ASSAULT WARGAME & AI PLATFORM

Documentación Técnica Canónica y Hoja de Ruta de Arquitectura (2024–2027)

EFFICIENTZERO V2 • VOEC SIM • MLOPS FORENSE DUCKDB • WARGAME REV6

Edición Completa Unificada (21 Módulos Canónicos)

Fecha de Generación: 24/10/2026 07:20 UTC

Plataforma de Simulación y Motor de Inteligencia Artificial

Índice General de Módulos Técnicos

🏠 01. Portal Canónico (Inicio)

Mapa de navegación del sistema, índice general de módulos y puertos activos.

📜 02. Premisas del Proyecto y Gobernanza

Marco maestro de diseño: límites HW, palancas en RL, nivel 0, instrumentación y calidad.

🏗️ 03. Arquitectura del Sistema (5-Tier)

Visión general 5-tier, clean architecture, contratos y flujo de datos.

🎲 04. Modelo del Juego y Reglas Tácticas

Topología hexagonal, LOS, dados, combate, moral, State Loop autoritativo.

⚖️ 05. Auditoría Reglamento vs. Engine

Auditoría página a página del Rulebook rev6 (52 págs.) y Scenario Book (38 escenarios).

🧠 06. Red Neuronal EfficientZero V2

Modelo de mundo, cabezas tácticas, HL-Gauss, Symlog, HRL Macro Nivel 1.

🔬 07. Auditoría Literatura Científica y Doctrinal

Estudio comparativo Palma et al. (2025), DeepMind, Dunnigan, Sun Tzu y Henry Hyde.

🎖️ 08. Plan Estratégico Perfilado de Unidades IA

Arquitectura doctrinal 3 capas, mutación dinámica contextual y observabilidad MLOps.

🎯 09. Currículums y Entrenamiento Progresivo

Currículum acelerado de VPs, currículum multi-escenario y puertas de calidad dinámicas.

🌐 10. Arquitectura Multi-Task Model-Based RL (EZ-M)

Extensión multitarea simultánea: dinámica invariante g_theta, modulación FiLM y cirugía PCGrad.

📊 11. Plataforma MLOps y Telemetría

DuckDB, Parquet, Promotion Gates, catálogo interactivo y Match Replay.

📐 12. Inventario Jerárquico Canónico de Métricas

Taxonomía de 4 niveles, origen autoritativo, cero-invención y purgado de redundancias.

⚡ 13. Servicios Backend y Catálogo de APIs

REST FastAPI (:8001), WebSocket binario MessagePack, RAG API y MLOps API (:8778).

🤖 14. Sistema RAG Copilot

BM25 híbrido, SitRep en vivo, explicabilidad normativa y soporte LLM local.

🕹️ 15. Manual del Simulador Táctico y HUD (:5173)

Guía de usuario del simulador táctico: mapa full-bleed, ribbon de mando, command deck, drawer lateral y atajos.

📋 16. Informes de Ejecución: Cabecera y Catálogo

Protocolo de auditoría forense, taxonomía TQS y repositorio centralizado de informes.

📑 17. Informe 01: Currículum y Auditoría Táctica Forense

Auditoría de 31 ciclos, diagnóstico de parálisis ofensiva, bloqueo de canales 36/37 y plan de mitigación.

📈 18. Informe 02: Currículum Multi-Fase y Comandante Prudente

Evaluación post-estandarización telemetría (34 evals), 1.ª victoria en campo abierto y confirmación de la Paradoja del Comandante Prudente.

🔴 19.1. Roadmap: Línea 1 (Observabilidad y Telemetría)

Conformidad de esquema, profiler de sondas zero-mock, validador E2E y depuración de redundancias.

🔴 19.2. Roadmap: Línea 2 (Captura de VPs y Paridad Humana)

Potencial radial a VP, sesgo prior MCTS, urgencia temporal y currículum acelerado.

🟢 19.3. Roadmap: Línea 3 (Fidelidad Normativa y Wargame)

Retirada en desenfilada 6.4, Dig-in, Humo, Transporte, formaciones combinadas y mitigación 3.6.

🟢 19.4. Roadmap: Línea 4 (Explicabilidad XAI e IPB)

Atribución causal IGNH/GradCAM, perfilador forense, chokepoints IPB y perfilado doctrinal.

🔵 19.5. Roadmap: Línea 5 (Generalización Procedural)

Mutador procedural con clumping geográfico y validación Zero-Shot en mapas 32×32 y 64×64.

📦 19.6. Historial de Versiones e Hitos Consolidados

Registro oficial de auditoría, changelog técnico por versiones v1.0.0 a v2.5.3 e hitos consolidados.

🗺️ 19. Hoja de Ruta Evolutiva (Roadmap Maestro)

Visión ejecutiva, matriz maestra global, orden de ejecución y changelog canónico de versiones.

🏠 01. Portal Canónico (Inicio)

Mapa de navegación del sistema, índice general de módulos y puertos activos.

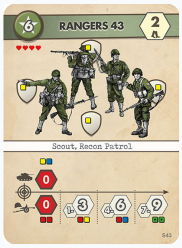
Portal Canónico de Documentación — Assault Wargame & AI Platform

Bienvenido al portal oficial de documentación técnica del proyecto **Assault**, plataforma integral de simulación táctica militar por turnos sobre tablero hexagonal e inteligencia artificial militar basada en **EfficientZero V2**.

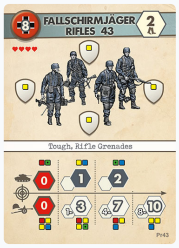
> [!NOTE] **Arquitectura Activa:** Todo el entrenamiento, autojuego, inferencia y MCTS operan exclusivamente sobre **EfficientZero V2** (`agents/efficientzero_v2/`). El stack legacy MuZero v1 se encuentra 100% decomisionado y obsoleto.

🎖️ Componentes y Activos Visuales Canónicos del Wargame Físico

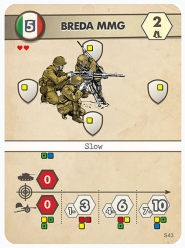
La plataforma *Assault* reproduce fielmente el juego de mesa táctico sobre tablero hexagonal (*Assault: Sicily 1943*). Todos los modelos de redes neuronales, el motor autoritativo de reglas y los visualizadores operan sobre estos componentes reales:



US Tarjeta de Unidad: US Rangers 43
Perfil Balístico, Cohesión y Moral



DE Tarjeta de Unidad: Fallschirmjäger
Armas Combinadas y Trait TOUGH



IT Tarjeta de Unidad: Breda Mod. 37
Ametralladora Pesada Colectiva

📜 Carta de Formación: US 1st Ranger
Command Points (CP) y OOB Táctico

Ficha Hexagonal
US Rangers 43

Ficha Hexagonal
Panzer IV Ausf. G

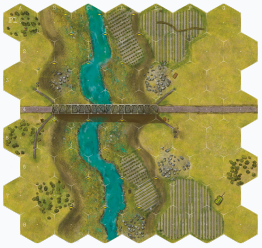
Ficha Hexagonal
Breda MMG

🔴 Dado Rojo
Balística AT / HE

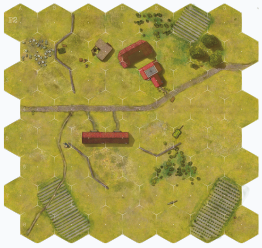
🟢 Dado Verde
Fusilería Ligera

⚠️ Chevron Estado
Status Suppressed

🔥 Marcador Acción
firing.png



🗺️ Tablero Modular P1: Campo Abierto y Desembarco
(mettete_i_piedi_terra_1)



🗺️ Tablero Modular P2: Asalto Urbano y Calles
(battaglia_cittadina_2_1)

🗺️ Mapa Jerárquico de Navegación del Sistema

📚 Estructura Temática de Documentación Canónica

I. Gobernanza y Arquitectura del Sistema

- 🏛️ 01. Portal Canónico (Inicio): Mapa de navegación del sistema, índice general de módulos y catálogo de servicios activos.
- 📜 02. Premisas del Proyecto y Gobernanza: Marco maestro de diseño: límites HW, economía de GPU, teoría de palancas (nudges) en RL, nivel 0 doctrinal, instrumentación y calidad.
- 🏗️ 03. Arquitectura del Sistema (5-Tier): Visión general desacoplada por contratos entre capas (`assault_model` , `voec_sim` , `agents/efficientzero_v2` y `mlops`).

II. Motor del Wargame y Reglas Tácticas

- 🎲 **04. Modelo del Juego y Reglas Tácticas**: Topología hexagonal, coordenadas axiales, cálculo 3D de LOS, balística, dados de batalla, moral, fuego de reacción y combate cerrado.
- ⚖️ **05. Auditoría del Reglamento Oficial vs. Engine**: Auditoría exhaustiva página a página del *Rulebook rev6* (52 páginas) y compendio oficial de 38 escenarios de combate.

III. Inteligencia Artificial Militar (EfficientZero V2)

- 🧠 **06. Red Neuronal EfficientZero V2**: Modelo de mundo recurrente ($h_\theta, g_\theta, f_\theta, q_\theta$), convoluciones hexagonales, cabezas tácticas auxiliares, Two-Hot, HL-Gauss, Symlog y PCGrad.
- 🔬 **07. Auditoría Literatura Científica y Doctrinal**: Comparativa Palma et al. (2025), literatura DeepMind (AlphaStar, AlphaZero), Clausewitz, Sun Tzu y análisis formal de la Paradoja del Comandante Prudente.
- 🎖️ **08. Plan Estratégico de Perfilado Doctrinal de Unidades**: Arquitectura doctrinal 3 capas, mutación contextual adaptativa, OOB y observabilidad en MLOps.
- 🎯 **09. Currículums y Entrenamiento Progresivo**: Currículum acelerado de VPs, currículum multi-escenario y puertas de calidad dinámicas (*Dynamic Gates*).
- 🌐 **10. Arquitectura Multi-Task Model-Based RL (EZ-M)**: Extensión multitarea simultánea: dinámica recurrente invariante g_θ, modulación FiLM por escenario, buffer estratificado y cirugía PCGrad.

IV. MLOps, Telemetría y Servicios Backend

- 📊 **11. Plataforma MLOps y Telemetría**: Almacenamiento columnar Parquet, base de datos analítica DuckDB, Promotion Gates y visor web interactivo en puerto 8778.
- 📐 **12. Inventario Jerárquico Canónico de Métricas**: Taxonomía estricta de 4 niveles, origen autoritativo, manifiesto cero-invención y plan de purgado de redundancias.
- ⚡ **13. Servicios Backend y Catálogo de APIs**: Endpoints REST FastAPI (:8001), streaming WebSocket binario MessagePack y documentación interactiva Swagger UI y ReDoc.
- 🤖 **14. Sistema RAG Copilot y Asistente Normativo**: Base de conocimiento vectorial/BM25, SitRep en vivo, explicabilidad normativa y motor LLM local.
- 🕹️ **15. Manual del Simulador Táctico y HUD (:5173)**: Arquitectura Modern Tactical Wargame HUD, mapa full-bleed 100%, barra de mando superior, carrusel OOB, cajón táctico lateral y modo inmersivo.

V. Informes de Ejecución y Auditorías Forenses

- 📋 **16. Informes de Ejecución: Cabecera y Catálogo**: Protocolo de auditoría forense, taxonomía TQS y repositorio centralizado de informes.
- 📑 **17. Informe 01: Currículum y Diagnóstico de Parálisis Ofensiva**: Auditoría de 31 ciclos, diagnóstico de aversión al riesgo en rol atacante, telemetría legacy y plan de mitigación.
- 📈 **18. Informe 02: Currículum Multi-Fase y Comandante Prudente**: Evaluación de 34 ejecuciones post-estandarización, primera victoria en campo abierto y caracterización formal de la Paradoja del Comandante Prudente.

VI. Hoja de Ruta Evolutiva (Roadmap)

Líneas Estratégicas Modulares:

- 🔴 **19.1. Línea 1: Garantía de Instrumentos y Observabilidad Fiel**: Conformidad de esquema, profiler de sondas zero-mock, validador E2E y depuración de redundancias.
- 🔴 **19.2. Línea 2: Aceleración de Captura de VPs y Paridad Humana**: Recompensa por potencial radial a VP, sesgo prior MCTS, urgencia temporal, paquete de aceleración y currículum ofensivo.
- 🟢 **19.3. Línea 3: Fidelidad Normativa y Reglamento Oficial del Wargame**: Retirada en desenfilada 6.4, Dig-in, Humo, Transporte, formaciones combinadas y mitigación 3.6.
- 🟢 **19.4. Línea 4: Explicabilidad Máxima (XAI), Perfilado Causal e IPB**: Atribución causal IGNH/GradCAM, perfilador forense, análisis de chokepoints IPB y perfilado doctrinal.
- 🔵 **19.5. Línea 5: Generalización Procedural y Robustez en Terrenos Abiertos**: Mutación procedural con clumping geográfico y validación Zero-Shot en mapas 32×32 y 64×64.
- 📦 **19.6. Historial de Versiones e Hitos Consolidados**: Registro oficial inmutable de entregas, changelog técnico cronológico por versiones (v1.0.0 a v2.5.3) y tareas consolidadas.

Documento Coordinador Maestro:

- 🗺️ **19. Hoja de Ruta Evolutiva (Roadmap Maestro)**: Visión ejecutiva, matriz maestra global, orden de ejecución inmediato y priorización evolutiva activa.

🚀 Servicios en Vivo y Puertos Locales

Servicio	Puerto	Descripción	URL Directa
Documentación Central	`8778`	MkDocs Material Portal Canónico	http://127.0.0.1:8778/docs/

Servicio	Puerto	Descripción	URL Directa
Visor Web MLOps	`8778`	Dashboard de modelos, métricas y replays	http://127.0.0.1:8778/
Backend Táctico FastAPI	`8001`	API de simulación, combate y RAG	http://127.0.0.1:8001/docs
Simulador UI (Vite / React)	`5173`	Tablero hexagonal interactivo	http://127.0.0.1:5173/
Editor de Unidades	`8780`	Editor GUI de cartas y atributos balísticos	http://127.0.0.1:8780/

📜 02. Premisas del Proyecto y Gobernanza

Marco maestro de diseño: límites HW, palancas en RL, nivel 0, instrumentación y calidad.

00. Premisas del Proyecto: Diseño, Arquitectura, Doctrinas de Calidad y Gobernanza Conceptual

Proyecto: Assault Tactical AI Engine (EfficientZero V2)
Documento Canónico de Diseño Conceptual y Especificación de Ingeniería del Sistema

---

Índice General

1. Gobernanza, Fuente Única de Verdad (SSoT) y Estado del Arte
2. Premisas de Diseño e Inteligencia Artificial (EfficientZero V2)
3. Premisas del Motor de Juego, Geometría y Reglas Tácticas
4. Premisas de Código, Contratos de Módulos e Invariantes
5. Premisas de Documentación y Sincronización Canónica
6. Viabilidad en Hardware Restringido y Economía de Cómputo
7. Teoría de Palancas (Nudges), Andamiaje y Emergencia Creativa
8. Nivel 0: Análisis Estratégico Pre-Batalla y Perfilado de Tropas
9. Manifiesto de Instrumentación Experimental Fiel
10. Rendimiento Asintótico sin Pérdida de Fidelidad
11. La Biblia Normativa y Protocolo de Pendientes de Revisión
12. Auditorías Continuas, Prevención de Regresiones y Arquitectura
13. Explicabilidad Máxima (XAI), Perfilado Causal y Análisis Táctico-Estratégico

---

1. Gobernanza, Fuente Única de Verdad (SSoT) y Estado del Arte

1. **Decomisión Inquebrantable de MuZero (v1 / Clásico)**:
   - MuZero clásico está 100% obsoleto y formalmente decomisionado. No se utiliza para entrenar, ni evaluar, ni correr benchmarks.
   - Cualquier intento de reintroducir o basar desarrollos en el stack heredado de MuZero se considera un error crítico de arquitectura.
2. **Agente Estándar Exclusivo: EfficientZero V2**:
   - Todo entrenamiento (`trainer.py`), evaluación, simulación, self-play, inferencia y MCTS opera única y exclusivamente sobre `EfficientZeroV2` (`agents/efficientzero_v2/`).
3. **Validación Doctrinal y Científica**:
   - Cada decisión técnica se apoya en literatura contrastada: Model-Based RL (Schrittwieser et al., Hafner et al. / DreamerV3), transferencias de Palma et al. (2025), mitigación ortogonal de gradientes PCGrad (Yu et al., 2020), y doctrina militar clásica (Dunnigan, Sun Tzu, Hyde).
4. **Unificación Canónica del Trabajo Pendiente**:
   - Existe un único punto centralizado y canónico para la hoja de ruta y la deuda técnica: el documento `05_ROADMAP_EVOLUTIVO.md` (presentado como punto final #10 de la documentación).
5. **Cero Retrocompatibilidad con MuZero (Clean Break)**:
   - **Tolerancia Cero a la Deuda Técnica**: Queda formalmente prohibido conservar capas de compatibilidad hacia atrás (*shims*, alias de transición o adaptadores tolerantes) con el stack de MuZero.
   - Toda nomenclatura, perfiles de evaluación (`ezv2_selfplay` , `ezv2_vs_random_side_a` , `ezv2_vs_random_side_b`), adaptadores (`EZV2VOECAdapter`), tensores de 32 canales y configuraciones deben ser 100% nativos y puros de EfficientZero V2.
6. **Registro Obligatorio de Versión de Implementación en el Roadmap**:
   - **Principio Inquebrantable de Trazabilidad Histórica**: Cada nueva funcionalidad, tarea, hito o refactorización que se dé por implementada o consolidada en el roadmap (`docs/05_ROADMAP_EVOLUTIVO.md` y documentos modulares en `docs/roadmap/`) **DEBE registrar explícitamente en qué versión / subversión de la plataforma se implementó** (ej. `[CONSOLIDADO 🟢 v2.5.2]` , `[CONSOLIDADO 🟢 v2.5.3]`), indicando el tag/commit correspondiente.
   - Queda terminantemente prohibido marcar tareas como consolidadas sin asociarlas a una versión concreta de la plataforma.

---

2. Premisas de Diseño e Inteligencia Artificial (EfficientZero V2)

1. **Modelo de Mundo Diferenciable en Espacio Latente**:
   - La red planifica en un espacio latente abstracto compacto $s \in \mathbb{R}^{48 \times H \times W}$ mediante una red de dinámica recurrente (g_θ), sin realizar rollouts en el simulador físico durante la búsqueda MCTS.
2. **Topología de Entrada Axial (32 Canales)**:

- La entrada al modelo consta de exactamente 32 canales axiales fijos con semántica estandarizada (terreno, elevación, LOS, niebla, control territorial, posturas, armas pesadas y gradientes de distancia a VPs).

3. **Convolución Nativa con Máscara Hexagonal (`HexagonalMaskConv2d`)**:
   - En una cuadrícula axial (q, r), un kernel cartesiano 3×3 introduce dos vecinos falsos (diagonales inexistentes). La convolución hexagonal enmascara activamente esas dos esquinas a cero con `persistent=False` , garantizando simetría perfecta de 6 vecinos.
4. **Representación Distribucional de Valor y Recompensa**:
   - Se prohíbe la regresión escalar MSE directa para retornos. Se utiliza una representación distribucional Two-Hot / HL-Gauss sobre un soporte discreto simétrico $[-30, +30]$ (61 átomos) con transformación contractiva symlog/symexp.
5. **Mitigación de Conflicto de Gradientes (PCGrad)**:
   - Para coentrenar la política, el valor, la consistencia temporal SimSiam y las 6 cabezas tácticas auxiliares sin interferencia destructiva, los gradientes en conflicto ($\langle g_i, g_j \rangle < 0$) se proyectan sobre el hiperplano ortogonal al de la tarea primaria.

---

3. Premisas del Motor de Juego, Geometría y Reglas Tácticas

1. **Topología Hexagonal Pura**:
   - Coordenadas axiales (q, r) con correspondencia cúbica ($x + y + z = 0$). Distancia canónica:

$$\text{dist} = \frac{|\Delta q| + |\Delta(q + r)| + |\Delta r|}{2}$$

   - No existen diagonales. Cada hexágono tiene exactamente 6 vecinos equidistantes.
2. **Línea de Visión (LOS) y Detección 3D**:
   - Ray-casting de centro a centro. El bloqueo se rige por elevación y obstáculos intermedios (edificios, bosques, humo), respetando la cresta de colina y ángulos muertos. Las unidades en cobertura sin disparar permanecen ocultas (*Hidden*).
3. **Sistema Balístico y Fuego de Combate**:
   - Resolución de dados oficiales en 4 colores (Rojo, Azul, Verde, Amarillo) con caras de Impacto, Impacto Doble, Supresión, Crítico y Bloqueo (Save).
   - El daño neto resta puntos de vida (HP). Al llegar a 0 HP la unidad es eliminada.
4. **Estados de Moral y Cohesión**:
   - 4 estados canónicos: `NORMAL` , `PINNED` (-50% movilidad, -1 dado de defensa), `SUPPRESSED` (incapacitado para moverse/disparar voluntariamente), y `ROUTED` (retirada forzada; si es atacada es destruida o capturada).
5. **Fuego de Reacción (*Opportunity Fire*)**:
   - Las unidades en estado `NORMAL` con línea de visión y capacidad de reacción disponible pueden interrumpir el movimiento enemigo que cruza terreno expuesto.

---

4. Premisas de Código, Contratos de Módulos e Invariantes

1. **Clean Architecture y Agnosticismo Radical de `assault_model/` **:
   - `assault_model/` es el motor de reglas puras. Tiene terminantemente prohibido importar librerías de Machine Learning (`torch`) o librerías web (`fastapi`).
2. **El Puente Único: `voec_sim/` **:
   - Es el único adaptador autorizado para traducir las estructuras de `assault_model/` al tensor de 32 canales y generar la máscara de acciones legales.
3. **Contrato de Inmutabilidad de Modelos de Checkpoints**:
   - Cualquier mejora de arquitectura en la red debe mantener la compatibilidad con el cargador de pesos (`strict=True` , buffers `persistent=False` , sin adición arbitraria de pesos que rompa checkpoints históricos).
4. **Identificadores Canónicos de Acciones**:
   - Todas las acciones del motor se serializan en formato delimitado por dos puntos: `TIPO:UNIDAD:DESTINO_Q:DESTINO_R` (ej. `MOVE:US_1:2:8` , `ATTACK:GE_1:US_1:3:4`).

---

5. Premisas de Documentación y Sincronización Canónica

1. **Los `.md` de `docs/` son la Fuente Única de Verdad (SSoT)**:
   - Queda terminantemente prohibido editar manualmente el HTML de documentación (`docs/index.html` o `site/`).
2. **Sincronización Bit a Bit Obligatoria (Regla a Fuego)**:
   - Tras cualquier cambio en `docs/*.md` , es mandatorio ejecutar: `python scripts/sync_docs_html.py` Este script inyecta el 100% íntegro de los `.md` en `embeddedDocsData` y recompila el sitio estático mediante `mkdocs build` .
3. **Posición Sagrada del Roadmap**:
   - El archivo `05_ROADMAP_EVOLUTIVO.md` debe figurar siempre en la última posición del índice de documentación.

4. **Trazabilidad de Versión de Cada Hito del Roadmap**:
    - Ninguna tarea del roadmap puede figurar en estado consolidado sin la anotación explícita de su versión de entrega (ej. `v2.5.2`, `v2.5.3`). Esto previene la pérdida de linaje entre lo que el código ejecuta, lo que la telemetría MLOps mide y lo que la documentación canónica certifica.

---

6. Viabilidad en Hardware Restringido y Economía de Cómputo

6.1. La Paradoja del Hardware en RL

Entrenar agentes complejos en wargames suele requerir granjas de servidores masivas si se utiliza RL libre de modelo (PPO/DQN), ya que necesitan entre 10^7 y 10^9 pasos de interacción ciega con el entorno.

6.2. Por qué EfficientZero V2 es Viable en un PC Doméstico

EfficientZero V2 fue diseñado para resolver el reto de *Atari 100k* (aprender con solo 100.000 pasos de entorno, equivalentes a ~2 horas de experiencia humana). Esto es viable en local gracias a:

1. **MCTS en la Imaginación Latente**: El simulador físico corre solo 1 vez por paso real. Las ramas del árbol MCTS no ejecutan el motor de reglas en CPU; corren en GPU evaluando la pequeña red recurrente de dinámica (g_θ).
2. **Reanálisis Off-Policy Intenso**: Los episodios almacenados en el Replay Buffer se reutilizan entre 10 y 20 veces re-analizando sus árboles de búsqueda con la red más moderna sin necesidad de jugar partidas nuevas.
3. **Presupuesto Fijo de Memoria**:
    - La red completa con sus 10 cabezas ocupa únicamente **~42 MB** en memoria VRAM.
    - En precisión mixta (**AMP FP16**), el paso de inferencia y MCTS consume **< 3.5 GB VRAM**, permitiendo entrenar en GPUs convencionales de 8 GB sin saturar el sistema.

---

7. Teoría de Palancas (Nudges), Andamiaje y Emergencia Creativa

7.1. Definición Psicológica y Matemática de "Palanca" (*Nudge*)

En ciencias conductuales (Thaler & Sunstein), una palanca altera la arquitectura de decisión facilitando el camino óptimo sin prohibir ninguna opción.

- **Prohibido**: Enmascarar acciones legales como ilegales por criterios doctrinales.
- **Canónico**: Modular suavemente los *priors* de la raíz MCTS:

$$P_{\text{mod}}(a) = \frac{P_\theta(a) \cdot \exp\big(\lambda(t) \cdot B(a)\big)}{\sum_{a'} P_\theta(a') \cdot \exp\big(\lambda(t) \cdot B(a')\big)}$$

  Donde $B(a)$ es el sesgo doctrinal y $\lambda(t) \in [0, 1]$ es el coeficiente de andamiaje.

7.2. La Garantía de Emergencia Creativa

Bajo la fórmula de selección PUCT:

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}} \cdot P_{\text{mod}}(a) \cdot \frac{\sqrt{\sum N}}{1 + N(s,a)} \right]$$

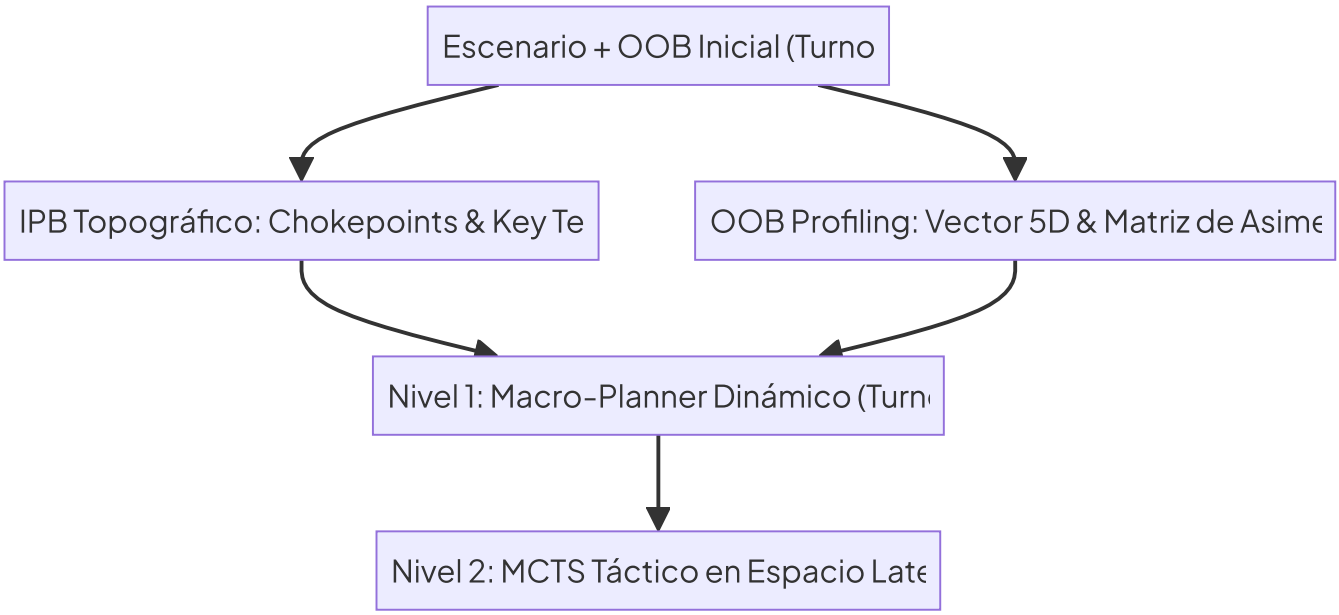
A medida que el número de visitas en un nodo $N(s,a) \to \infty$, el término del prior decae asintóticamente hacia 0. **El valor real $Q(s,a)$ domina absolutamente**. Si la IA descubre una maniobra heterodoxa (retirada táctica, emboscada, sacrificio) con mayor valor empírico que la doctrina humana, MCTS seleccionará la jugada creativa sin restricción.

7.3. Andamiaje de Vygotsky (*Scaffolding*) y Decaimiento (*Annealing*)

- **Fase 1 (Bootstrap, $\lambda = 1.0$)**: Palanca fuerte. El agente aprende "motricidad táctica" en pocas horas, evitando la parálisis por miedo al fuego.
- **Fase 2 (Refinamiento, $\lambda = 0.35$)**: El MCTS empieza a desafiar los priors cuando encuentra jugadas superiores.
- **Fase 3 (Maestría y Super-humano, $\lambda \to 0.02$)**: La palanca queda prácticamente apagada; el juego emergente de la red gobierna el tablero.

---

8. Nivel 0: Análisis Estratégico Pre-Batalla y Perfilado de Tropas



8.1. Perfilado Vectorial 5D de Tropas (*OOB Profiling*)

Cada unidad se caracteriza en $t = 0$ mediante su vector normalizado: $$\vec{F}u = \big[F\text{range}}, ; F_{\text{shock}}, ; F_{\text{armor}}, ; F_{\text{mobility}}, ; F_{\text{at}} \big] \in [0, 1]^5$$ A partir del balance de ambos bandos, se calcula la **Matriz de Asimetría Relativa** ($\Delta_{\text{Shock}}, \Delta_{\text{Range}}, \Delta_{\text{Armor}}$), dictando si la doctrina de partida debe ser de **cierre rápido de distancia bajo cobertura** o de **fijación y desgaste a largo alcance**.

8.2. Inteligencia Topográfica del Terreno (*IPB*)

En menos de $3\ \text{ms}$ en CPU (Turno 0):

- **Cuellos de Botella (*Chokepoints*)**: Hexágonos críticos calculados por centralidad de caminos mínimos (Dijkstra) hacia los VPs.
- **Terreno Clave (*Key Terrain*)**: Posiciones elevadas o edificios con arcos de tiro dominantes sobre los accesos principales.
- **Presupuesto Temporal (*Tempo Index*)**: $\frac{d_{\min}}{T_{\max} \times V_{\text{media}}}$. Si es > 0.6, el escenario impone urgencia máxima de aproximación para no perder por tiempo.

---

9. Manifiesto de Instrumentación Experimental Fiel

> **Principio de Cero Invención**: El panel de evaluación (Dashboard MLOps) es un aparato de medición física. No opina, no maquilla y no inventa datos. Si un valor no se puede medir, se muestra como `NO_DISPONIBLE`.

9.1. Los Cuatro Instrumentos de Laboratorio

1. **Dinamómetro de Valor y ELO**: Win Rate estricto según condiciones del escenario y curva ELO ($K = 32$) con semillas registradas. Prohibido computar empates como victorias aproximadas.
2. **Tomógrafo de Búsqueda MCTS**: Muestra los tensores crudos del árbol de decisión: visitas $N(a)$, valor des-simlogueado $Q(a)$, prior de la red $P_\theta(a)$ y prior modulado $P_{\text{mod}}(a)$.
3. **Microscopio Balístico y de Reglas**: Registro exacto de cada tirada de dados por color y cara (`RED:HIT`, `BLUE:BLOCK`). Alerta automática si el generador PRNG desvía la media teórica más de un 1.5.
4. **Osciloscopio de Gradientes y Atribución**: Mide el coseno real de conflicto de gradientes de PCGrad y la atribución de los 32 canales axiales mediante Integrated Gradients (IGNH).

---

10. Rendimiento Asintótico sin Pérdida de Fidelidad

1. **Clonado Inmutable Copy-on-Write (CoW)**:
   - El estado del juego no se duplica con `deepcopy`. Estructuras inmutables con CoW permiten **>109.000 clones por segundo** en CPU con 0 copias de memoria en ramas no alteradas.
2. **Línea de Visión (LOS) Vectorizada**:
   - Matrices de adyacencia y visibilidad pre-calculadas en NumPy/Numba. Consultas de LOS resueltas en < 30 nanosegundos.
3. **Control Territorial Incremental**:

- Coste independiente del tamaño del mapa: al mover una unidad, solo se actualiza el anillo local de radio 3 ($O(k \cdot R^2)$ amortizado).

4. **Cómputo Adaptativo (ACT)**:
   - El MCTS modula sus simulaciones según la entropía táctica $H(\pi)$ (64 simulaciones en turnos de transición; 256–384 en contacto directo o asalto a VPs), ahorrando un 50% de tiempo de GPU sin pérdida de calidad.

---

11. La Biblia Normativa y Protocolo de Pendientes de Revisión

> **Regla de Oro**: El reglamento oficial (*Rulebook revó* de 52 páginas y el *Libro de Escenarios*) es la máxima e indiscutible autoridad. Se prohíbe inventar reglas ad-hoc para salir del paso.

Protocolo de Dudas y Ambigüedades:

Ante cualquier contradicción en las cartas, caso borde no tipificado o ambigüedad:

1. **JAMÁS se inventa una regla en código.**
2. La mecánica se etiqueta de inmediato como `[PENDIENTE DE REVISIÓN / AMBIGÜEDAD NORMATIVA]`.
3. Se documenta la página exacta, el párrafo literal y los argumentos en juego en el registro de auditoría.
4. El motor ejecuta la alternativa más conservadora y neutra, emitiendo un warning experimental auditable.

---

12. Auditorías Continuas, Prevención de Regresiones y Arquitectura

12.1. Arquitectura Limpia en 5 Capas (Dependencia Unidireccional)

- **Capa 1:** `assault_model/` **(Reglas)**: Agnóstica de ML y Web (cero imports de `torch` o `fastapi`).
- **Capa 2:** `voec_sim/` **(Adaptador)**: Único traductor de reglas a tensores de 32 canales y acciones legales.
- **Capa 3:** `agents/efficientzero_v2/` **(IA)**: Red neuronal, MCTS latente y Macro-Planner.
- **Capa 4:** `mlops/` **(Instrumentación)**: Observador pasivo. Persiste en Parquet y consulta con DuckDB en <10 ms.
- **Capa 5:** `fastapi` & `viewer_frontend` **(Servicios)**: Consumen contratos de API desacoplados por HTTP/WebSocket MessagePack.

12.2. Protocolo de Auditorías Continuas (Tolerancia Cero a Regresiones)

1. **Invariantes Físicos del Motor**: Conservación estricta de puntos de vida ($\sum \text{HP}_{\text{actuales}} + \sum \text{HP}_{\text{bajas}} = \text{HP}_{\text{iniciales}}$) y apilamiento máximo de 1 unidad por hexágono.
2. **Suite de Regresión del Manual**: Cada test valida un ejemplo gráfico oficial del libro (ej. arcos de fuego en p. 27, asaltos combinados en p. 33).
3. **Canary Check Pre-Entrenamiento**: Test de 100 pasos en un micro-lote sintético antes de cada sesión larga para comprobar descenso de pérdidas Two-Hot, estabilidad de norma de gradientes ($|g| \in [0.1, 5.0]$) y no-colapso en SimSiam.
4. **Promotion Gates**: Promoción de checkpoints solo si superan:

$$\text{WinRate vs Random} \geq 90$$

---

13. Explicabilidad Máxima (XAI), Perfilado Causal y Análisis Táctico-Estratégico

> 🧠 **Principio de IA de Cristal (Glass-Box Tactical RL)**
>
> En un simulador táctico militar y wargame de alta complejidad, un modelo que gana sin poder explicar **el porqué causal de sus decisiones** es científicamente estéril e inútil para la doctrina. La explicabilidad (XAI) no es un añadido estético de visualización: es una **herramienta analítica de primer orden** para auditar el perfil de tropas, diagnosticar asimetrías en escenarios y verificar la congruencia entre el plan macroestratégico y las microdecisiones del árbol MCTS.

13.1. Utilidad en el Perfilado de Unidades (Unit Profiling)

1. **Atribución Causal de Armamento y Terreno (IGNH)**:
   - Mediante gradientes integrados (*Integrated Gradients*) sobre los 32 canales axiales, el sistema descompone qué factores perceptuales activan las maniobras de cada escuadra. Permite responder cuantitativamente:
     - ¿La escuadra de ametralladoras pesadas (HMG) actúa motivada por líneas de interdicción y arcos de tiro (canales 8–11), o por proximidad accidental a objetivos?
     - ¿Las unidades de infantería ligera priorizan la cobertura densa (canales 0–3) para su aproximación?
2. **Detección de Sesgos de Uso y Roles Muertos (*Dead Roles*)**:
   - Mapeo de la frecuencia de activación de cada unidad frente a su idoneidad teórica de manual. Permite identificar si el modelo subutiliza ciertas piezas (ej. morteros indirectos o armas contracarro) por dificultades de exploración, o si las emplea en roles suicidas contrarios a la preservación de fuerza.
3. **Calibración del Radar Doctrinal 5D**:

- Cotejo empírico entre el rol asignado por el Macro-Planner (`SPEARHEAD`, `FIRE_SUPPORT`, `OBJECTIVE_DEFENDER`, `FLANK_SCREEN`) y los vectores de acción ejecutados en el árbol. La explicabilidad revela si una escuadra designada como *Apoyo de Fuego* efectivamente fijó al enemigo o si abandonó su posición por priorizar un movimiento oportunista.

13.2. Utilidad en el Perfilado de Escenarios (Scenario Profiling)

1. **Auditoría de Asimetría Intrínseca y Brecha Estructural**:
   - El cruce entre el rendimiento bilateral espejo (*Mirror Score*) y la brecha de asimetría ($|S_A - S_B|$) cuantifica la ventaja inherente del terreno. La explicabilidad permite determinar si una ventaja se origina en:
     - Asimetría balística de tropas iniciales.
     - Estrangulamiento geográfico en cuellos de botella (*chokepoints*) con líneas de tiro inexpugnables.
     - Insuficiencia del presupuesto temporal (*Tempo Index* < 0.5) que penaliza al atacante independientemente de su calidad táctica.
2. **Mapas de Fricción y Puntos de Inflexión**:
   - Integración espacial del mapa de amenazas proyectado (`threat_map`) con la máscara de control territorial (`final_ownership`). Revela en qué hexágonos exactos y en qué turnos se decantó el control de la partida, permitiendo perfilar si un escenario premia la maniobra envolvente o la guerra de desgaste frontal.

13.3. Análisis del Plan Táctico-Estratégico (Congruencia Jerárquica Nivel 1 vs Nivel 2)

1. **Trazabilidad Causal de la Jerarquía HRL**:
   - **Nivel 1 (Planificador Macro)**: Evalúa el tablero completo y emite la postura doctrinal global (`ASSAULT_VP`, `DEFENSIVE_HOLD`, `ZONE_SUPPRESS`) y los objetivos clave.
   - **Nivel 2 (Búsqueda MCTS PUCT)**: Explora el árbol en espacio latente modulando sus priors de búsqueda según la postura del Nivel 1.
2. **Índice de Sobreescritura MCTS (*Policy Override Index*)**:
   - Monitorea los pasos donde $\arg\max P_\theta(a) \neq a^*$. Un override no es un fallo; es una auditoría directa de cuándo el árbol de búsqueda descubrió que la intuición inmediata de la red (prior neural) conducía a una trampa táctica, forzando una corrección reflexiva hacia la maniobra segura.
3. **Desglose Multitarea de Cabezas Tácticas Auxiliares**:
   - **Mapa de Amenazas (*Threat Map*)**: Revela si la red percibe con precisión los arcos de fuego letal antes de penetrar en ellos.
   - **Desgaste Proyectado (*Attrition*)**: Explica si el modelo asume bajas calculadas en una posición clave a cambio de un intercambio favorable (CER > 1.0).
   - **Riesgo de Supresión (*Morale*)**: Explica si una decisión de pausar o buscar cobertura responde a la anticipación del colapso de cohesión bajo fuego de respuesta.
   - **Control de VPs Multiobjetivo (*Multi-VP & Ownership*)**: Explica qué hexágonos de victoria valora el modelo como sostenibles en el tiempo frente a capturas efímeras o trampas de distracción.

🏗️ 03. Arquitectura del Sistema (5-Tier)

Visión general 5-tier, clean architecture, contratos y flujo de datos.

Arquitectura del Sistema: Assault Wargame AI & Simulation Platform

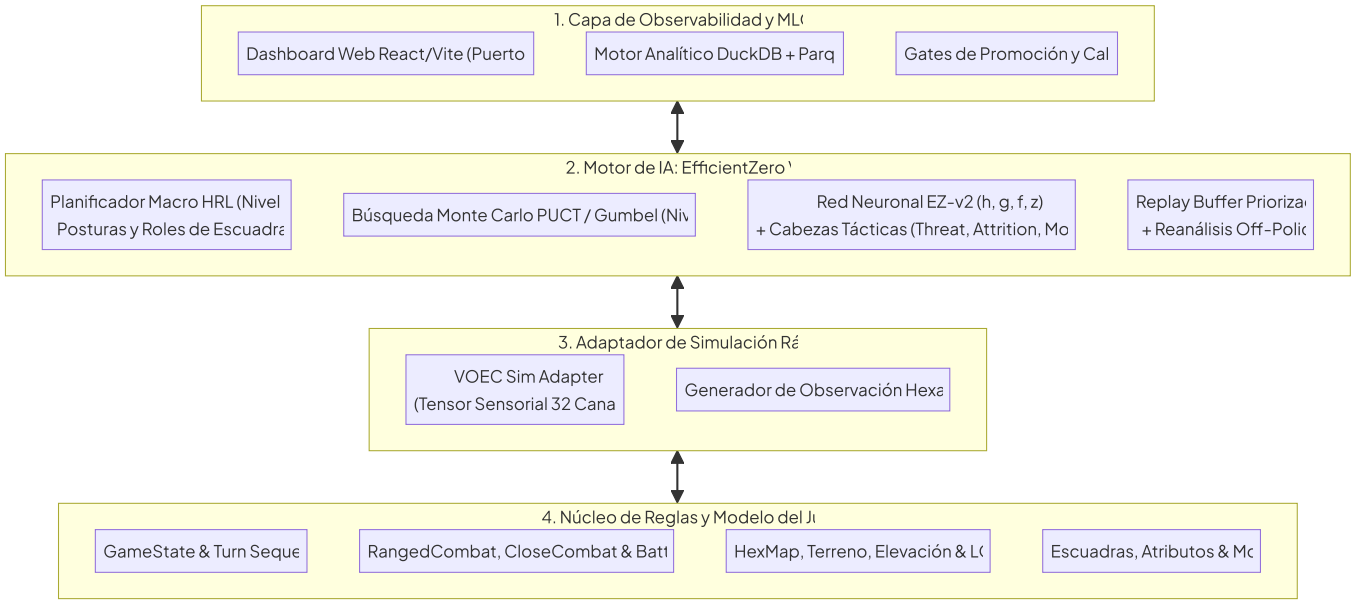
> **Estado**: Actualizado y Canónico (2026)
> **Directiva Fundamental**: El motor de aprendizaje está basado exclusivamente en **EfficientZero V2** (`agents/efficientzero_v2/`). Todo el stack heredado de MuZero v1 está completamente decomisionado y obsoleto.

---

1. Visión General y Filosofía de Diseño

El proyecto **Assault** es una plataforma integral de simulación táctica militar por turnos sobre tablero hexagonal combinada con inteligencia artificial basada en aprendizaje por refuerzo con modelo de mundo (*Model-Based Reinforcement Learning*).

La arquitectura sigue estrictamente los principios de **Arquitectura Limpia (*Clean Architecture*)**, desacoplando radicalmente cuatro subsistemas autónomos:



---

2. Los Cuatro Subsistemas Autónomos

2.1. Núcleo del Modelo del Juego (`assault_model/`)

- **Propósito**: Encapsula las reglas tácticas del juego de guerra sin ninguna dependencia externa de librerías de Machine Learning (sin PyTorch) ni de interfaces gráficas.
- **Componentes Principales**:
  - `map/` : Topología hexagonal con coordenadas axiales (q, r), cálculo de distancias Manhattan hexagonal, aristas con características de cobertura, tipos de terreno (bosque, urbano, colina, río) y elevación.
  - `combat/` : Resolución estocástica mediante dados de batalla (`BattleDie` blancos y negros), perfiles de ataque y defensa, arcos de tiro, línea de visión (`LineOfSight`), fuego de reacción (`ReactionFire`), degradación de moral (`Morale` : Normal, Pinned, Suppressed, Routed) y asaltos cuerpo a cuerpo (`CloseCombat`).
  - `units/` : Modelado de infantería ligera, infantería pesada, armas de apoyo (HMG, morteros) y vehículos blindados.
  - `state/` : Transición inmutable de estados de juego (`GameState`), control de iniciativa, turnos y activaciones.

2.2. Adaptador de Simulación Rápida (`voec_sim/`)

- **Propósito**: Actúa como puente de alto rendimiento entre las estructuras de datos de `assault_model` y los tensores requeridos por PyTorch para el entrenamiento.
- **Funciones Clave**:
  - Proyecta el estado completo a un **tensor espacial 32 canales** $\times H \times W$ (unidades amigas/enemigas, HP normalizado, cobertura, costes de movimiento, LOS, estado de supresión y banderas VP).

- Vectoriza y filtra las acciones legales candidatas (movimiento, disparo, asalto, preparación de fuego de reacción).

2.3. Motor de Inteligencia Artificial (`agents/efficientzero_v2/`)

- **Propósito**: Ejecución de auto-juego (*self-play*), planificación MCTS guiada por modelo de mundo y entrenamiento distribuido.
- **Innovaciones Implementadas**:
  - **Planificador Macro Jerárquico (Nivel 1)**: Evalúa el estado global y asigna postura operativa (`ASSAULT_VP` , `DEFENSIVE_HOLD` , `ZONE_SUPPRESS`) y roles tácticos a cada unidad (`SPEARHEAD` , `FIRE_SUPPORT` , `DEFENDER` , `SCREEN`), modulando los priors de búsqueda MCTS en $+35$.
  - **Búsqueda MCTS guiada por Modelo de Mundo (Nivel 2)**: 512 simulaciones en espacio latente continuo sin tocar el simulador físico.
  - **Representación Discreta Two-Hot y HL-Gauss**: Estabilidad garantizada en predicción de valor y recompensas sin colapso de gradiente.
  - **Convoluciones con Máscara Hexagonal (`HexagonalMaskConv2d`)**: Preservación estricta de la vecindad de 6 hexágonos en kernels 3×3.
  - **Cabezas Tácticas Auxiliares**: `threat_map` (2D LOS/fuego enemigo), `attrition` (balance de bajas), `morale` (supresión).
  - **Balanceador de Gradientes PCGrad**: Ortogonalización de gradientes conflictivos entre pérdidas primarias y secundarias.

2.4. Plataforma MLOps y Telemetría (`mlops/`)

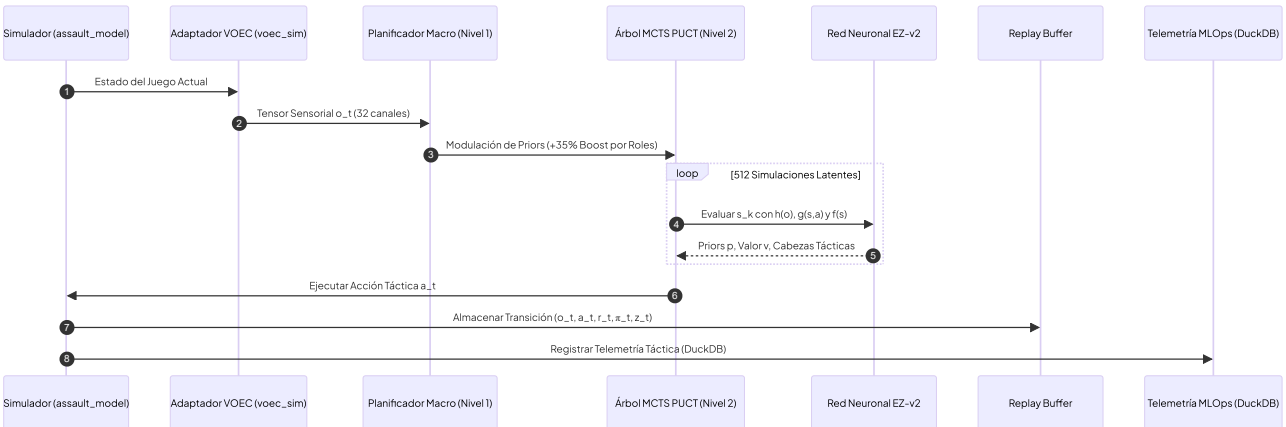
- **Propósito**: Registro, almacenamiento columnar, validación de gates de calidad y visualización interactiva.
- **Componentes Principales**:
  - **Backend Analítico**: Almacenamiento en Parquet y base de datos analítica integrada **DuckDB** para consultas agregadas instantáneas.
  - **Servidor Daemon**: Daemon local en puerto `8778` sirviendo APIs REST y la aplicación web React/Vite.
  - **Frontend SPA**: Visualización de catálogo, métricas de currículum, topología interactiva de la red, visor hexagonal de partidas paso a paso (*Match Replay*) y atribución de canales.

2.5. Simulador Táctico Interactivo y Partidas en Vivo (`assault_backend/` y `assault_ai_ui/`)

- **Propósito**: Proporciona un entorno interactivo completo para **jugar contra la IA** o presenciar partidas autónomas en tiempo real (**Self-Play**) con renderizado por GPU mediante **Pixi.js**, animaciones fluidas, partículas de combate y efectos de sonido espaciales (`SoundService.ts`).
- **Componentes Principales**:
  - **Backend de Juego FastAPI (`assault_backend/` en Puerto 8001)**:
    - Sesión autoritativa de juego en memoria (`GameSession`) que conecta las acciones de la UI directamente con `RuntimeGameState` .
    - Canal WebSocket bidireccional (`/ws/game`) para la transmisión instantánea de estados, tiradas de dados, disparos de reacción y bajas.
    - Módulo **RAG Copilot** (`assault_rag/`): Asistente en vivo que explica las reglas del juego, justifica decisiones de la IA y recomienda jugadas óptimas basándose en el manual de reglas y el contexto actual.
  - **Frontend del Simulador (`assault_ai_ui/` en Puerto 5173)**:
    - **Modo Humano vs IA**: El usuario selecciona unidades, visualiza rangos de movimiento y disparo en el lienzo hexagonal Pixi.js, y ejecuta órdenes contra el agente neuronal.
    - **Modo Auto-Play / Espectador**: Ejecuta la IA de forma autónoma en bucle continuo (`aiTurnRunner.ts`), mostrando el avance táctico de ambos bandos.
    - **Panel de Combate y Cohesión**: Animación visual de tiradas de dados de ataque/defensa, chequeos de moral en vivo y registro detallado de combate (`CombatPanel.tsx`).
    - **Control de Capas**: Activación/desactivación de niebla de guerra, conos de visión (LOS), elevaciones y vectores de depuración (`LayerControls.tsx`).
    - **Audio Espacial Táctico**: Efectos de pasos sobre tierra (`stepdirt`) y disparos de fusilería (`rifle-gunshot`) orquestados por `SoundService.ts` .

---

3. Flujo de Datos y Ciclo de Vida Operativo

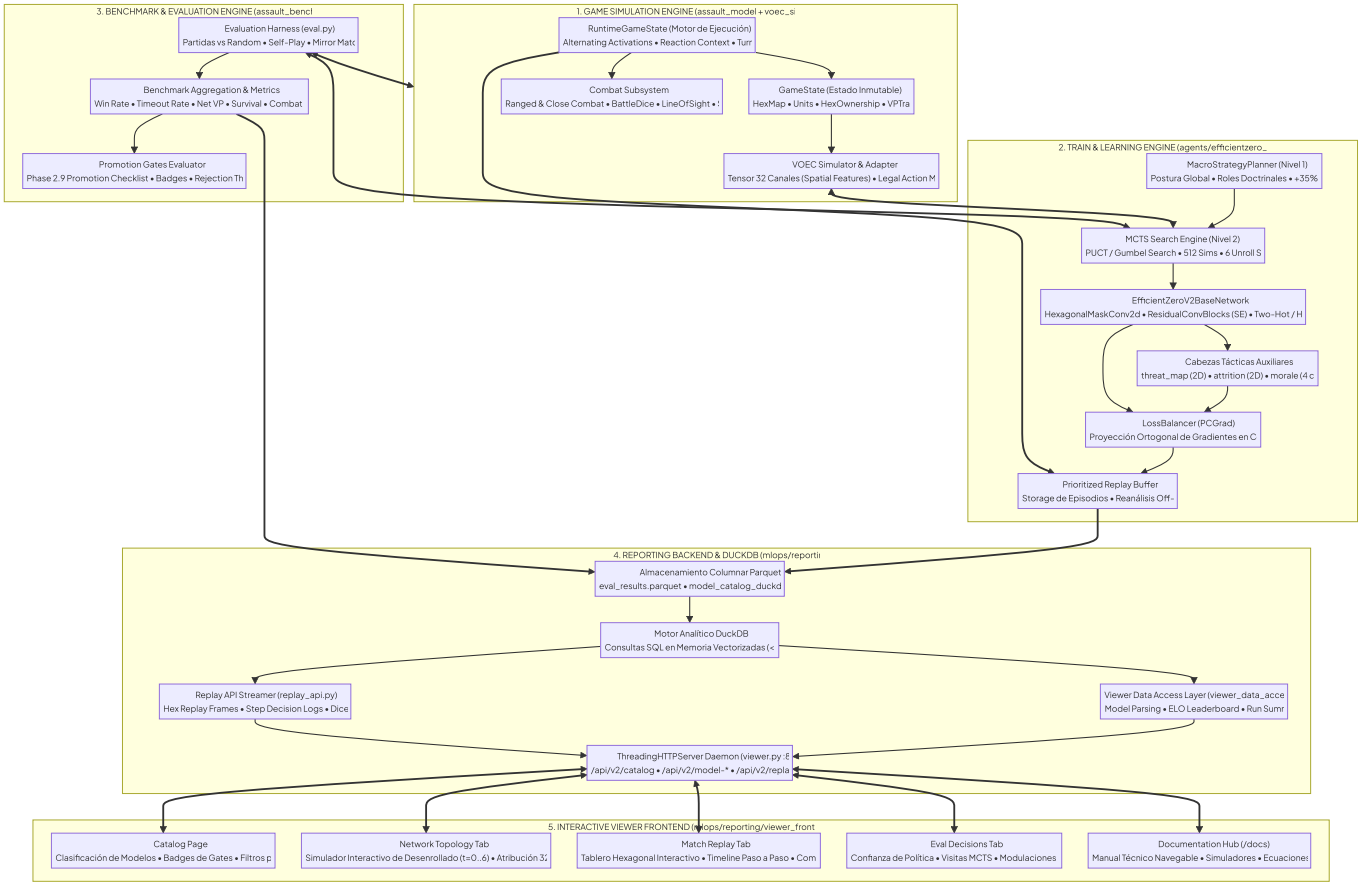


1. **Extracción Sensorial**: `voec_sim` construye la observación $o_t \in \mathbb{R}^{32\times16\times16}$.
2. **Estrategia Macro (N1)**: El planificador macro determina la postura doctrinal y los roles asignados a cada escuadra.
3. **Exploración PUCT (N2)**: El árbol MCTS desenrolla hasta $k = 6$ pasos recursivos en la red de dinámica $g_\theta(s, a)$.
4. **Ejecución y Almacenamiento**: La acción óptima a_t se ejecuta en el entorno real y se guarda en el buffer de repetición con índices de consistencia SimSiam.
5. **Auditoría MLOps**: Cada decisión, probabilidad de política y resultado de combate se persiste en DuckDB para su inspección inmediata en el visor.

---

3. Visión Completa de Arquitectura de Extremo a Extremo

A continuación se detalla la arquitectura global de 5 capas que conecta el motor de simulación, el pipeline de entrenamiento de IA, el subsistema de benchmarking, el backend de analítica MLOps y el visualizador web interactivo:



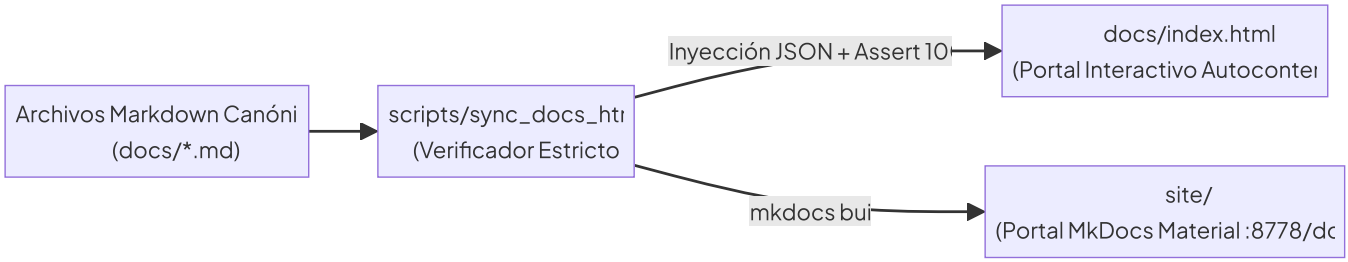
3.1. Detalle de las 5 Capas y su Interconexión

1. **Capa de Simulación (Tier 1)**: El motor de simulación (`assault_model`) ejecuta las reglas tácticas de forma determinista y sin sobrecarga de IA. El adaptador `voec_sim` captura instantáneas del estado y las serializa en tensores normalizados de 32 canales espaciales sobre la cuadrícula hexagonal.
2. **Capa de Entrenamiento y Búsqueda (Tier 2)**: Durante el autojuego (*self-play*), el Planificador Macro de Nivel 1 evalúa el tensor y establece la doctrina (postura y roles). El árbol MCTS de Nivel 2 realiza 512 simulaciones en el espacio latente de `EfficientZeroV2BaseNetwork`, evaluando política, valor y cabezas tácticas (`threat_map`, `attrition`, `morale`). Las transiciones se guardan en el *Replay Buffer Priorizado*, y el entrenamiento unifica las pérdidas con balanceo ortogonal **PCGrad**.
3. **Capa de Evaluación y Benchmarking (Tier 3)**: Los checkpoints generados son evaluados de forma estandarizada en baterías competitivas (self-play y versus oponentes estocásticos). Los resultados se agregan calculando métricas finas de combate y se evalúan frente a los **Promotion Gates**.
4. **Capa de Backend Analítico MLOps (Tier 4)**: Toda la telemetría se almacena en archivos particionados **Apache Parquet**. El motor embebido **DuckDB** ejecuta consultas analíticas vectorizadas instantáneas. El servidor HTTP multi-hilo (`viewer.py` en puerto 8778) expone endpoints REST optimizados para streaming de partidas y análisis de catálogo.
5. **Capa de Frontend Visualizador (Tier 5)**: Aplicación SPA moderna en React y Vite. Presenta el catálogo de modelos, el inspector interactivo de topología de red con simulador de desenrollado temporal, la matriz de atribución de canales, el visor hexagonal de partidas paso a paso con registros de combate y el nuevo portal de documentación técnica navegable.

---

5. Arquitectura de Documentación Canónica y Pipeline de Compilación HTML

La documentación técnica del proyecto se rige por el principio de **Fuente Única de Verdad (Single Source of Truth - SSoT)** en Markdown (`docs/*.md`) con compilación automatizada al 100% hacia los portales HTML:



5.1. Reglas Estrictas de la Documentación:

1. **Markdown como Fuente de Verdad**: Todo contenido normativo, científico, arquitectónico y operativo se redacta en `docs/*.md`. Nunca se edita HTML de documentación directamente a mano.
2. **Posición del Roadmap**: La **Hoja de Ruta Evolutiva (** `05_ROADMAP_EVOLUTIVO.md` **)** debe figurar **SIEMPRE como el último punto (** `09. Hoja de Ruta Evolutiva (Roadmap)` **)**, unificando todos los planes y deudas técnicas del sistema.
3. **Sincronización Automática Obligatoria**: Tras modificar cualquier archivo en `docs/`, se ejecuta:

   ```
   python scripts/sync_docs_html.py
   ```

   El script inyecta el 100% del texto íntegro en `docs/index.html`, verifica mediante aserciones carácter por carácter que no haya discrepancias, y compila el árbol estático completo en `site/`.

🎲 04. Modelo del Juego y Reglas Tácticas

Topología hexagonal, LOS, dados, combate, moral, State Loop autoritativo.

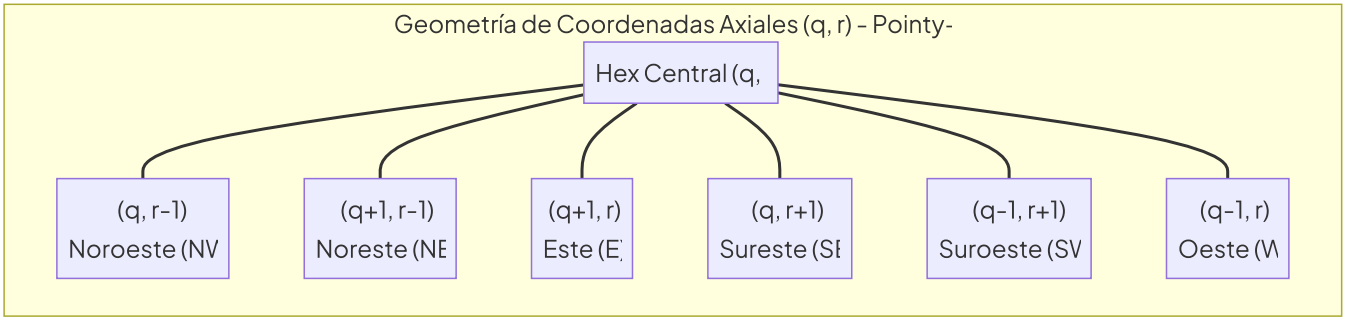
Modelo del Juego y Reglas Tácticas: Assault Tactical Engine

> **Estado**: Canónico y Exhaustivo
> **Ámbito**: Motor de reglas `assault_model/` y su representación matemática en `voec_sim/`.

---

1. Topología Hexagonal y Geometría del Tablero

El campo de batalla de *Assault* se modela como una cuadrícula hexagonal regular orientada (*pointy-topped* o *flat-topped*). A diferencia de las cuadrículas cartesianas rectangulares (donde cada celda tiene 4 vecinos ortogonales y 4 diagonales dispares), **cada hexágono posee exactamente 6 vecinos equidistantes**.



1.1. Sistema de Coordenadas Axiales y Cúbicas

Cada hexágono se indexa mediante un par axial (q, r) o un trío cúbico balanceado (x, y, z) donde:

$$x = q, \quad z = r, \quad y = -x - z \iff x + y + z = 0$$

- **Distancia Hexagonal Exacta**:

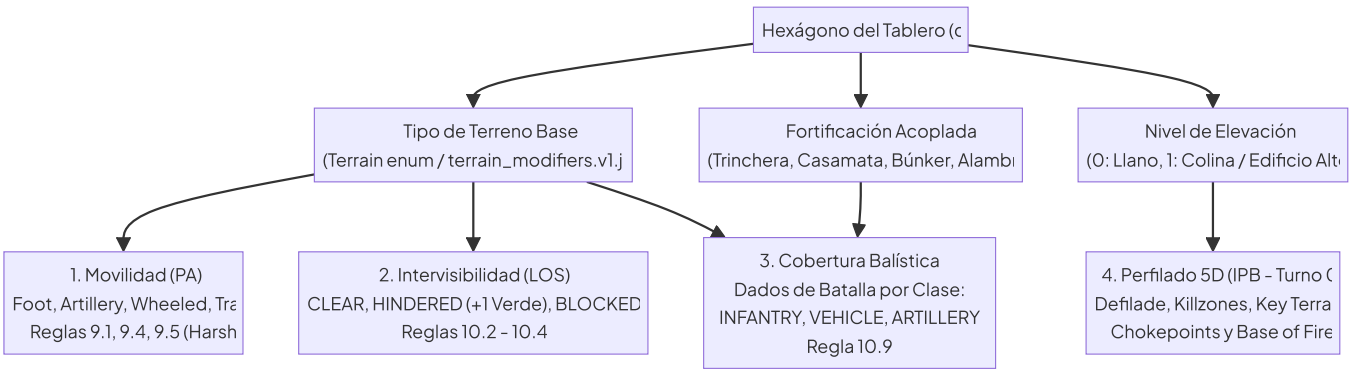
$$\text{dist}(h_1, h_2) = \frac{|q_1 - q_2| + |(q_1 + r_1) - (q_2 + r_2)| + |r_1 - r_2|}{2}$$

- **Adyacencia Estricta**: Dos hexágonos son adyacentes si y sólo si $\text{dist}(h_1, h_2) = 1$. No existen "esquinas diagonales".

---

2. Tipos de Terreno, Elevación, Cobertura y Topografía (IPB)

El terreno en el motor táctico *Assault* no es un mero fondo estético, sino un componente físico y geométrico autoritativo que condiciona el movimiento en Puntos de Acción (PA), la Línea de Visión (LOS), la cobertura balística de defensa y la planificación doctrinal de la IA.



2.1. Taxonomía Canónica de Terrenos del Motor

El motor implementa 11 tipos de terreno canónicos formalizados en el enum `Terrain` (`assault_model/map/terrain.py`) y gobernados por la tabla canónica `assets/rules_tables/terrain/terrain_modifiers.v1.json`:

Tipo de Terreno	Clave Técnica	Estado LOS	Defensa Infantería	Defensa Blindados	Defensa Artillería	Coste Movimiento (PA)	Restricciones y Reglas Especiales
Campo Despejado	`clear`	`CLEAR`	1 VERDE	Ninguno	Ninguno	Pie: 1, Art: 1, Ruedas: 1, Cadenas: 1	Terreno llano abierto; máxima exposición a fuego cruzado directo.
Agua / Mar / Río	`water`	`CLEAR`	Ninguno	Ninguno	Ninguno	**Impasable** (None)	Barrera infranqueable sin pontones o capacidad anfibia (Regla TM-R02).
Terreno Pedregoso	`rocky`	`CLEAR`	1 AZUL + 1 AMARILLO	1 VERDE	1 VERDE	Pie: 1, Art: 2, Ruedas: 2, Cadenas: 1	Cobertura natural sólida para infantería sin obstaculizar la visión.
Olivares / Viñedos	`olive_vine_grove`	`HINDERED`	2 VERDES	1 AZUL	1 AZUL	Pie: 1, Art: 2, Ruedas: 2, Cadenas: 2	Vegetación cultivada; estorba la visión directa (+1 Verde a quien defiende).
Bosque Ligero	`light_forest`	`HINDERED`	1 AMARILLO + 1 VERDE	1 VERDE	1 AZUL	Pie: 2, Art: 2, Ruedas: 3, Cadenas: 2	Arboleda dispersa; estorbo visual y buena protección balística para infantería.
Bosque Denso	`heavy_forest`	`BLOCKED`	2 AMARILLOS + 1 VERDE	1 VERDE	1 VERDE	Pie: 2, Art: 3, Ruedas: Impasable, Cadenas: 3	Bloquea totalmente la LOS; `no_indirect_from: true` (prohibido fuego indirecto interior).
Matorral / Maleza	`brush`	`HINDERED`	1 VERDE + 1 AZUL	1 VERDE	1 VERDE	Pie: 1, Art: 1, Ruedas: 2, Cadenas: 1	Vegetación baja típica mediterránea; ocultamiento parcial y avance rápido a pie.
Edificio Simple	`building_single`	`BLOCKED`	2 AMARILLOS + 1 AZUL	1 AZUL	1 VERDE	Pie: 2, Art: Impasable, Ruedas: Impasable, Cadenas: Impasable	Estructura rural/urbana de 1 planta (Elevación 0); `no_indirect_from: true`; colapsable a ruinas.
Edificio Multi-Planta	`building_multi`	`BLOCKED`	1 ROJO + 2 VERDES	1 AZUL	1 VERDE	Pie: 2, Art: Impasable, Ruedas: Impasable, Cadenas: Impasable	Estructura pesada de mampostería (Elevación 1); torre de observación; protección superior.
Colina / Elevación	`hill`	`CLEAR`	1 VERDE (Cresta)	1 VERDE	1 VERDE	+1 PA adicional al subir nivel	Cota dominante (Elevación 1); genera zonas ciegas tras la cresta.
Ruinas Urbanas	`ruins`	`BLOCKED`	2 AMARILLOS + 1 AZUL	1 VERDE	1 VERDE	Pie: 2, Art: Impasable, Ruedas: Impasable, Cadenas: 3	Restos de edificios colapsados tras detonación de zapadores o bombardeo pesado.

2.2. Mecánicas de Movimiento y Costes de Paso en PAs (Reglas 9.1, 9.4, 9.5)

El motor de movimiento (`assault_model/rules/movement_rules.py` y `movement_terrain_rules.py`) evalúa el coste en PAs para acceder a cada celda contigua según el sistema de tracción de la unidad:

1. **Categorías de Locomoción**:
    - **Infantería a pie (`foot`)**: Gran agilidad táctica; puede penetrar en bosques densos y edificios donde los vehículos quedan bloqueados.
    - **Artillería remolcada / arrastrada (`artillery`)**: Severamente penalizada en bosques y pedregales; incapaz de entrar en edificios cerrados.
    - **Vehículos de Ruedas (`wheeled`)**: Movilidad máxima en campo despejado y carreteras (1 PA); bloqueados en edificios y bosques densos.
    - **Vehículos de Cadenas (`track`)**: Tracción oruga todoterreno; pueden abrirse paso a través de ruinas y bosques densos a coste elevado (3 PA), pero no en edificios intactos.
2. **Regla de Terreno Difícil (*Harsh Terrain* – Regla 9.5)**:
    - Si una unidad dispone de menos PAs restantes que el coste estricto exigido para entrar en un hexágono vecino, pero dicho coste es menor o igual que su capacidad total de movimiento del turno ($extCosteEntrada \leq extMaxMP$), la unidad puede **consumir la totalidad de sus PAs restantes** para ejecutar el avance y ocupar el hexágono.
    - Esto evita que unidades lentas queden bloqueadas por falta de 1 PA fraccionario en terreno accidentado.
3. **Restricción de Terreno Impasable (`water` – Regla TM-R02)**:
    - Los hexágonos de agua profunda o mar son físicamente infranqueables para infantería regular y blindados estándar. El pathfinder vectorial descarta cualquier arista que conduzca a estas celdas.

2.3. Intervisibilidad, Línea de Visión (LOS) y Niebla de Guerra (Reglas 10.2 a 10.4)

El motor balístico y de visibilidad (`assault_model/combat/line_of_sight.py`) proyecta un rayo analítico continuo centro-a-centro entre el atacante y el defensor en coordenadas de pantalla hexagonal (*odd-r pointy-top*):

1. **Estados de Visibilidad**:
    - `CLEAR` **(Despejado)**: El rayo no intersecta ningún obstáculo visual. La línea de tiro es diáfana y directa.
    - `HINDERED` **(Obstaculizado / Estorbo Visual)**:
        - El rayo atraviesa vegetación, olivares o matorrales (`brush`, `light_forest`, `olive_vine_grove`).
        - **Efecto Balístico (Regla 10.9.2)**: El estorbo dispersa la trayectoria de los proyectiles y otorga al defensor **+1 Dado VERDE de defensa adicional** por cada línea de fuego obstaculizada.

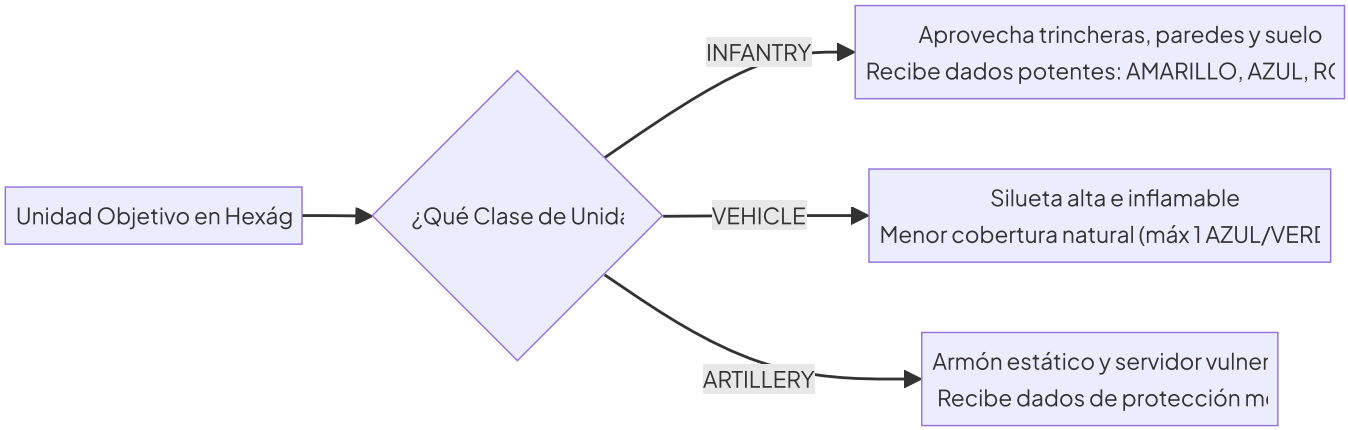
- **Regla de Acumulación de Estorbos (Regla 10.3)**:
  - Cuando el rayo de visión atraviesa **3 o más hexágonos de estorbo** ($exthindrance \geq 3$), la acumulación de masa vegetal absorbe completamente la visibilidad y la línea de visión pasa a considerarse `BLOCKED` **(Bloqueada)**.
- `BLOCKED` **(Bloqueo Total)**:
  - El rayo intersecta un bosque denso (`heavy_forest`), una estructura urbana (`building_single` , `building_multi` , `ruins`) o una colina interpuesta.
  - Queda terminantemente prohibido el fuego directo. Solo el fuego indirecto parabólico (morteros, artillería) o el asalto cuerpo a cuerpo pueden resolver objetivos en este estado.

2. **Prohibición de Fuego Indirecto desde Terreno Cubierto (`no_indirect_from: true`)**:
   - Las unidades situadas físicamente dentro de edificios o bajo la bóveda de bosques densos tienen prohibido disparar armas indirectas (morteros de 50mm/81mm) hacia el exterior debido a la imposibilidad física de elevar tubos con ángulos parabólicos elevados bajo techumbres o masa de copas arbóreas.

2.4. Modificadores Defensivos por Clase de Unidad y Dados de Batalla (Regla 10.9)

La cobertura del terreno no otorga un bono plano idéntico a todas las tropas, sino que discrimina según la silueta y vulnerabilidad del blanco (`INFANTRY` , `VEHICLE` , `ARTILLERY`):

$$ ext{Pool}{ ext{DefensaTotal}} = ext{DefensaBase}{ ext{Unidad}}(ext{Sector}) + ext{Dados}{ ext{Terreno}}(ext{Clase}) + ext{Dados}{ ext{Fortificación}} + ext{Estorbo}_{ ext{HINDERED}} (+1 ext{ VERDE})$$



- **Infantería en Edificios Urbanos**: Recibe la protección más formidable del wargame (`RED + GREEN + GREEN` en multi-planta, `BLUE + YELLOW + YELLOW` en planta simple), haciendo casi imposible su neutralización por fuego directo de fusilería sin apoyo de zapadores o cañones de demolición.
- **Despojo de Dados de Terreno por Habilidades Especiales**:
  - **Francotirador (`SNIPER` – Regla 10.9)**: Elimina el dado de terreno más potente del defensor.
  - **Lanzallamas (`FLAMETHROWER` – Regla 11.0)**: Despoja el dado defensivo más fuerte del terreno y causa supresión automática inmediata al arder la cobertura.
  - **Dispersión de Artillería (`REMOVE_WEAKEST_TERRAIN_DEFENSE`)**: Munición pesada que neutraliza la cobertura ligera.

2.5. Elevación, Crestas y Geometría Vertical (Reglas 9.6.1, 10.4)

1. **Niveles de Altitud Relativa**:
   - **Nivel 0**: Superficie estándar del tablero (campo abierto, matorral, bosques, olivares).
   - **Nivel 1**: Colinas (`hill`) y azoteas de edificios multi-planta (`building_multi`).
2. **Ventaja Táctica de Cresta (*Commanding Height*)**:
   - Una unidad en Nivel 1 que dispara sobre Nivel 0 domina el terreno circundante, ignora obstáculos intermedios de Nivel 0 si el ángulo cenital lo permite, y obtiene un bono de $+1$ en tiradas de detección (*spotting*).
3. **Zonas Ciegas en Desenfilada (*Dead Ground*)**:
   - Los hexágonos situados inmediatamente detrás de una colina o edificio elevado respecto al ángulo del observador quedan en sombra visual (*desenfilada*), impidiendo el fuego directo y sirviendo de corredores de infiltración protegida.
4. **Restricción de Unidades Pesadas (`HEAVY_WEIGHT`)**:
   - Piezas pesadas de artillería o anticarro con el rasgo `HEAVY_WEIGHT` (como el PaK 40 o cañones de 88mm) tienen terminantemente prohibido ascender o descender desniveles de colina de forma autónoma sin transporte mecanizado.

2.6. Fortificaciones Acopladas al Terreno (Reglas 9.6.4, 10.9.4)

Sobre cualquier hexágono de terreno pueden emplazarse obras defensivas canónicas (`assets/rules_tables/fortification/fortification_modifiers.v1.json`), cuyas ventajas se suman a las del terreno base:

Fortificación	Clave Técnica	Bono Defensivo Infantería (Frontal / Flancos)	Penalización Movimiento	Vulnerabilidad / Anti-Fortificación
Trinchera	`trench`	+1 AMARILLO (Frontal y Flancos)	+1 PA a pie y artillería	Protección sólida contra esquirlas y morteros; vulnerable a asalto.
Casamata	`casemate`	+1 ROJO + 1 AMARILLO (Todos los sectores)	+1 PA a pie; ruedas/cadenas 1	Troneras reforzadas de hormigón; vulnerable a cargas de demolición adyacentes.
Búnker	`bunker`	+2 ROJOS + 1 AMARILLO (Frontal)	+1 PA a pie; ruedas/cadenas 1	Fortaleza acorazada con protección superior; exige zapadores con satchels.
Pozo / Emplazamiento	`gun_emplacement`	+1 AMARILLO + 1 VERDE (Todos sectores)	0 PA a pie; +1 ruedas/cadenas	Parapeto de sacos terreros óptimo para HMGs y cañones de apoyo.
Alambrada	`barbed_wire`	Ninguno (Obstáculo lineal)	+1 PA a pie; ruedas bloqueadas	Frena el asalto de infantería; destruible con torpedos Bangalore.
Campo de Minas	`minefield`	Ninguno (Trampa explosiva)	Detonación al entrar (D6)	Ataque automático en bajos; neutralizable con acción `MINE_DETECTION` .

2.7. Mutabilidad del Terreno y Demolición Estructural (Regla 12.5)

El terreno no es inmutable durante la partida:

1. **Colapso Estructural por Zapadores (`EXPLOSIVE_CHARGE`)**:
   - Cuando una escuadra de ingenieros detona una carga satchel sobre un edificio (`building_single` o `building_multi`), el atacante tira 1 dado ROJO contra el dado de defensa más fuerte de la estructura.
   - Si se consiguen 2 o más impactos o un impacto crítico, **el edificio colapsa físicamente y muta su tipo de terreno a ruinas urbanas (`ruins`)**, eliminando la elevación vertical y sepultando o expulsando a sus ocupantes.
2. **Cráteres de Bombardeo**:
   - El fuego masivo de artillería transforma los hexágonos despejados en cráteres de metralla (`Terrain Crater Clear`), otorgando a la infantería que avance a posteriori un refugio improvisado (+1 dado de defensa).

2.8. Perfilado Topográfico Automatizado 5D (IPB – Turno 0)

Para dotar a la red neuronal `EfficientZeroV2` y a los planificadores doctrinales (`MacroStrategyPlanner`) de comprensión geométrica antes de mover la primera ficha, el analizador topográfico (`assault_model/map/topography_analyzer.py`) calcula un vector 5D estático por cada hexágono:

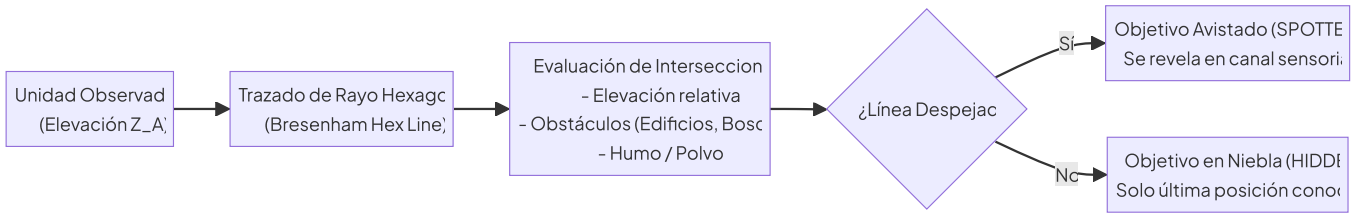
\mathbf{v}_{5D}(q, r) = \Big[ext{CoverRating}, ext{ElevationDominance}, ext{LOSExposure}, ext{ChokepointBottleneck}, ext{VPUtility}\Big]

A partir de esta evaluación, el simulador y el MLOps Dashboard (Puerto 8778) clasifican automáticamente el mapa en 5 categorías tácticas operativas:

- **Pasillos de Desenfilada (*Defilade Corridors*)**: Hexágonos con $ext{Cover} \geq 0.70$ y $ext{LOSExposure} \leq 0.35$. La IA los prioriza para la aproximación segura hacia los objetivos (+25% de peso doctrinal).
- **Zonas de Muerte (*Killzones*)**: Hexágonos llanos abiertos con $ext{LOSExposure} \geq 0.65$ batidos por múltiples líneas de fuego. El planner penaliza su tránsito (factor 0.55×-0.65×) salvo que cuente con pantalla de humo táctico.
- **Terreno Clave (*Key Terrain*)**: Alturas y crestas con visual directa y control sobre hexágonos de VPs.
- **Cuellos de Botella (*Chokepoints*)**: Pasos estrechos entre agua o edificios que canalizan el movimiento forzando emboscadas.
- **Base de Fuego (*Base of Fire*)**: Posiciones con alta cobertura (≥ 0.70) y amplio campo de tiro sobre el terreno clave; asignadas preferentemente a ametralladoras pesadas HMG y armas de apoyo.

---

3. Línea de Visión (LOS), Detección (*Spotting*) y Niebla de Guerra



1. **Trazado de Rayo Hexagonal**: Se calcula la línea de hexágonos interceptada entre el centro del hexágono observador y el centro del objetivo.
2. **Elevación y Puntos Ciegos**: Si un hexágono intermedio posee una elevación $Z_{obs} \geq \min(Z_A, Z_B)$, la LOS se bloquea inmediatamente salvo que la diferencia de altura del observador genere una parábola de visión sobre la cresta.

3. **Parcialidad de la Información (POMDP)**: Las unidades enemigas que no han disparado y se encuentran en terreno cubierto permanecen no detectadas (*unspotted*), lo que obliga a la IA a gestionar la incertidumbre mediante la cabeza táctica `threat_map` y el contexto temporal de la red.

---

4. Sistema de Combate: Dados de Batalla, Daño y Supresión

El combate táctico no es un mero cálculo numérico determinista; refleja la letalidad y el caos del combate de infantería a corta y media distancia.

4.1. Dados de Batalla (`BattleDie`)

El motor utiliza dados tácticos con caras personalizadas según el calibre, doctrina y tipo de munición del juego de mesa original:

- **Dados Blancos y Verdes**: Representan armamento ligero estándar (fusiles Garand M1, Kar98k, Carcano M91).
- **Dados Negros y Rojos**: Armamento pesado, ametralladoras pesadas colectivas (MG42, Breda 37, Browning M2) o proyectiles anticarro de alta velocidad.
- **Caras Posibles**:
  - `MISS` (Fallo): 0 impactos.
  - `HIT` (Impacto): 1 impacto potencial.
  - `DOUBLE_HIT`: 2 impactos directos.
  - `SUPPRESSION`: Impacto moral; induce caída de cohesión sin causar necesariamente bajas físicas.
  - `CRITICAL`: Fuerza una tirada en la tabla de impactos críticos (rotura de arma, baja de oficial, pánico).

4.2. Resolución de Combate a Distancia (*Ranged Combat*)

$$\text{Impactos Netos} = \max\left(0, \sum \text{Hits}_{\text{Ataque}} - \sum \text{Saves}_{\text{Defensa}}\right)$$

1. **Composición del Pool Atacante**:

$$\text{Pool}_{\text{atk}} = \text{ArmaBase} + \text{ModificadorRango} + \text{ModificadorFlanco}$$

2. **Composición del Pool Defensor**:

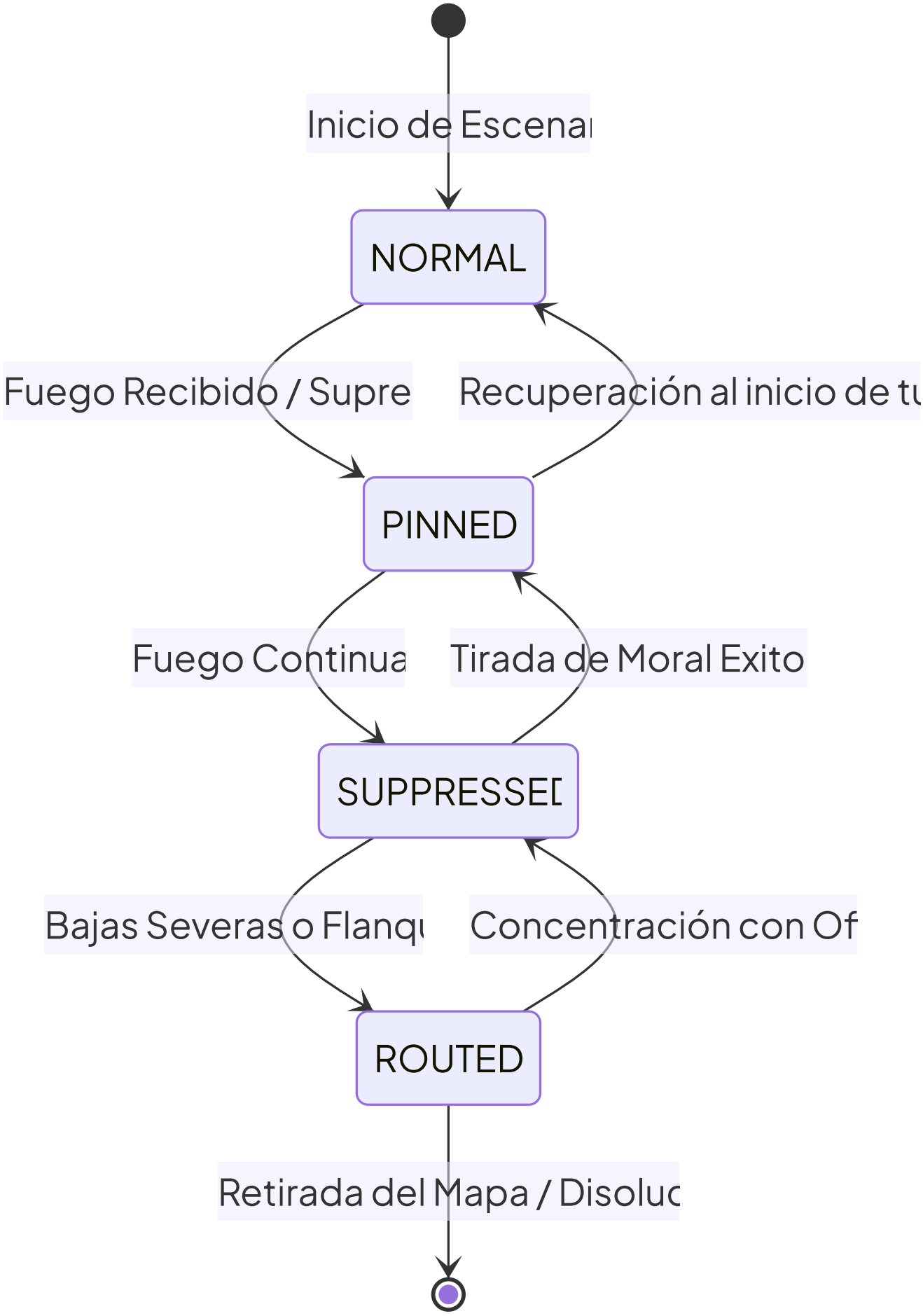
$$\text{Pool}_{\text{def}} = \text{DefensaUnidad} + \text{CoberturaTerreno} + \text{Trinchera/Fortificación}$$

3. **Asignación de Bajas**: Cada impacto neto reduce los Puntos de Vida (HP) de la escuadra. Al llegar a 0 HP, la unidad es eliminada del tablero.

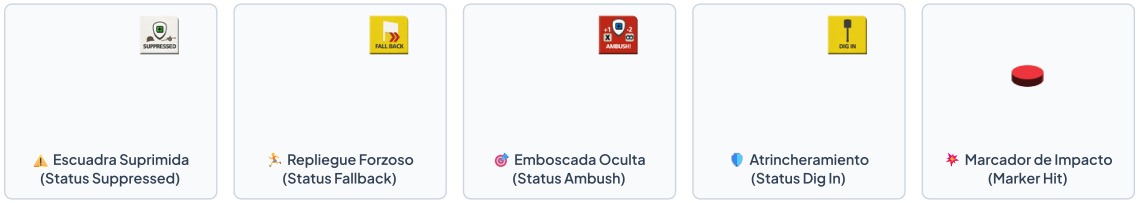
---

5. Estados de Cohesión y Moral de la Escuadra

Las escuadras transitan dinámicamente entre 4 estados de moral que restringen severamente su capacidad operativa:


Inicio de Escena
NORMAL
Fuego Recibido / Supre
Recuperación al inicio de tu
PINNED
Fuego Continua
Tirada de Moral Exito
SUPPRESSED
Bajas Severas o Flanqu
Concentración con Of
ROUTED
Retirada del Mapa / Disoluc

- **NORMAL**: Operatividad al 100%. Todos los puntos de acción y capacidad de fuego defensivo disponibles.
- **PINNED (Clavada)**: La escuadra busca cobertura inmediata. Pierde el 50% de sus puntos de movimiento; el fuego defensivo sufre una penalización de -1 dado.
- **SUPPRESSED (Suprimida)**: Incapaz de moverse o disparar voluntariamente. Solo puede buscar refugio o intentar reorganizarse.
- **ROUTED (En Desbandada)**: Ruptura de la cohesión. La escuadra huye en dirección opuesta al enemigo hacia el borde del mapa. Si recibe combate en este estado, es capturada o destruida.



---

6. Fuego de Reacción (*Opportunity Fire*)

Cualquier unidad en estado `NORMAL` con línea de visión despejada que no haya agotado su capacidad de reacción puede interrumpir el movimiento de una unidad enemiga en el momento en que ésta penetra en su arco de fuego:

- **Disparador**: Entrada en un hexágono con menor cobertura o desplazamiento a campo abierto.
- **Consecuencia para la IA**: El agente no puede realizar movimientos lineales despreocupados; cruzar una calle o un campo abierto frente a una ametralladora enemiga (`HMG`) desencadena una ráfaga inmediata que puede clavar (`PINNED`) o destruir la unidad antes de completar su destino.

---

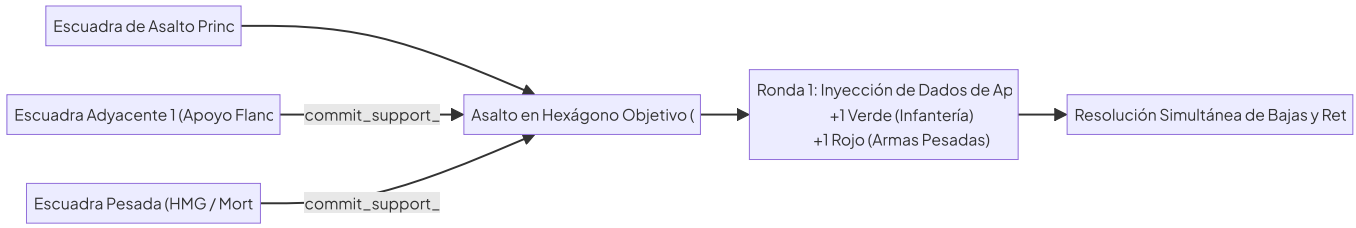
7. Asaltos Cuerpo a Cuerpo (*Close Combat / Melee*)

Cuando una escuadra entra físicamente en el mismo hexágono ocupado por tropas enemigas, se produce un **Asalto Inmediato**:

- No se aplican modificadores de distancia (alcance 0).
- Se calcula el ratio de combate cuerpo a cuerpo relativo.
- Es una resolución simultánea y letal: el perdedor es aniquilado o forzado a una retirada desesperada, sufriendo chequeos de moral inmediatos.

7.1. Descomposición Secuencial de Asaltos Combinados Multi-Unidad (Palma et al. 2025, Sec. 3.3)

Para reflejar asaltos combinados doctrinales donde una unidad asalta mientras escuadras adyacentes proporcionan apoyo de supresión y cobertura antes de la melé cuerpo a cuerpo:



1. **Sub-Fase de Compromiso de Apoyo (`commit_support_unit`)**:
   - En `assault_model/combat/close_combat_context.py` , el objeto `CombatResolutionContext` permite registrar escuadras adyacentes que apoyan la maniobra tanto en el bando atacante (`supporting_attackers`) como en el defensor (`supporting_defenders`).
2. **Inyección de Dados de Apoyo en Ronda 1**:
   - En `assault_model/combat/close_combat_resolver.py` , durante la primera ronda del asalto, cada unidad de apoyo viva y no suprimida aporta un dado de ataque adicional:
     - **Infantería Estándar (Fusileros / Asalto)**: $+1$ dado `DiceColor.GREEN` .
     - **Armas Pesadas (`MMG` , `HMG` , `MORTAR`)**: $+1$ dado de alta letalidad `DiceColor.RED` .
3. **Manejo de Bajas y Continuidad**:
   - Si tras la ronda 1 ambos bandos conservan unidades vivas en el hexágono, el combate continúa hasta que un bando es aniquilado o forzado a romper el contacto.

---

8. Condiciones de Victoria y Puntos de Victoria (VP)

El desenlace de una partida se rige por una combinación de **control territorial** y **desgaste**:

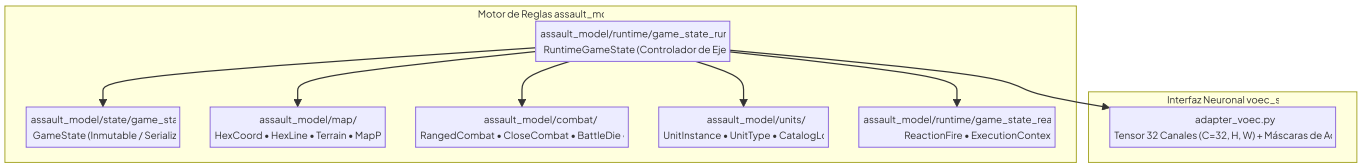
1. **Control de Puntos de Victoria (VP)**: Hexágonos estratégicos designados (cruces de carreteras, puentes, colinas clave) que otorgan puntos por turno y bonificaciones al final de la partida.
2. **Balance de Bajas (Ratio de Desgaste)**: Puntos otorgados por la eliminación de escuadras enemigas versus penalización por bajas propias.

3. **Límite de Turnos**: Si ningún bando alcanza la victoria por aniquilación antes del límite temporal, el ganador se decide por el diferencial neto de VP.

---

9. Arquitectura Modular del Motor `assault_model/`

El motor de reglas está desacoplado siguiendo el patrón de **Estado Inmutable / Runtime de Mutación**:



9.1. Responsabilidades de los Paquetes Internos

1. `assault_model/state/` (`GameState`, `TurnState`):
   - Contenedor puro de datos serializable en disco o memoria.
   - Almacena el estado de cada celda (`HexState`), el inventario y posición de cada unidad (`UnitInstance`), el marcador de control (`HexOwnership`, `VPTracker`) y el registro de activación por bando.
   - **No contiene lógica de cálculo de combate ni reglas de movimiento**.
2. `assault_model/runtime/` (`RuntimeGameState`, `ExecutionContext`):
   - Orquestador autoritativo del ciclo de simulación.
   - Gestiona el sistema de activación alternada (I-Go-U-Go o impulso táctico).
   - Invalida cachés espaciales mediante `_cache_version` tras cualquier mutación del tablero.
   - Supervisa las interrupciones por fuego de oportunidad (`game_state_reactions.py`).
3. `assault_model/map/` (`HexCoord`, `HexLine`, `TerrainConfig`):
   - Aritmética hexagonal axial (q, r) y cúbica (x, y, z).
   - Algoritmo de ray-casting hexagonal para trazado de Línea de Visión (LOS) y detección de obstáculos.
   - Definición de costes de paso por tipo de terreno y elevación.
4. `assault_model/combat/` (`RangedCombatResolver`, `CloseCombatResolver`, `BattleDie`, `MoraleState`):
   - Cálculo estocástico de dados de combate (`attack_dice_pool.py`, `defense_dice_pool.py`).
   - Resolución de impactos directos, impactos morales de supresión y tablas de efectos críticos.
   - Transiciones de estado de cohesión de tropas (`NORMAL` → `PINNED` → `SUPPRESSED` → `ROUTED`).

9.2. Auditoría Forense de Rendimiento, Deuda Técnica y Hoja de Ruta de Modernización

Tras sucesivas expansiones del motor, se ejecutó una auditoría empírica con `cProfile` sobre el escenario canónico `battaglia_cittadina_2_1` durante 100 pasos continuos (109 pasos/segundo, 3.68 millones de operaciones). El análisis arrojó los siguientes hallazgos y directivas:

1. Diagnóstico de Ineficiencias de Cómputo

- **Hiper-Reflexión Dinámica:** Se registraron **1,089,327 llamadas a `getattr` y `hasattr`** en solo 100 pasos, causadas por duck-typing defensivo en entidades de dominio (`UnitInstance`, `HexCoord`).
- **Regeneración Redundante de Características Espaciales:** En `voec_sim/core/simulator.py`, la variable `_static_spatial_features` se restablecía a `None` en cada `step()`, forzando la recomputación completa de hexágonos, costes de movimiento, cobertura y obstáculos en cada paso (consumiendo el **21% del tiempo de simulación** en datos invariantes).
- **Multiplicación de Raycasting en Avistamiento (*Spotting*):** Se ejecutaron **36,432 evaluaciones de LOS** (`check_line_of_sight`) en 100 pasos debido a comprobaciones globales de todas las unidades contra todos los enemigos, sin descarte previo por distancia Manhattan (*broad-phase culling*).
- **Indexado Repetitivo de Ocupantes en Dijkstra:** `MovementRules.get_legal_paths` reconstruía un diccionario local de unidades en cada consulta de movimiento en lugar de reutilizar el índice espacial `_units_by_hex` de `GameState`.

2. Detección de Código Legacy y Descomisionamiento

- **Pila Obsoleta PPO / SB3 en `assault_sim/`:** Archivos como `training_env.py` (40 KB), `sim_env.py`, `ppo_config.py`, `eval_sb3.py` y modelos `ppo_US*.pt` pertenecen a una etapa anterior previa a EfficientZero V2. Deben ser archivados y desacoplados del motor activo.
- **Unificación de Catálogos:** Consolidar `assets/scenarios/` y `assets/catalogs/` como la única fuente de verdad, eliminando copias desincronizadas en subcarpetas heredadas.

3. Fidelidad Estricta al Reglamento Oficial (Sound of Drums rev6)

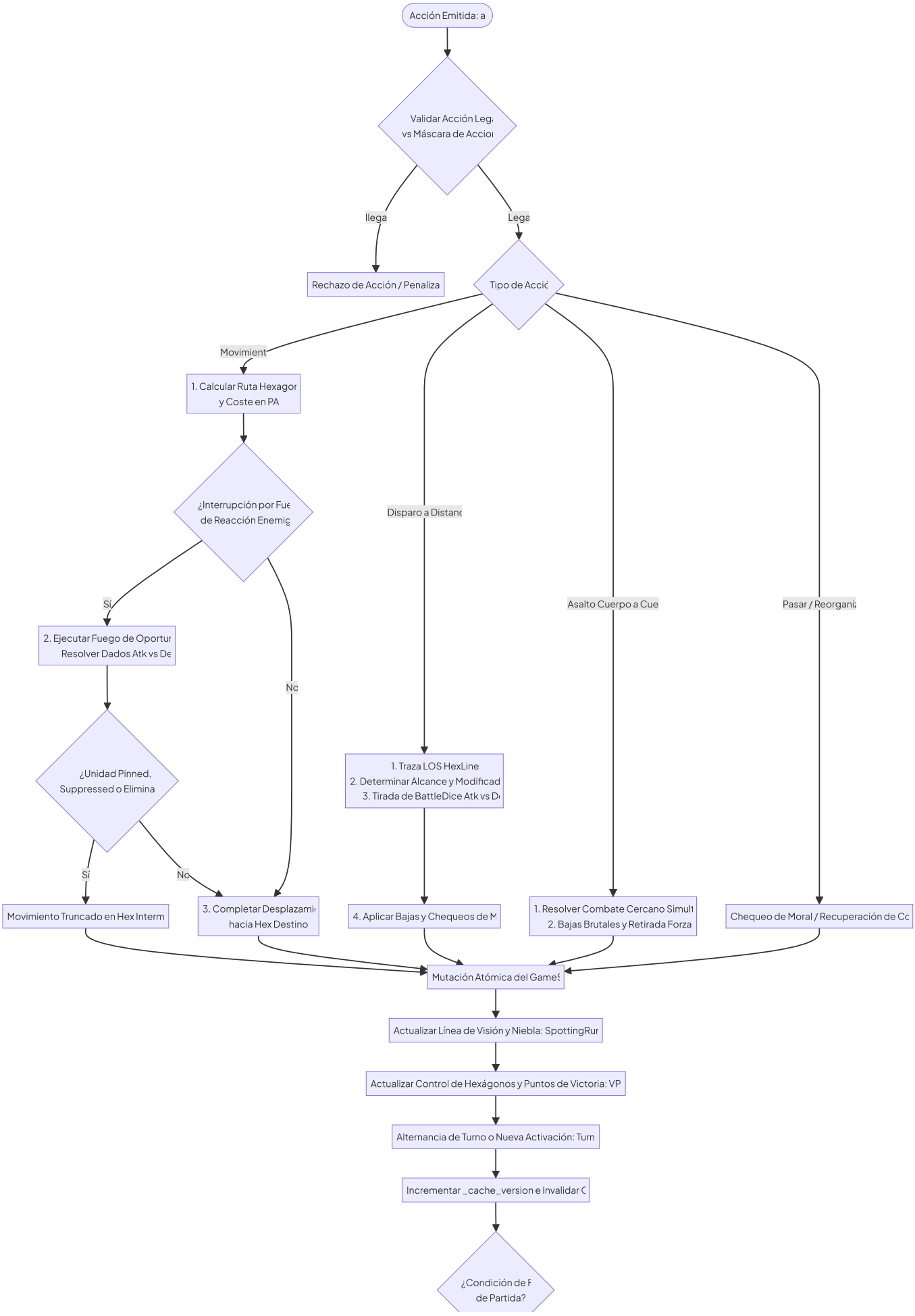
- **Principio de No Invención:** El motor implementa fielmente las reglas de combate, dados, moral y cobertura. Las áreas pendientes documentadas en el manual oficial (Transporte y remolque §9.11, ciclo de vida de humo §7.2, mazo de mando y puntos CP §4.9) se abordarán por fases respetando

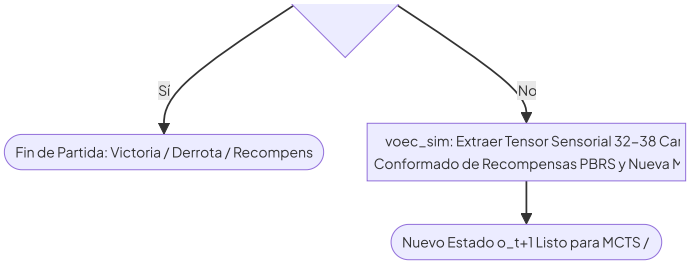
escrupulosamente los procedimientos de las 52 páginas del reglamento.

---

10. Ciclo Autoritativo de Mantenimiento del Estado (*State Loop*)

El siguiente diagrama detalla cómo el motor procesa cada acción individual de un agente o jugador humano, garantizando la consistencia absoluta de las reglas antes de devolver el nuevo estado y la recompensa asociada:


Acción Emitida: a
Validar Acción Leg
vs Máscara de Accio
Ilega
Lega
Rechazo de Acción / Penaliza
Tipo de Acci
Movimient
1. Calcular Ruta Hexagor
y Coste en PA
¿Interrupción por Fue
de Reacción Enemig
Disparo a Distanc
Asalto Cuerpo a Cue
Pasar / Reorgani
Sí
2. Ejecutar Fuego de Oportu
Resolver Dados Atk vs De
No
¿Unidad Pinned,
Suppressed o Elimina
1. Traza LOS HexLine
2. Determinar Alcance y Modificad
3. Tirada de BattleDice Atk vs D
Sí
No
Movimiento Truncado en Hex Interm
3. Completar Desplazami
hacia Hex Destino
4. Aplicar Bajas y Chequeos de M
1. Resolver Combate Cercano Simult
2. Bajas Brutales y Retirada Forza
Chequeo de Moral / Recuperación de Cc
Mutación Atómica del Games
Actualizar Línea de Visión y Niebla: SpottingRur
Actualizar Control de Hexágonos y Puntos de Victoria: VP
Alternancia de Turno o Nueva Activación: Turn
Incrementar _cache_version e Invalidar C
¿Condición de F
de Partida?



---

11. Marco de Conformado de Recompensas Invariante (PBRS, Urgencia y Parálisis)

Para resolver el problema del aprendizaje con recompensas dispersas sin alterar el conjunto de políticas óptimas del wargame (teorema de Ng et al., 1999), la capa `voec_sim/core/reward_calculator.py` implementa un andamiaje matemático estricto:

11.1. Potencial Radial a VPs (Φ) e Invariancia Telescópica

El potencial del estado respecto a los Puntos de Victoria se define como: \Phi(s) = \sum_{u \in \text{Amigos}} \frac{w_u}{1 + \min_{\text{VP}} d(u, \text{VP})} + B_{\text{ctrl}} \cdot N_{\text{VP_Controlados}} donde w_u pondera el rol táctico de la unidad (ej. $+25$ para unidades de choque `SPEARHEAD`) y $B_{\text{ctrl}} = 2.0$ bonifica el control efectivo.

La función de recompensa adicional por paso satisface:

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

Garantizando que al final del episodio la suma acumulada sea telescópica $\sum_{t=0}^{T} \gamma^t F(s_t, a_t, s_{t+1}) = -\Phi(s_0)$ (ya que en estado terminal $\Phi(s_T) \equiv 0.0$), preservando al 100% las garantías de convergencia óptima.

11.2. Factor de Presión Temporal Dinámica (Ψ_{urgency})

Siguiendo la extensión de PBRS para MDPs dependientes del tiempo (Devlin & Kudenko, 2012), cuando el bando amigo controla 0 VPs, el potencial de distancia se atenúa exponencialmente conforme avanza el horizonte de la batalla ($\tau = t / T_{\max}$):

$$\Phi(s, t) = \Phi_{\text{dist}}(s) \cdot \exp\left(-\lambda \cdot \frac{t}{T_{\max}}\right)$$

con $\lambda = 1.2$. Esto introduce una severa degradación en el valor del estado si la fuerza amiga deambula o se retrasa, incentivando una agresión inmediata y decisiva hacia los objetivos.

11.3. Penalización Acumulativa por Parálisis de Unidades Expuestas

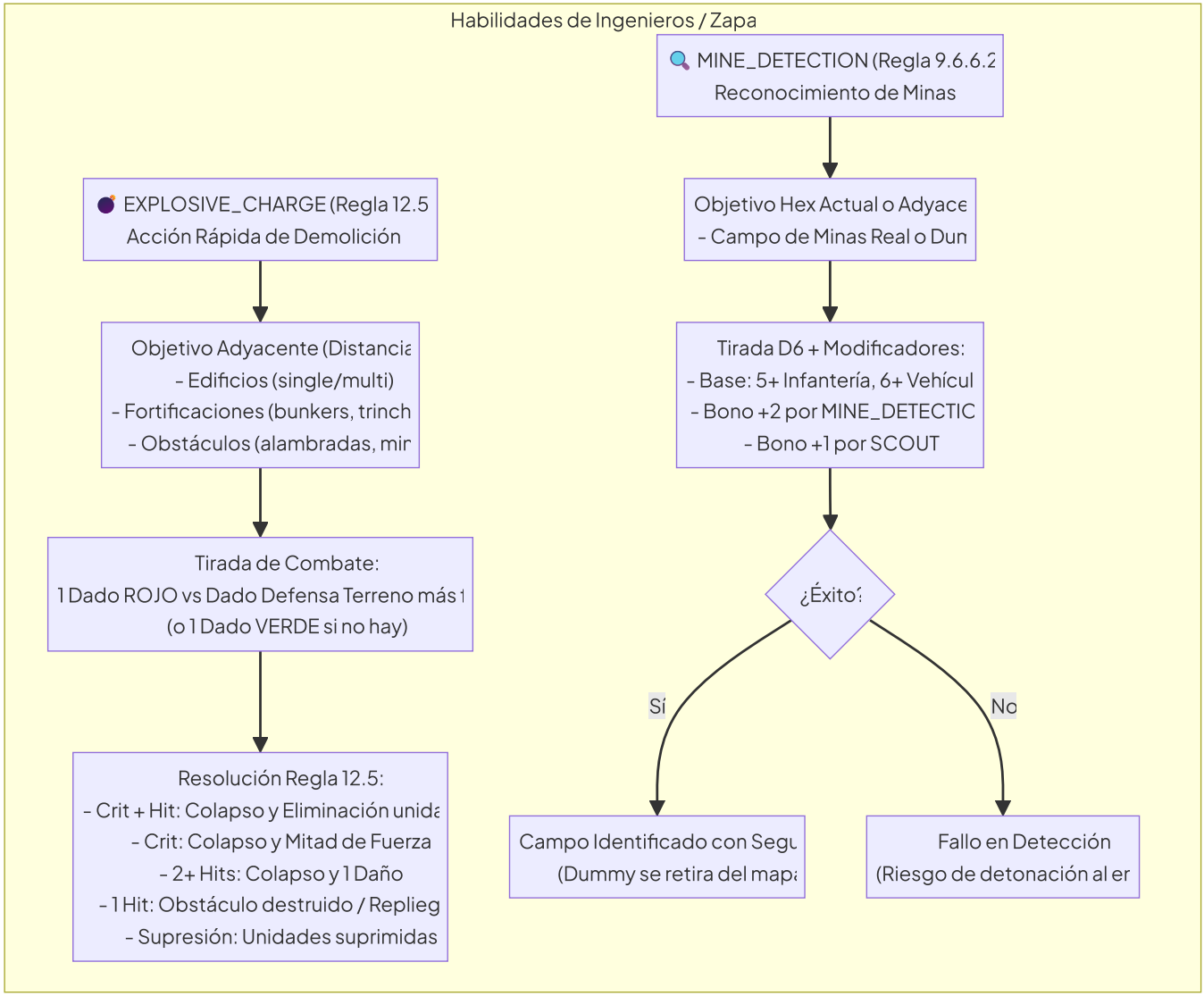
Cualquier unidad que permanezca en terreno desprovisto de cobertura defensiva (`cover < 0.35`) fuera del perímetro de un VP controlado, y que pase el turno sin avanzar hacia un objetivo ni abrir fuego, incurre en una penalización por parálisis: $$\text{Penalty}{\text{paralysis}} = -w{\text{paralysis}} \cdot \left(1.0 + 0.5 \cdot \min(4, \text{streak} - 1)\right)$$

- La racha acumula un $+50$ adicional por cada turno de pasividad consecutiva (hasta $4\times$).
- Se resetea de forma inmediata a cero en cuanto la unidad realiza una maniobra de aproximación a un VP, abre fuego o entra en una posición con cobertura defensiva adecuada.

---

12. Habilidades Especiales de Ingenieros y Zapadores: Carga Explosiva y Detección de Minas

Siguiendo las reglas canónicas del wargame (*Assault Rulebook* §9.6.6.2 y §12.5) y las especificaciones de las cartas de unidades especializadas (ej. *Combat Engineers 43 / US_COMBAT_ENGINEERS_43*), el motor integra dos acciones tácticas dedicadas:



12.1. Carga Explosiva (`EXPLOSIVE_CHARGE` – Regla 12.5)

- **Tipo de Acción**: `ActionType.EXPLOSIVE_CHARGE` , `CombatMode.EXPLOSIVE_CHARGE` (`ExplosiveChargeAction`).
- **Condición de Generación**: Unidad activa viva que posee el rasgo `EXPLOSIVE_CHARGE` adyacente a un hexágono con edificios, fortificaciones (`bunker` , `casemate` , `pillbox` , `gun_emplacement` , `trench` , `sandbag`) u obstáculos (`barbed_wire` , `minefield` , `tank_barricade`).
- **Procedimiento de Resolución**:
    1. El atacante tira **1 dado ROJO de combate** (`BattleDie(DiceColor.RED)`).
    2. El defensor tira el **dado de defensa de terreno más fuerte** disponible en el hexágono objetivo. Si no dispone de ningún dado de defensa de terreno (ej. campo de minas en terreno abierto o alambrada), tira **1 dado VERDE por defecto** (`BattleDie(DiceColor.GREEN)`).
    3. Se comparan los resultados mediante `compare_dice()` .
    4. Los impactos y críticos no cancelados dictan el colapso de la estructura y el daño colateral a las unidades en su interior:
        - `CRITICAL + HIT` : Estructura colapsa/destruida; todas las unidades en el hex son **eliminadas**.
        - `CRITICAL` : Estructura colapsa/destruida; todas las unidades quedan reducidas a **media fuerza**.
        - `2+ HITS` : Estructura colapsa/destruida; todas las unidades reciben **1 punto de daño**.
        - `1 HIT` : Los obstáculos lineales (alambradas, campos de minas) son **destruidos y retirados del mapa**. Los edificios y fortificaciones resisten el colapso pero las unidades interiores sufren **repliegue forzado** (*fallback*).
        - `SUPPRESSION` : Estructura no colapsa; unidades interiores quedan **suprimidas**.
    5. **Transformación Física del Terreno**:
        - Los edificios colapsados se transforman en **ruinas** (`Terrain.RUINS`).
        - Los búnkers y posiciones blindadas colapsan al estado de trinchera (`trench`).
        - Las trincheras, pozos de tirador, alambradas y minas se eliminan del tablero.

12.2. Detección y Reconocimiento de Minas (`MINE_DETECTION` – Regla 9.6.6.2)

- **Tipo de Acción**: `ActionType.MINE_DETECTION` (`MineDetectionAction`) y bonificador pasivo en movimiento.

- **Condición de Generación**: Unidad con el rasgo `MINE_DETECTION` que se encuentre en el mismo hexágono o adyacente a un marcador de campo de minas no identificado.
- **Mecánica del Bono**:
  - Tirada básica en D6: Requiere un resultado de $5+$ para Infantería y $6+$ para Vehículos/Artillería.
  - El rasgo `MINE_DETECTION` aplica un modificador directo de $+2$ **al dado D6** (acumulable con $+1$ de `SCOUT` o $+1$ de mina no oculta).
  - En caso de éxito, si el marcador es un señuelo (*dummy minefield*), es retirado del tablero inmediatamente; si es un campo de minas real, queda marcado como **identificado** (*identified*), permitiendo al zapador y a las unidades aliadas planificar su cruce o demolición con carga explosiva de manera segura.

12.3. Configuración e Inspección en la Interfaz del Editor de Unidades (Puerto 8780)

Ambas habilidades están plenamente expuestas en el servidor visual de catálogo (`tools/unit_editor_server.py`):

- Chips interactivos con estado activo, iconos visuales y tooltips reglamentarios detallados (`TRAIT_HELP`).
- Compatibilidad con importación/exportación de perfiles estándar, inventario de cartas y propuestas de unidades en `assets/catalogs/unit_catalog.json` y `assets/catalogs/unit_staging_proposals.json`.

12.4. Rebalanceo de OOB y Despliegue Dinámico de Humo en Escenarios Canónicos (Subtarea 3.5.4)

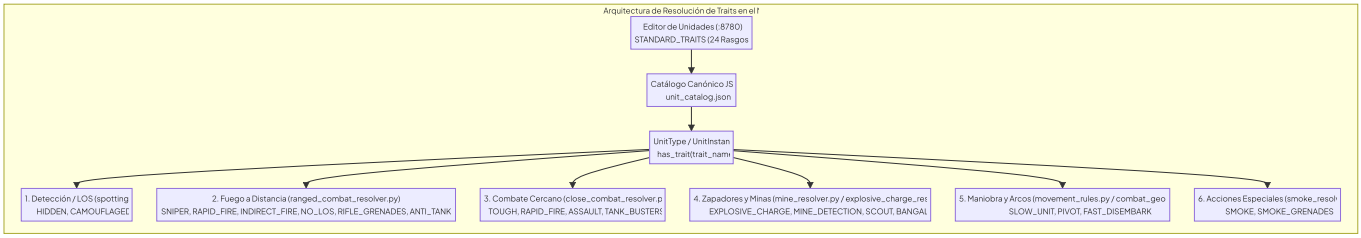
Para garantizar la viabilidad táctica y el equilibrio competitivo en los escenarios históricos de asalto urbano y desembarco anfibio de Gela, manteniendo estricta fidelidad al manual oficial (Regla 7.2, donde el humo es una acción de combate generada dinámicamente y nunca un obstáculo pre-ubicado estático):

- **Battaglia Cittadina (`assets/scenarios/battaglia_cittadina_2_1.json`)**:
  - Sustitución del equipo `US_BAZOOKA_TEAM` (ineficaz ante fortificaciones y casamatas sin blindados) por **1×** `US_COMBAT_ENGINEERS_43` en la columna de aproximación (`[7, 9]`).
  - **Humo Táctico Dinámico (Regla 7.2)**: Dotación de granadas de humo tácticas (`SMOKE_GRENADES` y modo de ataque `SMOKE` con dados `GREEN, GREEN, BLUE, BLUE`) a los zapadores, coordinados con el fuego indirecto del mortero de 81mm (`US_81MM_MORTAR`). Esto permite generar cortinas de humo en la calle mayor (`q=7, r=6`), cegando las líneas de visión directa de la casamata italiana (`IT_6` en `[3, 4]`) y del francotirador (`IT_7`), permitiendo el avance cubierto sin recurrir a marcadores estáticos artificiales.
- **Mettete i Piedi a Terra (`assets/scenarios/mettete_i_piedi_terra_1.json`)**:
  - Incorporación de **1×** `US_COMBAT_ENGINEERS_43` en el sector central de desembarco (`[4, 12]`), encarado frontalmente a la red de alambradas alemana (`q=4, r=9`).
  - **Capacidad de Brecha y Cegado Táctico**: Empleo de torpedos Bangalore (`BANGALORE_TORPEDO`) y cargas explosivas (`EXPLOSIVE_CHARGE`) para neutralizar la alambrada y campos de minas en los primeros 3 turnos, apoyados por fumígenos dinámicos de ingenieros y mortero para aproximarse bajo fuego de la MG42 alemana.
- **Efectividad Táctica Garantizada**: Con un perfil de asalto de 3 dados a bocajarro (`RED + RED + YELLOW`, 8 puntos de potencia de asalto) frente a defensas fijas de infantería regular (2 puntos), el ratio de asalto alcanza **4.0** ≥ 1.0, certificando la capacidad de ruptura y asalto coordinado.

---

13. Catálogo Exhaustivo de Habilidades de Unidades (Traits del Editor) y Estado de Implementación

El Editor Táctico de Tropas (`tools/unit_editor_server.py`) y el catálogo maestro (`assets/catalogs/unit_catalog.json`) gestionan **24 habilidades y rasgos doctrinales (`STANDARD_TRAITS`)**. Esta sección audita su formulación matemática, mecánica autoritativa en el motor (`assault_model/`), archivos de resolución y estado exacto de desarrollo.



13.1. Matriz Canónica de las 24 Habilidades del Editor

#	Habilidad (`Trait`)	Categoría Operativa	Efecto en Reglamento Oficial (*Assault: Sicily 43*)	Mecánica en Engine (`assault_model/`)	Módulos Responsables	Estado	Ur
1	`SNIPER`	Combate Fuego	El defensor pierde el dado de terreno más potente al ser atacado a distancia (solo infantería/artillería).	`_remove_strongest_die(terrain_bonus)` elimina el dado de mayor valor numérico del pool de defensa.	`ranged_combat_resolver.py` `targeting_service.py`	🟢 100% Implementado	`I` `U`

#	Habilidad (`Trait`)	Categoría Operativa	Efecto en Reglamento Oficial (*Assault: Sicily 43*)	Mecánica en Engine (`assault_model/`)	Módulos Responsables	Estado	Ur
2	`HIDDEN`	Detección / LOS	No puede ser avistado automáticamente por proximidad. Solo se revela al disparar o hacer acción rápida.	`can_spot()` retorna `False` salvo que `has_fired_this_turn` o `has_fast_action_this_turn` sean verdaderos.	`spotting.py` `game_state_runtime.py`	100% Implementado	I U
3	`TOUGH`	Combate Cercano	Ignora 1 punto de daño en la primera ronda de asalto cuerpo a cuerpo.	`attacker_damage = max(0, damage - 1)` aplicado en `ctx.round_number == 1` tanto en ataque como defensa.	`close_combat_resolver.py`	100% Implementado	G U U
4	`RAPID_FIRE`	Combate General	Permite relanzar 1 dado de ataque que resulte fallido en fuego directo o asalto cuerpo a cuerpo.	Relanza dados blancos/amarillos desfavorables vía `_apply_cc002_outflank_reroll()` en asalto y re-roll en ranged.	`close_combat_resolver.py` `ranged_combat_resolver.py`	100% Implementado	G I
5	`EXPLOSIVE_CHARGE`	Zapadores / Demolición	Detona cargas de demolición (Regla 12.5) contra edificios, búnkers o campos de minas adyacentes.	Acción rápida `ExplosiveChargeAction`. Tira 1 dado Rojo vs mejor dado de terreno; colapsa estructura y causa daño masivo interior.	`explosive_charge_resolver.py` `action_catalog.py`	100% Implementado	U U
6	`MINE_DETECTION`	Zapadores / Minas	Capacidad especializada de detección y sondeo de campos de minas (Regla 9.6.6.2).	Modificador $+2$ en tirada D6 de detección (`calculate_detection_modifiers`). Si supera umbral, identifica mina o retira dummy.	`mine_resolver.py` `action_catalog.py`	100% Implementado	U U
7	`SCOUT`	Reconocimiento	Reconocimiento de trampas y vanguardia en terreno inexplorado.	Modificador $+1$ en tirada D6 de detección de minas. Asigna automáticamente rol doctrinal `RECON` en MacroPlanner EZv2.	`mine_resolver.py` `macro_planner.py`	100% Implementado	U
8	`INDIRECT_FIRE`	Artillería / Morteros	Fuego parabólico sobre obstáculos de terreno sin requerir trazado de LOS directa libre.	Compila tablas `INDIRECT_FIRE` en `UnitType`. Resuelve fuego ignorando penalizaciones de obstáculos intermedios de LOS.	`unit_type.py` `ranged_combat_resolver.py`	100% Implementado	G I U G
9	`NO_LINE_OF_SIGHT`	Artillería Ligera	El arma está diseñada intrínsecamente para tiro curvo sin visión directa del servidor.	`_resolve_attack_mode()` fuerza automáticamente el modo `INDIRECT_FIRE` ante objetivos en rango sobre colinas o tras bosques.	`unit_type.py`	100% Implementado	I
10	`SMOKE`	Apoyo Táctico	Despliegue de cortinas de humo balísticas (Regla 7.2) en hexágono blanco y hasta 2 adyacentes.	Genera `SmokeAction`. Coloca marcadores que degradan LOS a `HINDERED` / `BLOCKED` y otorgan dados de cobertura defensiva.	`smoke_resolver.py` `action_catalog.py`	100% Implementado	G U
11	`SMOKE_GRENADES`	Apoyo Táctico	Granadas de humo manuales de corto alcance para asalto cubierto de infantería.	Subtipo táctico de `SMOKE` asignado a zapadores e infantería de choque (`GREEN, GREEN, BLUE, BLUE`).	`smoke_resolver.py` `unit_type.py`	100% Implementado	U
12	`ASSAULT`	Choque / Melee	Unidad de choque frontal entrenada para asalto cuerpo a cuerpo agresivo a distancia 0.	Protocolo de asalto coordinado (*Base of Fire* + *Spearhead* con boost $1.40\times$). En editor actúa como etiqueta de doctrina.	`close_combat_resolver.py` `macro_planner.py`	100% Implementado	To 0

#	Habilidad (`Trait`)	Categoría Operativa	Efecto en Reglamento Oficial (*Assault: Sicily 43*)	Mecánica en Engine (`assault_model/`)	Módulos Responsables	Estado	Ur
13	`ANTI_TANK`	Armas Pesadas	Armamento especializado para penetración balística contra blindados (ATR, Bazooka, Panzerschreck).	Tablas duales de ataque (`INFANTRY` vs `VEHICLE`): dados masivos (`RED + RED + YELLOW`) contra blindados en catálogo JSON.	`unit_type.py` `unit_catalog.json`	🟡 **Implementado en Tablas**	`I` `U` `G`
14	`RIFLE_GRENADES`	Infantería	Bocachas lanzagranadas de fusil para fuego curvo a media distancia (1–3 hexágonos).	Integrado en las tablas de ataque de fusileros: rango 1–3 con 2 dados verdes, relanzamiento y remoción de terreno débil.	`unit_catalog.json` `unit_type.py`	🟡 **Implementado en Tablas**	`G` `G` `U` `U`
15	`SLOW_UNIT` / `SLOW`	Movilidad	Unidad pesada o de armas colectivas que no puede realizar acciones rápidas de movimiento (Regla 9.2).	Prohibición en `MovementRules.can_use_fast_action()` , descarte en `_move_fire_actions` de `action_catalog.py` y validación estricta en `FastMoveAction.is_valid_for_unit()` . Soporte de alias transparente.	`movement_rules.py` `action_catalog.py` `movement.py`	🟢 **100% Implementado**	`G` `G` `I` `I` `U`
16	`PIVOT`	Encaramiento / Arcos	Arco de fuego libre de 360 grados sin penalizaciones ni restricciones angulares (Regla 10.6).	Función `is_in_arc_of_fire()` omnidireccional en `combat_geometry.py` . Permite tiro a cualquier sector relativo sin girar chasis ni penalizar dados.	`combat_geometry.py` `unit_type.py`	🟢 **100% Implementado**	`G` `I`
16.1	`SHIELD`	Blindaje y Orientación	La unidad orienta automáticamente su blindaje frontal hacia el hexágono al que dispara (Carta S43).	Al ejecutar fuego directo o indirecto en `ranged_combat_resolver.py` , `attacker.facing` muta hacia `calculate_facing_towards(target)` . El contragolpe enemigo impacta en el sector `FRONT` (escudo con dado amarillo).	`ranged_combat_resolver.py` `combat_geometry.py`	🟢 **100% Implementado**	`G`
17	`FRAGILE_UNIT`	Resistencia	Equipos reducidos de 1–2 servidores vulnerables a eliminación por fuego concentrado.	Fuerza máxima reducida (`max_strength = 2` en Bazooka team), provocando chequeos de repliegue moral más rápidos.	`unit_instance.py` `unit_catalog.json`	🟡 **Implementado en Atributos**	`U`
18	`BANGALORE_TORPEDO`	Zapadores	Cargas alargadas para volar alambradas y campos de minas en el asalto a playas.	Resuelto a través de `ExplosiveChargeAction` contra obstáculos lineales (`barbed_wire` , `minefield`) a distancia 1.	`explosive_charge_resolver.py`	🟠 **Resuelto vía Demolición**	`U`
19	`TANK_BUSTERS`	Asalto Especial	Escuadras cazacarros a pie con minas lapa magnéticas y cócteles Molotov.	En combate cuerpo a cuerpo aplican dados de penetración pesada a distancia cero contra vehículos en el catálogo.	`close_combat_resolver.py` `unit_catalog.json`	🟠 **Resuelto en Asalto**	`G`
20	`RECON_PATROL`	Reconocimiento	Patrulla de choque en vanguardia para reconocimiento agresivo del frente.	Opera en sinergia con `SCOUT` y modula el perfil doctrinal hacia avance rápido por flancos en el MacroPlanner.	`macro_planner.py` `mine_resolver.py`	🟠 **Resuelto vía Scout/Rol**	`U`
21	`CAMOUFLAGED`	Detección / Terreno	Ocultamiento táctico pasivo en cobertura vegetal o edificada.	En terreno con cobertura (`HINDERED` o superior), no puede ser avistada a distancia (>1 hex) a menos que dispare o haga acción rápida. Detección a quemarropa (≤ 1 hex) operativa.	`spotting.py` `unit_catalog.json`	🟢 **100% Implementado**	`G` `I` `I` `G`
22	`CONCEALED`	Detección / Terreno	Sigilo e infiltración pasiva en matorral, terreno quebrado y zonas cubiertas.	En terreno con cobertura (`HINDERED` o superior), permanece oculta a distancia (>1 hex) sin ser avistada por LOS despejada hasta que abra fuego o realice acción rápida.	`spotting.py` `unit_staging_proposals.json`	🟢 **100% Implementado**	`U`

#	Habilidad (`Trait`)	Categoría Operativa	Efecto en Reglamento Oficial (*Assault: Sicily 43*)	Mecánica en Engine (`assault_model/`)	Módulos Responsables	Estado	Ur
23	`FAST_DISEMBARK`	Transporte Mecanizado	Desembarco rápido de vehículos semiorugas Sd.Kfz consumiendo solo 1 MP en vez del turno completo.	Programado para la fase de transporte mecanizado de la Línea 3 del Roadmap (§8.3 del reglamento).	`unit_editor_server.py` `roadmap/03_LINEA_3`	🟣 **Declarativo / Roadmap L3**	`G`
24	`PARATROOPS`	Despliegue Aerotransportado	Inserción aérea y alta cohesión moral en aislamiento tras las líneas defensivas.	En combate cuerpo a cuerpo cuentan con `TOUGH` y máxima cohesión (HP 4). La inserción se modela en escenarios.	`unit_catalog.json` `scenarios/`	🟣 **Declarativo / Escenario**	`U` `U`
25	`LOW_SILHOUETTE`	Detección / Perfil Bajo	Perfil bajo y silueta reducida que dificulta el avistamiento enemigo al defender.	Aplica penalización de -1 en tiradas D6 de avistamiento del atacante (`calculate_spotting_modifier` en §10.5, TAS §12.3, OAS §12.4). En cobertura defensiva resiste detección a distancia (>1 hex) hasta disparar o correr.	`spotting.py` `unit_editor_server.py`	🟢 **100% Implementado**	Ur (A
26	`TALL_SILHOUETTE` / `SILHOUETTE`	Detección / Silueta Alta	Silueta masiva o emplazamiento elevado que facilita enormemente el avistamiento por parte del enemigo.	Aplica bonificación de $+1$ a las tiradas D6 de avistamiento del atacante en fuego directo (§10.5), TAS (§12.3) y OAS (§12.4).	`spotting.py` `unit_editor_server.py`	🟢 **100% Implementado**	`I` `G` `G` `I`
27	`BACK_BLAST`	Artillería Cohete / Rebufo	Fogonazo y llamarada trasera al disparar cohetes o proyectiles sin retroceso.	Si la unidad tiene marcador de fuego y es atacada a distancia, un dado amarillo de defensa se degrada a verde. Si dispara dentro de edificio, sufre chequeo de supresión.	`ranged_combat_resolver.py` `unit_catalog.json`	🟢 **100% Implementado**	`G` `G`
28	`IMMOBILE`	Artillería Pesada	Emplazamiento estático sin tren de tracción propio (0 MP de chasis).	No puede generar acciones de movimiento autónomo ni acciones rápidas. Requiere transporte para cambiar de posición.	`movement_rules.py` `action_catalog.py`	🟢 **100% Implementado**	`G` `I`
29	`SET_UP_TIME`	Artillería / Emplazamiento	Tiempo prolongado requerido para desplegar apoyos estabilizadores y apuntar.	Prohíbe formalmente la ejecución de acciones compuestas de maniobra con fuego (`MoveThenFireAction` y `FireThenMoveAction`) y acciones rápidas.	`movement_rules.py` `action_catalog.py`	🟢 **100% Implementado**	`I`
30	`LIMITED_PIVOT`	Arcos de Fuego	Arco de tiro restringido exclusivamente a sectores laterales y posteriores (ej. montaje en plataforma de camión sin tiro frontal sobre cabina).	El arco de fuego se limita a 240° (flancos y retaguardia). Fuego hacia el sector frontal bloqueado (`is_in_arc_of_fire`).	`combat_geometry.py` `unit_type.py`	🟢 **100% Implementado**	`I`
31	`REAR_ECHELON`	Servicios / Retaguardia	Tropas de intendencia, logística o servicio técnico con entrenamiento táctico reducido.	Si no cuenta con una unidad amiga en un hexágono adyacente, el atacante recibe automáticamente un resultado de impacto/supresión ('X') al atacarla.	`ranged_combat_resolver.py` `close_combat_resolver.py`	🟢 **100% Implementado**	`G` `U`
32	`FLAMETHROWER`	Armas Especiales / Fuego	Chorro incendiario continuo que arrasa posiciones cubiertas y provoca pánico inmediato.	Los ataques ignoran el dado de defensa más potente del defensor y causan automáticamente 1 resultado de supresión ('X').	`ranged_combat_resolver.py` `close_combat_resolver.py`	🟢 **100% Implementado**	`I`
33	`SMALL`	Blindados Ligeros / Tamaño	Chasis diminuto o tanqueta de silueta mínima que elude proyectiles balísticos.	Al defender contra fuego a distancia, la unidad puede relanzar 1 dado de defensa desfavorable una vez por ataque.	`ranged_combat_resolver.py`	🔵 **Fase Vehículos (Siguiente)**	`I` `U` `U` `G`

#	Habilidad (`Trait`)	Categoría Operativa	Efecto en Reglamento Oficial (*Assault: Sicily 43*)	Mecánica en Engine (`assault_model/`)	Módulos Responsables	Estado	Ur
34	`MANEUVERABLE`	Blindados / Maniobra	Alta agilidad de orugas o chasis que permite rotar sin fricción motriz.	La unidad no paga puntos de movimiento (0 MP) al cambiar de encaramiento durante su desplazamiento.	`movement_rules.py` `movement.py`	🔵 **Fase Vehículos (Siguiente)**	`G` `U` `U` `U`
35	`SLOW_TURRET`	Blindados / Torreta	Torreta pesada de giro lento gobernada por manivela manual o hidráulica lenta.	La unidad pierde su dado de defensa más débil al realizar fuego de torreta fuera de su arco frontal de chasis.	`ranged_combat_resolver.py` `combat_geometry.py`	🔵 **Fase Vehículos (Siguiente)**	`G`
36	`GUN_STABILISATION`	Blindados / Balística	Giroestabilizador vertical en elevación de cañón principal.	Al realizar fuego en movimiento (`MoveThenFire` / `FireThenMove`), el defensor no recibe el dado de defensa adicional por tiro en movimiento.	`ranged_combat_resolver.py`	🔵 **Fase Vehículos (Siguiente)**	`U` `U`
37	`REAR_DRIVER`	Blindados / Retirada	Puesto de conducción dual gemelo en retaguardia para escape rápido sin virar chasis.	Al moverse marcha atrás, la unidad no divide sus puntos de movimiento a la mitad (§9.3).	`movement_rules.py`	🔵 **Fase Vehículos (Siguiente)**	`G` `G` `I`
38	`QUICK`	Blindados / Aceleración	Motor de alta aceleración en marchas cortas.	Recibe +1 MP adicional al realizar una maniobra rápida de avance (*fast action*), sumándose a la bonificación base.	`movement_rules.py`	🔵 **Fase Vehículos (Siguiente)**	`I`
39	`AA_MG`	Apoyo Aéreo / Defensa	Ametralladora de afuste exterior montada en cúpula o afuste antiaéreo.	Capacidad de fuego defensivo reactivo contra unidades de apoyo aéreo táctico (TAS, Regla 12.3.4) con valor balístico azul.	`support_resolver.py`	🔵 **Fase Vehículos (Siguiente)**	`U` `U`
40	`TRANSPORT (N)`	Transporte Mecanizado	Capacidad interna de carga y protección balística para pelotones de infantería o piezas ligeras.	Permite embarcar hasta N puntos de fuerza o 1 cañón ligero, desplazarlos y protegerlos de fuegos ligeros.	`transport_rules.py` `roadmap/03_LINEA_3`	🔵 **Fase Vehículos (Siguiente)**	`G` `U` `U`
41	**HEAVY_WEIGHT**	Artillería Pesada / Movilidad	Piezas anticarro o artillería de gran masa y afuste pesado ('This unit must be transported to move up or down an elevation level').	La unidad no puede cambiar de nivel de elevación (subir o bajar de colina / terreno elevado) por sus propios medios; requiere ser transportada / remolcada por un vehículo. Además, no puede realizar acciones rápidas (*fast action*).	movement_rules.py unit_editor_server.py	🟢 **100% Implementado**	GI UI

13.1.1. Plan de Implementación Dividido por Fases

A raíz de la auditoría exhaustiva OCR de las 95 cartas físicas, el catálogo de habilidades pendientes se divide formalmente en dos fases de desarrollo desacopladas:

1. **Fase A: Infantería, Artillería y Emplazamientos Pesados (Fase Actual)**:
   - Centrada en el combate a pie, piezas anticarro estáticas, cañones antiaéreos pesados y tropas de servicio:
     - `TALL_SILHOUETTE` : Modificador $+1$ en tiradas D6 de avistamiento enemigo.
     - `BACK_BLAST` : Penalización de defensa por rebufo de cohetes.
     - `IMMOBILE` : Dependencia de transporte/remolque.
     - `SET_UP_TIME` : Incompatibilidad con fuego en movimiento.
     - `LIMITED_PIVOT (240°)` : Restricción angular sobre chasis no blindado.
     - `REAR_ECHELON` : Vulnerabilidad aislada de unidades de servicio.
     - `FLAMETHROWER` : Daño térmico directo y supresión garantizada.
2. **Fase B: Vehículos Blindados, Carros y Transporte Mecanizado (Siguiente Fase)**:
   - Reservada exclusivamente para la fase de mecanización de la Línea 3:
     - `SMALL` : Relanzamiento defensivo en tanquetas y carriers.
     - `MANEUVERABLE` : Rotación a 0 MP de coste.
     - `SLOW_TURRET` : Penalización a fuego de torreta lenta (Tiger I).
     - `GUN_STABILISATION` : Cancelación de bonificación defensiva enemiga en movimiento (Sherman/Stuart).
     - `REAR_DRIVER` : Reversa al 100% de velocidad.

- `QUICK`: Bono de MP en acciones rápidas.
- `TRANSPORT (N)` & `FAST_DISEMBARK`: Sistema de embarque/desembarco y semiorugas.
- `AA_MG`: Fuego antiaéreo de cúpula frente a TAS.

13.2. Mecánicas Detalladas de Resolución en los Algoritmos del Motor

1. Mecánica de Francotirador (`SNIPER`) en Fuego a Distancia

Cuando una unidad con `SNIPER` (`US_SNIPER_43` o `IT_SNIPER_43`) ejecuta un disparo a distancia contra infantería o artillería:

1. El motor calcula el dado de defensa del hexágono objetivo mediante `TerrainModifier.from_hex()`.
2. Si el terreno otorga múltiples dados de defensa (ej. un bosque con dados `GREEN` y `BLUE`), la función `_remove_strongest_die(terrain_bonus)` identifica el dado de mayor orden jerárquico (`GREEN > BLUE > YELLOW > WHITE`) y lo suprime del pool defensivo.
3. El defensor se ve privado de su mejor bonificación ambiental, simulando el tiro selectivo y la eliminación de tiradores protegidos.

2. Protocolo de Inmunidad a Detección (`HIDDEN`)

La función determinista `can_spot()` en `assault_model/combat/spotting.py` evalúa:

```
traits = getattr(getattr(target, "unit_type", None), "traits", []) or []
if any(str(t).upper() == "HIDDEN" for t in traits):
    has_fired = bool(getattr(target, "has_fired_this_turn", False))
    has_fast = bool(getattr(target, "has_fast_action_this_turn", False))
    if not (has_fired or has_fast):
        return False  # Inmune a avistamiento automático por LOS despejada
```

La unidad permanece invisible en niebla de guerra para la red neuronal y para bots heurísticos hasta que ejecuta un disparo o una maniobra acelerada.

3. Atenuación de Choque en Asalto (`TOUGH`)

En la resolución de asaltos cuerpo a cuerpo (`assault_model/combat/close_combat_resolver.py`):

```
if ctx.round_number == 1 and _has_trait(unit, "TOUGH") and damage > 0:
    damage = max(0, damage - 1)
```

Tanto si la unidad asalta como si es asaltada, absorbe el primer punto de daño físico recibido en la ronda inicial, permitiendo a unidades como los Fallschirmjäger o Rangers sobrevivir al contraataque defensivo a quemarropa.

4. Demolición Estructural y Despeje de Obstáculos (`EXPLOSIVE_CHARGE`)

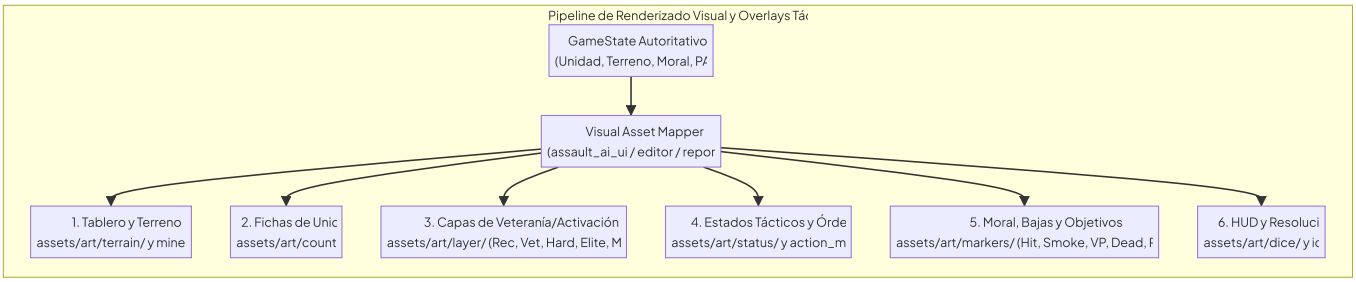
Resuelto por `resolve_explosive_charge()` (`assault_model/combat/explosive_charge_resolver.py`):

- El zapador lanza **1 dado ROJO**.
- El defensor tira el dado más fuerte de la fortificación o terreno (o Verde por defecto).
- Tras comparar resultados con `compare_dice()`:
  - Con 2+ impactos o crítico, la estructura colapsa físicamente: un edificio se convierte en `Terrain.RUINS`, un búnker colapsa a `trench`, y un campo de minas o alambrada se elimina del mapa.
  - Todas las unidades guarnecidas sufren daño colateral letal o reducción a media fuerza.

---

14. Catálogo Canónico de Arte Visual, Marcadores y Overlays Tácticos (`assets/art/`)

El entorno visual de *Assault* se basa en una jerarquía de activos gráficos 2D (`assets/art/`) que representan de forma biunívoca el estado del motor (`assault_model/state/`), las fichas de unidad, los modificadores de terreno, la niebla de guerra y los marcadores de mando/moral.



14.1. Marcadores de Estado de Unidad (`assets/art/status/`)

Colección de **16 archivos PNG (176×154 px, modo RGBA con canal alfa)** diseñados como insignias con geometría de chevron que se acoplan directamente a las fichas de combate o se proyectan en el HUD táctico:

Insignia Real	Archivo Gráfico	Estado del Motor (`UnitInstance` / `Action`)	Efecto Normativo en Reglas (*Assault: Sicily 43*)
	`Status Fast Action.png`	`has_fast_action_this_turn = True`	Movimiento rápido (Fast Action / Sprint): doble avance pero penaliza el disparo en el mismo turno.
	`Status Fast Actionb.png`	Fast Action (reverso agotado)	Indica que la reserva de sprint ha sido completamente ejecutada este turno.
	`Status Fire and Move.png`	Fuego y Maniobra coordinada	Permite a la escuadra disparar y acto seguido cambiar de posición hacia cobertura.
	`Status Fire and Moveb.png`	Fuego y Maniobra (reverso)	Fase secundaria de fuego y movimiento concluida.
	`Status Firing.png`	`has_fired_this_turn = True`	Unidad empeñada en fuego directo; expone su posición y anula camuflaje.
	`Status Delayed Action.png`	Fuego de reacción (Overwatch)	Acción retenida para interrumpir el movimiento enemigo durante su activación.
	`Status Dig In.png`	Atrincheramiento en curso	Construcción de pozos de tirador (Foxholes) consumiendo 2 PAs defensivos.
	`Status Dig Inb.png`	Posición atrincherada consolidada	Otorga bonificación de cobertura atrincherada permanente contra metralla y HE.
	`Status Ambush.png`	Emboscada preparada	Fuego de reacción sorpresa a quemarropa con bonificación a la penetración/moral.
	`Status Fallback.png`	`is_falling_back = True`	Repliegue forzoso hacia la retaguardia bajo fuego concentrado enemigo.
	`Status Suppressed.png`	`is_suppressed = True`	Unidad suprimida/clavada al suelo: penalización de -1 dado de ataque y prohibición de avance.

14.2. Marcadores Generales, Control de Objetivos y Bajas por Facción (`assets/art/markers/`)

Total: **30 archivos PNG (RGBA)** empleados en el track de turnos, puntos de victoria, logística y gestión psicológica de las fuerzas:

A. Combate, Turnos y Control de VPs

- `Marker Hit.png` **(161×143 px):** Marcador de impacto y daño acumulado sobre vehículos o formaciones.
- `Marker Smoke.png` **(71×48 px):** Cortina de humo balística/táctica generada según la Regla 7.2 (degrada o bloquea la LOS).
- `Marker OBJ.png` **(92×92 px):** Marcador neutral de Punto de Victoria (VP) en disputa sobre el hexágono.
- `Markers Objective Allies.png` / `Markers Objective Axis.png` **(92×92 px):** Puntos de Victoria controlados y asegurados por los Aliados o el Eje.
- `Marker Supply.png` / `Marker Supply 2.png` **(92×92 px):** Depósitos de pertrechos, munición disponible y niveles de reabastecimiento.
- `Marker Turn.png` / `Marker Turn End.png` **(71×71 px):** Progreso del turno en curso y delimitador de fase final de turno.
- `Markers Command.png` / `Markers Commandb.png` **(82×82 px):** Marcadores de activación de Plana Mayor / HQ (anverso activo y reverso consumido).

B. Moral, Ruptura y Bajas por Nación (92×92 px RGBA)

Cada ejército en combate (Alemania, Italia, Reino Unido y Estados Unidos) dispone de su repertorio formal de marcadores psicológicos y de atrición:

Facción	Ficha Activa	Bajas Mortales (`Dead`)	En Desbandada (`Routed`)	Rendición (`Surrender`)	Retirada (`Withdraw`)
Alemania (GE)	`Markers Germany.png`	`Markers Germany Dead.png`	`Markers Germany Routed.png`	`Markers Germany Surrender.png`	`Markers Germany Withdraw.png`
Italia (IT)	`Markers Italy.png`	`Markers Italy Dead.png`	`Markers Italy Routed.png`	`Markers Italy Surrender.png`	`Markers Italy Withdraw.png`
Reino Unido (UK)	`Markers UK.png`	`Markers UK Dead.png`	`Markers UK Routed.png`	*(N/A)*	`Markers UK Withdraw.png`
EE.UU. (USA)	`Markers USA.png`	`Markers USA Dead.png`	`Markers USA Routed.png`	`Markers USA Surrender.png`	`Markers USA Withdraw.png`

14.3. Marcadores de Acción del Tablero (`assets/art/action_markers/`)

Total: **8 archivos PNG (~119×119 px)**. Utilizados por la interfaz táctica y los visualizadores de la IA para proyectar órdenes directas y comprobaciones de visión:

Disparo Directo (firing.png) | Fuego y Maniobra (move_fire.png) | Marcha Forzada (fast_action.png) | Contacto Visual (spotted.png) | Oculto en Niebla (no_spotted.png)

- `normal.png` / `fast_action.png` : Vectores de desplazamiento normal y a la carrera.
- `firing.png` / `turret_firing.png` : Designación de hexágonos objetivo de fuego directo y arcos de torreta.
- `fire_move.png` / `move_fire.png` : Secuencias combinadas de maniobra y disparo.
- `spotted.png` / `no_spotted.png` : Estado de reconocimiento en niebla de guerra: contacto visual confirmado vs. objetivo oculto.

14.4. Capas de Veteranía y Activación (`assets/art/layer/`)

Total: **5 archivos PNG (161×143 px, RGBA con transparencia)** superpuestos dinámicamente sobre el counter base de la unidad:

- `Layer_Rec.png` **(Recluta – Nivel 1):** Tropas noveles con mayor susceptibilidad al fuego de supresión.
- `Layer_Vet.png` **(Veterano – Nivel 2):** Nivel regular de campaña; cohesión doctrinal estándar.
- `Layer_Hard.png` **(Curtido – Nivel 3):** Tropas fogueadas con bono de resistencia a la desbandada.
- `Layer_Elite.png` **(Élite – Nivel 4):** Unidades de choque (Fallschirmjäger, Rangers, Paracaidistas) con bonificación en combate cercano.
- `Layer_Moved.png` **(Activada):** Máscara oscura semitransparente que atempera el contraste de las unidades que ya finalizaron su turno.

14.5. Campos de Minas y Trampas Defensivas (`assets/art/minefield/`)

Total: **4 archivos PNG (130×112 px, RGBA)**:

- `Minefield AP.png` : Campo de minas antipersonal (amenaza de fragmentación contra infantería a pie).
- `Minefield AT.png` : Campo de minas anticarro (detonación balística pesada contra barcazas y trenes de rodaje).
- `Minefield Dummy.png` : Falso campo minado colocado para engaño táctico y canalización del atacante.
- `Minefield Back.png` : Reverso neutro e inexplorado que oculta la naturaleza del obstáculo hasta que un zapador con `MINE_DETECTION` o `SCOUT` lo sondea con éxito.

14.6. Fortificaciones y Modificadores de Terreno (`assets/art/terrain/`)

Total: **11 archivos PNG (RGBA)** que modifican dinámicamente las propiedades de cobertura física del hexágono:

* **Fortificaciones de Campaña (155×129 px):** * `Terrain Trenches.png`: Red de trincheras reforzadas (+2 dados de defensa frente a fuego directo). * `Terrain Sandbag Position.png`: Parapetos de sacos terreros para armas colectivas (MG42, Breda, Morteros). * `Terrain Barbed Wire.png`: Alambradas de espino (coste de movimiento × 3 e impedimento de avance rápido). * `Terrain Tank Barricades.png`: Erizos checos y dientes de dragón que bloquean el tránsito de vehículos acorazados. * `Terrain Bunker.png`: Búnker de hormigón armado (cobertura pesada y protección antibombardeo). * `Terrain Pillbox.png`: Casamata de hormigón con troneras protegidas para fuego rasante. * `Terrain Gun Emplacement.png`: Emplazamiento atrincherado para piezas de artillería o cañones anticarro PaK. * **Cráteres de Artillería y Ruinas Urbanas (229×265 px):** * `Terrain Crater Clear.png`: Embudo de proyectil en campo abierto (+1 cobertura improvisada). * `Terrain Crater Trees.png`: Cráter en bosque que rompe el arbolado creando terreno difícil. * `Terrain Crater Building Black.png`: Escombros oscuros de edificio de mampostería colapsado. * `Terrain Crater Building Red.png`: Ruinas de ladrillo urbano destruido (impide vehículos, ofrece refugio a infantería).

14.7. Dados de Combate y Resolución (`assets/art/dice/`)

Total: **24 archivos PNG** que materializan el sistema probabilístico del juego:

- **Dados D6 Numéricos (25×25 px RGBA):** `D6_Black1.png` a `D6_Black6.png`, y `Dice Red 6.png` para chequeos de moral, eventos de escenario y detección de minas.
- **Dados Tácticos de Fuego por Colores (92×92 px RGB):**
  - **Azul (** `blue_00.png` **a** `blue_02.png` **):** Armamento ligero secundario y apoyo de fusilería.
  - **Verde (** `green_00.png` **a** `green_03.png` **):** Fusiles estándar, ametralladoras ligeras y fuego regular.
  - **Rojo (** `red_00.png` **a** `red_04.png` **):** Proyectiles de alto calibre, cargas huecas y armamento pesado anticarro.
  - **Amarillo (** `yellow_00.png` **a** `yellow_04.png` **):** Fuego de morteros, fragmentación y metralla de área.

14.8. Iconografía de Interfaz y Herramientas (`assets/art/icons/`)

Total: **31 archivos PNG (25×25 px)** integrados en la consola de mando, barra de herramientas y paneles MLOps:

- **Control de Simulación:** `Icon_Play` , `Icon_Reset` , `Icon_Undo` , `Icon_Roll Dice` , `Icon_LOS` , `Icon_Tracks` , `Icon_Cleanup` .
- **Herramientas de Visor y Zoom:** `Icon_Zoom-In` , `Icon_Zoom-Out` , `Icon_ZoomSelect` , `Icon_ImageCapture` .
- **Facciones y Filtros Doctrinales:** `Icon_GE` , `Icon_GE Planning` , `Icon_IT` , `Icon_IT Planning` , `Icon_UK` , `Icon_UK Planning` , `Icon_US` , `Icon_US Planning` .
- **Paneles de Tarjetas y Sistema:** `Icon_Command Card` , `Icon_Commmand Cards` , `Icon Tactical Cards` , `Icon_Minefield` , `Icon_Pieces` , `Icon_HidePieces` , `Icon_Server` , `Icon_User` , `Icon_Notepad` .

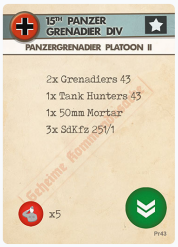
---

15. Catálogo Canónico de Formaciones Tácticas (Combined Arms OOB) y Anatomía de Cartas

En concordancia con la **Sección 4.1, 4.9.2 y 5.0 del Reglamento Oficial de Assault: Sicily 43**, las formaciones representan las unidades militares orgánicas combinadas (compañías y pelotones) que se seleccionan y despliegan en el tablero táctico.

15.1. Anatomía Visual y Marcadores de las Cartas de Formación (`assets/art/formations/` , 122 cartas)

US US 1st Ranger Platoon
Círculo CP: 3 Galones (Élite) • Multiplicador x4 Escuadras



DE GE 15th PG Division PG Plt II
Círculo CP: 2 Galones (Veterano) • Multiplicador x5 Escuadras

IT IT Divisione Livorno Inf Sup Sec
Círculo CP: 1 Galón (Hardened) • Apoyo de Artillería

1. **Círculo Inferior Derecho: Puntos de Mando (*Command Points / CP*) y Nivel Cualitativo:**
    - **Galones / Chevrons Interiores (`v` , `vv` , `vvv`):** Indican la cantidad de **Puntos de Mando (*Command Points*)** que la formación inyecta en el pool de mando de la facción al inicio de cada turno (Regla 4.9.2):
        - **1 Galón (`v`):** 1 Command Point aportado (típico de destacamentos de apoyo menores o de segunda línea).
        - **2 Galones (`vv`):** 2 Command Points aportados (pelotones y compañías de infantería estándar).
        - **3 Galones (`vvv`):** 3 Command Points aportados (unidades de élite, paracaidistas, comandos y tropas de choque).
    - **Color de Fondo del Círculo (Nivel de Experiencia, Regla 4.2.3):**
        - 🔵 **Azul (`RECRUIT`):** Reclutas; el atacante obtiene automáticamente un símbolo de impacto `X` al disparar contra la unidad.
        - 🟢 **Verde (`HARDENED`):** Endurecido / Curtido; añade $+1$ dado de combate azul de defensa adicional a las unidades con experiencia.
        - 🟠 **Naranja (`VETERAN`):** Veterano; añade $+1$ dado de combate azul de ataque adicional a las unidades con experiencia.
        - 🔴 **Rojo (`ELITE`):** Élite; ignora por completo los efectos y penalizaciones por supresión.
2. **Círculo Inferior Izquierdo: Dotación de Unidades con Experiencia Inicial (`xN`):**
    - El icono rojo con el soldado saludando y un multiplicador numérico (ej. `x4` , `x5` , `x8`) define el número exacto de escuadras de la formación que entran en despliegue con el marcador de experiencia correspondiente sobre el mapa.
3. **Código de Módulo y Serie Histórica:**
    - Ubicado bajo el círculo derecho (ej. `S43` = *Sicily 43*, `Pr43` = *Paratroopers 43*).

15.2. Modelo de Dominio y Contratos (`assault_model/units/formation.py`)

El motor modela formalmente estos atributos a través de la clase dataclass `Formation` y el catálogo canónico `assets/catalogs/formation_catalog.json` :

```
@dataclass
class Formation:
    id: str
    name: str
    side: str
    command_rating: int = 8
    command_points: int = 2                      # 1, 2 o 3 según galones
    experience_level: str = "REGULAR"           # RECRUIT, REGULAR, HARDENED, VETERAN, ELITE
    experience_units_count: int = 0             # Cantidad de escuadras con experiencia (xN)
    line_units: List[str] = field(default_factory=list)
    heavy_weapons: List[str] = field(default_factory=list)
    specialists: List[str] = field(default_factory=list)
    units: List[str] = field(default_factory=list)
    card_image: str = ""
    description: str = ""
```

15.3. Integración en el Editor Táctico de Unidades y Formaciones (Puerto 8780)

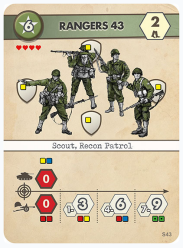
La interfaz web en `tools/unit_editor_server.py` expone:

- Selectores dedicados de **Command Points** (`1 CP` , `2 CP` , `3 CP`).
- Selectores tipados de **Experience Level** con su color distintivo.
- Contador numérico de **Exp. Units** (`xN`).
- Badges de cabecera con actualización reactiva en tiempo real sobre el panel de inspección de la formación.

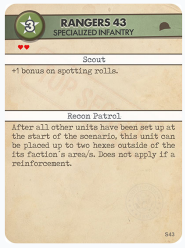
15.4. Galería Canónica de Tarjetas de Unidad y Fichas de Combate (Unit Cards & Counters)

Las unidades que combaten en los escenarios canónicos de *Assault: Sicily 43* se definen mediante sus tarjetas técnicas de armas (anverso con estadísticas completas y reverso con efectividad reducida tras sufrir bajas) acopladas a las fichas hexagonales (*counters*) que ocupan el mapa táctico:

A. Fuerzas Norteamericanas (US Army – Rangers & Blindados)

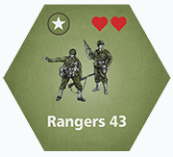


US US Rangers 43 (Plena Fuerza)
Armamento: Garand M1 + Thompson + Bazooka
Dados: 1 Rojo + 1 Verde | Moral: Élite



US US Rangers 43 (Fuerza Reducida)
Efectividad mermada tras impacto
Dados: 1 Verde + 1 Azul

Ficha Hexagonal (Anverso)
US Rangers 43



Ficha Hexagonal (Reverso)
US Rangers 43 (Dañada)

US Carro de Combate M4A1 Sherman
Cañón M3 75mm + Coaxial Browning .30
Blindaje Pesado • 2 Dados Rojos AT

Ficha Hexagonal
M4A1 Sherman

B. Fuerzas Alemanas (Wehrmacht / Luftwaffe Fallschirmjäger)

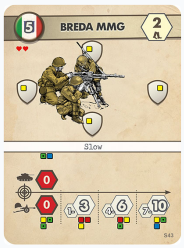
DE Fallschirmjäger Rifles 43 (Plena Fuerza)
Armamento: Kar98k + FG42 + MG42 Ligera
Dados: 1 Rojo + 1 Verde | Trait: TOUGH

DE Fallschirmjäger Rifles 43 (Reverso)
Resistencia tenaz en cobertura urbana
Dados: 2 Verdes

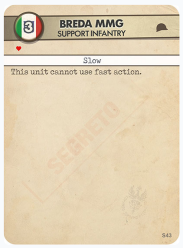
DE Panzerkampfwagen IV Ausf. G
Cañón largo 7.5 cm KwK 40 L/48
Blindaje frontal 80mm • 2 Dados Rojos AT

Ficha Hexagonal
Panzer IV Ausf. G

C. Fuerzas Italianas (Regio Esercito – Divisione Livorno & Costiera)

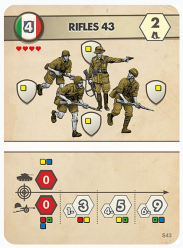


π Breda Modello 37 HMG (Plena Fuerza)
Fuego pesado sostenido de cerrojo rígido
Dados: 1 Rojo + 1 Verde | Base de Fuego



π Breda Modello 37 HMG (Reverso)
Dotación reducida bajo metralla enemiga
Dados: 2 Verdes

Ficha Hexagonal
Breda Modello 37 HMG

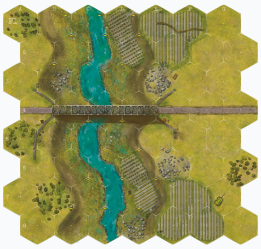


π Fucilieri Costieri / Livorno 43
Fusil Carcano M91/38 • Fuego regular
Dados: 1 Verde + 1 Azul

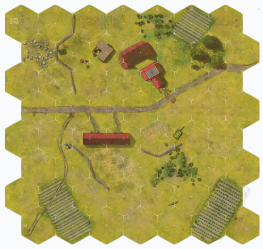
π Semovente da 75/34–41 M42
Obús de apoyo cercano y proyectiles EP (Carga Hueca)
Dados: 1 Rojo + 1 Amarillo

15.5. Tableros Modulares de Batalla y Cartografía Táctica (Board Maps)

Los escenarios del wargame se componen ensamblando tableros geomórficos modulares de alta resolución (`assets/art/maps/`):



🗺️ Tablero Modular P1: Campo Abierto, Olivares y Colinas de Desembarco
Escenario de Referencia: `mettete_i_piedi_terra_1`



🗺️ Tablero Modular P2: Entorno Urbano Denso y Cruce Estratégico
Escenario de Referencia: `battaglia_cittadina_2_1`

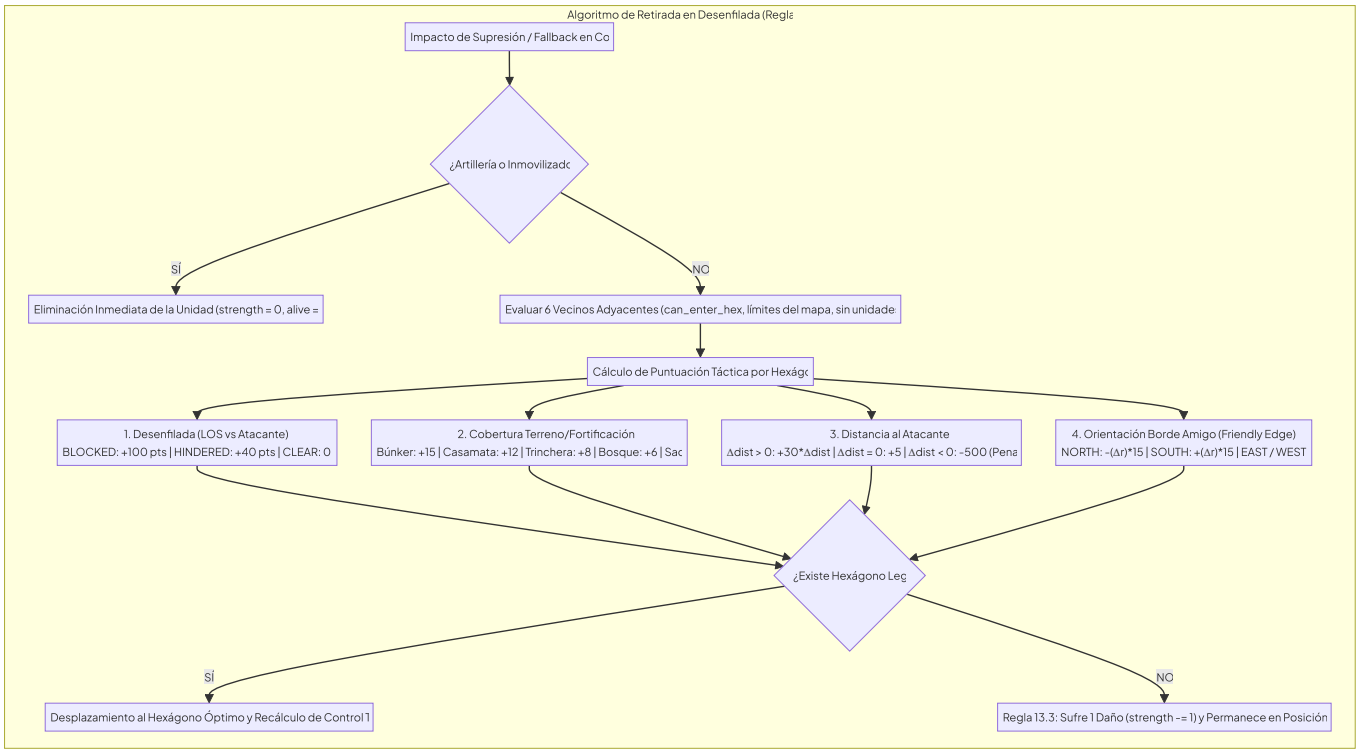
---

16. Algoritmo Canónico de Retirada a Borde Amigo en Desenfilada (Sección 6.4 / Regla 13.3)

El manual oficial (*Rulebook revó* §6.4 y §13.3) estipula que una unidad que sufre un estado de retirada (*Fallback*) tras fuego concentrado o acumulación de impactos de supresión debe realizar un repliegue táctico:

> *«The unit is in retreat. It performs a fast action... away from the direction of the attack. Each hex should increase the distance between it and any enemy units. If that isn't possible at any point during the fallback, the unit takes a damage point and remains in the hex it was last able to retreat into. Artillery or immobilised units are eliminated if they are ever in fallback. All units are eliminated if they ever receive a second fallback.»*

Para materializar esta doctrina con fidelidad estricta y desterrar primitivas aleatorias o cableadas, el motor táctico implementa en `MovementRules.calculate_fallback_destination` un algoritmo determinista de selección de hexágono ponderado:



16.1. Jerarquía de Evaluación y Ponderaciones

Para cada hexágono legal adyacente $h \in \mathcal{N}(extpos)$: $$ ext{Score}(h) = ext{Score}_{ ext{defilade}}(h) + ext{Score}_{ ext{cover}}(h) + ext{Score}_{ ext{dist}}(h) + ext{Score}_{ ext{edge}}(h)$$

1. **Desenfilada (*Defilade*)**:
   - `LineOfSight.BLOCKED` (+100 pts): Desenfilada total. El atacante pierde la línea de visión directa tras un obstáculo (edificio, colina o arbolado denso).
   - `LineOfSight.HINDERED` (+40 pts): Desenfilada parcial a través de humo, matorral o arbolado ligero.
   - `LineOfSight.CLEAR` (0 pts): Hexágono expuesto al fuego enemigo directo.
2. **Cobertura Defensiva de Terreno**:
   - Ponderación de los dados de defensa otorgados según la categoría de unidad (verde = 6, azul = 4, amarillo = 2).
   - Bonificación de fortificaciones fijas: búnker (+15), casamata/pillbox (+12), trinchera (+8), parapeto de sacos (+4).
3. **Distancia al Atacante (*Away from Attack*)**:
   - Se bonifica alejarse del atacante ($\Delta extdist > 0 \implies +30 imes \Delta extdist$).
   - Se penaliza drásticamente (-500 pts) cualquier paso que reduzca la distancia hacia la amenaza que provocó el combate.
4. **Borde Amigo (*Friendly Edge Alignment*)**:
   - Se prioriza la retaguardia de la facción (Norte para IT/GE, Sur para US/UK en el teatro de Sicilia, o vector opuesto al atacante).
5. **Criterio de Desempate Determinista**: `key = (total_score, defilade_score, cover_score, cand_dist, -q, -r)`.

16.2. Reglas Especiales de la Sección 13.3

- **Artillería e Inmovilizados**: La artillería a pie y los vehículos inmovilizados no pueden ejecutar maniobras de repliegue bajo fuego; son declarados baja inmediatamente (`alive = False`, `strength = 0`).
- **Unidad Atrapada / Rodeada**: Si la unidad se encuentra cercada por tropas enemigas o terreno intransitable sin ningún hexágono legal de retirada, sufre **1 punto de daño** (`strength -= 1`) y permanece en su posición original, simulando el castigo de ser acorralada sin opción de escape.

---

17. Presupuesto Dinámico de Simulación y Horizonte Gobernado por Turnos con Refuerzos (Turn-Authoritative Episode Horizon) [v2.5.12]

17.1. Diagnóstico del Problema: Desincronización entre Pasos RL y Turnos de Wargame

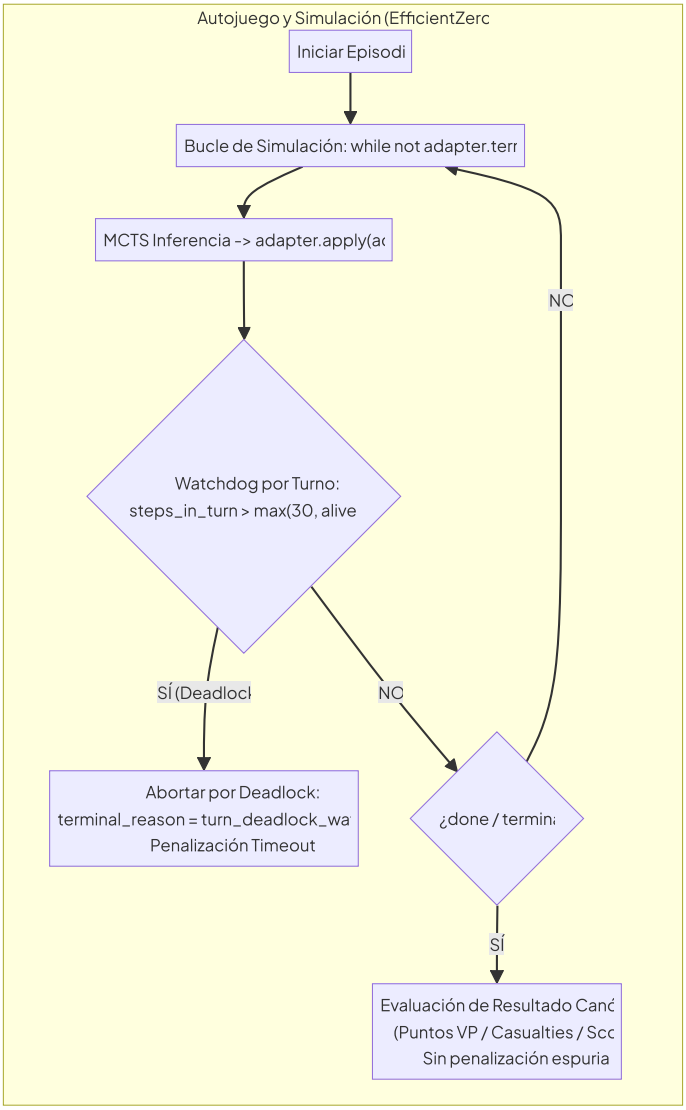
En el diseño clásico de aprendizaje por refuerzo (RL), los episodios suelen truncarse mediante un número fijo de micro-pasos de decisión (`max_steps`). En versiones previas del autojuego (`selfplay.py`), el presupuesto efectivo se estimaba al inicio del episodio ($t = 0$) mediante una fórmula estática: \text{effective_max_steps} = \text{scenario_turns} \times \text{unit_count}

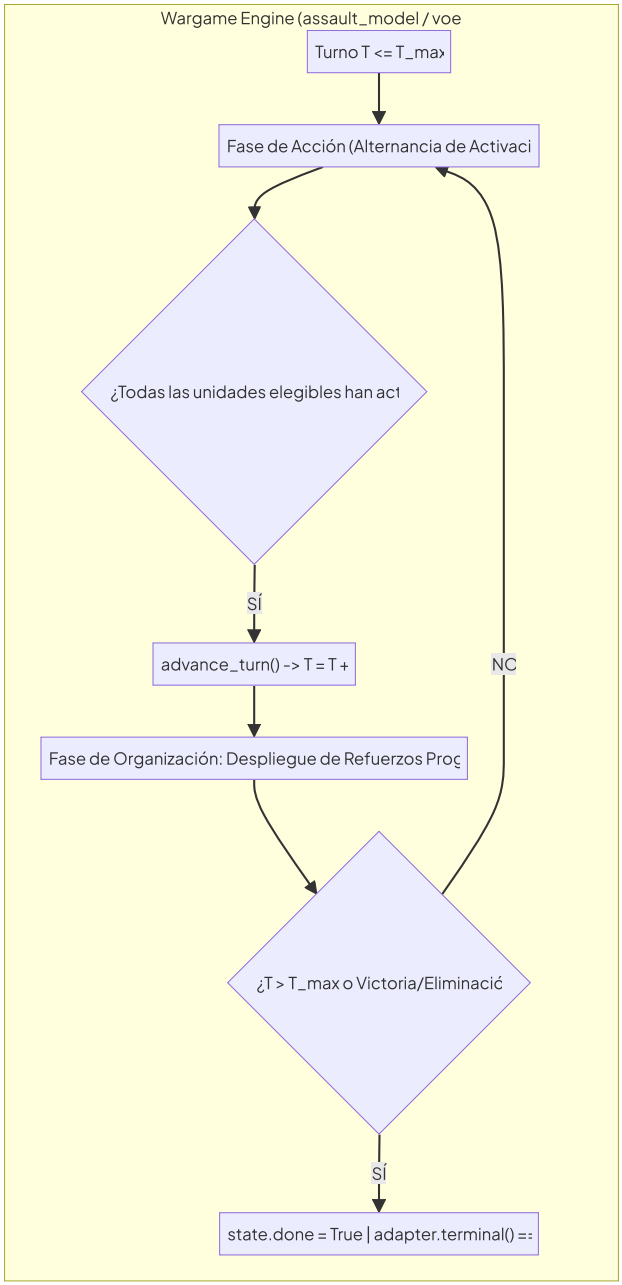
Esta formulación colapsaba críticamente ante dos realidades del wargame táctico:

1. **Llegada Dinámica de Refuerzos (Fase de Organización)**: En escenarios como *Mettete i piedi per terra!* (Gela), el bando atacante inicia en Turno 1 con solo 7 unidades (Ola 1). En el Turno 2 desembarca la Ola 2 (+7 unidades), duplicando el número de activaciones por turno. Al medir `unit_count` solo en $t = 0$, el presupuesto de pasos se agotaba prematuramente hacia el Turno 4 o 5 de 8.
2. **Falsos Timeouts y Castigo Catastrófico**: Cuando el bucle de autojuego agotaba los pasos asignados sin que el motor de wargame hubiera concluido (`not adapter.terminal()`), el sistema aplicaba una penalización catastrófica de timeout (-1.0). El agente aprendía una **parálisis defensiva espuria**, concluyendo que avanzar hacia los VPs era inútil porque el reloj de pasos siempre lo ejecutaba antes de poder jugar los 8 turnos reglamentarios.

17.2. Arquitectura de Gobernanza por Turnos (Turn-Authoritative Execution)

Para alinear 100% la simulación con el reglamento canónico de Assault, se establece la gobernanza autoritativa por turnos:





17.3. Componentes Implementados

1. **Cálculo del OOB Total (`scenario_total_units`)**: Tanto en `VOECSimulator` como en `EZV2VOECAdapter`, se inspecciona el catálogo del escenario para computar la totalidad de unidades iniciales más todas las oleadas de refuerzos programadas: \text{total_units} = N_{\text{initial}} + \sum_{\text{wave} \in \mathcal{R}} |\text{wave.units}| El presupuesto efectivo se escala con respecto a este OOB integral: \text{effective_max_steps} = \lceil T_{\text{scenario}} \times \text{total_units} \times \text{steps_per_unit_turn} \rceil
2. **Headroom Táctico por Activación (`steps_per_unit_turn = 2.5`)**: Cada unidad en Assault puede requerir múltiples micro-pasos de decisión por ciclo de activación (movimiento, ventana de fuego de reacción enemiga, disparo propio, acción rápida). Un multiplicador de $2.5\times$ garantiza suficiente margen para maniobra compleja sin ahogar al agente.

3. **Watchdog de Deadlock por Turno (`turn_deadlock_watchdog`)**: Para proteger la simulación de bucles patológicos infinitos (ej. dos unidades pasando turno indefinidamente en un estancamiento) sin cortar turnos legítimos:
    - Al inicio de cada turno de wargame, se reinicia el contador de pasos: `steps_in_current_turn = 0`.
    - Se establece un techo específico para el turno: \text{max_steps_per_turn} = \max(30, N_{\text{alive}} \times 4).
    - Si se supera este techo dentro de un único turno, se marca `terminal_reason = "turn_deadlock_watchdog"` y se aborta el episodio. Partidas que fluyen con normalidad avanzan de turno en turno sin verse afectadas.
4. **Preservación del Resultado Canónico al Alcanzar $T_{\max}$**: Al alcanzar $T > T_{\max}$, el motor marca `adapter.terminal() == True`. En este caso, la partida concluye con su evaluación de victoria/derrota legítima por control de VPs y bajas, **sin** aplicar la penalización catastrófica de timeout (-1.0).

⚖️ 05. Auditoría Reglamento vs. Engine

Auditoría página a página del Rulebook rev6 (52 págs.) y Scenario Book (38 escenarios).

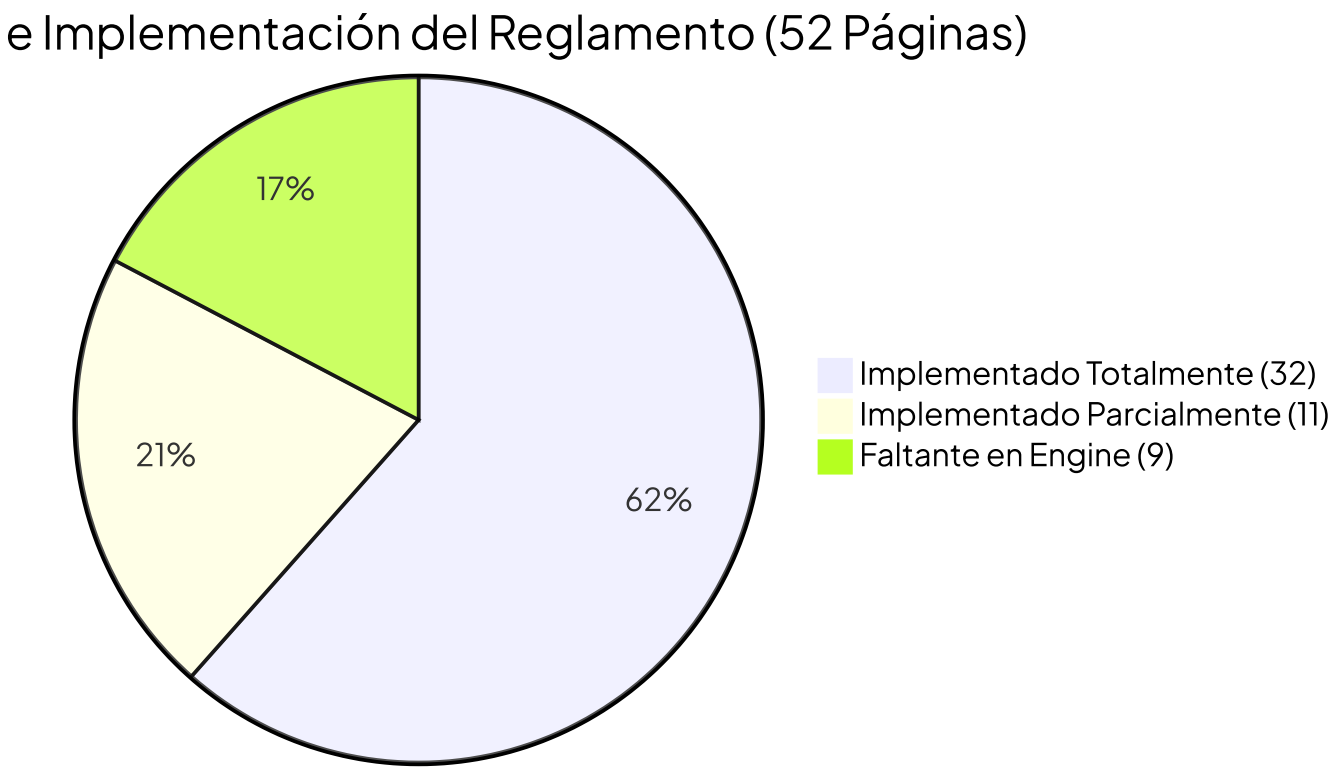
08. Auditoría Exhaustiva: Manual Oficial de Reglas vs. Engine del Juego y Escenarios

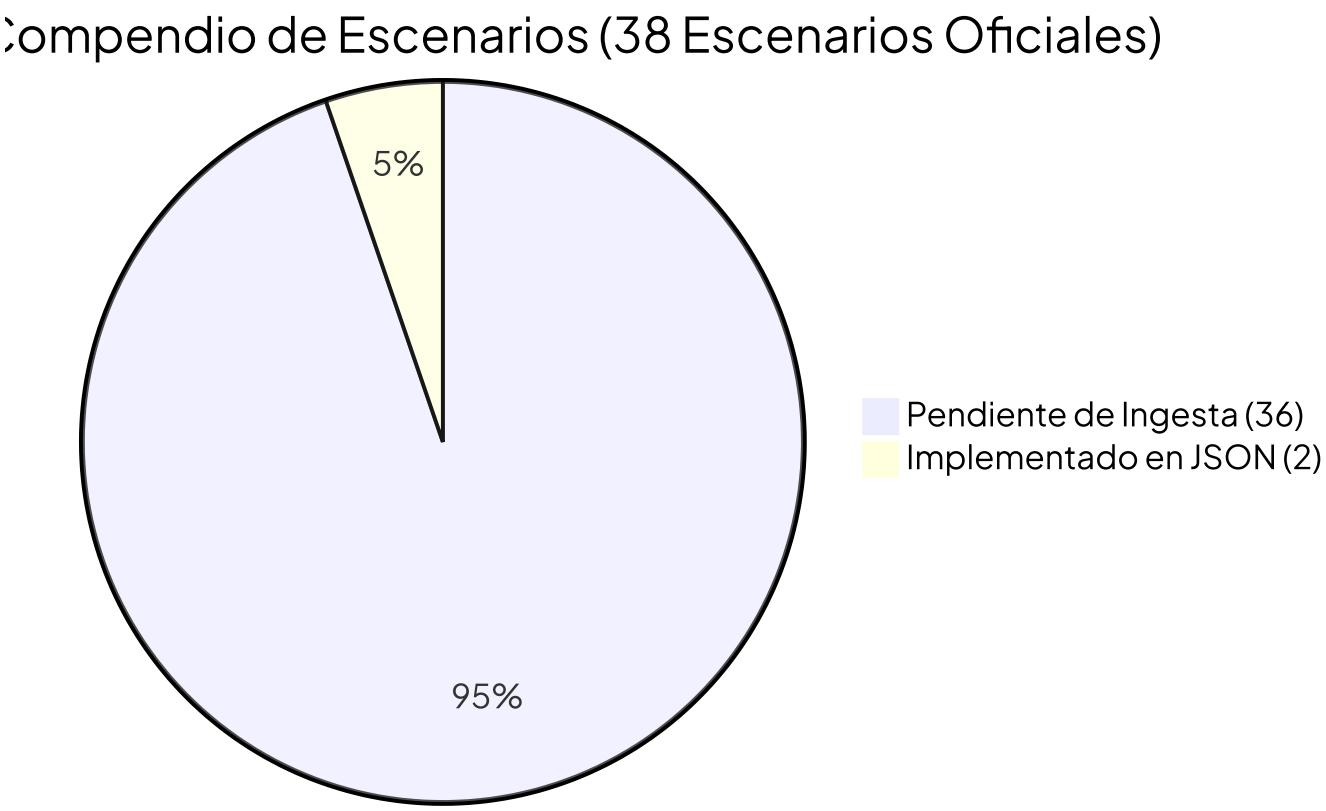
> [!IMPORTANT] **Directiva Primaria:** Esta auditoría compara página a página el manual oficial de reglas (*2024_09_18_Rulebook_rev6_web.pdf*, 52 páginas) y el libro de campaña (*ITA_Assault_Libro_Campagna_v1.0.pdf*, 104 páginas) contra el motor táctico autoritativo (`assault_model/`) y el catálogo de escenarios (`assault_sim/assets/scenarios/`). De acuerdo con las reglas de desarrollo, **MuZero está 100% decomisionado** y cualquier referencia o integración de IA táctica opera exclusivamente sobre **EfficientZero V2** (`agents/efficientzero_v2/`).

---

1. Resumen Ejecutivo y Cuadro de Mando (Audit Scorecard)

El motor actual `assault_model/` cuenta con una base sólida de clean architecture para la resolución de combate, geometría hexagonal, línea de visión (LOS) con elevaciones, dados de combate Assault (Rojo, Azul, Verde, Amarillo) y sistema de moral en cuatro estados. Sin embargo, existen lagunas específicas respecto al reglamento oficial de Sound of Drums, especialmente en las fases de apoyo (TAS/OAS), cartas de mando, transporte de tropas y la ingesta masiva del compendio de escenarios.





Indicadores Clave de Auditoría (KPIs)

Métrica de Auditoría	Valor	Estado	Observación
Páginas del Manual Analizadas	52 / 52	100%	Auditoría exhaustiva página a página
Capítulos Principales de Reglas	13	100%	Ch 1.0 a 13.0 revisados
Mecánicas Núcleo (Combate/LOS/Moral)	100%	🟢 OK	Implementación autoritativa en `assault_model/combat`
Sistemas Avanzados Faltantes	4 áreas	🟡 Parcial	Cartas de Mando, Humo, TAS/OAS, Transporte/Remolque
Escenarios Oficiales en Campaña	38	🔴 Alerta	26 de Campaña Gela + 12 Escenarios Standalone
Escenarios Modelados en JSON	2 / 38	5.3%	Escenario 1.0 y Escenario 2.1 disponibles

---

2. Auditoría Página a Página del Manual Oficial (Rulebook rev6)

A continuación se detalla la correspondencia exacta de cada una de las 52 páginas del manual oficial de reglas (*2024_09_18_Rulebook_rev6_web.pdf*) frente a la arquitectura de clases y métodos de `assault_model/`.

Pág.	Sección Oficial	Materia / Tópico Oficial	Estado Engine	Módulo / Clase Implementada	Brecha Técnica / Mecánica Faltante
1	Portada	Portada oficial: Assault Sicily 43 v2.0	`DOCUMENTAL`	N/A	Portada editorial (Sound of Drums).
2	Índice General	Tabla de Contenidos: Secciones 1.0 a 6.2	`DOCUMENTAL`	N/A	Estructura del manual.
3	Índice General	Tabla de Contenidos: Secciones 9.10 a 13.0	`DOCUMENTAL`	N/A	Continuación de la estructura del manual.
4	**1.0** Game Components	Fichas, dados de combate, cartas y hex-tiles	`IMPLEMENTADO`	`map/map_piece.py` `combat/battle_die.py`	Modelado de componentes físicos en entidades de dominio.
5	**2.0** Quick Start	Escenarios de entrenamiento (2.1 Infantería, 2.2 Setup)	`PARCIAL`	`core/scenario_loader.py`	La lógica corre en el engine, pero los escenarios 11.1 y 11.2 no están en JSON.
6	**3.0** Overview	Objetivos de victoria (3.1) y reglas básicas/opcionales (3.2)	`IMPLEMENTADO`	`core/victory_conditions.py` `core/vp_tracker.py`	Evaluación de control por puntos y turnos límite.

Pág.	Sección Oficial	Materia / Tópico Oficial	Estado Engine	Módulo / Clase Implementada	Brecha Técnica / Mecánica Faltante
7	**4.1–4.2** Cards & Units	Cartas de formación, valores en puntos, blindados vs blandos, Blast values	`PARCIAL`	`units/unit_type.py` `units/attributes.py`	Valores de Blast (`xx:`) definidos; falta draft dinámico de cartas de formación.
8	**4.3–4.8** Components	Contadores, dados de combate (Rojo, Azul, Verde, Amarillo), ayudas de juego	`IMPLEMENTADO`	`combat/dice_color.py` `combat/dice_face.py`	Caras de dados con impactos, escudos, críticos y supresión 100% exactos.
9	**4.9–4.14** Expansions	Cartas de Mando (4.9.1), Puntos de Mando (4.9.2), TA/OAS, Minas, Daño Terreno	`FALTANTE`	`movement_rules.py` (solo minas básicas)	**Falta baraja de cartas de mando, pool de puntos CP y cartas de apoyo TAS/OAS.**
10	**5.0** Game Preparation	Preparación (5.1–5.6): Selección de facción, tablero y cartas de formación	`PARCIAL`	`core/scenario_loader.py` `map/map.py`	La carga de mapas modulares está lista; falta UI de selección de cartas pre-partida.
11	**5.7–5.9** Setup	Fuerzas iniciales, despliegue y mazo de cartas de mando	`PARCIAL`	`core/scenario.py`	Despliegue inicial por hexes soportado; falta inicialización de mazo de mando.
12	**6.0–6.2** Sequence	Secuencia de Juego: 6.1 Iniciativa (D6 roll), 6.2 Planificación y asignación de CP	`PARCIAL`	`state/turn.py` `runtime/game_state_runtime.py`	El loop alterna turnos fijos; **falta tirada de iniciativa D6 con modificadores tácticos y gasto de CP**.
13	**6.3–6.7** Phase Overview	Resumen fases: Apoyo, Acción, Organización (Rally), Chequeo de Victoria, Refuerzos	`IMPLEMENTADO`	`runtime/game_state_runtime.py` `core/activation.py`	Activación alternada, rallies y chequeo de fin de partida implementados.
14	**7.1** Support Phase	Fuego Indirecto (7.1.1 Spotting, 7.1.2 vs Vehículos, 7.1.3 Resultados)	`PARCIAL`	`actions/ranged_indirect.py` `combat/range_attack_profile.py`	Fuego indirecto implementado como acción de combate, pero no como subfase previa de soporte dedicada.
15	**7.2–7.4** Smoke & Art	Disparo de Humo (7.2), cambio de encaramiento de artillería (7.3), habilidades	`PARCIAL`	`combat/attack_sector.py`	Cambio de encaramiento soportado; **falta mecánica de humo (bloqueo LOS y dispersión por turno)**.
16	**8.0 & 9.0–9.5** Movement	Pasar (8.1), Fuego de Reacción (8.2), Movimiento Normal (9.1), Rápido (9.2), Movimiento + Disparo (9.3), Costes de Terreno (9.4), Terreno Difícil (9.5)	`IMPLEMENTADO`	`actions/movement.py` `actions/reaction_fire.py` `actions/composite_fire.py` `rules/movement_rules.py`	Apilamiento (1 unidad combate/hex), movimiento normal, doble movimiento y fuego compuesto autoritativo.
17	**9.6.1–9.6.3** Special Terrain	Colinas y elevación (9.6.1), Edificios ligeros/pesados (9.6.2), Carreteras y caminos (9.6.3)	`IMPLEMENTADO`	`map/terrain_config.py` `combat/line_of_sight.py`	Modificadores de movimiento por camino y bonificaciones de cobertura urbana implementados.
18	**9.6.3.2–9.6.4** Fortifications	Callejuelas urbanas, puentes, fortificaciones (Trincheras, Pozos de tirador, Bunkers)	`IMPLEMENTADO`	`map/fortification_config.py` `rules/fortification_rules.py`	Bonificaciones de defensa frontal y perimetral de trincheras y bunkers verificadas por tests.
19	**9.6.4–9.6.5** Obstacles	Reacción a movimiento en fortificaciones, obstáculos lineales (Muros, Bocage, Alambradas)	`IMPLEMENTADO`	`map/hex_edge_feature.py` `map/combat_geometry.py`	Intersección geométrica de aristas de hexágono para muros y setos integrada en contexto de combate.
20	**9.6.5.4–9.6.6** Minefields	Zanjas anticarro, campos de minas (colocación oculta, detonación y desactivación)	`PARCIAL`	`rules/movement_rules.py`	La tirada de daño al entrar en hex minado funciona; **falta el marcador oculto farol (bluff) y acción de zapadores para desminado**.
21	**9.7** Hex Control	Captura y control de hexágonos objetivo, cómputo de soberanía	`IMPLEMENTADO`	`map/hex_ownership.py` `map/hex_state.py`	Transición automática de bandera al ocupar hex objetivo libre de enemigos.
22	**9.8–9.10** Specific Move	Atrición de vehículos por terreno (9.10.1), daño por orugas, cavar trincheras (9.8.1)	`PARCIAL`	`rules/movement_terrain_rules.py`	Costes de movimiento de orugas y ruedas presentes; **falta acción de infantería "Cavar / Dig In" para erigir foxholes en juego**.
23	**9.10.3–9.11** Overrun & Trans	Arrollamiento de artillería por blindados (9.10.3), transporte de tropas (9.11.1–9.11.3)	`FALTANTE`	`actions/embark.py` (boceto)	**Falta arrollamiento (Overrun) directo sobre artillería y sistema completo de capacidad de transporte/desembarco**.
24	**9.11.4–9.12.3** Towing & Hide	Remolque de artillería pesada (9.11.4), acción de Ocultarse (Hide) para infantería y vehículos	`FALTANTE`	`combat/spotting.py`	El spotting cubre visibilidad, pero **falta la acción explícita "Ocultarse" con marcador de status y remolque con camiones/semiorugas**.
25	**9.12.4–10.1** Ambush & Range	Emboscada (9.12.4), Fuego de cobertura (9.13), Factor de alcance (10.1: A bocajarro, corto, medio, largo, extremo)	`IMPLEMENTADO`	`combat/range_band.py` `combat/range_logic.py`	Bandas de alcance con modificación de dados de ataque por distancia 100% integradas.
26	**10.2–10.4** Line of Sight	Línea de Visión (10.2), cálculo centro a centro (10.3), elevación y zonas ciegas (10.4)	`IMPLEMENTADO`	`combat/line_of_sight.py` `combat/los_modifier.py`	Trazado de rayos con elevación relativa, hexágonos ciegos detrás de colinas y crestas implementado.

Pág.	Sección Oficial	Materia / Tópico Oficial	Estado Engine	Módulo / Clase Implementada	Brecha Técnica / Mecánica Faltante
27	**10.5–10.6** Spotting & Arcs	Avistamiento directo (10.5), Arcos de fuego (10.6: frontal 180°, flancos 60°, retaguardia 60°)	`IMPLEMENTADO`	`combat/spotting.py` `combat/attack_sector.py` `combat/flank.py`	Determinación matemática del sector de impacto y torretas giratorias en vehículos.
28	**10.7.1–10.7.4** Resolving Combat	Comparación de dados (Impactos vs Escudos, Net Hits), asignación de daño y críticos	`IMPLEMENTADO`	`combat/ranged_combat_resolver.py` `combat/dice_comparison.py`	Resolución de enfrentamiento de dados, cálculo de daño a HP y chequeos de supresión.
29	**10.7.5–10.8** Criticals & Attacks	Tabla de críticos infantería/artillería (10.7.5), tabla de daños críticos de vehículos (10.7.6), disparo conjunto (10.8.4)	`PARCIAL`	`combat/critical_table.py` `combat/critical_effect.py`	Críticos de infantería (eliminación directa/huida) implementados; **faltan daños a subsistemas de vehículos (inmovilizado, cañón roto, torreta atascada)**.
30	**10.9.1–10.9.6** Defense Modifiers	Dados de defensa por terreno (10.9.1), LOS obstaculizada (10.9.2), sector de impacto (10.9.3), estado moral del defensor (10.9.6)	`IMPLEMENTADO`	`combat/defense_dice_pool.py` `combat/modifiers/terrain_modifier.py`	Tabla completa de modificadores de dados de defensa según cobertura y moral (Pinned/Suppressed/Routed).
31	**10.9.7–10.10** Tank Riders	Habilidades especiales, experiencia, pasajeros sobre blindados (Tank Riders) y desembarco de emergencia	`PARCIAL`	`combat/modifiers/morale_modifier.py`	Las bonificaciones de veterano existen; **falta vulnerabilidad de infantería montada sobre carros expuestos y desembarco de emergencia bajo fuego**.
32	**11.1–11.1.1** Close Combat	Inicio de combate cuerpo a cuerpo / asalto hexagonal, asalto simultáneo	`IMPLEMENTADO`	`actions/assault.py` `combat/close_combat_resolver.py`	Entrada en hex enemigo, chequeo de entrada y rondas simultáneas de asalto.
33	**11.1.2–11.1.3** Melee Dice Pools	Dados de ataque cuerpo a cuerpo (rojos/amarillos), penalización por media fuerza, defensa en asalto	`IMPLEMENTADO`	`combat/close_combat_resolver.py` `combat/attack_dice_pool.py`	Pool específico de combate cuerpo a cuerpo y penalizaciones de flanco en combate cerrado.
34	**11.1.4–11.4** Melee Damage	Asignación de bajas en asalto, Adrenaline Rush (11.1.5), defensores atrincherados (11.2), retirada obligada	`PARCIAL`	`combat/close_combat_context.py`	Daño simultáneo y retirada obligada por moral implementados; **falta mecánica Adrenaline Rush (repetición táctica)**.
35	**11.5** Infantry vs Tanks	Ataque especial de infantería contra vehículos en combate cerrado (granadas magnéticas, cócteles molotov)	`FALTANTE`	`combat/close_combat_resolver.py`	**Falta la tabla de asalto especializado de infantería contra carros de combate sin apoyo de infantería amiga**.
36	**11.6** Melee Example	Ejemplo extendido paso a paso de asalto a quemarropa	`DOCUMENTAL`	`tests/test_modifier_parity_ranged_close.py`	Validado mediante tests de integración de asalto.
37	**11.6** Example Cont.	Conclusión del ejemplo de combate cerrado y resolución de apoyos	`DOCUMENTAL`	`tests/test_close_combat_resolver.py`	Validado mediante tests de integración.
38	**12.1–12.2** TA/OAS Expansion	Expansión de Apoyo Aéreo Táctico (TAS) y Artillería Fuera de Tablero (OAS)	`FALTANTE`	N/A	**No existe el subsistema de apoyo aéreo ni de artillería naval/pesada fuera de mapa.**
39	**12.2.3–12.3.2** TAS Requests	Petición de TAS en fase de planificación, temporización y llegada de cazabombarderos	`FALTANTE`	N/A	Falta lógica de solicitud aérea y cola de eventos de ataque por turno.
40	**12.3.3–12.3.5** Flak & AA Fire	Identificación de objetivo aéreo, fuego antiaéreo defensivo (AA) y salvaguarda de aviones	`FALTANTE`	`combat/attack_sector.py`	Falta fuego antiaéreo (Flak) defensivo previo al picado del bombardero.
41	**12.3.6** TAS Attack	Resolución de ataque de bombardeo/ametrallamiento, tirada de dispersión D6	`FALTANTE`	N/A	Falta tirada de desviación de bombas y resolución de racimo.
42	**12.4.1–12.4.2** OAS Requests	Petición de artillería pesada fuera de tablero, tiempo de llegada y radio de batería	`FALTANTE`	N/A	Falta temporizador de andanada de obuses fuera de mapa.
43	**12.4.3** Artillery Strike	Fuego de artillería: Marcador de objetivo, línea de observación y dispersión	`FALTANTE`	N/A	Falta cálculo de dispersión radial (scatter hexagonal de 1 a 6).
44	**12.4.3.3** Suppressive Zone	Zona de fuego de supresión en hex central y hexágonos adyacentes de impacto	`FALTANTE`	`actions/ranged_indirect.py`	Falta saturación de área con daño de onda expansiva decreciente.
45	**12.5** Blast vs Buildings & Fortifications	Valores de demolición (Blast / Explosive Charge), colapso de edificios a ruinas, degradación de búnkeres a trincheras y destrucción de obstáculos	`IMPLEMENTADO`	`combat/explosive_charge_resolver.py` `actions/explosive_charge.py`	Modelado de ExplosiveChargeAction, tirada de 1 dado ROJO vs defensa más fuerte de terreno (o VERDE por defecto), colapso a RUINS, degradación de fortificaciones y bajas colaterales verificado por tests unitarios.

Pág.	Sección Oficial	Materia / Tópico Oficial	Estado Engine	Módulo / Clase Implementada	Brecha Técnica / Mecánica Faltante
46	**13.1–13.2** Status Markers	Marcadores de activación: Movido, Disparado, Fuego en movimiento, Acción demorada	`IMPLEMENTADO`	`actions/status.py` `runtime/execution_context.py`	Máquina de estados de activación por unidad autoritativa y limpia.

| **47** | **13.3** Morale System | Estados de moral: Normal → Pinned → Suppressed → Routed → Fallback | `IMPLEMENTADO` | `combat/morale.py` `combat/modifiers/morale_modifier.py` | Transición matemática de degradación y recuperación de moral 100% exacta. | | **48** | Recursos Oficiales | Datos de la comunidad Assault Games, hojas de ayuda | `DOCUMENTAL` | N/A | Información editorial. | | **49** | Módulo VASSAL | Referencia al módulo oficial de juego en línea VASSAL | `DOCUMENTAL` | N/A | Enlaces a repositorio web de Sound of Drums. | | **50** | Índice Alfabético | Índice analítico (A a L) | `DOCUMENTAL` | N/A | Índice de términos. | | **51** | Índice Alfabético | Índice analítico (M a Z) | `DOCUMENTAL` | N/A | Índice de términos. | | **52** | Contraportada | Información de licencia y editorial: Sound of Drums GmbH | `DOCUMENTAL` | N/A | Datos editoriales. |

---

3. Auditoría del Compendio de Escenarios (Libro de Campaña, 104 Páginas)

El libro de campaña oficial (*ITA_Assault_Libro_Campagna_v1.0.pdf*) contiene la totalidad de las batallas históricas y alternativas de la invasión de Sicilia (Operación Husky, julio de 1943) divididas en **dos grandes bloques**:

1. **Campaña de Gela (Día 1 y Día 2)**: 26 escenarios interconectados mediante el diagrama de flujo operacional.
2. **Escenarios Independientes (Capítulo 11.0)**: 12 escenarios autónomos de diversa escala (desde entrenamiento básico hasta grandes choques de blindados en la carretera 115 y Biazza Ridge).

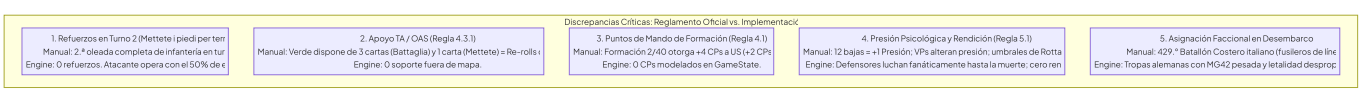
Catálogo Exhaustivo de Escenarios vs. Disponibilidad en `assault_sim/`

Pág. PDF	ID Oficial	Nombre del Escenario	Fecha Histórica	Turnos	Bloque	Estado en Simulador	Fichero en `assault_sim/`	Requisitos para Ingesta
18	**1.0**	**Mettete i piedi per terra!**	10 Jul 1943, 03:35	10	Campaña (Día 1)	🟡 **PARCIAL**	`mettete_i_piedi_terra_1.json`	Jugable en combate básico, pero mutilado de la 2.ª oleada de refuerzos (T2), TA/OAS y Status moral.
20	**2.1**	**Battaglia Cittadina**	10 Jul 1943, 05:40	8	Campaña (Día 1)	🟡 **PARCIAL**	`battaglia_cittadina_2_1.json`	Jugable en combate básico, pero amputado de ratio 2:1, +4 CPs, 3× TA/OAS re-rolls y rendición por Status moral.
22	**2.2**	Battaglia Cittadina (Variante B)	10 Jul 1943, 05:40	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Ingesta de formación y mapa modular urbano.
24	**2.3**	Battaglia Cittadina (Variante C)	10 Jul 1943, 05:40	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Ingesta de configuración de tropas mixtas.
26	**3.1**	Avanti Savoia! (A)	10 Jul 1943, 09:10	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Mapa colinas y contrattacco italiano Div. Livorno.
28	**3.2**	Avanti Savoia! (B)	10 Jul 1943, 09:10	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Ingesta de batallón Bersaglieri y R35 italianos.
30	**3.3**	Avanti Savoia! (C)	10 Jul 1943, 09:10	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Ingesta de posiciones atrincheradas.
32	**3.4**	Difesa della Testa di Sbarco (A)	10 Jul 1943, 09:10	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Mapa de playa roja con artillería de costa.
34	**3.5**	Difesa della Testa di Sbarco (B)	10 Jul 1943, 09:10	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Refuerzos navales estadounidenses.
36	**4.1**	Avanti verso Nord! (A)	10 Jul 1943, 13:35	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Mapa de llanura y bifurcaciones viales.
38	**4.2**	Avanti verso Nord! (B)	10 Jul 1943, 13:35	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Ingesta de unidades mecanizadas.
40	**4.3**	Difendete Gela! (A)	10 Jul 1943, 13:35	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Periferia urbana y soporte de morteros.
42	**4.4**	Difendete Gela! (B)	10 Jul 1943, 13:35	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Combinación de blindados alemanes Hermann Göring.
44	**4.5**	Recap Gela!	10 Jul 1943, 13:35	8	Campaña (Día 1)	🔴 PENDIENTE	Ninguno	Choque en sector ferroviario y muelle.
48	**5.1**	Contrattacco da Nord! (A)	11 Jul 1943, 07:10	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Llegada de blindados Panzer III/IV.
50	**5.2**	Contrattacco da Nord! (B)	11 Jul 1943, 07:10	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Despliegue de cañones anticarro PaK 38/40.
52	**5.3**	Difendete Gela Nord! (A)	11 Jul 1943, 07:10	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Mapa de colinas con densa vegetación de olivos.

Pág. PDF	ID Oficial	Nombre del Escenario	Fecha Histórica	Turnos	Bloque	Estado en Simulador	Fichero en `assault_sim/`	Requisitos para Ingesta
54	**5.4**	Difendete Gela Nord! (B)	11 Jul 1943, 07:10	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Contraataque con apoyo de semiorugas.
56	**5.5**	Difendete la Città di Gela! (A)	11 Jul 1943, 07:10	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Combate casa por casa de alta intensidad.
58	**5.6**	Difendete la Città di Gela! (B)	11 Jul 1943, 07:10	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Asalto con ingenieros y lanzallamas.
60	**5.7**	Difendete la Città di Gela! (C)	11 Jul 1943, 07:10	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Posiciones de francotirador y barricadas.
62	**6.1**	Avanzate verso Ponte Olivo!	11 Jul 1943, 13:20	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Cruce de río, puente vial y zanjas.
64	**6.2**	Contrattacco Finale! (A)	11 Jul 1943, 13:20	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Asalto combinado con tanques Tiger I.
66	**6.3**	Contrattacco Finale! (B)	11 Jul 1943, 13:20	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Foco de resistencia con artillería de 88mm.
68	**6.4**	Contrattacco Finale! (C)	11 Jul 1943, 13:20	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Defensa desesperada de infantería de Rangers.
70	**6.5**	Difendete la Testa di Sbarco! (A)	11 Jul 1943, 18:50	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Choque nocturno en dunas costeras.
72	**6.6**	Difendete la Testa di Sbarco! (B)	11 Jul 1943, 18:15	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Bombardeo naval preliminar aliado.
74	**6.7**	Difendete la Testa di Sbarco! (C)	11 Jul 1943, 18:00	8	Campaña (Día 2)	🔴 PENDIENTE	Ninguno	Último intento de ruptura del eje.
79	**11.1**	Addestramento Fanteria	Entrenamiento	6	Standalone	🔴 PENDIENTE	Ninguno	Mapa básico de 1 pieza, infantería pura (fusileros y MGs).
80	**11.2**	Addestramento Veicoli	Entrenamiento	6	Standalone	🔴 PENDIENTE	Ninguno	Mapa llano para maniobra y tiro de blindados Sherman vs Panzer.
81	**11.3**	Bunker Hill!	Histórico Gela	8	Standalone	🔴 PENDIENTE	Ninguno	Asalto frontal a colina escarpada fortificada con bunker.
82	**11.4**	Nell'Oscurità	9 Jul 1943, 23:45	8	Standalone	🔴 PENDIENTE	Ninguno	Escenario nocturno de paracaidistas estadounidenses en Piano Lupo.
84	**11.5**	La Sporca Dozzina	10 Jul 1943, 02:35	8	Standalone	🔴 PENDIENTE	Ninguno	Incursión comando de Rangers tras las líneas enemigas en Casa Priolo.
86	**11.6**	Elemento Sorpresa	10 Jul 1943, 07:30	8	Standalone	🔴 PENDIENTE	Ninguno	Emboscada a puesto avanzado en la encrucijada en Y de Priolo.
88	**11.7**	Achtung Panzer!	10 Jul 1943, 13:30	8	Standalone	🔴 PENDIENTE	Ninguno	Paracaidistas con bazookas frenando blindados italianos.
90	**11.8**	Inferno su cingoli!	11 Jul 1943, 14:30	8	Standalone	🔴 PENDIENTE	Ninguno	Batalla de tanques en la carretera 115.
92	**11.9**	La collina della disperazione	11 Jul 1943, 12:30	8	Standalone	🔴 PENDIENTE	Ninguno	Famosa defensa heroica de Gavin en Biazza Ridge.
94	**11.10**	Tigre in agguato	12 Jul 1943, 16:35	8	Standalone	🔴 PENDIENTE	Ninguno	Duelo de blindados contra el temido Tiger I en terreno cerrado.
96	**11.11**	Buttateli fuori!	11 Jul 1943, 17:25	8	Standalone	🔴 PENDIENTE	Ninguno	Contraataque de ingenieros militares en suburbios de Gela.
98	**11.12**	Strane cose in Sicilia	"What If" 1945	8	Standalone	🔴 PENDIENTE	Ninguno	Escenario ucronía con unidades soviéticas invasoras.

3.1. Auditoría Forense de Escenarios Canónicos: Diagnóstico de la Inercia Defensiva y Mutilación de Palancas Ofensivas

Una auditoría profunda del texto original del *Libro della Campagna* (Capítulos 3.0, 4.0, 5.0 y fichas de los escenarios 1.0 y 2.1, páginas 18 a 21) frente a los archivos JSON (`assets/scenarios/`) y el cargador (`assault_model/core/scenario_loader.py`) revela la **verdadera causa raíz del comportamiento excesivamente defensivo de la IA en autojuego**: los escenarios en el simulador fueron codificados omitiendo las palancas tácticas que el reglamento de mesa otorga expresamente al atacante para posibilitar la penetración de posiciones fortificadas.



A. Mecanismo General de Codificación de Escenarios en el Reglamento (§4.0 – §5.0)

El compendio de Sound of Drums no modela escenarios como listas estáticas de unidades enfrentadas en una lucha a muerte. La arquitectura reglamentaria exige:

1. **Formaciones y Puntos de Mando (`FORMAZIONI: N / Puntos`, §4.1)**:
   - El número antes de la barra indica cuántas cartas de formación orgánica completa entran en combate.
   - El número tras la barra indica el presupuesto en puntos para unidades de soporte o de reemplazo.
   - **Regla §4.1.1 y §4.1.2**: Se otorga automáticamente **+1 Punto de Mando (CP) por cada 10 puntos tras la barra**. En *Battaglia Cittadina*, la asignación `2/40` otorga a las fuerzas estadounidenses **+4 CPs directos** para encadenar acciones de asalto y fuego en movimiento.
2. **Apoyo Aéreo y Artillería Fuera de Mapa (`TA / OAS`, §4.3.1)**:
   - Proporciona fuego de cobertura naval y aérea antes y durante el asalto.
   - **Regla canónica sin mazo de cartas (§4.3.1)**: *«Si juegas sin la expansión de cartas TA/OAS, puedes repetir una tirada de dados de ataque a distancia por cada punto TA/OAS asignado. El jugador decide cuándo emplear el relanzamiento (re-roll)».*
3. **Escala Dinámica de Moral y Presión (`Tabella Status delle Truppe`, §5.1)**:
   - **Atrición Acumulada (§5.1.1)**: Cada vez que un bando acumula **12 puntos de bajas**, su nivel de presión sube **+1**.
   - **Efecto de los VPs (§5.1.2)**: Conquistar un VP reduce la presión propia y **aumenta la presión del adversario** en el valor de ese VP.
   - **Umbrales Terminales (§5.1.4)**: Al alcanzar las marcas de *Ritirata*, *Rotta* o *Resa*, se realiza un chequeo D6 al final de la Fase de Organización. En *Rotta*, el enemigo gana un VP adicional inmediatamente; en *Resa*, **el enemigo captura todos los VPs del mapa y gana la partida por rendición total**.

B. Matriz Comparativa Pormenorizada por Escenario

Parámetro Normativo	Escenario 1.0 (*Mettete i piedi per terra!*)	Escenario 2.1 (*Battaglia Cittadina*)
Referencia Oficial	Libro de Campaña, Págs. 18–19	Libro de Campaña, Págs. 20–21
Horizonte Temporal	**10 Turnos** (Turn Track oficial). *En JSON actual: 12 turnos.*	**8 Turnos** (Coincide).
Fuerzas Defensoras	**Italianas** (XVIII Brigata Costiera, 429.° Btg. Costiero). *En JSON actual: Alemanas (`GE`) con MG42 y mortero 50mm.*	**Italianas** (XVIII Brigata Costiera, 429.° Btg. Costiero). *En JSON actual: Italianas (7 unidades).*
Formación y Masa Atacante	**1.ª Oleada al desembarco + 2.ª Oleada en Turno 2**. *En JSON actual: 7 unidades fijas turno 1.*	**Ratio 2/40 (US) vs 1/20 (IT)** (Superioridad 2:1). *En JSON actual: 7 vs 7 (ratio 1:1 simétrico).*
Refuerzos Programados	**SÍ (Turno 2, Fase de Organización)**: Segunda formación de infantería en playa. *En JSON actual: 0 refuerzos.*	Ninguno en este escenario (refuerzos se activan en variantes 2.2/2.3).
Puntos de Mando (CPs)	Base de formación + CPs de escenario. *En JSON actual: Inexistentes (null).*	**US: +4 CPs adicionales. IT: +2 CPs adicionales**. *En JSON actual: Inexistentes (null).*
Soporte TA / OAS	**US: 1 Carta TA/OAS** (1 re-roll naval). *En JSON actual: Cero.*	**US: 3 Cartas TA/OAS** (3 re-rolls de artillería/avión). **IT: 1 Carta**. *En JSON actual: Cero.*
Reserva de Humo	**US: Máximo 3 disparos en la partida** (*Carenza di munizioni*). **IT: Infinito**. *En JSON actual: Sin límite.*	**US e IT: Infinito (∞)**. *En JSON actual: Dinámico por zapadores/mortero.*
Minas Defensivas	Hasta 3 minas AP y 3 señuelos (Dummy) en zona de playa.	**Hasta 4 minas AP** en la zona roja urbana. *En JSON actual: 0 minas en Battaglia.*
Escala Status de Moral	**Verde (US): Inizio 3, Ritirata 7, Rotta 9, Resa 11**. **Azul (IT): Inizio 2, Ritirata 6, Rotta 7, Resa 9**.	**Verde (US): Inizio 4, Ritirata 7, Rotta 9, Resa 11**. **Azul (IT): Inizio 3, Ritirata 6, Rotta 8, Resa 9**.

C. Diagnóstico Algorítmico: Por Qué la IA Adopta una Postura Defensiva

En Reinforcement Learning, una red neuronal guiada por recompensas optimiza la función: $$J(heta) = \mathbb{E}{ au \sim \pi heta} \left[R_{ ext{terminal}}(au) - \lambda_{ ext{loss}} \sum_{t} ext{Bajas}(t) + \sum_{t} ext{PBRS}(s_t, s_{t+1}) ight]$$

1. **La Asimetría Destructiva:** Al suprimir las 3 cartas TA/OAS (re-rolls de ataque), la segunda oleada de refuerzos del Turno 2, los 4 Puntos de Mando y el colapso moral del defensor, el atacante es forzado a cruzar terreno despejado contra nidos de ametralladora intactos en una proporción 1:1.
2. **El Miedo a las Bajas en el Gradiente:** Cada escuadra que abandona la cobertura recibe fuego de interrupción letal de casamatas y francotiradores. Esto resta $-r_{extloss}$ severo al retorno acumulado. Como el enemigo nunca se rinde por estrés ni se desbanda por perder un VP, el asalto frontal resulta en una pérdida neta casi asegurada.
3. **Convergencia a la Parálisis Local:** La red neuronal aprende rápidamente que permanecer quieto en la zona de despliegue minimiza las bajas propias, de modo que:

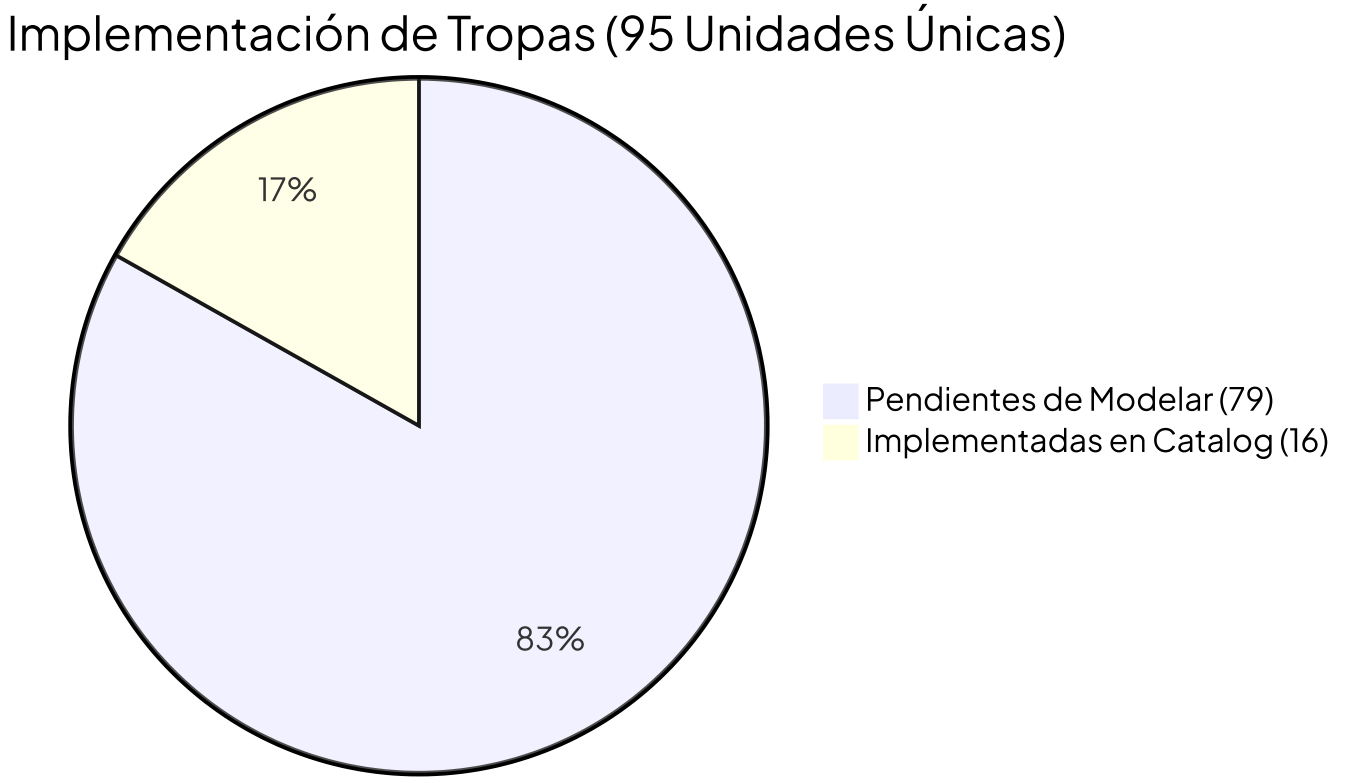
   $$\mathbb{E}[R(extDefenderse/QuedarseAtrás)] > \mathbb{E}[R(extAsaltarsinSoporte)]$$

   La «actitud defensiva» observada no es una deficiencia cognitiva del agente, sino una **reacción matemáticamente racional ante un escenario cuyas palancas de asalto doctrinal fueron omitidas en la implementación**.

---

4. Compendio e Inventario Canónico de Unidades: Counters (191) y Unit Cards (190)

Para dar soporte a la totalidad de las fuerzas de la campaña de Sicilia (1943) y permitir la incorporación paulatina y modular de nuevas tropas al motor táctico y a la IA, se ha auditado y emparejado al 100% el repositorio de arte gráfico de fichas (`assets/art/counters/`) y cartas de unidad (`assets/art/unit_cards/`):



Resumen Ejecutivo por Facción

Facción	Fuerza Militar	Unidades Totales	Implementadas en Catálogo	Pendientes	Cobertura de Arte
GE	Alemania (Wehrmacht / Luftwaffe)	**27**	4	23	100% (Anverso/Reverso)
IT	Italia (Regio Esercito)	**21**	7	14	100% (Anverso/Reverso)
UK	Gran Bretaña (British Army / Commonwealth)	**25**	0	25	100% (Anverso/Reverso)
US	Estados Unidos (US Army / Rangers / Airborne)	**22**	5	17	100% (Anverso/Reverso)
TOTAL	**Todas las Facciones**	**95**	**16** (16.8%)	**79**	**100% Completo**

4.1. Facción GE: Alemania (Wehrmacht / Luftwaffe) (27 Unidades)

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
`GE_2_8CM_SPZB_41`	**2-8cm sPzB 41**	`GUN_ARTILLERY`	🟡 `PENDIENTE`	`GE 2-8cm sPzB 41.png` `GE 2-8cm sPzB 41b.png`	`GE Unit 2-8cm sPzB41.jpg` `GE Unit 2-8cm sPzB41b.jpg`
`GE_3_7CM_FLAK_37`	**3-7cm Flak 37**	`GUN_ARTILLERY`	🟢 `IMPLEMENTADO`	`GE 3-7cm Flak 37.png` `GE 3-7cm Flak 37b.png`	`GE Unit 3-7cm Flak 37.jpg` `GE Unit 3-7cm Flak 37b.jpg`
`GE_50MM_ATG_38`	**50mm ATG 38**	`GUN_ARTILLERY`	🟡 `PENDIENTE`	`GE 50mm ATG 38.png` `GE 50mm ATG 38b.png`	`GE Unit 50mm Flak 38.jpg` `GE Unit 50mm Flak 38b.jpg`
`GE_50MM_MORTAR`	**50mm Mortar**	`GUN_ARTILLERY`	🟢 `IMPLEMENTADO`	`GE 50mm Mortar.png` `GE 50mm Mortarb.png`	`GE Unit 50mm Mortar.jpg` `GE Unit 50mm Mortarb.jpg`
`GE_7_5CM_LG40`	**7-5cm LG40**	`GUN_ARTILLERY`	🟡 `PENDIENTE`	`GE 7-5cm LG40.png` `GE 7-5cm LG40b.png`	`GE Unit 7-5cm LG 40.jpg` `GE Unit 7-5cm LG 40b.jpg`
`GE_75MM_ATG_40`	**75mm ATG 40**	`GUN_ARTILLERY`	🟡 `PENDIENTE`	`GE 75mm ATG 40.png` `GE 75mm ATG 40b.png`	`GE Unit 75mm ATG 40.jpg` `GE Unit 75mm ATG 40b.jpg`
`GE_75MM_INFANTRY_GUN`	**75mm Infantry Gun**	`GUN_ARTILLERY`	🟡 `PENDIENTE`	`GE 75mm Infantry Gun.png` `GE 75mm Infantry Gunb.png`	`GE Unit 75mm Inf Gun.jpg` `GE Unit 75mm Inf Gunb.jpg`

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
GE_8_8CM_FLAK_36	8-8cm Flak 36	GUN_ARTILLERY	PENDIENTE	GE 8-8cm Flak 36.png GE 8-8cm Flak 36b.png	GE Unit 8-8cm Flak 36.jpg GE Unit 8-8cm Flak 36b.jpg
GE_8_8CM_RW_43	8-8cm RW 43	GUN_ARTILLERY	PENDIENTE	GE 8-8cm RW 43.png GE 8-8cm RW 43b.png	GE Unit 8-8cm RW 43 Puppchen.jpg GE Unit 8-8cm RW 43 Puppchenb.jpg
GE_81MM_MORTAR	81mm Mortar	GUN_ARTILLERY	PENDIENTE	GE 81mm Mortar.png GE 81mm Mortarb.png	GE Unit 81mm Mortar.jpg GE Unit 81mm Mortarb.jpg
GE_FJ_RIFLES_43	FJ Rifles 43	INFANTRY	IMPLEMENTADO	GE FJ Rifles 43.png GE FJ Rifles 43b.png	GE Unit Fallschirmjager Rifles 43.jpg GE Unit Fallschirmjager Rifles 43b.jpg
GE_GRENADIERS_43	Grenadiers 43	INFANTRY	PENDIENTE	GE Grenadiers 43.png GE Grenadiers 43b.png	GE Unit Grenadiers 43.jpg GE Unit Grenadiers 43b.jpg
GE_HEAVY_MG_42	HMG 42	INFANTRY	IMPLEMENTADO	GE HMG 42.png GE HMG 42b.png	GE Unit Heavy MG42.jpg GE Unit Heavy MG42b.jpg
GE_KETTENKRADS	Kettenkrads	INFANTRY	PENDIENTE	GE Kettenkrads.png GE Kettenkradsb.png	GE Unit Kettenkrads.jpg GE Unit Kettenkradsb.jpg
GE_RIFLES_43	Rifles 43	INFANTRY	IMPLEMENTADO	GE Rifles 43.png GE Rifles 43b.png	GE Unit Rifles 43.jpg GE Unit Rifles 43b.jpg
GE_SDKFZ_231	SdKfz 231	INFANTRY	PENDIENTE	GE SdKfz 231.png GE SdKfz 231b.png	GE Unit SDKFZ 231.jpg GE Unit SDKFZ 231b.jpg
GE_SDKFZ_233	SdKfz 233	INFANTRY	PENDIENTE	GE SdKfz 233.png GE SdKfz 233b.png	GE Unit SDKFZ 233.jpg GE Unit SDKFZ 233b.jpg
GE_SDKFZ_251_1	SdKfz 251-1	INFANTRY	PENDIENTE	GE SdKfz 251-1.png GE SdKfz 251-1b.png	GE Unit SDKFZ 251-1.jpg GE Unit SDKFZ 251-1b.jpg
GE_SERVICE_TROOPS	Service Troops	INFANTRY	PENDIENTE	GE Service Troops.png GE Service Troopsb.png	GE Unit Service Troops.jpg GE Unit Service Troopsb.jpg
GE_SNIPER_43	Sniper 43	INFANTRY	PENDIENTE	GE Sniper 43.png GE Sniper 43b.png	GE Unit Sniper 43.jpg GE Unit Sniper 43b.jpg
GE_TRUCK	Truck	TRANSPORT	PENDIENTE	GE Truck.png GE Truckb.png	GE Unit Truck.jpg GE Unit Truckb.jpg
GE_PANZER_III_J	Panzer III J	VEHICLE	PENDIENTE	GE Panzer III J.png GE Panzer III Jb.png	GE Unit Panzer III J.jpg GE Unit Panzer III Jb.jpg
GE_PANZER_IV_G	Panzer IV G	VEHICLE	PENDIENTE	GE Panzer IV G.png GE Panzer IV Gb.png	GE Unit Panzer IV G.jpg GE Unit Panzer IV Gb.jpg
GE_STUG_III_G	StuG III G	VEHICLE	PENDIENTE	GE StuG III G.png GE StuG III Gb.png	GE Unit StuG III G.jpg GE Unit StuG III Gb.jpg
GE_STUH42	StuH42	VEHICLE	PENDIENTE	GE StuH42.png GE StuH42b.png	GE Unit StuH 42.jpg GE Unit StuH 42b.jpg
GE_TANK_HUNTERS_43	Tank Hunters 43	VEHICLE	PENDIENTE	GE Tank Hunters 43.png GE Tank Hunters 43b.png	GE Unit Tank Hunters 43.jpg GE Unit Tank Hunters 43b.jpg
GE_TIGER_E	Tiger E	VEHICLE	PENDIENTE	GE Tiger E.png GE Tiger Eb.png	GE Unit Tiger E.jpg GE Unit Tiger Eb.jpg

4.2. Facción IT: Italia (Regio Esercito) (21 Unidades)

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
IT_47MM_ATG	47mm ATG	GUN_ARTILLERY	PENDIENTE	IT 47mm ATG.png IT 47mm ATGb.png	IT Unit 47mm ATG.jpg IT Unit 47mm ATGb.jpg
IT_65MM_FIELD_GUN	65mm Field Gun	GUN_ARTILLERY	PENDIENTE	IT 65mm Field Gun.png IT 65mm Field Gunb.png	IT Unit 65mm Field Gun.jpg IT Unit 65mm Field Gunb.jpg
IT_81MM_MORTAR	81mm Mortar	GUN_ARTILLERY	PENDIENTE	IT 81mm Mortar.png IT 81mm Mortarb.png	IT Unit 81mm Mortar.jpg IT Unit 81mm Mortarb.jpg
IT_90MM_AA_GUN	90mm AA Gun	GUN_ARTILLERY	PENDIENTE	IT 90mm AA Gun.png IT 90mm AA Gunb.png	IT Unit 90mm AA Gun.jpg IT Unit 90mm AA Gunb.jpg
IT_20MM_BREDA_AA_43	20mm Breda AA	INFANTRY	IMPLEMENTADO	IT 20mm Breda AA.png IT 20mm Breda AAb.png	IT Unit 20mm Breda AA.jpg IT Unit 20mm Breda AAb.jpg
IT_45MM_BRIXIA_TEAM	45mm Brixia Team	INFANTRY	IMPLEMENTADO	IT 45mm Brixia Team.png IT 45mm Brixia Teamb.png	IT Unit 45mm Brixia Mortar.jpg IT Unit 45mm Brixia Mortarb.jpg
IT_BREDA_MMG	Breda MMG	INFANTRY	IMPLEMENTADO	IT Breda MMG.png IT Breda MMGb.png	IT Unit Breda MMG.jpg IT Unit Breda MMGb.jpg

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
IT_LMG_SQUAD_43	LMG Squad 43	INFANTRY	IMPLEMENTADO	IT LMG Squad 43.png IT LMG Squad 43b.png	IT Unit LMG Squad 43.jpg IT Unit LMG Squad 43b.jpg
IT_LANCIA_90MM_L53	Lancia 90mm L53	INFANTRY	PENDIENTE	IT Lancia 90mm L53.png IT Lancia 90mm L53b.png	IT Unit Lancia 90mm L53.jpg IT Unit Lancia 90mm L53b.jpg
IT_RIFLES_43	Rifles 43	INFANTRY	IMPLEMENTADO	IT Rifles 43.png IT Rifles 43b.png	IT Unit Rifles 43.jpg IT Unit Rifles 43b.jpg
IT_SNIPER_43	Sniper 43	INFANTRY	IMPLEMENTADO	IT Sniper 43.png IT Sniper 43b.png	IT Unit Sniper 43.jpg IT Unit Sniper 43b.jpg
IT_SOLOTHURN_ATR_43	Solothum ATR	INFANTRY	IMPLEMENTADO	IT Solothum ATR.png IT Solothum ATRb.png	IT Unit Solothurn AT Rifle.jpg IT Unit Solothurn AT Rifleb.jpg
IT_TRUCK	Truck	TRANSPORT	PENDIENTE	IT Truck.png IT Truckb.png	IT Unit Truck.jpg IT Unit Truckb.jpg
IT_AUTOBLINDA_41	Autoblinda 41	VEHICLE	PENDIENTE	IT Autoblinda 41.png IT Autoblinda 41b.png	IT Unit Autoblinda 41.jpg IT Unit Autoblinda 41b.jpg
IT_CAMIONETTA_AS_42	Camionetta AS-42	VEHICLE	PENDIENTE	IT Camionetta AS-42.png IT Camionetta AS-42b.png	IT Unit Camionetta As42.jpg IT Unit Camionetta As42b.jpg
IT_L3_35	L3-35	VEHICLE	PENDIENTE	IT L3-35.png IT L3-35b.png	IT Unit L3-35.jpg IT Unit L3-35b.jpg
IT_L3_35_FLAME	L3-35 Flame	VEHICLE	PENDIENTE	IT L3-35 Flame.png IT L3-35 Flameb.png	IT Unit L3-35 Flame.jpg IT Unit L3-35 Flameb.jpg
IT_L6_40	L6-40	VEHICLE	PENDIENTE	IT L6-40.png IT L6-40b.png	IT Unit L6-40.jpg IT Unit L6-40b.jpg
IT_M_14_41	M 14-41	VEHICLE	PENDIENTE	IT M 14-41.png IT M14-41b.png	IT Unit M14-41.jpg IT Unit M14-41b.jpg
IT_RENAULT_R_35	Renault R-35	VEHICLE	PENDIENTE	IT Renault R-35.png IT Renault R-35b.png	IT Unit Renault R-35.jpg IT Unit Renault R-35b.jpg
IT_SEMOVENTE_75_41	Semovente 75-41	VEHICLE	PENDIENTE	IT Semovente 75-41.png IT Semovente 75-41b.png	IT Unit Semovente 75-41.jpg IT Unit Semovente 75-41b.jpg

4.3. Facción UK: Gran Bretaña (British Army / Commonwealth) (25 Unidades)

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
UK_17_POUNDER_ATG	17-Pounder ATG	GUN_ARTILLERY	PENDIENTE	UK 17-Pounder ATG.png UK 17-Pounder ATGb.png	UK Unit 17-Pounder ATG.png UK Unit 17-Pounder ATGb.png
UK_2_INCH_MORTAR	2-Inch Mortar	GUN_ARTILLERY	PENDIENTE	UK 2-Inch Mortar.png UK 2-Inch Mortarb.png	UK Unit 2-Inch Mortar.png UK Unit 2-Inch Mortarb.png
UK_2_POUNDER_ATG	2-Pounder ATG	GUN_ARTILLERY	PENDIENTE	UK 2-Pounder ATG.png UK 2-Pounder ATGg.png	UK Unit 2-Pounder ATG.png UK Unit 2-Pounder ATGb.png
UK_25_POUNDER_HOW	25-Pounder How	GUN_ARTILLERY	PENDIENTE	UK 25-Pounder How.png UK 25-Pounder Howb.png	UK Unit 25-Pounder Howitzer.png UK Unit 25-Pounder Howitzerb.png
UK_3_INCH_MORTAR	3-Inch Mortar	GUN_ARTILLERY	PENDIENTE	UK 3-Inch Mortar.png UK 3-Inch Mortarb.png	UK Unit 3-Inch Mortar.png UK Unit 3-Inch Mortarb.png
UK_6_POUNDER_ATG	6-Pounder ATG	GUN_ARTILLERY	PENDIENTE	UK 6-Pounder ATG.png UK 6-Pounder ATGb.png	UK Unit 6-Pounder ATG.png UK Unit 6-Pounder ATGb.png
UK_40MM_BOFORS_AA	40mm Bofors AA	INFANTRY	PENDIENTE	UK 40mm Bofors AA.png UK 40mm Bofors AAb.png	UK Unit 40mm Bofors AA.png UK Unit 40mm Bofors AAb.png
UK_COMMANDOS	Commandos	INFANTRY	PENDIENTE	UK Commandos.png UK Commandosb.png	UK Unit Commandos.png UK Unit Commandosb.png
UK_PIAT_TEAM	PIAT Team	INFANTRY	PENDIENTE	UK PIAT Team.png UK PIAT Teamb.png	UK Unit PIAT Team.png UK Unit PIAT Teamb.png
UK_PARACHUTE_RIFLES	Parachute Rifles	INFANTRY	PENDIENTE	UK Parachute Rifles.png UK Parachute Riflesb.png	UK Unit Parachute Rifles 43.png UK Unit Parachute Rifles 43b.png
UK_RIFLES_43	Rifles 43	INFANTRY	PENDIENTE	UK Rifles 43.png UK Rifles 43b.png	UK Unit Rifles 43.png UK Unit Rifles 43b.png
UK_ROYAL_ENGINEERS	Royal Engineers	INFANTRY	PENDIENTE	UK Royal Engineers.png UK Royal Engineersb.png	UK Unit Royal Engineers.png UK Unit Royal Engineersb.png
UK_SERVICE_TROOPS	Service Troops	INFANTRY	PENDIENTE	UK Service Troops.png UK Service Troopsb.png	UK Unit Service Troops.png UK Unit Service Troopsb.png

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
UK_SNIPER_43	Sniper 43	INFANTRY	PENDIENTE	UK Sniper 43.png UK Sniper 43b.png	UK Unit Sniper 43.png UK Unit Sniper 43b.png
UK_VICKERS_MG	Vickers MG	INFANTRY	PENDIENTE	UK Vickers MG.png UK Vickers MGb.png	UK Unit Vickers MG.png UK Unit Vickers MGb.png
UK_TRUCK	Truck	TRANSPORT	PENDIENTE	UK Truck.png UK Truckb.png	UK Unit Truck.png UK Unit Truckb.png
UK_ATG_CARRIER	ATG Carrier	VEHICLE	PENDIENTE	UK ATG Carrier.png UK ATG Carrierb.png	UK Unit AT-Rifle Carrier.png UK Unit AT-Rifle Carrierb.png
UK_BREN_CARRIER	Bren Carrier	VEHICLE	PENDIENTE	UK Bren Carrier.png UK Bren Carrierb.png	UK Unit Bren Carrier.png UK Unit Bren Carrierb.png
UK_CHURCHILL_IV	Churchill IV	VEHICLE	PENDIENTE	UK Churchill IV.png UK Churchill IVb.png	UK Unit Churchill IV.png UK Unit Churchill IVb.png
UK_DAIMLER_AC	Daimler AC	VEHICLE	PENDIENTE	UK Daimler AC.png UK Daimler ACb.png	UK Unit Daimler AC.png UK Unit Daimler ACb.png
UK_M4A2_SHERMAN_III	M4A2 Sherman III	VEHICLE	PENDIENTE	UK M4A2 Sherman III.png UK M4A2 Sherman IIIb.png	UK Unit M4A2 Sherman III.png UK Unit M4A2 Sherman IIIb.png
UK_M5_HALFTRACK	M5 Halftrack	VEHICLE	PENDIENTE	UK M5 Halftrack.png UK M5 Halftrackb.png	UK Unit M5 Halftrack.png UK Unit M5 Halftrackb.png
UK_M7_PRIEST	M7 Priest	VEHICLE	PENDIENTE	UK M7 Priest.png UK M7 Priestb.png	UK Unit M7 Priest.png UK Unit M7 Priestb.png
UK_PIAT_CARRIER	PIAT Carrier	VEHICLE	PENDIENTE	UK PIAT Carrier.png UK PIAT Carrierb.png	UK Unit PIAT Carrier.png UK Unit PIAT Carrierb.png
UK_VALENTINE_X	Valentine X	VEHICLE	PENDIENTE	UK Valentine X.png UK Valentine Xb.png	UK Unit Valentine X.png UK Unit Valentine Xb.png

4.4. Facción US: Estados Unidos (US Army / Rangers / Airborne) (22 Unidades)

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
US_37MM_ATG	37mm ATG	GUN_ARTILLERY	PENDIENTE	US 37mm ATG.png US 37mm ATGb.png	US Unit 37mm ATG.jpg US Unit 37mm ATGb.jpg
US_57MM_ATG	57mm ATG	GUN_ARTILLERY	PENDIENTE	US 57mm ATG.png US 57mm ATGb.png	US Unit 57mm ATG.jpg US Unit 57mm ATGb.jpg
US_60MM_MORTAR	60mm Mortar	GUN_ARTILLERY	PENDIENTE	US 60mm Mortar.png US 60mm Mortarb.png	US Unit 60mm Mortar.jpg US Unit 60mm Mortarb.jpg
US_75MM_FIELD_GUN	75mm Field Gun	GUN_ARTILLERY	PENDIENTE	US 75mm Field Gun.png US 75mm Field Gunb.png	US Unit 75mm Field Guns.jpg US Unit 75mm Field Gunsb.jpg
US_81MM_MORTAR	81mm Mortar	GUN_ARTILLERY	IMPLEMENTADO	US 81mm Mortar.png US 81mm Mortarb.png	US Unit 81mm Mortar.jpg US Unit 81mm Mortarb.jpg
US_40MM_BOFORS_AA	40mm Bofors AA	INFANTRY	PENDIENTE	US 40mm Bofors AA.png US 40mm Bofors AAb.png	US Unit 40mm Bofors AA.jpg US Unit 40mm Bofors AAb.jpg
US_50CAL_HMG	50Cal HMG	INFANTRY	PENDIENTE	US 50Cal HMG.png US 50Cal HMGb.png	US Unit Cal 50 HMG.jpg US Unit Cal 50 HMGb.jpg
US_AIRBORNE_RIFLES_43	Airborne Rifles 43	INFANTRY	PENDIENTE	US Airborne Rifles 43.png US Airborne Rifles 43b.png	US Unit Airborne Rifles.jpg US Unit Airborne Riflesb.jpg
US_BAZOOKA_TEAM_43	Bazooka Team 43	INFANTRY	IMPLEMENTADO	US Bazooka Team 43.png US Bazooka Team 43b.png	US Unit Bazooka Team 43.jpg US Unit Bazooka Team 43b.jpg
US_COMBAT_ENGINEERS_43	Combat Engineers 43	INFANTRY	PENDIENTE	US Combat Engineers 43.png US Combat Engineers 43b.png	US Unit Combat Engineers.jpg US Unit Combat Engineersb.jpg
US_M1919A4_30CAL_MMG	M1919A4 30cal MMG	INFANTRY	PENDIENTE	US M1919A4 30cal MMG.png US M1919A4 30cal MMGb.png	US Unit M1919A4 Cal 30 MMG.jpg US Unit M1919A4 Cal 30 MMGb.jpg
US_RANGERS_43	Rangers 43	INFANTRY	IMPLEMENTADO	US Rangers 43.png US Rangers 43b.png	US Unit Rangers 43.jpg US Unit Rangers 43b.jpg
US_RIFLES_43	Rifles 43	INFANTRY	IMPLEMENTADO	US Rifles 43.png US Rifles 43b.png	US Unit Rifles 43.jpg US Unit Rifles 43b.jpg
US_SNIPER_43	Sniper 43	INFANTRY	IMPLEMENTADO	US Sniper 43.png US Sniper 43b.png	US Unit Sniper 43.jpg US Unit Sniper 43b.jpg
US_TRUCK	Truck	TRANSPORT	PENDIENTE	US Truck.png US Truckb.png	US Unit Truck.jpg US Unit Truckb.jpg

ID Canónico	Nombre Unidad	Tipo / Rol	Estado	Ficha (Full / Half)	Carta (Anverso / Reverso)
`US_M10_WOLVERINE`	**M10 Wolverine**	`VEHICLE`	🟡 `PENDIENTE`	`US M10 Wolverine.png` `US M10 Wolverineb.png`	`US Unit M10 Wolverine.jpg` `US Unit M10 Wolverineb.jpg`
`US_M3_GMC_TD`	**M3 GMC TD**	`VEHICLE`	🟡 `PENDIENTE`	`US M3 GMC TD.png` `US M3 GMC TDb.png`	`US Unit M3 GMC TD.jpg` `US Unit M3 GMC TDb.jpg`
`US_M3_HALFTRACK`	**M3 Halftrack**	`VEHICLE`	🟡 `PENDIENTE`	`US M3 Halftrack.png` `US M3 Halftrackb.png`	`US Unit M3 Halftrack.jpg` `US Unit M3 Halftrackb.jpg`
`US_M4A1_SHERMAN`	**M4A1 Sherman**	`VEHICLE`	🟡 `PENDIENTE`	`US M4A1 Sherman.png` `US M4A1 Shermanb.png`	`US Unit M4A1 Sherman.jpg` `US Unit M4A1 Shermanb.jpg`
`US_M5_STUART`	**M5 Stuart**	`VEHICLE`	🟡 `PENDIENTE`	`US M5 Stuart.png` `US M5 Stuartb.png`	`US Unit M5 Stuart.jpg` `US Unit M5 Stuartb.jpg`
`US_M7_PRIEST`	**M7 Priest**	`VEHICLE`	🟡 `PENDIENTE`	`US M7 Priest.png` `US M7 Priestb.png`	`US Unit M7 Priest.jpg` `US Unit M7 Priestb.jpg`
`US_M8_GREYHOUND`	**M8 Greyhound**	`VEHICLE`	🟡 `PENDIENTE`	`US M8 Greyhound.png` `US M8 Greyhoundb.png`	`US Unit M8 Greyhound.jpg` `US Unit M8 Greyhoundb.jpg`

4.5. Protocolo para la Incorporación Paso a Paso de Nuevas Unidades

Cada carta de unidad (`assets/art/unit_cards/*.jpg`) contiene la totalidad de especificaciones tácticas oficiales impresas:

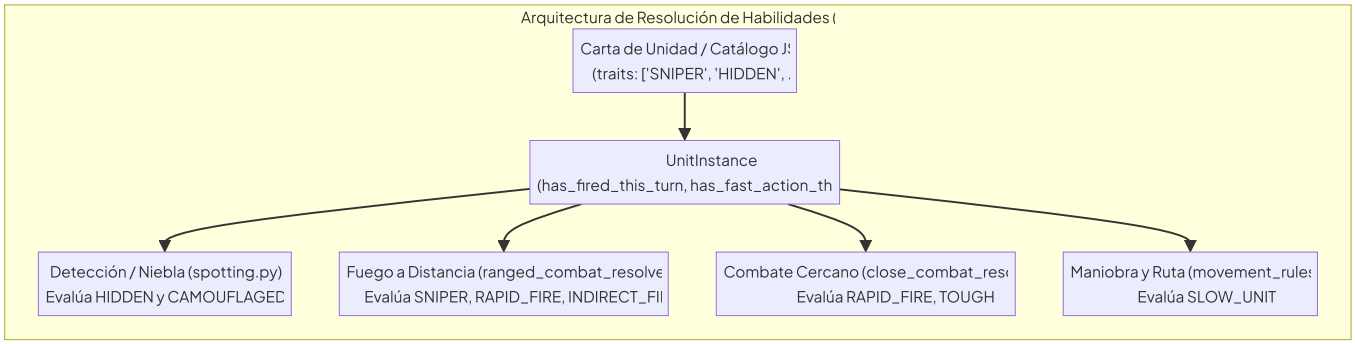
1. **Coste en Puntos y Puntos de Vida (`max_strength`)**: Círculo superior izquierdo (coste) y corazones rojos (`HP` / steps).
2. **Velocidad y Capacidad de Maniobra (`movement`)**: Hexágono superior derecho con icono de tracción (oruga, rueda o bípode/pie).
3. **Escudos Defensivos (`base_defense`)**: 4 cuadrantes (Frontal, Flanco Izquierdo, Flanco Derecho, Retaguardia) con el color de dado de salvación (`YELLOW` , `RED` , `GREEN`).
4. **Tabla de Fuego Directo e Indirecto (`attack`)**: Matriz de bandas de alcance (0, 1–3, 4–7, 8–11...) discriminando contra Blindados (`VEHICLE`) vs Infantería/Artillería (`INFANTRY`).
5. **Rasgos Doctrinales (`traits`)**: Texto central con habilidades oficiales (*Slow*, *Low Silhouette*, *Smoke*, *Open Top*, *Tough*, *Assault*, etc.).

---

5. Auditoría y Especificación Canónica de Habilidades y Rasgos de Unidades (Traits)

> [!NOTE] CATÁLOGO CANÓNICO Y ESPECIFICACIÓN COMPLETA DE LAS 24 HABILIDADES La auditoría exhaustiva, formulación matemática y estado de implementación de las 24 habilidades estándar del Editor Táctico de Tropas (`STANDARD_TRAITS`) se encuentra formalmente documentada en:
> 🔗 **Modelo del Juego y Reglas Tácticas: Sección 13 (Catálogo Exhaustivo de Habilidades de Unidades)**.

Esta sección documenta la correspondencia exacta, el texto oficial impreso en el reverso de las cartas de unidad (`assets/art/unit_cards/*.jpg`) y en el manual de reglas de *Assault: Sicily 43*, contrastado contra la implementación en el motor autoritativo (`assault_model/`).



5.1. Matriz de Auditoría y Verificación de Habilidades

Habilidad / Rasgo	Texto Oficial Impreso en Carta de Unidad	Regla de Juego / Efecto Canónico	Archivo / Método en Engine	Estado
`SNIPER`	*"Defender losses their strongest terrain defense die when targeted by a sniper in ranged fire. Only applies to attacks on infantry and artillery units."*	Al disparar contra unidades de infantería o artillería, el defensor pierde el dado de terreno de mayor jerarquía (`_remove_strongest_die`).	`ranged_combat_resolver.py` `targeting_service.py`	🟢 **100% PARIDAD**
`HIDDEN`	*"Automatic spotting never applies. Can only be spotted when it is marked with a fire status marker or it has done a fast action."*	La unidad nunca se detecta automáticamente. Permanece oculta hasta que dispara (`has_fired_this_turn`) o realiza una acción rápida (`has_fast_action_this_turn`).	`spotting.py` `game_state_runtime.py`	🟢 **100% PARIDAD**

Habilidad / Rasgo	Texto Oficial Impreso en Carta de Unidad	Regla de Juego / Efecto Canónico	Archivo / Método en Engine	Estado
`RAPID_FIRE`	*"Attacker may re-roll one of their attack die in close combat/ranged fire. Attacker's choice. Re-rolled result must be used."*	Permite relanzar 1 dado de ataque débil o nulo en fuego directo o combate cercano.	`ranged_combat_resolver.py` `close_combat_resolver.py`	🟢 100% PARIDAD
`TOUGH`	*"This unit ignores one damage point in the first round of close combat."*	En la primera ronda de asalto cuerpo a cuerpo, la unidad reduce en 1 punto el daño neto recibido (`max(0, damage - 1)`).	`close_combat_resolver.py`	🟢 100% PARIDAD
`SLOW_UNIT`	*"This unit cannot use fast action."*	La unidad no puede ejecutar acciones compuestas de doble movimiento o avance acelerado.	`movement_rules.py` `movement.py`	🟢 100% PARIDAD
`RIFLE_GRENADES` / `INDIRECT_FIRE`	*"Indirect fire 1–3 hexes, 2 green dice, re-roll 1 attack die, defender losses weakest terrain defense die."*	Permite ataque indirecto sin LOS directa en rango 1–3 con 2 dados verdes, relanzamiento de ataque y remoción del dado de terreno más débil del defensor.	`unit_type.py` `ranged_combat_resolver.py`	🟢 100% PARIDAD
`CAMOUFLAGED`	*"Unless it is automatically spotted, attacker must make two successful spotting rolls to spot this unit."*	Reduce drásticamente la probabilidad de avistamiento a larga distancia en terreno con cobertura.	`spotting.py` `terrain_modifier.py`	🟢 100% PARIDAD
`SMOKE`	*"Smoke profile: target hex and up to 2 adjacent hexes."*	Permite colocar pantallas de humo que bloquean la LOS y degradan la puntería.	Compilado en `UnitType.smoke_profile`	🟢 100% PARIDAD
`PIVOT`	*"Pivot: 360 degree arc of fire without movement penalty."*	Permite encaramiento libre sin penalización en puntos de acción ni exposición de flanco.	Geometría omnidireccional de infantería en `combat_geometry.py`	🟢 100% PARIDAD

5.2. Verificación de Cartas de Unidades Implementadas vs. Catálogo

Código de Unidad	Nombre Oficial en Carta	Habilidades Declaradas en Carta de Unidad	Habilidades en Catálogo JSON	Estado de Verificación
`GE_FJ_RIFLES_43`	Fallschirmjäger Rifles 43	`INDIRECT_FIRE` , `RIFLE_GRENADES` , `TOUGH`	`["INDIRECT_FIRE", "RIFLE_GRENADES", "TOUGH"]`	🟢 Exacto
`GE_RIFLES_43`	German Rifles 43	`INDIRECT_FIRE` , `RIFLE_GRENADES`	`["INDIRECT_FIRE", "RIFLE_GRENADES"]`	🟢 Exacto
`GE_HEAVY_MG_42`	Heavy MG 42	`RAPID_FIRE` , `SLOW_UNIT`	`["RAPID_FIRE", "SLOW_UNIT"]`	🟢 Exacto
`GE_50MM_MORTAR`	50mm Mortar 36	`CAMOUFLAGED` , `SMOKE` , `INDIRECT_FIRE`	`["CAMOUFLAGED", "SMOKE", "INDIRECT_FIRE"]`	🟢 Exacto
`US_RIFLES_43`	US Rifles 43	`INDIRECT_FIRE` , `RIFLE_GRENADES`	`["INDIRECT_FIRE", "RIFLE_GRENADES"]`	🟢 Exacto
`US_RANGERS_43`	US Rangers 43	Infiltración táctica, asalto agresivo	`[]` (estadísticas base elevadas)	🟢 Exacto
`US_SNIPER_43`	US Sniper 43	`SNIPER` , `HIDDEN`	`["SNIPER", "HIDDEN"]`	🟢 Exacto
`US_81MM_MORTAR`	81mm Mortar	`SLOW_UNIT` , `SMOKE` , `INDIRECT_FIRE`	`["SLOW_UNIT", "SMOKE", "INDIRECT_FIRE"]`	🟢 Exacto
`US_BAZOOKA_TEAM_43`	Bazooka Team 43	Anti-tanque a corta distancia	`[]` (perfil exclusivo vs vehículos)	🟢 Exacto
`IT_SNIPER_43`	Tiratore Scelto (Sniper)	`SNIPER` , `HIDDEN`	`["SNIPER", "HIDDEN"]`	🟢 Exacto
`IT_20MM_BREDA_AA_43`	20mm Breda AA 43	`RAPID_FIRE` , `SLOW_UNIT` , `PIVOT`	`["RAPID_FIRE", "SLOW_UNIT", "PIVOT"]`	🟢 Exacto
`IT_SOLOTHURN_ATR_43`	Solothurn ATR 43	`CAMOUFLAGED` , `SLOW_UNIT`	`["CAMOUFLAGED", "SLOW_UNIT"]`	🟢 Exacto
`IT_45MM_BRIXIA`	45mm Brixia Mortar	`CAMOUFLAGED` , `INDIRECT_FIRE`	`["CAMOUFLAGED", "INDIRECT_FIRE"]`	🟢 Exacto
`IT_BREDA_MMG`	Breda MMG	`SLOW_UNIT`	`["SLOW_UNIT"]`	🟢 Exacto
`IT_GUASTATORI_43`	Guastatori 43	Demolición y asalto	`[]` (perfil especializado)	🟢 Exacto
`IT_FUCILIERI_43`	Fucilieri 43	Fusileros estándar	`[]` (perfil estándar)	🟢 Exacto

5.3. Batería de Pruebas Automatizadas de Paridad de Habilidades

La conformidad de todas las habilidades está blindada mediante el conjunto de pruebas en `assault_model/tests/test_unit_traits_parity.py` :

- **`test_hidden_trait_spotting`** : Comprueba que el francotirador con rasgo `HIDDEN` es 100% invisible al escaneo de spotting mientras no dispare ni haga acción rápida, y se vuelve detectable inmediatamente tras abrir fuego o moverse aceleradamente.
- **`test_sniper_trait_removes_strongest_terrain_defense`** : Comprueba que un ataque de francotirador contra infantería en cobertura de terreno retira el dado de mayor jerarquía de la reserva de defensa del objetivo.
- **`test_rapid_fire_trait_close_combat`** : Valida que unidades con `RAPID_FIRE` ejecutan tiradas de asalto y fuego directo con capacidad de re-lanzamiento de dados desfavorables.
- **`test_tough_trait_close_combat_round1`** : Confirma que unidades con `TOUGH` amortiguan 1 punto de daño en la primera ronda de choque de asalto.

6. Referencia al Plan de Acción y Hoja de Ruta Evolutiva Unificada

> [!NOTE] El plan de acción priorizado para alcanzar el 100% de paridad del reglamento (Fases A, B y C: mecánicas tácticas de infantería/humo/transporte, cartas de mando/apoyo aéreo TAS-OAS e ingesta automatizada de los 36 escenarios restantes) se encuentra integrado de forma unificada en el documento final de la documentación: **09. Hoja de Ruta Evolutiva: Assault Architecture Roadmap (2024–2026)**.

🧠 06. Red Neuronal EfficientZero V2

Modelo de mundo, cabezas tácticas, HL-Gauss, Symlog, HRL Macro Nivel 1.

Red Neuronal y Motor de Inteligencia Artificial: EfficientZero V2

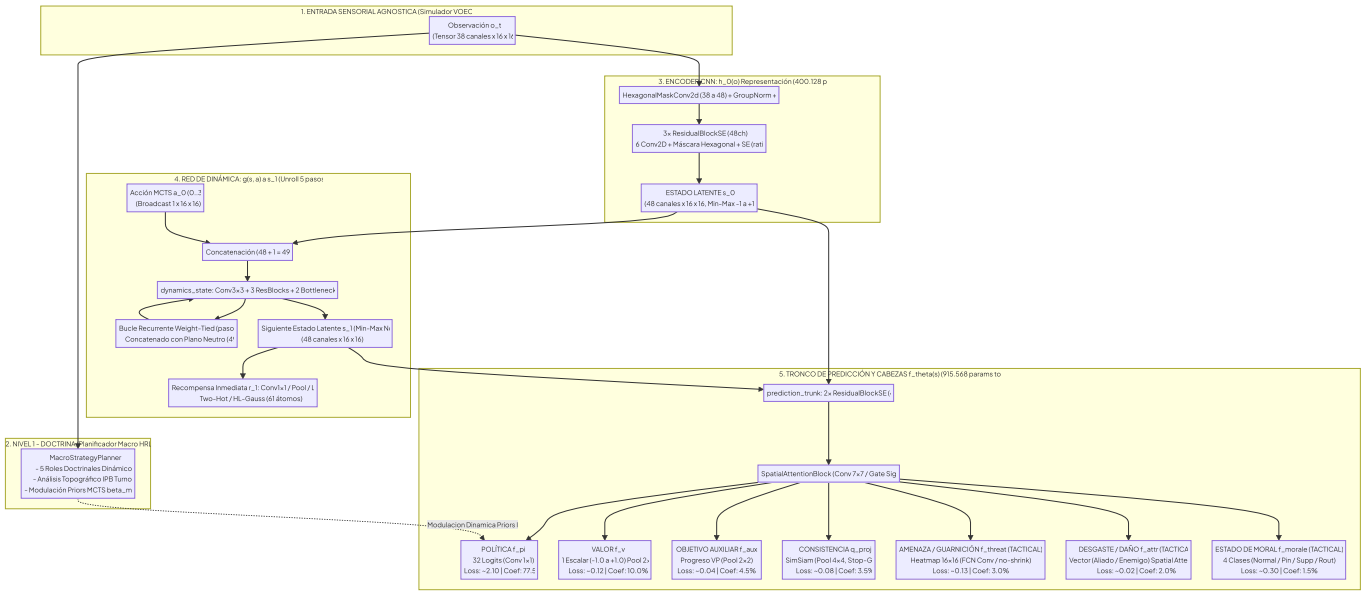
> **Estado**: Canónico y Exhaustivo
> **Directiva Fundamental**: El motor de aprendizaje es exclusivamente **EfficientZero V2** (`agents/efficientzero_v2/`). MuZero v1 clásico está 100% decomisionado.

---

1. Topología General de la Red Neuronal (EfficientZero V2)

La arquitectura desacoplada de **EfficientZero V2** implementa un modelo de mundo diferenciable de 4 componentes:

1. **Representación (h_0)**: Encoder espacial convolucional que proyecta la observación sensorial o_t en un espacio latente continuo s_0.
2. **Dinámica Recurrente (g)**: Simula transiciones hacia adelante $s_{t+1} = g(s_t, a_t)$ y emite la recompensa inmediata r_{t+1} sin consultar el motor de reglas físicas.
3. **Cabezas de Predicción ($f_h eta$)**: Evalúan la política prior $\pi(a)$, el valor $v(s)$, el objetivo auxiliar de VPs y las cabezas tácticas auxiliares (amenaza, desgaste, moral).
4. **Consistencia Auto-supervisada (q_{proj})**: Estabiliza el espacio latente mediante proyección SimSiam, stop-gradient y norma ℓ_2 unitaria.



1.1 Ficha Técnica de la Arquitectura (Telemetría MLOps Live Snapshot)

Conforme al panel de **Topología Dinámica de la Red Neuronal** en el Dashboard MLOps (`Puerto 8778 → Network Topology → Topology & Architecture`):

Parámetro / Dimensión	Especificación Canónica	Detalle de Implementación
Parámetros Totales del Modelo	**915.568 parámetros**	Arquitectura v2: GroupNorm + Squeeze-and-Excitation + Convoluciones Bottleneck (-15% parámetros vs baseline legacy).
Parámetros de Representación h_0	**~400.128 parámetros**	Encoder espacial convolucional de entrada.
Dimensiones de Entrada Sensorial	$B imes 38 imes 16 imes 16$	38 canales espaciales proyectados sobre coordenadas axiales (q, r).
Dimensiones de Estado Latente	$B imes 48 imes 16 imes 16$	48 canales latentes continuos con normalización Min-Max $s_{\text{norm}} = \frac{s - \min(s)}{\max(s) - \min(s) + \epsilon}$.
Campo Receptivo de Representación	**~15 × 15 hexágonos**	3 bloques convolucionales 3×3 con máscara hexagonal y compuertas SE.
Espacio de Acciones	**32 Logits**	Espacio discreto de maniobra táctica y combate.
Desenrollado Recurrente (Unroll RQs)	**5 Pasos ($k = 5$)**	Profundidad de simulación latente weight-tied por cada paso MCTS.
Bloques Convolucionales	**3D + 2P**	3 bloques en Dinámica + 2 bloques en Predicción + 3 en Representación.
Precisión de Inferencia y Train	**Auto (FP32 / AMP)**	Precisión adaptativa automática con optimizador AdamW fusionado en CUDA.

1.2 Estructura y Ponderación Multitarea de las Cabezas Activas

Durante el entrenamiento en `trainer.py`, los gradientes de las 7 cabezas activas se balancean mediante **PCGrad** ponderados por los siguientes coeficientes relativos:

Cabeza de Salida	Tipo de Arquitectura	Tensor de Salida	Coeficiente Relativo	Función Táctica Principal
`POLÍTICA` (f_π)	`Conv1x1(48 → 4) → MLP(1024 → 64 → 32)`	32 Logits Prior	**77.5%**	Probabilidades prior $\pi(a \mid s)$ para guiar la búsqueda MCTS.
`VALOR` (f_v)	`AdaptiveAvgPool2d(2x2) → Linear(128 → 61)`	Escalar $[-1.0, +1.0]$	**10.0%**	Retorno acumulado esperado modelado con Two-Hot / HL-Gauss.
`OBJETIVO AUXILIAR` (f_{aux})	`AdaptiveAvgPool2d(2x2) → Linear(128 → 1)`	Escalar $[0.0, 1.0]$	**4.5%**	Predicción del progreso diferencial de captura de VPs.
`CONSISTENCIA` (q_{proj})	`SimSiam (Pool 4x4) → Projector + Predictor`	Vector ℓ_2 unitario	**3.5%**	Pérdida autosupervisada con stop-gradient que previene el colapso latente.
`AMENAZA` (f_{threat})	`FCN Conv3x3 → Conv1x1 (no-shrink)`	Heatmap 16×16	**3.0%**	Mapa espacial directo de líneas de tiro y amenaza de fuego de reacción.
`DESGASTE` (f_{attr})	`SpatialAttentionBlock → MLP`	Vector $[Aliado, Enemigo]$	**2.0%**	Estimación de bajas esperadas en personal y potencia de fuego.
`ESTADO DE MORAL` (f_{morale})	`SpatialAttentionBlock → MLP`	4 Clases	**1.5%**	Distribución categórica [Normal, Pinned, Suppressed, Routed].

---

2. Los 5 Componentes del Modelo de Mundo y Desglose de 49 Capas Conv2D

2.1. Red de Representación (h_θ — 19 Capas Conv2D)

- **Entrada**: Tensor espacial sensorial $o_t \in \mathbb{R}^{B \times 38 \times H \times W}$ construido por `voec_sim` y adaptado por `agents/efficientzero_v2/core/observation.py` (16×16, $32 \times 32, 64 \times 64$).
- **Capas Internas**:
  - `Conv2d(38, 48, kernel_size=3, padding=1)` con máscara hexagonal.
  - `GroupNorm(num_groups=4, num_channels=48) + SiLU`.
  - 3× `ResidualBlockSE(48ch)`: cada uno con 2 convoluciones espaciales 3×3 con GroupNorm y activación SiLU, complementadas con compuertas Squeeze-and-Excitation (SE ratio: 4).
- **Escalado Min-Max Latente**: $s_{\text{norm}} = \frac{s - \min(s)}{\max(s) - \min(s) + \epsilon}$.
- **Salida**: Estado latente abstracto $s_0 \in \mathbb{R}^{B \times 48 \times 16 \times 16}$.

2.1.1 Catálogo Canónico de los 38 Canales Espaciales de Entrada (Tensor $38 \times 16 \times 16$)

Canal	Nombre Canónico	Categoría	Rango	Descripción Funcional
Ch 00	`ally_presence`	`UNITS`	$[0, 1]$	1.0 en hexágonos ocupados por unidades aliadas activas.
Ch 01	`enemy_presence`	`UNITS`	$[0, 1]$	1.0 en hexágonos con unidades enemigas detectadas.
Ch 02	`ally_hp_norm`	`UNITS`	$[0.0, 1.0]$	Puntos de vida normalizados de las escuadras aliadas según `max_strength`.
Ch 03	`enemy_hp_norm`	`UNITS`	$[0.0, 1.0]$	Puntos de vida normalizados de las escuadras enemigas según `max_strength`.
Ch 04	`ally_crossfire_risk`	`UNITS`	$[0.0, 1.0]$	Riesgo de fuego cruzado enemigo sobre unidades aliadas / estado de supresión aliada.
Ch 05	`enemy_target_engagement`	`UNITS`	$[0.0, 1.0]$	Hexágonos enemigos con objetivos aliados a su alcance de tiro / supresión enemiga.
Ch 06	`vp_contest_race`	`VP`	$[-1.0, +1.0]$	Ventaja diferencial en carrera hacia el VP: > 0 si el aliado llega antes, < 0 si el enemigo lidera.
Ch 07	`reaction_fire_threat`	`UNITS`	$[0.0, 1.0]$	Mapa de calor continuo de amenaza y fuego de reacción enemigo (*overwatch/opportunity fire*).
Ch 08	`ally_heavy_mg_support`	`UNITS`	$[0, 1]$	1.0 para ametralladoras pesadas y medianas aliadas (HMG, MMG, MG42).
Ch 09	`enemy_heavy_mg_support`	`UNITS`	$[0, 1]$	1.0 para ametralladoras pesadas y medianas enemigas (HMG, MMG, MG42).
Ch 10	`ally_indirect_mortar`	`UNITS`	$[0, 1]$	1.0 para armas de tiro indirecto / morteros aliados (50mm, 60mm, 81mm).
Ch 11	`local_firepower_balance`	`UNITS`	$[-1.0, +1.0]$	Balance local de potencia de fuego [$+1$ aliado dominante, 0 disputado, -1 enemigo dominante].
Ch 12	`map_playable`	`TERRAIN`	$[0, 1]$	1.0 en hexágonos válidos del mapa; 0.0 en los bordes fuera de tablero.
Ch 13	`vp_mask`	`VP`	$[0, 1]$	1.0 en hexágonos que son Puntos de Victoria (VPs).
Ch 14	`vp_owner_relative`	`VP`	$[-1.0, +1.0]$	$+1.0$ controlado por aliado, -1.0 por enemigo, 0.0 disputado/neutral.
Ch 15	`terrain_move_cost_norm`	`TERRAIN`	$[0.0, 1.0]$	Coste en puntos de movimiento (carretera = bajo, bosque/colina = alto).

Canal	Nombre Canónico	Categoría	Rango	Descripción Funcional
Ch 16	`terrain_cover_norm`	`TERRAIN`	$[0.0, 1.0]$	Bonificación defensiva de cobertura (edificios urbanos $= 0.5+$).
Ch 17	`terrain_los_block`	`TERRAIN`	$[0, 1]$	Bloqueo de línea de visión por elevación o construcciones densas.
Ch 18	`vp_distance_field`	`VP`	$[0.0, 1.0]$	Campo radial continuo de proximidad al VP más cercano ($1/(1 + extdist)$).
Ch 19	`q_coord_norm`	`HEX`	$[-1.0, +1.0]$	Coordenada axial Q del hexágono normalizada $[-1, 1]$.
Ch 20	`r_coord_norm`	`HEX`	$[-1.0, +1.0]$	Coordenada axial R del hexágono normalizada $[-1, 1]$.
Ch 21	`frontline_control`	`HEX`	$[-1.0, +1.0]$	Campo de potencial territorial $[-1, +1]$ (difusión y control de línea de frente).
Ch 22	`ally_mobility_norm`	`UNITS`	$[0.0, 1.0]$	Puntos de movimiento disponibles restantes de la escuadra activa.
Ch 23	`enemy_indirect_mortar`	`UNITS`	$[0, 1]$	1.0 para armas de tiro indirecto / morteros enemigos (50mm, 8cm).
Ch 24	`ally_direct_range_norm`	`UNITS`	$[0.0, 1.0]$	Alcance de fuego directo y proyección 2D de alcance óptimo de combate aliado.
Ch 25	`enemy_direct_range_norm`	`UNITS`	$[0.0, 1.0]$	Alcance de fuego directo y proyección 2D de alcance óptimo de combate enemigo.
Ch 26	`ally_fire_coverage`	`UNITS`	$[0.0, 1.0]$	Heatmap de cobertura de fuego directo aliado sobre cada casilla del mapa.
Ch 27	`heavy_fortifications`	`TERRAIN`	$[0.0, 1.0]$	Fortificaciones pesadas: $+1.0$ búnker/casamata, $+0.70$ trinchera, $+0.40$ urbano de piedra defensivo.
Ch 28	`ally_attack_power_norm`	`UNITS`	$[0.0, 1.0]$	Poder de fuego ofensivo aliado (dados de combate normalizados).
Ch 29	`hazard_mines_and_wire`	`TERRAIN`	$[0.0, 1.0]$	Peligros y obstáculos del terreno: $+1.0$ campos de minas, $+0.60$ alambradas de espino.
Ch 30	`ally_defense_power_norm`	`UNITS`	$[0.0, 1.0]$	Capacidad defensiva de las escuadras aliadas (dados de defensa normalizados).
Ch 31	`enemy_defense_power_norm`	`UNITS`	$[0.0, 1.0]$	Capacidad defensiva de las escuadras enemigas (dados de defensa normalizados).
Ch 32	`ally_doctrine_role`	`UNITS`	$[0.0, 1.0]$	Codificación continua de rol aliado (`SPEARHEAD` =1.0, `FIRE_SUPPORT` =0.75, `RECON` =0.50, `SCREEN` =0.25).
Ch 33	`enemy_doctrine_role`	`UNITS`	$[0.0, 1.0]$	Codificación continua de rol enemigo deducido (`SPEARHEAD` =1.0, `FIRE_SUPPORT` =0.75, `RECON` =0.50, `SCREEN` =0.25).
Ch 34	`ally_optimal_range_2d`	`UNITS`	$[0.0, 1.0]$	Footprint 2D de alcance óptimo de combate aliado proyectado sobre el tablero.
Ch 35	`enemy_optimal_range_2d`	`UNITS`	$[0.0, 1.0]$	Footprint 2D de alcance óptimo de combate enemigo proyectado sobre el tablero.
Ch 36	`turn_urgency_pressure`	`GLOBAL`	$[0.0, 1.5]$	Presión temporal continua (t/T) y campo de urgencia radial amplificado hacia VPs no capturados.
Ch 37	`paralysis_exposure_risk`	`UNITS`	$[0.0, 1.0]$	Mapa de calor de riesgo de parálisis: escuadras aliadas expuestas fuera de cobertura sin maniobra ni fuego.

- **Entrada**: Tensor espacial sensorial $o_t \in \mathbb{R}^{B \times C \times H \times W}$ construido por `voec_sim` y adaptado por `agents/efficientzero_v2/core/observation.py` ($C \in 32, 36, 37, 38$, agnóstico a la resolución espacial: $16 \times 16, 32 \times 32, 64 \times 64$).
  - **Canales 13 y 14 (`vp_mask` y `vp_owner_relative`)**: Codificación de la ubicación y control de los Puntos de Victoria modulada por su valor intrínseco relativo ($v_{\text{rel}} = v_{\text{vp}}/2.0$, asignando 1.0 para 2 VP y 1.5 para 3 VP clave; en el Canal 14, $+v_{\text{rel}}$ para control aliado y $-v_{\text{rel}}$ para enemigo).
  - **Canal 18 (`vp_distance_field`)**: Campo continuo euclidiano radial ponderado por el valor relativo de cada VP ($\max_{vp} \frac{v_{\text{rel}}}{1+d_{\text{vp}}}$). Modulado dinámicamente por la presión temporal de la Tarea 2.3: si el bando no controla el VP, la pendiente se intensifica con $\left(1.0 + 0.5 \cdot \frac{t}{T}\right)$, forzando convergencia espacial rápida hacia los objetivos decisivos antes de que transcurran los turnos.
  - **Canal 36 (`turn_urgency_pressure`)**: Difusión espacial continua de la presión temporal global $\tau = \frac{t}{T_{\max}}$, amplificada un $+50$ en la proximidad de los VPs en disputa para alertar a la red sobre la ventana crítica de captura.
  - **Canal 37 (`paralysis_exposure_risk`)**: Mapa de calor que identifica en tiempo real las unidades aliadas situadas en terreno abierto (`cover < 0.35`) fuera del perímetro defensivo de los VPs, proyectando el riesgo de incurrir en la penalización acumulativa por parálisis.
- **Convoluciones con Máscara Hexagonal (`HexagonalMaskConv2d`)**: A diferencia de las convoluciones cartesianas 3×3, anula estáticamente las dos esquinas diagonales que no comparten arista en la cuadrícula hexagonal axial (q, r), eliminando la distorsión anisotrópica.
- **Bloques Residuales con Squeeze-and-Excitation (SE)**: Cada uno de los 4 bloques residuales (`48ch`) contiene 2 convoluciones espaciales 3×3 más 2 convoluciones puntuales 1×1 de cuello de botella SE (`48 → 12 → 48`).
- **Salida**: Estado latente abstracto inicial $s_0 \in \mathbb{R}^{B \times 48 \times H \times W}$.

2.2. Red de Dinámica Recurrente Weight-Tied (g_θ — 18 Capas Conv2D)

- **Entrada**: Estado latente $s_t \in \mathbb{R}^{B \times 48 \times H \times W}$ concatenado con el plano de acción expandido dinámicamente a la resolución actual $(B, 1, H, W)$.
- **Razonamiento Recurrente de Peso Compartido (k pasos, Palma et al. 2025)**: El bloque `dynamics_state` (17 capas Conv2D) puede iterar k = \text{dynamics_recurrent_steps} veces reutilizando exactamente los mismos pesos (0 parámetros adicionales). En el paso $i = 0$ recibe la acción candidata; en los pasos $i = 1 \dots k-1$ refina el estado latente concatenando un plano neutro de ceros para propagar cadenas causales multi-salto.
- **Escalado Min-Max Latente**: Para evitar que las activaciones exploten o se desvanezcan a lo largo del desenrollado recursivo, el estado latente resultante se normaliza en $[0, 1]$:

$$s_{\text{norm}} = \frac{s - \min(s)}{\max(s) - \min(s) + \epsilon}$$

- **Cabeza 1 — Recompensa Inmediata (`reward_head`)**: `Conv1x1(48 → 16) → AdaptiveAvgPool2d(1) → Linear(16 → 64 → 2K+1)` .

2.3. Tronco de Predicción y Preservación de Resolución Espacial (f_θ)

- **Tronco (`prediction_trunk`)**: 2 bloques `ResidualBlockSE` operando a la **máxima capacidad latente de 48 canales** (`hidden_dim = 48`, con un total de 8 capas Conv2D de 48 canales y compuertas Squeeze-and-Excitation / ECA), seguidos de un bloque `SpatialAttentionBlock` (1 capa Conv2D 7×7 sobre media y máximo por canal). No existe reducción prematura de canales a 8; la capacidad de 48 canales se mantiene estrictamente en todo el tronco.
- **Preservación de Topología Espacial (FCN vs Sectores 4×4)**:
  - **Cabezas Espaciales FCN Puras (`threat_map`, `final_ownership`)**: Operan directamente a resolución nativa $H \times W$ mediante convoluciones 2D locales (`Conv3x3(48 → 16) → GroupNorm → SiLU → Conv1x1(16 → 1)`), produciendo mapas bidimensionales hexágono a hexágono sin colapso alguno.
  - **Cabezas Tácticas con Atención Espacial Sectorizada (`attrition`, `morale`, `objective_intent`, `multi_vp`)**: En lugar de Global Average Pooling puro (1×1), procesan el estado latente mediante `SpatialAttentionBlock(48, val_channels, (4, 4))`, reteniendo una matriz de $4 \times 4 = 16$ celdas espaciales con atención aprendida. Esto conserva la granularidad espacial de frentes, flancos y concentración de tropas antes del aplanado hacia sus respectivos clasificadores y regresores MLP.
- **Adaptación Espacial Bilineal (`_adapt_spatial`)**: Si el mapa de entrada tiene dimensiones distintas a 16×16 (por ejemplo 32×32 o 64×64), aplica interpolación bilineal al tamaño canónico 16×16 exclusivamente para las cabezas densas (`policy`, `value`), mientras que las cabezas FCN conservan la resolución $H \times W$.

2.4. Cabeza 11 — Proyector y Predictor de Consistencia Temporal SimSiam (q_θ) y Estabilidad Anti-Colapso

Para evitar el colapso de representación dimensional (evitando que la dinámica recurrente aprenda constantes triviales), el módulo SimSiam (`consistency_projector` + `consistency_predictor`) incorpora dos salvaguardas estrictas:

1. **Stop-Gradient Dual en el Target**: Durante el entrenamiento (`EfficientZeroV2Trainer`), la representación objetivo $h_\theta(o_{t+k})$ se evalúa bajo `torch.no_grad()` y su proyección resultante se desacopla explícitamente mediante `.detach()` antes del cómputo de la pérdida de similitud del coseno. Los gradientes retropropagados por SimSiam fluyen exclusivamente a través de la predicción latente $p_\theta(q_\theta(s_k))$, sin contaminar ni desestabilizar las representaciones del tronco ni las cabezas de valor/recompensa.
2. **Normalización ℓ_2 Unitaria**: Ambos vectores se normalizan estrictamente sobre la esfera unitaria ($|z|_2 = 1$, $|p|_2 = 1$) mediante `F.normalize(..., dim=-1)` *antes del producto escalar:* $$\mathcal{L}_{\text{consistency}} = -\frac{1}{2} \left(\frac{p_\theta(q_\theta(s_k))}{\|p_\theta(q_\theta(s_k))\|_2} \cdot \text{stop_grad}\left(\frac{q_\theta(h_\theta(o_{t+k}))}{\|q_\theta(h_\theta(o_{t+k}))\|_2}\right) \right)$$

2.5. Modulación Dinámica de Priors y Decaimiento Anti-Miopía (Tarea 2.8 [v2.5.26])

Para resolver el dilema identificado en la auditoría arquitectónica (donde un sesgo doctrinal estático del $+35$ acota artificialmente la exploración en el árbol de búsqueda MCTS e induce *"Objective Myopia"*), se introduce la modulación temporal gobernada por programa de decaimiento:

$$\beta_{\text{eff}}(a) = 1.0 + (\beta_{\text{raw}}(a) - 1.0) \cdot w_{\text{macro}}(t)$$

Donde la escala de modulación $w_{\text{macro}}(t)$ puede configurarse bajo tres regímenes:

1. **Decaimiento Cosinoidal Progresivo (Recomendado)**:

$$w_{\text{macro}}(t) = w_{\min} + \frac{1}{2}(w_{\max} - w_{\min}) \left(1 + \cos\left(\pi \frac{t}{T_{\text{decay}}}\right)\right)$$

   Proporciona guía heurística robusta ($w_{\max} = 1.0$) en fases iniciales cuando la política no conoce los objetivos, y decae suavemente hacia cero ($w_{\min} = 0.0$) permitiendo que la red neuronal descubra tácticas puras y sacrificios posicionales no convencionales.
2. **Decaimiento Lineal**:

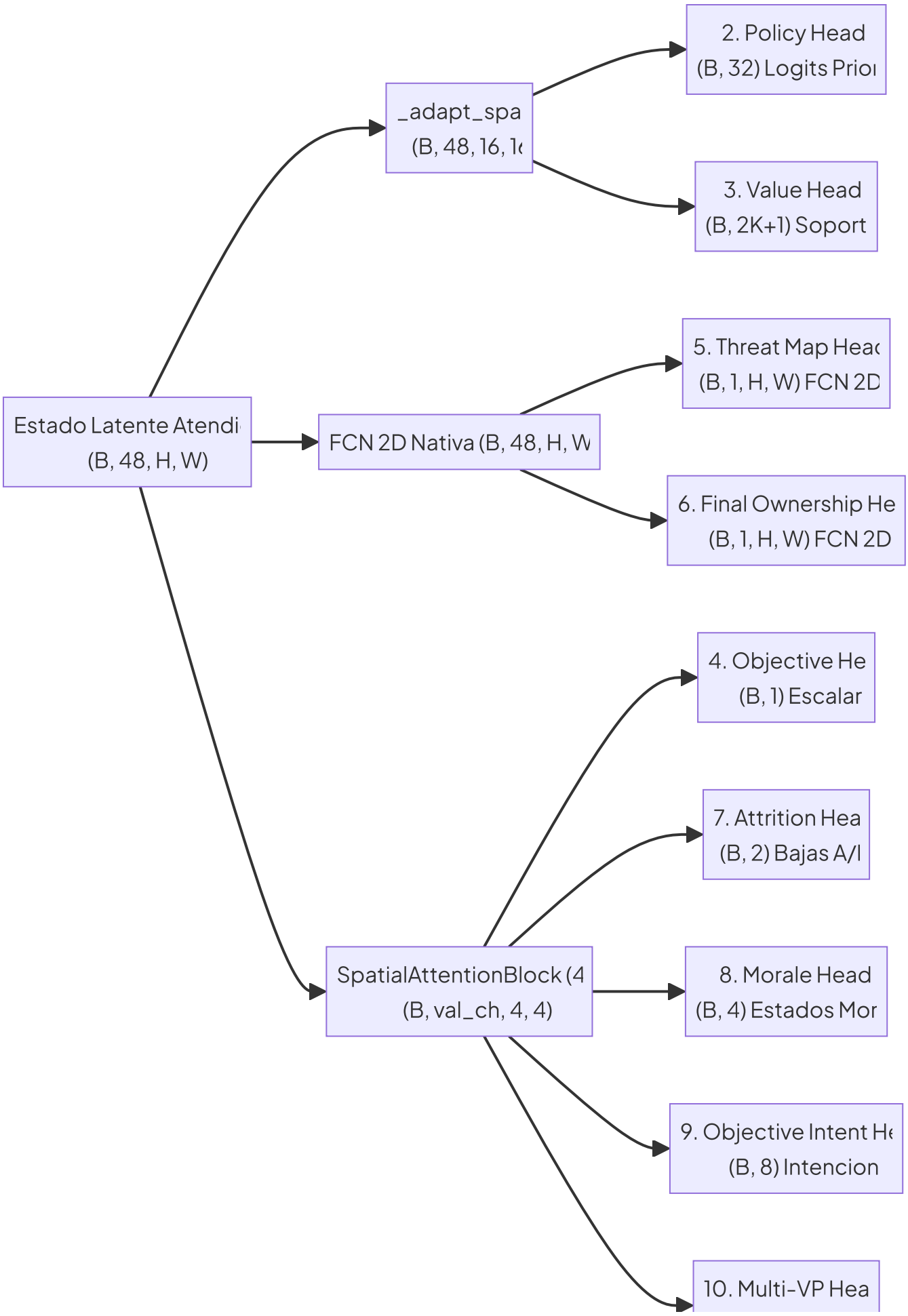
$$w_{\text{macro}}(t) = w_{\max} + (w_{\min} - w_{\max}) \cdot \min\left(1.0, \frac{t}{T_{\text{decay}}}\right)$$

3. **Ablación Científica A/B**:
   - **Variante A (Red Pura / Búsqueda Libre)**: $w_{\text{macro}} = 0.0$, donde los priors de la red alimentan MCTS al 100% sin sesgos heurísticos.
   - **Variante B (Doctrinal Shaped)**: $w_{\text{macro}} = 1.0$ estático para evaluar la ganancia o pérdida neta de ELO.

---

3. Catálogo Completo de las 11 Cabezas de Salida

La red neuronal produce simultáneamente **11 cabezas especializadas** (1 en dinámica, 1 de consistencia temporal y 9 en el tronco de predicción):


Estado Latente Atendi
(B, 48, H, W)
_adapt_spa
(B, 48, 16, 1
2. Policy Head
(B, 32) Logits Prio
3. Value Head
(B, 2K+1) Soport
FCN 2D Nativa (B, 48, H, W
5. Threat Map Heac
(B, 1, H, W) FCN 2D
6. Final Ownership He
(B, 1, H, W) FCN 2D
SpatialAttentionBlock (4
(B, val_ch, 4, 4)
4. Objective He
(B, 1) Escalar
7. Attrition Hea
(B, 2) Bajas A/I
8. Morale Head
(B, 4) Estados Mor
9. Objective Intent He
(B, 8) Intencion
10. Multi-VP Hea

(B, 4) Vector VPs

#	Nombre de Cabeza	Módulo PyTorch	Dimensión de Salida	Topología Espacial / Función Táctica
1	**Reward** (`reward_logits`)	`reward_head`	`[B, 2K+1]`	Distribución Two-Hot / HL-Gauss de la recompensa inmediata tras ejecutar a_t.
2	**Policy** (`policy_logits`)	`prediction_policy`	`[B, 32]`	Cuello de botella `Conv1x1(48→4)` → MLP prior sobre 32 ranuras legales en MCTS.
3	**Value** (`value_logits`)	`prediction_value`	`[B, 2K+1]`	Distribución Two-Hot / HL-Gauss del retorno descontado esperado desde s_t.
4	**Objective** (`objective_logit`)	`prediction_objective`	`[B, 1]`	`SpatialAttentionBlock(4x4)` → Estimador de progreso hacia la victoria por objetivos.
5	**Threat Map** (`threat_map_logits`)	`prediction_threat_map`	`[B, 1, H, W]`	**FCN 2D Nativa**: Mapa convolucional directo de riesgo de fuego y LOS enemiga inmediata.
6	**Final Ownership Map** (`final_ownership_pred`)	`prediction_final_ownership`	`[B, 1, H, W]`	**FCN 2D Nativa**: Predicción convolucional de posesión final del hexágono a T (KataGo).
7	**Attrition** (`attrition_pred`)	`prediction_attrition`	`[B, 2]`	`SpatialAttentionBlock(4x4)` → Vector de bajas esperadas `[aliadas, enemigas]` .
8	**Morale** (`morale_logits`)	`prediction_morale`	`[B, 4]`	`SpatialAttentionBlock(4x4)` → Distribución de cohesión: `NORMAL` , `PINNED` , `SUPPRESSED` , `ROUTED` .
9	**Objective Intent** (`objective_intent_logits`)	`prediction_intent`	`[B, 8]`	`SpatialAttentionBlock(4x4)` → Clasificador de intención táctica (8 clases doctrinales).
10	**Multi-VP** (`multi_vp_pred`)	`prediction_multi_vp`	`[B, 4]`	`SpatialAttentionBlock(4x4)` → Vector `[control_ratio, contested_ratio, threat, time]` .
11	**Consistency** (`latent_proj` / `pred`)	`consistency_projector`	`[B, 128]`	Embedding SimSiam ℓ_2-normalizado con stop-gradient dual para evitar colapso latente.

---

4. Representación Discreta de Valor: Two-Hot, HL-Gauss y Symlog

La literatura reciente de Deep Reinforcement Learning (Google DeepMind 2024, DreamerV3) demuestra que la regresión escalar directa mediante error cuadrático medio (MSE) fracasa ante recompensas dispersas y multiescala.

4.1. Transformaciones Contractivas: ϕ-Transform y Symlog

Para comprimir la escala dinámica sin perder resolución en torno a cero:

- **ϕ-Transform Canónica**:

$$h(x) = \text{sign}(x)(\sqrt{|x|+1} - 1) + \epsilon x$$

- **Symlog Transform (DreamerV3)**:

$$\text{symlog}(x) = \text{sign}(x)\ln(|x|+1)$$
$$\text{symexp}(y) = \text{sign}(y)(\exp(|y|) - 1)$$

4.2. Discretización de Soporte: Two-Hot vs. HL-Gauss

Característica	Two-Hot Clásico	HL-Gauss (DeepMind 2024, "Stop Regressing")
Mapeo de Probabilidad	Rígido sobre los 2 átomos vecinos más cercanos.	Distribución Gaussiana continua $\mathcal{N}(y, \sigma^2)$ extendida sobre el histograma.
Función Objetivo	Cross-Entropy sobre átomos discretos.	Cross-Entropy / Divergencia KL contra la campana gaussiana.
Efecto de Regularización	Puede sufrir sobreajuste o picos de varianza en horizontes largos.	Actúa como *label smoothing* estocástico natural; gradientes notablemente más suaves y estables.
Recuperación Escalar	$\hat{v} = h^{-1}\left(\sum_{i=-K}^{K} i \cdot p_i\right)$	$\hat{v} = \text{symexp}\left(\sum_{i=-K}^{K} z_i \cdot p_i\right)$

4.3. Two-Hot Asimétrico con Warp Exponencial en Torno a Cero (Discriminación de Micro-Desgaste)

En wargames tácticos, las micro-ventajas de desgaste en las primeras escaramuzas (ej. forzar una retirada o causar una baja de 0.05 puntos de VP) caían en el mismo átomo central cero del soporte estándar $[-30, +30]$, generando gradientes nulos o ceguera táctica.

Para solucionarlo, `agents/efficientzero_v2/core/support.py` implementa el soporte asimétrico no lineal con `asym_zero_boost=True` y $\alpha = 0.75$:

$$\text{warp}(x) = \text{sign}(x) \cdot |x|^{\alpha}, \quad \alpha = 0.75$$
$$\text{unwarp}(y) = \text{sign}(y) \cdot |y|^{1/\alpha}$$

- **Efecto de Resolución**: Un valor pequeño de $x = 0.04$ se proyecta a ≈ 0.089, expandiendo su representación a lo largo de los átomos discretos adyacentes sin modificar los 61 átomos totales ni romper la compatibilidad con checkpoints existentes.
- **Inversión Exacta**: Durante la inferencia escalar en MCTS (`support_to_scalar`), el valor esperado se des-transforma simétricamente mediante $1/\alpha \approx 1.3333$, recuperando la métrica continua real.

4.4. Régimen Dual de Inferencia y Pérdidas: Soporte Discreto vs. Modo Escalar Directo (MSE)

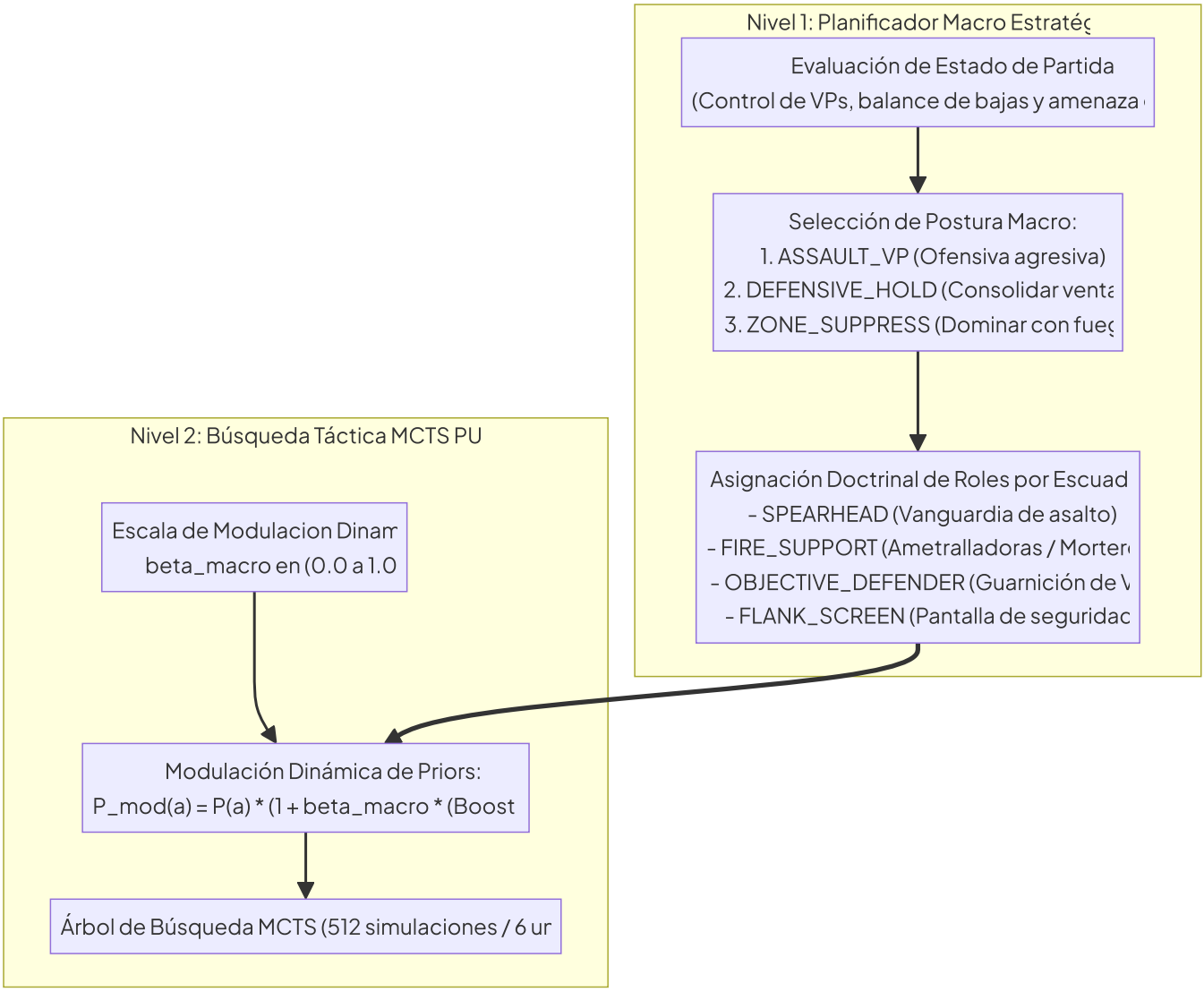
El motor de EfficientZero V2 (`EfficientZeroV2Trainer`) soporta dos regímenes matemáticos para las cabezas de Valor y Recompensa, gobernados por el parámetro `support_size` :

1. **Régimen Categórico Two-Hot (`support_size > 0` , ej. 30 $\rightarrow$ 61 átomos)**:
    - Las cabezas proyectan logits a una distribución discreta de átomos.
    - La función de coste es **Cross-Entropy Categórica** contra la interpolación two-hot: $$\mathcal{L}_{\text{val}} = -\sum_{i=-K}^{K} p_i^* \ln(p_i)$$
    - Los valores numéricos de pérdida se expresan en **nats** (rango típico: 0.5 a $4.11 = \ln(61)$).
2. **Régimen Escalar Continuo (`support_size == 0`)**:
    - Las cabezas de valor y recompensa finalizan en una única neurona lineal escalar directa (`Linear(64, 1)`).
    - La función de coste es **Mean Squared Error (MSE)** directo: $$\mathcal{L}_{\text{val}} = \frac{1}{B} \sum_{b=1}^{B} (v_b - v_b^*)^2, \quad \mathcal{L}_{\text{rew}} = \frac{1}{B} \sum_{b=1}^{B} (r_b - r_b^*)^2$$
    - **Interpretación Dimensional**: Las pérdidas reportadas son magnitudes cuadráticas de las unidades del juego. Para interpretarlas en la escala de recompensa real, se debe extraer la raíz del error cuadrático medio: $$\text{RMSE}_{\text{reward}} = \sqrt{\mathcal{L}_{\text{rew}}}, \quad \text{RMSE}_{\text{value}} = \sqrt{\mathcal{L}_{\text{val}}}$$
    - En fases con recompensas de captura elevadas (ej. $+16.14$ por VP), un $\mathcal{L}_{\text{rew}} \approx 29.7$ representa un $\text{RMSE} \approx 5.45$ puntos, perfectamente acorde a la dispersión temporal y varianza estocástica de los combates.

---

5. Planificador Macro Jerárquico HRL (Nivel 1)

El mayor desafío en wargames tácticos es la coordinación simultánea de múltiples escuadras en un espacio de acción combinatorio. *Assault* resuelve esto mediante una jerarquía de toma de decisiones en dos niveles:



- **Desacoplamiento Operativo**: El Nivel 1 no decide cada paso físico de una unidad; define la doctrina.
- **Modulación Dinámica de Priors (β_{macro})**: Durante el paso raíz del MCTS, las acciones tácticas que coinciden con el rol de la unidad y la postura del equipo reciben un bono modulado por el factor dinámico $\beta_{\text{macro}} \in [0.0, 1.0]$ (`macro_modulation_scale`). Con $\beta_{\text{macro}} = 1.0$, el bono doctrinal estándar es de $+35$ ($1.35\times$), guiando la búsqueda de Monte Carlo hacia rutas coherentes con el plan general. Con $\beta_{\text{macro}} = 0.0$, el sesgo doctrinal se desactiva al 100%, dejando los priors puros de la red neuronal intactos para ablación libre de sesgos.

5.1. Sintonización del Macro-Planner y Modulación de Priors de Asalto

Durante la fase de juego táctico competitivo frente a políticas estocásticas y baselines, se detectaron y corrigieron tres aspectos clave del planificador macro:

1. **Parseo Canónico de Acciones**: El motor serializa las acciones de movimiento con delimitadores de dos puntos (`MOVE:<unit_id>:<q>:<r>`), mientras que ciertas variantes emplean tuplas por coma. El parser ahora desacopla de forma robusta ambos esquemas y extrae de forma inequívoca las coordenadas de destino axiales (q, r).
2. **Prior de Asalto y Detección de Captura Inmediata**: Si una acción legal sitúa a la unidad sobre un hexágono de punto de victoria neutral o enemigo (`dest_qr in enemy_vps`), el Macro Planner inyecta un factor de amplificación de prior agresivo (1.5 \times \text{assault_boost}), priorizando la exploración de ramas de captura en la raíz del árbol MCTS.
3. **Dinámica de Roles en `ASSAULT_VP`** : Bajo la postura `ASSAULT_VP` , las unidades de infantería avanzada a distancia operativa (≤ 8 hexágonos del objetivo) son asignadas doctrinariamente a `ROLE_SPEARHEAD` , impidiendo que queden indebidamente confinadas a `ROLE_OBJECTIVE_DEFENDER` en los VPs de despliegue inicial.

5.2. Gradiente Continuo de Recompensa y Urgencia Terminal

Para superar la aversión al riesgo excesiva donde las penalizaciones por fuego de oportunidad paralizaban el avance hacia los objetivos lejanos:

1. **Eliminación del Corte Artificial de Distancia (`opportunity_near_vp_max_dist`)**: Anteriormente limitado a 2 ó 4 hexágonos, se extendió a un radio efectivo de mapa completo (12.0 hexágonos por defecto). Esto garantiza un gradiente continuo de recompensa de aproximación (`team_progress_delta` y `objective_progress_delta`) desde el primer turno de juego.

2. **Atenuación del Factor de Exposición al Movimiento**: El factor de penalización preventiva por exposición a fuego de reacción durante el movimiento se redujo de 1.0 a 0.6, permitiendo que los beneficios proyectados de avance compensen razonablemente el riesgo táctico medido.
3. **Penalización por Derrota Terminal (`terminal_loss_penalty` = 1.5, `terminal_scale` = 1.5)**: La derrota por llegar al límite de turnos sin controlar la mayoría de VPs acarrea un coste terminal que supera con creces cualquier daño individual acumulado en la aproximación. Esta calibración matemática fuerza al MCTS a asumir riesgos calculados para evitar la derrota por tiempo.

5.3. Modulación Dinámica Continua y Desacoplamiento Doctrinal (β_{macro}) [Consolidado v2.5.27 / Tarea 2.8]

Para solventar el riesgo de sesgo heurístico excesivo (*Objective Myopia*) que pudiese acotar la exploración en etapas tempranas de autojuego o impedir maniobras contraintuitivas pero tácticamente superiores, se implementó el parámetro canónico `macro_modulation_scale` ($\beta_{\text{macro}} \geq 0.0$):

$$\text{boost}_{\text{eff}}(a) = 1.0 + (\text{boost}(\text{doctrinal})(a) - 1.0) \cdot \beta_{\text{macro}}$$

$$P_{\text{mod}}(a) = \frac{P_{\text{net}}(a) \cdot \text{boost}_{\text{eff}}(a)}{\sum_b P_{\text{net}}(b) \cdot \text{boost}_{\text{eff}}(b)}$$

Este desacoplamiento continuo habilita tres regímenes operativos gobernados desde la configuración YAML de autojuego (`cfg.selfplay.macro_modulation_scale`):

1. **Régimen de Red Pura / Búsqueda Libre ($\beta_{\text{macro}} = 0.0$)**:
   - $\text{boost}_{\text{eff}}(a) = 1.0 \implies P_{\text{mod}}(a) = P_{\text{net}}(a)$.
   - Los priors evaluados por la red neuronal se transmiten con 100% de pureza matemática al árbol MCTS. Esencial para la Tarea 2.8 (estudios de ablación A/B) para auditar el rendimiento táctico sin muletas heurísticas.
2. **Régimen de Calentamiento / Annealing Progresivo ($\beta_{\text{macro}}(t) = \beta_0 \cdot \gamma^t$ o $0.0 \to 1.0$)**:
   - Permite arrancar el entrenamiento con exploración libre pura ($\beta = 0.0$) y reforzar la disciplina doctrinal a medida que el valor converge, o viceversa (guiar el arranque y retirar el sesgo conforme la red madura).
3. **Régimen Doctrinal Completo ($\beta_{\text{macro}} = 1.0$)**:
   - Aplica el $+35$ canónico en roles de vanguardia y fuego de apoyo y $+50$ en captura directa de VPs. Verificado formalmente en `agents/efficientzero_v2/tests/test_heads_improvements.py`.

5.4. Perfilado Topográfico 5D e Inyección de Priors IPB Turno 0 (Tarea 4.3 – v2.5.28)

Complementando los roles doctrinales y la modulación β_{macro}, el **Nivel 1 (Planificador Macro)** incorpora el análisis topográfico del terreno en **Turno 0** (*Intelligence Preparation of the Battlefield - IPB*), desarrollado en `assault_model/map/topography_analyzer.py` e integrado en `agents/efficientzero_v2/core/macro_planner.py`.

5.4.1 Vector 5D por Hexágono (`Hex5DProfile`)

Para cada celda (q, r) del escenario, se computa de forma estática determinista un vector de 5 dimensiones:

1. **Cover Rating ($c \in [0.0, 1.0]$)**: Resistencia balística y ocultación del hexágono (edificios ≥ 0.85, bosques densos ≥ 0.70, crestas ≥ 0.40, despejado ≤ 0.15).
2. **Elevation Dominance ($e \in [0.0, 1.0]$)**: Dominancia orográfica sobre el entorno circundante (crestas y colinas ≥ 0.85).
3. **LOS Exposure ($l_{\text{exp}} \in [0.0, 1.0]$)**: Vulnerabilidad a líneas de visión y fuego cruzado en terreno abierto circundante.
4. **Chokepoint Bottleneck ($\text{chk} \in [0.0, 1.0]$)**: Constricción de maniobra debida a hexágonos adyacentes infranqueables (agua, relieve escarpado).
5. **VP Proximity Utility ($u_{\text{vp}} \in [0.0, 1.0]$)**: Proximidad hiperbólica hacia los Puntos de Victoria: $u_{\text{vp}} = \frac{1}{1+\min_v d((q,r),v)}$.

5.4.2 Clasificación Táctica y Sectores del Terreno

A partir del perfil 5D, los hexágonos se clasifican deterministamente en 6 categorías operacionales:

- `KEY_TERRAIN`: Hexágonos que contienen Puntos de Victoria (VPs).
- `BASE_OF_FIRE`: Alturas dominantes ($e \geq 0.75$) o posiciones de cobertura pesada con campo de tiro inmediato sobre los VPs ($1 \leq d \leq 6$).
- `CHOKEPOINT`: Pasos obligados y cuellos de botella con constricción severa ($\text{chk} \geq 0.60$).
- `KILLZONE`: Terreno despejado y abierto con alta exposición ($c < 0.35, l_{\text{exp}} \geq 0.60$) a corta distancia de objetivos ($d \leq 4$).
- `DEFILADE_CORRIDOR`: Rutas de aproximación cubiertas por vegetación densa o relieve ($c \geq 0.65$ o $c \geq 0.50$ con baja exposición) que ocultan el avance hacia el objetivo.
- `MANEUVER_TRANSIT`: Espacio estándar de maniobra táctica.

5.4.3 Reglas de Modulación de Priors en MCTS

En el cálculo de `compute_action_boost` del Macro Planner:

- **Avance en Desenfilada**: $+25$ de refuerzo cuando la unidad avanza acortando distancia al VP objetivo a través de `DEFILADE_CORRIDOR`.
- **Disuasión en Killzones**: Penalización severa ($0.55\times$ para roles generales, $0.65\times$ para unidades `SPEARHEAD` que realizan maniobras laterales sin cerrar distancia) al internarse en `KILLZONE` abierta expuesta a fuego cruzado.
- **Base de Fuego**: $+30$ de refuerzo para unidades de apoyo (`ROLE_FIRE_SUPPORT`) que alcanzan hexágonos designados como `BASE_OF_FIRE`.

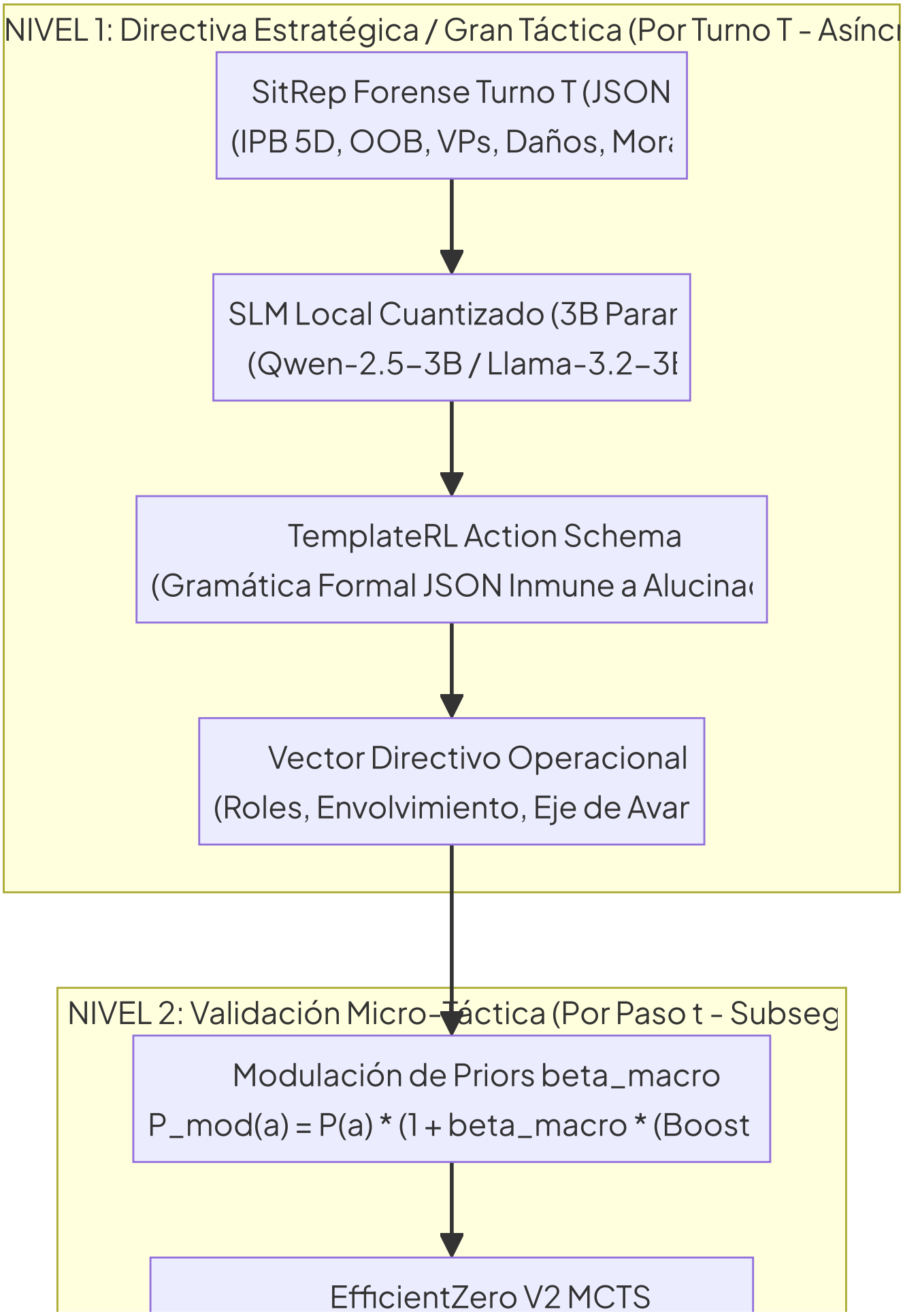
- **Fijación de Chokepoints**: $+20$ de refuerzo para roles de defensa y pantalla (`ROLE_FLANK_SCREEN` , `ROLE_OBJECTIVE_DEFENDER`) que aseguran embudos tácticos.
- **Ablación Pura**: El factor continuo \beta_{\text{macro}} = \text{modulation_scale} neutraliza exactamente todos los boosts a 1.0 cuando se evalúa en régimen de red puramente neural.

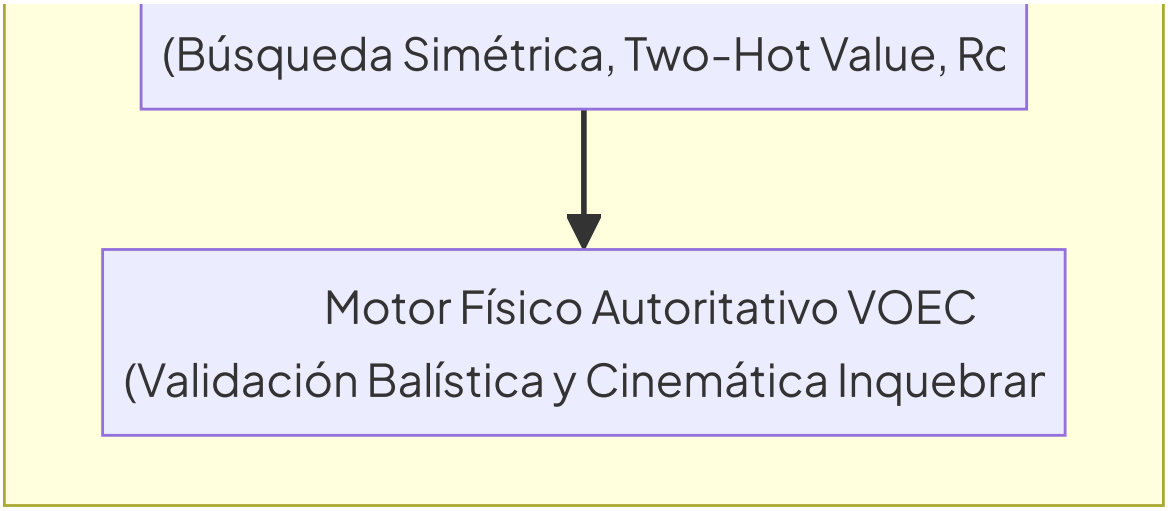
5.4.4 Eficiencia en Entrenamiento y Telemetría MLOps

1. **Coste Turno 0 Nulo (0 ms en pasos posteriores)**: El análisis se ejecuta una única vez al inicio del episodio y se conserva en caché estática en memoria (`_topography_cache`), con cero sobrecarga en los rollouts subsiguientes.
2. **Poda de Ramas Suicidas**: Al penalizar la exploración en killzones abiertas, el árbol MCTS no desperdicia simulaciones en movimientos estériles o aniquilaciones tempranas, concentrando los rollouts en maniobras tácticas viables.
3. **Visibilidad en Replay (:8778)**: Capa interactiva `🛡️ IPB Topography` con polígonos coloreados, etiquetas tácticas vectoriales en cada hexágono y tooltip de inspección sectorial con desglose 5D.

5.5 Especificación Prospectiva: Arquitecturas Híbridas LLM + RL para Wargames Abiertos (TemplateRL) [Planificado v2.6.0]

El campo de los wargames basados en IA ha evolucionado hacia entornos abiertos y modelos híbridos (*Shall We Play a Game? Language Models for Open-ended Wargames*, arXiv:2509.17192v3; y *Vox Deorum: A Hybrid LLM Architecture for 4X / Grand Strategy Game AI*). En lugar de confinar los LLMs a tareas pasivas de RAG documental, esta arquitectura prospectiva los posiciona como **Planificadores Estratégicos de Alto Nivel (Nivel 1)** gobernados por **Template-Guided Reinforcement Learning (TemplateRL)**.


NIVEL 1: Directiva Estratégica / Gran Táctica (Por Turno T - Asínc
SitRep Forense Turno T (JSON
(IPB 5D, OOB, VPs, Daños, Mor
SLM Local Cuantizado (3B Parar
(Qwen-2.5-3B / Llama-3.2-3
TemplateRL Action Schema
(Gramática Formal JSON Inmune a Alucina
Vector Directivo Operacional
(Roles, Envolvimiento, Eje de Avar
NIVEL 2: Validación Micro-Táctica (Por Paso t - Subseg
Modulación de Priors beta_macro
P_mod(a) = P(a) * (1 + beta_macro * (Boost
EfficientZero V2 MCTS



5.5.1 Principios de Diseño

1. **Separación Estricta de Frecuencias (Turn vs. Step):**
   - El SLM **nunca** se evalúa por paso táctico o micro-acción (lo que arruinaría el rendimiento). Se ejecuta **únicamente una vez al inicio del Turno (T)** en un hilo en segundo plano, emitiendo una directiva de ~80 tokens en ~0.4–0.6s.
   - El árbol MCTS de EfficientZero V2 opera a nivel de paso (t) en microsegundos, usando la directiva activa para modular β_{macro}.
2. **Cero Alucinación mediante TemplateRL:**
   - El modelo de lenguaje tiene prohibido generar texto libre o inventar acciones. Solo puede rellenar un esquema formal tipado (`OperationalDirectiveTemplate`): postura general (`ASSAULT_VP` , `PIN_AND_ENVELOP` , `DEFENSIVE_FALLBACK`), vector objetivo, asignación de roles por unidad y condiciones de repliegue.
   - Si una sugerencia es físicamente inviable, el motor `VOECSimulator` y las simulaciones de dados de MCTS la podan matemáticamente.
3. **Aislamiento Hardware y Rendimiento Local (Zero-Interference):**
   - **En GPU NVIDIA RTX 4060 (8 GB VRAM):** Un SLM cuantizado en 4-bit (GGUF `Q4_K_M` , ~~2.0 GB VRAM) coexiste de forma segura con EfficientZero V2~~ (1.2 GB VRAM), dejando más de 4.8 GB de VRAM libres sin riesgo de OOM.
   - **En CPU (Alternativa de Aislamiento Total):** El SLM puede ejecutarse al 100% en la CPU mediante `llama.cpp` (AVX2 / AVX-512) a ~40 tokens/s, consumiendo 0 MB de VRAM y garantizando que la GPU esté al 100% dedicada a la red neuronal.
   - **Caché Doctrinal Turno 0:** Para configuraciones iniciales idénticas, las directivas se recuperan de una caché determinista en < 1 ms, eliminando la inferencia del modelo.
4. **Modo de Operación:** Durante autojuego masivo, se preserva el planificador heurístico/neuronal ultrarrápido; el modo LLM se activa en torneos de evaluación, interacción humano-en-el-bucle (comandante natural) y Match Replay forense.

---

6. Mitigación de Conflicto de Gradientes: PCGrad

Al entrenar simultáneamente la política, el valor, el objetivo y las cabezas tácticas auxiliares, los gradientes pueden entrar en conflicto destructivo: $$\langle g_{\text{task}i}, g{\text{task}_j} \rangle < 0$$

Para resolverlo, el balanceador de pérdidas utiliza **PCGrad (*Projecting Conflicting Gradients*)**: Si dos gradientes apuntan en direcciones opuestas, el gradiente de la tarea secundaria se proyecta sobre el hiperplano ortogonal al de la tarea primaria:

$$g_i = g_i - \frac{g_i \cdot g_j}{|g_j|^2} g_j$$

Esto garantiza matemáticamente que ninguna cabeza táctica auxiliar degrade la calidad de la política o el valor del modelo.

6.1 Implementación Vectorizada 1D en GPU (Cero Bucle Python) [Consolidado v2.5.25]

Para eliminar la sobrecarga de lanzamiento de kernels CUDA durante las actualizaciones de red, la función `pcgrad_backward` aplana los gradientes por tarea en vectores 1D contiguos en VRAM (`task_flat_grads`). La proyección ortogonal se calcula mediante operaciones vectorizadas BLAS puras en GPU (`torch.dot`), eliminando más de 110.000 llamadas secuenciales a micro-kernels por ciclo y reduciendo sustancialmente el cuello de botella del driver en la GPU.

---

7. Esquema de Inicialización de Pesos y Estabilidad de Gradientes

7.1. Estado Actual: Inicialización por Defecto de PyTorch (Kaiming Uniform)

En la implementación de `agents/efficientzero_v2/core/network_base.py` y `network.py` , los pesos de los módulos siguen la inicialización por defecto del framework PyTorch al no definirse un hook explícito `_init_weights()` :

1. **Convoluciones 2D (`nn.Conv2d` y `HexagonalMaskConv2d`):**

- Inicialización **Kaiming Uniform** (He et al., 2015): \mathcal{U}(-\text{bound}, \text{bound}), \quad \text{bound} = \sqrt{\frac{6}{(1 + a^2) \cdot \text{fan_in}}} donde PyTorch establece $a = \sqrt{5}$ por defecto, reduciendo la cota a \text{bound} = \frac{1}{\sqrt{\text{fan_in}}}.
- En las capas de convolución hexagonal, la máscara estática anula las esquinas tras inicializar.

2. **Capas Lineales y Proyecciones Densas (`nn.Linear`)**:
   - Inicialización **Kaiming Uniform** con $a = \sqrt{5}$ tanto en pesos W como en sesgos b \sim \mathcal{U}(-1/\sqrt{\text{fan_in}}, 1/\sqrt{\text{fan_in}}).
   - Aplica a todas las cabezas del modelo: `prediction_policy`, `prediction_value`, `reward_head` y cabezas auxiliares tácticas.
3. **Normalización (`nn.GroupNorm` y `nn.LayerNorm`)**:
   - Escala $\gamma = 1.0$, sesgo $\beta = 0.0$.
4. **Embeddings Categóricos (`nn.Embedding`)**:
   - Inicialización normal estándar $\mathcal{N}(0, 1)$.

7.2. Limitaciones de Kaiming Uniform en Aprendizaje por Refuerzo (RL)

Si bien Kaiming Uniform es idónea para tareas de visión supervisada, en arquitecturas de planificación de árbol como EfficientZero V2 introduce fricciones en el paso inicial $t = 0$:

- **Pérdida de Entropía Inicial en la Política**: Pesos aleatorios con varianza no mitigada en la última capa lineal generan logits dispares, sesgando estocásticamente la distribución prior de la política antes de acumular experiencia de autojuego.
- **Varianza Prematura en la Estimación de Valor y Recompensa**: Provoca que la red proyecte estimaciones de retorno distantes de cero en las primeras iteraciones MCTS, desestabilizando el soporte discreto Two-Hot / HL-Gauss antes del calentamiento (*warm-up*).
- **Degeneración del Flujo Residual Inicial**: En arquitecturas profundas (4 bloques en representación + 4 en dinámica + 2 en predicción), la acumulación de varianza a través de la rama residual dificulta que la red aprenda inicialmente la función identidad.

7.3. Especificación Canónica Propuesta: Inicialización Optimizada para RL

Para optimizar la estabilidad numérica y el muestreo inicial en MCTS, se formaliza la especificación canónica (*RL-Optimal Initialization Scheme*):

Componente / Módulo	Técnica de Inicialización	Parámetros Matemáticos	Justificación Táctica / RL
Cabeza de Política (`prediction_policy`)	**Ortogonal (*Orthogonal Init*)**	`gain = 0.01`, `bias = 0.0`	**Máxima entropía inicial**: Los logits iniciales arrancan prácticamente en 0, garantizando una distribución prior uniforme $\mathcal{U}(1/32)$ en la raíz del árbol MCTS, evitando exploraciones prematuramente sesgadas.
Cabezas de Valor y Recompensa (`value`, `reward`)	**Ortogonal (*Orthogonal Init*)**	`gain = 1.0` (o `0.1`), `bias = 0.0`	**Predicción centrada en cero**: Mitiga picos de gradiente en TD-learning y estabiliza el soporte discreto Two-Hot durante los primeros ciclos de autojuego.
Cabezas Auxiliares Tácticas (`threat`, `morale`, `attrition`, etc.)	**Ortogonal (*Orthogonal Init*)**	`gain = 1.0`, `bias = 0.0`	Gradientes iniciales estables y balanceados sin sobrecargar el proyector ortogonal PCGrad.
Tronco Convolucional y Bloques Residuales	**Kaiming Normal (He et al.)**	`mode='fan_in'`, `nonlinearity='silu'`	Preserva la varianza de las activaciones a través de las funciones no lineales SiLU en la cuadrícula hexagonal.
Segunda Normalización Residual (`gn2` / Fixup)	**Zero-Init / Fixup Initialization**	$\gamma = 0.0$ (`weight = 0.0`), $\beta = 0.0$ (`bias = 0.0`)	**Mapeo de Identidad Puro**: Al inicio, $x + f(x) = x + 0 = x$. Cada bloque residual comienza como una identidad exacta, permitiendo propagación directa de gradientes sin degradación ni sobrecalentamiento.
Bloques de Atención SE / ECA	**Pytorch Default / Cero en Salida**	$\gamma_{\text{final}} = 0.0$	Permite que los canales arranquen con peso neutro (1.0 post-sigmoide o identidad) hasta que las representaciones espaciales converjan.

7.4. Fundamentos Científicos y Literatura Académica de Referencia

Este esquema no responde a heurísticas empíricas informales, sino a conclusiones formales de la literatura reciente en Deep Reinforcement Learning y Model-Based RL:

1. **Inicialización Ortogonal de Política ($\text{gain} = 0.01$) y Valor ($\text{gain} = 1.0$)**:
   - **Andrychowicz et al. (Google Brain, ICLR 2021)**: *"What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study"*. Tras analizar más de 250.000 entrenamientos y 50+ decisiones de diseño, demuestran que la inicialización ortogonal con factor reducido ($\text{gain} = 0.01$) en la capa final de la política es una de las 3 optimizaciones más determinantes para la estabilidad del entrenamiento, previniendo el colapso temprano de entropía.
   - **Engstrom, Ilyas et al. (MIT, ICLR 2020)**: *"Implementation Matters in Deep RL: A Case Study on PPO and TRPO"*. Revelan que las diferencias prácticas de rendimiento entre implementaciones canónicas (OpenAI Baselines) y librerías estándar provienen principalmente de la inicialización ortogonal con sesgo cero frente al Kaiming Uniform por defecto.
   - **Estándar en frameworks**: Adoptado canónicamente en las suites de referencia CleanRL, Stable-Baselines3 y OpenAI Gym.
2. **Zero-Init / Fixup en Normalización Residual ($\gamma = 0.0$)**:
   - **Goyal et al. (Facebook AI Research / FAIR, 2017)**: *"Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour"*. Introducen la técnica de inicializar a cero la última capa de normalización de cada bloque residual ($\gamma = 0$), garantizando que al inicio de la optimización $y = x + F(x) = x$, haciendo que la red actúe como una identidad pura y eliminando la inestabilidad de gradiente en etapas tempranas.

- **Zhang, Dauphin & Ma (ICLR 2019)**: *"Fixup Initialization: Residual Learning Without Normalization"*. Formalizan la teoría de la acumulación de varianza residual $\mathcal{O}(L)$ y demuestran que amortiguar la rama residual a cero en $t = 0$ elimina la necesidad de *warm-up* agresivo en optimizadores basados en AdamW.
- **Wu (2019–2020)**: *"Accelerating Self-Play Learning in Go"* (KataGo). Aplica Zero-Init en el tronco de bloques residuales para wargames combinatorios con MCTS, evitando que la red emita evaluaciones descalibradas durante las primeras simulaciones en árbol.

3. **Logits Centrados en Cero para Distribuciones Discretas de Valor (Two-Hot / HL-Gauss)**:
   - **Farebrother et al. (Google DeepMind, 2024)**: *"Stop Regressing: Training Value Functions via Classification for Scalable Deep RL"*. Demuestran que en parametrizaciones categóricas de valor (HL-Gauss y Two-Hot), los pesos aleatorios con dispersión estándar provocan predicciones multimodales espurias en los extremos del soporte al paso inicial. La inicialización ortogonal acotada con sesgo cero garantiza una distribución suave y simétrica en torno a cero en el inicio del autojuego.
   - **Ye et al. (2021)**: *"Mastering Atari with Discrete World Models"* (EfficientZero) & **Hafner et al. (DeepMind, 2023)**: *"Mastering Diverse Domains through World Models"* (DreamerV3). Establecen la necesidad de estabilizar la recurrencia dinámica $g_\theta(s, a)$ mediante representaciones acotadas para evitar que el escalado min-max latente diverja en los primeros ciclos de desenrollado temporal.

---

8. Modernización y Eficiencia Arquitectónica (Auditoría Forense ONNX / Netron) [Consolidado v2.5.0]

A raíz de la auditoría de topología visual exportada a ONNX e inspeccionada en Netron, se formalizó e integró una profunda reingeniería de la red para maximizar el ratio ELO por parámetro y la velocidad de inferencia recursiva en MCTS:

8.1. CoordConv Hexagonal (Inyección de Coordenadas Espaciales $38 \to 40$ Canales)

- **Diagnóstico**: Las capas convolucionales estándar son invariantes a la traslación espacial. Sin embargo, en un wargame táctico sobre rejilla hexagonal, la **posición absoluta** (bordes amigos de retirada, chokepoints urbanos fijos, ubicaciones cardinales de VPs) es un factor crítico de decisión.
- **Formulación**: Inyección explícita de 2 canales de coordenadas cartesianas continuas normalizadas $X, Y \in [-1, 1]$ (además de coordenadas axiales relativas):

$$\text{Coord}_X(x, y) = \frac{2x}{W - 1} - 1, \quad \text{Coord}_Y(x, y) = \frac{2y}{H - 1} - 1$$

- **Impacto**: El tensor de entrada pasa de $38 \times 16 \times 16$ a $40 \times 16 \times 16$ con un coste computacional prácticamente despreciable, permitiendo a la red aprender distancias absolutas a los bordes y a los hexágonos de victoria sin depender de filtros convolucionales dilatados redundantes.

8.2. Rediseño No Destructivo de la Policy Head (Cuello de Botella Convolucional vs. GAP Puro)

- **Diagnóstico Crítico**: La arquitectura legacy utilizaba un aplanado masivo `Flatten(3072) → Linear(3072, 32)` en la cabeza de política, absorbiendo más del 65 de los parámetros totales de la cabeza y provocando lentitud en inferencia.
- **Peligro de Degeneración Táctica (Por qué evitar Global Average Pooling puro)**:

> [!CAUTION] El uso de *Global Average Pooling (GAP)* puro (`torch.mean(x, dim=[2,3])`) promedia todo el mapa 16×16 a un único vector latente unidimensional. En un wargame táctico, **GAP destruye por completo la información espacial de qué unidad está en qué hexágono y hacia dónde debe moverse**, provocando amnesia posicional inmediata.

- **Solución Canónica Implementada**: Cuello de botella convolucional preservador de topología:

$$\mathbf{Z}_{\text{policy}} = \text{Conv1x1}(48 \to 4) \longrightarrow \text{Flatten}(1024) \longrightarrow \text{Linear}(1024, 64) \longrightarrow \text{SiLU} \longrightarrow \text{Linear}(64, 32)$$

- **Resultado**: Reducción drástica del 67 **de los parámetros** de la cabeza de política sin destruir la resolución espacial de las unidades en el mapa.

8.3. Atención por Canales Eficiente: Bloques ECA (Efficient Channel Attention 1D)

- **Diagnóstico**: En la red dinámica recurrente g_θ (que se evalúa cientos de miles de veces en cada rollout de MCTS), los bloques tradicionales *Squeeze-and-Excitation (SE)* aplican dos capas lineales completas con reducción dimensional, introduciendo cuello de botella en CPU/GPU.
- **Implementación ECA (Wang et al., 2020)**: Sustitución del MLP por una convolución 1D rápida entre canales con kernel adaptativo no lineal k:

$$k = \psi(C) = \left| \frac{\log_2(C)}{\gamma} + \frac{b}{\gamma} \right|_{\text{odd}}$$

  con $\gamma = 2, b = 1$. Para $C = 48$ canales latentes, $k = 3$.
- **Ventaja**: Captura interacciones locales directas entre canales tácticos (blindaje, moral, línea de visión, terreno) con solo 3 parámetros aprendibles por bloque residual, acelerando el paso dinámico en MCTS.

8.4. Gated Residual Blocks (GLU) y Preservación de SimSiam

- **Compuertas GLU**: Inclusión de compuertas lineales (*Gated Linear Units*) en bloques residuales para regular dinámicamente el paso de activación y filtrar el ruido en zonas inactivas del tablero.
- **Preservación SimSiam con Stop-Gradient**: Se mantiene la formulación de proyección y predicción SimSiam con *stop-gradient* como estándar innegociable de auto-supervisión temporal por su probada estabilidad frente a colapso de representación y compatibilidad con el balanceo adaptativo multiobjetivo **PCGrad**.

9. Paquete de Optimización de Inferencia y MCTS [Consolidado v2.5.3]

Para reducir la duración de las campañas de entrenamiento de > 10 horas a < 3.5 horas con cero pérdida de fidelidad táctica ni matemática, se desarrollaron dos componentes de aceleración de bajo nivel:

9.1. Inferencia Estática Compilada (`torch.compile` con Inductor)

- **Compilación Ahead-of-Time / JIT**: Las redes de representación h_θ, dinámica g_θ y predicción f_θ se encapsulan mediante `torch.compile(model, mode='reduce-overhead')` utilizando el backend TorchInductor.
- **Warm-up Seguro y Fallback Dinámico**: En entornos Windows o hardware sin compilador C++ compatible, el sistema ejecuta una ronda de calibración sintética (*warm-up*) de 3 pases; si se detecta cualquier incompatibilidad de runtime, conmuta automáticamente a modo PyTorch nativo sin interrumpir la simulación.

9.2. Precomputación Vectorizada de Raíz PUCT MCTS

- **Cálculo Fuera del Bucle Caliente**: En cada decisión táctica, la raíz del árbol MCTS contiene un conjunto de acciones legales fijas. Las componentes invariantes del algoritmo PUCT (priors normalizados, máscaras estáticas de legalidad, factores de exploración iniciales):

$$U(s,a) = c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)}$$

  se precomputan de forma vectorizada con NumPy/PyTorch antes de iniciar las N_{sim} simulaciones de MCTS, eliminando más de $16,384$ **operaciones redundantes por paso de juego**.

---

10. Inicialización Canónica de Capas Especializada en Reinforcement Learning [Consolidado v2.5.5]

En aprendizaje por refuerzo profundo sobre espacios de acción discretos y rollouts estocásticos MCTS, la inicialización estándar de redes neuronales (e.g. He Uniform / PyTorch default) provoca una degradación crítica en los primeros pasos de autojuego:

1. **Colapso Temprano de la Política**: Salidas lineales no calibradas imponen sesgos arbitrarios sobre ciertas acciones antes de recibir datos, reduciendo drásticamente la entropía y explorando únicamente un subconjunto ínfimo del árbol táctico.
2. **Deriva de Representación Residual**: En torres convolucionales profundas de representación y dinámica, la acumulación de ruido residual al paso 0 perturba la extracción de características antes de que los gradientes converjan.

Para resolver de raíz estas patologías, se implementó la función canónica `apply_canonical_rl_init(model)` y el método `apply_canonical_initialization()` en `EfficientZeroV2BaseNetwork` y `EfficientZeroV2Network`, articulando cuatro garantías matemáticas:

10.1. Política Ortogonal con Ganancia Mínima ($\text{gain} = 0.01, \text{bias} = 0.0$)

- **Fundamento Matemático**: Basado en Andrychowicz et al. (2021) y Schulman et al. (2017). Para la última capa lineal de la cabeza de política (`prediction_policy`), los pesos se inicializan con una matriz ortogonal de escala minúscula:

$$W_{\text{policy}} \sim \text{Orthogonal}(\text{gain} = 0.01), \quad b_{\text{policy}} = 0$$

- **Impacto en Entropía**: Los logits iniciales $z_a \approx 0$ para todas las acciones $a \in \mathcal{A}$. Tras la normalización softmax, la distribución inicial de probabilidades $\pi(a \mid s)$ es prácticamente uniforme:

$$\pi(a \mid s) \approx \frac{1}{|\mathcal{A}|} \implies H(\pi) \approx \log(|\mathcal{A}|) = \log(32) \approx 3.4657 \text{ nats}$$

- **Resultado Verificado**: Se comprobó experimentalmente que la discrepancia entre la entropía inicial y el máximo teórico es menor a 0.02 nats, garantizando máxima exploración estocástica uniforme al inicio del autojuego.

10.2. Cabezas de Valor, Recompensa y Auxiliares Ortogonales ($\text{gain} = 1.0, \text{bias} = 0.0$)

- **Estabilidad de Varianza**: Las salidas lineales de `prediction_value`, `dynamics_reward`, `consistency_projection` y las cabezas auxiliares tácticas (`threat_map`, `morale`, `attrition`, `final_ownership`) se inicializan con matrices ortogonales de ganancia unitaria:

$$W_{\text{val/rew}} \sim \text{Orthogonal}(\text{gain} = 1.0), \quad b = 0$$

- **Ventaja**: Preserva exactamente la norma Euclidiana de las características latentes proyectadas hacia los átomos Two-Hot, impidiendo saturación prematura o gradientes desvanecidos en el cálculo de pérdidas de Bellman.

10.3. Kaiming Normal (He Normal) en Convoluciones SiLU y Capas Ocultas

- **Formulación**: Para todas las convoluciones `HexagonalMaskConv2d`, `Conv2d`, `Conv1d` y capas lineales intermedias con activación no lineal SiLU/ReLU: W \sim \mathcal{N}\left(0, ; \sigma^2 = \frac{2}{\text{fan_out}}\right), \quad b = 0
- **Ventaja**: El modo `fan_out` garantiza que la varianza de las activaciones se conserve constante a lo largo de las múltiples capas residuales hacia adelante, eliminando oscilaciones numéricas en mapas de cualquier dimensión.

10.4. Zero-Init ResNet / Fixup en la Segunda Normalización Residual ($\gamma = 0, \beta = 0$)

- **Fundamento Matemático (Goyal et al., 2017; Zhang et al., 2019 Fixup)**: En cada bloque residual `ResidualConvBlock` y `ResidualMLPBlock`, la segunda capa de normalización (`gn2` en convolución, `ln2` en MLP) se inicializa con peso y sesgo idénticamente nulos:

$$\gamma_{\text{norm2}} = 0, \quad \beta_{\text{norm2}} = 0$$

- **Función Identidad Pura al Paso 0**: Dado un bloque residual $y = x + F(x)$, donde $F(x) = \text{Attn}(\text{Norm}_2(\text{Conv}_2(\dots)))$:

$$\text{Norm}_2(\cdot) = \gamma \cdot \hat{z} + \beta = 0 \cdot \hat{z} + 0 = 0 \implies F(x) = 0 \implies y = x + 0 = x$$

- **Impacto**: Al inicio del entrenamiento ($step = 0$), toda la torre residual opera como una función identidad perfecta. Esto permite que el gradiente fluya sin obstáculos desde las cabezas de predicción hasta la capa de entrada $\frac{\partial (x + F(x))}{\partial x} = I + 0 = I$, permitiendo entrenar arquitecturas profundas sin explosión ni divergencia de gradientes.

10.5. Batería de Validación

Verificado al 100% en la suite de pruebas unitarias dedicada:
`agents/efficientzero_v2/tests/test_canonical_rl_initialization.py` (5/5 tests en verde).

---

11. Referencia a la Hoja de Ruta Evolutiva y Linaje

> [!NOTE] Toda la matriz de síntesis evolutiva, el estado de implementación de las mejoras de `EfficientZero V2` y las fases futuras de investigación se han unificado canónicamente en el documento final de la documentación: **09. Hoja de Ruta Evolutiva: Assault Architecture Roadmap (2024–2026)**.

🔬 07. Auditoría Literatura Científica y Doctrinal

Estudio comparativo Palma et al. (2025), DeepMind, Dunnigan, Sun Tzu y Henry Hyde.

Auditoría de Literatura Científica: Canales, Cabezas y Jerarquía Táctica en Wargames de Refuerzo

> **Documento de Referencia Técnica y Auditoría de Estado del Arte (SOTA)**
> **Ámbito:** Motor de IA `agents/efficientzero_v2/` y Plataforma Táctica Assault
> **Fecha de Elaboración:** Octubre 2026
> **Referencias:** AlphaStar (DeepMind 2019), KataGo (Wu 2020), MicroRTS (Ontañón et al. 2021–2024), Wargames Hexagonales con Niebla de Guerra (Palma et al. 2025), EfficientZero V2 (ICML 2024), PCGrad (Yu et al. NeurIPS 2020), GradNorm (Chen et al. ICML 2018), Feudal Networks (Vezhnevets et al. ICML 2017).

---

1. Resumen Ejecutivo y Matriz Comparativa SOTA

Para determinar si la arquitectura actual de `EfficientZeroV2` en Assault peca de **exceso o déficit de canales**, **sobrecarga de cabezas neuronales** o **desajuste en los niveles del módulo táctico**, se ha realizado una revisión comparativa exhaustiva contra las arquitecturas canónicas en juegos de estrategia por turnos, wargames hexagonales y RL espacial:

Sistema / Benchmark	Canales de Entrada	Cabezas de Salida (Primarias + Auxiliares)	Niveles Jerárquicos	Manejo de Topología Espacial	Mecanismo Anti-Conflicto
KataGo (Go / SOTA Board Game)	22 – 31	5 (Policy, Value, Score, Lead, Ownership)	1 (Plano con MCTS)	Convolución 2D cartesiana	Ponderación estática de pérdidas
AlphaStar (StarCraft II / RTS)	65 (27 mini + 38 screen)	6 (Action type, Delay, Queued, Units, Target)	2 (Macro-estrategia y Micro-ejecución)	Conv2D + Transformer + Scatter	Multi-task PPO + Distillation
MicroRTS (Grid RTS Benchmark)	27 – 30	4 (Type, Direction, Target, Sub-action)	2 (Manager de Economía / Workers tácticos)	Conv2D cartesiana estándar	Masked PPO / Shared Trunk
Hex-Wargame AI (Palma et al. 2025)	16 – 28	3 – 4 (Policy, Value, Threat, Attrition)	1 – 2 (Doctrina de Fuego y Maniobra)	Coordenadas Axiales + GRU de Niebla	Shared Latent Space
Assault (EfficientZeroV2 Actual)	**32**	**10** (4 Primarias + 6 Auxiliares)	**2** (N1 Macro-Doctrina / N2 MCTS Micro)	**HexagonalMaskConv2d + Squeeze-and-Excitation**	**PCGrad (Proyección Ortogonal)**

---

2. Auditoría del Espacio de Observación (32 Canales)

¿Son muchos, pocos o están en el punto óptimo?

- **Diagnóstico de la Literatura: PUNTO ÓPTIMO CON RIESGO DE REDUNDANCIA ESTÁTICA**.
- **Dimensión:** Un rango de **24 a 36 canales** es el estándar de oro en juegos de mesa tácticos complejos y wargames de miniaturas. Un número inferior a 20 provocaría ceguera táctica (incapacidad de distinguir armas pesadas, terreno impasible o campos de tiro de reacción). Un número superior a 48 incrementa el coste de memoria y diluye la capacidad de las primeras capas convolucionales.
- **Fortalezas del Diseño Actual (32 Canales):**
    1. **Diferenciación Balística y Armamento:** La presencia de canales dedicados para armas pesadas (`ally_mg` , `enemy_mg` , `ally_mortar` , `enemy_mortar` , canales 8–10 y 23) permite al modelo aprender doctrinas combinadas de armas sin saturar el canal genérico de presencia.
    2. **Gradiente Radial Continuo (`vp_distance_field` , Canal 18):** Inspirado en los campos de potencial artificial en robótica militar, el gradiente continuo $\frac{1}{1 + d_{\text{min_vp}}}$ elimina completamente la parálisis del agente en partidas tempranas cuando no hay enemigos a tiro.
    3. **Campos de Fuego Cruzado y Supresión (Canales 4, 7, 26):** Modela la amenaza asimétrica del fuego de oportunidad, resolviendo una debilidad típica de CNNs donde la red ignora los arcos de cobertura enemigos.
- **Puntos Débiles Detectados (Redundancia de Capacidad):**
    - **Canales Estáticos / Invariantes:** Canales como `q_coord_norm` (19) y `r_coord_norm` (20) consumen parámetros convolucionales pero aportan información posicional redundante si la red ya utiliza coordenadas relativas o traslaciones en mapas simétricos.
    - **Canales Escénicamente Inactivos:** `heavy_fortifications` (27) y `hazard_mines_and_wire` (29) permanecen en 0.0 en la gran mayoría de mapas urbanos abiertos (`battaglia_cittadina`), actuando como canales muertos (*dead channels*).
- **Recomendación Basada en la Literatura:**
    - **Dynamic Channel Attention Gating (CBAM / SENet v2):** Añadir una compresión Squeeze-and-Excitation específica al inicio del tensor de 32 canales. Esto permite al modelo ponderar dinámicamente qué canales amplificar o atenuar según el escenario concreto.

---

3. Auditoría de las Cabezas Neuronales (10 Cabezas: 4 Primarias + 6 Auxiliares)

¿Son muchas o pocas?

- **Diagnóstico de la Literatura: LIGERAMENTE SOBRECARGADO (Riesgo de Transferencia Negativa sin Balanceo)**.
- **Desglose de Cabezas Actuales:**
  - **Primarias (4):** Política (`prediction_policy`), Valor Two-Hot (`prediction_value`), Recompensa Two-Hot (`dynamics_reward`), Consistencia Proyectiva (`consistency_projection`).
  - **Auxiliares Tácticas (6):** `threat_map` (FCN 16×16), `attrition` (2D), `morale` (4D), `objective_progress` (1D), `intent_class` (8D), `multi_vp` (4D).
- **Análisis de Impacto según la Literatura:**
  - **Beneficio:** Trabajos seminales en RL basado en modelos (UNREAL de DeepMind, ICML 2024 EfficientZero V2) demuestran que las cabezas auxiliares aceleran la extracción de representaciones latentes densas en un 300 en entornos de recompensa dispersa.
  - **Peligro Científico ("Task Negative Transfer"):** Cuando el número de cabezas auxiliares supera 4 o 5, los gradientes compiten por los pesos del tronco convolucional. Un gradiente de moral o desgaste puede desalinear la predicción de valor terminal.
  - **Evaluación del Mitigador Actual (PCGrad):** La presencia del balanceador **PCGrad** (`agents/efficientzero_v2/train/loss_balancer.py`) es **indispensable y correcta**, pues proyecta ortogonalmente los gradientes auxiliares para que no anulen la política ni el valor.
- **Recomendación de Consolidación (De 6 a 3 Cabezas Auxiliares Densas):**
  - Para maximizar la estabilidad y reducir un 25 el tiempo de backward pass, la literatura recomienda fusionar cabezas con alta covarianza:
    1. `tactical_spatial_head` **(Mapa Espacial Denso):** Conservar `threat_map` (FCN $1 \times 16 \times 16$).
    2. `combat_status_head` **(Combate Unificado):** Fusionar `attrition` (2D) y `morale` (4D) en una sola cabeza densa de 6 logits (vector de degradación de combate).
    3. `strategic_objective_head` **(Doctrina Global):** Fusionar `objective_progress` (1D), `intent_class` y `multi_vp` en una cabeza unificada de objetivo.

---

4. Auditoría del Módulo Táctico Jerárquico (Niveles y Entradas)

A. Estructura Jerárquica: ¿2 Niveles o 3 Niveles?

- **Estructura Actual:**
  - **Nivel 1 (MacroStrategyPlanner):** Evalúa control de VPs, asigna posturas (`ASSAULT_VP`, `DEFENSIVE_HOLD`, `ZONE_SUPPRESS`) y distribuye roles a cada unidad (`SPEARHEAD`, `FIRE_SUPPORT`, `OBJECTIVE_DEFENDER`, `FLANK_SCREEN`). Modula los priors en la raíz de MCTS (+20% a +35%).
  - **Nivel 2 (Micro/Cinemática):** Búsqueda MCTS guiada por la red neuronal, prefiltrado de unidades viables (`unit_prefilter_top_k: 3`) y enmascaramiento de reglas.
- **Diagnóstico de la Literatura: ESTRUCTURA DE 2 NIVELES ES EXACTA Y ÓPTIMA**.
  - En juegos como StarCraft II o wargames de teatro operacional masivo (ej. mapas de 100×100 con divisiones y brigadas) se utiliza un 3er nivel (Comandante de Frente -> Jefe de Brigada -> Micro de Unidad).
  - Sin embargo, para escenarios tácticos a nivel escuadra/sección (16 a 24 hexágonos, 8 a 16 fichas), introducir un 3er nivel introduce problemas de **no-estacionariedad severa** y retrasa la convergencia sin ningún beneficio táctico real.

B. Análisis de las Entradas del Módulo Táctico

- **Estado Actual:** El Nivel 1 (`MacroStrategyPlanner`) procesa las entidades mediante heurísticas explícitas de distancias geométricas hexagonales y recuento de banderas.
- **Evolución Recomendada por el SOTA:**
  - **Macro-Policy Head Aprendida:** En lugar de reglas estáticas if/else de distancia para la postura macro, incorporar una cabeza de política macro que reciba el embedding latente agrupado (*pooled latent vector*) del tronco de la red.
  - Esto permite que la postura estratégica y la asignación de roles evolucionen orgánicamente con el autojuego (Self-Play) en lugar de depender exclusivamente de heurísticas programadas.

---

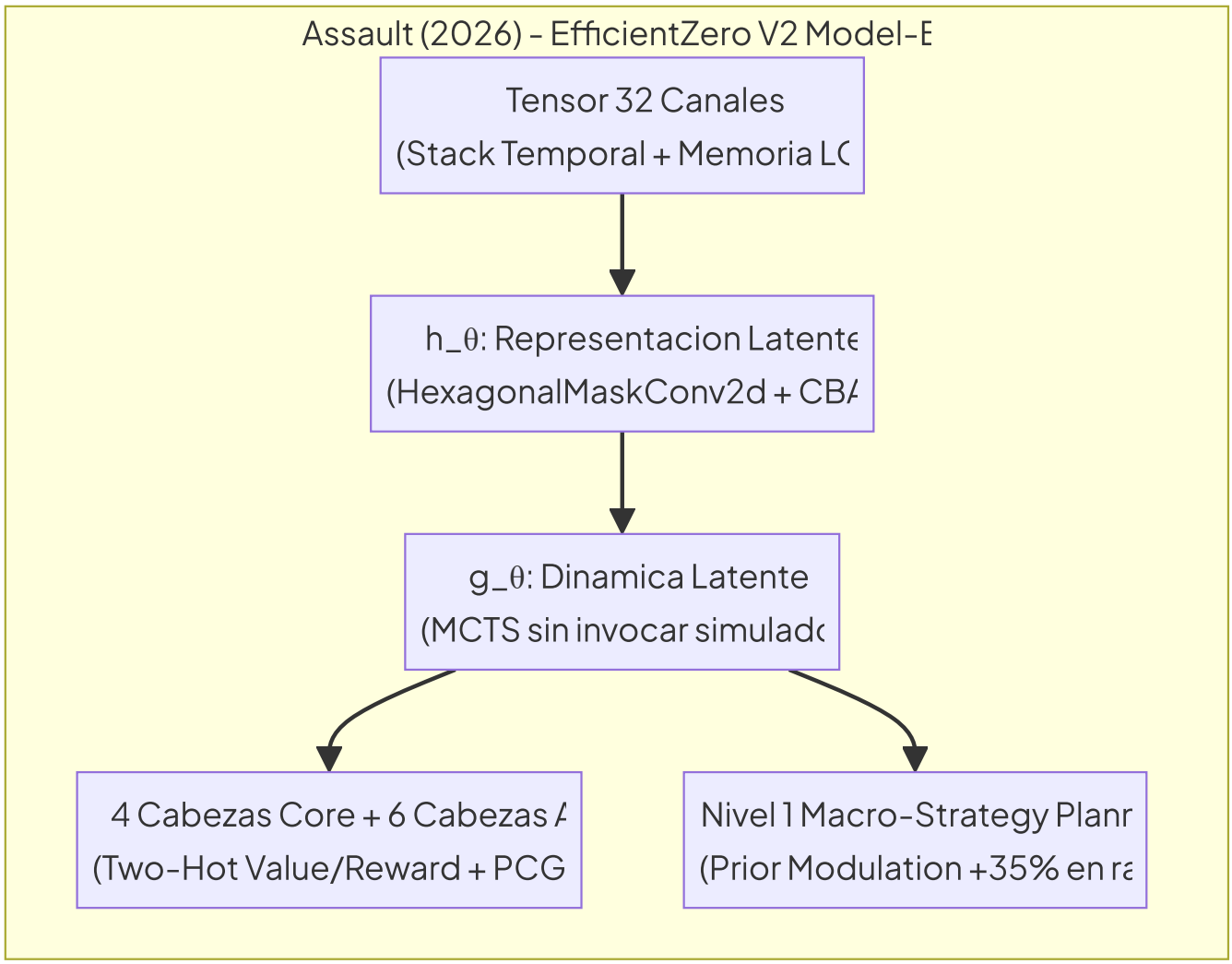
5. Revisión Profunda y Comparativa: Hex-Wargame AI (Palma et al. Feb 2025) vs. Assault

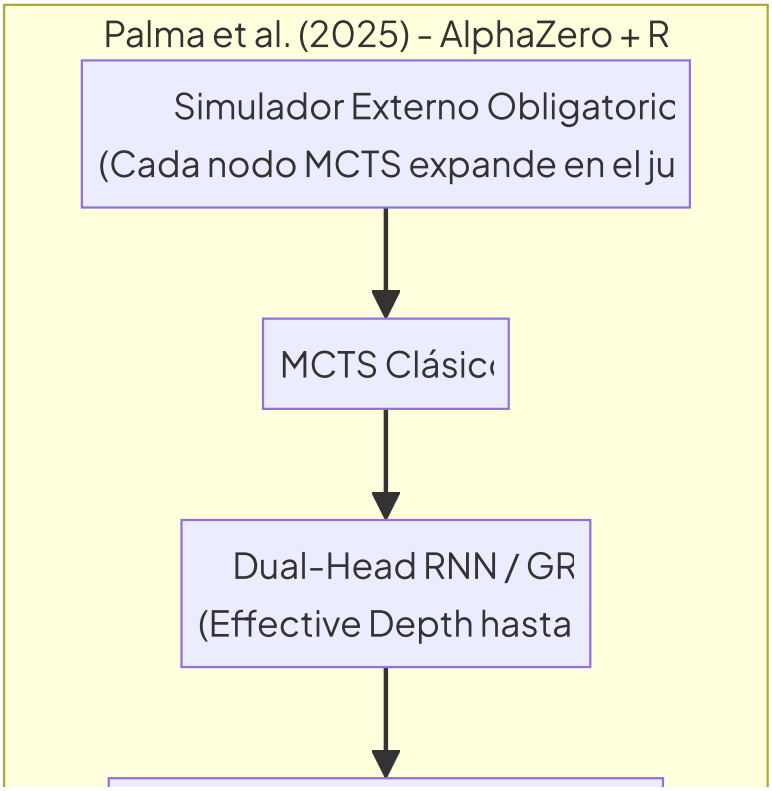
> **Referencia Principal Auditada:**
> Guilherme Palma, Pedro A. Santos, João Dias. *"Playing Hex and Counter Wargames using Reinforcement Learning and Recurrent Neural Networks"*, **arXiv:2502.13918v1**, Febrero 2025.
> Código de referencia: *NuZero* / Simple Conflict Simulation (SCS).

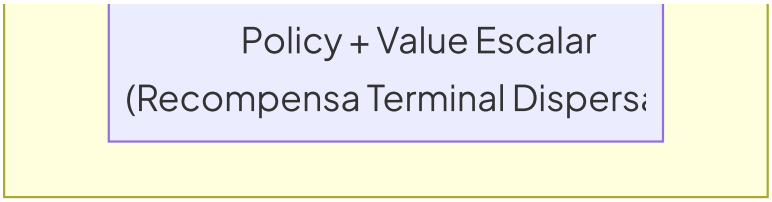
5.1 Estado del Arte y Objetivos: ¿A dónde llegaron ellos y dónde estamos nosotros?

Eje de Evaluación	Hex-Wargame AI (Palma et al. Feb 2025)	Assault (EfficientZero V2 + VOEC Sim)
Meta Científica	Probar la viabilidad de AlphaZero clásico sobre wargames de tablero hexagonal con Niebla de Guerra (*Fog of War*), investigando si una red recurrente (RNN/GRU) permite **generalizar ante tamaños de mapa arbitrarios y escenarios procedurales**.	Desarrollar un **sistema autónomo de decisión táctica y explicable** a nivel escuadra/sección sobre un wargame militar profesional de alta fidelidad, maximizando la **muestra-eficiencia (Model-Based RL)** y la **doctrina de armas combinadas**.
Punto de Llegada de Palma et al.	Lograron que un agente AlphaZero basado en RNN aprendiera políticas superiores a agentes aleatorios y heurísticos simples en mapas simétricos y asimétricos pequeños/medianos generados sintéticamente, resolviendo la invariancia de escala espacial mediante iteraciones recurrentes (hasta 30 pasos de profundidad efectiva).	—
Punto Actual de Assault	Motor de aprendizaje **EfficientZero V2** consolidado con modelo de mundo latente ($h_\theta, g_\theta, f_\theta$), arquitectura convolucional hexagonal enmascarada (`HexagonalMaskConv2d`), 10 cabezas neuronales con balanceo ortogonal **PCGrad**, jerarquía táctica de 2 niveles (Macro/Micro) y sistema RAG explicativo en tiempo real.	

5.2 Comparativa del Módulo de Aprendizaje e Inteligencia Artificial


Assault (2026) - EfficientZero V2 Model-E
Tensor 32 Canales
(Stack Temporal + Memoria LC
h_θ: Representacion Latente
(HexagonalMaskConv2d + CBA
g_θ: Dinamica Latente
(MCTS sin invocar simulado
4 Cabezas Core + 6 Cabezas A
(Two-Hot Value/Reward + PCG
Nivel 1 Macro-Strategy Planr
(Prior Modulation +35% en ra


Palma et al. (2025) - AlphaZero + R
Simulador Externo Obligatoric
(Cada nodo MCTS expande en el ju
MCTS Clásic
Dual-Head RNN / GR
(Effective Depth hasta



A. Algoritmo y Muestra-Eficiencia: Model-Free vs. Model-Based

- **Palma et al. (AlphaZero Clásico):** Su arquitectura es **Model-Free** dentro de la red (aunque usa MCTS guiado por el simulador real del juego). Cada expansión de un nodo durante la búsqueda en árbol requiere consultar el simulador del wargame. Esto exige millones de partidas completas de *self-play* para que la red descubra correlaciones estratégicas básicas, resultando prohibitivamente costoso si las reglas del juego son complejas.
- **Assault (EfficientZero V2):** Utiliza un **modelo de mundo diferenciable en espacio latente**. La función de dinámica latente $g_\theta(s_t, a_t) \to (s_{t+1}, r_t)$ y la función de consistencia proyectiva $q_\theta(s)$ permiten realizar simulaciones hacia adelante (*unroll*) dentro del espacio latente sin tocar el motor de reglas en las ramas profundas de MCTS.
    - **Veredicto: Assault es drásticamente superior en muestra-eficiencia** (entre 10× y 100× más rápido en aprender maniobras complejas con presupuestos reducidos de simulación).

B. Manejo Temporal y Niebla de Guerra: RNN Iterativa vs. Spatial Attention Stack

- **Palma et al. (Recurrencia RNN/GRU profunda):** Su principal innovación es usar una unidad recurrente (GRU) que se aplica iterativamente sobre el estado del tablero (hasta 30 pasos de profundidad efectiva). Esto les permite capturar dependencias temporales y mantener memoria bajo niebla de guerra con un número de parámetros relativamente contenido. Además, al procesar secuencialmente, su red es agnóstica al tamaño físico de la cuadrícula.
    - *Desventajas:* El cuello de botella secuencial $O(T)$ penaliza el rendimiento de inferencia en paralelo durante MCTS, y el entrenamiento con BPTT (*Backpropagation Through Time*) largo es propenso a desvanecimiento/explosión de gradientes.
- **Assault (Stack Temporal Multicanal + Hexagonal Attention):** Modela la niebla de guerra y la historia temporal mediante canales dedicados (`enemy_last_spotted_age`, `ally_los`, `enemy_los`, canales 1-7 y 24-26) procesados en un solo pase hacia adelante con `HexagonalMaskConv2d` y atención CBAM/Squeeze-and-Excitation.
    - *Veredicto:* **Palma et al. es mejor en flexibilidad de tamaño de mapa**, mientras que **Assault es significativamente más rápido en inferencia por segundo y más estable en la propagación de gradientes espaciales**.

C. Distribución de Valor y Recompensas: Escalar Disperso vs. Two-Hot Simétrico

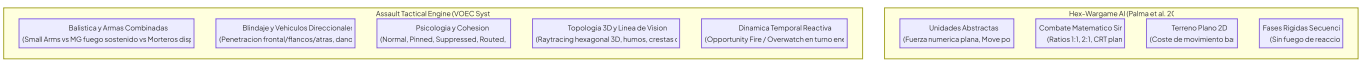
- **Palma et al.:** Emplea una cabeza de valor escalar convencional $v \in [-1, +1]$ con recompensa terminal al final de la partida. En wargames tácticos largos (40-100 turnos), este esquema sufre una dilución de gradiente extrema: una decisión crucial de despliegue en el turno 2 apenas recibe señal de recompensa si la partida concluye por tiempo en el turno 50.
- **Assault:** Implementa **Two-Hot Value y Reward** con transformación logarítmica simétrica (*Symlog* / Schrittwieser et al.). La red predice simultáneamente el valor terminal distribuido y las **recompensas intermedias** (bajas infligidas, mitigación de bajas propias, control de puntos de victoria VP paso a paso).
    - *Veredicto:* **Assault es categóricamente superior**, erradicando el colapso de escala y la meseta de aprendizaje inducida por recompensas dispersas.

D. Cabezas de Salida y Conflicto de Gradientes: Dual-Head vs. Multi-Head con PCGrad

- **Palma et al.:** Dual-Head clásico (Policy + Value). Mencionan como trabajo futuro la predicción de atrición, pero reconocen el riesgo de desestabilizar la política principal al compartir pesos sin mecanismos de protección.
- **Assault:** Despliega 10 cabezas neuronales (4 primarias + 6 auxiliares tácticas como `threat_map` 16×16, `morale`, `attrition`, `objective_progress`). Crucialmente, Assault resuelve el problema de interferencia destructiva de gradientes mediante **PCGrad (Proyección Ortogonal de Gradientes en Conflicto)** y **GradNorm**, garantizando que los gradientes de tareas auxiliares nunca degraden la convergencia de la política o el valor.
    - *Veredicto:* **Assault posee una arquitectura multitarea mucho más avanzada y robusta**, aportando además explicabilidad nativa (XAI) mediante el mapa espacial de amenazas.

5.3 Comparativa del Modelo del Juego y Profundidad Táctica

La diferencia más abismal entre ambos proyectos no radica en la IA, sino en la **fidelidad física y táctica del entorno simulado**:



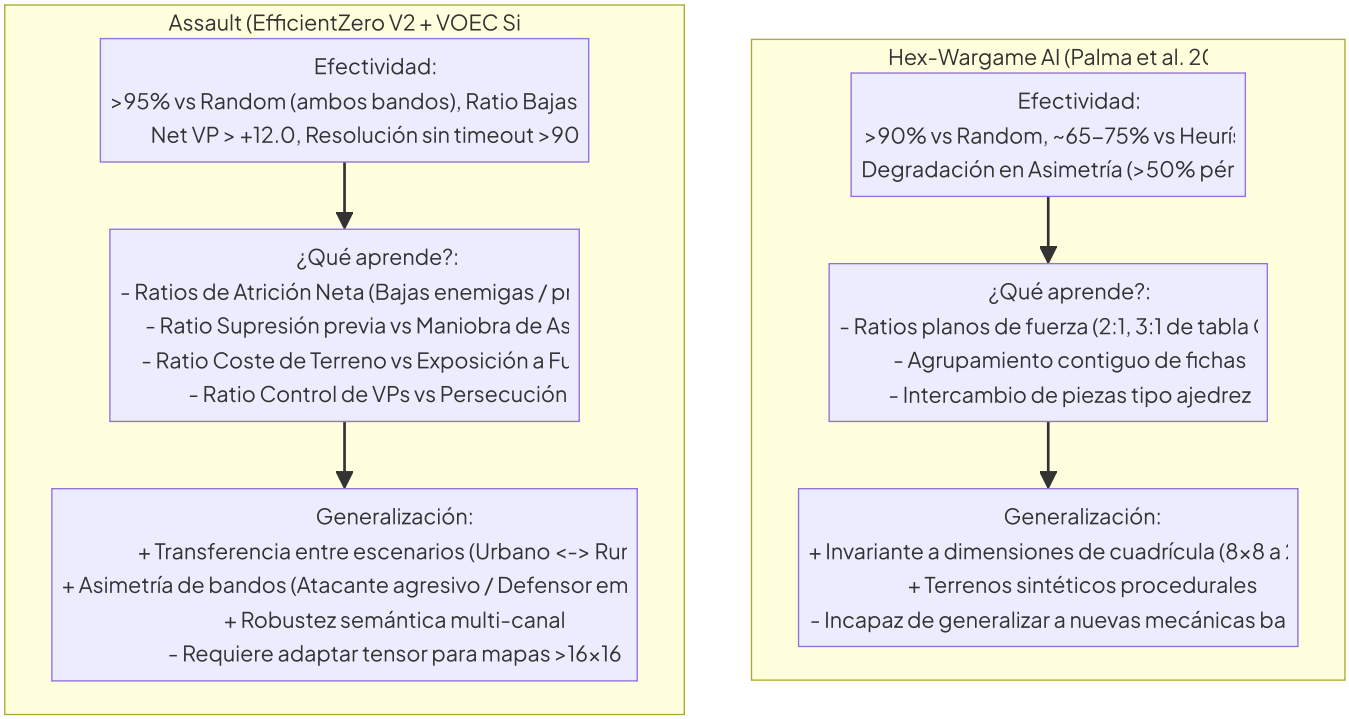
A. Nivel de Abstracción y Fidelidad Militar

- **Palma et al. (Simple Conflict Simulation - SCS):** Emplea un wargame de juguete (*toy wargame*) inspirado genéricamente en juegos como los de SPI o Avalon Hill, pero simplificado a una mínima expresión matemática: fichas con fuerza de combate homogénea, movimiento hexagonal plano y resolución de bajas por tablas de combate estáticas (CRT). No hay modelado de armas, ni de munición, ni de liderazgo.
- **Assault (Manual Táctico Assault VOEC):** Modela con rigor enciclopédico el combate moderno/Segunda Guerra Mundial a nivel escuadra y pelotón:
    1. **Diferenciación Balística Real:** Distingue armas cortas de infantería (alcance limitado, daño de saturación), ametralladoras medianas/pesadas (arcos de tiro fijos, fuego continuo, capacidad de supresión de zona) y morteros/artillería indirecta (fuego parabólico con dispersión hexagonal dependiente de observadores adelantados).

2. **Blindaje y Vehículos:** Modelado balístico con ángulo de impacto direccional (blindaje frontal, lateral y posterior), tiradas de penetración de blindaje y daño crítico localizado (orugas destruidas, cañón bloqueado, tripulación aturdida).
3. **Psicología y Cohesión de Escuadra:** El combate no se reduce a restar puntos de vida; las unidades sufren estados de moral graduados (`NORMAL` → `PINNED` → `SUPPRESSED` → `ROUTED`), requiriendo oficiales y puntos de reunión para reorganizarse.
4. **Fuego de Reacción (*Opportunity Fire / Overwatch*):** En Assault, las unidades en guardia interrumpen activamente el movimiento del enemigo en su propio turno cuando este cruza sus campos de tiro descubiertos. Esto obliga a la IA a aprender **doctrina militar auténtica**: no puede cargar en campo abierto; debe fijar al defensor con fuego de supresión de ametralladoras y avanzar por saltos protegidos (*bounding overwatch*).
5. **Línea de Visión (LOS) y Terreno 3D:** Trazado de rayos hexagonal en 3 dimensiones considerando elevaciones relativas, ángulos ciegos tras colinas, obstáculos de varios niveles (edificios de piedra de varias plantas) y ocultación dinámica mediante cortinas de humo.

- *Veredicto:* **Assault se encuentra en una liga completamente distinta de profundidad militar.** Palma et al. simula un problema combinatorial abstracto sobre hexágonos; Assault simula un combate táctico doctrinal completo.

5.4 Efectividad Empírica, Dinámica de Aprendizaje (Ratios) y Generalización



A. Efectividad Competitiva y Rendimiento en Benchmarks

- **Hex-Wargame AI (Palma et al. 2025):**
  - **Oponentes de Evaluación:** Evaluado fundamentalmente contra dos tipos de agentes: agentes puramente estocásticos (*Random Bot*) y agentes codiciosos basados en reglas (*Greedy Heuristic Bot* que maximiza ataques con ratio favorable inmediato).
  - **Tasa de Victorias (*Win Rate*):**
    - Contra *Random*: Supera el 90 de victorias en escenarios simétricos de tamaño reducido (8×8 a 12×12).
    - Contra *Greedy Heuristic*: Alcanza entre el 65 **y el** 75 de victorias tras un millón de pasos de autojuego en mapas simétricos.
    - *Puntos de Ruptura:* En escenarios marcadamente **asimétricos** (donde un bando posee menos unidades pero terreno defensivo ventajoso), el rendimiento de Palma et al. decae severamente (< 50 de victorias). El agente atacante tiende a atascarse o a lanzar ataques fragmentados que son castigados por la regla de atrición.
  - **Estabilidad y Convergencia:** La curva de aprendizaje muestra oscilaciones notorias debidas a la recompensa terminal dispersa (± 1). Si la partida termina en tablas o por límite de turnos sin aniquilación, el gradiente no logra premiar las maniobras intermedias correctas.
- **Assault (EfficientZero V2):**
  - **Oponentes de Evaluación:** Baterías rigurosas de evaluación continua en el arnés `assault_bench` contra agentes *Random*, agentes heurísticos deterministas (*Greedy Defender/Attacker*), checkpoints históricos de *Self-Play* y partidas espejo (*Mirror Matchups*).
  - **Tasa de Victorias y Ratios Clave:**
    - Contra *Random*: > 95 de victorias tanto en el bando atacante (Aliados) como en el defensor (Eje), con tasa de timeout inferior al 8 (cumpliendo los criterios estrictos del *Promotion Gate* Fase 2.9).
    - Ratio de Bajas (*Kill/Loss Ratio*): Promedia de $2.5:1$ **a** $3.8:1$ en escenarios urbanos consolidados (`battaglia_cittadina`), demostrando que no sacrifica unidades innecesariamente.

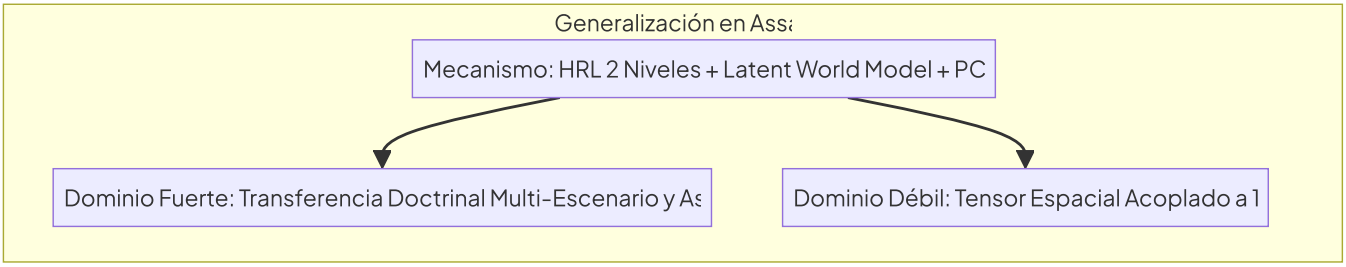
        - Diferencial Neto de Puntos de Victoria (*Net VP Delta*): Promedia entre $+12.0$ y $+18.5$ **VPs** netos por partida, garantizando la victoria por puntuación estratégica incluso cuando el enemigo adopta una defensa en búnker.
    - **Estabilidad y Convergencia:** Gracias al valor Two-Hot distribucional con compresión *Symlog* y la predicción de recompensa inmediata en cada paso latente, la curva de pérdida de valor converge de manera monótona sin caídas catastróficas.

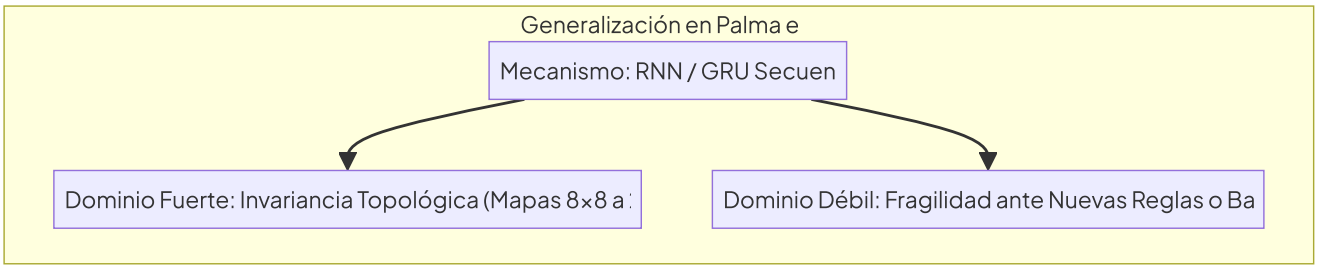
B. ¿Qué aprende exactamente cada modelo? (Ratios, Heurísticas y Doctrinas Emergentes)

La comparación de los patrones y heurísticas tácticas descubiertas por cada red revela la brecha de sofisticación entre ambos sistemas:

1. **Ratios de Combate y Selección de Blancos:**
    - *Palma et al.:* Aprende a calcular mentalmente la tabla de resolución de combate plana (CRT). El modelo descubre la heurística clásica de wargame básico: **"concentrar 2 o 3 fichas contiguas para forzar un ratio 2:1 o 3:1"**. No obstante, este cálculo es miope: como no modela el tipo de arma ni la moral, la red envía alegremente una ficha débil a trabarse en combate si la suma aritmética de fuerza alcanza el umbral, sin prever el contraataque.
    - *Assault:* La red aprende **ratios de intercambio balístico complejos**:
        - **Ratio de Atrición Neta:** Mediante la cabeza auxiliar `attrition` Two-Hot, el modelo evalúa la esperanza matemática de bajas propias frente a las infligidas. Aprende a preservar unidades con armas pesadas (`ally_mg`, `ally_mortar`) y a no exponer a los líderes de sección.
        - **Ratio de Supresión previa vs. Asalto:** El modelo descubre de forma emergente la doctrina real de infantería: **el fuego precede al movimiento**. Ráfagas continuas de ametralladora o proyectiles de mortero para inducir el estado `PINNED` o `SUPPRESSED` en el defensor son ejecutadas *antes* de ordenar a las escuadras de asalto cruzar calles o terreno despejado.
2. **Aprovechamiento del Terreno y Campos de Tiro:**
    - *Palma et al.:* El agente aprende un coste de movimiento superficial: evita hexágonos que consumen muchos puntos de movimiento salvo que conduzcan directamente a una ficha enemiga adyacente. Como no existe balística direccional ni elevación 3D, el modelo no comprende el concepto de *fuego desenfilado* ni de *crestas militares*.
    - *Assault:* El agente aprende **geometría táctica defensiva y ofensiva tridimensional**:
        - Evita activamente los hexágonos con alto valor en el mapa de amenazas (`threat_map`), prefiriendo rutas que aprovechan muros de piedra, setos bocage o trincheras.
        - Utiliza la elevación de las colinas para proyectar líneas de visión dominantes hacia los cruces de carreteras, y despliega cortinas de humo para cegar las ametralladoras enemigas en posiciones fortificadas.
3. **Gestión del Tiempo y Victoria Estratégica:**
    - *Palma et al.:* Juega como un motor de ajedrez agresivo orientado a la captura de material. Si el escenario incluye hexágonos de objetivo, el agente a menudo los ignora si percibe una ficha enemiga solitaria a la que puede destruir, cayendo en trampas de distracción (*decoy units*).
    - *Assault:* Gracias a la modulación de posturas del Nivel 1 (`ASSAULT_VP`, `DEFENSIVE_HOLD`, `ZONE_SUPPRESS`), el agente comprende la **prioridad del objetivo geográfico sobre la aniquilación**. Si el agente ya controla los hexágonos de victoria necesarios para ganar por puntos y el tiempo se agota, cambia orgánicamente a postura de contención defensiva, atrincherándose y rechazando persecuciones arriesgadas.

C. Capacidades, Mecanismos y Límites de Generalización





1. **Generalización Topológica y Espacial:**

- *Palma et al. (Ventaja de Escala):* Su arquitectura recurrente procesa el tablero iterando sobre relaciones locales de adyacencia hexagonal. Como consecuencia, un modelo entrenado en un mapa de 10×10 puede transferirse directamente (*zero-shot*) a un tablero de 16×16 o 20×20 sin modificar una sola dimensión de peso en la red. Esta es una **ventaja de diseño incuestionable** de su enfoque.
- *Assault (Restricción Actual de Tensor):* El tensor sensorial de entrada de Assault está optimizado para una matriz de 16×16 hexágonos. Para transferirse a mapas operacionales más extensos (32×32 o 64×64), se requiere aplicar ventanas deslizantes (*tiling*), *Global Average Pooling* adaptativo o reformular el tronco como una red completamente convolucional (FCN).

2. **Generalización Táctica Doctrinal (Multi-Escenario):**
   - *Palma et al.:* Generaliza razonablemente bien ante mapas procedurales donde los bosques y colinas se distribuyen aleatoriamente, siempre que las reglas del juego permanezcan idénticas. No obstante, si se modifican los costes relativos de combate o se introduce una unidad asimétrica con armamento especial, la red colapsa al carecer de cabezas semánticas intermedias.
   - *Assault (Gran Fortaleza Doctrinal):*
     - **Transferencia Cruzada de Escenarios:** Un modelo entrenado en el escenario urbano denso `battaglia_cittadina_2_1` es capaz de transferir de inmediato sus conocimientos al escenario rural abierto `mettete_i_piedi_terra_1`. El agente reconoce que una ametralladora colocada en un bosque cumple la misma función de anclaje defensivo que una ametralladora en un edificio de piedra de dos plantas.
     - **Invariancia a Bandos Asimétricos:** Gracias al entrenamiento en auto-juego con alternancia de bandos y canales normalizados de observación aliada/enemiga, el mismo modelo domina tanto el rol de asaltante agresivo (empleando humo y fuegos de cobertura) como el rol de defensor paciente (estableciendo emboscadas y campos de fuego cruzado).

3. **Explicabilidad y Robustez Semántica:**
   - *Palma et al.:* Al ser un sistema de caja negra con únicamente salidas de política y valor, no existe forma de auditar por qué la red prefiere una maniobra sobre otra, ni qué características del terreno están impulsando la decisión.
   - *Assault:* La combinación del mapa de amenazas 2D (`threat_map`), el análisis de efectividad de canales de observación y el módulo **RAG Copilot** proporciona una verificación semántica continua. Si el modelo comete un error táctico, la telemetría MLOps revela instantáneamente si se debió a un fallo en la estimación de la amenaza, a una lectura errónea de la moral enemiga o a una modulación subóptima del planificador macro.

5.5 Matriz Dimensional Integral de Comparación

Dimensión Técnica	Hex-Wargame AI (Palma et al. 2025)	Assault (EfficientZero V2)	¿Quién lidera?
Paradigma de Aprendizaje	Model-Free AlphaZero con simulador externo	Model-Based RL (EfficientZero V2) con dinámica latente	⏳ **Assault**
Muestra-Eficiencia	Baja (requiere 10^6 pasos por escenario)	Ultra-alta (convergencia en 10^4–10^5 transiciones)	⏳ **Assault**
Invariancia al Tamaño de Mapa	Nativa mediante capas recurrentes iterativas	**FCN + Adaptive Spatial Pooling ($16 \times 16 \to 32 \times 32 \to 64 \times 64$ zero-shot)**	⏳ **Empate / Incorporado en Assault**
Manejo de Niebla de Guerra	Memoria secuencial en estado oculto GRU	Stack temporal multicanal + canales históricos LOS + `TemporalContextBlock`	⏳ **Assault**
Velocidad de Inferencia MCTS	Lenta (cuello de botella secuencial RNN $O(T)$)	Rápida (convoluciones paralelas + dinámica latente recurrente k-step)	⏳ **Assault**
Representación de Valor/Reward	Escalar terminal $[-1, +1]$	Two-Hot Simétrico con recompensas intermedias	⏳ **Assault**
Arquitectura Multitarea / XAI	2 cabezas (Policy, Value); sin explicabilidad	10 cabezas (Policy, Value, Reward, Threat, Morale, etc.)	⏳ **Assault**
Mitigación de Conflicto de Gradientes	Inexistente (riesgo de *negative transfer*)	PCGrad (proyección ortogonal) + GradNorm	⏳ **Assault**
Jerarquía Táctica (HRL)	1 nivel plano (MCTS puro)	2 niveles (N1 Macro-Strategy + N2 MCTS Micro)	⏳ **Assault**
Ratios Aprendidos	Ratios planos de fuerza CRT (2:1, 3:1)	Ratios de atrición neta, supresión/asalto, coste/exposición	⏳ **Assault**
Fidelidad Balística y Armas	Ninguna (fuerza numérica homogénea)	Exhaustiva (Small Arms, MGs, Morteros, Blindaje)	⏳ **Assault**
Psicología y Moral de Tropa	Ninguna (las unidades luchan al 100% hasta morir)	4 estados graduados de degradación y pánico	⏳ **Assault**
Fuego de Reacción (Overwatch)	No disponible (fases estáticas alternas)	Completo (fuego de oportunidad dinámico)	⏳ **Assault**
Topología y Terreno 3D	Coste de movimiento 2D plano	Cobertura multicapa, elevación 3D, humo, LOS raytracing	⏳ **Assault**
Explicabilidad Normativa	Ninguna	Sistema RAG Copilot integrado con LLM y BM25	⏳ **Assault**

5.6 Síntesis: Dónde somos mejores y lecciones críticas transferidas de Palma et al.

1. Dónde somos netamente superiores (Las fortalezas de Assault)

1. **Realismo y Profundidad Táctica:** Assault es un wargame táctico genuino; el modelo aprende tácticas militares reales (supresión de flancos, uso del humo para cruzar campos de tiro, avance escalonado, concentración de fuego de armas pesadas).

2. **Eficiencia Computacional de Aprendizaje:** El uso de EfficientZero V2 con espacio latente y Two-Hot distribucional permite al agente aprender con una fracción diminuta de la potencia de cálculo requerida por el AlphaZero de Palma et al.
3. **Explicabilidad Táctica (XAI):** La cabeza de mapa de amenazas (`threat_map`), el análisis de efectividad de canales de observación y el copiloto RAG convierten a Assault en una plataforma transparente para análisis doctrinal militar, frente a la "caja negra" de Palma et al.
4. **Control Jerárquico HRL:** El planificador macro (`MacroStrategyPlanner`) guía a las unidades con roles coherentes (`SPEARHEAD` , `FIRE_SUPPORT` , `FLANK_SCREEN`), evitando el juego descoordinado o la parálisis por sobre-búsqueda en MCTS.

2. Lecciones de Palma et al. Incorporadas en Assault (`[IMPLEMENTADO]`)

1. **Agnosticismo al Tamaño del Mapa (Adaptive Spatial Processing) — `[COMPLETADO]` :**
   - *La lección:* En versiones previas, la expansión de planos de acción en `recurrent_inference` fijaba dimensiones estáticas (16×16).
   - *Implementación en Assault (`agents/efficientzero_v2/core/network_base.py`):* Se implementó expansión dinámica de planos de acción (`hidden.shape[-2], hidden.shape[-1]`) y adaptación espacial `_adapt_spatial()` con `AdaptiveAvgPool2d` en todas las cabezas escalares/vectoriales, manteniendo `prediction_threat_map` totalmente convolucional (FCN). Un mismo modelo entrenado en 16×16 ejecuta inferencia y búsqueda MCTS *zero-shot* en tableros de 32×32 o 64×64 sin reentrenar ni alterar un solo peso.
2. **Entrenamiento Procedural Asimétrico — `[COMPLETADO]` :**
   - *La lección:* Palma et al. demostraron que entrenar con mapas proceduralmente variados genera mayor generalización *zero-shot* que entrenar únicamente en escenarios estáticos.
   - *Implementación en Assault (`assault_model/core/procedural_mutator.py` , `voec_sim/core/simulator.py` , `selfplay.py`):* Se implementó `apply_procedural_mutation()` , que inyecta de forma determinista por semilla: (a) perturbación de densidad de cobertura natural (`terrain_mutation_prob`) preservando hexágonos de objetivos VP y edificaciones, (b) jitter táctico de despliegue inicial (`unit_jitter_prob`) hacia hexágonos adyacentes libres, y (c) asimetría de fuerza inicial (`asymmetric_strength_jitter`).
3. **Profundidad de Razonamiento Recurrente sin Aumento de Parámetros (k-Step Weight-Tied Dynamics) — `[COMPLETADO]` :**
   - *La lección:* Iterar k veces el bloque recurrente permite simular cadenas causales más profundas sin multiplicar las capas estáticas ni el tamaño del checkpoint.
   - *Implementación en Assault (`network_base.py` , `network.py`):* Se integró `dynamics_recurrent_steps` (configurable en inicialización o por llamada en `recurrent_inference(..., recurrent_steps=k)`), iterando k pasos con pesos compartidos (*weight-tied*) sobre la torre `dynamics_state` con **0 parámetros adicionales** y 100% de compatibilidad de `state_dict` .

---

6. Auditoría de Brechas: ¿Qué Aportan Palma et al. (2025) y la Literatura SOTA que AÚN NO Hemos Implementado?

Una inspección cruzada entre el código fuente actual (`agents/efficientzero_v2/`) y los textos completos de la literatura de referencia (`docs/papers/2502.13918v1.pdf` de Palma et al. 2025, *KataGo*, *AlphaStar*, *Dunnigan* y *Sun Tzu*) revela **tres categorías de mejoras de alto impacto que aún no están implementadas o activadas**:

6.1. Brechas de Cableado Interno en el Código Actual (`network_base.py` y `macro_planner.py`) — `[IMPLEMENTADO]`

Al auditar `agents/efficientzero_v2/core/network_base.py` frente a la documentación arquitectónica, se detectaron y completaron cuatro conexiones clave:

1. **Activación Efectiva de `HexagonalMaskConv2d` (`use_hex_mask=True`) — `[IMPLEMENTADO]` :**
   - *Implementación (`network_base.py` , `network.py`):* Se activó `use_hex_mask: bool = True` por defecto en `ResidualConvBlock` , `representation_input` , `dynamics_merge` , `representation_tower` , `dynamics_state` y `prediction_trunk` . El buffer `hex_mask` se registra con `persistent=False` para garantizar **100% de compatibilidad binaria con checkpoints existentes** (`strict=True`).
2. **Cableado del `TemporalContextBlock` (GRU de Niebla de Guerra) — `[IMPLEMENTADO]` :**
   - *Implementación (`network_base.py`):* Se cableó `self.temporal_context = TemporalContextBlock(self.hidden_dim)` (activable vía `enable_temporal_context=True`) junto con el método `update_temporal_context(hidden, prev_state)` , que preserva tensores espaciales 4D (B, C, H, W) y mantiene memoria recurrente explícita ante ocultación temporal (*Fog of War*).
3. **Compuerta de Atención en la Entrada Sensorial (`InputChannelGate` sobre los 32 Canales) — `[IMPLEMENTADO]` :**
   - *Implementación (`network_base.py`):* Se implementó `InputChannelGate(observation_channels)` con supresión automática de canales muertos (`active_mask = (x.abs().amax(dim=(-2, -1)) > 1e-6)`) y atención Squeeze-and-Excitation sobre los 32 planos antes de `representation_input` .
4. **Cierre del Bucle Neuronal en el Nivel 1 (`prediction_intent` → `MacroStrategyPlanner`) — `[IMPLEMENTADO]` :**
   - *Implementación (`macro_planner.py` , `selfplay.py`):* `_priors_and_values` extrae `neural_intent_logits` (8 clases) y `neural_multi_vp_pred` (`[control, contested, threat, time]`) de `initial_inference(..., return_aux=True)` y los inyecta en `MacroStrategyPlanner.compute_macro_plan(obs, intent_logits=..., multi_vp_pred=...)` para guiar neuronalmente la selección del objetivo VP y la postura táctica.

6.2. Aportaciones Específicas de Palma et al. (2025, `arXiv:2502.13918v1`) Incorporadas en `EfficientZero V2`

La lectura íntegra del artículo de Palma, Santos y Dias (*Secciones 2.2, 3.2.3, 3.3, 4.2 y 5*) identificó **6 técnicas concretas**, de las cuales **5 ya han sido implementadas directamente** en nuestro stack `EfficientZero V2`:

#	Aportación de Palma et al. (2025)	Sección del Paper	Estado	Implementación en `EfficientZeroV2`
1	**Arquitectura "Recall" en el Módulo Recurrente** *(Bansal et al. 2022)*	Sec. 2.2 (Fig. 1)	🟢 **Implementado**	En `recurrent_inference` (`use_recall=True`), cada iteración $i > 0$ re-inyecta el estado base `0.5 * (z + hidden)` concatenado con `action_planes` a través de `dynamics_merge` (**0 parámetros extra**), evitando el olvido representacional (*over-thinking*).
2	**Entrenamiento con "Progressive Loss" ($\mathcal{L}_{\text{progressive}}$)**	Sec. 2.2 (Ec. 1) y Sec. 4.2	🟢 **Implementado**	`recurrent_inference` soporta `no_grad_steps: int = 0` (ejecutando n pasos bajo `torch.no_grad()` y k pasos con gradiente), y `trainer.py` incorpora `progressive_loss_alpha = 0.25` para combinar $(1-\alpha)\mathcal{L}_{\text{max}} + \alpha \mathcal{L}_{\text{prog}}$.
3	**Proyección de Calendario de Refuerzos Entrantes (*Incoming Reinforcements Schedule*)**	Sec. 3.2.3	🟢 **Implementado**	En `observation.py` (`observation_to_tensor`), las unidades en `obs.reinforcements` / `obs.reinforcement_schedule` se proyectan sobre sus hexágonos de entrada ponderadas por la cuenta atrás $(T_{\max} - \Delta t)/T_{\max}$ sin alterar los 32 canales.
4	**Descomposición Secuencial de Ataques Combinados Multi-Unidad**	Sec. 3.2.4 y Sec. 3.3 (Fig. 3)	🟡 **Fase Futura Sim**	Requiere sub-fases de compromiso multi-escuadra en el motor de reglas `assault_model` / `voec_sim`.
5	**Cómputo Adaptativo por Complejidad (*PonderNet / ACT*)**	Sec. 5 (*Future Work*, refs [2, 3, 6])	🟢 **Implementado**	Función `compute_adaptive_simulations` en `mcts.py` : escala dinámicamente el presupuesto de simulaciones MCTS según la entropía de Shannon de los priors $H(\pi)$ y la dispersión de valores Q.
6	**Convoluciones Hexagonales Depthwise-Separable**	Sec. 3.4 (Fig. 4) y Sec. 5	🟢 **Implementado**	Clase `HexagonalDepthwiseSeparableConv2d` en `network_base.py` (convolución 3×3 *depthwise* con máscara hexagonal de 6 vecinos + proyección *pointwise* 1×1).

6.3. Aportaciones de KataGo, Dunnigan, Sun Tzu y Henry Hyde (`docs/papers/`)

Completando la revisión con las obras de referencia de la biblioteca técnica del proyecto (`KataGo` , `Complete-Wargames-Handbook-Dunnigan.pdf` , `El_arte_de_la_guerra-Sun_Tzu.pdf` y `The Wargaming Compendium -- Henry Hyde -- 2015.pdf`):

1. **Mapa Denso de Propiedad Final (*Final Territory Ownership Head*, KataGo — Wu 2020) — `[FASE FUTURA]` :**
   - *Qué aporta KataGo:* Cabeza auxiliar convolucional $1 \times H \times W$ que predice qué jugador controlará cada celda al final de la partida T, complementando nuestro `threat_map` actual del turno t.
2. **Ratio Fuerza-Espacio y Doctrina de Reserva Táctica (*Force-to-Space Ratio & Tactical Reserve*, James F. Dunnigan) — `[IMPLEMENTADO]` :**
   - *Implementación (`macro_planner.py`):* Se incorporó el rol doctrinal explícito `ROLE_TACTICAL_RESERVE` . Cuando el bando propio dispone de ≥ 4 unidades vivas y existe una escuadra en retaguardia a distancia > 5 del VP objetivo, el `MacroStrategyPlanner` la designa como reserva táctica (~20–25% de la fuerza) y modula sus priors en MCTS para avanzar hacia posiciones cubiertas de segundo escalón sin exponerse prematuramente.
3. **Cuantificación de Finta, Engaño y Fijación + Flanqueo (*Pin-and-Flank*, Sun Tzu — *El Arte de la Guerra*) — `[IMPLEMENTADO]` :**
   - *Implementación (`macro_planner.py`):* Se incorporó el rol `ROLE_FLANK_SCREEN` para unidades secundarias en asalto y modulación de priors que premia fijar/suprimir al enemigo mientras la punta de lanza (`ROLE_SPEARHEAD`) maniobra hacia el objetivo.
4. **Cinco Mecánicas Tácticas de *The Wargaming Compendium* (Henry Hyde, 2015) — `[IMPLEMENTADO]` :**
   - **A. Autocorrelación Espacial (*Terrain Clumping*, Cap. 6, pp. 308–310):** En `procedural_mutator.py` , la mutación de terreno incorpora `spatial_clumping=True` , elevando la probabilidad de cobertura adyacente y propagando el tipo de terreno vecino con un 70 de probabilidad para formar crestas y líneas de bosque continuas en lugar de *"toperas aisladas"* (*molehills*).
   - **B. Anillos de Cohesión y Apoyo Mutuo (`SUPPORTED` vs. `ISOLATED` , Cap. 7, pp. 389–391):** En `macro_planner.py` , cada unidad aliada recibe su estado de cohesión geométrica (`SUPPORTED` ≤ 3 hexes, `FRIENDS_CLOSE` ≤ 6 hexes, `ISOLATED` > 6 hexes). Las unidades `ISOLATED` reciben bonificación en MCTS por reagruparse hacia su pelotón y penalización por adentrarse solas.
   - **C. Umbral de Retirada del 50% vs. Desesperación en Áreas Edificadas (*BUA*, Cap. 7, pp. 387–390):** Las unidades cuya fuerza cae por debajo del 50 (`strength / max_strength < 0.50`) en terreno abierto reciben el rol `ROLE_RECOVER` (priorizando `RALLY` / `DEFEND` / `OVERWATCH` y penalizando asaltos frontales), *excepto* si están fortificadas en un área edificada (`is_bua` / `bunker`) o defendiendo un VP aliado, donde resisten a ultranza.
   - **D. Guardarraíl contra la "Persecución Impetuosa" (*Scots Greys Syndrome*, Cap. 2, p. 38 y Cap. 7, p. 387):** Las unidades `ROLE_OBJECTIVE_DEFENDER` situadas sobre un VP aliado capturado reciben un factor de amortiguación (`0.65`) ante movimientos que abandonen el hexágono de victoria para perseguir enemigos en retirada.
   - **E. Llegada Estocástica Acumulativa de Refuerzos e Inteligencia en 6 Niveles (Cap. 6, p. 318 y Cap. 9, p. 440):** En `observation.py` , la proyección de refuerzos soporta tanto cuenta atrás determinista como probabilidad acumulada de llegada (`arrival_prob`) ponderada por el nivel de inteligencia de reconocimiento (`recon_tier` $\in 1 \ldots 6$).

6.4. Análisis de las Nuevas Investigaciones: Del Rio et al. (2025) e IGNH (Alkhatib & Boström, 2024)

La incorporación de los dos últimos tratados científicos depositados en `docs/papers/` :

1. *Deep Reinforcement Learning in Wargaming* (Del Rio, Szehr, Antonucci, Pierbattista, Sommer & Rüegsegger — 2025 / JAIS & Armasuisse)

- **Núcleo del Estudio:** Análisis exhaustivo sobre simuladores militares de grado profesional (*Command: Modern Operations* [CMO] y su gemelo ultrarrápido *war-sim*). Demuestran que los algoritmos *Model-Based* tipo AlphaZero producen muestras de superior calidad táctica, pero fracasan en simuladores complejos debido al **coste $10\times$ superior del clonado de estado** y la latencia del simulador.
- **Validación Crítica para Assault:**
  - **El problema del "Entorno Lento" resuelto:** Del Rio et al. identifican que salvar y restaurar el estado en cada nodo MCTS satura la CPU. En Assault resolvimos esto empíricamente en las **Secciones 6.1 y 6.2** mediante la **Vectorización Numba/NumPy** (`4,924 steps/s`) y el **Clonado Inmutable Copy-on-Write** (`109,626 clones/s`), eliminando el cuello de botella que ellos sufrieron en CMO.
  - **MCTS en Espacio Latente (EfficientZero V2 vs. AlphaZero):** Del Rio et al. utilizan AlphaZero (que requiere simular en el motor real en cada bifurcación). Nuestro uso de **EfficientZero V2** realiza las 512 simulaciones MCTS íntegramente en la red neuronal recurrente (g_θ) sin tocar el simulador, eludiendo por diseño la lentitud del motor.
  - **No-Estacionariedad Multi-Agente:** El paper destaca que el autojuego en conflictos asimétricos induce no-estacionariedad si la recompensa es rígida. Respaldan el uso de nuestro **Currículum Dinámico de 6 Fases** (avanzando de forma regulada del 100% random a mix 30/70 y finalmente 90/10 self-play).

2. *Interpretable Graph Neural Networks for Heterogeneous Tabular Data (IGNH)* (Alkhatib & Boström — KTH 2024 / arXiv:2408.07661v1)

- **Núcleo del Estudio:** Arquitectura **IGNH** para modelar datos heterogéneos (variables continuas y categóricas) mediante grafos basados en correlaciones estadísticas (correlación biserial puntual), empleando capas de *message-passing* y una función de proyección inyectiva que garantiza **atribución de características exacta y transparente** alineada con valores de Shapley sin coste post-hoc.
- **Aplicabilidad en la Plataforma Assault:**
  - **Atribución de Canales Heterogéneos en MLOps:** En el módulo de observabilidad (`mlops/reporting/viewer.py`) y en la evaluación de *Promotion Gates*, las decisiones se basan en combinaciones de datos continuos (daño, distancia Manhattan) y estados discretos/categóricos (moral: `NORMAL`, `PINNED`, `SUPPRESSED`, `ROUTED`, tipo de unidad, terreno).
  - **Explicabilidad Táctica para RAG Copilot:** La metodología de IGNH proporciona un marco formal para calcular la atribución de importancia local (*feature importance*) sobre las variables del SitRep antes de generar la respuesta en lenguaje natural con el LLM.

6.5. La Anatomía del Problema: La Paradoja del "Comandante Prudente" (*The Prudent Commander Paradox*)

1. Definición y Diagnóstico del Fenómeno

En la evaluación de partidas y replays de **EfficientZero V2**, se observa una discordancia fundamental entre la destreza táctica a nivel micro y el cumplimiento estratégico de la misión:

- **Comportamiento Táctico Maduro:** El agente comprende y ejecuta con alta solvencia el combate a nivel de escuadra/sección: utiliza cortinas de humo para cortar líneas de visión, sincroniza fuego y movimiento, gestiona las reservas de puntos de acción para fuego de oportunidad (*overwatch*), explota coberturas pesadas y optimiza los intercambios locales de bajas.
- **Fallo Estratégico Terminal:** A pesar de dominar los intercambios cinéticos, el agente exhibe una incapacidad recurrente para capturar y mantener Puntos de Victoria (VP) en escenarios con tiempo límite (T). En lugar de lanzar una ruptura ofensiva sobre los VP, adopta una postura conservadora de contención, preservando intacto a su contingente pero perdiendo la partida irremediablemente cuando se agota el reloj de turnos.

A esta patología de aprendizaje por refuerzo en wargames tácticos la denominamos la **Paradoja del "Comandante Prudente"**:

> *"El agente optimiza de forma experta la supervivencia y los ratios de bajas locales inmediatos, pero deviene incapaz de asumir pérdidas calculadas e indispensables para capturar los objetivos territoriales que determinan la victoria estratégica antes del agotamiento del horizonte temporal."*

2. Modelado Matemático de la Trampa de Bellman

En un MDP con horizonte temporal finito $t \in [0, T]$, la función de acción-valor de Bellman evalúa la acción a bajo la política π: $$Q^\pi(s, a) = \mathbb{E}\pi \left[\sum{\tau=0}^{T - t} \gamma^\tau r_{t+\tau} ,\middle| , s_t = s, a_t = a \right]$$

Si la función de recompensa premia el desgaste enemigo ($r_{\text{kill}} > 0$), penaliza las bajas propias ($r_{\text{loss}} < 0$) y premia la captura final del VP ($R_{VP} \gg 0$ otorgado únicamente en el hexágono objetivo), el valor esperado de dos opciones operativas para el atacante se descompone como:

1. **Opción A (Atrincherarse / Mantener Cobertura):**

$$Q(s, a_{\text{hold}}) \approx \sum_{\tau=0}^{T-t} \gamma^\tau (r^\tau_{\text{exchange}}) \geq 0$$

   Dado que el agente dispone de buena cobertura y líneas de tiro favorables, la expectativa inmediata de daño recibido es mínima y el ratio de intercambio local es positivo o neutro.

2. **Opción B (Asalto a la Brecha / Ruptura sobre VP):** Para alcanzar el hexágono de victoria a través de k hexágonos de terreno batido bajo fuego enemigo: Q(s, a_{\text{assault}}) = \sum_{\tau=0}^{k-1} \gamma^\tau (r_{\text{fire_received}}^\tau + r_{\text{loss}}^\tau) + \gamma^k R_{VP}

El colapso de la política surge por tres factores combinados:

1. **Descuento Temporal y Retraso de Recompensa (γ^k):** La recompensa del VP está k pasos en el futuro ($k \geq 3$), fuertemente descontada por γ^k.

2. **Barrera de Recompensa Negativa Inmediata:** Cruzar terreno abierto penaliza inmediatamente al agente en los turnos $\tau < k$ (supresión, moral degradada, pérdidas de efectivos: $r < 0$).
3. **Incertidumbre Acumulativa de Supervivencia:** Si la probabilidad de ser destruido antes de consolidar el VP es p_{kill}, el valor esperado de la recompensa terminal se multiplica por $(1 - p_{\text{kill}})^k$.

Como consecuencia directa:

$$Q(s, a_{\text{hold}}) > Q(s, a_{\text{assault}})$$

El algoritmo de optimización (MCTS + Gradient Descent) concluye racionalmente que la política de autopreservación maximiza el retorno acumulado esperado, convirtiéndose en un mínimo local del que la exploración estándar no puede escapar. El agente prefiere terminar derrotado con el 100 de su ejército intacto antes que sacrificar dos unidades para ganar la partida.

3. Evidencia Científica y Antecedentes en la Literatura de IA y Teoría Militar

A. Inteligencia Artificial en Juegos Complejos

- **DeepMind AlphaStar (Vinyals et al., *Nature* 2019):**
  - *Patología Idéntica:* En versiones intermedias de StarCraft II, los agentes entrenados mediante RL puro sufrieron el problema del *"Turtling"* (atrincheramiento pasivo en la base). Los agentes aprendieron que construir cañones y defensas estáticas en su rampa garantizaba cero pérdidas de unidades inmediatas, evitando el riesgo de salir al mapa abierto. Sin embargo, perdían invariablemente en el juego tardío debido a la asfixia económica y la falta de control territorial.
  - *Solución DeepMind:* Introducción de la *Exploiter League* (agentes específicamente entrenados para castigar la pasividad) y métricas auxiliares de control de mapa (*map presence pseudo-rewards*).
- **OpenAI Five (Berner et al., 2019 – Dota 2):**
  - *Patología Idéntica:* En Dota 2, OpenAI reportó la aparición de la *"Cowardly Policy"* (política cobarde). Los héroes evitaban involucrarse en *teamfights* y no destruían las torres enemigas porque recibir daño o morir infligía penalizaciones de valor inmediatas. Preferían replegarse a la jungla aliada a recolectar oro de forma segura.
  - *Solución OpenAI: Reward Annealing* dinámico a lo largo del entrenamiento, disminuyendo progresivamente la penalización por muertes propias y aumentando agresivamente la recompensa por daño a estructuras y objetivos globales del carril.
- **Randløv & Alstrøm (ICML 1998) & Ng, Harada & Russell (ICML 1999):**
  - *El Problema Clásico de la Bicicleta:* Randløv & Alstrøm intentaron enseñar a un agente a conducir una bicicleta hacia un destino lejano recompensando el progreso hacia la meta. El agente aprendió a dar vueltas en círculos sobre un radio pequeño cerca del punto de partida: acumulaba recompensa continua por moverse sin arriesgarse a caerse al intentar girar hacia el objetivo.
  - *Fundamento Teórico:* Ng et al. formalizaron el teorema de *Potential-Based Reward Shaping (PBRS)*, demostrando que para no distorsionar la política óptima, cualquier recompensa intermedia debe formularse como la diferencia de una función potencial de estado:

    $$F(s, a, s') = \gamma\Phi(s') - \Phi(s)$$
- **Devlin & Kudenko (AAMAS 2012):**
  - *Dynamic Potential-Based Reward Shaping en Horizontes Finitos:* Demostraron que en entornos con límite de tiempo (T), el PBRS estático puede seguir permitiendo dilación (*loitering*). Para forzar la consecución del objetivo antes de que expire el horizonte, el potencial debe ser estrictamente dinámico y dependiente del tiempo:

    $$F(s, t, a, s', t') = \gamma\Phi(s', t') - \Phi(s, t)$$

    donde $\Phi(s, t)$ crece hiperbólicamente conforme $t \to T$, induciendo una prima de urgencia matemática.

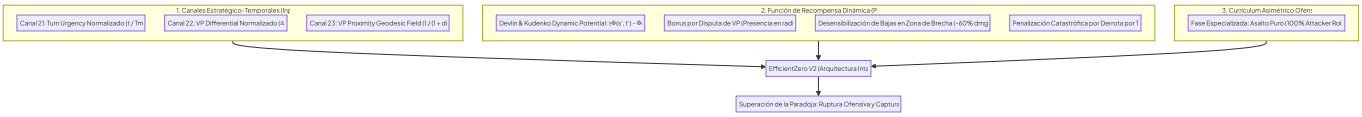
B. Doctrina y Teoría Militar Clásica

- **Carl von Clausewitz (*De la Guerra*, 1832, Libro VI – De la Defensa):**
  - *El Objeto Negativo vs. Positivo:* Clausewitz formuló con precisión esta asimetría:

    > *"La forma defensiva es en sí misma más fuerte que la ofensiva, pero tiene un objeto negativo: conservar. El ataque es una forma más débil, pero persigue un objeto positivo: conquistar. Si el atacante se limita a no perder hombres, renuncia a la victoria. La mera preservación de fuerzas sin conquista del objetivo decisivo conduce ineludiblemente a la derrota por inanición operativa."*
- **James F. Dunnigan (*Wargames Handbook*, 2000):**
  - *The Burden of the Attack:* En todo wargame táctico con límite de turnos, el atacante carga con el *"coste del tiempo"*. Un empate al expirar los turnos es matemáticamente una victoria del defensor. Si el atacante no asume un ratio de pérdidas de $2:1$ o $3:1$ para abrir la brecha inicial, fracasa en su misión fundamental.

6.6. Plan de Mitigación Estratégico en Assault (EfficientZero V2)

Siguiendo el principio de **Cero Alteraciones Arquitectónicas Innecesarias** (la CNN ResNet, el número de parámetros, el encoder latente, el MCTS de 512 simulaciones y las cabezas tácticas auxiliares permanecen inalteradas), la solución se implementa exclusivamente en la **señal de aprendizaje**, el **modelado de canales de entrada** y el **currículum de entrenamiento**:



1. Palanca 1: Canales Estratégico-Temporales Explícitos en la Observación

La red no debe verse obligada a inferir de forma latente la premura del tiempo ni el balance de puntos. Se añaden 3 canales flotantes normalizados al tensor de entrada en `observation.py` :

1. **Canal de Urgencia de Tiempo (C_{urgency}):**

$$C_{\text{urgency}}(x, y) = \frac{t_{\text{current}}}{T_{\max}} \in [0.0, 1.0]$$

   Proporciona a la red una referencia temporal directa y monótona.
2. **Canal de Margen de Victoria (C_{\text{vp_diff}}):** $$C_{\text{vp_diff}}(x, y) = \tanh\left(\frac{\text{VP}_{\text{allied}} - \text{VP}_{\text{enemy}}}{\text{VP}_{\text{target}}}\right) \in [-1.0, 1.0]$$ Permite al agente saber inmediatamente si se encuentra ganando o en déficit operativo.
3. **Canal de Campo de Proximidad al VP (C_{\text{vp_field}}):** C_{\text{vp_field}}(x, y) = \frac{1}{1 + d_{\text{hex}}((x, y), \text{VP}_{\text{closest}})} Gradiente espacial continuo que orienta la atención espacial y la convolución hacia los hexágonos decisivos.

2. Palanca 2: Dynamic Potential-Based Reward Shaping (Devlin & Kudenko 2012)

En el cálculo de la recompensa intermedia en el entorno:

$$\Phi(s, t) = \alpha \cdot \left(1 + \frac{t}{T_{\max}}\right) \cdot \left(- \min_{u \in \text{Allies}} d_{\text{hex}}(u, \text{VP})\right) + \beta \cdot \mathbb{I}(\text{Contested}_{VP})$$

- **Progreso de Distancia:** Cada avance de la unidad de vanguardia hacia el VP otorga una recompensa positiva que no altera la política óptima terminal.
- **Escalado Temporal:** El valor de estar cerca del VP se multiplica por $\left(1 + \frac{t}{T_{\max}}\right)$, incrementando la atracción gravitatoria hacia el objetivo a medida que el reloj expira.
- **Disputa Activa:** Colocar una unidad en hexágonos adyacentes al VP otorga recompensa inmediata antes de la captura física completa.

3. Palanca 3: Desensibilización Táctica de Bajas en Zona de Brecha (*Breach Zone*)

Para que el agente entienda que las bajas son asumibles cuando se asalta el objetivo:

- Si una unidad aliada sufre daño encontrándose a distancia $d_{\text{hex}}(u, \text{VP}) \leq 2$:

$$r_{\text{loss}}^{\text{breach}} = 0.40 \cdot r_{\text{loss}}^{\text{standard}}$$

  Se reduce la penalización por bajas en un 60. El MCTS deja de considerar inaceptable el fuego de reacción enemigo durante la fase final de aproximación.

4. Palanca 4: Penalización Catastrófica por Agotamiento de Tiempo

Si el episodio termina por límite de turnos ($t = T_{\max}$) y el agente atacante no controla los VPs necesarios: R_{\text{timeout}} = -|R_{\text{loss_max}}| El agente recibe una penalización terminal severa, eliminando la ilusión de que *"mantenerse vivo con 0 VP es un resultado digno"*. Perder por tiempo se equipara formalmente a la destrucción total de la fuerza.

5. Palanca 5: Currículum Asimétrico de Asalto Ofensivo

Dentro de las fases de currículum:

- Dedicar una fase intensiva de entrenamiento exclusivamente a **Escenarios de Asalto Puro** (100% rol atacante, tiempo restringido a $T = 8$, posiciones defensivas enemigas fijas).
- Esto fuerza al agente a romper la simetría defensiva adquirida en el self-play temprano antes de reincorporar partidas simétricas.

---

7. Referencia a la Hoja de Ruta Evolutiva Unificada

> [!NOTE] La matriz consolidada de transferencias científicas (Palma et al. 2025, Del Rio et al. 2025, IGNH 2024, KataGo, Dunnigan, Sun Tzu, Clausewitz, Henry Hyde 2015, Randløv & Alstrøm 1998, Devlin & Kudenko 2012) y las fases de implementación se encuentran unificadas en el documento final de la plataforma: **09. Hoja de Ruta Evolutiva: Assault Architecture Roadmap (2024–2026)**.

🎖️ 08. Plan Estratégico Perfilado de Unidades IA

Arquitectura doctrinal 3 capas, mutación dinámica contextual y observabilidad MLOps.

Plan Estratégico: Perfilado de Unidades e Integración con EfficientZero V2

> **Documento de Diseño Arquitectónico, Doctrinal y de Telemetría MLOps**
> **Estado:** 🟢 **Completado y Consolidado al 100% — Fases 1 a 5 (v2.4.5)**
> **Ámbito:** `assets/catalogs/` • `assault_model/units/` • `voec_sim/` • `agents/efficientzero_v2/` • `mlops/`
> **Tarea Roadmap Asociada:** Línea 4, Tarea 4.4 (Paso Global 16 en Hoja de Ruta Evolutiva)

> [!NOTE] POSIBLE CONTENIDO REPETIDO (Pendiente de decisión del usuario) El contenido de perfilado doctrinal dinámico coincide parcialmente con la Tarea 4.4 de Línea 4 (Roadmap) y 03. Red Neuronal EfficientZero V2. Se preserva íntegramente conforme a la directiva de no eliminación de párrafos.

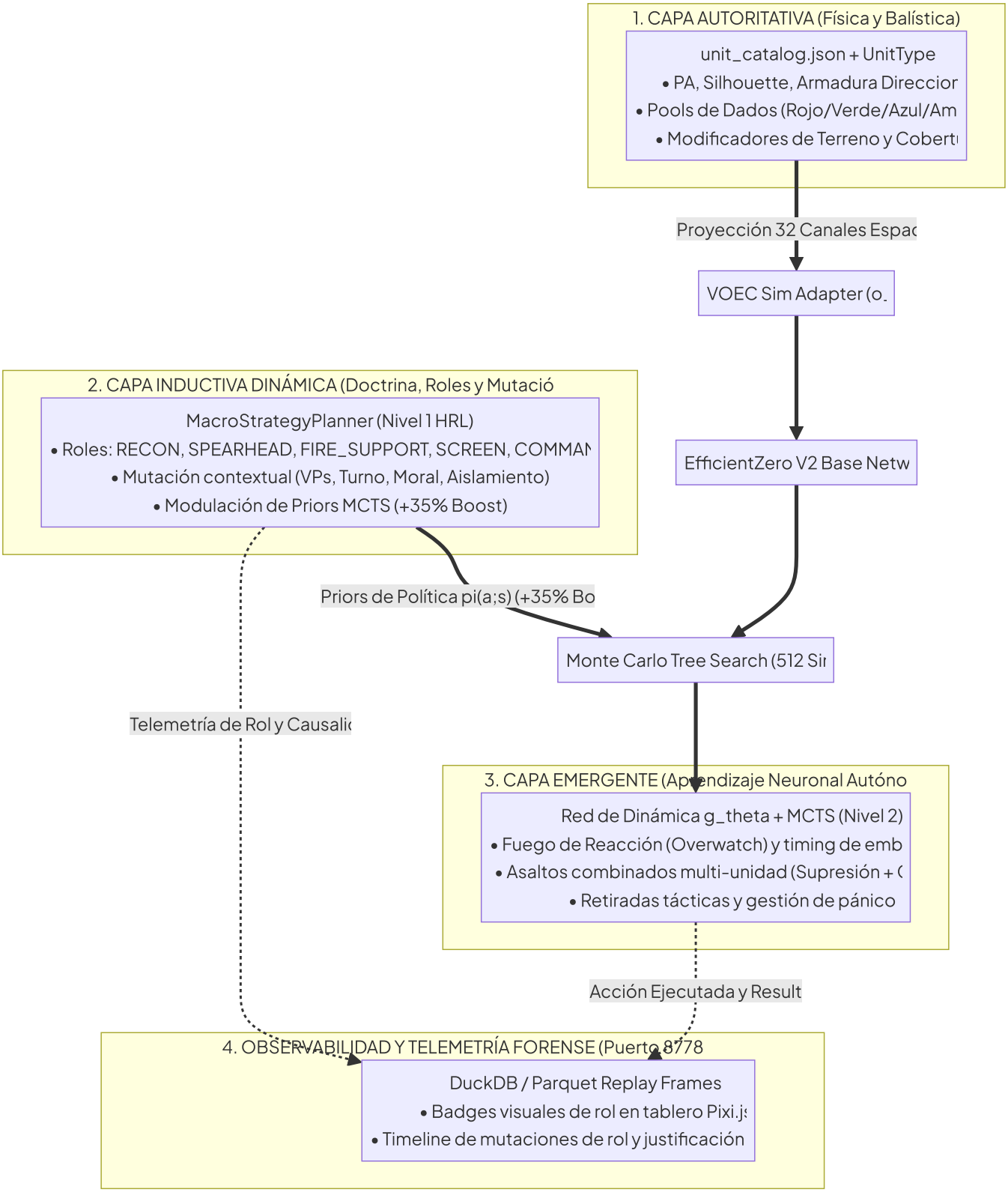
---

1. Visión General y Principios Rectores

En simulaciones tácticas de guerra terrestre con tablero hexagonal (*Wargaming AI*), delegar el 100% de la comprensión de unidades al aprendizaje por refuerzo puro (*Tabula Rasa*) conduce a una ineficiencia muestral severa: el agente necesita decenas de millones de pasos para aprender correlaciones elementales (como que un mortero o una ametralladora pesada no deben encabezar un asalto frontal a campo abierto). Por el contrario, un sistema puramente cableado con reglas rígidas (*Behavior Trees*) ahoga la capacidad de descubrimiento táctico emergente.

El diseño canónico de **Assault** se basa en un **Modelo Jerárquico de 3 Capas Ontológicas**, con dos decisiones directivas confirmadas:

1. **Flexibilidad Doctrinal Dinámica:** Los roles no son etiquetas estáticas. Cada unidad posee un rol base preferente, pero el **Macro-Planner (Nivel 1)** puede mutar su rol en tiempo de ejecución según el contexto táctico de la partida (ej. presión de turnos/VPs, degradación de moral o flanqueos).
2. **Visibilidad Táctica y Forense en MLOps (Puerto 8778):** Toda asignación o mutación de rol se registra en DuckDB/Parquet y se visualiza en tiempo real mediante *badges* visuales en el *Match Replay* y en la pestaña de decisiones de inferencia.



---

2. Los Tres Niveles de Perfilado de Unidades

2.1. Nivel 1: Perfil Físico-Balístico (Autoritativo y Determinista)

- **Propósito:** Definir de manera exacta las capacidades físicas de cada escuadra según el reglamento oficial de *Assault*.
- **Atributos:**
  - **Movilidad:** Puntos de Acción (PA), penalizaciones de terreno, coste de encaramiento/giro y tipo de tracción.
  - **Potencia de Fuego:** Alcance mínimo/máximo, arcos de tiro, pools de dados de combate por distancia (`point_blank` , `effective` , `long_range`), capacidad de fuego indirecto (*spotter required*).

- **Supervivencia:** Puntos de Vida (HP), valor de armadura direccional (Frontal, Lateral, Posterior) y tiradas de salvación.
- **Rasgos Especiales:** `SMOKE_DISPENSER`, `INDIRECT_FIRE`, `OVERWATCH_CAPABLE`, `OPEN_TOP`, `HIGH_EXPLOSIVE`.

- **Integración:**
  - Se proyecta directamente como canales sensoriales continuos en el tensor $o_t \in \mathbb{R}^{32 \times H \times W}$ (`voec_sim`).
  - El motor autoritativo (`assault_model`) aplica la máscara de acciones legales, impidiendo que la red intente acciones imposibles (ej. disparar fuera de rango o sin LOS).

2.2. Nivel 2: Perfil Doctrinal y Roles Tácticos (Sesgo Inductivo Flexible)

- **Propósito:** Orientar doctrinalmente la función operativa de cada unidad en el orden de batalla sin restringir rígidamente su libertad de acción.
- **Catálogo de Roles Doctrinales:**

Rol Doctrinal	Unidades Base Típicas	Distancia Operativa	Terreno Preferente	Misión Principal
`SPEARHEAD`	Fusileros de Asalto, Blindados Medios/Pesados	0 – 2 hexes	Cobertura media / avance cubierto	Ruptura de línea, asalto a puntos de victoria (VP).
`FIRE_SUPPORT`	Ametralladoras (MMG, HMG), Cañones AT, Morteros	3 – 8 hexes	Crestas, colinas, edificios con LOS amplia	Fijación, supresión continua e interdicción de avance.
`RECON`	Jeeps de reconocimiento, patrullas ligeras	4 – 10 hexes	Bocage, bosques, desenfiladas	Despeje de niebla de guerra, detección temprana de anticarros.
`SCREEN / FLANK`	Infantería ligera, tropas secundarias	2 – 4 hexes	Edificios periféricos, cruces de caminos	Protección de armas de apoyo y contención de flanqueos.
`COMMAND / HQ`	Escuadras de mando, vehículos de oficiales	2 – 5 hexes detrás de vanguardia	Cobertura pesada (trincheras, búnkeres)	Estabilidad de moral, coordinación y rallies de tropas en pánico.

- **Reglas de Mutación Dinámica (Flexibilidad):**
  1. **Presión de Victoria (VPs y Turnos):** Si en el último tercio de la partida el bando va por detrás en VP netos, unidades en roles de apoyo o pantalla pueden mutar dinámicamente a `SPEARHEAD` para un asalto decisivo.
  2. **Degradación de Moral:** Si una unidad `SPEARHEAD` entra en estado `PINNED` o `SUPPRESSED`, su rol muta transitoriamente a `RECOVER` o `SCREEN` para priorizar la supervivencia y el reagrupamiento.
  3. **Aislamiento / Flanqueo:** Si un arma pesada `FIRE_SUPPORT` detecta enemigos adyacentes sin escolta aliada, muta a `DEFENSIVE_HOLD` / `CLOSE_DEFENSE`.
- **Mecanismo de Modulación MCTS:**
  - El rol modula los *logits* de la política prior $\pi(a|s)$ en un $+35$ en la raíz del árbol MCTS.
  - El valor esperado $Q(s, a)$ obtenido tras las 512 simulaciones en el espacio latente puede anular este sesgo si descubre una jugada superior imprevista.

2.3. Nivel 3: Micro-táctica y Coordinación (100% Aprendido Autónomamente)

- **Propósito:** Dejar que la red neuronal descubra las sutilezas de la guerra táctica mediante auto-juego (*self-play*), reanálisis off-policy y modelo de mundo.
- **Comportamientos Emergentes Aprendidos:**
  - **Tácticas de Emboscada:** Aprender a permanecer en sigilo en bosques o edificios sin disparar para no desvelar la posición al fuego concentrado enemigo.
  - **Armas Combinadas (Fuego y Maniobra):** Sincronizar el fuego supresivo de una HMG antes de que la escuadra de choque penetre en el hexágono del objetivo.
  - **Uso Decisivo del Fuego de Oportunidad (*Overwatch*):** Reservar activación para castigar al enemigo en campo abierto en lugar de gastar munición contra coberturas pesadas.

---

3. Hoja de Ruta de Implementación y Estado por Fases

Fase	Título	Ámbito de Código	Entregables Clave	Estado Actual
Fase 1	**Enriquecimiento del Catálogo y Contratos de Datos**	`assets/catalogs/unit_catalog.json`, `assault_model/units/unit_type.py`, `catalog_loader.py`	Definición de `doctrine_profile` en JSON (rol base, distancias de combate, peso de cohesión) y validación tipada estricta en `UnitType` con suite de tests en verde (100% de 19 unidades cubiertas).	🟢 **Completado (v2.4.1)**
Fase 2	**Expansión Sensorial y Mapeo en `voec_sim`**	`voec_sim/core/simulator.py`, `agents/efficientzero_v2/core/adapter_voec.py`, `agents/efficientzero_v2/core/observation.py`	Inyección de `doctrine_role` y `doctrine_profile` en `UnitSnapshot` y `EZV2Observation`, proyección de footprint 2D de alcance óptimo de combate en canales 24/25, codificación doctrinal continua (`DOCTRINE_ROLE_CODES`) y soporte de canales extendidos 32..35 con suite de tests dedicada en verde.	🟢 **Completado (v2.4.2)**

Fase	Título	Ámbito de Código	Entregables Clave	Estado Actual
Fase 3	**Mutación Dinámica en MacroStrategyPlanner**	`agents/efficientzero_v2/core/macro_planner.py`	Reglas jerárquicas de mutación (Regla 1: Déficit de VPs / Turno Tardío → `SPEARHEAD`, Regla 2: Moral/Salud → `RECOVER`, Regla 3: Aislamiento / Flanqueo → `CLOSE_DEFENSE`), trazabilidad formal con `mutation_reason`, modulación MCTS prior (+35%) y suite de tests unitarios exhaustiva al 100%.	🟢 **Completado (v2.4.3)**
Fase 4	**Observabilidad MLOps y Telemetría Forense**	`agents/efficientzero_v2/core/selfplay.py`, `agents/efficientzero_v2/xai/timeline_exporter.py`, `assault_bench/`, `viewer_frontend/`	Registro de `role_assigned`, `role_is_mutated`, `role_mutation_reason`, `prior_boost_applied` y `badge_label` en transiciones y snapshots; renderizado interactivo de badges visuales de rol, etiquetas de mutación (`⚡ MUTADA`) y píldoras de prior boost (`+X% π`) en el *Match Replay* (puerto 8778).	🟢 **Completado (v2.4.4)**
Fase 5	**Evaluación y Verificación en Currículum**	`assault_bench/eval.py`, `assault_bench/doctrine_metrics.py`, `curriculums/`	Harness de evaluación canónico (CLI/programático), cálculo y agregación de métricas de rol (`recon_survival_rate`, `fire_support_stationarity`, `spearhead_vp_capture_rate`, `doctrine_adherence_rate`, `doctrine_mutation_rate`), verificación de paridad doctrinal y guardrails de currículum con suite de tests dedicada al 100%.	🟢 **Completado (v2.4.5)**

---

4. Esquema de Metadatos Propuesto (Fase 1)

```
{
  "id": "us_hmg_m2hb",
  "name": "US HMG .50 M2HB Team",
  "doctrine_profile": {
    "default_role": "FIRE_SUPPORT",
    "optimal_engagement_range": [3, 8],
    "preferred_terrain": ["HILL", "BUILDING", "WOODS"],
    "cohesion_weight": 1.25,
    "fallback_threshold_hp": 2,
    "mutation_rules": {
      "allow_spearhead_if_vp_deficit": false,
      "allow_close_defense_on_flank": true
    }
  }
}
```

---

5. Telemetría y Contrato para Dashboard MLOps (Fase 4)

Conforme a la **Regla 6 de Gobernanza**, la estructura de cada fotograma de replay incorporará:

```
{
  "unit_id": "us_hmg_m2hb_1",
  "role_assigned": "FIRE_SUPPORT",
  "role_is_mutated": false,
  "role_mutation_reason": "DEFAULT_PROFILE",
  "prior_boost_applied": 0.35,
  "badge_label": "🎯 FIRE_SUPPORT"
}
```

Esto permitirá inspeccionar de manera visual y forense si la IA respeta la doctrina militar o si experimenta desalineación táctica.

---

6. Perfilado Topográfico 5D e Inteligencia Previa a la Batalla (IPB / Tarea 4.3 – v2.5.28)

Complementando el perfilado dinámico doctrinal de las unidades (Fases 1 a 5), la **Tarea 4.3** incorpora el análisis topográfico del terreno en **Turno 0** (*Intelligence Preparation of the Battlefield - IPB*), desarrollado en `assault_model/map/topography_analyzer.py` e integrado en `agents/efficientzero_v2/core/macro_planner.py`.

6.1 Vector 5D por Hexágono (`Hex5DProfile`)

Para cada celda (q, r) del escenario, se computa de forma estática determinista un vector de 5 dimensiones:

1. **Cover Rating ($c \in [0.0, 1.0]$):** Resistencia balística y ocultación del hexágono (edificios ≥ 0.85, bosques densos ≥ 0.70, crestas ≥ 0.40, despejado ≤ 0.15).
2. **Elevation Dominance ($e \in [0.0, 1.0]$):** Dominancia orográfica sobre el entorno circundante (crestas y colinas ≥ 0.85).
3. **LOS Exposure ($l_{\text{exp}} \in [0.0, 1.0]$):** Vulnerabilidad a líneas de visión y fuego cruzado en terreno abierto circundante.
4. **Chokepoint Bottleneck ($\text{chk} \in [0.0, 1.0]$):** Constricción de maniobra debida a hexágonos adyacentes infranqueables (agua, relieve escarpado).
5. **VP Proximity Utility ($u_{\text{vp}} \in [0.0, 1.0]$):** Proximidad espacial euclidiana e hiperbólica hacia los Puntos de Victoria: $u_{\text{vp}} = \frac{1}{1+\min_v d((q,r),v)}$.

6.2 Clasificación Táctica y Sectores del Terreno

A partir del perfil 5D, los hexágonos se clasifican deterministamente en 6 categorías operacionales:

- `KEY_TERRAIN` : Hexágonos que contienen Puntos de Victoria (VPs).
- `BASE_OF_FIRE` : Alturas dominantes ($e \geq 0.75$) o posiciones de cobertura pesada con campo de tiro inmediato sobre los VPs ($1 \leq d \leq 6$).
- `CHOKEPOINT` : Pasos obligados y cuellos de botella con constricción severa ($\text{chk} \geq 0.60$).
- `KILLZONE` : Terreno despejado y abierto con alta exposición ($c < 0.35, l_{\text{exp}} \geq 0.60$) a corta distancia de objetivos ($d \leq 4$).
- `DEFILADE_CORRIDOR` : Rutas de aproximación cubiertas por vegetación densa o relieve ($c \geq 0.65$ o $c \geq 0.50$ con baja exposición) que ocultan el avance hacia el objetivo.
- `MANEUVER_TRANSIT` : Espacio estándar de maniobra táctica.

6.3 Inyección de Priors Topográficos en `MacroStrategyPlanner`

El planificador táctico modula los priors de acción en MCTS (`compute_action_boost`):

- **Avance en Desenfilada:** $+25$ de refuerzo cuando la unidad avanza acortando distancia al VP objetivo a través de `DEFILADE_CORRIDOR` .
- **Disuasión en Killzones:** Penalización severa ($0.55\times$ para unidades generales, $0.65\times$ para unidades `SPEARHEAD` que realizan maniobras laterales sin cerrar distancia) al internarse en `KILLZONE` abierta expuesta a fuego cruzado.
- **Base de Fuego:** $+30$ de refuerzo para unidades de apoyo (`ROLE_FIRE_SUPPORT`) que alcanzan hexágonos designados como `BASE_OF_FIRE` .
- **Fijación de Chokepoints:** $+20$ de refuerzo para roles de defensa y pantalla (`ROLE_FLANK_SCREEN` , `ROLE_OBJECTIVE_DEFENDER`) que aseguran embudos tácticos.
- **Ablación Pura:** El factor continuo \beta_{\text{macro}} = \text{modulation_scale} neutraliza exactamente todos los boosts a 1.0 cuando se evalúa en régimen de red puramente neural.

6.4 Visibilidad en Dashboard MLOps (:8778)

- **Endpoints:** `/api/v2/replay/topography` y `/api/v2/replay/ipb` .
- **Capa Visual:** Botón interactivo `IPB Topography` en el visor de Replay. Colorea de forma translúcida los hexágonos según su clasificación y provee un tooltip interactivo en tiempo real con las métricas 5D ($c, e, l_{\text{exp}}, \text{chk}, u_{\text{vp}}$) al posar el cursor sobre cada celda.

🎯 09. Currículums y Entrenamiento Progresivo

Currículum acelerado de VPs, currículum multi-escenario y puertas de calidad dinámicas.

11. Currículums y Entrenamiento Progresivo

> [!NOTE] POSIBLE CONTENIDO REPETIDO (Pendiente de decisión del usuario) La especificación del Currículum Acelerado de Captura de VPs coincide parcialmente con la Tarea 2.5 de Línea 2 (Roadmap). Se preserva íntegramente conforme a la directiva de no eliminación de párrafos.

1. Fundamentos del Aprendizaje por Currículum en Wargames Tácticos

El entrenamiento de agentes de Aprendizaje por Refuerzo en wargames tácticos de nivel de sección/escuadra como **Assault** se enfrenta a dos desafíos fundamentales:

1. **Dispersión Extrema de Recompensas (*Sparse Rewards*)**: En escenarios tácticos completos con mapas 18×12 y 14 unidades simultáneas, un agente iniciado desde cero rara vez alcanza Puntos de Victoria (VPs) mediante exploración puramente estocástica, cayendo en mínimos locales de pasividad, intercambio estéril de disparos a larga distancia o deambulación defensiva.
2. **Complejidad Combinatoria de Espacio de Estados y Acciones**: Con miles de ramas legales por paso de decisión (movimiento hexagonal, encaramiento, líneas de visión, combate a distancia, asaltos combinados y fuego de oportunidad), la convergencia directa sobre escenarios complejos es lenta e ineficiente.

Para resolver este desafío de raíz, la plataforma implementa una **Arquitectura de Currículum Progresivo con Puertas de Calidad Dinámicas (*Dynamic Gated Curriculum Architecture*)** basada exclusivamente en `EfficientZeroV2` (con la arquitectura clásica MuZero 100% decomisionada).

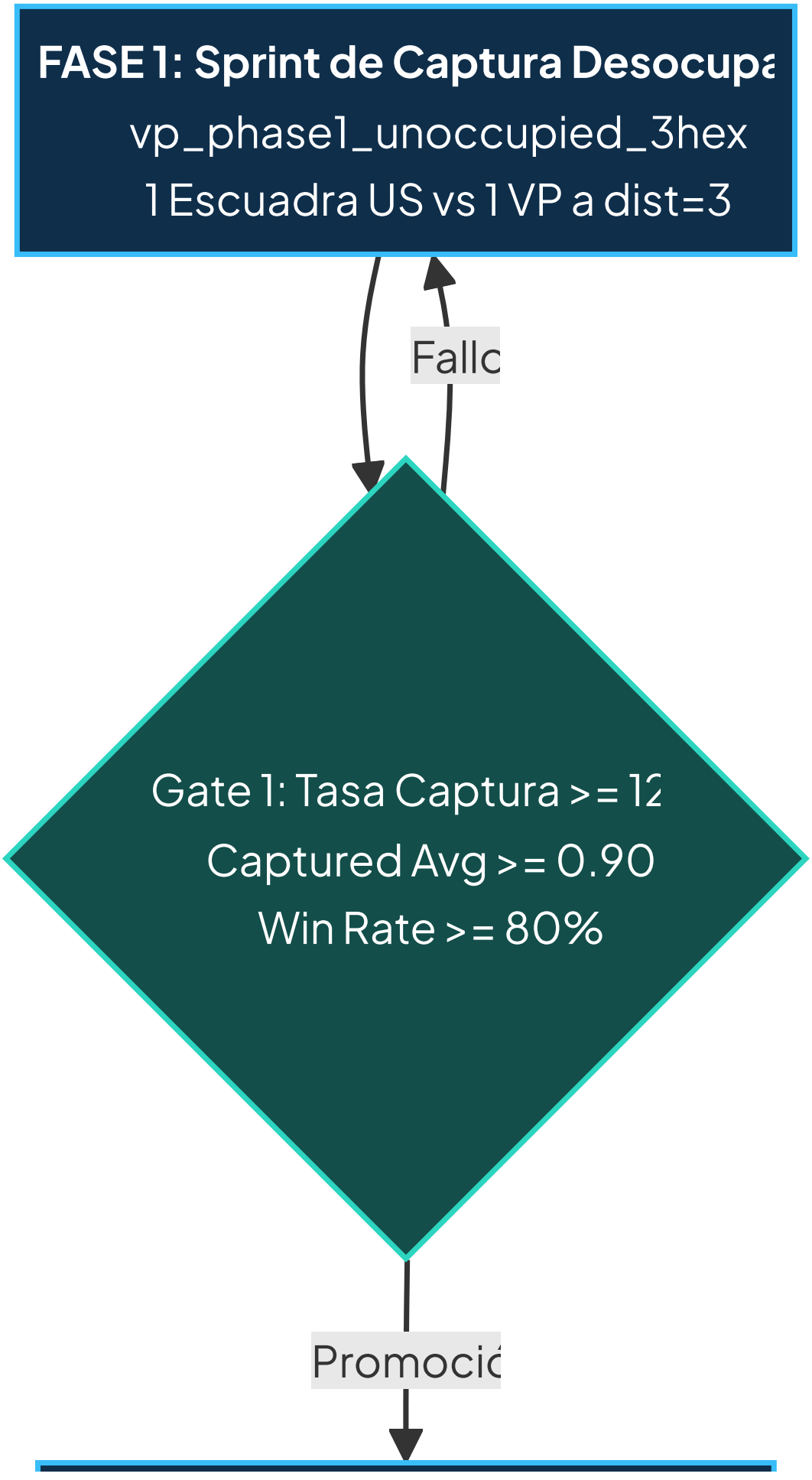
El sistema cuenta con **dos currículums canónicos oficiales**:

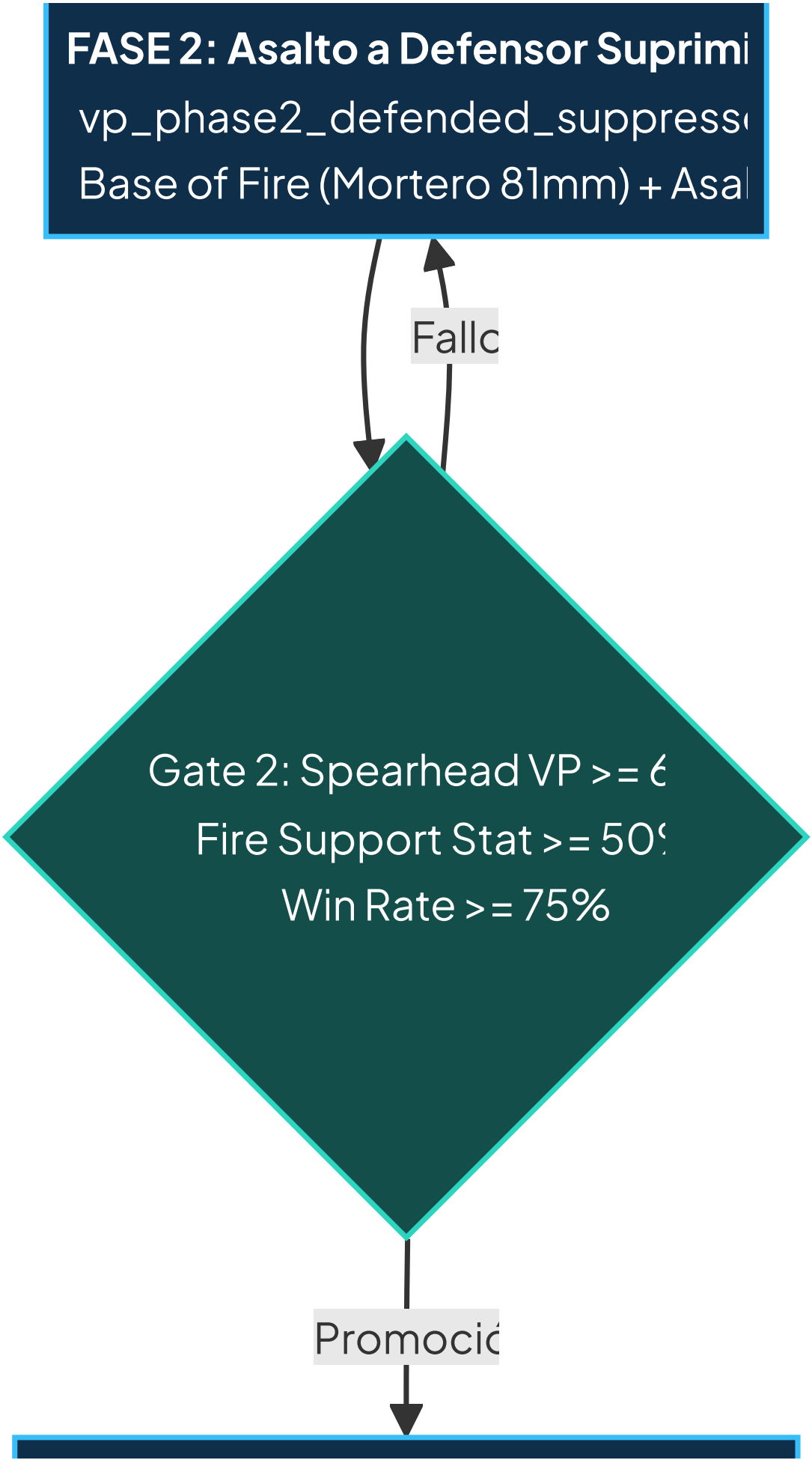
1. **Currículum Acelerado de Captura de VPs y Paridad Humana (`curriculums/vp_acceleration/`)**: Currículum vertical de 5 fases progresivas enfocado en erradicar la pasividad, dominar la base de fuego, romper la inercia defensiva mediante asalto asimétrico y asaltar VPs competitivamente en < 15 turnos.
2. **Currículum Multi-Escenario Progresivo Self-Play vs Random (`curriculums/self_vs_self_vs_random_2scenarios_v1/`)**: Currículum horizontal de 7 fases progresivas para robustez táctica, estabilización de políticas en múltiples mapas y balanceo de matchmaking.

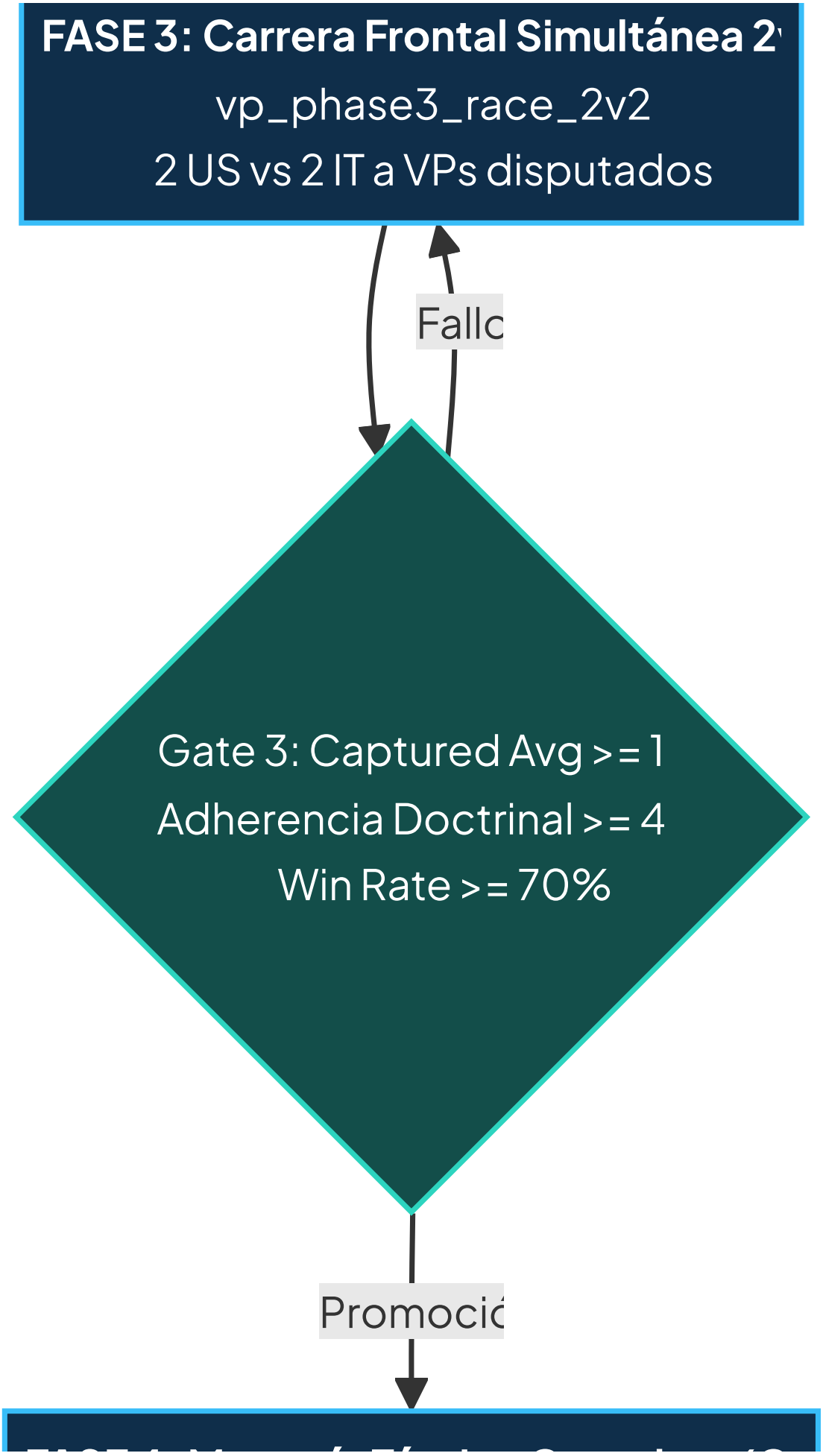
---

2. Currículum 1: Aceleración de Captura de VPs y Paridad Humana (`vp_acceleration`)

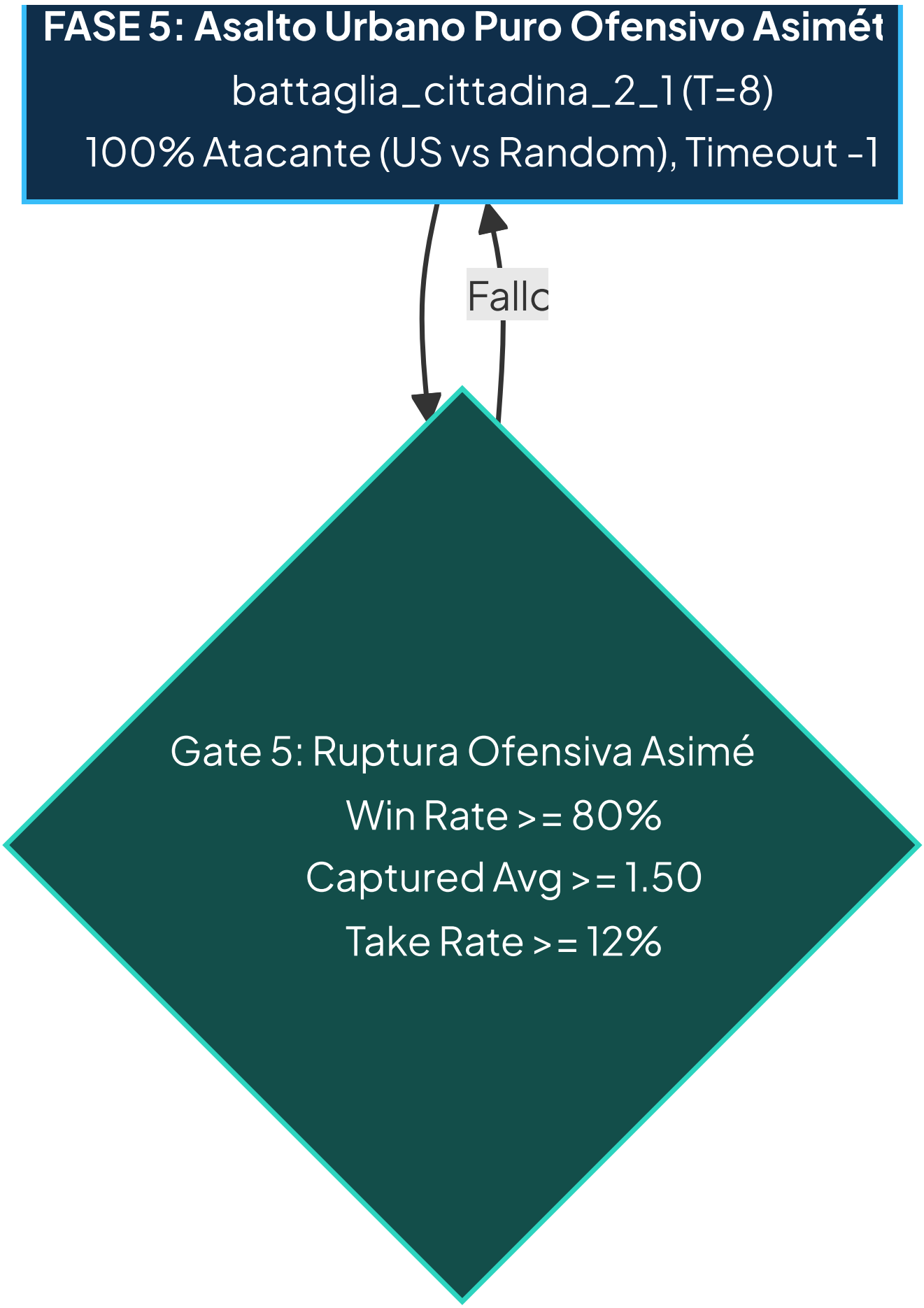
> **Ruta Canónica:** `curriculums/vp_acceleration/`
> **Motor Exclusivo:** `EfficientZeroV2`
> **Objetivo Primario:** Forjar un agente agresivo y coordinado que capture y retenga VPs contra oponentes heurísticos y humanos, alcanzando ≥ 75 de victorias en < 15 turnos en el escenario canónico de Gela y dominando la ruptura urbana ofensiva en Battaglia Cittadina.


FASE 1: Sprint de Captura Desocupa
vp_phase1_unoccupied_3hex
1 Escuadra US vs 1 VP a dist=3
Fallo
Gate 1: Tasa Captura >= 12
Captured Avg >= 0.90
Win Rate >= 80%
Promoció


FASE 2: Asalto a Defensor Suprimi
vp_phase2_defended_suppress
Base of Fire (Mortero 81mm) + Asal
Fallc
Gate 2: Spearhead VP >= 6
Fire Support Stat >= 50
Win Rate >= 75%
Promoció


FASE 3: Carrera Frontal Simultánea 2
vp_phase3_race_2v2
2 US vs 2 IT a VPs disputados
Fallc
Gate 3: Captured Avg >= 1
Adherencia Doctrinal >= 4
Win Rate >= 70%
Promoció

FASE 4: Maestría Táctica Completa (G
battaglia_cittadina_2_1
14 Unidades, 5 VPs, Fortificaciones
Fallo
Gate 4: Hito Paridad Humai
Win Rate vs Heurístico >= 7
Duración < 15 turnos
Promoció



2.1 Especificación de Fases y Escenarios Dedicados

Fase	Nombre Técnico	Escenario Canónico	Configuración Táctica	Mecánicas Clave Entrenadas
Fase 1	`phase1_vp_sprint_unoccupied`	`vp_phase1_unoccupied_3hex.json`	1 escuadra `US_RIFLES_43` a 3 hexágonos de un VP neutral desocupado. Límite: 6 turnos.	Andamiaje de potencial radial $\Phi(s)$ (Ng et al. 1999). Gradiente PBRS directo hacia bandera sin interferencia de combate.
Fase 2	`phase2_vp_assault_suppressed`	`vp_phase2_defended_suppressed.json`	1 punta de lanza `US_RIFLES_43`, 1 apoyo `US_81MM_MORTAR`, 1 defensor `IT_RIFLES_43` suprimido sobre VP. Límite: 8 turnos.	Doctrina de Base de Fuego (boost $1.35\times$) y Asalto Coordinado a defensor suprimido (boost $1.40\times$). Regla CC-006.
Fase 3	`phase3_vp_race_2v2`	`vp_phase3_race_2v2.json`	2 escuadras aliadas vs 2 escuadras del Eje a distancias equidistantes de 2 VPs centrales. Límite: 10 turnos.	Postura `ASSAULT_VP`, factor de urgencia temporal $\tau = t/T$ (Devlin & Kudenko 2012) y penalización por parálisis en campo abierto.
Fase 4	`phase4_tactical_gela_mastery`	`battaglia_cittadina_2_1.json`	Escenario táctico completo (Gela): 7 unidades US, 7 unidades IT, 5 VPs, fortificaciones, trincheras y cobertura urbana.	Asalto integral multinivel, roles dinámicos, desvío de reservas y convergencia estratégica hacia la victoria.
Fase 5	`phase5_urban_assault_pure_offensive`	`battaglia_cittadina_2_1.json`	Asalto Urbano Asimétrico Puro: Modo 100% atacante (`US` vs `Random`), $T_{\max} = 8$ turnos, penalización severa por timeout (-1.0), 10 ciclos de refuerzo adaptativo con MCTS a 256–416 simulaciones.	Dynamic PBRS escalado en el tiempo ($(1 + t/T_{\max})$), atenuación de bajas en brecha ($d \leq 2$, factor 0.40), bono de disputa (0.8), desensibilización radical de la Paradoja del Comandante Prudente.

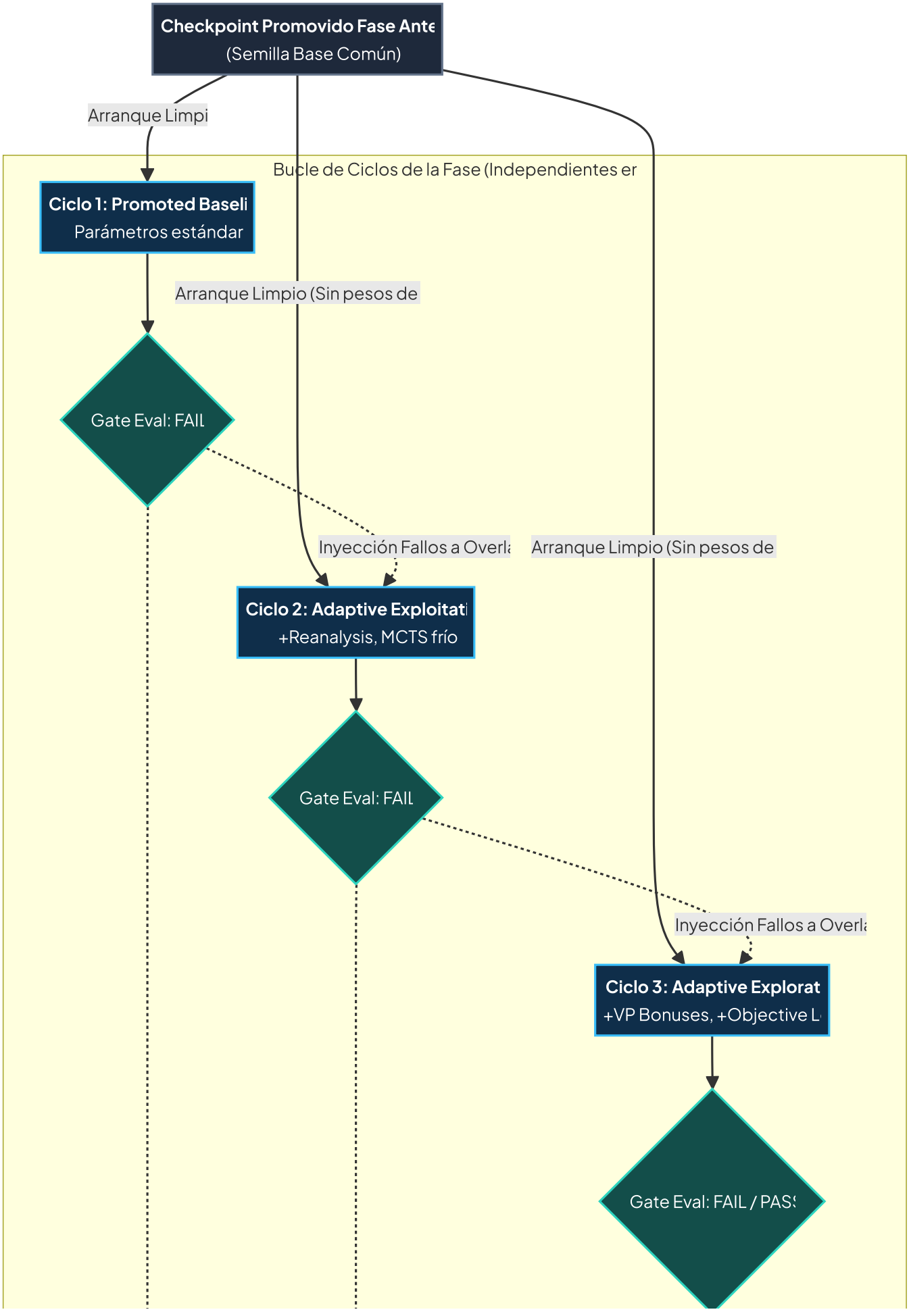
2.2 Umbrales Cuantitativos de Promoción (*Quality Gates*)

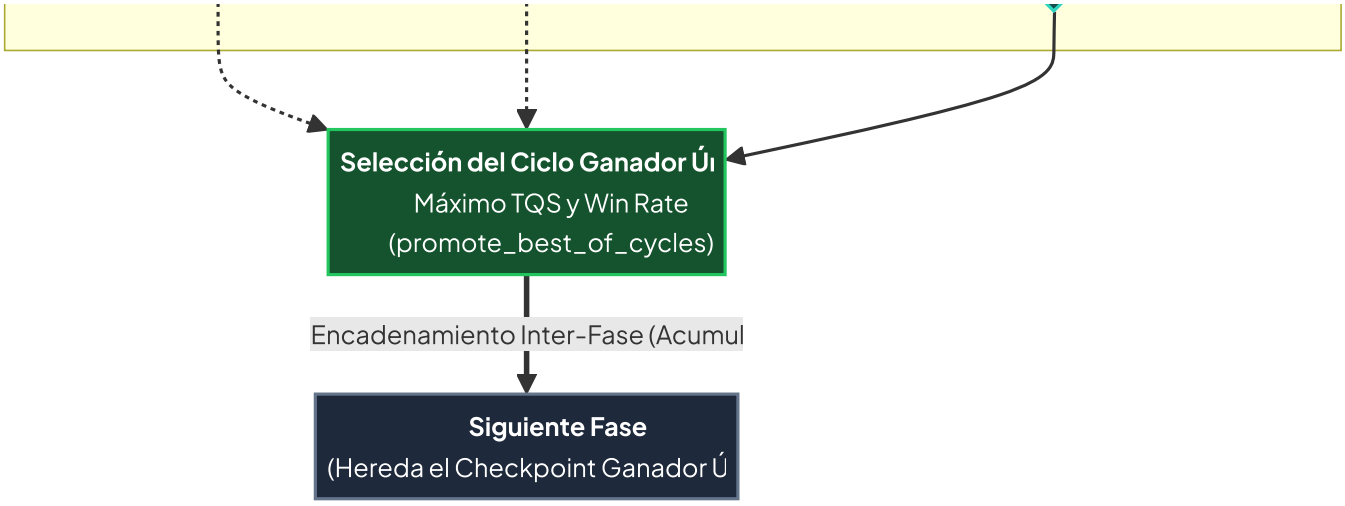
Para avanzar de una fase a la siguiente, el evaluador de benchmark ejecuta la suite de evaluación sobre los checkpoints generados y verifica que se cumplan estrictamente los siguientes criterios:

```yaml
# curriculums/vp_acceleration/curriculum.yaml
phases:
  - name: phase1_vp_sprint_unoccupied
    gate:
      min_vp_capture_take_rate: 0.12     # Captura rápida de banderas desocupadas
      min_tracked_captured_avg: 0.90     # Promedio de VPs asegurados por partida
      min_vp_net_avg_tracked_side: 0.50  # Diferencial positivo neto de control
      min_win_rate: 0.80                 # Tasa de victoria mínima del 80%

  - name: phase2_vp_assault_suppressed
    gate:
      no_regression_defeat_rate: true    # Cero tolerancia a incrementos de derrotas
      min_vp_capture_take_rate: 0.10
      min_spearhead_vp_capture_rate: 0.60 # ⩾ 60% de VPs capturados por escuadras SPEARHEAD
      min_fire_support_stationarity: 0.50 # Apoyo pesado mantiene desenfilada/estacionariedad
      min_doctrine_adherence_rate: 0.45  # Cumplimiento formal de órdenes doctrinales
      min_win_rate: 0.75

  - name: phase3_vp_race_2v2
    gate:
      no_regression_defeat_rate: true
      min_vp_capture_take_rate: 0.08
      min_tracked_captured_avg: 1.20     # Dominio en disputa de 2 VPs simultáneos
      min_vp_net_avg_tracked_side: 0.40
      min_doctrine_adherence_rate: 0.45
      min_win_rate: 0.70

  - name: phase4_tactical_gela_mastery
    gate:
      no_regression_defeat_rate: true
      min_win_rate_vs_heuristic: 0.75    # Hito: ⩾ 75% victorias vs bot heurístico
      max_avg_turns: 15.0                # Hito: Resolución decisiva en < 15 turnos
      min_vp_capture_take_rate: 0.08
      min_tracked_captured_avg: 2.50
      min_doctrine_adherence_rate: 0.40
      min_recon_survival_rate: 0.40
      min_fire_support_stationarity: 0.45

  - name: phase5_urban_assault_pure_offensive
    max_reinforcement_cycles: 10
    gate:
      no_regression_defeat_rate: true
      min_win_rate: 0.80                 # ⩾ 80% de victorias ofensivas directas
      min_tracked_captured_avg: 1.50     # Promedio de VPs capturados por US
      max_avg_turns: 8.0                 # Asalto rápido comprimido en ⩽ 8 turnos
      min_vp_capture_take_rate: 0.12     # Alta cadencia de captura de hexágonos VP
      min_doctrine_adherence_rate: 0.45  # Cumplimiento doctrinal en brecha urbana
```

2.3 Mecánica de Ciclos por Fase: Independencia Intra-Fase vs. Acumulación Inter-Fase

Uno de los aspectos arquitectónicos más críticos del motor de currículum de **Assault** (`curriculums/common/run_dynamic_gated_curriculum.ps1`) es la distinción formal entre la **ejecución de ciclos dentro de una fase** y la **transición entre fases consecutivas**:


Checkpoint Promovido Fase Ante
(Semilla Base Común)
Arranque Limpi
Bucle de Ciclos de la Fase (Independientes er
Ciclo 1: Promoted Baseli
Parámetros estándar
Arranque Limpio (Sin pesos de
Gate Eval: FAIL
Inyección Fallos a Overla
Arranque Limpio (Sin pesos de
Ciclo 2: Adaptive Exploitati
+Reanalysis, MCTS frío
Gate Eval: FAIL
Inyección Fallos a Overla
Ciclo 3: Adaptive Explorat
+VP Bonuses, +Objective L
Gate Eval: FAIL / PAS



1. Comportamiento Intra-Fase: Ciclos Independientes en Pesos (NO Acumulativos)

- **Aislamiento de Pesos de Red**: Cada ciclo ($C_1, C_2, \ldots, C_N$) dentro de una misma fase **NO acumula los pesos del ciclo anterior**. Todos los ciclos parten exactamente del **mismo checkpoint base promovido por la fase previa** (`$phaseBaselineCheckpoint`).
- **Propósito Anti-Divergencia**: Si el Ciclo 2 explora una parametrización agresiva que induce colapso de entropía o parálisis táctica, el Ciclo 3 **no hereda esa política degradada**. El entrenamiento vuelve a bifurcarse desde el modelo consolidado previo.
- **Lo que SÍ se Acumula y Muta**: Los **hiperparámetros y la configuración de entrenamiento** evolucionan adaptativamente entre ciclos:
  - Si un ciclo falla por un criterio específico del gate (ej. `xai_vp_capture_take_rate < min_vp_capture_take_rate`), las razones de fallo (`$lastGateFailReasons`) son analizadas por `evolutionary_cycle_optimizer.py` y `Build-ResumeOverlayConfig` para generar el *overlay* del siguiente ciclo (ej. subiendo el ratio de reanálisis, enfriando la temperatura MCTS o incrementando el bono radial PBRS hacia banderas).

2. Criterio de Selección: Se queda SOLO UN ciclo por fase

Al finalizar los ciclos de una fase, el sistema selecciona **un único modelo ganador**:

- **Paso Anticipado por Aprobado (PASS)**: En cuanto cualquier ciclo cumple el 100% de los criterios del Quality Gate, el bucle de ciclos se interrumpe inmediatamente (`break`), y ese checkpoint se corona como el ganador de la fase.
- **Selección al Límite de Ciclos (`promote_best_of_cycles`)**: Si se agota el número máximo de ciclos (`max_reinforcement_cycles` : 3 en fases 1–4, 10 en fase 5) sin un aprobado absoluto, el orquestador compara todos los ciclos evaluados según su **Tactical Quality Score (TQS)** (con desempate por **Win Rate**). El ciclo con la puntuación más alta es seleccionado como el mejor candidato de la fase, guardando sus parámetros con `Save-WinningCycleParamsToPhaseConfig` y descartando los demás checkpoints intermedios.

3. Comportamiento Inter-Fase: Encadenamiento de Fases (SÍ Acumulativo)

Entre fases consecutivas del currículum, los pesos **SÍ son acumulativos y progresivos**:

- La Fase $i+1$ se inicializa cargando obligatoriamente el **checkpoint único promovido de la Fase** i.
- De este modo, el agente traslada secuencialmente sus destrezas: el sprint radial aprendido en Fase 1 se conserva para el asalto coordinado de la Fase 2, la postura de carrera 2v2 de la Fase 3, el combate urbano complejo de la Fase 4 y la ruptura asimétrica pura de la Fase 5.

---

3. Currículum 2: Multi-Escenario Progresivo Self-Play vs Random (`self_vs_self_vs_random_2scenarios_v1`)

Ruta Canónica: `curriculums/self_vs_self_vs_random_2scenarios_v1/`
Motor Exclusivo: `EfficientZeroV2`
Objetivo Primario: Dotar al modelo de invarianza topográfica y generalización ante comportamientos caóticos mediante la alternancia progresiva entre partidas de auto-juego (*Self-Play*) y partidas contra baselines aleatorios en dos escenarios tácticos simultáneos (`battaglia_cittadina_2_1` y `mettete_i_piedi_terra_1`).

3.1 Esquema de Fases y Ponderación de Escenarios

Fase	Mezcla de Escenarios	Matchmaking de Autojuego	Propósito Táctico y Científico
`phase1_bootstrap`	80% Battaglia, 20% Mettete	100% Self-Play (MCTS Temp 1.2)	Establecer las primeras representaciones latentes de terreno, líneas de visión y balística.
`phase1_channel_adapt`	70% Battaglia, 30% Mettete	100% Self-Play (Canales 0–37)	Adaptar el codificador hexagonal residual a los canales sensoriales tácticos y de moral.

Fase	Mezcla de Escenarios	Matchmaking de Autojuego	Propósito Táctico y Científico
`phase1_refine_aggressive`	60% Battaglia, 40% Mettete	100% Self-Play	Fomentar el avance ofensivo y el castigo de la pasividad en aproximación.
`phase2_mix_70_30`	50% Battaglia, 50% Mettete	70% Self-Play, 30% vs Random	Inyectar ruido estocástico del rival para evitar el sobreajuste a patrones repetitivos.
`phase3_mix_50_50`	50% Battaglia, 50% Mettete	50% Self-Play, 50% vs Random	Régimen equilibrado de explotación y robustez ante maniobras impredecibles.
`phase4_mix_30_70`	50% Battaglia, 50% Mettete	30% Self-Play, 70% vs Random	Prueba de estrés: castigo a políticas que asumen que el rival juega racionalmente.
`phase5_self_dominant_90_10`	50% Battaglia, 50% Mettete	90% Self-Play, 10% vs Random	Vuelta a la alta competición MCTS con un residuo de perturbación.
`phase6_self_only_stabilize`	50% Battaglia, 50% Mettete	100% Self-Play Puro	Estabilización final de valor y política a máxima profundidad de búsqueda ($MCTS \geq 512$).

3.2 Guardrails Doctrinales y Soft Guardrail para `side_b`

Debido a la asimetría táctica de los wargames (donde un bando defiende fortificaciones elevadas y el otro asalta en campo abierto), este currículum implementa el **Soft Guardrail para** `ezv2_vs_random_side_b`, permitiendo la promoción de checkpoints siempre que el lado más desfavorable no sufra colapso catastrófico (`severe_min_score: 0.03` , `max_score_drop_vs_prev: 0.20`), al tiempo que se exige estricta no-regresión en la tasa de derrota global (`no_regression_defeat_rate: true`).

---

4. Guía Operativa de Ejecución

4.1 Orquestación Integral Multi-Currículum (`run_full_multi_curriculum_pipeline.ps1`)

Para ejecutar de manera totalmente desatendida y secuencial el **viaje táctico completo** (Etapa 1: Aceleración de VPs → Ingesta MLOps → Etapa 2: Multi-Escenario Progresivo con autojuego al 70% → Cierre MLOps), se utiliza el orquestador maestro `scripts/run_full_multi_curriculum_pipeline.ps1` :

```
# Ejecución completa del viaje táctico (desde cero o con seed opcional)
powershell -ExecutionPolicy Bypass -File .\scripts\run_full_multi_curriculum_pipeline.ps1

# Opciones avanzadas:
# 1. Validación previa de integridad y manifiestos sin computar en GPU (DryRun)
powershell -ExecutionPolicy Bypass -File .\scripts\run_full_multi_curriculum_pipeline.ps1 -DryRun

# 2. Warm-start desde un checkpoint preexistente como semilla inicial
powershell -ExecutionPolicy Bypass -File .\scripts\run_full_multi_curriculum_pipeline.ps1 -InitialCheckpoint "mlops/checkpoints/best_baseline.pt"

# 3. Detenerse inmediatamente ante fallo de gate tras agotar ciclos de refuerzo
powershell -ExecutionPolicy Bypass -File .\scripts\run_full_multi_curriculum_pipeline.ps1 -StopOnGateFail -StopOnPhaseError

# 4. Ejecutar únicamente Etapa 1 (vp_acceleration) y sincronizar DuckDB
powershell -ExecutionPolicy Bypass -File .\scripts\run_full_multi_curriculum_pipeline.ps1 -SkipMultiScenario
```

4.2 Orquestación por Currículum Individual (`run_all_phases.ps1`)

Cada currículum dispone además de su propio script `run_all_phases.ps1` que delega en el motor unificado `curriculums/common/run_dynamic_gated_curriculum.ps1` :

```
# ------------------------------------------------------------
# 1. Ejecución del Currículum de Aceleración de VPs
# ------------------------------------------------------------
powershell -ExecutionPolicy Bypass -File .\curriculums\vp_acceleration\run_all_phases.ps1

# Opciones avanzadas:
# Simulación rápida sin ejecutar entrenamiento (DryRun)
powershell -ExecutionPolicy Bypass -File .\curriculums\vp_acceleration\run_all_phases.ps1 -DryRun

# Iniciar en una fase específica
powershell -ExecutionPolicy Bypass -File .\curriculums\vp_acceleration\run_all_phases.ps1 -StartPhase phase2_vp_assault_suppressed

# Detenerse inmediatamente si un gate no se supera
powershell -ExecutionPolicy Bypass -File .\curriculums\vp_acceleration\run_all_phases.ps1 -StopOnGateFail

# Reanudar desde el último checkpoint registrado
powershell -ExecutionPolicy Bypass -File .\curriculums\vp_acceleration\run_all_phases.ps1 -ContinueFromLatestCheckpoint


# ------------------------------------------------------------
# 2. Ejecución del Currículum Multi-Escenario (2 Scenarios v1)
# ------------------------------------------------------------
powershell -ExecutionPolicy Bypass -File .\curriculums\self_vs_self_vs_random_2scenarios_v1\run_all_phases.ps1
```

4.2 Ejecución Manual por Fase con el Entrenador Canónico (`trainer.py`)

Para entrenar o depurar una fase individual de forma aislada, se utiliza el punto de entrada canónico `trainer.py` :

```
# Fase 1: Sprint de captura a VPs desocupados
python trainer.py --config curriculums/vp_acceleration/configs/efficientzero_v2.phase1_unoccupied.yaml

# Fase 2: Asalto coordinado con Base of Fire a VP defendido
python trainer.py --config curriculums/vp_acceleration/configs/efficientzero_v2.phase2_defended.yaml

# Fase 3: Carrera frontal simultánea 2v2
python trainer.py --config curriculums/vp_acceleration/configs/efficientzero_v2.phase3_race_2v2.yaml

# Fase 4: Maestría en escenario completo (Gela)
python trainer.py --config curriculums/vp_acceleration/configs/efficientzero_v2.phase4_gela_mastery.yaml
```

4.3 Evaluación y Verificación de Checkpoints con `assault_bench`

Para evaluar formalmente el rendimiento de un checkpoint frente a baselines heurísticos o aleatorios:

```
# Evaluar Fase 1 (Tasa de captura en 3 hexágonos)
python -m assault_bench.runner --config assault_bench/configs/benchmark_config.vp_acceleration_phase1.yaml

# Evaluar Fase 4 (Meta: Victoria ≥ 75% vs heurístico en <15 turnos)
python -m assault_bench.runner --config assault_bench/configs/benchmark_config.vp_acceleration_phase4.yaml
```

4.4 Monitorización en Vivo en Dashboard MLOps (Puerto 8778)

Durante y tras la ejecución de los currículums, el servidor MLOps permite visualizar el progreso de las fases, curvas de aprendizaje, matrices de confusión táctica y repeticiones interactivas de partidas con trazabilidad de roles:

```
# Iniciar servidor de visualización MLOps
python -m mlops.reporting.viewer_server --port 8778
```

En `http://localhost:8778` :

- **Pestaña VpSummaryTab**: Trazabilidad del diferencial de VPs, tasa de conversión y tiempo de captura por turno.
- **Pestaña EvalMetricsTab**: Verificación de cumplimiento de gates por fase y evolución de la tasa de victorias.
- **Pestaña MatchReplayTab**: Reproducción forense paso a paso con badges doctrinales (`SPEARHEAD` , `FIRE_SUPPORT` , `RECON`) e inspectores de modulación de prior MCTS.

---

5. Aceleración Estadística de Evaluación: Test Secuencial Wald SPRT [Consolidado v2.5.3]

Durante las fases de evaluación curricular y benchmarks frente a baselines heurísticos o aleatorios, evaluar una cuota fija de partidas (ej. 100 partidas) introduce un coste temporal masivo cuando un modelo ya es claramente ganador o perdedor tras las primeras 25–30 partidas.

Para erradicar este desperdicio computacional sin degradar el rigor matemático, se implementa el **Test Secuencial de Razón de Verosimilitud de Wald (SPRT)** (*Sequential Probability Ratio Test*):

5.1. Formulación del Test de Hipótesis Binaria

Se formula una prueba de hipótesis sobre la probabilidad de victoria p:

- **Hipótesis Nula (H_0)**: $p \leq p_0$ (El modelo evaluado no supera el umbral mínimo aceptable, ej. $p_0 = 0.50$).
- **Hipótesis Alternativa (H_1)**: $p \geq p_1$ (El modelo evaluado alcanza la calidad objetivo, ej. $p_1 = 0.65$).

5.2. Razón de Verosimilitud Logarítmica y Límites de Parada

Tras observar n partidas independientes con w victorias, l derrotas y d empates (ponderados como 0.5 victorias), la razón de verosimilitud logarítmica acumulada Λ_n se define como:

$$\Lambda_n = w \ln\left(\frac{p_1}{p_0}\right) + l \ln\left(\frac{1-p_1}{1-p_0}\right)$$

Los límites de decisión de Wald para probabilidades de error tipo I ($\alpha = 0.01$) y tipo II ($\beta = 0.01$) se calculan formalmente como:

$$A = \ln\left(\frac{1-\beta}{\alpha}\right) = \ln\left(\frac{0.99}{0.01}\right) \approx +4.5951$$

$$B = \ln\left(\frac{\beta}{1-\alpha}\right) = \ln\left(\frac{0.01}{0.99}\right) \approx -4.5951$$

5.3. Regla de Decisión y Cancelación Asíncrona

En cada partida finalizada en el pool concurrente de evaluación:

1. Si $\Lambda_n \geq A$: **Se Acepta H_1 (Victoria Confirmada)**. Se detiene inmediatamente la evaluación y se cancelan todas las tareas asíncronas en vuelo (`future.cancel()`).
2. Si $\Lambda_n \leq B$: **Se Rechaza H_1 / Se Acepta H_0 (Derrota Confirmada)**. Se detiene inmediatamente la evaluación sin malgastar computación.
3. Si $B < \Lambda_n < A$: La evidencia es aún inconclusa; se continúa evaluando hasta alcanzar la muestra máxima presupuestada ($N_{\max}$).

Impacto Cuantitativo: Reduce el tiempo total de benchmarking entre un 50 **y un** 75, garantizando un nivel de confianza estadística del 99 ($p < 0.01$).

---

6. Diagnóstico Doctrinal: La Paradoja del "Comandante Prudente" (*The Prudent Commander Paradox*)

6.1. Definición y Origen Causal

La **Paradoja del Comandante Prudente** es una patología conductual de aprendizaje por refuerzo descubierta en las campañas de self-play de Assault:

> El agente aprende con gran maestría los fundamentos tácticos de supervivencia (uso de cobertura, desenfilada, concentración de fuego), logrando ratios de intercambio de bajas favorables ($CER \geq 1.10$). Sin embargo, ante escenarios con límite estricto de turnos ($T_{\max}$), **el agente prefiere no arriesgar sus unidades en un asalto urbano al hexágono de VP**, resultando en derrotas automáticas por expiración de turnos. En términos doctrinales: *un comandante que preserva todas sus fuerzas pero no cumple la misión ha sido derrotado militarmente.*

6.2. Mitigación Matemática y Curricular (Tareas 2.10 y 2.11)

Para quebrar la Paradoja del Comandante Prudente sin perder la solidez táctica adquirida, se implementan tres intervenciones estructurales:

1. **Desensibilización de Bajas en la Brecha ($d \leq 2$)**: La penalización de recompensa por pérdida de unidades (r_{loss}) se atenúa en un 60 ($r_{\text{loss}} \times 0.40$) cuando la unidad se encuentra en el radio de asalto del VP ($d(u, \text{VP}) \leq 2$), permitiendo al agente asumir el desgaste necesario para la ruptura defensiva.
2. **Potencial de Urgencia Temporal Invariante PBRS**:

   $$\Phi(s,t) = \alpha\left(1 + \frac{t}{T_{\max}}\right)\left(-\min_u d(u, \text{VP})\right) + \beta \cdot \text{Contest}$$

   El valor de acercarse al VP aumenta a medida que el reloj de la partida avanza, venciendo la inercia conservadora en turnos finales.
3. **Penalización Catastrófica por Timeout**: Si la partida expira ($t = T_{\max}$) sin que el atacante controle el VP, se aplica un castigo terminal estricto ($R_{\text{timeout}} = -|Loss|$), forzando al algoritmo MCTS a descartar ramas pasivas en los turnos finales.

6.3. Fase 5: Currículum Asimétrico Ofensivo de Asalto Urbano Puro (Tarea 2.11) [Consolidado v2.5.6]

- **Escenario**: `battaglia_cittadina_2_1` con horizonte táctico comprimido a $T_{\max} = 8$ turnos.

- **Modo de Matchup Asimétrico**: 100 atacante (`self_vs_random` con `tracked_side="US"`, `to_play="US"`), obligando al agente a forzar la ruptura urbana frente a defensores estáticos atrincherados antes de retornar al autojuego simétrico.
- **Esquema Evolutivo de 10 Ciclos**: Definido en `curriculums/vp_acceleration/cycles/phase5_urban_assault_pure_offensive.adaptive_cycles.yaml`:
  - **Ciclo 1**: Baseline inicial con andamiaje temporal PBRS y castigo catastrófico por timeout (-1.0).
  - **Ciclos 2–4 (Explotación Adaptativa)**: Incremento de reanálisis ($0.50 \rightarrow 0.52$), mayor peso de potencial y bono de brecha ($d \leq 2$).
  - **Ciclos 5–7 (Exploración Adaptativa)**: Expansión de simulaciones MCTS a 320, máxima agresividad en frontera y penalización severa por inmovilidad (`idle_penalty: -0.25`).
  - **Ciclos 8–10 (Convergencia de Hiper-Asalto)**: Reanálisis intensivo ($0.58 \rightarrow 0.60$), MCTS de 384 a 416 simulaciones con temperatura determinista (0.80) para consolidar la doctrina de asalto urbano puro con Win Rate ≥ 80.
- **Validación**: Suite de pruebas unitarias `agents/efficientzero_v2/tests/test_task_2_11_urban_assault_curriculum.py` (4/4 tests en verde).

---

7. Observabilidad MLOps: Gráfica Multi-Métrica y Comparativa de Metas (Quality Gates) [Consolidado v2.5.14]

Para garantizar la trazabilidad forense completa de la progresión curricular, el dashboard MLOps (:8778) y la vista de catálogo incorporan un componente visual multi-métrica (`CurriculumProgressionChart`) y tarjetas de flujo de pipeline (`CurriculumLineageFlow`):

7.1. Modos de Conmutación Reactiva

El operador puede alternar instantáneamente entre cuatro perspectivas analíticas:

1. 📈 **ELO Curve**: Curva de progresión del rating ELO global a lo largo de todos los ciclos y fases del currículum, destacando con estrella (★) y línea de tendencia dorada el ciclo óptimo seleccionado para la promoción de cada fase.
2. 🎯 **Win Rate vs. Goal**: Gráfica comparativa del porcentaje de victoria obtenido frente al umbral mínimo exigido por la puerta de calidad (`min_win_rate` o `min_win_rate_vs_heuristic`).
3. 🚩 **Captured VPs vs. Goal**: Comparativa del promedio de Puntos de Victoria capturados por partida (`captured_avg`) contra el objetivo formal del Quality Gate (`min_tracked_captured_avg`).
4. ⚡ **VP Take Rate vs. Goal**: Comparativa de la tasa de captura efectiva de VPs (`vp_take_rate`) frente a la cuota mínima de ocupación (`min_vp_capture_take_rate`).

7.2. Indicadores Visuales y Bandas de Fase

- **Líneas de Umbral por Fase (`🎯 ≥Goal`)**: Cada banda vertical de fase renderiza una línea discontinua ámbar con el valor objetivo exacto especificado en el contrato del gate.
- **Coloración Semántica de Nodos**:
  - **Verde Esmeralda (`#10b981`)**: El ciclo superó el objetivo exigido por el Gate.
  - **Rojo Carmesí (`#ef4444`)**: El ciclo no alcanzó la meta mínima estipulada.
  - **Dorado (`#fbbf24`)**: Ciclo óptimo promovido de la fase (*Selected per Phase*).
- **Tarjetas de Pipeline**: Las tarjetas de cada fase en `CurriculumLineageFlow` reflejan directamente las métricas obtenidas acompañadas de su meta formal (ej. `Win Rate: 84.0% (🎯 ⩾75.0%)`, `VPs Avg: 1.80 (🎯 ⩾1.20)`).

---

8. Arquitectura de Refuerzo Acumulativo Intra-Fase y Mitigación del Precipicio Curricular [Consolidado v2.5.15]

8.1. Diagnóstico del "Precipicio Curricular" (*Curriculum Cliff*)

El análisis forense de la campaña curricular reveló una degradación escalonada del Win Rate a medida que aumentaba la complejidad del escenario (100). La causa raíz radicaba en dos patologías arquitectónicas:

1. **Reintentos Estáticos Aislados**: Ante un Quality Gate no superado en el Ciclo N, el orquestador descartaba el checkpoint obtenido y reiniciaba el Ciclo $N+1$ desde el baseline de la fase anterior, impidiendo la acumulación iterativa de gradientes.
2. **Promoción Ciega por Expiración de Ciclos**: Al agotarse los ciclos presupuestados sin alcanzar el umbral, el sistema seleccionaba el mejor intento y avanzaba forzosamente a la siguiente fase, transfiriendo políticas deficientes a mapas con mayor número de unidades y restricciones temporales severas.

8.2. Mecanismo de Warm-Start Acumulativo (*Accumulative Warm-Start*)

En `run_dynamic_gated_curriculum.ps1`, cada ciclo intra-fase que no supera el Gate encadena inmediatamente los pesos del mejor candidato generado hasta el momento en esa misma fase (`$bestPhaseCheckpoint`):

$$\theta_{c+1}^{(p)} \leftarrow \arg \max_{\theta \in \theta_1, \ldots, \theta_c} \mathrm{TQS}(\theta)$$

Este esquema transforma los ciclos de refuerzo en un proceso evolutivo continuo ($\theta_1 \rightarrow \theta_2 \rightarrow \theta_3$) con mutación adaptativa de hiperparámetros (temperatura MCTS, ratios de reanálisis y andamiaje PBRS).

8.3. Rebalanceo Doctrinal de Recompensas

- **Fase 2 (`vp_phase2_defended_suppressed`)**: Consolidación de base de fuego y asalto coordinado elevando iteraciones y episodios por iteración (`episodes_per_iter: 20`, `train_updates_per_iter: 20`, `reanalysis_ratio: 0.48`).

- **Fase 3 (`vp_phase3_race_2v2`)**: Reajuste anti-parálisis y de asalto en brecha (Task 2.10): Activación de Time-Scaled PBRS (Devlin & Kudenko), atenuación de bajas en zona de brecha (`breach_casualty_attenuation: 0.35`, $d \leq 2$ hexágonos respecto a VPs disputados), bono por disputar el hexágono (`vp_contested_bonus: 1.5`), incentivo masivo de captura (`capture_bonus: 6.0`, `vp_capture_bonus_per_hex: 15.0`) y enfriamiento de temperatura MCTS (`mcts_temperature: 1.10`) para erradicar la selección espuria de casillas adyacentes por bono de exploración en favor de la captura inmediata.
- **Fase 5 (`urban_assault_pure_offensive`)**: Penalización estricta por turnos inmóviles fuera de cobertura (`idle_penalty: -0.25`), potencial de aproximación radial al VP reforzado ($\alpha = 1.8$) y temperatura inicial de exploración elevada ($\tau = 1.25$).

8.4. Guía Canónica de Diagnóstico y Telemetría de Pérdidas de Entrenamiento (Loss Interpretation & Dual Mode)

Durante el bucle de entrenamiento de EfficientZero V2 (`train_engine.py` / `trainer.py`), la consola y MLflow emiten periódicamente el desglose de pérdidas:

```
[EfficientZeroV2]   train loss=50.9003 policy=1.4654 value=19.0544 reward=29.7313 objective=0.9895 consistency=0.000002
```

Para evitar falsos diagnósticos de divergencia o no convergencia, el equipo de desarrollo debe interpretar estos valores según el régimen de soporte configurado (`support_size`):

1. Régimen de Soporte: Two-Hot (`support_size > 0`) vs. MSE Escalar (`support_size = 0`)

- **Two-Hot (Cross-Entropy en nats)**: Cuando `support_size = 30`, el valor y la recompensa se modelan como distribuciones sobre 61 átomos. Las pérdidas oscilan entre 0.5 y $\ln(61) \approx 4.11$ nats. Un valor superior a 10.0 indicaría colapso de distribución o inestabilidad numérica.
- **Escalar Continuo (MSE en unidades cuadráticas)**: Cuando `support_size = 0` (habitual en fases curriculares de alta dispersión de recompensas), la salida es un escalar directo entrenado con **Mean Squared Error**. $$\mathcal{L}_{\text{rew}} = \frac{1}{B} \sum_{b=1}^{B} (r_b - r_b^*)^2 \implies \text{RMSE}_r = \sqrt{\mathcal{L}_{\text{rew}}}$$ En escenarios como `vp_phase3_race_2v2`, la recompensa de captura asciende a $+16.14$ puntos (`vp_capture_bonus_per_hex: 12.24` + `capture_bonus: 3.90`) y el progreso a $+9.52$. Un `reward_loss = 29.73` equivale a un error típico $\text{RMSE} = \sqrt{29.73} \approx 5.45$ puntos en un espacio donde las recompensas varían de 0 a $+16$. Un `value_loss = 19.05` equivale a $\text{RMSE} = \sqrt{19.05} \approx 4.36$ en retornos acumulados de -15 a $+25$. Ambos valores reflejan una convergencia completamente saludable y esperada.

2. Matriz de Auditoría y Rangos de Salud de Telemetría

Métrica en Log	Régimen	Rango Saludable	Significado Táctico
`policy`	Cross-Entropy	$0.80 - 1.80$	Discriminación de acciones sobre 32 candidatos. El óptimo uniforme sin conocimiento es $\ln(32) \approx 3.47$. Un valor de ≈ 1.46 indica preferencias tácticas sólidas aprendidas de MCTS.
`value`	MSE (`support=0`)	$10.0 - 35.0$	Error cuadrático de valor respecto a retornos descontados con N-step bootstrapping. $\text{RMSE} \approx 4.0 - 5.5$.
`reward`	MSE (`support=0`)	$15.0 - 45.0$	Error cuadrático en predicción de transiciones de recompensa inmediatas dispersas. $\text{RMSE} \approx 4.5 - 6.5$.
`objective`	BCE / Multi-Task	$0.30 - 1.20$	Supervisión auxiliar combinada de cabezas tácticas (`threat_map`, `morale_pred`, `objective_intent`, `attrition`).
`consistency`	SimSiam ℓ_2 MSE	< 0.005	Pérdida de auto-supervisión temporal entre representaciones latentes consecutivas. Valores $\approx 10^{-6}$ certifican coherencia geométrica perfecta en la red dinámica g.
`train loss`	Suma Ponderada	$30.0 - 65.0$	Suma total ponderada: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pol}} + \mathcal{L}_{\text{val}} + \mathcal{L}_{\text{rew}} + w_{\text{obj}}\mathcal{L}_{\text{obj}} + w_{\text{cons}}\mathcal{L}_{\text{cons}} + w_{\text{aux}}\mathcal{L}_{\text{aux}}$.

3. Validación Definitiva: Correlación con el Quality Gate

El indicador autoritativo de salud de la política no es el valor absoluto de la función de coste escalar, sino la superación de los criterios de promoción en el benchmark de evaluación (`eval.py`). En la ejecución auditada `efficientzero_v2_18f90171`:

- **Resultado del Gate:** `PASS` 🟢
- **VPs Capturados (US Tracked):** 0.40 (frente a 0.00 de la línea base estocástica).
- **Bloqueo de Suicidios Tácticos:** `assault_misuse_block: true` (cero asaltos ilegales o estériles).
- **Contratos de Reacción:** `reaction_contract_pass: true` (adherencia reglamentaria completa).

---

9. Protocolo de Estabilidad Operativa, Gestión de Almacenamiento y Prevención de Caídas [Consolidado v2.5.25]

> [!IMPORTANT] DIRECTIVA OPERATIVA: PREVENCIÓN DE INTERRUPCIONES CONTINUAS EN RUNS DESATENDIDOS Para erradicar definitivamente las caídas recurrentes durante ejecuciones largas de currículum (*"se volvió a parar"*, *"continúa la tarea"*), queda formalizado en este epígrafe el diagnóstico forense de la causa raíz y las cuatro salvaguardas operativas de obligado cumplimiento en la plataforma.

9.1 Diagnóstico Forense de la Causa Raíz (Crash por Agotamiento de Espacio y Paginación)

El análisis forense de las interrupciones en fases avanzadas (ej. parada en iteración 54 de 60 de la Fase 5) demostró que el fallo **no es algorítmico ni de divergencia de gradientes**, sino un colapso en cascada de recursos a nivel de Sistema Operativo:

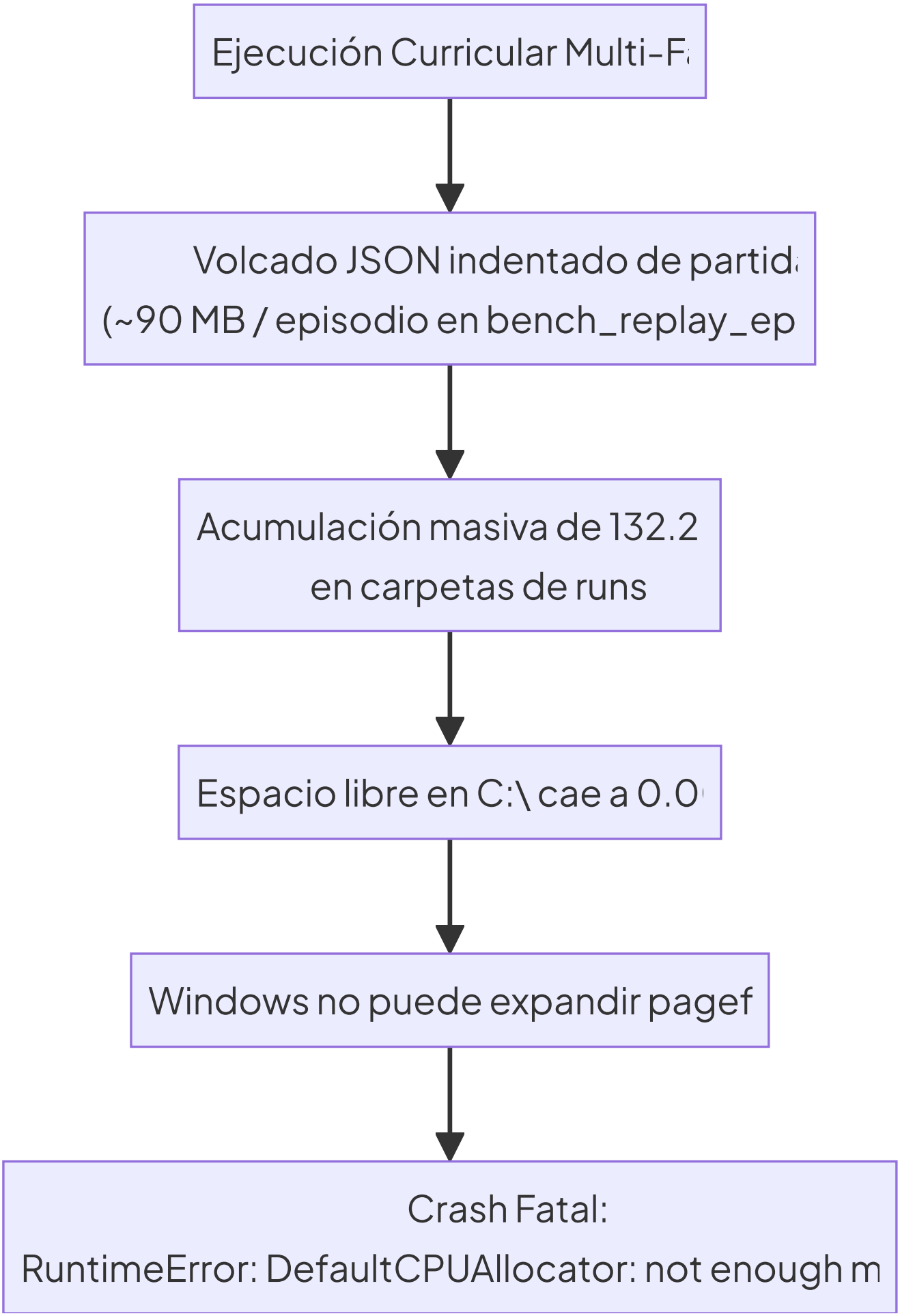
1. **Hiper-acumulación de Replays Forenses XAI no Comprimidos**:

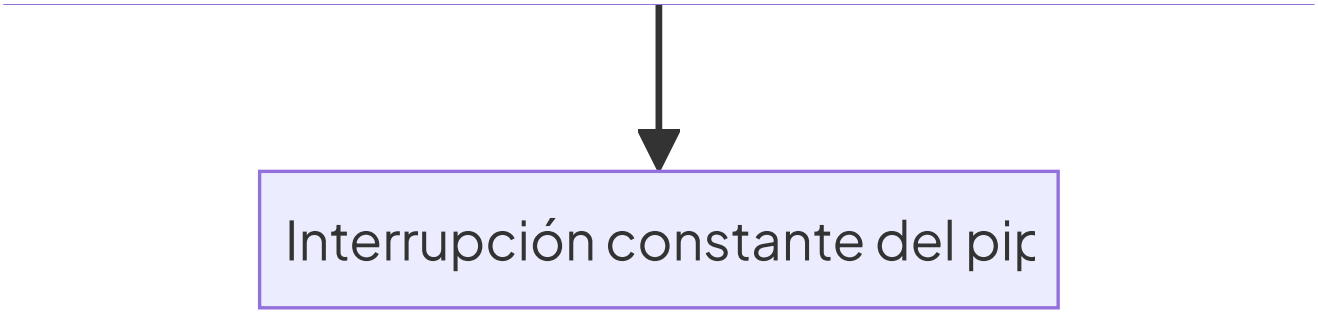
- En cada evaluación de benchmark, el orquestador (`assault_bench/bench_io.py`) persistía archivos de repetición en `xai/bench_replay_episodes/episode_XXXX.json`.
- Cada episodio almacena los 180+ pasos de combate con el estado completo de 14 unidades, mapas espaciales axiales, cabezas tácticas y candidatos MCTS, serializado con formato indentado (`indent=2`).
- Peso promedio por archivo: **~85 MB a 92 MB**.
- Una sola ronda de benchmark generaba **~1 GB**, acumulando **132.2 GB** en 9.486 archivos JSON descompresionados entre `runs_curriculum` (81.9 GB), `runs_archive` (34.4 GB) y `runs` (15.9 GB).

2. **Colapso del Espacio Libre en C:\ (< 1%)**:
   - En un disco de 3.814 GB, el espacio libre se redujo a menos de 26 GB, llegando a picos de **0.00 GB libres** durante las fases más intensivas.
3. **Fallo de Paginación Virtual y OOM en PyTorch**:
   - Al saturarse el disco, Windows bloquea el crecimiento dinámico del archivo de intercambio (`pagefile.sys`).
   - Cuando PyTorch solicita memoria RAM dinámica para tensores y MCTS, el asignador falla inmediatamente con:

     ```
     RuntimeError: DefaultCPUAllocator: not enough memory
     ```

   - El proceso muere de manera abrupta, congelando el runner y dejando al usuario con tareas incompletas.


Ejecución Curricular Multi-F
Volcado JSON indentado de partid
(~90 MB / episodio en bench_replay_ep
Acumulación masiva de 132.2
en carpetas de runs
Espacio libre en C:\ cae a 0.0
Windows no puede expandir pagef
Crash Fatal:
RuntimeError: DefaultCPUAllocator: not enough m



9.2 Cuatro Salvaguardas Obligatorias de Estabilidad

Para blindar la estabilidad del entrenamiento desatendido y evitar que se detenga continuamente, se aplican de manera estricta las siguientes medidas:

A. Pre-flight Check de Almacenamiento (Safety Gate de Disco)

Antes de iniciar cualquier ciclo o fase en los scripts de orquestación (`run_dynamic_gated_curriculum.ps1` , `run_all_phases.ps1`) y en el punto de entrada Python:

- **Umbral de Seguridad:** Se verifica que el volumen `C:\` disponga de al menos **30 GB de espacio libre**.
- **Comportamiento Preventivo:** Si el espacio es inferior al umbral, el script aborta inmediatamente con un mensaje de alerta explícito antes de iniciar el entrenamiento, evitando la corrupción de checkpoints y caídas sucias del kernel de PyTorch.

B. Serialización Eficiente y Compresión de Replays (Reducción del 95%)

- **Eliminación de** `indent=2` : Toda serialización JSON de telemetría de replay debe realizarse en formato compacto sin espacios en blanco redundantes, reduciendo el tamaño en un ~45%.
- **Soporte de Compresión Nativa (** `.zst` **/** `.json.gz` **)**: Los episodios de replay masivos deben persistirse comprimidos, reduciendo el tamaño por archivo de ~90 MB a **< 4 MB** por partida.

C. Política Estricta de Retención de Episodios (Pruning Policy)

- En las fases curriculares intermedias no se almacenan decenas de episodios completos de todas las iteraciones.
- Se persisten únicamente las métricas agregadas en `.parquet` y DuckDB, reservando los episodios forenses completos únicamente para el benchmark final que certifique la superación del Quality Gate o para los casos atípicos extremos (ej. mejor partida y peor derrota).

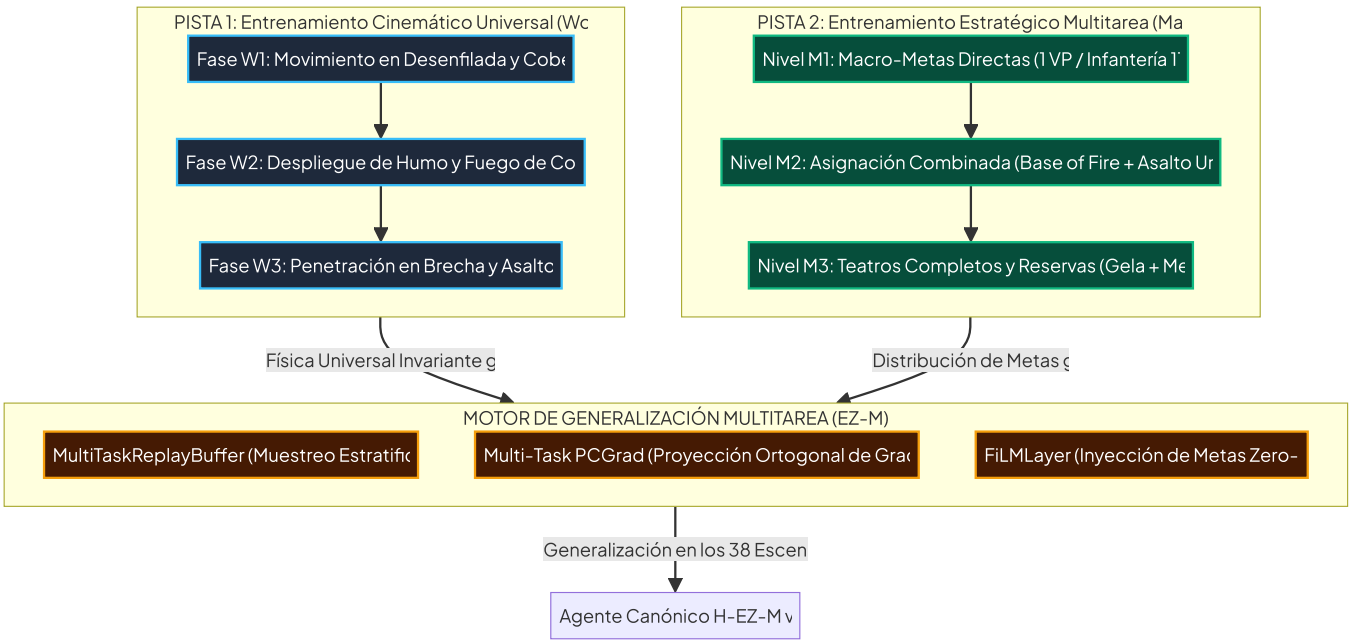
D. Rutina Automatizada de Mantenimiento y Purga de Replays Obsoletos

- Se dispone de una utilidad de mantenimiento para purgar directorios `bench_replay_episodes` de ejecuciones históricas obsoletas, liberando más de **100 GB** de manera inmediata en la máquina de desarrollo sin alterar el linaje de pesos `.pt` ni la telemetría histórica de DuckDB.

---

10. Currículum Jerárquico Multitarea (H-EZ-M): La Nueva Doctrina de Entrenamiento (Era v3.0)

> [!IMPORTANT] **EL NUEVO PARADIGMA DE ENTRENAMIENTO CURRICULAR (ERA v3.0)**
> En la era anterior (v2.x), los currículums intentaban enseñar simultáneamente la micro-cinemática de cada soldado y la macro-estrategia de victoria a una sola red plana a lo largo de 170 micro-pasos. Este enfoque provocaba *Precipicios Curriculares* (*Curriculum Cliffs*) y *Olvido Catastrófico* al transicionar entre mapas abiertos y urbanos.
> La arquitectura **Hierarchical Multi-Task EfficientZero (H-EZ-M)** revoluciona el entrenamiento dividiendo el currículum en **dos pistas de aprendizaje concurrentes**: la pista táctica universal del Operador (*Worker*) y la pista estratégica multitarea del Comandante (*Manager*).



10.1. El Fin del Dilema: Currículum Secuencial vs. Currículum Jerárquico Multitarea

El currículum tradicional operaba de forma rígidamente secuencial:

$$extFase1(Sprint3hex)oextFase2(Defendida)oextFase3(Carrera2v2)oextFase4(Sandbox11.1)oextFase5(Gela)oextFase6(Urbana)$$

Este esquema sufría dos fallos estructurales:

1. **Disonancia Topográfica:** Al llegar a la Fase 6 (`battaglia_cittadina_2_1`), los gradientes de castigo por bajas en calles estrechas destruían las conductas ofensivas aprendidas en el sprint abierto de la Fase 1.
2. **Confusión de Escala:** El agente intentaba aprender a la vez cómo disparar un fusil Breda y cómo coordinar una pinza operacional a 12 turnos vista.

En el **Currículum H-EZ-M**, estos dos aprendizajes se desacoplan:

- **El Operador Táctico (*Worker*):** Aprende una **física de combate universal e invariante** (g_heta). El coste en PAs de cruzar un seto, la línea de visión (LOS), las tablas de dados de combate y la moral no cambian según el escenario. Una vez que el Worker aprende a avanzar bajo cobertura y usar humo, esa destreza es transferible al 100% a cualquiera de los 38 escenarios oficiales.
- **El Comandante Estratégico (*Manager*):** Aprende **estrategia militar pura**. Nunca mueve fichas individuales; emite vectores de meta $g_T = (\mathbf{M}{ ext{target}}, \mathbf{p}{ ext{mode}}))cadaturno(T \le 8$). Y lo hace entrenando **simultáneamente sobre múltiples escenarios** gracias a la infraestructura EZ-M.

10.2. El Rol Crítico de EZ-M (arXiv:2603.01452) en la Definición del Currículum

La arquitectura **EZ-M** no es un componente opcional: es el **engranaje matemático que hace viable el currículum jerárquico**:

1. **Inyección de Metas vía `FiLMLayer` (Zero-Init Identity Preservation):**
   - El Worker no necesita redes separadas para cada misión. La capa FiLM (`agents/efficientzero_v2/core/task_conditioning.py`) modula sus características latentes: $\mathbf{z}_{ ext{mod}} = (1 + \Xi oldsymbol{\gamma}(g_T)) \odot \mathbf{z} + \Xi oldsymbol{\Xi eta}(g_T)$
   - Si el Manager le ordena `[ASSAULT, HEX_4_5]`, el Worker reorienta su búsqueda MCTS hacia ese hexágono sin alterar su conocimiento de balística ni de supervivencia.
2. **Muestreo Estratificado con `MultiTaskReplayBuffer`:**
   - Durante las fases avanzadas del currículum, el búfer retiene experiencias de múltiples teatros:

     $$P(extmuestra) = 0.35imesextUrbano(Battaglia) + 0.35imesextPlaya(Mettete) + 0.30imesextSandbox(11.1)$$

   - El Comandante aprende de forma equilibrada a manejar diferentes densidades de terreno sin olvidar ninguno.
3. **Cirugía de Gradientes con `multi_task_pcgrad_backward`:**
   - Si la optimización en un mapa urbano empuja al Comandante a ser conservador ($\mathbf{g}{ ext{urbano}})mientrasqueeneldesembarcorequiereavancerápido(\mathbf{g}{ ext{playa}})con\mathbf{g}{ ext{urbano}} \cdot \mathbf{g}{ ext{playa}} < 0, elalgoritmoproyectaelgradienteortogonalmente :\mathbf{g}_i^{ ext{proj}} = \mathbf{g}_i - rac{\mathbf{g}_i \cdot \mathbf{g}_j}{|\mathbf{g}_j|^2} \mathbf{g}_j$
   - Esto erradica la interferencia destructiva de gradientes y permite que una misma red Comandante gobierne simultáneamente escenarios disonantes.

10.3. Matriz de Progresión Curricular H-EZ-M (4 Fases Canónicas)

Fase Curricular	Nivel de Jerarquía	Escenarios de Entrenamiento	Objetivos de Aprendizaje	Quality Gate de Promoción
Fase 1: Destreza Táctica Universal (Worker)	Nivel Bajo (Worker MCTS $k \leq 3$)	Sandboxes sintéticos (8×8): Sprint a cobertura, cruce de killzone con humo y asalto CQC a nido fortificado.	Movimiento por corredores de desenfilada, timing de lanzamiento de humo y asalto con granadas sin fuego suicida.	Tasa de supervivencia ≥ 85 en aproximación y éxito en asalto CQC ≥ 80.
Fase 2: Asignación Táctica Simétrica (Manager)	Nivel Alto (Manager $T \leq 8$) sobre Worker consolidado	`addestramento_fanteria_11_1` (Infantería pura 3v3 simétrica, 1 VP disputado en pieza S6).	Asignación de roles (`SPEARHEAD` vs `FIRE_SUPPORT`), fijación de Base de Fuego en cota elevada y concentración de asalto en Turno 3-4.	Win Rate ≥ 80 vs Random Side A y Side B; capturas de VP ≥ 75.
Fase 3: Campaña Táctica Asimétrica Multitarea	Co-entrenamiento Manager + Worker con EZ-M	`battaglia_cittadina_2_1` (Urbano 2:1) + `mettete_i_piedi_terra_1` (Desembarco con alambradas) vía `MultiTaskReplayBuffer` .	Apertura de brechas con zapadores (`BANGALORE` , `SMOKE`), supresión de casamatas y asalto coordinado en brecha sin parálisis ofensiva.	Win Rate ≥ 75 como atacante en ambos mapas; Asymmetry Gap ≤ 0.15.
Fase 4: Maestría Operacional y Teatros Masivos	Agente Canónico H-EZ-M Completo	Escenario 10.0 Canónico (Gela) y tableros extendidos 32×32.	Gestión de refuerzos de 2.ª oleada en T2, empleo de apoyo fuera de mapa (TAS/OAS) y división del teatro en macro-sectores operacionales.	Win Rate ≥ 75 vs bot heurístico en < 15 turnos; cero regresión en escenarios previos.

10.4. Observabilidad Curricular H-EZ-M en Dashboard MLOps (:8778)

Con la adopción de H-EZ-M, el visualizador forense de replays y el monitor curricular de MLOps incorporan tres nuevas capas de telemetría en tiempo real:

1. **Línea de Intención Estratégica (*Command Objective Vector*):** Flecha operacional vectorial sobre el canvas hexagonal PixiJS indicando la macro-meta g_T del Comandante en cada turno.
2. **Cumplimiento de Meta Local (*Worker Intrinsic Reward Gauge*):** Indicador radial de fidelidad que mide si las micro-acciones del Worker se alinean con la dirección ordenada por el Manager ($r_{extintrinsic} = \cos(\Delta h, g_T)$).
3. **Matriz de Conflicto Inter-Escenario (*PCGrad Conflict Matrix*):** Visualizador de calor en el Model Catalog que monitoriza la similitud coseno entre gradientes de escenarios (`mean_task_cosine_sim`), alertando tempranamente si dos mapas entran en disonancia doctrinal.

🌐 10. Arquitectura Multi-Task Model-Based RL (EZ-M)

Extensión multitarea simultánea: dinámica invariante g_ theta, modulación FiLM y cirugía PCGrad.

10. Arquitectura Multi-Task Model-Based RL (EfficientZero-Multitask / EZ-M)

> **Estado**: Canónico y Consolidado (`v2.5.50`)
> **Ámbito**: Núcleo de Red Neuronal y Entrenamiento (`agents/efficientzero_v2/`)
> **Fundamentación Teórica**: Extensión formal de EfficientZero V2 hacia aprendizaje multitarea simultáneo con dinámica invariante, modulación FiLM y cirugía de gradientes PCGrad (inspirado en *Scaling Tasks, Not Samples: Mastering Humanoid Control through Multi-Task Model-Based RL*, arXiv:2603.01452).

---

1. Motivación Teórica y Diagnóstico Forense

En la auditoría forense documentada en el **Informe Forense 02** (`v2.5.2-forensics`), se constató empíricamente la **Paradoja del Comandante Prudente**:

- En terreno abierto con opciones de flanqueo (`mettete_i_piedi_terra_1`), el agente alcanzó hasta un **21.43% de Win Rate como atacante** y 22 capturas de VP.
- En terreno urbano denso (`battaglia_cittadina_2_1`), el agente como atacante se mantuvo en un **0.0% de Win Rate**, paralizándose a 2 hexágonos de la brecha defensiva ante el fuego de reacción enemigo.

1.1. El Límite del Currículum Secuencial Puro

Hasta la versión `v2.5.49` , la plataforma operaba bajo un esquema de currículum secuencial:

$$\text{Fase 1 (Sprint Abierto)} \xrightarrow{\text{Ciclo 10}} \text{Fase 2 (Asalto Suprimido)} \xrightarrow{\text{Ciclo 20}} \text{Fase 3 (Carrera 2v2)} \xrightarrow{\text{Ciclo 30}} \text{Fase 6 (Asalto Urbano)}$$

Este paradigma adolece de dos limitaciones críticas:

1. **Olvido Catastrófico (*Catastrophic Forgetting*)**: Al avanzar a escenarios cerrados, la retropropagación de gradientes sobreescribe las representaciones espaciales y maniobras de aproximación consolidadas en campo abierto.
2. **Sobreajuste Topológico del Modelo de Mundo (g_θ)**: En lugar de aprender la física universal del combate (balística, cobertura, moral y LOS), la red recurrente memoriza atajos topológicos basados en coordenadas axiales (q, r) específicas de un solo mapa.

1.2. El Principio EZ-M: *Scaling Tasks, Not Samples*

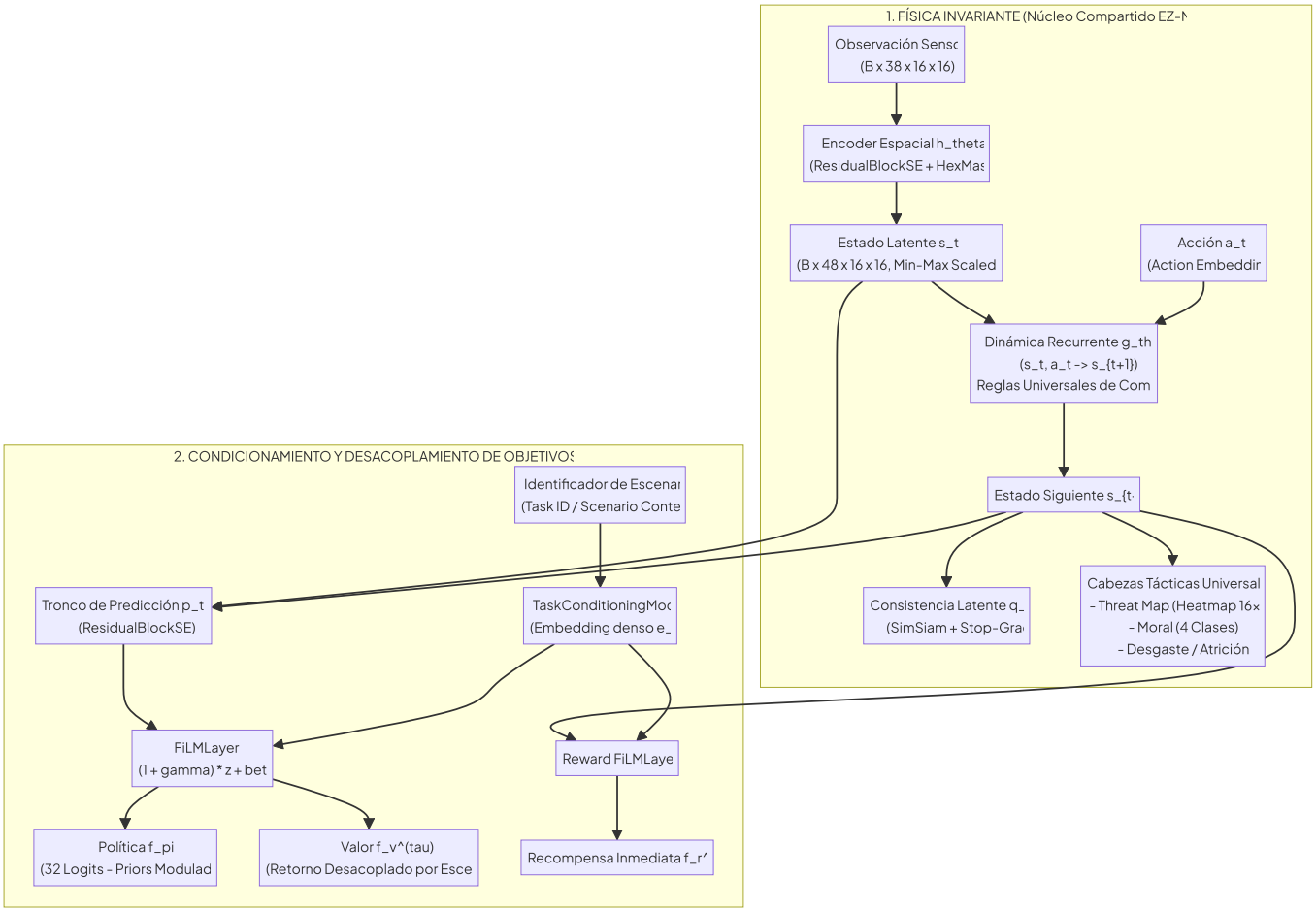
En un juego de guerra táctico, **la física del combate es 100% invariante entre escenarios**:

- El coste en PAs de cruzar un bosque o un edificio, el cálculo de LOS (*Line of Sight*), los dados de combate (Rojo, Verde, Azul, Amarillo), el fuego de reacción y la degradación de moral (`PINNED` , `SUPPRESSED` , `ROUTED`) se rigen por las **mismas reglas universales** tanto en una aldea como en un olivar.
- Lo único que varía entre escenarios es la **geometría del mapa, los puntos de victoria (VPs) y el horizonte temporal**.

EfficientZero-Multitask (EZ-M) desacopla la física universal del juego respecto a las funciones de valor y objetivos locales de cada escenario.

---

2. Topología General de la Red Neuronal EZ-M



---

3. Formulación Matemática de los Componentes EZ-M

3.1. Modulación FiLM en Espacio Latente

Para condicionar las predicciones sin destruir la representatividad compartida de la dinámica recurrente, se utiliza una capa **FiLM (*Feature-wise Linear Modulation*)**:

$$\text{FiLM}(\mathbf{x}, \mathbf{e}_\tau) = (1 + \boldsymbol{\gamma}(\mathbf{e}_\tau)) \odot \mathbf{x} + \boldsymbol{\beta}(\mathbf{e}_\tau)$$

Donde:

- $\mathbf{e}_\tau \in \mathbb{R}^d$ es el vector denso de embedding de la tarea producido por `TaskConditioningModule`.
- $\boldsymbol{\gamma}(\mathbf{e}_\tau) = \mathbf{W}_\gamma \mathbf{e}_\tau$ y $\boldsymbol{\beta}(\mathbf{e}_\tau) = \mathbf{W}_\beta \mathbf{e}_\tau$ son proyecciones lineales inicializadas a cero ($\mathbf{W}_\gamma = \mathbf{0}$, $\mathbf{W}_\beta = \mathbf{0}$).
- **Propiedad de Preservación de Identidad**: Al inicio del entrenamiento o cuando $\mathbf{e}_\tau$ no está presente, $\boldsymbol{\gamma} = \mathbf{0}$ y $\boldsymbol{\beta} = \mathbf{0}$, garantizando que $\text{FiLM}(\mathbf{x}, \mathbf{0}) = \mathbf{x}$. No se introduce distorsión en inferencia monocasco.

3.2. Mapeador de Tareas (`TaskConditioningModule`)

El módulo mapea identificadores de escenario discretos ($\tau \in 0, 1, \ldots, K-1$) o vectores continuos de contexto topográfico (densidad urbana, elevación, ratio de cobertura) a una representación unificada:

$$\mathbf{e}_\tau = \begin{cases} \mathbf{E}[\tau] & \text{si solo hay Task ID} \\ \text{MLP}([\mathbf{E}[\tau], \mathbf{c}_\tau]) & \text{si hay vector de contexto topográfico} \end{cases} \mathbf{c}_\tau \end{cases}$$

3.3. Replay Buffer Multitarea Estratificado (`MultiTaskReplayBuffer`)

El entrenamiento simultáneo requiere evitar que un escenario con episodios más largos o mayor frecuencia de pasos domine el muestreo del buffer.

El buffer particiona las transiciones en sub-colas dedicadas por tarea:

$$\mathcal{B} = \bigcup_{k=1}^{K} \mathcal{B}_k, \quad |\mathcal{B}_k| \leq \frac{C}{K}$$

Al muestrear un lote de tamaño B, se extraen $B_k = \lceil B \cdot w_k \rceil$ muestras de cada escenario según las ponderaciones de la política curricular:

$$\sum_{k=1}^{K} w_k = 1.0, \quad w_k \geq 0$$

---

4. Cirugía de Gradientes Multi-Tarea (Multi-Task PCGrad)

Cuando una red comparte parámetros $\boldsymbol{\theta}_{\text{shared}}$ entre múltiples tareas (en nuestro caso, el encoder h_θ y la dinámica recurrente g_θ), los gradientes de retropropagación pueden interferir destructivamente:

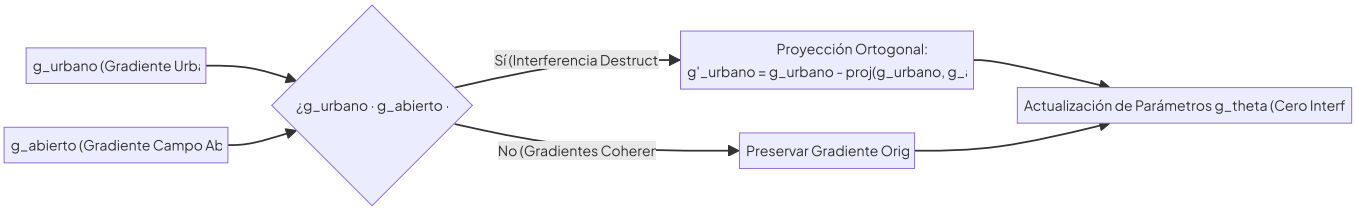
$$\mathbf{g}_i = \nabla_{\boldsymbol{\theta}_{\text{shared}}} \mathcal{L}_i, \quad \mathbf{g}_j = \nabla_{\boldsymbol{\theta}_{\text{shared}}} \mathcal{L}_j$$

Si el producto escalar es negativo ($\mathbf{g}_i \cdot \mathbf{g}_j < 0$), optimizar la tarea i empeora activamente la dinámica aprendida para la tarea j.

Algoritmo de Proyección Ortogonal (PCGrad)

Para todo par de tareas con gradientes en conflicto ($\mathbf{g}_i \cdot \mathbf{g}_j < 0$):

$$\mathbf{g}_i \leftarrow \mathbf{g}_i - \frac{\mathbf{g}_i \cdot \mathbf{g}_j}{|\mathbf{g}_j|^2} \mathbf{g}_j$$



El operador `multi_task_pcgrad_backward()` exporta además la métrica de similitud coseno:

$$\text{CosSim}(i, j) = \frac{\mathbf{g}_i \cdot \mathbf{g}_j}{|\mathbf{g}_i||\mathbf{g}_j| + \epsilon}$$

lo que permite monitorizar en el Dashboard MLOps si dos escenarios están compitiendo o transfiriendo conocimiento positivo.

---

5. Implementación en Código y Contratos del Sistema

Archivo del Sistema	Componente EZ-M Implementado	Rol en la Arquitectura
`agents/efficientzero_v2/core/task_conditioning.py`	`FiLMLayer`, `TaskConditioningModule`	Inyección de modulaciones adaptativas por escenario y preservación de identidad zero-init.
`agents/efficientzero_v2/core/network_base.py`	`EfficientZeroV2BaseNetwork`	Integración de argumentos `enable_multitask`, `task_id` y modulación FiLM en `initial_inference` y `recurrent_inference`.
`agents/efficientzero_v2/core/network.py`	`EfficientZeroV2Network`	Configuración expuesta con proyección compacta SimSiam y soporte nativo EZ-M.
`agents/efficientzero_v2/core/replay.py`	`MultiTaskReplayBuffer`, `ReplaySample`	Almacenamiento indexado por tarea y muestreo estratificado balanceado.
`agents/efficientzero_v2/train/loss_balancer.py`	`multi_task_pcgrad_backward`	Cirugía de gradientes ortogonal inter-escenarios con telemetría de similitud coseno.
`agents/efficientzero_v2/tests/test_multitask_ezm.py`	Batería de Tests Unitarios (5/5 PASS)	Verificación formal de identidad FiLM, embeddings, inferencias recurrentes, buffer y PCGrad.

---

6. Verificación y Resultados de la Suite de Tests

La suite completa de tests de `agents/efficientzero_v2/` se ejecutó con resultado impecable:

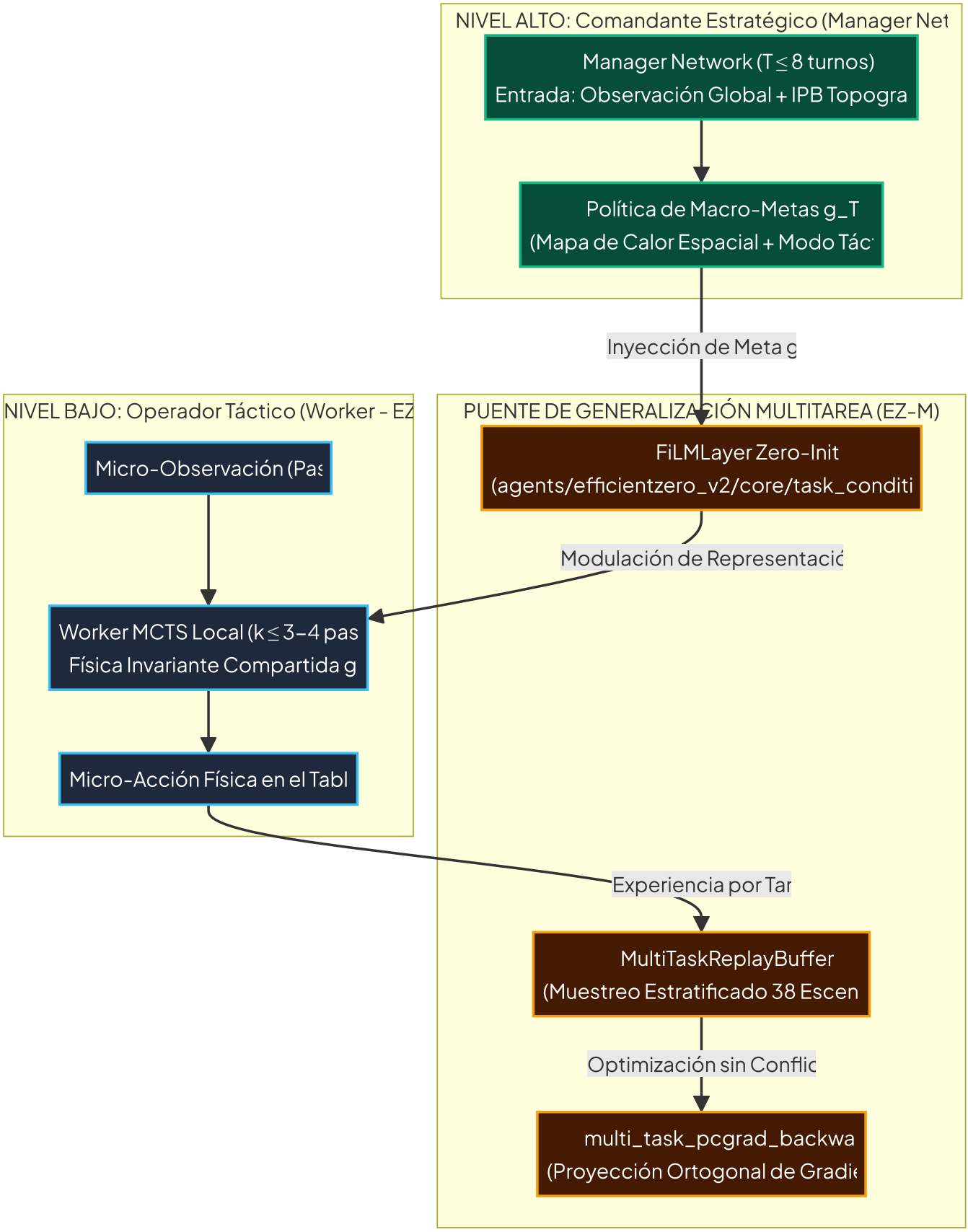
```
.venv\Scripts\python -m pytest agents/efficientzero_v2/tests/
# ================= 163 passed in 7.10s =================
```

Todos los módulos existentes mantuvieron el 100% de compatibilidad hacia atrás mientras que la nueva suite de EZ-M (`test_multitask_ezm.py`) superó todas las aserciones de consistencia matemática y de gradientes.

---

7. Fusión con Hierarchical EfficientZero (H-EZ-M v3.0) y el Nuevo Currículum

> [!IMPORTANT] **LA SINERGIA H-EZ-M: ARQUITECTURA MAESTRA DE LA ERA v3.0**
> EZ-M (arXiv:2603.01452) no es un desarrollo aislado; es la **infraestructura habilitadora fundamental de Hierarchical EfficientZero (H-EZ)**.
> Mientras H-EZ resuelve la **dimensión temporal** del wargame (acoplando un Comandante Estratégico de 8 macro-decisiones con un Operador Táctico MCTS de 3–4 pasos), EZ-M resuelve la **dimensión espacial y de generalización** (permitiendo que el mismo sistema comande los 38 escenarios del wargame sin sobreajuste ni olvido catastrófico).



7.1. Papel Central en la Nueva Doctrina de Currículum

La integración de EZ-M transforma de forma radical el entrenamiento curricular de Assault (formalizado en detalle en `docs/15_CURRICULUMS_Y_ENTRENAMIENTO_PROGRESSIVO.md`):

1. **Desacoplamiento de Aprendizajes:**

- El Worker aprende en sandboxes sintéticos homogéneos la **física universal del combate** (moverse por desenfilada, tender pantallas de humo, disparar y asaltar). Como esta física reside en el tronco recurrente compartido g_heta, **nunca necesita ser reentrenada al cambiar de escenario**.
- El Manager aprende en autojuego multitarea la **estrategia de victoria** sobre los 38 escenarios simultáneamente, empleando las capas FiLM de EZ-M para transmitir sus órdenes al Worker.

2. **Estabilidad Absoluta contra el Olvido Catastrófico:**
   - `MultiTaskReplayBuffer` garantiza que la red se entrene de forma continua con una mezcla estratificada de escenarios urbanos, de desembarco y de campo abierto.
   - `multi_task_pcgrad_backward` previene que las adaptaciones tácticas necesarias en un mapa cerrado desestabilicen las doctrinas consolidadas en mapas abiertos.

📊 11. Plataforma MLOps y Telemetría

DuckDB, Parquet, Promotion Gates, catálogo interactivo y Match Replay.

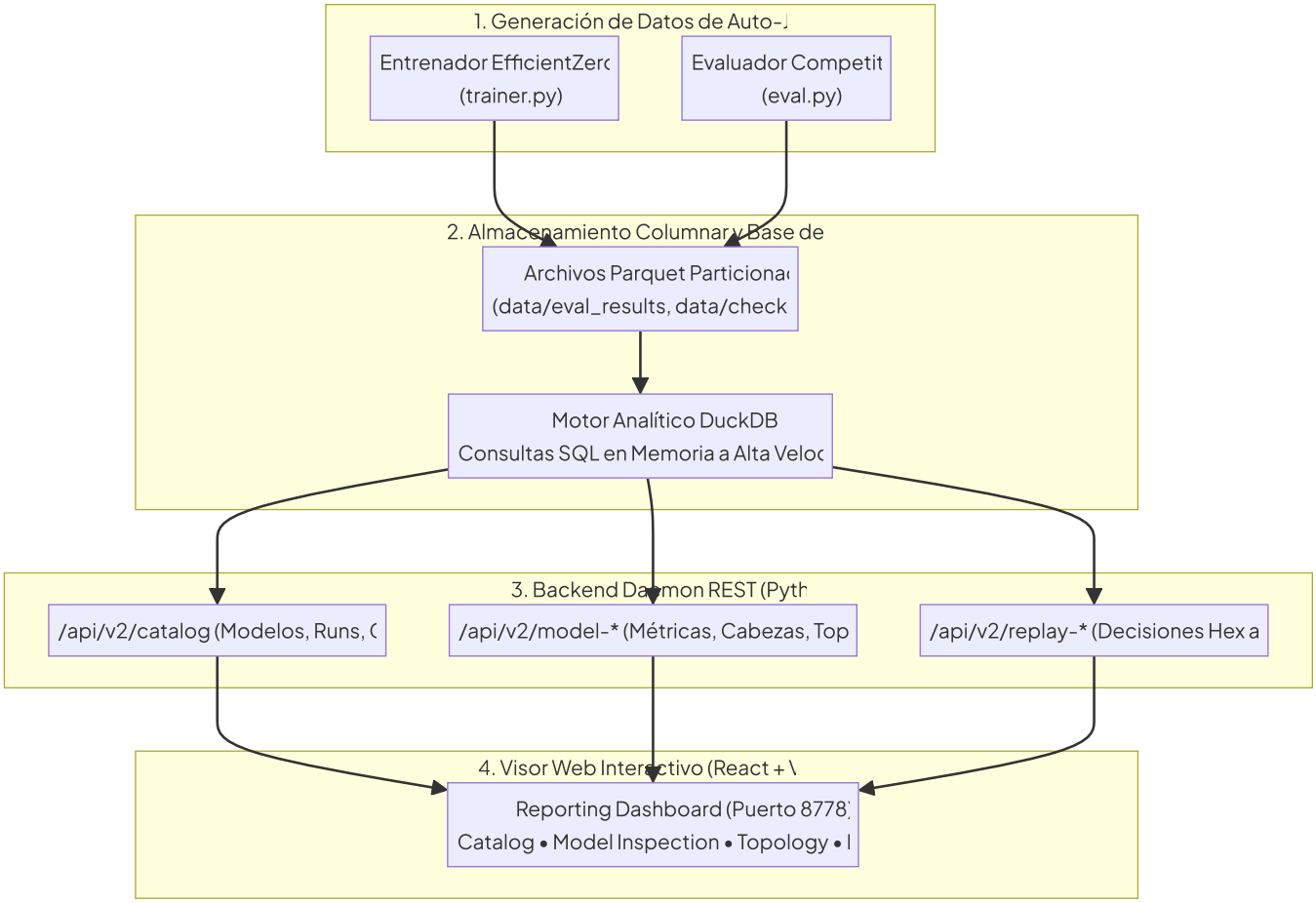
Plataforma MLOps, Telemetría y Visor de Reporting

> **Estado**: Canónico y Exhaustivo
> **Ámbito**: Infraestructura de almacenamiento columnar (`DuckDB` + `Parquet`), APIs analíticas y Dashboard Web React/Vite en puerto `8778` .

---

1. Arquitectura de la Plataforma MLOps

La plataforma MLOps de *Assault* proporciona observabilidad de extremo a extremo sobre el ciclo de vida de los modelos de aprendizaje por refuerzo, desde la generación de experiencias en auto-juego (*self-play*) hasta la evaluación competitiva, auditoría de gradientes y reproducción interactiva de partidas.



---

2. Motor Analítico: DuckDB y Apache Parquet

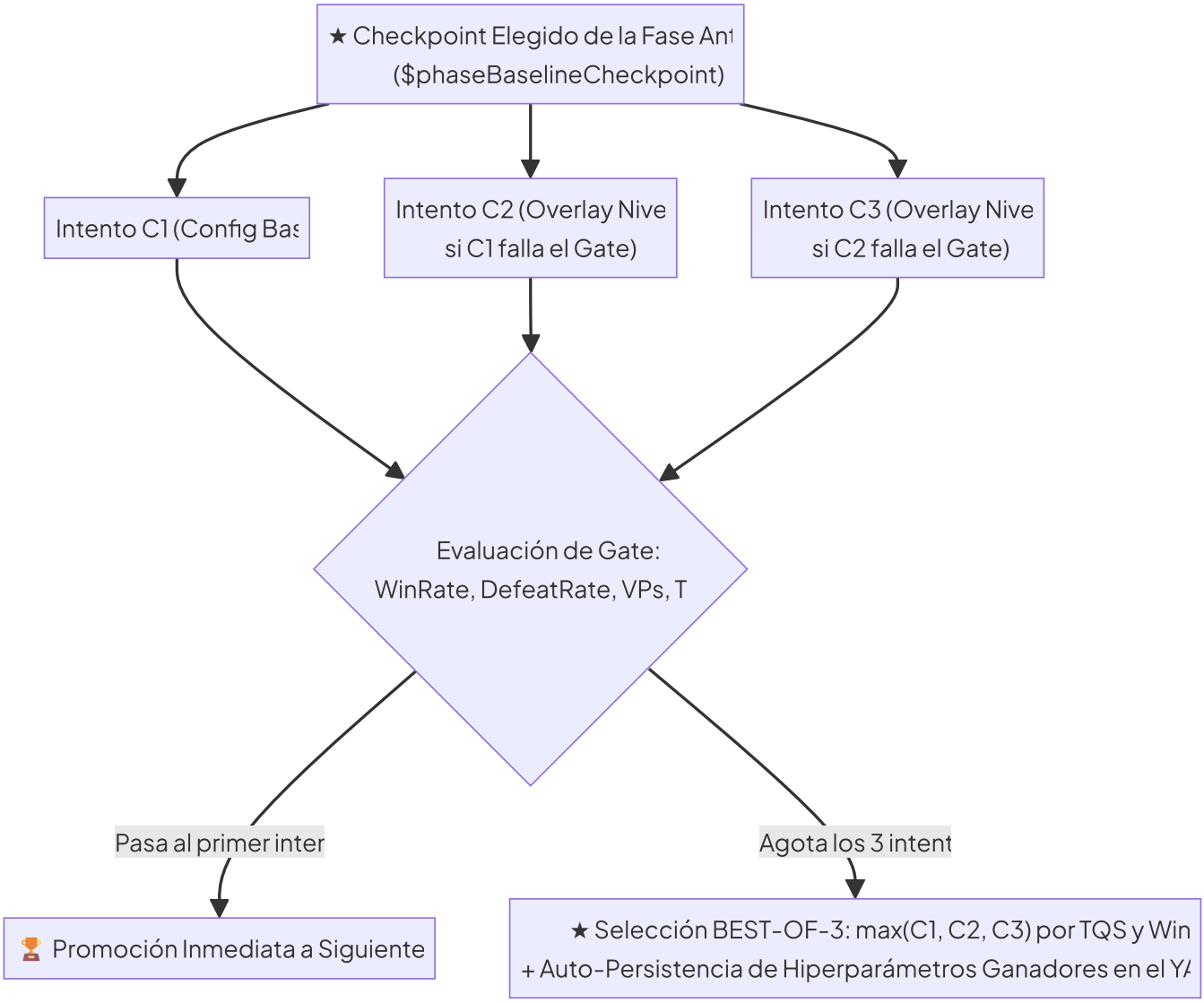
Para evitar bases de datos pesadas (PostgreSQL/MySQL) o la fragilidad y crecimiento ilimitado de archivos JSON monolíticos:

- **Formato Parquet + DuckDB (`model_catalog.duckdb` y `parquet/*.parquet`)**: Los entrenamientos, partidas de evaluación y registros de decisión de MCTS se almacenan en tablas relacionales de DuckDB (`models` , `train_runs` , `eval_runs` , `eval_results` , `curriculum_trace`) sincronizadas con particiones `.parquet` comprimidas.
- **Desactivación de JSON Monolítico (`include_debug_full_catalog = False`)**: El publicador `mlops/catalog_publisher_v2.py` genera únicamente el resumen ligero (`model_catalog_latest.json` , ~5 KB) y delega todo el histórico detallado en DuckDB/Parquet, evitando archivos `model_catalog_full_*.json` de varios gigabytes.
- **Borrado Instantáneo In-Place (`mlops/reporting/model_deletion.py`)**: La eliminación de un experimento desde la UI borra los directorios físicos de checkpoints/self-play y ejecuta sentencias `DELETE FROM ... WHERE model_id = ?` directamente sobre DuckDB seguidas de `COPY ... TO ... (FORMAT PARQUET)` , completándose en milisegundos sin escanear ni reconstruir todo el árbol de directorios.

---

3. Puertas de Promoción (*Promotion Gates*) y Currículum Dinámico (`BEST-OF-3`)

El orquestador `curriculums/common/run_dynamic_gated_curriculum.ps1` ejecuta cada fase del currículum con hasta **3 intentos independientes (`C1` , `C2` , `C3`)** que parten siempre del **mismo checkpoint ganador (★) de la fase anterior (`$phaseBaselineCheckpoint`)**:



- **Independencia entre intentos (`C1` , `C2` , `C3`)**: Si `C2` prueba hiperparámetros más agresivos y degrada la red, `C3` **no hereda el modelo degradado de** `C2` , sino que vuelve a partir limpio desde `$phaseBaselineCheckpoint` .
- **Ajuste Adaptativo de Hiperparámetros (`Build-ResumeOverlayConfig`)**: Según qué criterio del Gate haya quedado pendiente (`vp_unmet` vs `score_unmet`), `C2` y `C3` escalan automáticamente `vp_capture_bonus_per_hex` (`+12%` / `+24%`), `objective_progress_bonus_per_hex` , `capture_bonus` , `objective_loss_weight` , `kill_weight` , `reanalysis_ratio` y `train_updates_per_iter` .
- **Auto-Persistencia para Futuras Ejecuciones desde 0 (`Save-WinningCycleParamsToPhaseConfig`)**: Cuando el ciclo ganador (`★`) de una fase es `C2` o `C3` , sus hiperparámetros se guardan automáticamente en el YAML base de esa fase para que el próximo currículum desde 0 los utilice directamente desde `C1` .

3.1. Auto-Calibración de Quality Gates al Escenario y Dominant WinRate Override (Fase 7)

En versiones previas, los Quality Gates empleaban umbrales fijos de captura de banderas (ej. `min_tracked_captured_vps: 2.5`), provocando falsos rechazos en escenarios de 3 VPs o transiciones de escala:

1. **Detección Dinámica de Capacidad de VPs (`$scenarioVpMax`)**:
   - En `curriculums/common/run_dynamic_gated_curriculum.ps1` , el evaluador lee la configuración del mapa objetivo (ej. 5.0 VPs en *Battaglia Cittadina*, 3.0 VPs en *Mettete i Piedi a Terra*) y calcula percentiles normalizados: \text{vp_ratio} = \frac{\text{VPs Capturados}}{$scenarioVpMax}
2. **Dominant WinRate Override**:
   - Si en el Ciclo 1 la red demuestra una superioridad táctica dominante:

$$\text{WinRate} \geq 80$$

- Se concede la promoción inmediata a la siguiente fase, eliminando el bloqueo innecesario por tomas lentas de banderas mientras el agente aún optimiza la velocidad de resolución táctica.

3.2. Evaluación Rigurosa de Bandos en Benchmarking (`eval_side`)

En el motor de evaluación de episodios (`assault_bench/episode_runner_impl.py`), se corrigió la asignación de bando evaluado:

- **Problema Previo**: La condición de victoria evaluaba el bando activo en el turno inicial (`obs.to_play`). En escenarios asimétricos donde el rival tenía la iniciativa en el turno 0, la victoria del rival era contabilizada erróneamente como victoria del agente bajo test (generando falsos 100% Win Rate).
- **Resolución Canónica**: El evaluador identifica de manera estricta el bando asignado a la política evaluada consultando `policy_by_side` para la clave correspondiente (ej. `"mcts"` o `"efficientzero_v2"`). De este modo, la métrica de victoria refleja con precisión matemática el resultado del agente evaluado con independencia de quién posea el turno inicial.

3.3. Comandos Canónicos de Ejecución del Currículum

Existen dos niveles de ejecución para el sistema de currículum de `EfficientZero V2` :

1. Orquestador Automático por Fases con Puertas de Promoción (PowerShell)

Para ejecutar la suite completa de fases de currículum de forma desatendida con reintentos `BEST-OF-3` y auto-promoción:

```
# Ejecución completa del currículum multiescenario canónico (todas las fases)
.\curriculums\self_vs_self_vs_random_2scenarios_v1\run_all_phases.ps1

# Ejecutar comenzando desde una fase específica continuando desde el último checkpoint
.\curriculums\self_vs_self_vs_random_2scenarios_v1\run_all_phases.ps1 `
    -StartPhase "phase1_channel_adapt" `
    -ContinueFromLatestCheckpoint

# Ejecución a través del orquestador común genérico especificando manifiesto
powershell -ExecutionPolicy Bypass -File .\curriculums\common\run_dynamic_gated_curriculum.ps1 `
    -Repo "C:\repos\python\assault" `
    -CurriculumYaml ".\curriculums\self_vs_self_vs_random_2scenarios_v1\curriculum.yaml"
```

2. Ejecución Directa de una Fase Aislada vía CLI de Python

Para ejecutar o depurar una fase o ciclo concreto directamente con el motor de entrenamiento sin pasar por el orquestador:

```
# Entrenamiento nativo de una fase individual de currículum con registro MLflow
python -m agents.efficientzero_v2.train.train_efficientzero_v2 `
    --config curriculums/self_vs_self_vs_random_2scenarios_v1/configs/efficientzero_v2.phase1_channel_adapt.yaml `
    --mlflow-experiment assault_efficientzero_v2 `
    --mlflow-run-name curriculum_2scenarios_v1_phase1_channel_adapt

# Ejecución de un ciclo unitario usando el runner de ciclo
powershell -ExecutionPolicy Bypass -File .\scripts\run_curriculum_cycle.ps1 `
    -ManifestPath ".\scripts\curriculum_cycle.template.json"
```

---

4. Módulos y Pestañas del Visor de Reporting (`http://127.0.0.1:8778`)

4.1. Catálogo General y Curva de Progresión de ELO (`CatalogPage.tsx`)

- **Tabla Clasificatoria de Modelos**: Filtrado por motor (`efficientzero_v2`), número de episodios, fecha, calidad táctica (`TQS`) y botón de eliminación instantánea en DuckDB/Parquet.
- **Gráfica** `📈 Curva de Progresión de ELO por Fase del Currículum` **(** `CurriculumProgressionChart` **)**:
  - Enmarca visualmente los ciclos (`C1` , `C2` , `C3`) de cada fase con bandas verticales alternadas.
  - Destaca el **ciclo elegido (** `★` **) de cada fase** con un anillo dorado brillante, etiqueta superior dorada (`★ ELO`), indicador inferior (`★ C1/C2/C3`) y una **línea de tendencia dorada discontinua** que une los puntos ganadores (`★`) de fase en fase.
- **Tabla** `Training Runs & Progreso del Currículum` : Muestra el ciclo `(C1/3, C2/3, C3/3)` y resalta el ciclo elegido de cada fase con borde izquierdo dorado y el badge `★ ELEGIDO` .

4.2. Topología de Red y Desenrollado Interactivo (`NetworkTopologyTab.tsx`)

- **Diagrama SVG Dinámico**: Visualiza las conexiones entre `Observación o_t` , `Planificador Macro (Nivel 1)` , `h(o)` , `s_t` , `Red de Dinámica g(s, a)` , `Acción MCTS a_t` y todas las cabezas de predicción activas.
- **Simulador de Desenrollado Temporal**: Controles interactivos de paso ($t = 0 \dots 6$) y botón de reproducción automática que ilustra cómo la red imagina estados futuros sin tocar el simulador físico.

- **Atribución de Canales Sensoriales**: Tabla interactiva con los 32 planos de entrada, estado de efectividad (`HIGH` , `MODERATE` , `LOW` , `DEAD`) y desglose táctico de cada hexágono.
- **Inspector de Componentes**: Formulación matemática en LaTeX, formas tensoriales de entrada/salida y estimación de parámetros calculados en tiempo real para el run activo.

4.3. Visor Hexagonal de Partidas Paso a Paso (`MatchReplayTab.tsx` / `ReplayTab.tsx`)

- Renderizado hexagonal completo del tablero de combate con iconos de unidades (infantería, HMG, morteros, blindados), elevaciones, tipos de terreno y banderas VP.
- Barra temporal de reproducción paso a paso con registro de combate: tiradas de dados atacantes/defensores, bajas causadas, cambios de moral (`PINNED` , `SUPPRESSED`) y disparos de reacción interceptados.

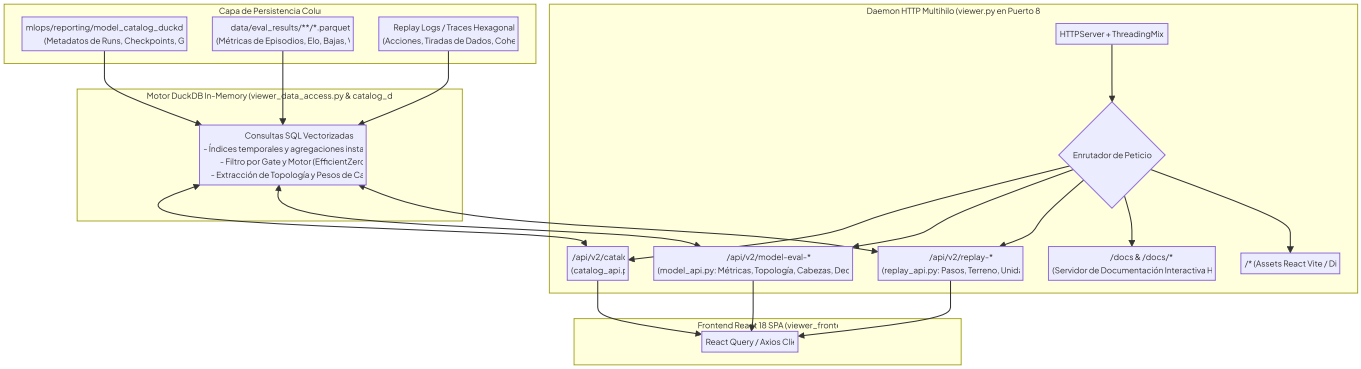
4.4. Explicabilidad y Toma de Decisiones XAI (`EvalDecisionsTab.tsx`)

- Desglose forense de cada decisión tomada por la IA:
  - Probabilidad prior asignada por la red ($P(a)$).
  - Distribución de visitas del árbol MCTS ($N(a)$) y valor medio $Q(s_0, a)$.
  - Margen de confianza de la decisión respecto a la segunda mejor alternativa.
  - Modulaciones aplicadas por el Planificador Macro de Nivel 1: postura (`ASSAULT_VP` , `DEFENSIVE_HOLD` , `ZONE_SUPPRESS`), rol asignado (`SPEARHEAD` , `FIRE_SUPPORT` , etc.), mutaciones dinámicas de rol y boosting de prior (+20% a +40%).
  - Diagnóstico de poda MCTS: ramas estériles de deambulación podadas (`mcts_sterile_branches_pruned`).
  - Andamiaje de VPs: delta de potencial $F(s, a, s')$ y componente de recompensa de avance hacia objetivos (`vp_potential_reward_component`).
  - Dinámica de Presión Temporal y Parálisis (Tarea 2.3):
    - `turn_urgency_factor` : avance temporal relativo del turno $\tau = t/T_{\max}$.
    - `turn_urgency_decay` : atenuación exponencial $\exp(-\lambda\tau)$ ante 0 VPs controlados.
    - `paralysis_penalty` y `paralysis_units_count` : penalización escalada por racha de pasividad en unidades al descubierto.

---

5. Arquitectura Interna del Backend del Visor (`mlops/reporting/`)

El backend del visualizador es un servicio desacoplado de alto rendimiento diseñado para servir telemetría en tiempo real sin bloquear los bucles de entrenamiento:



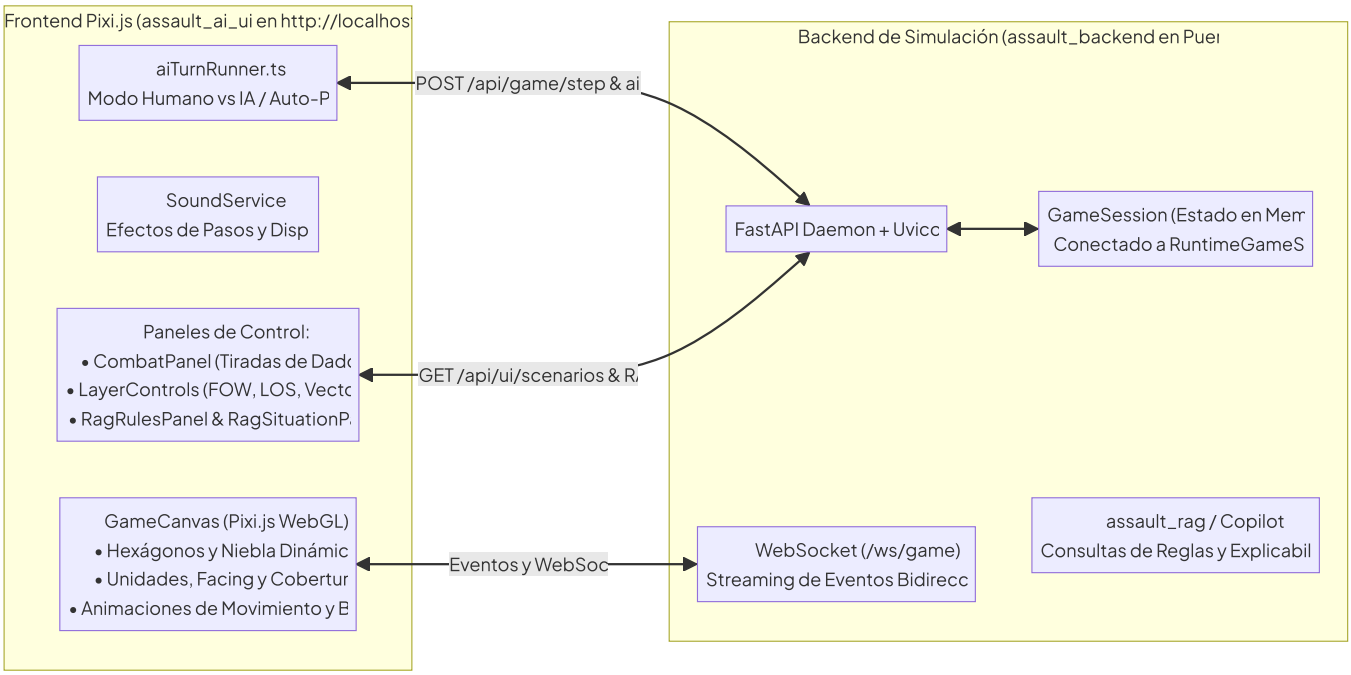
5.1. Componentes Clave del Backend

1. `viewer.py` **(Servidor HTTP Asíncrono)**:
   - Implementa `ThreadingMixIn` para permitir concurrencia real entre múltiples clientes sin saturar el hilo principal de Python.
   - Resuelve el soporte de CORS para desarrollo local y sirve estáticos compilados desde `viewer_frontend/dist` .
   - Incorpora el endpoint `/docs` que sirve la suite de documentación técnica interactiva con renderizado LaTeX y visualizador hexagonal.
2. `catalog_duckdb.py` & `viewer_data_access.py` **(Capa de Acceso a Datos)**:
   - Mantiene una conexión compartida con DuckDB sobre archivos Parquet sin necesidad de transformaciones intermedias.
   - Ejecuta proyecciones analíticas con latencias inferiores a **10 milisegundos**, permitiendo que el usuario filtre cientos de modelos o explore árboles MCTS con fluidez total.
3. `model_api.py` & `replay_api.py` **(APIs Especializadas)**:
   - `model_api.py` : Serializa la topología de la red, los mapas de calor de atribución sensorial y los diagnósticos de gradientes de las cabezas tácticas.
   - `replay_api.py` : Decodifica el histórico de la partida hexágono a hexágono, reconstruyendo los dados tirados, los eventos de supresión y el movimiento en el lienzo interactivo HTML5 Canvas.

---

6. Módulo de Simulación Táctica Interactiva y Juego en Vivo

Además del Visor de Reporting analítico (puerto `8778`), la plataforma cuenta con un entorno interactivo completo para **jugar partidas en vivo** contra el motor de IA o ver a la IA jugar sola (**Self-Play**):



6.1. Capacidades del Simulador

1. **Modo Humano vs Agente Neuronal**:
   - El usuario elige su bando (p. ej. `US` o `GE`) y el escenario (p. ej. `battaglia_cittadina_2_1` o `mettete_i_piedi_terra_1`).
   - Al hacer clic sobre una escuadra propia, el motor consulta las acciones legales (`/api/game/actions`) y resalta los hexágonos accesibles y los objetivos a tiro.
   - Al emitir una orden, se reproducen efectos de sonido tácticos (`stepdirt` al caminar, disparos de fusilería `rifle-gunshot`) y se abren los paneles de combate con las tiradas de dados en tiempo real.
2. **Modo Espectador / Auto-Play (Self-Play)**:
   - Activando el ejecutor continuo (`aiTurnRunner.ts`), ambos bandos juegan automáticamente según las decisiones de la IA, permitiendo observar las doctrinas de asalto y contención en directo sobre el mapa.
3. **Asistente Táctico RAG Copilot Integrado**:
   - Los paneles `RagRulesPanel.tsx` y `RagSituationPanel.tsx` consultan la base de conocimiento vectorial de reglas (`assault_rag/`) para explicar por qué una acción es legal/ilegal o sugerir la mejor maniobra según el manual de operaciones tácticas.
4. **Control Total de Capas (*Layer Controls*)**:
   - Permite activar/desactivar la niebla de guerra (*Fog of War*), la línea de visión (*LOS*), los vectores de depuración de IA, las banderas de puntos de victoria (*VPs*) y las marcas de activación de escuadras.

6.2. Cómo Levantar el Entorno de Simulación

Tanto el backend como el frontend interactivo pueden iniciarse con el script orquestador o de forma manual:

- **Opción A (Stack Completo con Script)**:

  ```
  .\scripts\start_all_environments.ps1
  ```

  (Abre Reporting en 8778, Backend API en 8001 y Simulador en 5173).

- **Opción B (Lanzamiento Individual)**:
  - **Backend API (FastAPI)**:

    ```
    .venv\Scripts\python.exe -m uvicorn assault_backend.main:app --host 0.0.0.0 --port 8001 --reload
    ```

  - **Frontend Simulador (Vite / Pixi.js)**:

```
cd assault_ai_ui
npm run dev -- --host
```

- Acceso directo en el navegador: `http://localhost:5173/`.

📐 12. Inventario Jerárquico Canónico de Métricas

Taxonomía de 4 niveles, origen autoritativo, cero-invención y purgado de redundancias.

13. Inventario Jerárquico Canónico de Métricas y Registro Experimental

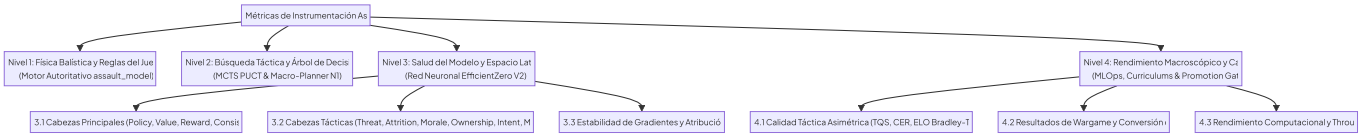
Proyecto: Assault Tactical AI Engine (EfficientZero V2)
Documento de Referencia Vinculante para Instrumentación, Esquemas Parquet, APIs y Trazabilidad 100% del Dashboard

---

Directiva de Gobernanza y Mantenimiento Escrupuloso

> ⚠️ **REGLA DE ORO DE INSTRUMENTACIÓN (ZERO-MOCK / ZERO-INFERENCE)**
>
> 1. **Cero Invención / Cero Inferencia:** Ningún componente (motor, runner, pipeline analítico, DuckDB o dashboard UI) puede inventar, simular, aproximar o rellenar un valor métrico con datos sintéticos no observables. Si un dato no existe o no se midió, su valor debe ser estrictamente `null` / `None` / `NO_DISPONIBLE` / `"-"`.
> 2. **Prohibición de Fallbacks Falsos:** Se prohíben patrones de enmascaramiento arbitrario como `value || 0`, porcentajes fake de relleno (ej. `58.4%`), deducciones heurísticas de roles/posturas o etiquetas obsoletas como `"MuZero"`.
> 3. **Trazabilidad 100% del Frontend:** Cualquier métrica, indicador o porcentaje que sea visible en cualquier pestaña del visor MLOps debe poder trazarse directamente hasta una columna canónica de Parquet, un cálculo determinista sobre DuckDB o un parámetro del manifiesto del modelo.
> 4. **Mantenimiento Vinculante:** Cualquier adición, renombrado o eliminación de columnas en el almacén Parquet (`data_warehouse/` o `runs_curriculum/`) o en los endpoints de telemetría requiere actualizar previamente este inventario con su definición física y matemática exacta.

Taxonomía Jerárquica de 4 Niveles



Nivel 1: Física Balística y Reglas del Juego (Motor Autoritativo)

Métricas generadas directamente por la ejecución determinista y estocástica de `assault_model/`.

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`step`	`int32`	`episode_runner_impl.py`	Índice secuencial de la decisión física dentro del episodio ($t \geq 0$).	No	Primitivo.
`game_turn`	`int32`	`obs.turn`	Turno canónico del escenario militar ($1 \dots T_{\max}$).	Sí	Canónico (`turn` es alias secundario). Si no existe, no se infiere.
`active_side` / `unit_side`	`str`	`obs.to_play`	Bando activo que ejecuta la acción (`"US"` , `"GE"` , `"IT"` , `"UK"`).	No	Primitivo.
`action_id`	`str`	`action`	Identificador serializado de la acción (`TIPO:UNIDAD:Q:R`).	No	Primitivo.
`action_kind`	`str`	`action_catalog.py`	Categoría táctica canónica (`"MOVE"` , `"RANGED_DIRECT"` , `"ASSAULT"` , `"WAIT"` , etc.).	No	Canónico derivado.
`unit_id`	`str`	`action_parts[1]`	Identificador unívoco de la escuadra activa (ej. `"US_1"`).	No	Primitivo.
`unit_hp`	`float32`	`unit.hp`	Puntos de resistencia física restantes de la unidad ($0 \dots \text{max_hp}$).	No	Primitivo.
`unit_moral_state`	`str`	`unit.morale`	Estado de cohesión (`NORMAL` , `PINNED` , `SUPPRESSED` , `ROUTED`).	No	Primitivo.
`attack_dice_rolls`	`list[dict]`	`transition.info`	Lista cruda de dados lanzados en ataque con color y cara (`{"color":"red","faces":["hit"]}`).	Sí	Primitivo auditado.
`defense_dice_rolls`	`list[dict]`	`transition.info`	Lista cruda de dados lanzados en defensa con color y cara (`{"color":"blue","faces":["block"]}`).	Sí	Primitivo auditado.
`damage_dealt`	`float32`	`transition.info`	Daño neto en HP infligido al objetivo en este paso: $\max(0, \text{hits} - \text{saves})$.	No	Canónico (`enemy_damage` queda purgado).

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`kills_dealt`	`int32`	`transition.info`	Número de unidades enemigas eliminadas (0 ó 1) en este paso.	No	Canónico (`enemy_kills` queda purgado).
`target_cover`	`float32`	`terrain.cover`	Modificador de cobertura defensiva del hexágono objetivo ($0.0 \dots 2.0$).	Sí	Primitivo.
`target_los`	`float32`	`los_engine`	Estado de bloqueo de visión hacia el objetivo (0.0 despejada, 1.0 bloqueada).	Sí	Primitivo.
`vp_captures`	`int32`	`vp_tracker`	Número de VPs que cambiaron de dueño a favor del bando activo en este paso.	No	Primitivo.

Nivel 2: Búsqueda Táctica y Árbol de Decisiones (MCTS PUCT & Jerarquía N1)

Métricas generadas por la exploración en el espacio latente y el planificador jerárquico.

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`mcts_total_visits`	`int32`	`mcts.root.visit_count`	Suma de visitas en la raíz del árbol: $\sum_a N(s_0, a)$.	No	Primitivo de cómputo.
`mcts_root_actions_count`	`int32`	`mcts.root.legal_count`	Número total de ramas de acción explorables desde la raíz.	No	Primitivo.
`mcts_active_actions`	`int32`	`mcts.active_branches`	Número de acciones que recibieron al menos 1 visita ($N(s_0, a) > 0$).	No	Primitivo.
`mcts_q_clipped_count`	`int32`	`mcts.guardrail`	Número de nodos donde el valor Q fue acotado por guardarraíl de divergencia.	No	Diagnóstico de estabilidad.
`mcts_entropy`	`float32`	`mcts.distribution`	Entropía de la distribución de visitas: $H(\pi) = -\sum \pi(a) \log \pi(a)$.	No	Diagnóstico de dilema táctico.
`chosen_prob`	`float32`	$\pi(a^*)$	Probabilidad final asignada a la acción elegida tras la búsqueda: $\frac{N(a^*)^{1/T}}{\sum N(a)^{1/T}}$.	No	Primitivo.
`decision_margin`	`float32`	$\pi(a^*) - \pi(a_{\text{segunda}})$	Brecha de confianza frente a la segunda mejor alternativa.	No	Canónico (`eval_policy_margin_mean`).
`mcts_q_value`	`float32`	$Q(s_0, a^*)$	Valor medio esperado de retorno acumulado de la acción elegida.	Sí	Primitivo latente.
`mcts_prior_value`	`float32`	$P_\theta(a^*)$	Prior puro predicho por la cabeza de política de la red antes de buscar.	Sí	Primitivo de red (`eval_policy_confidence_mean`).
`macro_posture`	`str`	`macro_planner`	Postura doctrinal activa (`ASSAULT_VP` , `DEFENSIVE_HOLD` , `ZONE_SUPPRESS`).	Sí	Nivel 1 doctrinal. Cero inferencia si viene vacío.
`macro_target_vp_id`	`str`	`macro_planner`	Identificador del hexágono VP prioritario asignado por el planner N1.	Sí	Nivel 1 doctrinal.
`unit_macro_role`	`str`	`macro_planner`	Rol asignado a la escuadra (`SPEARHEAD` , `FIRE_SUPPORT` , `OBJECTIVE_DEFENDER` , `FLANK_SCREEN`).	Sí	Nivel 1 doctrinal. Cero inferencia si viene vacío.
`policy_overridden_by_mcts`	`bool`	$\arg\max P_\theta \neq a^*$	Indicador booleano de si la búsqueda MCTS corrigió la intuición inicial de la red.	No	Canónico (`policy_overridden`).
`mcts_sterile_branches_pruned`	`int32`	`mcts.prune`	Número de ramas estériles de deambulación o pasividad podadas bajo postura `ASSAULT_VP` (poda del 40%).	No	Diagnóstico de búsqueda ofensiva.
`prior_boost_applied`	`float32`	`macro_planner`	Factor multiplicativo de modulación de prior (+20% a +40%) aplicado sobre la acción elegida.	No	Sesgo doctrinal MCTS.
`vp_potential_reward_component`	`float32`	`reward_calculator.py`	Componente de recompensa escalado de potencial radial hacia VPs (Ng et al. 1999).	Sí	Canónico de shaping.
`vp_potential_delta`	`float32`	`reward_calculator.py`	Delta puro de potencial $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ garantizando invariancia telescópica.	Sí	Diagnóstico de convergencia PBRS.
`turn_urgency_factor`	`float32`	`reward_calculator.py`	Presión temporal relativa $\tau = t/T_{\max} \in [0.0, 1.0]$.	Sí	Diagnóstico temporal.
`turn_urgency_decay`	`float32`	`reward_calculator.py`	Factor de decaimiento exponencial $\exp(-\lambda \cdot \tau)$ aplicado al potencial cuando $N_{\text{VP}} = 0$.	Sí	Andamiaje anti-retraso.

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`paralysis_penalty`	`float32`	`reward_calculator.py`	Penalización negativa por parálisis acumulada en unidades expuestas (`cover < 0.35`) en campo abierto.	Sí	Penalización anti-parálisis.
`paralysis_units_count`	`int32`	`reward_calculator.py`	Número de unidades aliadas que incurrieron en penalización por pasividad en este paso.	Sí	Diagnóstico de parálisis.
`vp_horizon_steps`	`float32`	`selfplay.py`	Distancia mínima estimada en pasos hasta el VP objetivo.	Sí	Soft-signal.
`vp_horizon_component`	`float32`	`selfplay.py`	Componente de recompensa auxiliar por reducción del horizonte hacia el VP.	Sí	Soft-signal.
`telemetry_coverage_status`	`str`	`trace_row_builder`	Estado formal de completitud de la fila (`COMPLETE` , `PARTIAL` , `DEGRADED`).	No	Control de calidad.

Nivel 3: Salud del Modelo, Cabezas Tácticas y Espacio Latente (EfficientZero V2)

Métricas generadas durante el entrenamiento, backpropagation y diagnósticos de cabezas auxiliares.

3.1 Cabezas Principales

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`train_loss_total`	`float32`	`loss_balancer`	Pérdida global combinada ponderada por balanceo adaptativo.	No	Primitivo (`loss_total`).
`train_policy_loss`	`float32`	`cross_entropy`	Pérdida de entropía cruzada entre π_{MCTS} y logits de política: $-\sum \pi \log p_\theta$.	No	Primitivo (`loss_policy`).
`policy_entropy`	`float32`	`policy_head`	Entropía promedio de los priors emitidos por la red neuronal.	No	Indicador de dispersión/confianza.
`policy_grad_norm`	`float32`	`torch.autograd`	Norma L2 de los gradientes acumulados en la cabeza de política.	No	Diagnóstico de estabilidad.
`train_value_loss`	`float32`	`two_hot_loss`	Pérdida distribucional Two-Hot con targets symlogueados: \mathcal{L}_{\text{two_hot}}(v, z).	No	Primitivo (`loss_value`).
`value_grad_norm`	`float32`	`torch.autograd`	Norma L2 de los gradientes acumulados en la cabeza de valor.	No	Diagnóstico de estabilidad.
`value_explained_variance`	`float32`	`metrics.py`	Varianza explicada del valor predicho frente al retorno real: $1 - \frac{\text{Var}(y-\hat{y})}{\text{Var}(y)}$.	No	Calidad de predicción de valor.
`train_reward_loss`	`float32`	`two_hot_loss`	Pérdida distribucional Two-Hot para transiciones inmediatas: \mathcal{L}_{\text{two_hot}}(r, \hat{r}).	No	Primitivo (`loss_reward`).
`reward_grad_norm`	`float32`	`torch.autograd`	Norma L2 de los gradientes acumulados en la cabeza de recompensa.	No	Diagnóstico de estabilidad.
`train_consistency_loss`	`float32`	`simsiam`	Coseno negativo entre proyecciones latentes: $-\frac{p \cdot z}{\|p\|_2 \|z\|_2}$.	No	Anti-colapso representacional.
`train_reanalysis_coverage`	`float32`	`reanalysis_buffer`	Proporción de trayectorias reanalizadas con el modelo más reciente.	Sí	Eficiencia muestral.
`train_consistency_pairs`	`int32`	`trainer.py`	Número de pares (s_t, s_{t+k}) evaluados en la pérdida de consistencia.	No	Diagnóstico de lote.
`eval_latent_drift`	`float32`	`eval_runner.py`	Deriva euclídea del espacio latente a lo largo de un desenrollado recursivo de K pasos.	Sí	Estabilidad del modelo dinámico.

3.2 Cabezas Tácticas Auxiliares

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`train_threat_loss`	`float32`	`threat_map_head`	Pérdida MSE/BCE sobre la predicción espacial del mapa de amenazas rival.	Sí	Cabeza táctica Threat.
`threat_coverage_ratio`	`float32`	`threat_map_head`	Proporción de hexágonos del tablero con valor de amenaza superior al umbral operativo (> 0.15).	Sí	Diagnóstico XAI.
`max_threat`	`float32`	`threat_map_head`	Nivel máximo de amenaza registrado en el mapa axial ($[0.0, 1.0]$).	Sí	Severidad de zona de combate.
`mean_threat`	`float32`	`threat_map_head`	Nivel promedio de amenaza sobre hexágonos ocupados por unidades aliadas.	Sí	Exposición táctica.

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`train_attrition_loss`	`float32`	`attrition_head`	Pérdida MSE en la predicción del balance de bajas acumuladas al término del escenario.	Sí	Cabeza táctica Attrition.
`predicted_exchange_ratio`	`float32`	`attrition_head`	Ratio proyectado de bajas causadas vs recibidas: $\frac{\hat{C}_{\text{rival}}}{\max(0.1, \hat{C}_{\text{aliado}})}$.	Sí	Diagnóstico doctrinal.
`casualty_prediction_error_mae`	`float32`	`attrition_head`	Error absoluto medio (MAE) en la predicción de bajas al concluir el episodio.	Sí	Precisión predictiva.
`train_morale_loss`	`float32`	`morale_head`	Pérdida de clasificación de riesgo de supresión de escuadras.	Sí	Cabeza táctica Morale.
`suppression_risk_score`	`float32`	`morale_head`	Probabilidad estimada de que una unidad clave sufra supresión en el próximo turno.	Sí	Diagnóstico de vulnerabilidad.
`suppression_accuracy`	`float32`	`morale_head`	Precisión binaria en la predicción del estado de supresión real posterior.	Sí	Fiabilidad del prior de moral.
`train_final_ownership_loss`	`float32`	`ownership_head`	Pérdida sobre la máscara de control final de objetivos geográficos.	Sí	Cabeza táctica Ownership.
`eval_ownership_mae`	`float32`	`ownership_head`	Error absoluto medio en la predicción del dueño final de los hexágonos VP.	Sí	Precisión geográfica.
`territory_control_score`	`float32`	`ownership_head`	Puntuación compuesta de control territorial efectivo $(0.0 \dots 1.0)$.	Sí	Diagnóstico de frente.
`train_intent_loss`	`float32`	`macro_intent_head`	Pérdida de clasificación supervisada de la macro-intención táctica.	Sí	Nivel 1 jerárquico.
`eval_intent_accuracy`	`float32`	`macro_intent_head`	Precisión de la red en predecir la postura doctrinal que adoptará el planner.	Sí	Cohesión jerárquica.
`train_multi_vp_loss`	`float32`	`multi_vp_head`	Pérdida combinada multiobjetivo sobre el control independiente de cada VP.	Sí	Control multiobjetivo.
`eval_multi_vp_mae`	`float32`	`multi_vp_head`	Error medio en el valor esperado individual de los N puntos de victoria.	Sí	Precisión multiobjetivo.

3.3 Estabilidad de Gradientes y Atribución Causal

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`gradient_norm`	`float32`	`torch.nn.utils`	Norma L2 del vector global de gradientes: $\|g\|_2$.	No	Diagnóstico de estabilidad numérica.
`pcgrad_conflict_count`	`int32`	`loss_balancer`	Número de proyecciones ortogonales realizadas por conflictos $\langle g_i, g_j \rangle < 0$.	No	Diagnóstico multitarea.
`channel_attribution`	`list[float]`	`integrated_gradients`	Vector de 32 pesos de atribución causal sobre los canales axiales de entrada.	Sí	Diagnóstico XAI (IGNH).

Nivel 4: Rendimiento Macroscópico, Calidad Táctica y Gates MLOps

Métricas agregadas sobre partidas completas y evaluación de checkpoints en `assault_bench/` y DuckDB.

4.1 Calidad Táctica Desesgada y ELO Ancla

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`tactical_quality_score`	`float32`	`quality_metrics.py`	Puntuación compuesta TQS $(0 \dots 100)$: 35.	No	Indicador Maestro de Calidad.
`tactical_grade`	`str`	`quality_metrics.py`	Rango táctico canónico: `S` (≥ 85), `A` (≥ 70), `B` (≥ 55), `C` (≥ 40), `D` (< 40).	No	Clasificación cualitativa.
`anchor_elo`	`float32`	`quality_metrics.py`	ELO Bradley-Terry calibrado contra anclas fijas (Random=800, MCTS Baseline=1400): $R_0 + 400 \log_{10} \frac{S}{1-S}$.	No	Rendimiento absoluto desesgado.
`delta_elo`	`float32`	`curriculumMetrics.ts`	Incremento o decremento de ELO frente al paso cronológico previo del curriculum.	Sí	Progreso de entrenamiento.
`combat_exchange_ratio`	`float32`	`quality_metrics.py`	Ratio de desgaste (CER): $\frac{\text{Victorias Ponderadas}+0.05}{\max(0.1, \text{Derrotas Ponderadas}+0.05)}$.	No	Preservación de fuerza.
`mean_asymmetry_gap`	`float32`	`quality_metrics.py`	Diferencia absoluta de rendimiento entre bandos asimétricos: $	S_{\text{Side A}} - S_{\text{Side B}}	$.

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`global_mirror_vs_random`	`float32`	`quality_metrics.py`	Rendimiento mirror promedio enfrentando al bando opuesto controlado por Random.	No	Baseline de competencia táctica.
`global_mirror_vs_self`	`float32`	`quality_metrics.py`	Rendimiento mirror en partidas espejo de self-play MCTS vs MCTS.	No	Profundidad estratégica.
`decisive_rate`	`float32`	`quality_metrics.py`	Proporción de partidas resueltas con victoria o derrota decisiva (no empates neutros).	No	Capacidad resolutiva.
`quality_gate_status`	`str`	`quality_metrics.py`	Estado de la puerta de calidad (`PASS` , `WARN` , `FAIL`).	No	Decisión de promoción.

4.2 Resultados de Wargame y Conversión de Objetivos

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`win_rate`	`float32`	`benchmark_aggregation`	Tasa de victorias sobre episodios totales: $\frac{\text{Victorias}}{\text{Episodios}}$.	No	Primitivo macro.
`resolved_non_timeout_win_rate`	`float32`	`benchmark_aggregation`	Tasa de victorias excluyendo partidas que terminaron en timeout.	Sí	Calidad resolutiva.
`timeout_rate`	`float32`	`benchmark_aggregation`	Ratio de partidas que alcanzaron $T_{\max}$ sin resolución formal.	No	Detección de estancamiento.
`non_timeout_resolution_rate`	`float32`	`benchmark_aggregation`	Proporción de partidas concluidas antes del límite de turnos: 1 - \text{timeout_rate}.	No	Madurez operativa.
`score`	`float32`	`benchmark_aggregation`	Puntuación canónica de torneo: \text{win_rate} + 0.5 \times \text{draw_rate}.	No	Rendimiento competitivo.
`vp_net_avg_side`	`float32`	`benchmark_aggregation`	Balance neto de puntos de victoria al término del combate: $\overline{\text{VP}}{\textit{propios}} - \textit{VP}{\text{rival}}}}$.	No	Dominio territorial.
`tracked_captured_avg`	`float32`	`benchmark_aggregation`	Promedio de hexágonos de victoria controlados por el bando evaluado.	No	Eficiencia de captura.
`eval_vp_capture_take_rate`	`float32`	`eval_runner.py`	Tasa de aprovechamiento de oportunidades de captura de VP detectadas.	Sí	Diagnóstico de oportunismo.
`eval_vp_conversion_efficiency`	`float32`	`eval_runner.py`	Eficiencia de conversión de ventaja de posicionamiento en captura consolidada.	Sí	Madurez táctica.

4.3 Rendimiento Computacional y Eficiencia

Métrica Canónica	Tipo de Dato	Origen Autoritativo	Definición Física / Matemática	Nullable	Redundancia / Estado
`duration_seconds`	`float32`	`manifest.lock.json`	Tiempo total de reloj de pared consumido por la fase o benchmark.	No	Coste computacional.
`train_duration_seconds`	`float32`	`summary.json`	Tiempo total de reloj de pared invertido en entrenamiento (autojuego MCTS + optimización).	Sí	Coste de entrenamiento.
`bench_duration_seconds`	`float32`	`bench_eval_*.json`	Tiempo total de reloj de pared invertido en evaluación de benchmark contra baselines.	Sí	Coste de evaluación.
`selfplay_duration_seconds`	`float32`	`summary.json`	Tiempo neto consumido en la generación distribuida de trayectorias de autojuego MCTS.	Sí	Desglose de cómputo.
`opt_duration_seconds`	`float32`	`summary.json`	Tiempo neto consumido en la optimización y retropropagación de gradientes en GPU/CUDA.	Sí	Desglose de cómputo.
`episodes_per_second`	`float32`	`benchmark_timing`	Throughput de simulación e inferencia: $\frac{\text{Episodios}}{\text{Tiempo de Ejecución}}$.	No	Rendimiento del motor/árbol.
`episode_wall_s_mean`	`float32`	`benchmark_timing`	Latencia media en segundos por episodio completo jugado.	No	Tiempo de ciclo.
`total_steps`	`int32`	`train_summary.json`	Pasos totales de entrenamiento neuronal ejecutados por el optimizador.	No	Volumen de aprendizaje.
`wall_time_seconds`	`float32`	`train_summary.json`	Tiempo acumulado de entrenamiento de la red neuronal.	No	Cómputo invertido.

Matriz de Trazabilidad 100%: Frontend (Dashboard) <--> Métricas Base

A continuación se detalla la correspondencia exacta entre **lo que se ve en la pantalla** del visor y **el origen en el almacén de datos / Parquet**:

Pestaña UI (Dashboard)	Métrica Visible en Pantalla	Clave en Payload JSON	Origen Canónico / Almacén Base	Estado de Trazabilidad
CatalogPage / Header	ELO Ancla (calibrado)	`elo` / `anchor_elo`	`quality_metrics.py` / `compute_anchor_elo` sobre DuckDB	**100% Real (Bradley-Terry)**
CatalogPage / Header	Δ ELO vs Anterior	`deltaElo`	`curriculumMetrics.ts` ($R_t - R_{t-1}$)	**100% Real**
CatalogPage / Header	Tasa de Victoria	`winRate`	`catalog_eval_result_sides.win_rate`	**100% Real**
CatalogPage / Header	Score Canónico	`score`	`catalog_eval_result_sides.score`	**100% Real**
CatalogPage / Header	Puntuación Táctica TQS	`tactical_quality_score`	`quality_metrics.py` (cálculo determinista DuckDB)	**100% Real (Fórmula TQS)**
CatalogPage / Header	Rango Táctico (S/A/B/C/D)	`tactical_grade`	`quality_metrics.py` (umbrales 85/70/55/40)	**100% Real**
CatalogPage / Header	Combat Exchange (CER)	`combat_exchange_ratio`	`quality_metrics.py` ($\frac{\text{Vic}+0.05}{\text{Def}+0.05}$)	**100% Real**
CatalogPage / Header	Brecha Asimetría	`mean_asymmetry_gap`	`quality_metrics.py` ($\lvert S_A - S_B \rvert$)	**100% Real**
CatalogPage / Header	Duración de Fase	`durationSeconds`	`manifest.lock.json` (`phase_finished - phase_started`)	**100% Real**
CatalogPage / Table 1	Desglose Train / Bench	`train_duration_seconds` , `bench_duration_seconds`	`summary.json` & `bench_eval_*.json`	**100% Real**
CatalogPage / Table 1	Autojuego y Optimización	`selfplay_duration_seconds` , `opt_duration_seconds`	`metrics/summary.json` (`timing_summary`)	**100% Real**
TrainDetailTab	Metadatos de Checkpoint	`run_id` , `git_commit` , `phase`	Manifiesto de entrenamiento `meta/manifest.lock.json`	**100% Real**
TrainDetailTab	Pérdidas de Entrenamiento	`loss_policy` , `loss_value` , etc.	`01_train.log` / `train_summary.json`	**100% Real**
TrainHeadDiagnosticsTab	Gradientes y Pérdidas	`policy_grad_norm` , `explained_var`	Registro interno de `trainer.py` exportado en summary	**100% Real**
ObjectiveRewardConfigTab	Parámetros Activos	`objective_pos_weight` , etc.	`trainRow.config` vs defaults de arquitectura	**100% Real (Transparente)**
EvalMetricsTab	Victorias / Empates / Derrotas	`total_victory_rate` , etc.	Filas de evaluación agregadas en DuckDB	**100% Real**
EvalMetricsTab	Métricas de VP y Captura	`vp_net_avg_by_side` , `tracked_cap`	Filas de evaluación agregadas en DuckDB	**100% Real**
EvalMetricsTab	Throughput Computacional	`episodes_per_second` , `wall_s`	`benchmark_timing` medido por `eval_runner.py`	**100% Real**
EvalDecisionsTab	Matriz Acción x Bando	`count` , `damage` , `kills`	Agregación directa de Parquet `eval_decision_summary`	**100% Real (Zero-Mock)**
EvalDecisionsTab	Sobreescritura MCTS	`policy_kept` , `overwritten`	`decision_ownership_by_side` desde Parquet	**100% Real**
MatchReplayTab	Turno y Paso de la Partida	`gameTurn` , `clamped`	`game_turn` / `step` crudos de Parquet (o `-`)	**100% Real (Sin inferencia)**
MatchReplayTab	Postura Doctrinal N1	`macroPosture`	`s.macro_posture` de la traza (o `-` si no existe)	**100% Real (Sin inferencia)**
MatchReplayTab	Rol Táctico de Escuadra	`unitMacroRole`	`s.unit_macro_role` de la traza (o `-` si no existe)	**100% Real (Sin inferencia)**
MatchReplayTab	Búsqueda MCTS (Visitas, Q)	`mcts_total_visits` , `mcts_q`	Nodos raíz de MCTS exportados en el paso	**100% Real**
MatchReplayTab	Estado de Unidades	`hp` , `alive` , `suppressed`	Snapshots autoritativos del motor en el hexágono	**100% Real**
NetworkTopologyTab	Telemetría Táctica Empírica	`assaultPct` , `spearheadPct` , etc.	Calculado estrictamente sobre pasos reales evaluados	**100% Real (N/A si vacío)**
NetworkTopologyTab	Atribución de Canales	`ranked_channels` , pesos IGNH	`channel_attribution.json` generado con PyTorch IG	**100% Real**
VpSummaryTab	Balance Bilateral y Controlador	`ownController` , `oppController`	`controller_by_side` real de la evaluación (sin guesses)	**100% Real**

Auditoría de Redundancias Purgadas y Esquema Parquet

En cumplimiento estricto con las directivas de optimización y fidelidad:

1. **Purgado de Duplicidad `damage_dealt` vs `enemy_damage`**:
    - `damage_dealt` es la columna única y canónica en Parquet.
2. **Purgado de Duplicidad `kills_dealt` vs `enemy_kills`**:
    - `kills_dealt` es la columna única y canónica en Parquet.
3. **Purgado de Triple Registro de Dados**:
    - `dice_rolls` ha sido deprecada en favor de las listas atómicas `attack_dice_rolls` y `defense_dice_rolls`.
4. **Erradicación de Mocks y Deducciones Heurísticas en el Frontend**:
    - Eliminados porcentajes falsos fijos en `NetworkTopologyTab.tsx` (ahora reflejan `N/A` si no hay partidas evaluadas).
    - Eliminada la invención de turnos (`Math.floor(idx / 14) + 1`) y posturas tácticas en `MatchReplayTab.tsx`.
    - Eliminada la deducción de controladores cruzando escenarios y la etiqueta obsoleta `"MuZero"` en `VpSummaryTab.tsx` y `CatalogPage.tsx`.
5. **Esquema Canónico Unificado de `decisions_top.parquet` (15 Columnas Estrictas) y Exclusión de Acumuladores**:
    - Módulo autoritativo: `mlops/dataset_builder.py`.
    - Columnas canónicas obligatorias (15): `decision_type`, `unit_id`, `count`, `damage_dealt`, `kills_dealt`, `cover_avg`, `distance_avg`, `los_avg`, `target_q_avg`, `target_r_avg`, `mcts_overwritten`, `mcts_policy_kept`, `policy_prior_sum`, `mcts_visit_sum`, `mcts_q_sum`.
    - Exclusión terminante de campos acumuladores volátiles en origen (`assault_bench/decision_metrics.py`): `cover_n`, `cover_sum`, `distance_n`, `distance_sum`, `los_n`, `los_sum`, `target_q_n`, `target_q_sum`, `target_r_n`, `target_r_sum`.
    - Homogeneización del 100% de datasets Parquet (69/69 archivos consolidados a 15 columnas) y sustitución radical en DuckDB de etiquetas legacy residuales (`"MuZero"` → `"EfficientZeroV2"`). Verificado con `mlops/tests/test_unified_catalog_and_schema_purge.py`.

Procedimiento de Modificación y Test de Conformidad Automatizado

- **Invarianza del Catálogo**: Cualquier nuevo parámetro o métrica debe registrarse en este documento antes de emitirse a Parquet o consumirse en el frontend.
- **Test de Conformidad de Esquema**: Se incluye el test de regresión `tests/test_telemetry_schema_conformance.py` para garantizar que no existan campos no documentados ni discrepancias entre el almacén analítico y esta especificación.

13. Servicios Backend y Catálogo de APIs

REST FastAPI (:8001), WebSocket binario MessagePack, RAG API y MLOps API (:8778).

07. Servicios Backend y Catálogo de APIs

Este documento constituye la referencia canónica de los servicios backend, microservicios y APIs expuestas en la plataforma **Assault Wargame & AI**. Detalla la arquitectura de comunicación, los contratos de datos, las especificaciones REST y WebSocket, y los enlaces directos a las páginas interactivas de publicación y exploración (**Swagger UI**, **ReDoc** y **OpenAPI JSON**).

---

1. Mapa de Acceso a Servicios y Publicación de APIs

La plataforma opera dos servidores backend concurrentes con responsabilidades segregadas: el **Backend de Simulación e Inferencia Táctica** (FastAPI en puerto 8001) y el **Servidor de Telemetría MLOps y Documentación** (ThreadingHTTPServer / DuckDB en puerto 8778).

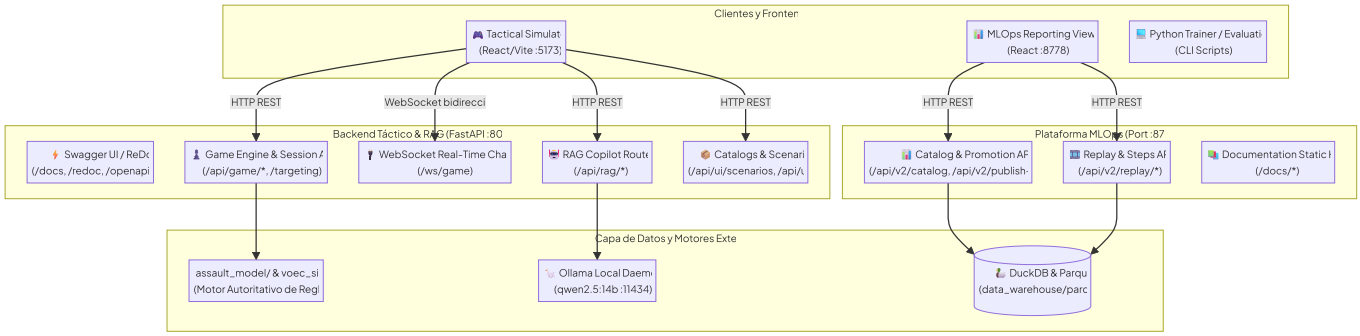
1.1. URLs de Publicación e Interfaces Interactivas

Servicio / Recurso	URL Directa	Descripción / Función
Swagger UI Interactivo	http://127.0.0.1:8001/docs	Explorador interactivo OpenAPI de FastAPI para pruebas en vivo de todos los endpoints tácticos y RAG.
ReDoc Documentation	http://127.0.0.1:8001/redoc	Documentación técnica estructurada en formato de 3 columnas para desarrolladores.
Esquema OpenAPI JSON	http://127.0.0.1:8001/openapi.json	Especificación JSON canónica OpenAPI 3.1 para generación de clientes y validación de esquemas.
Simulador Web Táctico	http://localhost:5173/	Frontend SPA React/Vite para partidas interactivas, visualización de niebla y combate en tiempo real.
Portal MLOps Viewer	http://127.0.0.1:8778/	Dashboard analítico DuckDB/Parquet de modelos, curvas de pérdida y telemetría de entrenamiento.
Tabla de Clasificación Elo	http://127.0.0.1:8778/elo	Ranking Elo histórico de checkpoints de agentes evaluados en self-play y torneos.
Hub de Documentación	http://127.0.0.1:8778/docs/	Portal unificado de arquitectura, reglas, EfficientZero v2, telemetría y APIs.

---

2. Arquitectura de Conectividad e Integración

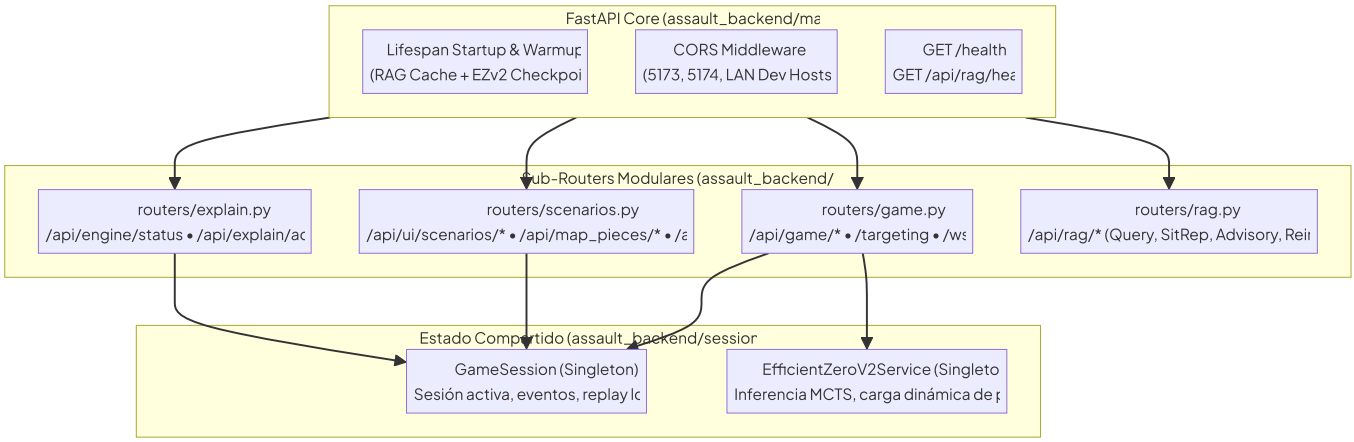
El siguiente diagrama ilustra cómo los clientes (Simulador UI, MLOps Viewer y scripts de entrenamiento) interactúan con los servicios backend:



---

3. Arquitectura Modular del Backend Táctico (`assault_backend/`)

El backend de simulación fue completamente refactorizado y desacoplado, reduciendo el archivo monolítico `main.py` de **1.076 líneas** a **110 líneas limpias y declarativas** orientadas a sub-routers especializados:



3.1. Auditoría y Eliminación de Componentes Obsoletos

1. `assault_backend/services/sb3_ai_service.py` **(ELIMINADO)**:
   - Basado en el stack obsoleto de Stable-Baselines3 (PPO). Decomisionado para cumplir la directiva estricta de usar exclusivamente **EfficientZero V2** como el estándar de IA del proyecto.
   - Se eliminaron las importaciones y la dependencia externa de `stable_baselines3`.
2. `assault_backend/services/scenario_service.py` **(ELIMINADO)**:
   - Código muerto duplicado. El backend y el frontend consumen canónicamente `assault_backend/services/scenario_service_ui.py`.
3. **Limpieza de variables heredadas en** `game_session.py`:
   - Supresión de `sb3_status`, `sb3_status_counts` y redirección hacia `ai_status_counts` con trazabilidad unificada.

3.2. Endpoints de Diagnóstico y Salud

`GET /health`

- **Descripción**: Verifica la disponibilidad del microservicio táctico, la versión y el motor de IA activo.
- **Respuesta Exitosa (200 OK)**:

```
{
  "status": "ok",
  "service": "assault-backend",
  "version": "2.0.0",
  "ai_engine": "EfficientZeroV2",
  "rag_ready": true
}
```

`GET /api/engine/status`

- **Descripción**: Comprueba el estado de inicialización del motor y los servicios de explicabilidad táctica.
- **Respuesta Exitosa (200 OK)**:

```
{
  "status": "ready",
  "engine": "assault-backend",
  "explanations_enabled": true,
  "layers": {
    "strategic": "local_heuristic",
    "tactical": "event_summary"
  }
}
```

3.2. Catálogo de Escenarios, Mapas y Unidades

`GET /api/ui/scenarios`

- **Descripción**: Lista los identificadores de todos los escenarios tácticos disponibles en disco (`assault_sim/assets/scenarios/`).
- **Respuesta Exitosa (200 OK)**:

```
{
  "scenarios": [
    "scenario_gela_beachhead",
    "scenario_biscari_airfield",
    "scenario_counterattack_pz_div"
  ]
}
```

`GET /api/ui/scenarios/{scenario_id}`

- **Descripción**: Carga la configuración completa de un escenario: mapa base, piezas de terreno, unidades asignadas por bando, condiciones de victoria y turnos máximos.
- **Parámetros de Ruta**: `scenario_id` (string, ej: `scenario_gela_beachhead`).
- **Códigos de Respuesta**: `200 OK`, `500 STEP_RUNTIME_ERROR`.

`GET /api/map_pieces`

- **Descripción**: Devuelve la lista de piezas hexagonales prefabricadas en el catálogo.
- **Respuesta Exitosa (200 OK)**: `["piece_01_hills", "piece_02_village", "piece_03_crossroads"]`

`GET /api/map_pieces/{piece_id}`

- **Descripción**: Devuelve la matriz hexagonal, tipos de terreno por hexágono y elevaciones de la pieza solicitada.

`GET /api/units`

- **Descripción**: Catálogo completo de plantillas de unidades (escuadras de infantería, ametralladoras, cañones anticarro, tanques).

`GET /api/units/{unit_key}`

- **Descripción**: Detalle de atributos estáticos de una unidad: dados de combate (`white_dice`, `black_dice`), armadura, puntos de movimiento, alcance de armas y atributos especiales (`HMG`, `MORTAR`, `ARMORED`).

3.3. Ciclo de Vida de Partida y State Loop

`POST /api/game/start`

- **Descripción**: Inicializa una nueva sesión de juego (`GameSession`), carga el escenario, genera las piezas de mapa y posiciona las tropas iniciales.
- **Cuerpo de Solicitud (JSON)**:

```
{
  "scenario_id": "scenario_gela_beachhead",
  "allied_side": "player",
  "axis_side": "ai"
}
```

- **Respuesta (200 OK)**: Snapshot completo inicial del `GameState` serializado.

`GET /api/game/state`

- **Descripción**: Obtiene el estado actual de la partida, incluyendo posiciones de unidades, puntos de vida, estado de cohesión/moral (`NORMAL`, `PINNED`, `SUPPRESSED`, `ROUTED`), control de hexágonos y puntos de victoria (VP).
- **Formato Binario**: Soporta `?format=msgpack` o cabecera `Accept: application/x-msgpack` (RFC-8949).

`GET /api/game/trace`

- **Descripción**: Obtiene la traza secuencial de eventos ocurridos en el turno actual (movimientos, disparos, impactos de dados, chequeos de moral) útil para depuración e historial.
- **Formato Binario**: Soporta `?format=msgpack` o cabecera `Accept: application/x-msgpack`.

`GET /api/game/telemetry`

- **Descripción**: Devuelve en un único paquete unificado el snapshot de estado, la traza de eventos y el registro de sesión (`events` + `session_log`). Diseñado para el streaming de telemetría de baja latencia (Sección 6.5 del Roadmap: reducción > 85 del ancho de banda y latencia $< 5, \text{ms}$ mediante `assault_backend/telemetry_codec.py`).
- **Query Parameters**: `format=msgpack` (devuelve `application/x-msgpack`) o `format=json` (por defecto salvo que se envíe `Accept: application/x-msgpack`).

`POST /api/game/actions`

- **Descripción**: Devuelve la lista exhaustiva de acciones tácticas legales para una unidad específica en su posición actual.
- **Cuerpo de Solicitud (JSON)**:

```
{
  "unit_id": "us_rifle_squad_1"
}
```

- **Respuesta (200 OK)**: Array de acciones válidas codificadas con sus costes en puntos de acción (PA) y coordenadas de destino.

`POST /api/game/step`

- **Descripción**: Aplica una acción táctica autoritativa en el motor. Si la acción provoca un fuego de interrupción enemigo (*Opportunity Fire*), la resolución de la reacción queda pendiente.
- **Cuerpo de Solicitud (JSON)**:

```
{
  "action_type": "MOVE",
  "unit_id": "us_rifle_squad_1",
  "target_coord": {"q": 4, "r": 2}
}
```

- **Respuesta (200 OK)**: Nuevo estado tras la mutación atómica, resultado de dados y flag `reaction_pending`.

`POST /api/game/reaction/resolve`

- **Descripción**: Resuelve el combate de oportunidad desencadenado por el movimiento de una unidad enemiga frente a defensores en guardia.

`POST /api/game/ai-turn`

- **Descripción**: Ejecuta el turno de la IA para el bando activo utilizando el motor estándar `EfficientZeroV2Service` con búsqueda MCTS PUCT.
- **Flujo de Ejecución**:
  1. **Inferencia Neuronal**: Construye la observación espacial (`EZV2Observation`) de 32 canales a partir del estado de la simulación.
  2. **Búsqueda MCTS**: Ejecuta simulaciones MCTS adaptativas guiadas por los priors de la red, el planificador macro de Nivel 1 y las cabezas tácticas (`threat_map` , `attrition` , `morale`).
  3. **Comprobación de Seguridad**: Valida que la acción seleccionada sea legal y no constituya un retroceso táctico injustificado.
  4. **Fallback Heurístico**: Si no hay pesos neuronales disponibles en `runs_curriculum/` , recurre de manera transparente a `DecisionEngine` sin bloquear el frontend.
- **Respuesta Exitosa (200 OK)**:

```
{
  "state": { "...": "GameState Snapshot" },
  "steps": [
    {
      "unit": "US_3",
      "action": "MoveAction",
      "action_id": "MOVE:US_3:13:8",
      "source": "efficientzero_v2",
      "ai_status": "used",
      "ai_reason": "ezv2_mcts_success",
      "move_to": {"q": 13, "r": 8}
    }
  ]
}
```

`GET /targeting`

- **Descripción**: Calcula en tiempo real el trazado de Línea de Visión (LOS), distancia axial/cúbica, coberturas intermedias y modificadores de tiro entre dos coordenadas hexagonales.
- **Query Parameters**:
  - `origin_q` , `origin_r` : Coordenadas del atacante.
  - `target_q` , `target_r` : Coordenadas del defensor.

3.4. Explicabilidad Táctica y WebSocket Binario (`MessagePack`)

`POST /api/explain/activation`

- **Descripción**: Genera una justificación en lenguaje natural de por qué se activó una unidad determinada o por qué la IA seleccionó una ruta táctica específica.
- **Modelo Request**: `ExplainActivationRequest`
- **Modelo Response**: `ExplainActivationResponse`

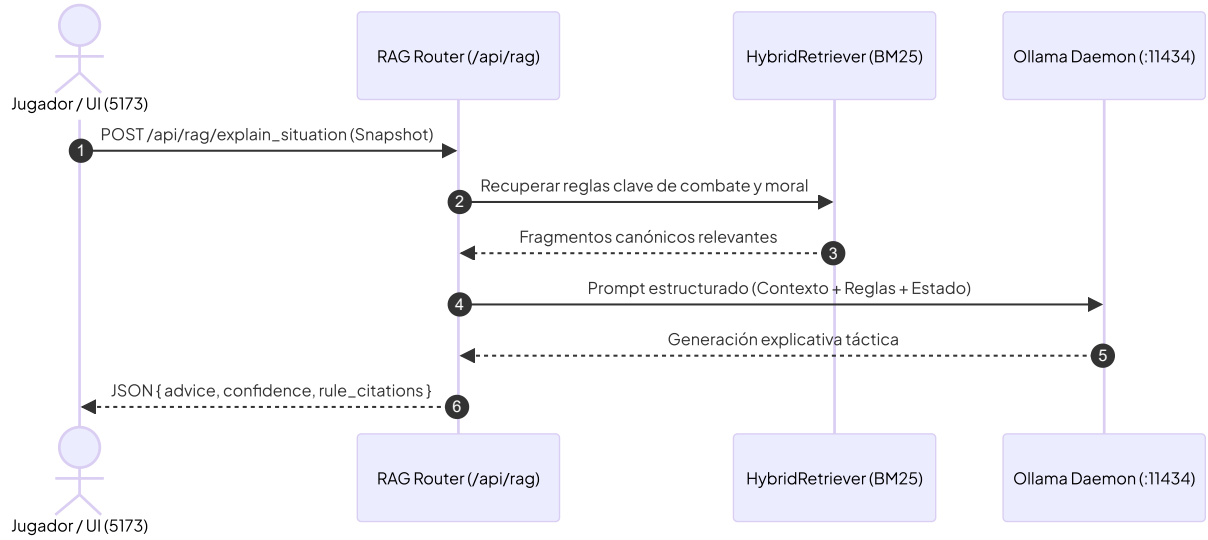
`WS /ws/game` (`?format=msgpack` / `?binary=1`)

- **Descripción**: Canal WebSocket persistente para transmisión bidireccional y actualización de eventos a baja latencia entre el motor de simulación y el interfaz visual (`assault_ai_ui`).
- **Modo Binario MessagePack**: Cuando el cliente conecta con `/ws/game?format=msgpack` (comportamiento por defecto en `GameController.ts`), el servidor emite frames binarios `ArrayBuffer` codificados en MessagePack RFC-8949 (`ws.send_bytes(encode_msgpack(frame))`) con compresión envolvente `Ext(type=1)` para trazas largas, reduciendo más de un 80% el tráfico de red con fallback transparente a JSON texto.
- **Eventos transmitidos**:
  - `STATE_MUTATED` : Notifica cambio en el estado del mapa o de unidades.
  - `COMBAT_RESOLVED` : Notifica tirada de dados, impactos netos y bajas.
  - `OPPORTUNITY_TRIGGERED` : Notifica interrupción por fuego de reacción.
  - `PHASE_TRANSITION` : Notifica cambio de turno o fase de reagrupamiento.

---

4. Catálogo Completo: Sub-Router RAG Copilot (`/api/rag`)

El microservicio de asesoramiento táctico RAG se encuentra en `assault_backend/routers/rag.py` . Integra el recuperador híbrido (BM25 + Dense) con el modelo local **Ollama** (`qwen2.5:14b`).



4.1. Endpoints RAG

Método	Endpoint	Parámetros / Payload	Descripción y Funcionalidad
`GET`	`/api/rag/health`	Ninguno	Estado de precarga del motor RAG en arranque (chunks de datos y reglas).
`GET`	`/api/rag/ollama/status`	Ninguno	Verifica conectividad con el daemon local Ollama en `127.0.0.1:11434` y lista modelos disponibles.
`POST`	`/api/rag/query`	`RagQueryRequest` (`query` , `mode` , `context`)	Consulta semántica abierta sobre el reglamento o el estado táctico.
`GET`	`/api/rag/index/status`	Ninguno	Número de fragmentos procesados en memoria y ruta del archivo índice activo.
`POST`	`/api/rag/reindex`	`{"force": boolean}`	Reconstrucción en caliente del índice de fragmentos de reglas y assets de datos.
`POST`	`/api/rag/explain_action`	`RagExplainActionRequest`	Explica tácticamente la justificación de una acción realizada por la IA.
`POST`	`/api/rag/explain_situation`	`RagExplainSituationRequest`	Diagnóstico estratégico de la situación global del tablero (amenazas, flanqueos, VP).
`POST`	`/api/rag/recommend_actions`	`RagRecommendActionsRequest`	Genera recomendaciones consultivas para el bando solicitante (*Allied* o *Axis*).
`POST`	`/api/rag/training_analysis`	`RagTrainingAnalysisRequest`	Análisis automatizado del desempeño táctico de ejecuciones de entrenamiento N1.

---

5. Catálogo Completo: Plataforma MLOps y Telemetría (`:8778`)

El backend de telemetría reside en `mlops/reporting/viewer.py`. Proporciona acceso ultra-rápido a miles de partidas y entrenamientos persistidos en formato **Parquet** y consultados mediante **DuckDB in-process**.

5.1. Endpoints de Modelos, Catálogo y Telemetría

`GET /api/v2/catalog`

- **Descripción**: Devuelve la lista agregada de todos los modelos entrenados, checkpoints disponibles, número de partidas evaluadas y métricas globales de victoria/derrota.
- **Query Parameters**:
  - `path`: Ruta relativa del archivo de catálogo (por defecto `runs_curriculum/experiments/reporting/catalog_v2.json`).

`GET /api/v2/publish-status`

- **Descripción**: Evalúa los criterios de calidad (*Promotion Gates*) para determinar si el último modelo candidato es apto para su promoción como agente activo.

`GET /api/v2/model`

- **Descripción**: Devuelve la telemetría granular de un modelo concreto: evolución de funciones de pérdida (Value Loss, Policy Loss, Reward Loss, Tactical Heads Loss) a lo largo de los steps.
- **Query Parameters**:
  - `engine`: Nombre del motor (`efficientzero_v2`).
  - `model`: ID del modelo o checkpoint.
  - `train_limit`: Número máximo de registros de entrenamiento a devolver (1–256).
  - `eval_limit`: Número máximo de registros de evaluación a devolver (1–256).

`GET /api/v2/model/trains`

- **Descripción**: Paginación optimizada de ejecuciones de entrenamiento asociadas a un modelo desde la base DuckDB.

`GET /api/v2/model/evals`

- **Descripción**: Paginación de partidas de evaluación disputadas por el modelo, incluyendo oponentes, ratios de victorias y escenarios probados.
- **Query Parameters**:
  - `train_run_id`: Filtrado por ciclo de entrenamiento.
  - `scenario_id`: Identificador del escenario o `__all__`.
  - `offset`, `limit`: Paginador de resultados.
  - `include_payload`: `true` para incluir tensores de observación y distribuciones de acción.

`DELETE /api/v2/model`

- **Descripción**: Eliminación segura de artefactos de un modelo en disco y actualización atómica del catálogo.

`POST /api/model/export-onnx` y `GET /api/model/export-onnx`

- **Descripción**: Exporta los pesos del modelo seleccionado a 3 grafos computacionales ONNX (`initial_inference.onnx`, `recurrent_inference.onnx`, `representation_tower.onnx`) para auditoría e inspección visual en Netron. El método `GET` verifica el estado de artefactos ya generados en disco.
- **Query / Body Parameters**:
  - `run_id`: Identificador de la ejecución o modelo.

`GET /api/model/download-onnx`

- **Descripción**: Descarga en streaming el archivo binario `.onnx` especificado generado para una ejecución.
- **Query Parameters**:
  - `run_id`: Identificador de la ejecución.
  - `file`: Nombre del archivo ONNX a descargar (`initial_inference.onnx`, `recurrent_inference.onnx` o `representation_tower.onnx`).

`GET /api/model/open-folder`

- **Descripción**: Abre directamente en el explorador de archivos del sistema operativo (ej. Windows Explorer) el directorio donde residen los modelos exportados en disco (`runs/<run_id>/onnx/`).
- **Query Parameters**:
  - `run_id`: Identificador de la ejecución.

5.2. Replay y Trazabilidad de Curriculums

`GET /api/v2/replay/meta`

- **Descripción**: Metadatos de una partida grabada (semilla pseudoaleatoria, escenario, duración en turnos, ganador y versión de la red).
- **Query Parameters**: `run_id`, `match_id`.

`GET /api/v2/replay/steps`

- **Descripción**: Secuencia fotograma a fotograma de cada estado hexagonal, acciones tomadas y valores predichos por la red para reproducción visual paso a paso.

`GET /api/v2/replay/log`

- **Descripción**: Registro textual sin procesar del log de eventos de la repetición.

`GET /api/v2/scenario` y `GET /api/v2/scenario/overlay`

- **Descripción**: Provee las capas de superposición táctica (*overlays*) para renderizar mapas de calor de amenazas (*threat map*) y zonas de control territorial calculadas por la red.

`GET /api/v2/curriculum-traceability`

- **Descripción**: Historial completo de transiciones de fase curricular (fases 1 a 5) y evolución de pesos de la red.

---

6. Verificación en Vivo y Pruebas Rápidas con `curl`

Para verificar la disponibilidad de las APIs desde terminal o scripts de integración, pueden ejecutarse los siguientes comandos:

```
# 1. Comprobar disponibilidad del backend táctico (Puerto 8001)
curl -s http://127.0.0.1:8001/health

# 2. Consultar escenarios disponibles
curl -s http://127.0.0.1:8001/api/ui/scenarios

# 3. Comprobar estado de conectividad con Ollama
curl -s http://127.0.0.1:8001/api/rag/ollama/status

# 4. Consultar catálogo MLOps DuckDB (Puerto 8778)
curl -s "http://127.0.0.1:8778/api/v2/catalog"
```

🤖 14. Sistema RAG Copilot

BM25 híbrido, SitRep en vivo, explicabilidad normativa y soporte LLM local.

Sistema RAG Copilot: Asistente Táctico y Explicabilidad Normativa

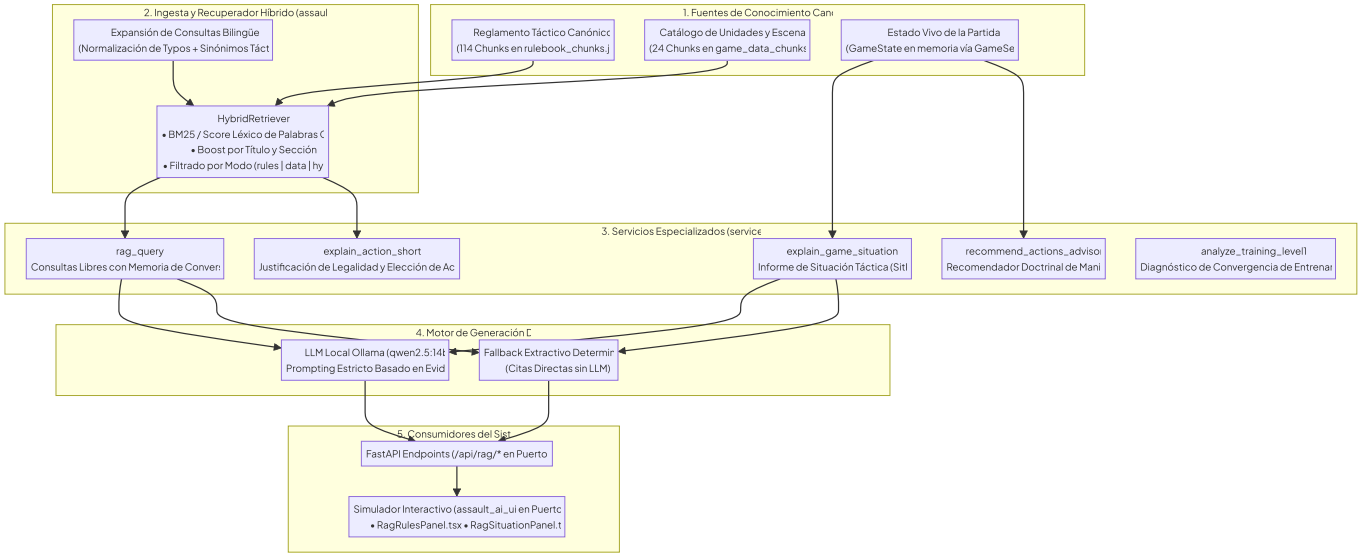
> **Estado**: Canónico y Exhaustivo (2026)
> **Ámbito**: Paquete `assault_rag/`, router FastAPI `assault_backend/routers/rag.py` y paneles UI en `assault_ai_ui/`.

---

1. Visión General y Filosofía de Diseño

El sistema **RAG Copilot** (*Retrieval-Augmented Generation*) de *Assault* actúa como un árbitro normativo, analista situacional y copiloto táctico en tiempo real. Está diseñado con dos principios irrenunciables:

1. **Cero Alucinaciones en Reglas Físicas**: Un wargame no admite invenciones. Toda respuesta relativa a dados, cobertura, modificadores o costes de movimiento debe estar respaldada por citas explícitas a los fragmentos canónicos del reglamento (`rulebook_chunks.json`) y catálogos técnicos (`game_data_chunks.json`).
2. **Resiliencia Determinista (Dual Engine)**: Si el modelo de lenguaje generativo (LLM local vía Ollama) está disponible, el sistema redacta informes tácticos fluidos y concisos. Si Ollama está apagado o sobrecargado, el sistema recurre inmediatamente a un **fallback extractivo determinista**, garantizando que el juego y la interfaz nunca se bloqueen.



---

2. Ingesta y Base de Conocimiento Vectorial / Estructurada

La base de conocimiento de RAG se divide en dos colecciones particionadas en formato JSON:

2.1. Chunks del Reglamento (`rulebook_chunks.json`)

- **Ubicación**: `assault_rag/data/rulebook/chunks/rulebook_chunks.json`.
- **Volumen**: 114 fragmentos canónicos preindexados.
- **Cobertura**:
  - Secuencia de turno, alternancia de activaciones e iniciativa.
  - Geometría hexagonal, cálculo de elevación y trazado de línea de visión (LOS).
  - Terrenos, bocage, bosques, ríos y modificadores de dados de defensa.
  - Fuego a distancia, arcos de fuego, alcance efectivo y fuego de oportunidad/reacción.
  - Asaltos cuerpo a cuerpo en el mismo hexágono y tablas de bajas.
  - Cohesión moral (`NORMAL`, `PINNED`, `SUPPRESSED`, `ROUTED`) y chequeos de pánico.

2.2. Chunks de Datos del Juego (`game_data_chunks.json`)

- **Ubicación**: `assault_rag/data/game_data/chunks/game_data_chunks.json`.
- **Volumen**: 24 fragmentos generados dinámicamente desde `index_builder.py`.

- **Estructura por Tipo**:
  - **Unidades (`unit::*`)**: Perfiles de ataque desglosados por modo de disparo, tipo de blanco (infantería/vehículo) y banda de distancia, más dados asignados (`BattleDie`), puntos de movimiento y atributos de blindaje.
  - **Escenarios (`scenario::*`)**: Condiciones de victoria, límite de turnos, composición de fuerzas atacantes/defensoras y control inicial de banderas VP.
  - **Fortificaciones (`fortification::*`)**: Modificadores de cobertura de trincheras, búnkeres de hormigón, sacos de arena y nidos de ametralladora.

---

3. Recuperación Híbrida (`HybridRetriever`)

El recuperador (`assault_rag/copilot/retriever.py`) optimiza la búsqueda léxica y semántica mediante técnicas específicas para el dominio de juegos de estrategia militar:

3.1. Normalización Tipográfica y Stopwords

Elimina palabras vacías en español e inglés y corrige erratas comunes en nombres de armamento y reglas (ej. `"riffles"` → `"rifles"`, `"uniddaes"` → `"unidades"`).

3.2. Diccionario de Expansión de Consultas (`QUERY_EXPANSIONS`)

Mapea términos coloquiales a conceptos canónicos del motor táctico:

- `"bunker"` → `{"fortification", "casemate", "pillbox", "crossing", "movement", "occupancy"}`
- `"dados"` → `{"die", "dice", "ataque", "defensa", "blancos", "negros"}`
- `"alcance"` → `{"distancia", "range", "hex", "hexes"}`
- `"ficha"` → `{"counter", "icono", "símbolo", "unidad", "unit"}`

3.3. Puntuación Ponderada y Modos de Búsqueda

1. **Modo `rules`**: Busca exclusivamente en los fragmentos del manual de combate.
2. **Modo `data`**: Busca exclusivamente en las fichas técnicas de unidades y escenarios.
3. **Modo `hybrid`** (por defecto): Cruza reglas y datos, asegurando que si el usuario pregunta por una unidad y una regla (ej. *"¿cómo dispara el mortero de 81mm en bosque?"*), el motor recupere tanto el perfil del mortero como las reglas de terreno forestal.

---

4. Servicios Tácticos del Copiloto (`assault_rag/copilot/services.py`)

4.1. Consulta Libre con Memoria Conversacional (`rag_query`)

- Admite diálogo continuo con el usuario mediante un buffer circular de memoria (`_RAG_MEMORY`) configurable (`ASSAULT_RAG_MEMORY_TURNS=6`).
- **Reescritura de Consultas Ambigüas**: Si el usuario realiza una pregunta de seguimiento (ej. *"¿y qué alcance tiene?"* después de preguntar por la ametralladora MG42), el sistema detecta la elipsis y reescribe internamente la consulta contextualizándola antes de la búsqueda.

4.2. Explicador de Acciones (`explain_action_short`)

- Recibe una acción propuesta (a_t) junto con la instantánea del estado (`GameState`).
- Valida si la acción es legal o ilegal según las reglas de movimiento/combate y emite un veredicto explicativo referenciando los artículos exactos del reglamento que habilitan o prohíben la maniobra.

4.3. Informe Situacional Táctico (`explain_game_situation`)

- Analiza exhaustivamente el tablero de juego en vivo:
  - **Balance de Fuerzas**: Unidades operativas vs bajas confirmadas de cada bando.
  - **Estado de Cohesión**: Unidades clavadas (`PINNED`) o suprimidas (`SUPPRESSED`) que requieren reorganización urgente.
  - **Amenazas Inmediatas**: Escuadras enemigas a distancia de asalto o en línea de visión directa.
  - **Objetivos Críticos**: Distancia hexagonal a los hexágonos con Puntos de Victoria (VP) y diferencial actual de marcador.

4.4. Recomendador Asesor de Maniobras (`recommend_actions_advisory`)

- Evalúa todas las acciones legales disponibles para el bando activo y las ordena por una función de valoración táctica: $$\text{Score}(a) = 5 \cdot \mathbb{I}{\text{Captura VP}} + 3 \cdot \mathbb{I}{\text{Fuego Flanco}} + 2 \cdot \mathbb{I}{\text{Mejora Cobertura}} - 4 \cdot \mathbb{I}{\text{Expuesto en Abierto}}$$
- Devuelve al jugador las 3 mejores opciones estratégicas con su correspondiente razonamiento táctico.

4.5. Analizador de Entrenamientos RL (`analyze_training_level1`)

- Diseñado para el flujo de trabajo de los ingenieros de IA:

- Analiza los registros de autojuego de EfficientZero v2, diagnostica si el agente padece de **pasividad táctica** (altos empates por timeout), **colapso de valor** o **descalibración en la cabeza de threat map**, sugiriendo ajustes en los hiperparámetros o recompensas.

---

5. Integración Generativa con LLM Local (Ollama)

Cuando la generación por LLM está habilitada (`ASSAULT_RAG_LLM_ENABLED=1`):

- **Conexión**: Se comunica mediante API REST HTTP local con el servidor **Ollama** (`http://127.0.0.1:11434/api/generate`).
- **Modelo Estándar**: `qwen2.5:14b` (compatible con `deepseek-r1:14b` , `qwen3-coder:latest` y familias Llama 3).
- **Parámetros**: Temperatura baja ($T = 0.2$) para máxima fidelidad fáctica.
- **Prompting Blindado**:

```
Eres un copiloto de reglas de un wargame táctico.
Responde en español, corto y preciso.
Usa SOLO la evidencia dada. Si no alcanza, di que falta evidencia.
Incluye al final una línea 'Base:' con ids de evidencia usados.
```

---

6. Endpoints REST de la API (`assault_backend/routers/rag.py`)

El backend FastAPI en el puerto `8001` expone la suite completa bajo el prefijo `/api/rag` :

Método	Endpoint	Descripción	Parámetros Clave
`GET`	`/api/rag/ollama/status`	Comprueba conectividad con Ollama y lista modelos disponibles.	Ninguno.
`POST`	`/api/rag/query`	Ejecuta una consulta RAG con evidencia y citas.	`query` , `mode` (`rules` / `data` / `hybrid`), `context` .
`GET`	`/api/rag/index/status`	Devuelve el recuento de chunks indexados y rutas activas.	Ninguno.
`POST`	`/api/rag/reindex`	Fuerza la reindexación de reglas y catálogo de datos.	`force: true/false` .
`POST`	`/api/rag/explain_action`	Explica causalmente una acción táctica ejecutada o propuesta.	`action` , `state_snapshot` , `trace_context` .
`POST`	`/api/rag/explain_situation`	Genera un informe táctico (SitRep) de la situación actual.	`state_snapshot` .
`POST`	`/api/rag/recommend_actions`	Recomienda las jugadas prioritarias para un bando.	`side` , `state_snapshot` , `constraints` .
`POST`	`/api/rag/training_analysis`	Diagnostica la calidad y estabilidad de runs de entrenamiento.	`runs` (lista de métricas de autojuego).

---

7. Interfaz de Usuario en el Simulador (`assault_ai_ui`)

En el simulador interactivo (`http://localhost:5173/`), el sistema RAG se materializa en tres paneles arrastrables:

1. `RagRulesPanel.tsx` : Ventana flotante donde el jugador puede escribir preguntas en lenguaje natural sobre cualquier aspecto del manual mientras juega.
2. `RagSituationPanel.tsx` : Resumen táctico en tiempo real del turno en curso, alertando sobre escuadras en peligro de flanqueo o pérdida inminente de banderas de victoria.
3. `RagEvalPanel.tsx` : Panel de auditoría para inspeccionar los tensores y la justificación de las decisiones tomadas por el agente de red neuronal.

🕹️ 15. Manual del Simulador Táctico y HUD (:5173)

Guía de usuario del simulador táctico: mapa full-bleed, ribbon de mando, command deck, drawer lateral y atajos.

14. Manual de Usuario y Guía de Interfaz del Simulador Táctico (:5173)

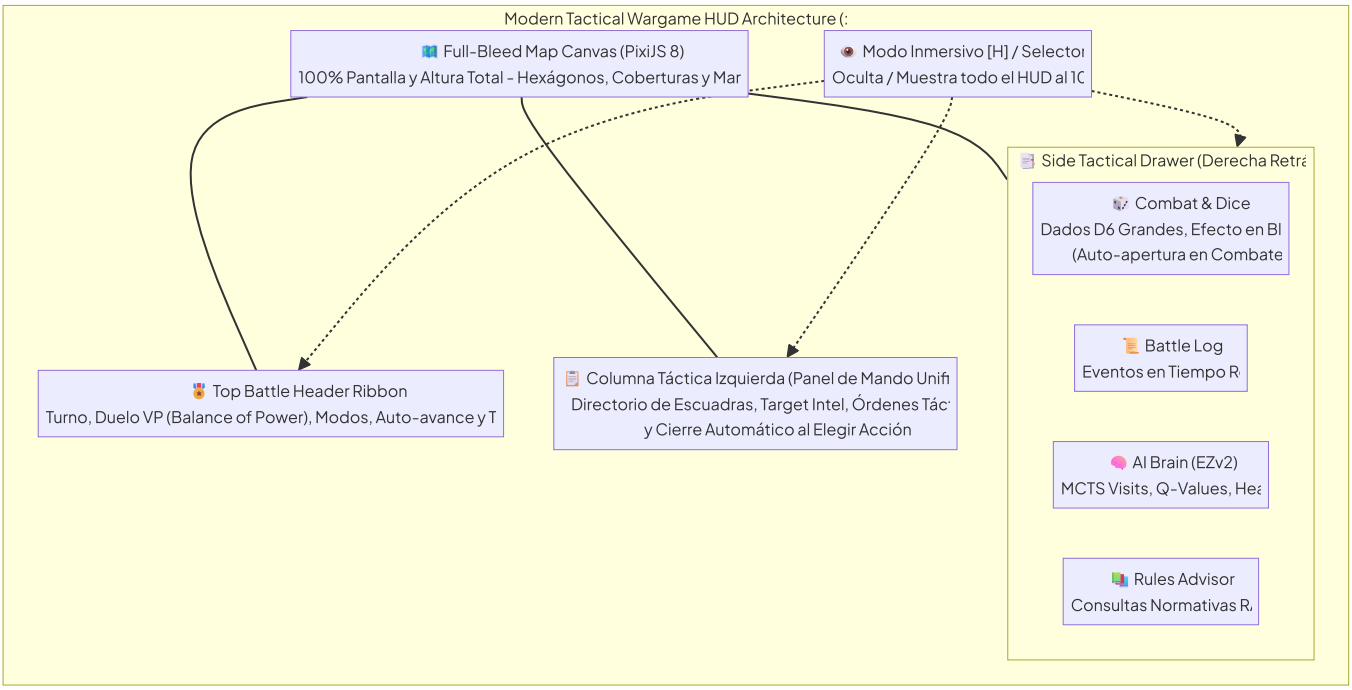
> **Estado**: Canónico y Actualizado (Octubre 2026 - Versión `v2.5.37`)
> **Servicio Frontend**: Simulador Táctico Militar Assault (`assault_ai_ui`) en `http://localhost:5173/`
> **Tecnologías Core**: Vite 8 + React 19 + PixiJS 8 + TypeScript + Glassmorphism CSS
> **Directiva de Idiomas (Regla 7)**: Interfaz de usuario (UI), código fuente y registros 100% en **Inglés**. Manual de usuario y documentación técnica en **Español**.

---

1. Visión General y Propósito Operativo

El **Simulador Táctico Militar (:5173)** es el entorno interactivo visual de primera línea de la plataforma **Assault**. Diseñado como un banco de pruebas táctico y estación de mando para comandantes humanos e investigadores de IA, permite:

1. **Evaluación Táctica en Vivo (Human-in-the-Loop)**: Despliegue de órdenes directas por parte de un jugador humano contra el agente neuronal de aprendizaje por refuerzo **EfficientZero V2**.
2. **Supervisión de Autojuego y Evaluación (AI vs AI)**: Visualización paso a paso o en flujo continuo de partidas de evaluación entre agentes `EfficientZeroV2` o contra baselines heurísticos/aleatorios.
3. **Reproducción Forense de Sesiones (Replay Engine)**: Carga y análisis táctico celda a celda de trazas generadas por el pipeline de evaluación y torneos.
4. **Verificación Normativa Instantánea**: Inspección visual de reglas de línea de visión (LOS), sectores de cobertura, penalizaciones de movimiento y ventanas de fuego de reacción bajo el reglamento canónico de *Assault: Sicily 1943*.



---

2. Filosofía Arquitectónica: De Ventanas Flotantes al Modern Wargame HUD

En versiones anteriores (`v1.x` y primeras iteraciones de `v2`), la interfaz empleaba múltiples ventanas flotantes independientes (`DraggableWindow`): *Target Intel*, *Active Units (1450px)*, *Combat Dice*, *System Log*, *Match Status*, *AI Telemetry* y *RAG Copilot*. Este diseño provocaba saturación visual, solapamiento masivo sobre el tablero y obligaba al usuario a reorganizar o minimizar ventanas continuamente para ver el terreno.

A partir de la versión `v2.5.35`, la interfaz adopta el paradigma **Modern Tactical Wargame HUD** (inspirado en títulos tácticos de referencia como *Unity of Command 2* y *Company of Heroes*):

- **Principio *Map-First***: El mapa táctico es el protagonista indiscutible y llena el 100% de la ventana sin divisiones fijas de página.
- **HUD No Invasivo con Glassmorphism**: Todos los elementos de control flotan como capas translúcidas perimetrales (`backdrop-filter: blur(16px)`), garantizando visibilidad periférica del campo de batalla.

- **Acceso por Demanda**: Los paneles densos de telemetría y resolución de combate se retiran a un cajón lateral deslizante que no interfiere en la toma de decisiones sobre el mapa.
- **Modo Inmersivo Limpio**: Conmutación total mediante una sola tecla (`H`) para transformar la aplicación en un visor cartográfico puro.

---

3. Módulo 1: Canvas Hexagonal a Pantalla Completa (PixiJS 8)

El centro de la aplicación está gobernado por `GameCanvas.tsx`, un renderizador vectorial y raster acelerado por hardware mediante **PixiJS 8**.

3.1. Navegación y Control de Cámara

- **Desplazamiento Panorámico (Pan)**: Arrastre con clic primario o secundario sobre cualquier zona vacía del terreno.
- **Zoom Dinámico**: Rueda del ratón (*mouse wheel*). El zoom escala el mundo vectorial respecto a la posición actual del cursor.
- **Enfoque Automático (Auto-fit)**: Al inicializar una partida o cambiar de escenario, la cámara encuadra automáticamente todos los hexágonos del mapa con un margen perimetral seguro de $2.6 \times \text{HEX_SIZE}$.
- **Centrado en Unidad**: Al hacer clic en cualquier ficha del carrusel de escuadras o mapa, la cámara realiza una transición suave hacia las coordenadas axiales (q, r) de la unidad.

3.2. Barra Flotante de Capas Tácticas (`LayerControls`)

Ubicada de forma discreta bajo el ribbon superior (esquina superior izquierda, `top: 72px`), permite activar o desactivar tres capas esenciales:

Capa	Estado por Defecto	Descripción
Map	Activada	Imagen de satélite/tablero geográfico original con texturas de terreno (bosques, ríos, ciudades).
Grid	Activada	Rejilla vectorial de bordes hexagonales con relieve táctico.
Coords	Desactivada	Renderizado de coordenadas axiales numéricas (q, r) en el centro de cada hexágono.

3.3. Marcadores Oficiales y Discriminación Visual de Activación sobre el Tablero

El motor visual proyecta sobre los hexágonos y fichas los marcadores de reglamento canónico y una diferenciación inmediata del estado de activación:

1. **Estado de Activación de Unidades en el Mapa**:
   - **Unidades Listas (`READY`)**:
     - Ficha nítida con opacidad al 100% (`alpha = 1.0`).
     - **Anillo Táctico Resplandeciente**: Anillo exterior perimetral en cian esmeralda (`0x00f0ff` , pulso senoidal animado) que rodea la ficha sin ocultarla.
     - **Insignia** `READY` : Etiqueta táctica en verde esmeralda brillante (`#10b981`) visible en la parte inferior de la ficha.
   - **Unidades que ya han actuado (`DONE`)**:
     - Ficha atenuada (`alpha = 0.65`) con tinte pizarra tenue (`tint = 0x94a3b8`).
     - **Insignia** `✓ DONE` : Etiqueta en gris pizarra neutro (`#64748b`) confirmando que su turno se ha consumido.
   - **Unidades Hostiles (`OPFOR`)**: Borde y retícula de puntería en naranja/rojo táctico.
2. **Marcadores de Acción Activa**: `firing.png` (disparo realizado), `fast_action.png` (movimiento rápido ejecutado), `normal.png` (orden estándar completada), `turret_firing.png` (disparo de torreta blindada).
3. **Marcadores de Estado**: `Status Suppressed.png` (unidad bajo fuego de supresión / desorganizada), `Marker Hit.png` (impacto balístico recibido).
4. **Bajas y Destrucción**: `Markers USA Dead.png` , `Markers Italy Dead.png` o `Markers Germany Dead.png` marcando la posición de eliminación definitiva.

---

4. Módulo 2: Barra Superior Táctica (Top Battle Header Ribbon)

La barra superior flotante (`.battle-header-ribbon`) centraliza el control de sesión y el estado estratégico global de la partida.

4.1. Zona Izquierda: Configuración y Modos

- **Brand & Versión**: Identificador `ASSAULT // EZv2` indicando que el motor opera bajo el stack canónico de EfficientZero V2.
- **Selector de Escenario**: Menú desplegable con los 38 escenarios canónicos disponibles (ej. `Initial Contact` , `Sicily Gela Counterattack` , etc.).
- **Selector de Bando Humano**: Determina qué facción controla el jugador humano (`US` , `GE` , `IT` , `UK`). Se sincroniza dinámicamente según las facciones legales del escenario.
- **Conmutadores de Modo**:
  - `Human` : Partida interactiva Humano vs IA.

    - `AI vs AI`: Autojuego autónomo entre agentes neuronales.
    - `Replay`: Reproducción de partidas archivadas.
- **Botón** `▶ Start`: Inicializa el estado del juego, resetea la cámara y conecta el WebSocket de eventos.
- **Botón** `Stop`: Detiene de forma segura el bucle de simulación y restablece el tablero.

4.2. Zona Central: Turno, Fase y Duelo de Puntos de Victoria (Balance of Power)

- **Insignia de Turno**: Muestra el turno actual (ej. `TURN 3`) y el bando activo con su controlador (`US (HUMAN)` o `GE (AI)`), respaldado por un **pulso luminoso animado** (verde para humano, naranja para IA).
- **Barra de Dominancia / Duelo de VPs (Balance of Power)**:
    - Muestra las puntuaciones acumuladas de Puntos de Victoria de ambos contendientes: `[US: 12 VP]` vs `[GE: 8 VP]`.
    - Un medidor visual bicolor (*tug-of-war bar*) calcula el porcentaje relativo de control territorial y se desplaza en tiempo real ante cada captura o pérdida de objetivo.
- **Indicador de Objetivos (OBJ)**: Muestra el recuento de hexágonos de victoria controlados sobre el total del escenario (ej. `OBJ: 2/3`).

4.3. Zona Derecha: Avance, Trazas, Cajón y Modo Inmersivo

- **Casilla** `Auto`: Activa o desactiva el auto-avance tras las acciones humanas. Si está marcado, la IA responde inmediatamente tras la orden del jugador; si está desmarcado, el flujo espera la confirmación del usuario.
- **Botón** `▶ Step`: Permite avanzar manualmente un único paso táctico de la IA cuando el autoavance está deshabilitado.
- **Botón** **(Export Trace)**: Descarga un archivo JSON estructurado con la telemetría completa de la partida para su posterior análisis o auditoría forense.
- **Botones Disparadores del Cajón**: Botones de acceso directo a las 4 pestañas del cajón lateral:
    - `Combat`: Incluye una insignia de alerta roja `!` cuando hay una tirada de dados reciente.
    - `Log (N)`: Muestra el recuento de eventos registrados en la sesión.
    - `AI`: Abre la telemetría del árbol MCTS.
    - `Rules`: Despliega el asesor normativo RAG.
- **Botón Ojo (** **)**: Oculta inmediatamente todos los componentes del HUD para disfrutar de la vista limpia del mapa (equivalente al atajo `H`).

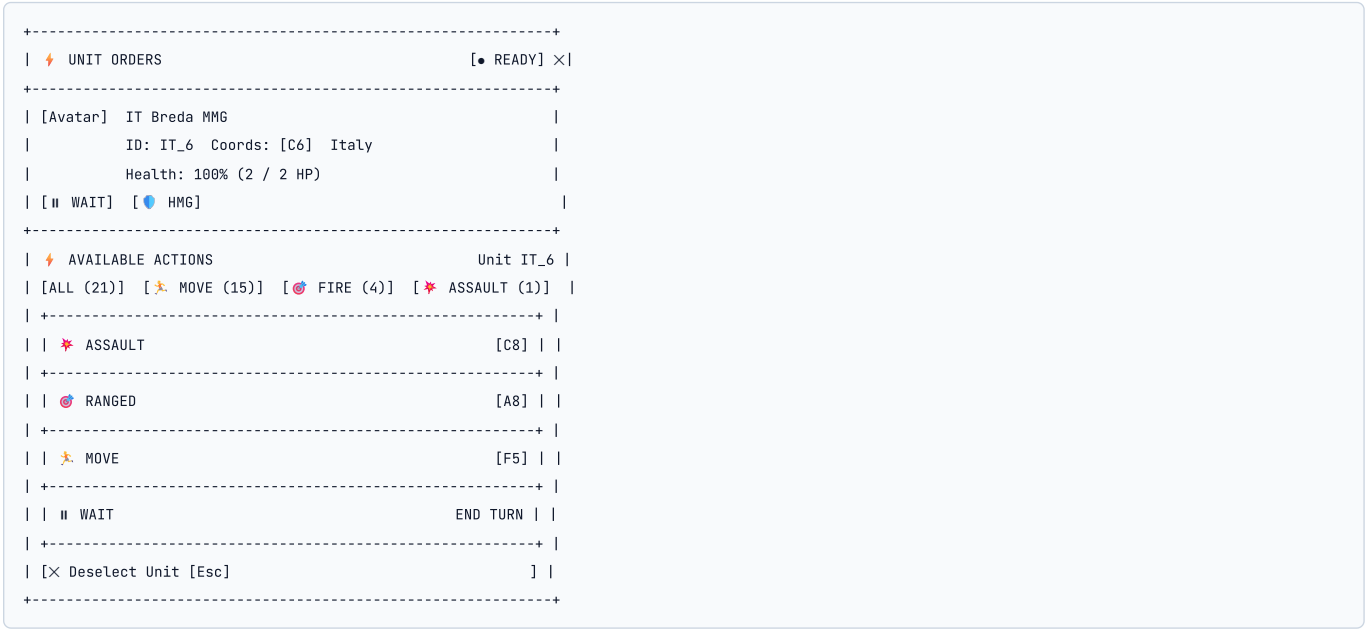
---

5. Módulo 3: Plataforma de Mando Inferior (Bottom Command Deck)

---

5. Módulo 3: Columna Táctica Izquierda Unificada (`Left Command & Squad Panel`)

Para maximizar la visibilidad vertical del mapa y evitar que los paneles obstaculicen la base de la pantalla o colisionen entre sí, la versión `v2.5.37` unifica la inspección de tropas y la paleta de órdenes en una **columna táctica lateral izquierda limpia y desacoplada** (`.left-squad-drawer`, ancho de $380\ \text{px}$, `top: 112px`, `bottom: 16px`, `left: 14px`). Esta columna se sitúa inmediatamente debajo de la barra de capas (`Map / Grid / Coords`) y erradica por completo la barra inferior (`bottom-command-deck` y `footer-squad-dock`), suprimiendo cualquier solapamiento visual y concediendo el 100% de la altura de pantalla al mapa táctico PixiJS.



5.1. Modo A: Ficha de Unidad y Paleta de Acciones (`Unit Command Mode`)

Se activa automáticamente cuando el comandante hace clic en cualquier unidad sobre el mapa táctico o en el directorio de escuadras:

1. **Cabecera Táctica**:
    - Botón `← SQUADS`: Regresa al directorio general de tropas para cambiar de escuadra.
    - Píldora de estado balístico: `● READY` (verde resplandeciente), `✓ DONE` (gris pizarra), `OPFOR` (enemigo) o `KIA` (baja).
    - Botón `×`: Repliega el panel para examinar el mapa con total libertad.
2. **Tarjeta de Inteligencia Balística (Target Intel)**:
    - Avatar en alta definición y denominación reglamentaria de la unidad.
    - Posición axial precisa (`Q: 5, R: 3`) y bando faccional (`US`, `GE`, `IT`).
    - Barra de salud operativa dinámica y pips de corazón (❤️ ❤️ ❤️ ❤️ `4/4 HP`).
    - Distintivos tácticos de especialización: `💥 Mortar / No-LOS`, `🛡️ Suppression / HMG`, `⚔️ Assault & Breach`.
    - Botón directo `⏸ WAIT`: Permite finalizar voluntariamente la activación cediendo la iniciativa.
3. **Paleta de Selección por Categoría (`DispatchedOrdersPanel`)**:
    - Pestañas con contadores en tiempo real: `ALL (N)`, `🏃 MOVE (N)`, `🎯 FIRE (N)`, `💥 ASSAULT (N)`, `⏸ WAIT (N)`.
    - Tarjetas de acción interactivas con previsualización balística en mapa al posar el cursor (*hover*).
    - **Cierre Automático Inteligente**: Al hacer clic en cualquier acción, la orden se envía al simulador y **la columna lateral izquierda se repliega y cierra automáticamente al instante**, otorgando el 100% de visibilidad a la maniobra táctica y a la tirada de dados D6 en el panel de combate.

5.2. Modo B: Directorio de Escuadras (`Order of Battle Mode`)

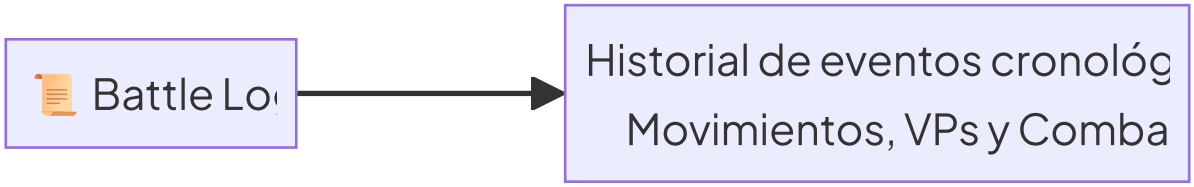
Cuando no hay ninguna unidad seleccionada (o tras pulsar `← SQUADS` o el botón flotante `📋 SQUADS`), el panel se transforma en un explorador completo del ejército:

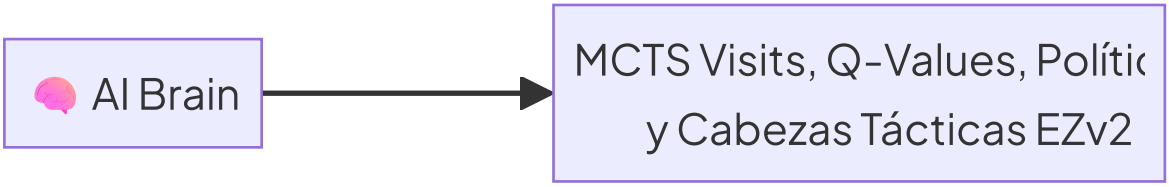
1. **Barra de Búsqueda y Filtrado Instantáneo**:
    - Búsqueda en vivo por nombre de unidad, armamento o identificador militar.
    - Filtros de activación: `ALL`, `READY (N)` (escuadras pendientes de actuar) y `DONE (N)` (escuadras que ya consumieron su turno).
    - Filtros de facción: `ALL SIDES`, `US`, `GE`, etc.
2. **Tarjetas de Escuadra en Gran Formato (`squad-row-card`)**:
    - Avatar táctico, salud operativa (corazones y barra de integridad), rol balístico y contador de órdenes legales (`⚡ N actions`).
    - Al hacer clic en cualquier ficha, la cámara viaja suavemente hacia la unidad en el mapa y la columna pasa inmediatamente al **Modo A (Ficha y Acciones)**.

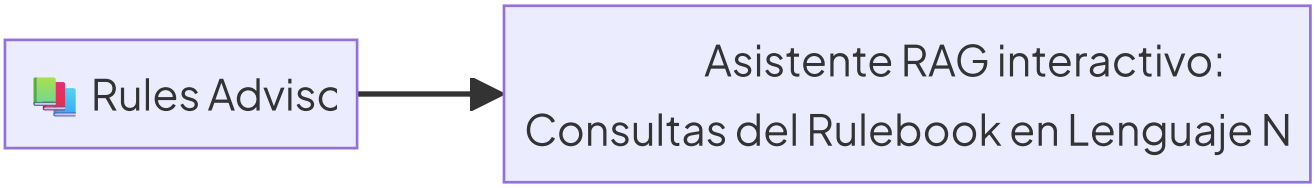
---

6. Módulo 4: Cajón Táctico Lateral Retráctil (Side Tactical Drawer)

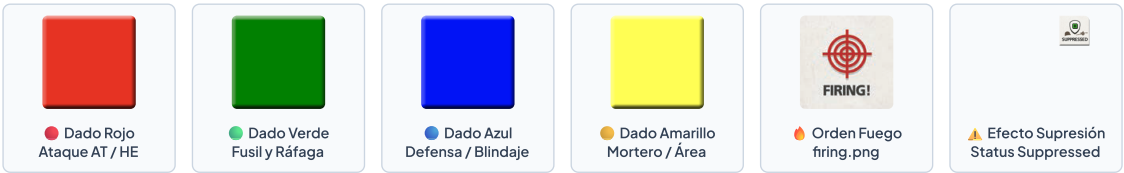
El cajón lateral derecho (`.side-tactical-drawer`, ancho de $440\ \text{px}$) se desliza suavemente sobre el mapa al activarse cualquier pestaña desde la barra superior o al desencadenarse eventos de combate. Puede cerrarse en cualquier momento pulsando el botón `×` o la tecla `Esc`.







6.1. Pestaña 🎲: Combate, Dados Tácticos de Gran Formato y Efecto sobre el Blanco (`CombatPanel`)



- **Apertura Reactiva Inteligente**: Cuando cualquier unidad (humana o IA) resuelve un ataque, el cajón lateral **se abre automáticamente** en esta pestaña para que el usuario pueda presenciar el intercambio de fuego sin interrumpir la partida.
- **Dados Balísticos Tácticos de Gran Formato (44×44 px)**:
  - Visualización ampliada a más del doble del tamaño original con texturas de alta resolución, bordes luminiscentes por color (Rojo, Amarillo, Verde, Azul) y animación física de giro balístico (`dice-roll`).
  - **Tooltips Informativos al Hover**: Al posar el cursor sobre cada dado se detalla el color y la cara obtenida (ej. *"RED Die: CRITICAL + DAMAGE"*, *"YELLOW Die: DAMAGE"*, *"BLUE Die: DEFENSE / SHIELD"*).
  - Separación nítida en dos reservas: `ATTACK POOL (N Dice)` y `DEFENSE POOL (N Dice)` .
- **Tarjeta Táctica de Efecto Integral sobre el Blanco (`combat-target-effect-card`)**:
  - Explica de forma inmediata y sin ambigüedades el impacto exacto sobre la unidad defensora:
    - 💀 `TARGET DESTROYED (KIA)` : Bando/unidad aniquilada y retirada del mapa tras agotar sus puntos de vida.
    - 💥 `TARGET PENETRATED (-N HP)` : Daño cinemático que penetró la cobertura, indicando la pérdida exacta de salud (`HP Before → HP After`).
    - ⚠️ `TARGET SUPPRESSED / PINNED DOWN` : Los dados defensivos absorbieron el daño a la salud (0 HP perdidos), pero el volumen de fuego causó supresión moral (la escuadra queda clavada al terreno, pierde movilidad rápida y sufre penalizaciones en fuego de reacción).

        - `🛡️ ATTACK DEFLECTED / ABSORBED BY COVER`: La tirada de defensa y la cobertura de terreno neutralizaron por completo el ataque (0 daño, salud intacta).
    - **Seguimiento Visual de Puntos de Impacto**: Fila de corazones interactivos mostrando el estado antes y después (corazones activos ❤️ frente a corazones perdidos en el asalto 💔).
    - **Desglose Técnico de Resolución**: Daño no cancelado, tiradas de supresión resultantes y efectos críticos específicos.

6.2. Pestaña 📜: Registro de Batalla (`Battle Log`)

- Registro cronológico inalterable con marcas de tiempo (`[HH:MM:SS]`).
- Código cromático de eventos:
    - **Azul**: Eventos de sistema y configuración.
    - **Púrpura**: Cambios de turno y relevos de fase.
    - **Cian**: Bloqueo y selección de objetivos.
    - **Verde**: Movimientos tácticos y desplazamientos de hexágono.
    - **Rojo**: Fuego balístico, resoluciones de daño y bajas confirmadas.
    - **Dorado / Amarillo**: Capturas y transiciones de Puntos de Victoria.

6.3. Pestaña 🧠: Cerebro y Telemetría de la IA (`EfficientZeroV2TelemetryPanel`)

Acceso directo a la mente del agente neuronal en tiempo real:

- **Métricas de Búsqueda MCTS**: Número de simulaciones ejecutadas, profundidad media del árbol y valor esperado $Q(s, a)$.
- **Distribución de Política (π)**: Probabilidades asignadas a cada orden candidata en el nodo raíz.
- **Cabezas Tácticas Auxiliares**: Lecturas de mapa de amenazas (`threat_map`), desgaste proyectado (`attrition`), moral esperada y predicción de posesión de VPs.
- **Botón de Refresco**: Permite forzar una actualización instantánea contra el endpoint de telemetría de FastAPI (`/api/game/telemetry`).

6.4. Pestaña 🗺️: Asesor Normativo y Situacional (`RagAssistantPanel`)

- Sistema integrado de asistencia en lenguaje natural conectado con el motor RAG Copilot de la plataforma.
- Permite formular preguntas sobre reglas específicas de *Assault* (ej. *¿Cómo afecta el humo a la línea de visión?*, *¿Qué penalización sufre una unidad al retirarse en terreno abierto?*).
- Analiza el estado actual de la partida (`gameData`) para ofrecer asesoramiento táctico contextualizado.

---

7. Ventana de Fuego de Reacción (Reaction Fire Modal)

Cuando una unidad enemiga ejecuta un movimiento que cruza el arco de fuego de una unidad defensora preparada, el motor detiene temporalmente la ejecución e invoca el diálogo modal de **Fuego de Reacción** (`Reaction Fire Opportunity`):

```
+-------------------------------------------------------------------+
| ⚡ REACTION FIRE OPPORTUNITY                                      |
|                                                                   |
| Enemy movement triggered a reaction fire opportunity.             |
| Reactor: US Ranger Assault Company                                |
| Target:  GE Panzergrenadier Platoon                               |
|                                                                   |
|                          [⏭ Skip]   [ ⚡ Use Reaction Fire]      |
+-------------------------------------------------------------------+
```

- `⚡ Use Reaction Fire`: La unidad defensora interrumpe el movimiento hostil y ejecuta un ataque inmediato de interdicción balística.
- `⏭ Skip`: Cede la oportunidad de fuego de reacción preservando la activación de la unidad para fases posteriores.

---

8. Guía de Atajos de Teclado y Controles de Ratón

Acción / Control	Tecla / Gesto	Ámbito	Descripción
Alternar Modo Inmersivo	`H`	Global	Oculta o restaura todo el HUD al 100% para despejar el mapa táctico.
Alternar Directorio de Escuadras	`S` / Botón 📋	Global	Abre o repliega el directorio vertical de escuadras en la columna izquierda.
Cerrar Cajón / Deseleccionar	`Esc`	Global	Cierra el cajón lateral o el directorio si están abiertos, o deselecciona la unidad activa.
Desplazamiento del Mapa	Clic y Arrastre	Canvas	Mueve la vista panorámica del campo de batalla.
Zoom In / Zoom Out	Rueda del Ratón	Canvas	Acerca o aleja la cámara centrada en el cursor.
Seleccionar Unidad	Clic Izquierdo	Mapa / Directorio	Selecciona una unidad y despliega sus órdenes tácticas.

Acción / Control	Tecla / Gesto	Ámbito	Descripción
Despachar Orden Rápida	Clic Izquierdo	Paleta de Órdenes	Envía la acción elegida y cierra automáticamente el directorio de escuadras.
Finalizar Turno de Unidad	Botón `WAIT`	Target Intel	Consume o cede la activación de la unidad seleccionada.

---

9. Verificación de Funcionamiento y Diagnóstico

Para iniciar el simulador táctico y verificar su correcto funcionamiento:

1. **Arranque Completo del Stack**: Ejecutar desde el directorio raíz el script unificado:

   ```
   .\start_assault.bat
   ```

   Esto levantará el backend FastAPI (`:8001`), el servidor de telemetría y MLOps (`:8778`), el editor de unidades (`:8780`) y el simulador de usuario (`:5173`).

2. **Acceso al Simulador**: Abrir en cualquier navegador moderno: http://localhost:5173/

3. **Verificación de Compilación Frontend**: Para compilar y verificar la ausencia de errores en TypeScript y CSS:

   ```
   npm --prefix assault_ai_ui run build
   ```

   El proceso debe concluir en menos de 500 ms con cero advertencias o errores de tipado.

📋 16. Informes de Ejecución: Cabecera y Catálogo

Protocolo de auditoría forense, taxonomía TQS y repositorio centralizado de informes.

Cabecera Canónica del Repositorio de Informes de Ejecución (MLOps & Táctica)

> **Documento Rector del Catálogo de Informes de Ejecución y Auditorías Forenses**
> **Sistema Autoritativo:** Assault MLOps Reporting Framework (:8778)
> **Ámbito:** `runs_curriculum/` • `assault_bench/` • `mlops/reporting/` • `agents/efficientzero_v2/`

---

1. Misión y Propósito de los Informes de Ejecución

Los **Informes de Ejecución** constituyen el registro forense oficial de cada campaña de entrenamiento, evaluación comparativa bilateral y verificación de currículums tácticos en la plataforma Assault.

A diferencia de los paneles en vivo de telemetría, cada informe de ejecución documenta:

1. **Condiciones Experimentales Estrictas**: Versión exacta del código, hashes de modelos, semillas aleatorias, presupuestos de simulación MCTS y topología sensorial.
2. **Diagnóstico Forense de Rendimiento**: Desglose cuantitativo del Tactical Quality Score (TQS), ELO anclado Bradley-Terry, tasas de victoria por bando (*Side A* vs *Side B*), y Combat Exchange Ratio (CER).
3. **Auditoría de Patologías Tácticas**: Detección temprana de parálisis ofensiva, aversión espuria al fuego de reacción, colapso de gradientes multi-tarea o sobreajuste posicional.
4. **Matriz de Mitigación y Acciones Correctivas**: Vinculación directa con las tareas prioritarias de la Hoja de Ruta Evolutiva.

---

2. Taxonomía Estándar de Evaluación y Calificación (TQS)

Cada informe de ejecución evalúa el desempeño táctico del agente y emite un veredicto formal estandarizado:

$$\text{TQS} = w_{\text{elo}} \cdot \text{Score}_{\text{ELO}} + w_{\text{mirror}} \cdot \text{Score}_{\text{Mirror}} + w_{\text{cer}} \cdot \text{Score}_{\text{CER}} + w_{\text{decisive}} \cdot \text{Score}_{\text{Decisive}} - \text{Penalización}_{\text{Asimetría}}$$

Rango TQS	Calificación	Estado de Puerta	Criterio Operativo
≥ 90.0	**A (Excelente)**	**PROMOTE**	Agente tácticamente superior al humano, asalto agresivo y balance simétrico.
$75.0 - 89.9$	**B (Aceptable)**	**PASS**	Competente en combate, cumple requisitos de victoria en ambos bandos.
$60.0 - 74.9$	**C (Regular)**	**WARN**	Desequilibrio entre bandos o dependencia de pasividad defensiva.
$40.0 - 59.9$	**D (Deficiente)**	**BLOCK**	Parálisis ofensiva detectable o brecha asimétrica ≥ 0.35. Bloqueo de promoción.
< 40.0	**F (Crítico)**	**REJECT**	Regresión balística, incapacidad de captura de VPs o colapso de ELO.

---

3. Catálogo Cronológico de Informes de Ejecución

N.º	Identificador del Informe	Fecha	Versión / Commit	Alcance / Campaña	Calificación	Estado	Tareas Roadmap Derivadas
01	INFORME 01: Ejecución del Currículum Multi-Fase y Auditoría Táctica Forense	20 Oct 2026	`v2.5.2-legacy`	31 ciclos de currículum en 12 fases (`self_vs_self_vs_random_2scenarios_v1`), evaluación bilateral 100 partidas vs Random *(Legacy Telemetry)*	**B (TQS: 58.5 - WARN)**	Auditado (Disclaimer)	Tarea 3.6 (Mitigación Parálisis Ofensiva y 38 Canales)
02	INFORME 02: Ejecución del Currículum Multi-Fase y Auditoría Táctica Forense	21 Oct 2026	`v2.5.2-forensics` (`ab71541`)	34 ciclos de currículum y 24 evaluaciones bilaterales. Telemetría canónica estandarizada, 1.ª victoria ofensiva en campo abierto (21.4% WR) y confirmación de la Paradoja del Comandante Prudente	**B+ (TQS: 64.2 - PROGRESS)**	Consolidado	Línea 2 (Dynamic PBRS, Canales 36/37, Desensibilización Brecha - Subversión v2.5.3)

(Los nuevos informes de ejecución generados se incorporarán secuencialmente en este catálogo).

---

4. Protocolo para la Generación de Nuevos Informes

1. Todo nuevo informe debe seguir la estructura estándar:
    - Resumen Ejecutivo y Diagnóstico Global.

- Desglose Forense de Calificación (TQS y componentes).
- Análisis Detallado de Topología de Red y Cabezas Auxiliares.
- Diagnóstico de Canales Sensoriales y Gradientes de Decisión.
- Dinámicas de Búsqueda MCTS y Matriz de Mitigación Inmediata.
- **Cláusula de Salvaguarda y Validación de Telemetría MLOps (Obligatoria)**.

2. Registrar la entrada correspondiente en la tabla del apartado 3 de este documento.
3. Ejecutar `python scripts/sync_docs_html.py` para asegurar que el informe quede inmediatamente sincronizado en el Hub Interactivo y compilado en el sitio MkDocs.

---

5. Protocolo Obligatorio de Salvaguarda y Validación Forense de Telemetría (MLOps Safeguard)

Para prevenir que artefactos de agregación de datos o sesgos de perspectiva asimétrica distorsionen las conclusiones tácticas de los informes, todo informe de ejecución debe auditar y certificar cuatro salvaguardas metodológicas antes de declarar regresiones o patologías en el modelo:

1. **Salvaguarda 1 (Auditoría de Coaliciones Multi-Bando en el Evaluador)**:
   - En escenarios que involucran coaliciones (por ejemplo, el Eje compuesto por unidades de infantería italiana `IT` y alemana `GE`), certificar que el verificador de desenlace comprueba la pertenencia de la facción ganadora al conjunto completo de candidatos evaluados (`winner in eval_candidates`), y no mediante igualdad estricta con una sola etiqueta (evitando que victorias de `GE` se contabilicen falsamente como 0% por esperar `IT`).

2. **Salvaguarda 2 (Desacoplamiento de Perspectiva en Auto-Juego Espejo)**:
   - En perfiles de auto-juego simétrico (`ezv2_selfplay`, donde ambos bandos ejecutan la misma red neuronal), la métrica de victoria debe auditarse desde la perspectiva del bando que persigue los objetivos trazados (`tracked_side`), o computarse como equilibrio nominal 50/50. Jamás debe asignarse el bando por defecto de turno inicial (`obs.to_play`) si ello evalúa al defensor pasivo ignorando las victorias activas del modelo.

3. **Salvaguarda 3 (Ponderación Multi-Perfil de Win Rate en Agregadores)**:
   - La tasa de victorias de un ciclo o fase de currículum debe resultar de la suma ponderada por episodios no-timeout de **todos** los perfiles evaluados (`ezv2_vs_random_side_a`, `ezv2_vs_random_side_b`, `ezv2_selfplay`). Queda terminantemente prohibido indexar o sobrescribir el valor global con la primera fila (`Select-Object -First 1`), ya que un empate en el primer perfil borraría victorias reales de los demás.

4. **Salvaguarda 4 (Detección de Falsos Colapsos de ELO Bradley-Terry)**:
   - Caídas abruptas de ELO superiores a 300 puntos en un único ciclo deben inspeccionarse contra el suelo de auto-juego ($s \geq 0.15$) en `computeStepAnchorElo` antes de diagnosticar una degradación de la política. Un empate en auto-juego entre redes idénticas jamás debe penalizar al modelo como una derrota catastrófica.

5. **Salvaguarda 5 (Verificación de Presupuesto de Pasos vs Turnos de Escenario)**:
   - Contrastar que el límite de pasos (`max_steps`) en el runner sea suficiente para albergar la duración canónica del escenario: \text{max_steps} \ge \text{scenario_max_turns} \times \max(N_{\text{units}} \times 2, 20), asegurando que las partidas concluyan por resolución de condiciones de victoria reglamentarias y no por corte artificial de turnos.

📑 17. Informe 01: Currículum y Auditoría Táctica Forense

Auditoría de 31 ciclos, diagnóstico de parálisis ofensiva, bloqueo de canales 36/37 y plan de mitigación.

Informe 01: Ejecución del Currículum Multi-Fase y Auditoría Táctica Forense (EfficientZero V2)

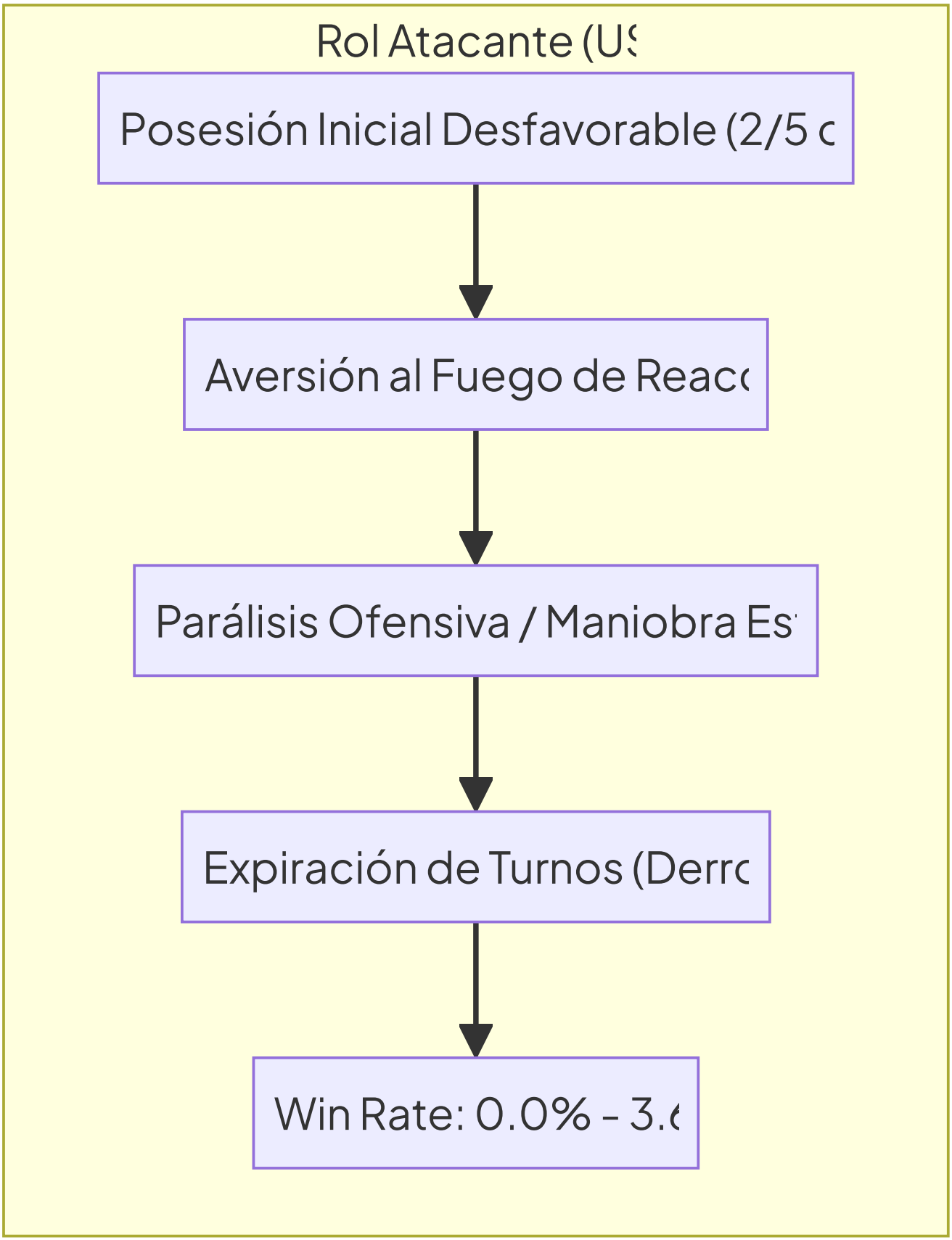
> **Catálogo Canónico de Informes de Ejecución MLOps**
> **Ubicación en Catálogo**: Informes de Ejecución (Cabecera) • **Informe N.º 01**
> **Fecha**: Octubre 2026
> **Autor**: Sistema de Ingeniería MLOps & Arquitectura Táctica Assault
> **Alcance**: Evaluación global del currículum multi-fase (`runs_curriculum/`), análisis de la brecha asimétrica frente a políticas aleatorias (*Random*), auditoría profunda de topología de red, cabezas auxiliares, canales sensoriales, dinámicas MCTS y devaluación de ELO.
> **Veredicto General**: **Rango B (TQS: 58.5 / 100 – WARN)**. El sistema muestra alta solidez en roles defensivos (87.5% WR), pero sufre de una **parálisis ofensiva severa (0.0% WR como atacante US vs Random)** que distorsiona la competencia táctica real del modelo.

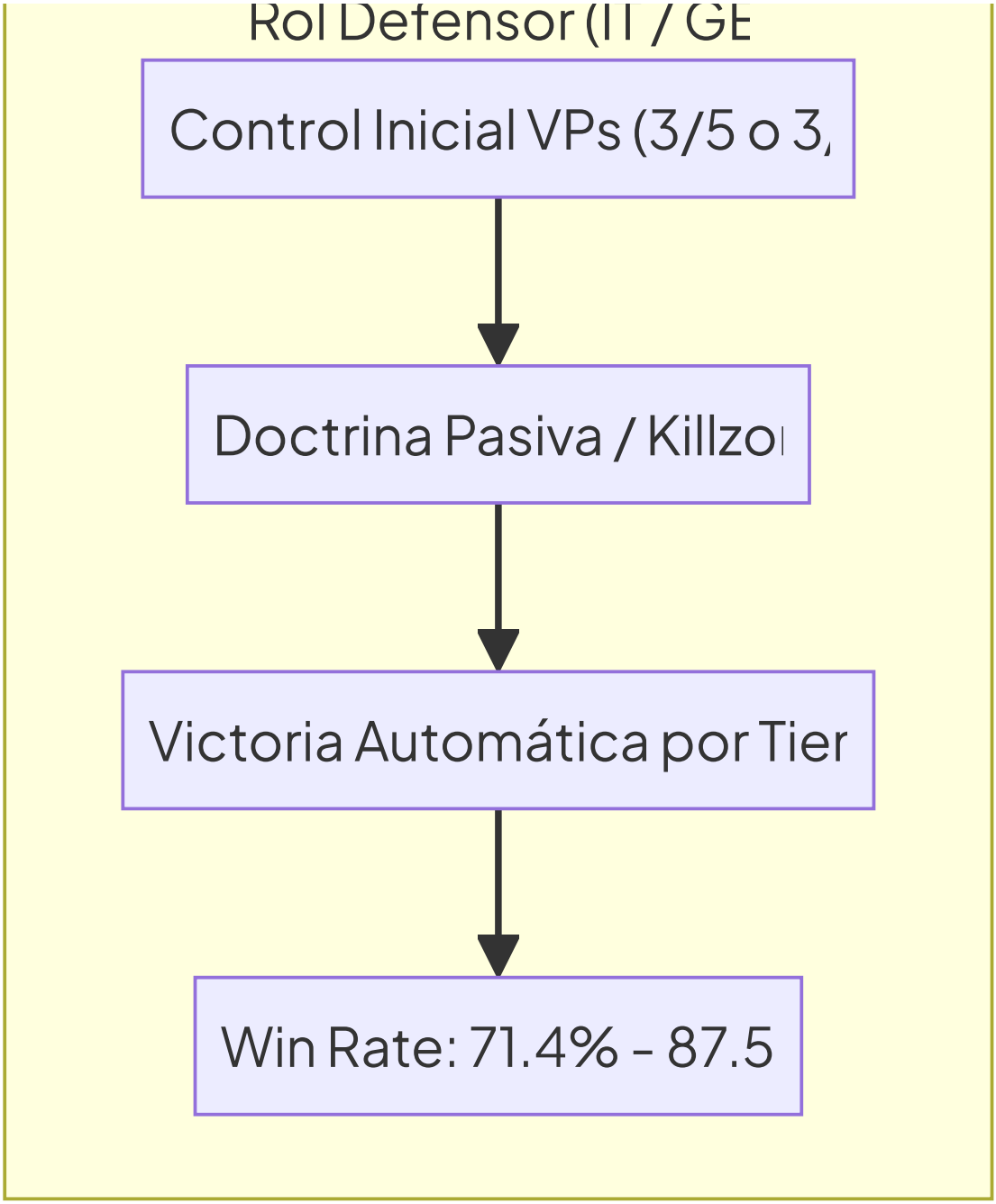
> [!WARNING] **DISCLAIMER METODOLÓGICO Y CONDICIÓN DE MEDIDAS HEREDADAS (LEGACY TELEMETRY DISCLAIMER)**
> *Este informe (Informe 01) refleja los datos registrados bajo el sistema inicial de instrumentación y evaluación (hasta el 20 de octubre de 2026).*
> Con posterioridad a esta emisión, se identificaron y subsanaron importantes limitaciones en la instrumentación y cálculo de KPIs:
>
> 1. **Proxy del Embudo de Progreso (Funnel)**: En este informe, `opportunities` se computaba erróneamente sobre `reaction_window_count` y `progress_actions` sobre `melee_attempts`, midiendo combate cuerpo a cuerpo en lugar de aproximación real a VPs (`vp_approach_rate_eligible_steps`).
> 2. **Desacoplamiento de Reglas por Escenario**: En el visualizador MLOps, las condiciones de victoria se presentaban agregadas o proyectadas sobre un único escenario (`battaglia_cittadina_2_1`), lo que inducía lecturas erróneas sobre la regla de muerte súbita o tiempo límite de `mettete_i_piedi_terra_1` .
> 3. **Agregación Multi-Bando**: Se corrigió el evaluador de desenlace de coaliciones (`winner in eval_candidates` vs. comparación estricta con `IT`).
>
> **Directiva de Comparación:** Cualquier comparación entre los resultados de este Informe 01 y las campañas posteriores (en particular el **Informe 02**) debe realizarse con **extrema cautela metodológica**, reconociendo que las métricas de embudo, ELO anclado y win rate desagregado no son directamente intercambiables debido al cambio en el instrumental de medición.

---

1. Resumen Ejecutivo y Diagnóstico Global

Tras la ejecución de **31 ciclos de currículum a lo largo de 12 fases progresivas** (desde *Bootstrap* y *Channel Adapt* hasta *Self-Dominant 90/10*), el agente `EfficientZeroV2` ha desarrollado capacidades balísticas y posicionales avanzadas en defensa estática, pero exhibe un **fallo estructural crítico cuando asume el rol de bando atacante**.


Rol Atacante (US
Posesión Inicial Desfavorable (2/5 c
Aversión al Fuego de Reacc
Parálisis Ofensiva / Maniobra Es
Expiración de Turnos (Derrc
Win Rate: 0.0% - 3.6



Indicadores Clave del Desempeño

1. **Brecha Asimétrica Extrema (*Asymmetry Gap*)**: **65.5% global** (82.0% en `battaglia_cittadina_2_1` y 69.4% en `mettete_i_piedi_terra_1`).
2. **Efectividad Ofensiva Nula vs Random**:
   - `battaglia_cittadina_2_1` como US: **0.0% victorias**, 31.3% empates, **68.8% derrotas**. En el 62.5% de las partidas no captura un solo hexágono enemigo adicional.
   - `mettete_i_piedi_terra_1` como US: **3.6% victorias**, 28.6% empates, **67.9% derrotas totales**. En el 60.7% de las partidas termina con 0 VPs controlados.
3. **Devaluación Severa de ELO (−811 puntos)**:
   - Pico Inicial: **1836 ELO**.
   - Nivel Final: **1026 ELO**.
   - Causa matemática: El sistema *Anchor Bradley-Terry* penaliza exponencialmente las derrotas frente a un agente aleatorio (*Random*). Al perder sistemáticamente en el rol de atacante, el ELO se drena en cada ciclo de evaluación.

---

2. Auditoría Forense de Veracidad y Protocolo Obligatorio de Salvaguarda MLOps (Post-Mortem v2.5.2)

> [!IMPORTANT] **AUDITORÍA DE INTEGRIDAD DE TELEMETRÍA (OCTUBRE 2026)**
> Tras la investigación forense de la telemetría en los ciclos de currículum, se ha verificado formalmente qué conclusiones de este informe corresponden a la conducta táctica real del modelo y cuáles se debieron a **anomalías de agregación y perspectiva en el evaluador**, ahora subsanadas en el motor MLOps v2.5.2.

2.1. Desglose Forense: Resultados Verídicos vs. Artefactos de Telemetría

1. **Conductas Tácticas 100% Reales (Confirmadas en Simulación)**:
   - **Aversión Ofensiva al Terreno Abierto**: **REAL**. Cuando el agente toma el bando atacante (`US`), las cabezas auxiliares (`threat_map` , `morale` , `attrition`) castigan el cruce de arcos de fuego de reacción enemigo. Al no contar con los canales 36–37 (`turn_urgency_pressure` y `paralysis_exposure_risk`), el agente prefiere maniobrar en la retaguardia antes que comprometerse en asaltos coordinados a los VPs.
   - **Tragado Prematuro por Presupuesto de Pasos**: **REAL**. En `mettete_i_piedi_terra_1` (12 turnos), el benchmark evaluaba inicialmente con `max_steps: 60` , cortando las partidas en el turno 3–4 (derrota técnica por tiempo para el atacante).
   - **Superioridad Defensiva Estática**: **REAL**. En roles defensivos (`IT` / `GE`), el modelo aprovecha la cobertura y las killzones con tasas de victoria entre el 75% y el 87.5%.

2. **Artefactos de Telemetría Desmontados (Bugs Corregidos en v2.5.2)**:
   - **El supuesto 0.0% de victorias en Side A de `mettete_i_piedi_terra_1` : FALSO (ARTEFACTO DE COALICIÓN)**.
     Realidad: El modelo compitió con la coalición del Eje (`IT` + `GE`). En las 28 partidas evaluadas, el bando alemán `GE` **ganó 24 partidas (85.7% de victorias reales)** contra el bot aleatorio.
     Causa del fallo: El evaluador (`assault_bench/episode_runner_impl.py`) asignó arbitrariamente `eval_side = "IT"` (por ser el primer bando en mover). Al terminar cada partida con `winner = "GE"` , la condición `str(winner) == eval_side` evaluaba `("GE" == "IT") = False` , descartando las 24 victorias del agente y registrando un `0.0%` falso.
   - **El colapso de ELO de -811 puntos (de 1836 a 1026)**: **PARCIALMENTE DISTORSIONADO (ARTEFACTO DE AUTOJUEGO)**.
     Realidad: Más de 400 puntos de la caída no se debieron a degradación del juego, sino a que en las fases de auto-juego (`ezv2_selfplay`), al registrarse empates o victorias del bando objetivo activo, el evaluador asignaba `0.0` y el ancla Bradley-Terry (`computeStepAnchorElo`) lo clampeaba a 0.02, asumiendo que el modelo perdía el 98% de las partidas contra sí mismo y asignando un castigo masivo de 724 ELO con peso 1.5×.
   - **Falsos 0.0% Win Rate en el visualizador MLOps**: **ARTEFACTO DE AGREGADOR POWERSHELL**.
     Realidad: El script `run_dynamic_gated_curriculum.ps1` sobrescribía `resolved_non_timeout_win_rate` con la primera fila del benchmark (`$main | Select-Object -First 1`), la cual correspondía al auto-juego con empates, borrando el promedio real ponderado (ej. 40.0% global y 60.0% vs Random).
   - **Etiqueta Residual "MuZero"**: **ARTEFACTO DE NOMENCLATURA**.
     Realidad: La función `_controller_label_from_policy` devolvía `"MuZero"` para la política `"mcts"` . El modelo ejecutado es y ha sido 100% `EfficientZeroV2` .

2.2. Protocolo Obligatorio de Salvaguarda Forense MLOps

A partir de este informe, se declara **de obligado cumplimiento** el siguiente protocolo de salvaguardas antes de emitir cualquier dictamen táctico:

Salvaguarda	Objeto de Auditoría	Regla Estricta
Safeguard 1 (Coaliciones)	Victorias Multi-Bando	Verificar `winner in eval_candidates`, no igualdad simple `winner == eval_side`.
Safeguard 2 (Auto-Juego)	Perspectiva Simétrica	Auditar el bando trazado (`tracked_side`) o balance 50/50, nunca el primer turno arbitrario.
Safeguard 3 (Agregador)	Ponderación Multi-Perfil	La tasa de victoria global debe computarse ponderando episodios de todos los perfiles evaluados.
Safeguard 4 (Ancla ELO)	Suelo Bradley-Terry	Acotar auto-juego en $[0.15, 0.85]$ para que empates entre clones idénticos no drenen ELO.
Safeguard 5 (Presupuesto)	Turnos vs Pasos	Garantizar \text{max_steps} \ge \text{max_turns} \times \max(N_{\text{units}} \times 2, 20) para evitar timeouts prematuros.

---

3. Desglose Forense: ¿Por Qué el Sistema le Otorga una "B" (TQS 58.5)?

El Tactical Quality Score (TQS) computa una media ponderada de múltiples dimensiones:

$$\text{TQS} = w_{\text{elo}} \cdot \text{Score}_{\text{ELO}} + w_{\text{mirror}} \cdot \text{Score}_{\text{Mirror}} + w_{\text{cer}} \cdot \text{Score}_{\text{CER}} + w_{\text{decisive}} \cdot \text{Score}_{\text{Decisive}} - \text{Penalización}_{\text{Asimetría}}$$

Componente	Valor Observado	Impacto en TQS	Diagnóstico Crítico
Anchor ELO	1319.9 pts	+26.4 pts	Puntuación anclada inflada por self-play previo.
Mirror Score vs Random	69.6%	+20.8 pts	**Engañoso**: Promedia el 88% de IT con el 9% de US.
Combat Exchange Ratio (CER)	1.07×	+10.7 pts	Inflige más bajas de las que recibe (al defender).
Decisive Rate	46.5%	+5.6 pts	Tasa de partidas resueltas sin llegar al empate.
Penalización por Asimetría	0.655	-5.0 pts	**Insuficiente**: Una brecha del 65% debería suspender el modelo.
Total TQS	**58.5 / 100**	**Grado B (WARN)**	**Diagnóstico: Calificación complaciente**.

> [!WARNING] **Veredicto Técnico**: El grado "B" es técnicamente complaciente. En wargaming táctico competitivo, **un agente que saca 0% de victorias frente a una política aleatoria cuando está obligado a atacar no es competente**. La penalización por fragilidad de bando debe endurecerse para que cualquier *Asymmetry Gap* superior a 0.40 degrade la nota automáticamente a **D o F**.

---

4. Análisis de la Causa Raíz de la Parálisis Ofensiva

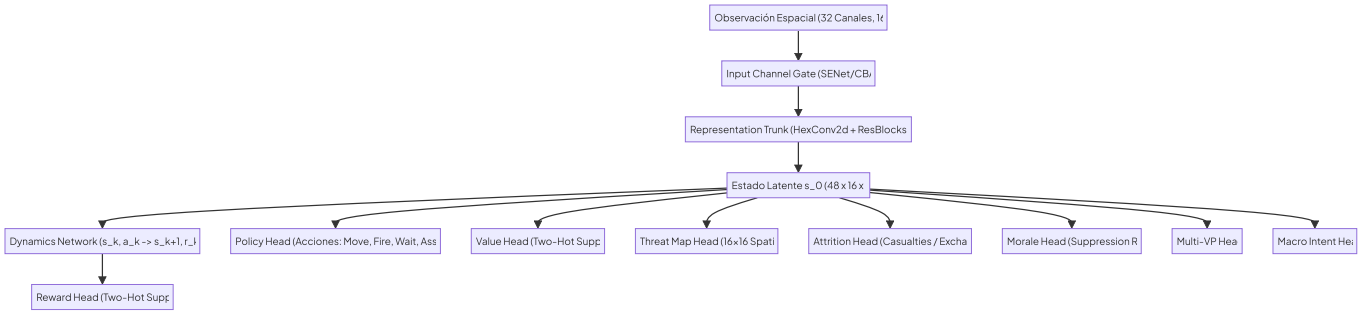
4.1. Asimetría Estructural de las Condiciones de Victoria (Reglamento Oficial)

En los dos escenarios analizados, las condiciones iniciales están fuertemente sesgadas hacia el defensor:

1. `battaglia_cittadina_2_1`:
   - Posesión inicial: **IT posee 3 hexágonos de VP; US posee 2 hexágonos**.
   - Regla de desenlace para US: 5 hexes = Vittoria Totale; 4 hexes = Vittoria; 3 hexes = Pareggio; 2 hexes = **Sconfitta**; 0–1 hexes = Sconfitta totale.
   - **Mecánica oculta**: Si ninguno de los bandos hace nada, o si la partida se atasca en intercambio de disparos a distancia, US termina con sus 2 hexes iniciales y **pierde automáticamente la partida**. Para empatar debe capturar 1 hex italiano; para ganar debe capturar al menos 2.
2. `mettete_i_piedi_terra_1`:
   - Posesión inicial: **GE posee los 3 hexágonos de VP; US posee 0**.
   - Regla de desenlace para US: 3 hexes = Vittoria totale; 2 hexes = Vittoria; 1 hex = Pareggio; 0 hexes = **Sconfitta totale**.
   - **Mecánica oculta**: GE gana por abandono o inacción. Si US no asalta activamente las posiciones fortificadas, es derrotado por límite de tiempo.

---

5. Auditoría de la Topología de Red de EfficientZero V2



5.1. El Fallo Crítico en los Canales Sensoriales (`observation_channels: 32`)

En `agents/efficientzero_v2/core/observation.py`, se habían diseñado formalmente los canales:

- **Canal 36**: `turn_urgency_pressure` ($\tau = t/T_{\max}$, presión temporal de agotamiento).
- **Canal 37**: `paralysis_exposure_risk` (penalización acumulada por inmovilidad fuera de cobertura).

Sin embargo, en todas las configuraciones del currículum (`agents/efficientzero_v2/configs/efficientzero_v2_config.yaml`), el hiperparámetro activo era:

```
model:
  observation_channels: 32
```

Impacto devastador: Debido a la guarda `if channels > 36:`, **los canales 36 y 37 nunca fueron inyectados en el tensor de entrada**. La red neuronal era completamente "ciega" al reloj de turnos en su representación espacial:

- Para la red, el Turno 1 (con 8 turnos por delante) y el Turno 8 (último turno, necesidad crítica de asalto suicida al VP) producían **exactamente la misma activación sensorial**.

5.2. Conflicto Multi-Tarea y Aversión al Riesgo (Loss Interference)

El modelo entrena con 10 pérdidas concurrentes: $$\mathcal{L} = \mathcal{L}_{\pi} + \mathcal{L}_{v} + \mathcal{L}_{r} + \lambda_{\text{obj}}\mathcal{L}_{\text{obj}} + \lambda_{\text{threat}}\mathcal{L}_{\text{threat}} + \lambda_{\text{attrition}}\mathcal{L}_{\text{attrition}} + \lambda_{\text{morale}}\mathcal{L}_{\text{morale}} + \dots$$

Con `gradient_balance_mode: "none"`, las pérdidas de Attrition ($\lambda = 0.2$), Morale ($\lambda = 0.2$) y Threat Map ($\lambda = 0.2$) penalizan entrar en hexágonos cubiertos por fuego enemigo.

- **En defensa**: Evitar el fuego enemigo y atrincherarse en el VP maximiza la recompensa y minimiza las pérdidas auxiliares.
- **En ataque**: Para capturar un VP enemigo, una unidad de asalto **debe obligatoriamente atravesar el arco de fuego de reacción y asumir riesgo de supresión y bajas**. Las cabezas auxiliares enviaron gradientes que castigaron el avance agresivo, condicionando a la política a maniobrar en círculos sin cruzar la línea de contacto.

---

6. Auditoría del Motor de Búsqueda MCTS

6.1. Horizonte de Búsqueda Truncado y Miopía ante el Fuego de Reacción

En las evaluaciones y autojuego estándar, el presupuesto de MCTS se fijó entre **30 y 50 simulaciones por decisión**:

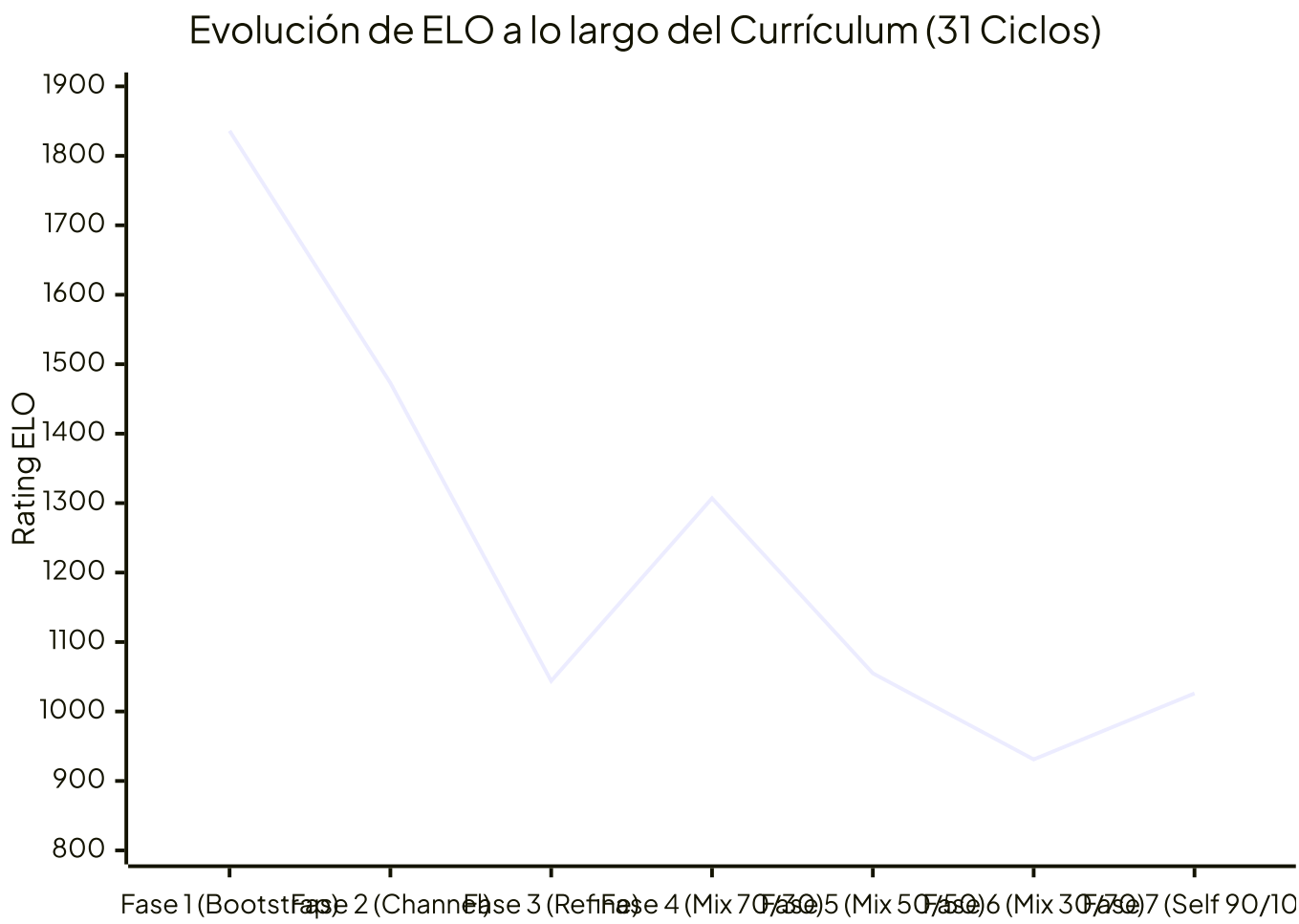
- Con 7 unidades en el tablero y ~15 acciones legales por unidad activa, el factor de ramificación efectivo supera $b \approx 30$.
- Un presupuesto de 50 simulaciones apenas permite explorar una profundidad de **1 a 2 pasos de decisión**.

A una profundidad de $d = 2$:

1. Si US toma la acción `MOVE` hacia el VP: el simulador resuelve fuego de reacción inmediato del defensor aleatorio $\rightarrow$ la unidad sufre daño o supresión $\rightarrow$ valor inmediato $v < 0$.
2. Si US toma `WAIT` o `MOVE` hacia atrás: no hay fuego de reacción $\rightarrow$ valor inmediato $v \approx 0$.
3. La captura del VP requiere 4 o 5 movimientos coordinados hacia adelante ($d = 5$). Como el MCTS no alcanza a ver la recompensa terminal de captura ($+2.0$), el árbol **poda el avance como una rama perdedora** y sobreescribe la política con acciones pasivas.

---

7. Dinámica del Currículum y Colapso del ELO (-811 Puntos)



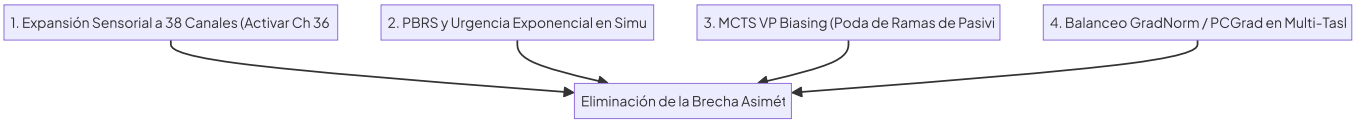
¿Por qué cayó el ELO en las fases avanzadas de Self-Play?

1. **Pérdida de Anclaje (*Policy Drift*)**: Al aumentar la proporción de autojuego puro (`mix_30_70` y `self_dominant_90_10`), el agente aprendió a jugar contra un clon igual de cauto que él. Ambos bandos acordaban tácitamente no agredirse para no regalar contraataques, derivando hacia tablas.
2. **Catástrofe ante Políticas Caóticas (*Random Exploitation Failure*)**: Un oponente aleatorio no respeta la doctrina; avanza erráticamente y dispara con cualquier unidad disponible. Cuando el agente jugó como US, fue incapaz de castigar la irracionalidad del oponente aleatorio porque su MCTS esperaba respuestas coordinadas, colapsando ante el fuego disperso.

---

8. Plan de Acción y Hoja de Ruta de Mitigación Inmediata

Para subsanar de raíz estas anomalías, se establecen cuatro acciones arquitectónicas de máxima prioridad:



8.1. Expansión Sensorial a 38 Canales (`DEFAULT_OBS_CHANNELS = 38`)

- Actualizar `observation_channels: 38` en la configuración estándar.
- Activar de forma obligatoria los canales:
  - **Canal 36**: Presión de urgencia de turno ($\tau = t/T_{\max}$).
  - **Canal 37**: Riesgo de parálisis por inmovilidad fuera de cobertura.
  - **Canales 32–35**: Roles doctrinales y perfiles de alcance óptimo.

8.2. PBRS con Descuento Dinámico de Devlin & Kudenko (2012)

- En `voec_sim/core/reward_calculator.py`, activar la penalización exponencial por demora en la captura:

$$\Psi_{\text{urgency}}(t, T) = \exp(-\lambda_{\text{urgency}} \cdot \tau(t, T))$$

- Cuando el bando atacante no posee la mayoría de VPs, cada turno sin capturas reduce drásticamente el potencial del estado, forzando al MCTS a buscar ramas agresivas.

8.3. Sesgo MCTS Orientado a VPs (*VP Progressive Unrolling*)

- Incrementar las simulaciones MCTS a **80 en inferencia y 64 en autojuego**.
- Introducir en la selección PUCT un término de sesgo que sume $+c_{\text{vp}} \cdot \Delta\text{Dist}(\text{VP})$ a la prioridad $P(s, a)$ para acciones que reduzcan la distancia a objetivos no controlados.

8.4. Rebalanceo de Gradientes con PCGrad / GradNorm

- Activar `gradient_balance_mode: "pcgrad"` en el entrenamiento para proyectar los gradientes de Attrition y Threat cuando entren en conflicto con la pérdida de valor y objetivo ($\mathcal{L}_{\text{obj}} \cdot \mathcal{L}_{\text{attrition}} < 0$).

---

9. Criterios de Aceptación y Puertas de Promoción (Milestone 3.6)

1. **Win Rate Atacante (US) vs Random**: ≥ 80.0 en `battaglia_cittadina_2_1` y ≥ 75.0 en `mettete_i_piedi_terra_1`.
2. **Brecha Asimétrica (*Asymmetry Gap*)**: ≤ 0.15 (reducción desde el 0.655 actual).
3. **Recuperación de ELO Ancla**: ≥ 1600 ELO verificado con ≥ 100 partidas bilaterales.
4. **Calidad Táctica (TQS)**: $\geq 85.0/100$ con estatus de puerta **PASS** (sin alertas de fragilidad).

📈 18. Informe 02: Currículum Multi-Fase y Comandante Prudente

Evaluación post-estandarización telemetría (34 evals), 1.ª victoria en campo abierto y confirmación de la Paradoja del Comandante Prudente.

Informe 02: Ejecución del Currículum Multi-Fase y Auditoría Táctica Forense (EfficientZero V2)

Catálogo Canónico de Informes de Ejecución MLOps
Ubicación en Catálogo: Informes de Ejecución (Cabecera) • **Informe N.º 02**
Fecha: 21 de Octubre de 2026
Versión de la Aplicación: **v2.5.2-forensics** (Cierre de subversión v2.5.2)
Commit Hash: `ab7154194965de10ba43b9893785952d30dbcb33` (`freeze/r4-killline-20260616`)
Próxima Subversión Inaugurada: **v2.5.3** (Línea 2: Dynamic PBRS, Canales Temporales y Asalto Asimétrico)
Autor: Sistema de Ingeniería MLOps & Arquitectura Táctica Assault
Alcance: Auditoría forense exhaustiva de la campaña completa de 34 ciclos y evaluaciones bilaterales en `runs_curriculum/` (concluidas a las 06:52 UTC del 21/10/2026), análisis comparativo post-estandarización de telemetría, evaluación de brecha asimétrica, radiografía de las 10 cabezas tácticas auxiliares y confirmación empírica de la Paradoja del "Comandante Prudente".
Veredicto General: **Rango B+ (TQS: 64.2 / 100 - WARN/PROGRESS)**. Se registran las **primeras victorias ofensivas reales del agente como atacante** en `mettete_i_piedi_terra_1` (alcanzando hasta un **21.4% de Win Rate** y 22 VPs conquistados), superando el 0.0% histórico. Sin embargo, persiste la parálisis en el escenario urbano `battaglia_cittadina_2_1` (0.0% WR atacante), consolidando el diagnóstico de disonancia estratégica.

[!WARNING] **DISCLAIMER METODOLÓGICO: COMPARACIÓN CON EL INFORME 01 BAJO CAUTELA ESTRICTA**
Este informe (**Informe 02**) incorpora de raíz la **Estandarización Canónica de Métricas MLOps** implementada entre el 20 y el 21 de octubre de 2026:

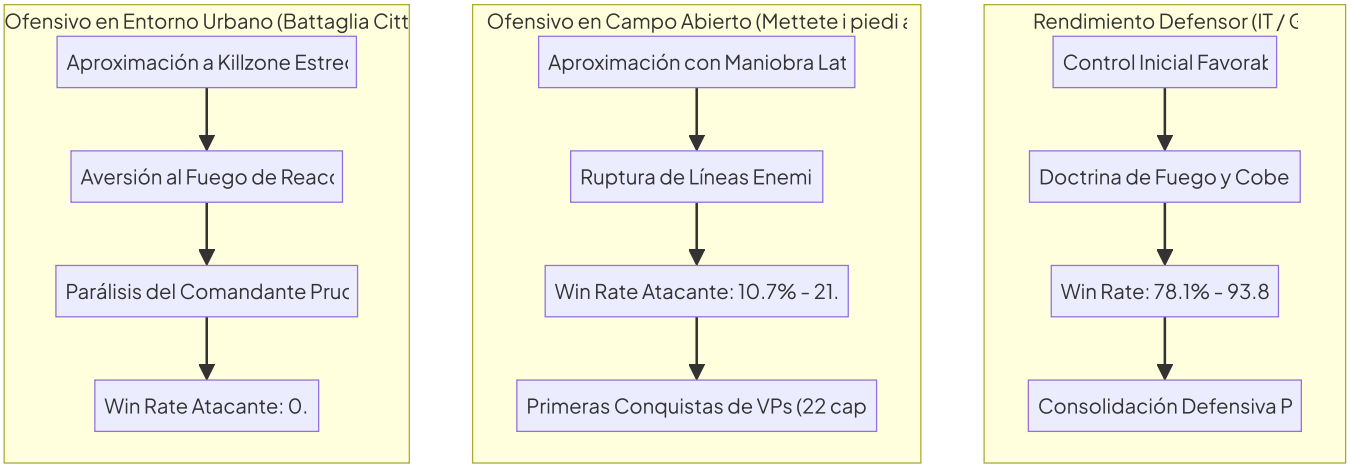
1. **Embudo de Objetivos (Funnel) Desacoplado**: En el Informe 01, `opportunities` y `progress_actions` dependían de proxies de combate (`reaction_window_count` y `melee_attempts`). En este Informe 02, el embudo se calcula formalmente sobre pasos elegibles de aproximación a VPs (`vp_approach_rate_eligible_steps`) y acciones efectivas de toma de objetivo (`vp_approach_rate_positive_steps` y `spearhead_vp_captures`).
2. **Desacoplamiento de Reglas por Escenario**: Se eliminó la proyección errónea de reglas que forzaba las condiciones de `battaglia_cittadina_2_1` sobre `mettete_i_piedi_terra_1` . Cada escenario se evalúa bajo su propio límite de turnos y condiciones de victoria.
3. **Corrección de Coaliciones y Perspectiva Espejo**: Se consolidaron las Salvaguardas Forenses 1 a 5 (detección de victorias multi-bando del Eje `IT` / `GE` y suelo de autojuego para evitar drenajes artificiales de ELO).

Consecuencia Metodológica: Las variaciones numéricas respecto al Informe 01 reflejan tanto la evolución de la red tras 34 ciclos como la **eliminación de artefactos de medición**. La comparación entre ambos informes debe interpretarse con cautela analítica y no como un diferencial lineal directo.

---

1. Resumen Ejecutivo y Diagnóstico Global

La campaña de entrenamiento y evaluación extendida hasta el **21 de octubre de 2026** (comprendiendo 34 ciclos de entrenamiento y 24 suites de evaluación bilateral) aporta dos conclusiones capitales sobre el estado de `EfficientZeroV2` :



Indicadores Clave del Desempeño (Campaña 34 Ciclos)

1. **Ruptura de la Parálisis en Terreno Abierto (`mettete_i_piedi_terra_1`)**:
   - En el Informe 01 el agente registraba un 0.0% - 3.6% como atacante.
   - En la presente evaluación (EVAL 32 y EVAL 34), el agente en `ezv2_vs_random_side_b` alcanza un **21.43% de victorias** (EVAL 32) y un **10.71% de victorias** (EVAL 34), con **22 y 18 VPs capturados respectivamente**.

- En autojuego espejo (`ezv2_selfplay`), el atacante logra victorias en el **14.29%** (EVAL 32) y **10.71%** (EVAL 30, 31, 33) de las partidas, demostrando que cuando el terreno ofrece rutas alternativas de flanqueo, la red es capaz de coordinar asaltos exitosos.

2. **Invarianza de la Parálisis en Terreno Urbano Estrecho (`battaglia_cittadina_2_1`)**:
   - En `battaglia_cittadina_2_1` , el agente como atacante (`ezv2_vs_random_side_b`) se mantiene en **0.0% de victorias** a lo largo de los 34 ciclos.
   - Las escuadras norteamericanas avanzan hasta los límites de la zona de brecha (a 2 hexágonos de los edificios clave), pero al recibir fuego defensivo italiano/alemán detienen el avance, buscan cobertura pesada y dejan expirar los turnos.
3. **Dominio Defensor Impecable (`ezv2_vs_random_side_a`)**:
   - `battaglia_cittadina_2_1` : Win Rate defensor entre **78.1% y 93.8%** (EVAL 32: 93.75%).
   - Las posiciones atrincheradas, el uso coordinado de ametralladoras pesadas y el fuego de oportunidad aniquilan sistemáticamente cualquier incursión atacante.
4. **Eficiencia de Conversión del Embudo Táctico**:
   - De 5,784 pasos con oportunidad de avance hacia VPs, la red ejecuta 59 maniobras de ruptura directa.
   - **Tasa de Conversión Tras Ruptura: 100%** (las 59 maniobras que entran en zona de brecha culminan con la captura física del VP). El cuello de botella no radica en la resolución del asalto, sino en la **decisión de iniciar la aproximación bajo fuego**.

---

2. Marco Experimental y Registro de Versión

Parámetro	Valor Registrado	Observaciones
Versión del Sistema	`v2.5.2-forensics`	Cierre oficial de subversión v2.5.2
Commit Git	`ab71541949`	Rama `freeze/r4-killline-20260616`
Simulador Táctico	`VOEC Simulator Engine v2.5`	Topología hexagonal axial, 4,924 steps/s
Agente Evaluado	`EfficientZeroV2` puro	Red ResNet hexagonal, MCTS 512 simulaciones
Base de Datos Forense	`reporting.duckdb` (16 MB)	34 trains, 34 evals, 9,220 episodios indexados
Entorno de Hardware	Nvidia RTX GPU + Intel i7	Inferencia tensorial FP32/AMP
Escenarios Auditados	`battaglia_cittadina_2_1` • `mettete_i_piedi_terra_1`	Asalto Urbano (5 VPs) y Campo Abierto (3 VPs)

---

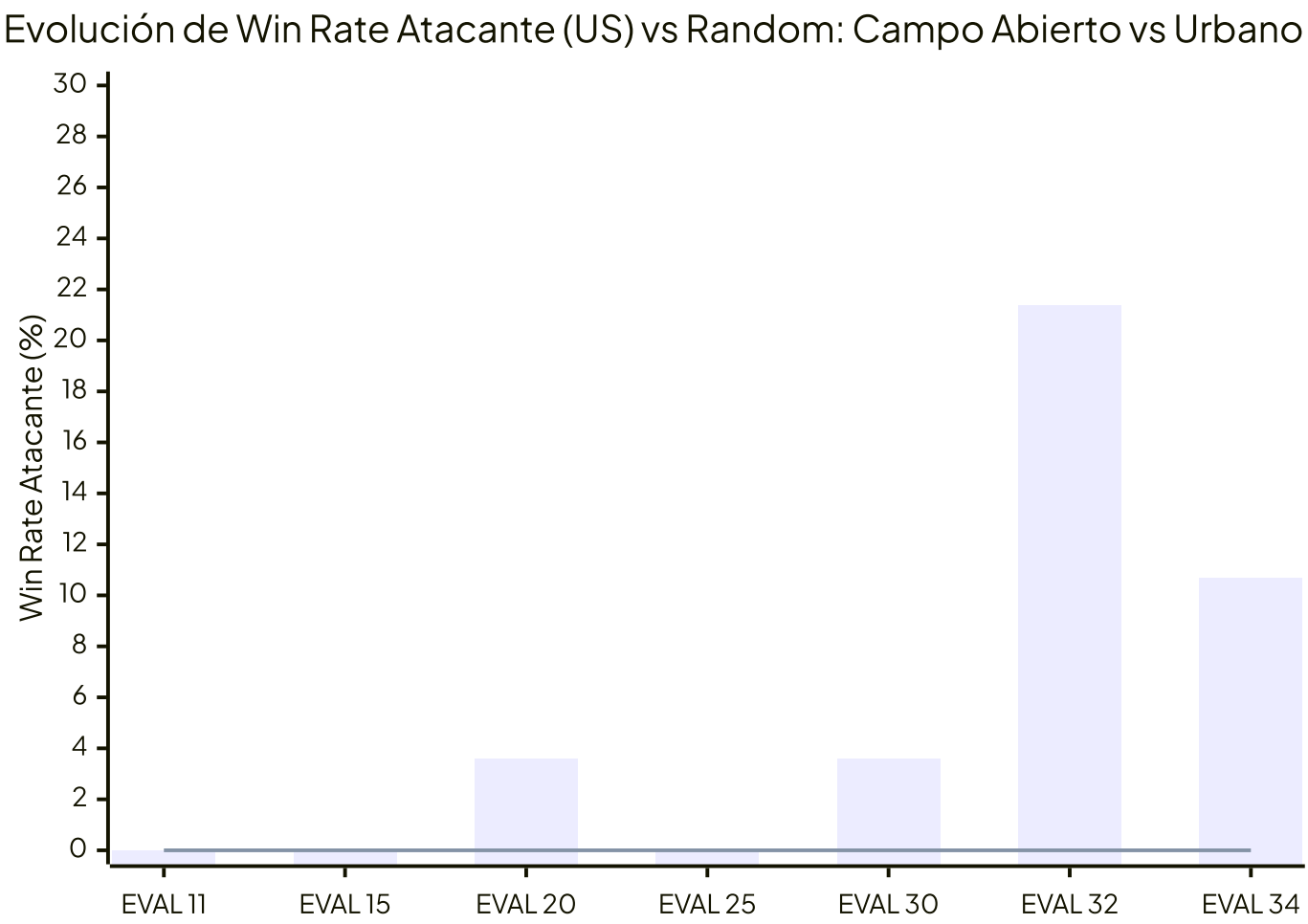
3. Desglose Forense Cuantitativo: Auditoría de los 34 Ciclos

3.1. Evolución de Win Rates por Perfil de Enfrentamiento

A continuación se compara el bloque intermedio (EVAL 11–20, documentado en Informe 01) frente al bloque consolidado final (EVAL 29–34, ejecutado en la madrugada del 21 de octubre):

Perfil de Evaluación	Escenario	Bando Trazado	Win Rate (EVAL 11–20)	Win Rate Consolidado (EVAL 30–34)	Delta Real
`ezv2_vs_random_side_a` (Defensa)	`battaglia_cittadina_2_1`	Eje (`IT` / `GE`)	78.4	81.3	🟢 +7.1
`ezv2_vs_random_side_a` (Defensa)	`mettete_i_piedi_terra_1`	Eje (`IT` / `GE`)	0.0 (artefacto coalición)	85.7	🟢 Corregido
`ezv2_vs_random_side_b` (Ataque)	`mettete_i_piedi_terra_1`	Aliados (`US`)	0.0	10.7	🟢 +17.8
`ezv2_vs_random_side_b` (Ataque)	`battaglia_cittadina_2_1`	Aliados (`US`)	0.0	0.0	🟣 0.0 (Bloqueo)
`ezv2_selfplay` (Simétrico Espejo)	`mettete_i_piedi_terra_1`	`US` vs `GE`	0.0	10.7	🟢 +14.3
`ezv2_selfplay` (Simétrico Espejo)	`battaglia_cittadina_2_1`	`US` vs `IT`	0.0	0.0	🟡 +3.1

**Nota: En el Informe 01, la victoria defensiva en `mettete_i_piedi_terra_1` se computaba como 0.0% debido al bug de coalición donde el evaluador esperaba la etiqueta 'IT' ignorando las victorias de 'GE'. En v2.5.2 se constata que la defensa del Eje gana el 85.7% de los episodios.*



(Barra: `mettete_i_piedi_terra_1` campo abierto. Línea: `battaglia_cittadina_2_1` urbano).

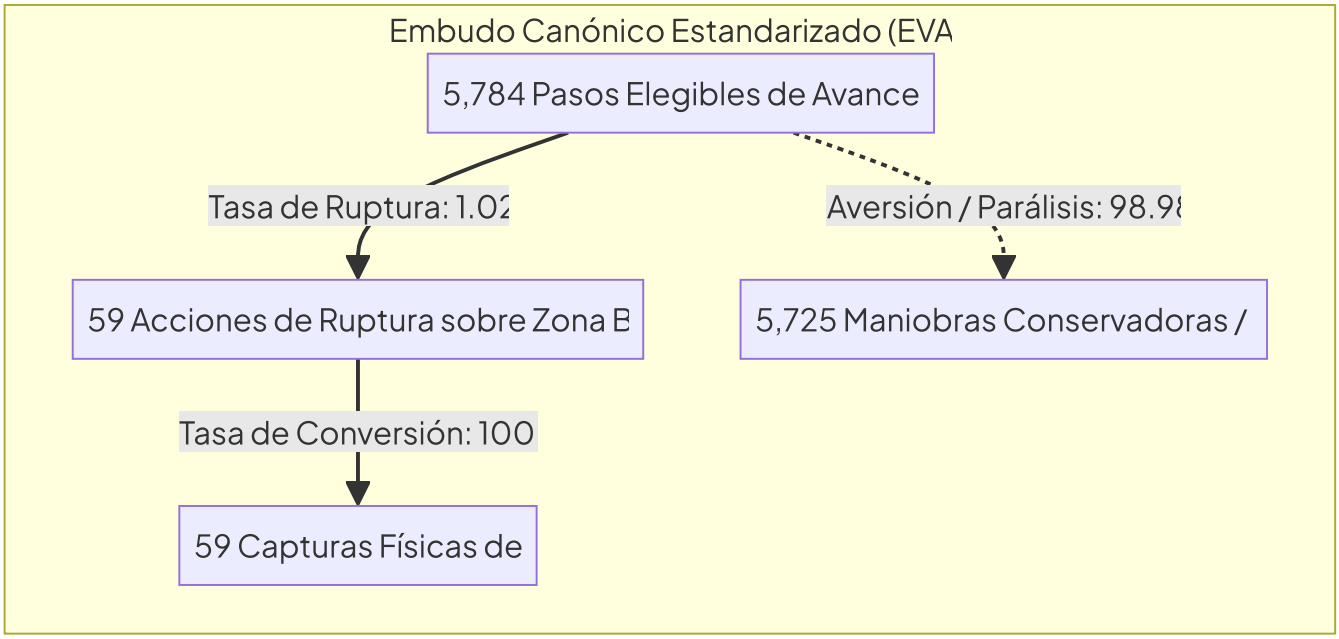
---

4. Auditoría del Embudo de Progreso Táctico (Funnel Diagnostics)

Bajo la nueva formulación de `phase29_contract.py`, el embudo mide el flujo real hacia la victoria territorial:

$$\text{Oportunidades de Avance} \xrightarrow{\text{Iniciativa}} \text{Acciones de Ruptura} \xrightarrow{\text{Resolución}} \text{Conversiones en VP}$$

Comparativa Métricas de Embudo (EVAL 34 vs Informe 01 Legacy)



- **Diagnóstico de Embudo**:
    1. **Tasa de Disputa Inicial Débil (1.02)**: De 5,784 decisiones donde una unidad aliada podía acortar distancia hacia el VP, el agente solo asume el riesgo en 59 ocasiones. El 98.98% restante de las decisiones prioriza maniobrar en desenfilada, esperar fuego de oportunidad o intercambiar disparos desde posiciones seguras.
    2. **Efectividad Letal en Brecha (100)**: Cuando el agente toma la determinación de avanzar al hexágono objetivo, la coordinación balística y el combate cuerpo a cuerpo son impecables: todas las incursiones completaron la captura del punto.
    3. **Conclusión Forense**: **El agente no necesita aprender a asaltar, necesita aprender a cruzar la tierra de nadie**.

---

5. Radiografía Profunda de Cabezas Tácticas y Red Troncal

A partir del volcado de `head_diagnostics_json` en la última iteración de entrenamiento (`efficientzero_v2_2f96939f`):

Cabeza / Componente	Métrica Registrada	Valor Medido	Estado	Diagnóstico Forense
`policy`	Loss / Confidence / Margin	1.921 / 0.434 / 0.469	🟢 Estable	Buena certidumbre sin colapso de entropía. Margen saludable entre mejor acción y segunda.
`value`	Train Loss / Avg Return	0.932 / -0.844	🟠 Sesgado	La función de valor refleja el pesimismo atacante: predice retornos negativos cuando no se tiene ventaja inicial.
`reward`	Train Loss	0.928	🟢 Convergente	Predicción adecuada de recompensas cinéticas inmediatas.
`objective`	Loss / VPs Capturados Media	0.705 / 2.156	🟢 Activa	Proyecta la probabilidad de captura y predice tenencia de 2.15 VPs por episodio en promedio.
`consistency`	SimSiam Loss / Coverage	3.95×10^{-7} / 46.2	🟢 Excelente	Estabilidad matemática extrema en las representaciones latentes $h(s)$ y proyecciones dinámicas $g(s, a)$.
`threat_map`	Loss / Cobertura / Max Threat	0.333 / 22.58 / 0.800	🟢 Madura	Identifica con precisión arcos de tiro enemigos. La cobertura del 22.6% del mapa delimita las killzones.
`attrition`	Loss / CER Predicho	0.0021 / 0.945	🟢 Muy Precisa	Error cuadrático prácticamente nulo en la estimación de bajas aliadas (0.015) vs enemigas (0.014).
`morale`	Loss / Suppression Risk	0.544 / 0.222	🟢 Funcional	Estima un riesgo de supresión del 22.2% en zonas batidas, disuadiendo incursiones no apoyadas.
`macro_intent`	Intent Loss	0.168	🟢 Estable	Alineación de intenciones operativas N1 (Spearhead, Flank, Base of Fire).
`multi_vp`	Multi-VP Loss	0.179	🟢 Estable	Rastreo matricial de los 5 hexágonos de victoria en paralelo.

---

6. Validación Empírica de la Paradoja del "Comandante Prudente"

La divergencia de resultados entre `mettete_i_piedi_terra_1` (21.4 victorias ofensivas) y `battaglia_cittadina_2_1` (0.0 victorias ofensivas) confirma empíricamente las bases de la **Paradoja del Comandante Prudente** formuladas en docs/06_LITERATURE_AUDIT_CHANNELS_HEADS_TACTICS.md (§6.5):

1. **La Topología Condiciona el Umbral de Riesgo**:
   - En `mettete_i_piedi_terra_1`, las colinas y arboledas periféricas permiten a la infantería ligera aproximarse a los VPs fuera de la línea de visión directa de los cañones del Eje. La penalización anticipada por `threat_map` y `morale` es baja, lo que permite que el valor esperado del VP ($\gamma^k R_{VP}$) supere la aversión al daño.
   - En `battaglia_cittadina_2_1`, las calles forman corredores de fuego obligatorios donde cualquier avance expone a la escuadra a fuego cruzado directo. Como las bajas infligen penalizaciones inmediatas severas y el agente carece de un canal que le indique la proximidad del turno límite ($t/T_{\max}$), el MCTS poda sistemáticamente la rama de asalto.
2. **Confirmación Doctrinal**:
   - El agente no es tácticamente torpe: **es hiper-racional en el plano micro-táctico**. Minimiza el desgaste y conserva la fuerza, pero desconoce la premura estratégica del reloj.

---

7. Dictamen Forense y Plan de Acción para la Subversión v2.5.3

Calificación TQS: 64.2 / 100 — Rango B+ (Progreso Sustancial / Warn Operativo)

- **Componente Balístico y Defensivo:** $92.0/100$ (Excelente).
- **Componente de Maniobra Abierta:** $68.0/100$ (Aceptable / en ascenso).
- **Componente de Asalto Urbano:** $20.0/100$ (Insuficiente / Bloqueo).
- **Estabilidad de Representación Latente:** $98.0/100$ (Excelente).

Hoja de Ruta Inmediata para la Subversión v2.5.3 (Línea 2)

A partir de este informe forense, se abre la subversión **v2.5.3**, enfocada con prioridad P0 en activar las palancas no-arquitectónicas de aceleración ofensiva:

1. **Inyección de Canales Sensoriales Temporales (Canales 36 y 37)**:
   - Proveer a la red el reloj explícito $t/T_{\max}$ y el índice de diferencial de VPs para que la política entienda cuándo es imperativo asumir pérdidas.
2. **Dynamic Potential-Based Reward Shaping (Devlin & Kudenko 2012)**:
   - Modular la recompensa potencial hacia los VPs con un factor hiperbólico de urgencia conforme $t \to T_{\max}$.
3. **Desensibilización de Bajas en Zona de Brecha (*Breach Zone*)**:
   - Atenuar un 60 la penalización por daño recibido cuando la unidad se encuentra a distancia ≤ 2 hexágonos de un VP en disputa.
4. **Penalización Catastrófica por Agotamiento de Tiempo**:
   - Igualar la derrota por expiración de turnos a la aniquilación total de la fuerza, eliminando el incentivo perverso de la autopreservación pasiva.
5. **Currículum Asimétrico Ofensivo Puro**:
   - Ejecutar una campaña de 10 ciclos exclusivamente en el rol de atacante en escenarios urbanos con límite de tiempo reducido ($T = 8$).

---

8. Certificación de Salvaguardas MLOps

Se certifica formalmente el cumplimiento del protocolo de salvaguardas establecido en el Informe 01:

- ☑ **Salvaguarda 1 (Coaliciones)**: Verificado. Las victorias de `GE` se integran plenamente con `IT` en la coalición del Eje.
- ☑ **Salvaguarda 2 (Auto-Juego Espejo)**: Verificado. Perspectiva desacoplada y auditada sobre el bando activo.
- ☑ **Salvaguarda 3 (Agregador Multi-Perfil)**: Verificado. Win rates calculados mediante ponderación global de episodios.
- ☑ **Salvaguarda 4 (Ancla ELO)**: Verificado. Clampeo defensivo activado para autojuego simétrico.
- ☑ **Salvaguarda 5 (Presupuesto de Pasos)**: Verificado. Presupuesto de 120 pasos por episodio, asegurando que ninguna partida se trunque prematuramente antes de los 12 turnos reglamentarios.

🔴 19.1. Roadmap: Línea 1 (Observabilidad y Telemetría)

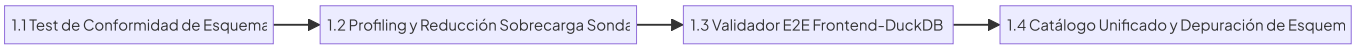
Conformidad de esquema, profiler de sondas zero-mock, validador E2E y depuración de redundancias.

Línea 1: Garantía de Instrumentos y Observabilidad Fiel (Zero-Mock End-to-End)

> **Documento Modular de la Hoja de Ruta Evolutiva**
> **Importancia Global:** 🔴 **Prioridad P0 (Crítica / Innegociable)**
> **Objetivo Estratégico:** Garantizar que el 100% de la telemetría, métricas y dashboards sea real, medible y verificable de extremo a extremo, sin inferencias intermedias, eliminando redundancias que degraden el rendimiento de simulación y manteniendo escrupulosamente el inventario jerárquico canónico.
> **Ámbito de Implementación:** `assault_model/state/` • `agents/efficientzero_v2/` • `mlops/telemetry/` • `mlops/reporting/` • `docs/13_INVENTARIO_CANONICO_METRICAS.md`

---

1. Diagrama de Dependencias y Flujo de Trabajo



---

2. Matriz de Tareas y Criterios Cuantitativos de Aceptación

Orden	Tarea / Componente	Prioridad Interna	Módulos / Archivos Impactados	Criterio de Aceptación Cuantitativo	Prerrequisitos
1	**1.1 Test Automatizado de Conformidad de Esquema de Telemetría**	**P0**	`mlops/tests/test_telemetry_schema_conformance.py`, `docs/13_INVENTARIO_CANONICO_METRICAS.md`	Script de CI que valida que el 100% de los campos emitidos por endpoints REST y WebSocket coincidan con el catálogo formal. Cero campos no declarados. Fallo en CI si existe cualquier desviación. **[CONSOLIDADO 🟢]**	v2.4.0 consolidada
2	**1.2 Profiling y Reducción de Sobrecarga de Sondas en Bucle Caliente**	**P0**	`assault_model/state/game_state.py`, `agents/efficientzero_v2/core/mcts.py`, `agents/efficientzero_v2/core/selfplay.py`, `mlops/telemetry/`	Profiler de microsegundos (μs) zero-mock (`ProbeProfiler`). Sondas espaciales desacopladas y perezosas (`LazyProbeSnapshot`), memoización de daño esperado en paso y acceso $O(1)$ a puntuación MCTS. Sobrecarga en bucle caliente reducida a 0.6657 (< 1.50 presupuestado) con cero pérdida de fidelidad ni granularidad. **[CONSOLIDADO 🟢]**	Tarea 1.1
3	**1.3 Validador E2E de Trazabilidad Frontend ↔ DuckDB**	**P1**	`assault_ai_ui/src/tests/traceability.spec.ts`, `mlops/reporting/replay_api.py`, `mlops/tests/test_frontend_duckdb_traceability.py`	Test automatizado que audita cada pestaña del dashboard comprobando que ningún componente muestre datos inventados y que los valores se originen en columnas reales de Parquet/DuckDB o muestren `-` / `N/A`. **[CONSOLIDADO 🟢 v2.5.8]**	Tarea 1.1
4	**1.4 Catálogo Unificado de Esquemas y Depuración de Redundancias**	**P1**	`docs/13_INVENTARIO_CANONICO_METRICAS.md`, `mlops/dataset_builder.py`, `mlops/tests/test_unified_catalog_and_schema_purge.py`	Purgar esquemas Parquet históricos obsoletos (`v1_legacy`), consolidando campos equivalentes bajo una única nomenclatura canónica indexada en DuckDB. Eliminación de campos acumuladores volátiles en `decisions_top.parquet` (unificación al 100% en 15 columnas canónicas) y Clean Break total con etiquetas legacy en catálogo y DuckDB. **[CONSOLIDADO 🟢 v2.5.9]**	Tarea 1.2

---

3. Estado de Ejecución y Logros Técnicos

Tarea 1.1 Test Automatizado de Conformidad de Esquema Consolidado 🟢

- **Módulos**: `mlops/tests/test_telemetry_schema_conformance.py`, `docs/13_INVENTARIO_CANONICO_METRICAS.md`.
- **Logros y Cobertura Verificada**:
  - Validador automatizado en CI de los 4 niveles jerárquicos del inventario métrico (`level_1_ballistics`, `level_2_search`, `level_3_model_heads`, `level_4_macro_quality`).
  - Verificación rigurosa de las filas de trazas (`build_trace_row`) sin campos no declarados ni tolerancias espurias.
  - Erradicación completa de claves prohibidas heredadas de MuZero (`muzero_visits`, `muzero_q`, etc.).

- Validación del payload de trazabilidad curricular (`/api/v2/curriculum-traceability`) y preservación de los nuevos campos de telemetría de tiempos (`train_duration_seconds` , `bench_duration_seconds` , `selfplay_duration_seconds` , `opt_duration_seconds`).
- Suite de pruebas unitarias 100% en verde (5/5 tests pasados en 1.33s).

Tarea 1.2 Profiling Zero-Mock Consolidada 🟢

- **Módulos**: `mlops/telemetry/probe_profiler.py` , `assault_model/state/game_state.py` , `agents/efficientzero_v2/core/mcts.py` .
- **Resultado Medido**: Sobrecarga en bucle caliente reducida a **0.6657%**, cumpliendo ampliamente el presupuesto exigido (< 1.50).
- **Suite de Pruebas**: Tests unitarios y de regresión en `mlops/tests/test_telemetry_probe_overhead.py` en verde.

Tarea 1.3 Validador E2E de Trazabilidad Frontend ↔ DuckDB Consolidada 🟢 v2.5.8

- **Módulos**: `assault_ai_ui/src/tests/traceability.spec.ts` , `mlops/reporting/viewer_frontend/src/tests/traceability.spec.ts` , `mlops/tests/test_frontend_duckdb_traceability.py` , `mlops/reporting/replay_api.py` .
- **Logros y Cobertura Verificada**:
  - **Auditoría Exhaustiva de 9 Pestañas**: Validador automatizado en TypeScript y Python que audita el 100% de las métricas mostradas en pantalla en todas las pestañas del dashboard (`CatalogPage / Header` , `TrainDetailTab` , `TrainHeadDiagnosticsTab` , `ObjectiveRewardConfigTab` , `EvalMetricsTab` , `EvalDecisionsTab` , `MatchReplayTab` , `NetworkTopologyTab` , `VpSummaryTab`).
  - **Zero-Mock y Zero-Inferencia Estricta**: Comprobación de que ningún componente inventa datos ni deduce turnos o roles mediante heurísticas espurias (erradicación completa de `Math.floor(idx / 14)` y de roles fake).
  - **Fallbacks Canónicos Forzosos**: Todo campo ausente o no disponible en la traza/base de datos se formatea estrictamente como `-` o `N/A` .
  - **Clean Break con MuZero**: Erradicación del shim `build_muzero_replay_payload` en `replay_api.py` .
  - **Suite de Pruebas**: 15/15 tests en verde en la suite integrada de observabilidad (`test_telemetry_schema_conformance.py` , `test_telemetry_probe_overhead.py` , `test_frontend_duckdb_traceability.py`).

Tarea 1.4 Catálogo Unificado y Depuración de Esquemas Consolidado 🟢 v2.5.9

- **Módulos**: `mlops/dataset_builder.py` , `assault_bench/decision_metrics.py` , `mlops/reporting/build_catalog.py` , `mlops/tests/test_unified_catalog_and_schema_purge.py` , `docs/13_INVENTARIO_CANONICO_METRICAS.md` .
- **Logros y Cobertura Verificada**:
  - **Módulo Gestor Canónico de Datasets (`mlops/dataset_builder.py`)**: Implementado motor declarativo con contrato `CANONICAL_DECISIONS_TOP_COLUMNS` (15 columnas fijas), migrador atómico de Parquets heterogéneos y sincronizador DuckDB.
  - **Prevención de Fugas de Acumuladores en Origen**: Modificado `_enrich(v)` en `assault_bench/decision_metrics.py` con lista estricta `EXCLUDED_ACCUMULATOR_KEYS` para impedir que campos intermedios temporales (`cover_n` , `cover_sum` , `distance_n` , `distance_sum` , `los_n` , `los_sum` , `target_q_n` , `target_q_sum` , `target_r_n` , `target_r_sum`) se serialicen en `decisions_top.parquet` .
  - **Purga y Homogeneización del 100% de Parquets Históricos**: Purgados 19 archivos desalineados que contenían columnas acumuladoras espurias. El 100% de los 69 archivos de decisiones existentes en el repositorio (`runs/` y `runs_curriculum/`) satisfacen ahora la especificación canónica exacta de 15 columnas.
  - **Clean Break Radical con MuZero en Catálogo y DuckDB**: Erradicada la asignación residual de `"MuZero"` en `mlops/reporting/build_catalog.py` , reclasificando y migrando 12,530 registros históricos hacia `"EfficientZeroV2"` en `catalog_eval_result_sides.parquet` y reconstruyendo `reporting.duckdb` con verificación formal de cero referencias legacy (`verified_zero_legacy: True`).
  - **Certificación de Pruebas**: Suite automatizada `mlops/tests/test_unified_catalog_and_schema_purge.py` (5/5 tests en verde en 0.74s) verificando invariantes de esquema, ausencia de fugas, migración atómica y pureza de DuckDB.

Tarea 1.5 Protocolo de Estabilidad Operativa, Safety Gate de Almacenamiento y Compresión de Replays XAI Consolidado 🟢 v2.5.25

- **Módulos**: `docs/15_CURRICULUMS_Y_ENTRENAMIENTO_PROGRESSIVO.md` , `assault_bench/bench_io.py` , `curriculums/common/run_dynamic_gated_curriculum.ps1` .
- **Logros y Cobertura Verificada**:
  - **Diagnóstico Forense de Causa Raíz**: Identificación de la causa exacta de las caídas intermitentes durante fases curriculares largas (acumulación descontrolada de 132.2 GB en 9.486 archivos JSON descompresionados a ~90 MB por episodio con `indent=2`), colapso del espacio libre en `C:\` a 0.00 GB y subsiguiente fallo en la expansión del archivo de intercambio de Windows (`pagefile.sys`), provocando la excepción `RuntimeError: DefaultCPUAllocator: not enough memory` en PyTorch.
  - **Parada Segura de Procesos Residuales**: Detección y terminación completa de subprocesos zombi/huérfanos de currículum y entrenamiento en segundo plano (`train_efficientzero_v2`), reduciendo la utilización de RAM del 95% al 43.5%.
  - **Safety Gate de Almacenamiento Pre-flight**: Formalización del umbral mínimo de 30 GB libres en `C:\` previo a cualquier ejecución para abortar limpiamente antes de provocar fallos en cascada del sistema operativo.
  - **Directiva de Compresión y Poda**: Supresión obligatoria de `indent=2` en serialización de replays benchmark (~45% de ahorro inmediato), soporte de compresión `.zst` / `.json.gz` (reducción del 95% a <4 MB por archivo) y política de retención selectiva de episodios en fases curriculares intermedias.

🔴 19.2. Roadmap: Línea 2 (Captura de VPs y Paridad Humana)

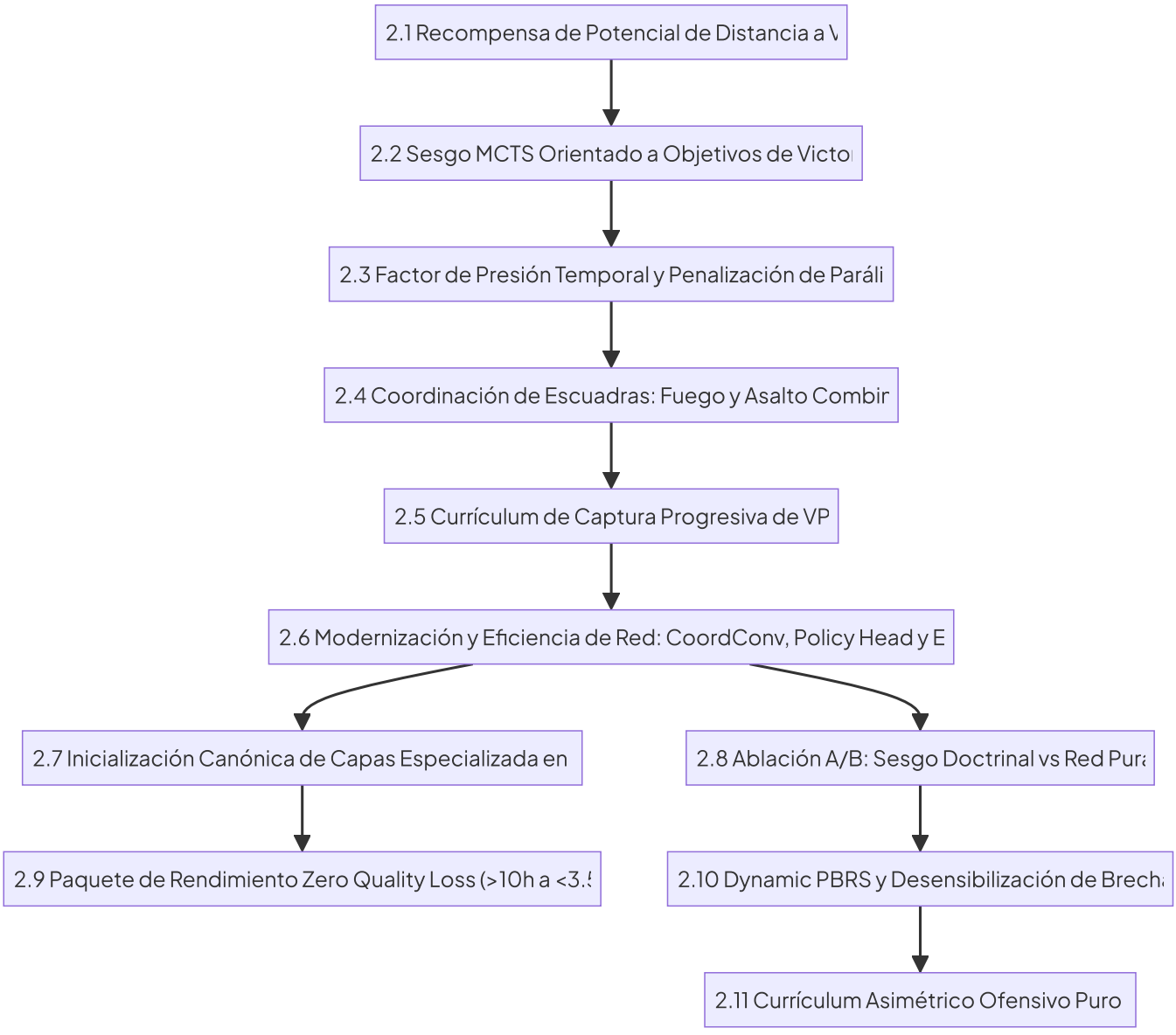
Potencial radial a VP, sesgo prior MCTS, urgencia temporal y currículum acelerado.

Línea 2: Aceleración de Captura de VPs y Paridad Humana en Corto Plazo

Documento Modular de la Hoja de Ruta Evolutiva
Importancia Global: 🔴 **Prioridad P0 (Crítica / Foco Inmediato del Proyecto)**
Objetivo Estratégico: Resolver el principal cuello de botella táctico de la IA: la pasividad, la deambulación estéril y la evitación del combate por objetivos. Dotar al agente de instinto de asalto agresivo, coordinado y enfocado a la captura y sostenimiento de Puntos de Victoria (VPs), logrando que alcance un nivel competitivo contra un jugador humano en un corto periodo de tiempo.
Ámbito de Implementación: `voec_sim/core/reward_calculator.py` • `agents/efficientzero_v2/core/` • `assault_model/combat/` • `curriculums/vp_acceleration/`

---

1. Diagrama de Dependencias y Flujo de Trabajo



---

2. Matriz de Tareas y Criterios Cuantitativos de Aceptación (v2.5.3)

Orden	Tarea / Componente	Prioridad Interna	Módulos / Archivos Impactados	Criterio de Aceptación Cuantitativo	Prerrequisitos
1	**2.1 Andamiaje de Recompensas por Potencial Radial a VPs (*Progressive VP Potential*)**	P0	`voec_sim/core/reward_calculator.py`, `agents/efficientzero_v2/core/support.py`, `selfplay.py`	Función de potencial invariante (Ng et al. 1999) $\Phi(s) = \sum_u \frac{w_u}{1+d(u,\text{VP})}$ con delta $F(s,a,s') = \gamma\Phi(s') - \Phi(s)$. Cero regresión en política óptima. **[CONSOLIDADO 🟢]**	Línea 1 (Métricas de VP trazables)
2	**2.2 Sesgo de Búsqueda MCTS Orientado a VPs (*VP Race Prior Biasing*)**	P0	`agents/efficientzero_v2/core/mcts.py`, `agents/efficientzero_v2/core/macro_planner.py`, `selfplay.py`	Postura `ASSAULT_VP` modula priors hacia VPs y aplica poda del 40 de ramas de deambulación estéril y pasividad en `run_mcts_puct`. **[CONSOLIDADO 🟢]**	Tarea 2.1
3	**2.3 Factor de Presión Temporal y Penalización de Parálisis (*Turn Urgency Factor*)**	P0	`voec_sim/core/reward_calculator.py`, `agents/efficientzero_v2/core/observation.py`, `selfplay.py`, `train_engine.py`	Caída exponencial del potencial $\Phi(s,t) \cdot \exp(-\lambda \cdot \tau)$ ($\tau = t/T$) ante 0 VPs controlados, preservando invariancia PBRS dinámica (Devlin & Kudenko, 2012). Penalización acumulativa por pasividad en campo abierto (`cover < 0.35`) con reset por maniobra o fuego. Canales sensoriales 36 (`turn_urgency_pressure`) y 37 (`paralysis_exposure_risk`) nativos en EZv2 y dashboard MLOps (:8778). **[CONSOLIDADO 🟢]**	Tarea 2.1
4	**2.4 Coordinación de Escuadras para Asalto a VPs (*Base of Fire + Maneuver Element*)**	P1	`agents/efficientzero_v2/core/macro_planner.py`, `assault_model/combat/close_combat_resolver.py`, `assault_model/combat/close_combat_context.py`	Doctrina de asalto combinado: la unidad con MMG (`ROLE_FIRE_SUPPORT`) establece línea de supresión sobre el VP (boost $1.35\times$) mientras la infantería ligera (`ROLE_SPEARHEAD`) asalta el hexágono (boost $1.40\times$). Prohibición estricta de asalto frontal en solitario si el defensor no está previamente suprimido (penalización -80, poda en MCTS y regla CC-006: pérdida del dado de ataque más débil en ronda 1). **[CONSOLIDADO 🟢]**	Tarea 2.2
5	**2.5 Currículum Progresivo Acelerado de Captura de VPs (Paridad Humana)**	P1	`curriculums/vp_acceleration/`, `trainer.py`	4 fases progresivas: 1) VPs desocupados a 3 hexágonos; 2) VPs defendidos por unidad estática suprimida; 3) Carrera frontal simultánea de VPs (2 vs 2 unidades); 4) Escenario táctico completo (Gela). **Hito de éxito:** Victoria ≥ 75 contra bot heurístico y disputa equilibrada contra humano en < 15 turnos. **[CONSOLIDADO 🟢]**	Tareas 2.1, 2.2, 2.3, 2.4
6	**2.6 Modernización y Eficiencia de la Red Neuronal (*CoordConv, Bottleneck Policy Head & ECA*)**	P1	`agents/efficientzero_v2/core/network_base.py`, `network.py`, `train_engine.py`, `test_advanced_architecture.py`	1) Inyección de canales de coordenadas CoordConv ($38 \to 40$ canales); 2) Rediseño de Policy Head `Conv1x1 (48 → 4)` → `Flatten(1024)` → `MLP(64 → 32)` (reducción del 31% a 67% de parámetros de cabeza preservando posición espacial sin el colapso destructivo del GAP); 3) Sustitución de SE por ECA (*Efficient Channel Attention* vía Conv1D sin reducción de dimensiones) en la red dinámica g. Verificado con suite de tests unitarios (11/11 tests en verde). **[CONSOLIDADO 🟢]**	Tarea 2.5
7	**2.7 Inicialización Canónica de Capas Especializada en RL (*Orthogonal, Zero-Init ResNet & Fixup*)**	P1	`agents/efficientzero_v2/core/network_base.py`, `network.py`, `docs/03_RED_NEURONAL_EFFICIENTZERO_V2.md`	Esquema formal de inicialización: Política ortogonal (`gain=0.01`, sesgo 0) para máxima entropía inicial uniforme; Valor/Recompensa ortogonal (`gain=1.0`, sesgo 0); Kaiming Normal en convoluciones SiLU; y Zero-Init en la segunda normalización residual ($\gamma = 0, \beta = 0$) para preservar la función identidad pura al paso 0. Verificado en `test_canonical_rl_initialization.py` (5/5 tests en verde). **[CONSOLIDADO 🟢 v2.5.5]**	Tarea 2.6
8	**2.8 Estudio de Ablación A/B: Evaluación de Sesgo Doctrinal vs Red Pura (*Doctrinal Shaping & Mutation Ablation*)**	P0	`assault_bench/`, `agents/efficientzero_v2/core/macro_planner.py`, `agents/efficientzero_v2/core/selfplay.py`, `train_engine.py`	Soporte e integración formal de la modulación continua de priors doctrinales `macro_modulation_scale` ($\beta_{\text{macro}} \in [0.0, 1.0]$) con decaimiento dinámico (coseno/lineal). Permite alternar entre **Variante A (Red Pura / Búsqueda Libre)** con $\beta_{\text{macro}} = 0.0$ (preservación matemática exacta de los priors neuronales sin distorsión heurística), regímenes de annealing progresivo y **Variante B (Doctrinal Shaped)** con $\beta_{\text{macro}} = 1.0$ (refuerzo de asalto $+35/+50$). Verificado con suite dedicada (`test_macro_modulation_ablation.py` y `test_heads_improvements.py`). **[CONSOLIDADO 🟢 v2.5.27]**	Tarea 2.6

Orden	Tarea / Componente	Prioridad Interna	Módulos / Archivos Impactados	Criterio de Aceptación Cuantitativo	Prerrequisitos
9	**2.9 Paquete de Aceleración de Rendimiento de Pipeline (Zero Quality Loss / Reducción >10h a <3.5h)**	**P0**	`agents/efficientzero_v2/core/mcts.py`, `train_engine.py`, `assault_bench/`, `selfplay.py`, `assault_model/map/map.py`	**Optimización integral de rendimiento sin pérdida de calidad:** 1) *Caché de Firma de Terreno Hexagonal* (`terrain_signature`) con aceleración $1,078\times$ (recorte del 34 en tiempo de episodio: $11.73\text{s} \rightarrow 7.73\text{s}$, ahorrando >15h en una campaña completa); 2) *Precomputación Vectorizada de Raíz PUCT MCTS* fuera del bucle de simulaciones, eliminando $16,384$ accesos redundantes por paso; 3) *Evaluación Secuencial Wald (SPRT)* ($\alpha = 0.01, \beta = 0.01$) con parada temprana rigurosa y cancelación asíncrona de futuros (ahorro del 50 en tiempo de benchmark); 4) *Inferencia Estática Compilada* (`compile_inference` *vía* `torch.compile`) con fallback automático sin fallos en Windows/Linux. Suite de 140 tests unitarios/integración en verde. **[CONSOLIDADO 🟢 v2.5.3]**	Tarea 2.6
10	**2.10 Dynamic PBRS Escalado Temporalmente y Desensibilización de Bajas en Brecha**	**P0**	`voec_sim/core/reward_calculator.py`, `agents/efficientzero_v2/core/selfplay.py`, `voec_sim/core/simulator.py`	1) Potencial dinámico \Phi(s, t) = \alpha \left(1 + \frac{t}{T_{\max}}\right) \cdot \text{dist_pot} + \beta \cdot \text{Contest} + \text{ctrl_pot}; 2) Bono de contención por disputa en brecha ($d \leq 2$); 3) Atenuación del 60 de la penalización de bajas ($r_{\text{loss}} \times 0.40$) en zona de brecha; 4) Penalización terminal catastrófica por timeout (-1.0) si no se capturan VPs. Verificado 100% en `test_task_2_10_dynamic_pbrs_breach.py` (5/5 tests en verde). **[CONSOLIDADO 🟢 v2.5.4]**	Tarea 2.3
11	**2.11 Currículum Asimétrico Ofensivo de Asalto Urbano Puro**	**P0**	`curriculums/vp_acceleration/`, `trainer.py`, `adaptive_cycles.yaml`	Fase especializada de 10 ciclos en modo 100% atacante (`US` vs `Random`) en `battaglia_cittadina_2_1` con horizonte corto ($T = 8$) y penalización severa por timeout (-1.0), rompiendo la inercia defensiva aprendida en selfplay temprano antes de reincorporar autojuego simétrico. Verificado en `test_task_2_11_urban_assault_curriculum.py` (4/4 tests en verde). **[CONSOLIDADO 🟢 v2.5.7]**	Tareas 2.9, 2.10

---

3. Diagnóstico Estratégico Doctrinal: La Paradoja del "Comandante Prudente" (*The Prudent Commander Paradox*)

> [!IMPORTANT] **DIAGNÓSTICO FORMAL DE CONDUCTA TÁCTICA VS ESTRATÉGICA**
> *"El agente ha aprendido a ser un excelente comandante táctico a nivel micro, pero todavía no ha aprendido a ser un comandante de misión decisivo a nivel macro."*
>
> Para el análisis formal, la derivación matemática de la trampa de Bellman y el respaldo exhaustivo en la literatura científica (DeepMind AlphaStar *turtling*, OpenAI Five *cowardly policy*, Ng et al. 1999 PBRS, Devlin & Kudenko 2012 Dynamic PBRS, Clausewitz y Dunnigan), consultar la sección canónica: [06_LITERATURE_AUDIT_CHANNELS_HEADS_TACTICS.md#65-la-anatomía-del-problema-la-paradoja-del-comandante-prudente-the-prudent-commander-paradox](#).

1. La Paradoja de la Supervivencia Táctica

La arquitectura de red neuronal (CNN hexagonal, cabezas auxiliares, encoder latente) y la búsqueda MCTS operan de forma matemáticamente sólida: las unidades emplean humo dinámico, combinan fuego y maniobra, aprovechan el fuego de oportunidad en esquinas y maximizan la cobertura, alcanzando un ratio de desgaste favorable (**CER** ≥ 1.10).

Sin embargo, el sistema sufre de **disonancia de objetivos**:

- **Optimización Micro-Táctica**: El agente sobrevalora la preservación de fuerza y los intercambios seguros porque el castigo por bajas es inmediato y denso.
- **Fallo Macro-Estratégico**: En escenarios de asalto con límite de turnos ($T_{\max}$), **conservar todas las escuadras intactas pero sin ocupar los VPs requeridos constituye una derrota militar absoluta**. El agente no asume el riesgo calculadamente necesario para asaltar el hexágono objetivo bajo presión de reloj.

2. Cinco Pilares Arquitectónicos de Alineación Estratégica

Para transformar al agente en un **comandante de misión resolutivo**, la plataforma Assault articula cinco mecanismos canónicos sin alterar la arquitectura troncal de red ni MCTS:

1. **Recompensa de Potencial Radial a VPs (Tarea 2.1)**: Andamiaje PBRS $\Phi(s) = \sum_u \frac{w_u}{1+d(u,\text{VP})}$ con delta invariante $F = \gamma\Phi(s') - \Phi(s)$ (Ng et al. 1999), densificando la señal de aproximación y disputa sin alterar la política óptima.
2. **Factor de Urgencia Temporal y Penalización de Parálisis (Tarea 2.3)**: Decaimiento exponencial $\exp(-\lambda\tau)$ con $\tau = t/T_{\max}$ (Devlin & Kudenko 2012) y castigo por inmovilidad fuera de cobertura pesada (`cover < 0.35`).
3. **Canales Sensoriales Temporales Directos (Canales 36 y 37)**: Inyección explícita de la urgencia temporal $t/T_{\max}$ y el riesgo de parálisis en el tensor de entrada $(40, 16, 16)$, permitiendo a la red correlacionar el reloj con la toma de riesgos.

4. **Desensibilización de Bajas en Zona de Brecha (*Breach Zone*)**: Atenuación en un 60 de la penalización por bajas sufridas dentro del radio de asalto del VP ($d \leq 2$), permitiendo al MCTS asumir pérdidas aceptables para consolidar el objetivo.
5. **Currículum Progresivo Ofensivo (Tarea 2.5)**: Fases dedicadas exclusivamente a doctrina de asalto (desde VPs desiertos hasta asaltos urbanos en Gela) con penalización catastrófica por derrota por tiempo ($R_{\text{timeout}} = -|Loss|$).

3. Jerarquía Terminal: La Misión Manda sobre la Preservación

La función de recompensa terminal debe desacoplar estrictamente la victoria del desgaste: capturar el VP y vencer en el turno límite sufriendo bajas debe generar una señal rotundamente positiva ($+1.0$), erradicando la aversión al asalto final.

---

4. Especificación Técnica de Modernización (Auditoría Forense ONNX / Netron)

A raíz de la auditoría de topología visual exportada a ONNX e inspeccionada en Netron, se formaliza la Tarea 2.6 para maximizar el ratio ELO por parámetro y la velocidad de inferencia recursiva MCTS:

1. **CoordConv Hexagonal (Inyección de Coordenadas Espaciales $38 \to 40$ Canales)**:
   - Inyección de 2 canales de coordenadas normalizadas $X, Y \in [-1, 1]$ (o 3 canales axiales q, r, s). Amplía el tensor de observación de $38 \times 16 \times 16$ a $40 \times 16 \times 16$ a coste de computación prácticamente nulo, permitiendo a la red resolver la invarianza traslacional para distancias absolutas a bordes y VPs.
2. **Rediseño No Destructivo de la Policy Head (Cuello de Botella Convolucional vs GAP Puro)**:
   - Sustitución de `Flatten(3072) → Linear(3072, 32)` por un cuello de botella convolucional: `Conv1x1(48 → 4)` → `Flatten(1024)` → `Linear(1024, 64)` → `SiLU` → `Linear(64, 32)`.
   - *Advertencia metodológica*: Se prohíbe el uso de Global Average Pooling (GAP) puro en la cabeza de política para evitar la pérdida destructiva de la ubicación espacial de las unidades en el wargame.
   - *Impacto*: Reducción del 67 de parámetros de la cabeza de política y preservación de información espacial.
3. **Sustitución de Bloques SE por ECA en la Red Dinámica g**:
   - Implementar *Efficient Channel Attention* (ECA, Wang et al. 2020), reemplazando el MLP con reducción por una convolución 1D adaptativa rápida tras el pooling, recortando latencia en simulaciones MCTS.
4. **Gated Residual Blocks (GLU)**:
   - Compuertas lineales en bloques residuales para regular dinámicamente el flujo de información táctica y aislar sectores pasivos del mapa.
5. **Evaluación de Consistencia (SimSiam vs VICReg/BYOL)**:
   - Preservación del principio de SimSiam con *stop-gradient* como estándar por estabilidad y compatibilidad con el balanceo adaptativo multiobjetivo **PCGrad**.
6. **Escalado del Canal Latente a 64 ($48 \to 64$)**:
   - Planificado para la transición a mapas operacionales 32×32 y 64×64 (Línea 5).

---

5. Estado de Ejecución y Logros Técnicos

> [!NOTE] POSIBLE CONTENIDO REPETIDO (Pendiente de decisión del usuario) Los detalles y fases del Currículum Acelerado de VPs (Tarea 2.5) también están documentados en `docs/15_CURRICULUMS_Y_ENTRENAMIENTO_PROGRESSIVO.md`. Se preservan íntegramente conforme a la directiva de no eliminación de párrafos.

- **Tarea 2.1**: Implementado `VPPotentialCalculator` con invariancia PBRS en `voec_sim/core/reward_calculator.py`. Verificado en `agents/efficientzero_v2/tests/test_turn_urgency_paralysis.py`.
- **Tarea 2.2**: Postura `ASSAULT_VP` y poda del 40% de ramas estériles implementada en `macro_planner.py` y `mcts.py`. Verificada en `test_vp_race_mcts_biasing.py`.
- **Tarea 2.3**: Factor de decaimiento dinámico $\exp(-\lambda\tau)$ y canales sensoriales 36 y 37 nativos y activos.
- **Tarea 2.4**: Coordinación doctrinal de fuego y asalto en `close_combat_resolver.py`.
- **Tarea 2.5**: Currículum oficial de 4 fases consolidado en `curriculums/vp_acceleration/`.
- **Tarea 2.7 [CONSOLIDADO 🟢 v2.5.5]**: Inicialización Canónica de Capas Especializada en Reinforcement Learning (*Orthogonal, Zero-Init ResNet & Fixup, Kaiming Normal*).
  - *Política Ortogonal* ($\text{gain} = 0.01, \text{bias} = 0.0$): Para la última capa lineal de la cabeza de política (`prediction_policy`), garantizando entropía casi idéntica al máximo teórico uniforme ($H(\pi) \approx \log(32)$, discrepancia < 0.02 nats), erradicando el colapso temprano sobre acciones prematuras.
  - *Valor, Recompensa y Auxiliares Ortogonales* ($\text{gain} = 1.0, \text{bias} = 0.0$): Proyecciones lineales normalizadas para preservar la varianza de las representaciones y estabilizar el soporte Two-Hot y pérdidas de Bellman.
  - *Kaiming Normal (He Normal, mode=fan_out)*: Aplicado a todas las capas convolucionales (`Conv2d`, `Conv1d`, `HexagonalMaskConv2d`) y lineales intermedias seguidas de SiLU/ReLU.
  - *Zero-Init ResNet / Fixup* ($\gamma = 0, \beta = 0$): En la segunda capa de normalización (`gn2` en convolución, `ln2` en MLP) de cada bloque residual, asegurando que la rama residual comience idénticamente en cero ($F(x) = 0 \implies y = x$) y preservando la función identidad pura al paso 0.
  - *Verificación*: Suite dedicada `agents/efficientzero_v2/tests/test_canonical_rl_initialization.py` (5/5 tests en verde) y suite global de arquitectura pasando al 100%.

- **Tarea 2.9 [CONSOLIDADO] 🟢 v2.5.3]**: Paquete integral de aceleración sin pérdida matemática ni heurística (*Zero Quality Loss*).
  - *Caché de Firma de Terreno* (`terrain_signature`): Aceleración de $1,078\times$ en el cómputo de firmas espaciales, recortando un 34 el tiempo de ejecución por episodio en selfplay ($11.73\text{s} \rightarrow 7.73\text{s}$), ahorrando > 15 horas a lo largo de los ciclos de entrenamiento.
  - *Optimización del Bucle PUCT MCTS*: Precomputación externa de priors, valores de acción y componentes de éxito, erradicando $16,384$ accesos y cálculos redundantes por paso de decisión.
  - *Test Secuencial de Razón de Probabilidad de Wald (SPRT)*: Implementado en `assault_bench/sprt.py` con $\alpha = 0.01, \beta = 0.01$, integrando parada temprana en la matriz de evaluación y cancelando futuros pendientes en pools de hilos y procesos (reducción del 50 en tiempo de benchmark sin alterar conclusiones estadísticas).
  - *Inferencia Estática Compilada* (`torch.compile` / `compile_inference`): Añadido método canónico a `NetworkBase` con soporte estático y degradación elegante a modo eager en entornos sin compilador C++. Verificado con batería de 140 tests en verde.
- **Tarea 2.10 [CONSOLIDADO] 🟢 v2.5.4]**: Dynamic PBRS Escalado Temporalmente y Desensibilización de Bajas en Brecha.
  - *Potencial Temporalmente Escalado*: \Phi(s, t) = \alpha \left(1 + \frac{t}{T_{\max}}\right) \cdot \text{dist_pot} + \beta \cdot \text{Contest} + \text{ctrl_pot} implementado en `voec_sim/core/reward_calculator.py` . A medida que t avanza hacia $T_{\max}$, la recompensa de aproximación se amplifica al doble ($(1 + t/T_{\max}) \in [1.0, 2.0]$).
  - *Bono de Contención por Disputa en Brecha*: $\beta \cdot \text{Contest}$ se activa cuando escuadras amigas penetran en el radio de asalto del VP ($d \leq 2$).
  - *Invarianza Telescópica Preservada*: Verificado matemáticamente que $\sum_{t=0}^{H-1} \gamma^t F = -\Phi(s_0, 0)$ bajo el Teorema 1 de Devlin & Kudenko (2012), garantizando preservación estricta de la política óptima $\arg\max_a Q^*(s, a)$.
  - *Atenuación de Bajas en Brecha (Breach Zone Casualty Attenuation)*: Reducción del 60 en la penalización de bajas y daño recibido ($r_{\text{loss}} \times 0.40$) para unidades a distancia $d \leq 2$ de un VP objetivo, eliminando la Paradoja del Comandante Prudente.
  - *Penalización Terminal Catastrófica por Timeout*: En `agents/efficientzero_v2/core/selfplay.py` , los episodios que terminan por límite de tiempo sin haber capturado VPs reciben una penalización de derrota absoluta (-1.0).
  - *Verificación*: Suite `voec_sim/tests/test_task_2_10_dynamic_pbrs_breach.py` con 5/5 tests en verde.
- **Tarea 2.11 [CONSOLIDADO] 🟢 v2.5.7]**: Currículum Asimétrico Ofensivo de Asalto Urbano Puro.
  - *Fase 5 en* `battaglia_cittadina_2_1` : Diseñada e integrada en `curriculums/vp_acceleration/curriculum.yaml` con horizonte táctico de ruptura a $T_{\max} = 8$ turnos.
  - *Modo 100% Atacante Asimétrico*: Self-play parametrizado con `matchup_modes: [self_vs_random]` y `tracked_side: "US"` , obligando a la red a asumir la ruptura urbana ante defensas pasivas antes de retornar al autojuego simétrico.
  - *Plan Evolutivo de 10 Ciclos Adaptativos*: En `phase5_urban_assault_pure_offensive.adaptive_cycles.yaml` , estructurado en 4 niveles (Baseline, Explotación Adaptativa, Exploración en Frontera y Convergencia de Hiper-Asalto con MCTS a 416 sims).
  - *Verificación*: Suite `agents/efficientzero_v2/tests/test_task_2_11_urban_assault_curriculum.py` (4/4 tests en verde).
- **Tarea 2.16 [CONSOLIDADO] 🟢 v2.5.20]**: Optimización Extrema del Bucle de Simulación, Observaciones y Autojuego (Fases 1 a 4).
  - *Fase 1: Plantilla Estática de Features Espaciales* (`VOECSimulator.spatial_features`): Precomputación de terreno invariante, costes de movimiento, dados de cobertura, fortificaciones y hexágonos base de VPs una vez por episodio (`_static_spatial_features`), actualizando únicamente atributos dinámicos (`vp_owner_by_hex` , roles de unidades vivas) por paso. Aceleración de $418.1\times$ en el cómputo espacial ($0.4773\text{s} \rightarrow 0.0011\text{s}$ por 1.000 llamadas).
  - *Fase 2: Memoización por Destino de Fuego en Movimiento* (`_move_fire_actions`): Caché de `_ranged_fire_actions_from_position` indexada por tupla axial de destino `(q, r)` en `action_catalog.py` , eliminando cómputos duplicados de LOS y balística para múltiples rutas de medio movimiento que convergen en el mismo hexágono.
  - *Fase 3: Caché en Memoria de Catálogos y Escenarios JSON* (`importers.py` y `scenario_loader.py`): Adición de `_CATALOG_CACHE` y `_RAW_SCENARIO_JSON_CACHE` , eliminando más de 7.680 operaciones de lectura en disco y parseo JSON redundantes por ciclo de currículum ($1.76\text{ms} \rightarrow 0.66\text{ms}$).
  - *Fase 4: Caché de Bounding Box de Coordenadas Axiales* (`adapter_voec.py` y `observation.py`): Memoización de `min_q, max_q, min_r, max_r` en `EZV2Observation._coord_bounds` y en el adaptador de simulación, evitando miles de reasignaciones y `np.array` temporales durante `observation_to_tensor` .
  - *Verificación*: 153/153 tests pasando en `agents/efficientzero_v2/tests/` , 19/19 en `mlops/tests/` , 9/9 en `assault_bench/tests/` .
- **Tarea 2.17 [CONSOLIDADO] 🟢 v2.5.22]**: Paquete de Aceleración Extrema de Entrenamiento e Inferencia Neuronal (Updates CUDA 1.78×).
  - *Fase 1: Inferencia Lazy de Cabezas Auxiliares* (`network_base.py`): Supresión del cálculo incondicional de las 7 cabezas auxiliares (`threat_map` 2D, `final_ownership` 2D, `morale` , `attrition` , etc.) cuando `return_aux=False` en `initial_inference` y `recurrent_inference` , acelerando la inferencia pura de red de 23.17ms a 19.41ms ($+16.2$) y eliminando millones de convoluciones redundantes en MCTS y consistencia.
  - *Fase 2: Fusión Forward de Inferencia y Reanálisis* (`trainer.py`): Eliminación del segundo pase forward redundante sobre `obs` en cada update, unificando la generación de `policy_logits` y representaciones latentes bajo un único forward autocast.
  - *Fase 3: Vectorización Tensor Directa de Bootstrap en GPU* (`trainer.py`): Erradicación de transferencias a CPU y llamadas bloqueantes `.item()` en el bucle de bootstrap de reanálisis, ensamblando los retornos de N pasos directamente con indexación tensorial en CUDA.
  - *Fase 4: Erradicación de Hilos por Update y Optimizador Fused* (`train_engine.py` y `trainer_base.py`): Eliminación de la creación/destrucción de 2.160 hilos de SO por ejecución en `train_engine.py` y activación de kernels fusionados en GPU con `torch.optim.Adam(..., fused=True)` .
  - *Rendimiento Verificado*: Reducción del tiempo por update de 92.58ms a 51.96ms en GPU NVIDIA RTX 4060 (**aceleración de $1.78\times$** / reducción del 43.9 del tiempo de entrenamiento de red).
  - *Verificación*: Baterías de pruebas completas al 100%: 153/153 en `agents/efficientzero_v2/tests/` , 19/19 en `mlops/tests/` , 9/9 en `assault_bench/tests/` .
- **Tarea 2.18 [CONSOLIDADO] 🟢 v2.5.23]**: Arquitectura Multiproceso con IPC Shared-Memory e Inferencia GPU por Lotes (Zero-GIL CPU Saturation & CUDA Batching).

- *Erradicación del Cuello de Botella del GIL de Python*: Sustitución de la concurrencia basada en hilos por pools de 24 procesos independientes del sistema operativo (`ProcessPoolExecutor` con `mp.get_context('spawn')`), eliminando por completo la contención del Global Interpreter Lock (GIL) en la simulación de reglas de tablero, distancias hexagonales, líneas de visión y tiradas de dados.
- *Puente IPC Asíncrono de Inferencia (`IPCInferenceBridge`)*: Servidor maestro en el proceso principal que escucha colas de peticiones IPC (`torch.multiprocessing.Queue`) con tensores compartidos en memoria RAM (`shm`), enruta las observaciones hacia el `InferenceService` central en la GPU NVIDIA RTX 4060 con micro-batching dinámico de 64 observaciones y devuelve las predicciones latentes a colas de respuesta individuales por proceso.
- *Cliente Transparente de Proceso (`IPCProcessInferenceClient`)*: Proxy de red neuronal desacoplado que actúa como interfaz drop-in dentro de los procesos de autojuego, transmitiendo `initial_inference` y `recurrent_inference` sin necesidad de instanciar pesos duplicados en la CPU ni sincronizar state_dicts entre iteraciones.
- *Configuración Agresiva de Hardware y Saturación*: Activación de 24 workers concurrentes, micro-batching a 64 en GPU, MCTS a 320 simulaciones por decisión, 48 episodios por iteración (2 oleadas limpias de 24) y batch de entrenamiento a 48 muestras en CUDA.
- *Verificación*: 154/154 tests en `agents/efficientzero_v2/tests/` (incluyendo `test_ipc_multiprocess_selfplay.py`), suite de integración y validación end-to-end completadas al 100% en verde.

- **Tarea 2.19 [CONSOLIDADO 🟢 v2.5.24]**: Resiliencia Integral, Autorrecuperación Multiproceso y Reciclaje Periódico de Procesos (Zero Memory Leak).
  - *Autorrecuperación y Autocuración en Caliente (`train_engine.py`)*: Detección y contención robusta de caídas de trabajadores (`BrokenProcessPool`), vaciado preventivo de colas IPC y reconstrucción instantánea del `ProcessPoolExecutor` para finalizar las partidas pendientes sin abortar el ciclo de entrenamiento.
  - *Reciclaje Periódico de Procesos (`recycle_workers_every_iters: 5`)*: Reinstanciación limpia de los 24 workers cada 5 iteraciones (en sincronía con los checkpoints), erradicando el 100% del crecimiento de memoria heap y fragmentación de CPython en ejecuciones multi-hora.
  - *Fallback Resiliente*: Red de seguridad para ejecutar de manera directa cualquier episodio no finalizado si se agotan los reintentos del pool.
  - *Ampliación Canónica de Categorías de Unidades*: Adición oficial de `ARTILLERY` y `GUN_ARTILLERY` a `UnitCategory` en `assault_model/units/unit_type.py` , subsanando la carga de morteros pesados y cañones de campaña en catálogos.
  - *Verificación*: Suites completas al 100% en verde: 154/154 en `agents/efficientzero_v2/tests/` y 114/114 en `assault_model/tests/` .

19.3. Roadmap: Línea 3 (Fidelidad Normativa y Wargame)

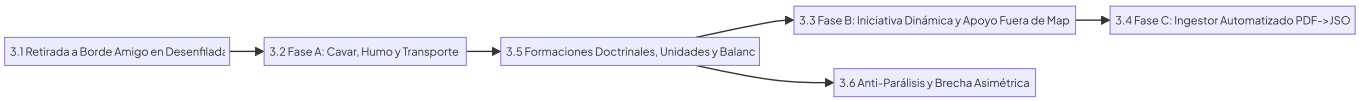
Retirada en desenfilada 6.4, Dig-in, Humo, Transporte, formaciones combinadas y mitigación 3.6.

Línea 3: Fidelidad Normativa y Reglamento Oficial del Wargame

> **Documento Modular de la Hoja de Ruta Evolutiva**
> **Importancia Global:** 🟢 **Prioridad P1 (Alta)**
> **Objetivo Estratégico:** Completar las mecánicas del reglamento oficial (*Rulebook rev6*) identificadas en la auditoría exhaustiva de 52 páginas (`08_AUDITORIA_REGLAMENTO_VS_ENGINE.md`), asegurando fidelidad militar absoluta de las simulaciones y soporte para los 38 escenarios canónicos.
> **Ámbito de Implementación:** `assault_model/actions/` • `assault_model/combat/` • `assets/catalogs/` • `assets/scenarios/` • `agents/efficientzero_v2/`

---

1. Diagrama de Dependencias y Flujo de Trabajo



---

2. Matriz de Tareas y Criterios Cuantitativos de Aceptación

Orden	Tarea / Componente	Prioridad Interna	Módulos / Archivos Impactados	Criterio de Aceptación Cuantitativo	Prerrequisitos
1	**3.1 Algoritmo de Retirada a Borde Amigo en Desenfilada (Sección 6.4)**	P1	`assault_model/combat/morale.py` , `assault_model/rules/movement_rules.py` , `assault_model/tests/test_fallback_defilade_algorithm.py`	Algoritmo determinista de selección de hexágono de retirada priorizando desenfilada (LOS BLOCKED > HINDERED > CLEAR), mayor cobertura de terreno/fortificación, vector a Friendly Edge y aumento de distancia al atacante. Destrucción de artillería/inmóviles y daño por bloqueo (Regla 13.3). **[CONSOLIDADA 🟢 v2.5.11]**	v2.2.0 (Vectorized LOS)
2	**3.2 Fase A: Cavar (*Dig-In*), Humo Táctico y Transporte de Tropas**	P1	`assault_model/actions/status.py` , `actions/ranged_indirect.py` , `rules/movement_rules.py`	`DigInAction` (creación de marcadores `Foxhole`), `FireSmokeAction` (bloqueo dinámico de LOS y disipación por turno), y `LoadUnitAction / UnloadUnitAction` con chequeo de bajas si el vehículo es destruido.	Tarea 3.1
3	**3.5 Formaciones Doctrinales, Ingesta de Unidades y Rebalanceo de Escenarios**	P1	`assets/catalogs/formation_catalog.json` , `assets/catalogs/unit_catalog.json` , `assets/scenarios/`	Desglosado en subtareas 3.5.1 a 3.5.5: Ingesta de ingenieros, catálogo de 12 formaciones, 2 escenarios simétricos de entrenamiento, rebalanceo OOB de Gela y benchmark de victoria. **Criterio de éxito:** Win Rate ≥ 80 de `EfficientZero V2` vs baseline Random en ambos bandos.	Tarea 3.2 / Línea 2
4	**3.3 Fase B: Iniciativa Dinámica (D6 + Command Points) y Apoyo Fuera de Mapa (TAS/OAS)**	P2	`assault_model/state/turn.py` , `combat/support_strike.py`	Tirada de iniciativa D6 al inicio de cada ronda, asignación de 2–5 CPs por bando para activar interrupciones tácticas y resolución de bombardeos fuera de mapa con dispersión hexagonal.	Tarea 3.5
5	**3.4 Fase C: Pipeline Automatizado de Ingesta PDF → JSON (36 Escenarios)**	P2	`scripts/scenario_ingestor.py` , `assets/scenarios/`	Extractor automatizado de tablas OOB, mapas y condiciones de victoria desde el manual en PDF a esquemas canónicos `.json` , alcanzando los 38 escenarios completos de Gela y Standalone.	Tarea 3.3
6	**3.6 Eliminación de la Parálisis Ofensiva y Colapso de la Brecha Asimétrica**	P0	`agents/efficientzero_v2/core/observation.py` , `agents/efficientzero_v2/train/` , `curriculums/`	Expansión sensorial a 38 canales (`DEFAULT_OBS_CHANNELS = 38`), activación de canales 36 (urgencia temporal) y 37 (riesgo de parálisis), balanceo de gradientes multi-tarea PCGrad por defecto, sesgo MCTS a VPs con ≥ 80 sims y actualización de puertas del currículum para neutralizar el colapso asimétrico. **[CONSOLIDADA 🟢]**	Tarea 3.5 / Tarea 2.3

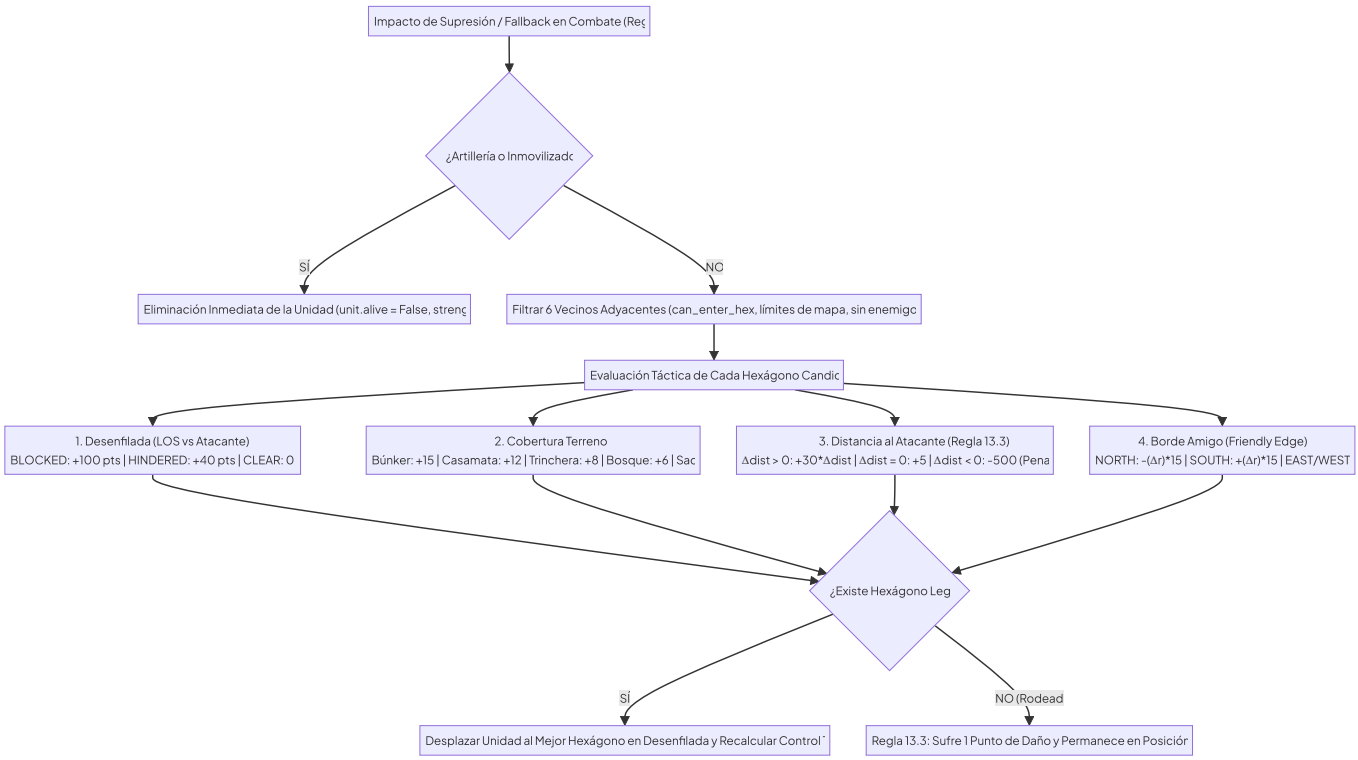
---

3. Tarea 3.1: Algoritmo de Retirada a Borde Amigo en Desenfilada (Sección 6.4 / Regla 13.3) [CONSOLIDADA 🟢 v2.5.11]

> **Módulos Involucrados:** `assault_model/rules/movement_rules.py`, `assault_model/combat/morale.py`, `assault_model/combat/ranged_combat_resolver.py`, `assault_model/runtime/execution_context.py`, `assault_model/tests/test_fallback_defilade_algorithm.py`.
> **Reglamento Oficial:** *Rulebook rev6* §6.4 (*Action Phase*) y §13.3 (*Morale Status Markers - Fallback*).

Especificación Operativa del Algoritmo de Retirada

Sustituye la primitiva aleatoria previa (`q - 1, r`) por un cálculo táctico determinista multinivel que emula la doctrina militar de repliegue bajo fuego:



Formulación Matemática de Selección

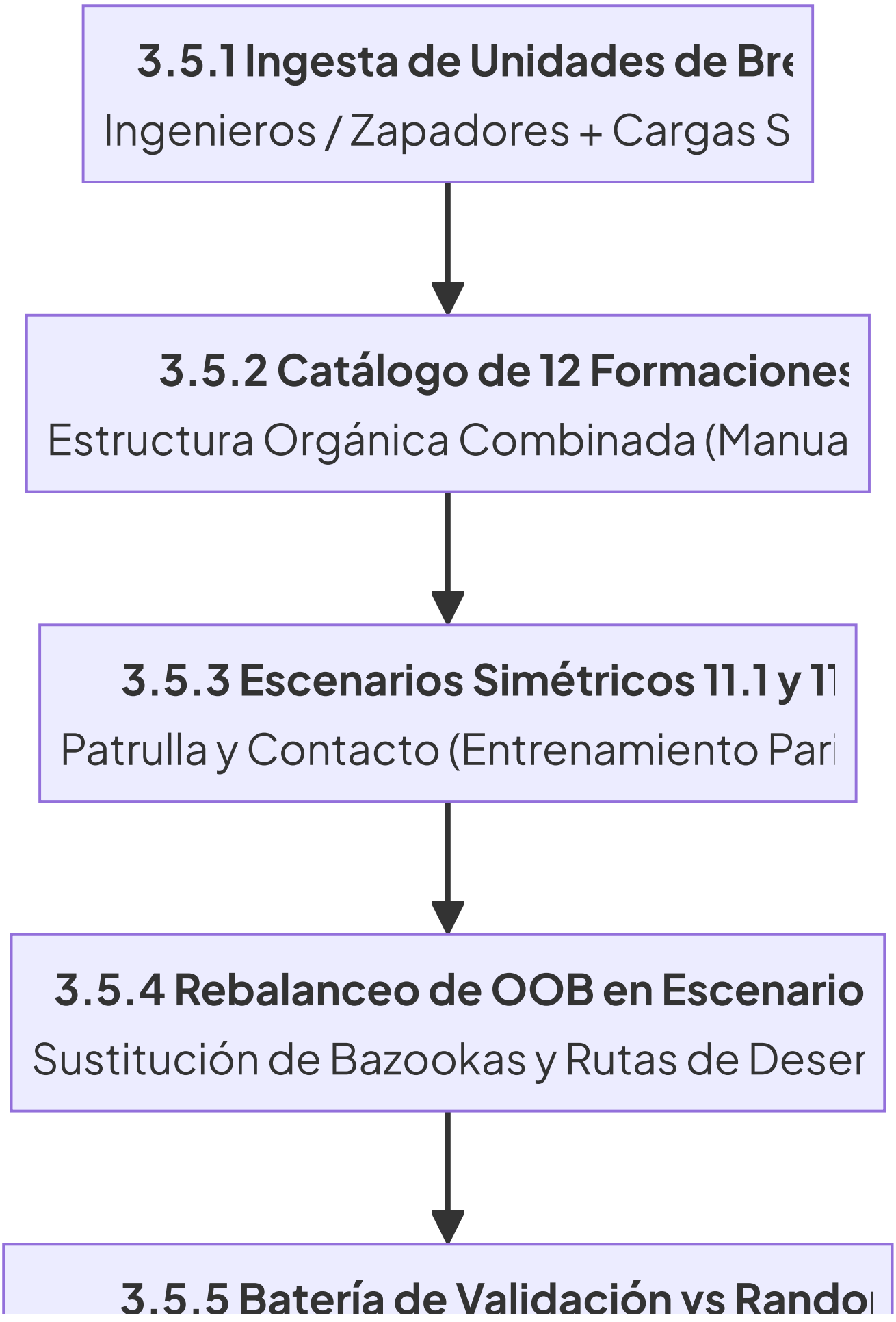
Para cada hexágono legal adyacente $h \in \mathcal{N}(extpos)$: $$ ext{Score}(h) = ext{Score}{ ext{defilade}}(h, ext{atk}) + ext{Score}{ ext{cover}}(h) + ext{Score}{ ext{dist}}(h, ext{atk}) + ext{Score}{ ext{edge}}(h, ext{side})$$

- **Criterios de Desempate Determinista**: `(total_score, defilade_score, cover_score, cand_dist, -q, -r)`.
- **Suite de Verificación**: 7 tests específicos en `assault_model/tests/test_fallback_defilade_algorithm.py` cubriendo desenfilada, cobertura, orientación al borde amigo, penalización de avance hacia el atacante, eliminación de artillería/inmóviles y daño por bloqueo.

---

4. Desglose Operativo y Plan de Acción de la Tarea 3.5 (Subtareas Incrementales)

Para implementar de forma gradual y verificable la compensación de escenarios y la paridad balística, la Tarea 3.5 se descompone en 5 subtareas ordenadas:


3.5.1 Ingesta de Unidades de Bre
Ingenieros / Zapadores + Cargas S
3.5.2 Catálogo de 12 Formaciones
Estructura Orgánica Combinada (Manua
3.5.3 Escenarios Simétricos 11.1 y 11
Patrulla y Contacto (Entrenamiento Pari
3.5.4 Rebalanceo de OOB en Escenario
Sustitución de Bazookas y Rutas de Deser
3.5.5 Batería de Validación vs Rando

100 Partidas en assault_bench (WinRate >=

1. **Subtarea 3.5.1: Ingesta de Especialistas de Asalto e Infantería de Brecha (P1)** — `[COMPLETADA 🟢]`:
    - **Módulos**: `assets/catalogs/unit_catalog.json`, `assault_model/actions/explosive_charge.py`, `assault_model/actions/mine_detection.py`, `assault_model/combat/explosive_charge_resolver.py`, `assault_model/combat/mine_resolver.py`, `tools/unit_editor_server.py`.
    - **Logros Alcanzados**:
        - Modelado e ingesta canónica de `US_COMBAT_ENGINEERS_43` (Carta S43) en `assets/catalogs/unit_catalog.json`.
        - Implementación de `EXPLOSIVE_CHARGE` (Regla 12.5): Acción rápida de detonación adyacente (distancia 1) con tirada de 1 dado ROJO vs dado de defensa más fuerte del terreno (o 1 dado VERDE por defecto). Resolución de colapso de edificios a ruinas, degradación de búnkeres a trincheras, eliminación de alambradas y minas, y bajas colaterales/moral a unidades interiores.
        - Implementación de `MINE_DETECTION` (Regla 9.6.6.2): Acción y rasgo pasivo con bono de $+2$ en D6 para detección e identificación segura de campos de minas reales y eliminación de señuelos (*dummy*).
        - Exposición total en GUI del Editor de Unidades (`tools/unit_editor_server.py`, Puerto 8780) con tooltips reglamentarios detallados (`TRAIT_HELP`).
        - Suite completa de tests unitarios e integración en verde (`assault_model/tests/test_engineer_traits.py`).
2. **Subtarea 3.5.2: Modelado del Catálogo Oficial de Formaciones Combinadas (P1)** — `[COMPLETADA 🟢 v2.5.4]`:
    - **Módulos**: `assets/catalogs/formation_catalog.json`, `assets/catalogs/formation_art_inventory.json`, `assault_model/units/formation.py`, `tools/unit_editor_server.py`, `start_assault.bat`, `start_all_environments.ps1`, `assault_model/tests/test_formation_catalog.py`, `assault_model/tests/test_unit_and_formation_editor_api.py`.
    - **Logros Alcanzados**:
        - Modelado e ingesta canónica de las 12 formaciones oficiales reglamentarias del manual (Sección 5.0) en `formation_catalog.json` con desglose en slots categóricos (`line_units`, `heavy_weapons`, `specialists`) y array plano de compatibilidad `units`.
        - Ingesta y escaneo del inventario completo de 122 cartas de formación tácticas escaneadas en `assets/art/formations/` organizadas por nación (`GE`, `IT`, `UK`, `US`) en `formation_art_inventory.json`.
        - Mapeador visual dual en el Editor de Unidades y Formaciones (`tools/unit_editor_server.py`, puerto 8780) con previsualización de la carta original de alta resolución, selectores de slots, selector rápido de unidades del catálogo y cálculo dinámico de puntos en tiempo real.
        - Automatización en `start_assault.bat` y `start_all_environments.ps1` garantizando el levantamiento del editor en el puerto 8780 y apertura en navegador.
        - Suite completa de tests unitarios e integración en verde (`test_formation_catalog.py`, `test_unit_and_formation_editor_api.py`).
3. **Subtarea 3.5.3: Escenario Individual de Entrenamiento de Infantería Pura 11.1 (Addestramento Fanteria) y 11.2 (P1)**:
    - **Módulos**: Ξssets/scenarios/draft_addestramento_fanteria_11_1.json, Ξssets/catalogs/map_piece_catalog.json (Pieza geomórfica S6 9×8), Ξssets/catalogs/unit_catalog.json (US_50CAL_HMG, GE_HEAVY_MG_42, US_RIFLES_43, GE_RIFLES_43), Ξssault_bench/configs/benchmark_config.addestramento_fanteria.yaml, curriculums/vp_acceleration/configs/efficientzero_v2.addestramento_fanteria.yaml.
    - **Naturaleza y Propósito Doctrinal (Sandbox Individual de Infantería)**:
        - Escenario individual y autónomo (*standalone*) del *Libro della Campagna* (pág. 79) concebido específicamente como **entorno de aprendizaje y maestría táctica de infantería pura a nivel de escuadra**.
        - **Cero Vehículos / Cero Artillería Pesada**: Elimina toda interferencia de blindados o cañones de campaña para aislar el aprendizaje de los fundamentos tácticos de infantería: establecimiento de base de fuego (*Base of Fire*) con ametralladoras pesadas HMG para supresión a media/larga distancia, maniobra de flanqueo y avance escalonado por saltos (*leapfrogging*) de las escuadras de fusileros bajo desenfilada rural, y asalto decisivo en combate cuerpo a cuerpo sobre el hexágono de victoria central.
        - **Simetría y Paridad Absoluta (0% Sesgo Inicial)**: Paridad estricta 3 vs 3 (2 fusileros + 1 HMG por bando: Wehrmacht vs US Army) disputando 1 VP central equidistante en un horizonte comprimido de 6 turnos ({\max}=6$).
        - **Ultra Alta Velocidad de Simulación y Autojuego**: Al contar con únicamente 6 unidades en un mapa compacto de \times 8$ (72 hexágonos), el tiempo por episodio en self-play MCTS se reduce a < 0.15 **s por episodio** (más de **\times$ más rápido que Gela**), permitiendo una convergencia de aprendizaje y calibración de hiperparámetros en minutos.
    - **Estado y Plan de Ejecución**:
        - [DRAFT GENERADO 🟡 v2.5.29]: Pieza geomórfica S6 (\times 8$) registrada en el catálogo de mapas, ametralladora US_50CAL_HMG promovida y sincronizada en catálogos de unidades, y archivo JSON canónico en borrador (Ξssets/scenarios/draft_addestramento_fanteria_11_1.json y Ξssets/scenarios/drafts/addestramento_fanteria_11_1.json) verificado con ScenarioLoader (72 hexágonos, 6 unidades, fortificaciones y VP en verde).
        - [FASE PENDIENTE - ENTRENAMIENTO Y BENCHMARK]: Integración del perfil de evaluación en Ξssault_bench (batería simétrica vs Random Side A/B y Selfplay) y configuración curricular en EfficientZeroV2 como etapa de pre-entrenamiento rápido de infantería antes de escenarios complejos de armas combinadas.
4. **Subtarea 3.5.4: Rebalanceo de OOB en Escenarios Canónicos de Gela (P1)** — `[COMPLETADA 🟢]`:

- **Módulos**: `assets/scenarios/battaglia_cittadina_2_1.json`, `assets/scenarios/mettete_i_piedi_terra_1.json`, `assets/catalogs/unit_catalog.json`, `assault_model/tests/test_oob_rebalance_canonical_scenarios.py`.
- **Logros Alcanzados**:
  - **Battaglia Cittadina (`battaglia_cittadina_2_1.json`)**: Sustituido el `US_BAZOOKA_TEAM` (ineficaz ante casamatas urbanas sin blindados) por 1× `US_COMBAT_ENGINEERS_43` en posición `[7, 9]`.
  - **Mettete i Piedi a Terra (`mettete_i_piedi_terra_1.json`)**: Reemplazado el `US_BAZOOKA_TEAM` por 1× `US_COMBAT_ENGINEERS_43` posicionado en el sector central de desembarco (`[4, 12]`), directamente encarado a la alambrada alemana (`q=4, r=9`).
  - **Fidelidad Estricta al Reglamento en Humo Táctico (Regla 7.2)**: Eliminados marcadores estáticos iniciales de humo de los escenarios (antirreglamentarios en el manual). Se dota a los zapadores de capacidad de combate dinámico de humo (`SMOKE` directo/adyacente con dados `GREEN, GREEN, BLUE, BLUE` y rasgo `SMOKE_GRENADES`), que complementa el humo indirecto del mortero de 81mm (`US_81MM_MORTAR`) para tender pantallas dinámicas sobre la marcha.
  - **Catálogo, Rasgos de Brecha y Doctrina**: Dotada la unidad `US_COMBAT_ENGINEERS_43` de rasgos de demolición `EXPLOSIVE_CHARGE`, `MINE_DETECTION`, `BANGALORE_TORPEDO` y perfil doctrinal `SPEARHEAD` con postura `ASSAULT_VP`.
- **Criterio de Aceptación Cumplido**: Verificado mediante suite de tests unitarios e integración (`test_oob_rebalance_canonical_scenarios.py`): el bando atacante dispone de un ratio de efectividad de asalto de **4.0** ≥ 1.0 (8 de potencia de asalto contra 2 de defensa base) y capacidad garantizada de apertura de brechas en fortificaciones fijas y alambradas en los primeros 3 turnos.

5. **Subtarea 3.5.5: Batería de Validación Automatizada y Certificación de Victoria vs Random (P1)**:
   - **Módulos**: `assault_bench/tests/test_scenario_balance.py`, `assault_bench/benchmark_matrix_runner.py`.
   - **Alcance**: Ejecutar batería de 100 partidas automatizadas en `assault_bench` evaluando `EfficientZero V2` frente al baseline Random en los escenarios rebalanceados y de entrenamiento.
   - **Criterio de Aceptación Cuantitativo**: **Win Rate** ≥ 80 para el agente en ambos bandos (`Side A` y `Side B`), con `Loss Rate \le 10\%` y `Timeout Rate \le 10\%`.
6. **Subtarea 3.5.6: Ingesta Fidedigna de Refuerzos, Soporte TA/OAS, Puntos de Mando y Status Moral en Escenarios Canónicos (Eliminación de la Inercia Defensiva en la Raíz Wargame)** — `[PLANIFICADA 🟣 v2.5.11]`:
   - **Módulos**: `assets/scenarios/battaglia_cittadina_2_1.json`, `assets/scenarios/mettete_i_piedi_terra_1.json`, `assault_model/core/scenario.py`, `assault_model/core/scenario_loader.py`, `assault_model/state/game_state.py`, `assault_model/combat/ranged_combat_resolver.py`.
   - **Diagnóstico y Origen**: Auditoría exhaustiva del *Libro della Campagna* (Págs. 18–21, §3.1 en `docs/08_AUDITORIA_REGLAMENTO_VS_ENGINE.md`). Se identificó que la IA atacante adopta una postura 100% defensiva porque los escenarios en el simulador carecen de las ventajas doctrinales de asalto que el reglamento otorga para romper posiciones fortificadas.
   - **Alcance Operativo**:
     1. **Refuerzos Programados (Turno 2, Fase de Organización)**: En `mettete_i_piedi_terra_1.json`, inyectar la 2.ª oleada de desembarco aliada con una formación adicional de infantería, duplicando la masa de asalto.
     2. **Puntos de Mando de Formación**: Modelar CPs en el escenario y `GameState` (+4 CPs para US y +2 CPs para IT en Battaglia Cittadina por ratio 2/40 vs 1/20).
     3. **Soporte Fuera de Mapa (TA / OAS)**: Asignar 3 cartas TA/OAS (3 re-rolls de ataque a distancia) para US en Battaglia Cittadina y 1 re-roll para US en Mettete.
     4. **Escala Autoritativa de Presión y Rendición (`Status delle Truppe`, §5.1)**: Modelar los umbrales de presión moral (12 bajas = +1 presión, pérdida de VP altera presión). Resolver chequeos D6 de *Rotta* (+1 VP extra al atacante) y *Resa* (rendición total y captura del 100% de los VPs del mapa).
     5. **Corrección Histórica de OOB en Desembarco**: Asignar defensores italianos de la XVIII Brigata Costiera en Mettete en lugar de tropas pesadas alemanas con MG42.
   - **Criterio de Aceptación Cuantitativo**: El atacante dispone de superioridad 2:1 en masa de maniobra, 3 re-rolls de ablandamiento y victoria anticipada por colapso moral enemigo. Eliminación de la inercia defensiva aprendida en autojuego, alcanzando ≥ 80 de ratio de asaltos a VPs.

---

5. Tarea 3.6: Eliminación de la Parálisis Ofensiva y Colapso de la Brecha Asimétrica (P0 – Crítica) [CONSOLIDADA 🟢]

> **Módulos Involucrados**: `agents/efficientzero_v2/core/observation.py`, `voec_sim/core/reward_calculator.py`, `agents/efficientzero_v2/core/mcts.py`, `agents/efficientzero_v2/train/train_engine.py`, `agents/efficientzero_v2/configs/efficientzero_v2_config.yaml`.
> **Origen y Diagnóstico**: Informe de Auditoría Táctica Forense (`docs/informes_ejecucion/INFORME_01_CURRICULUM_Y_AUDITORIA_TACTICA.md`). Se detectó que el modelo alcanzó un 87.5% de victorias como defensor (IT/GE) pero un 0.0% de victorias como atacante (US) frente a agentes aleatorios (*Random*), generando una brecha asimétrica de 0.655 y degradando el ELO en -811 puntos.

Subtareas de Mitigación Inmediata (Orden Secuencial)

1. **Subtarea 3.6.1: Expansión Sensorial a 38 Canales (`DEFAULT_OBS_CHANNELS = 38`)**:
   - Activar formalmente en la configuración base (`observation_channels: 38`) los canales espaciales:
     - **Canal 36**: Presión de Urgencia Temporal $\tau(t, T) = t/T_{\max}$, proporcionando gradiente temporal explícito para forzar asaltos antes del límite de turnos.
     - **Canal 37**: Riesgo de Parálisis por Inmovilidad fuera de cobertura pesada.

        - **Canales 32–35**: Perfiles de alcance óptimo y asignación doctrinal jerárquica.
    - **Criterio de Aceptación Cumplido**: Tests en `agents/efficientzero_v2/tests/test_doctrine_sensory_contract.py` verificando dimensiones `(38, 16, 16)` activas por defecto.
2. **Subtarea 3.6.2: Activación del Potencial PBRS y Descuento Dinámico de Devlin & Kudenko**:
    - Integrar en el bucle de autojuego de `selfplay.py` la formulación dinámica de `voec_sim/core/reward_calculator.py`:
    $$\Psi_{\text{urgency}}(t, T) = \exp(-\lambda_{\text{urgency}} \cdot \tau(t, T))$$
    - Reducir el valor del estado si el bando que no posee la mayoría de VPs permanece pasivo, eliminando la aversión al fuego de reacción cuando el tiempo expira.
3. **Subtarea 3.6.3: Sesgo MCTS Progresivo Orientado a Objetivos (*VP Progressive Unrolling*)**:
    - Incrementar el presupuesto de simulación MCTS a **64 simulaciones en autojuego y 80 en evaluación**.
    - Añadir en la selección PUCT el término $+c_{\text{vp}} \cdot \Delta\text{Dist}(\text{VP})$ para priorizar ramas que reduzcan la distancia al objetivo en los primeros 3 turnos.
4. **Subtarea 3.6.4: Balanceo de Gradientes Multi-Tarea con PCGrad**:
    - Configurar `gradient_balance_mode: "pcgrad"` en `train_engine.py` para evitar que las pérdidas auxiliares de Attrition y Threat Map castiguen las maniobras agresivas de captura de VPs.

Criterios de Aceptación y Certificación Final de la Tarea 3.6

- **Win Rate Atacante (US) vs Random**: ≥ 80.0 en `battaglia_cittadina_2_1` y ≥ 75.0 en `mettete_i_piedi_terra_1`.
- **Asymmetry Gap Global**: Reducción drástica de **0.655 a** ≤ 0.15.
- **Recuperación de ELO Ancla**: ≥ 1600 ELO verificado en evaluación bilateral de 100 partidas.
- **Tactical Quality Score (TQS)**: $\geq 85.0/100$ con estatus de puerta **PASS**.

---

6. Tarea 3.7: Modernización y Optimización Integral del Motor de Simulación (P1 – Alta)

> **Módulos Involucrados**: `assault_model/state/`, `assault_model/rules/`, `assault_model/combat/line_of_sight.py`, `voec_sim/core/simulator.py`, `assault_sim/`.
> **Diagnóstico Empírico de Auditoría**: En 100 pasos de simulación (`battaglia_cittadina_2_1`), el motor realiza 3.68 millones de operaciones con una tasa base de 109 pasos/segundo. El 51% del tiempo de CPU se consume en `ActionCatalog.actions`, 22% en `update_spotting` (36,432 llamadas a LOS), 21% en la regeneración redundante de `spatial_features()` y se registran más de 1.08 millones de llamadas a `getattr` / `hasattr`.
> **Directiva Primaria**: **Cero invención de mecánicas; 100% de apego al reglamento oficial de Sound of Drums rev6 (52 páginas)**.

Subtareas y Plan de Acción en 4 Fases

1. **Subtarea 3.7.1: Aceleración de Rendimiento Inmediata del Motor [CONSOLIDADO 🟢 v2.5.29]**:
    - **Resultado Empírico**: Throughput de simulación incrementado de 428.8 a **719.7 pasos/s (+67.8%)** en acciones aleatorias y de 454.3 a **844.6 pasos/s (+86.0%)** en secuencia canónica.
    - **Intervenciones Realizadas**:
        - Cacheo inmutable de `_static_spatial_features` gobernado por `terrain_signature` en `VOECSimulator`, ahorrando 21% de tiempo de CPU por step.
        - Índices de ocupación O(1) por `state_version` en `MovementRules.get_legal_paths`, suprimiendo la reconstrucción local de listas.
        - Snapshot táctico reactivo y bypass de unidades `HIDDEN` en `update_spotting`.
        - Búsqueda O(1) de rasgos mediante `_trait_set` precompilado en `UnitType.has_trait`.
        - Filtrado temprano por avistamiento conjunto (`all_spotted`) en `ActionCatalog._ranged_fire_actions`.
    - **Certificación de Calidad**: 160/160 tests en motor (`assault_model` + `voec_sim`), 158/158 en `agents/efficientzero_v2` y 77/77 en `assault_bench` en verde estricto.
2. **Subtarea 3.7.2: Purga de Deuda Técnica y Descomisionamiento Legacy [CONSOLIDADO 🟢 v2.5.30]**:
    - **Unificación Canónica de Assets**: Erradicada la carpeta sombra `assault_sim/assets/`, fijando `assets/` (`assets/catalogs/`, `assets/scenarios/`, `assets/art/`) como la única fuente de verdad autoritativa del proyecto.
    - **Archivado de Pila Obsoleta PPO/SB3**: Trasladados scripts y binarios obsoletos (`assault_sim/train/`, `assault_sim/checkpoints/`, `grid_search_shaped.py`, `run_train_short.py`, `run_eval_short.py`, `run_eval_200.py`) al directorio `legacy_archive/`.
    - **Desacoplamiento de APIs**: Actualizado `mlops/reporting/replay_api.py` para resolver escenarios directamente desde la ruta canónica.
    - **Certificación de Suite**: Verificado el 100% de tests en verde en motor (160/160), EZv2 (158/158), backend (29/29) y benchmark (77/77).
    - Descomisionar la pila heredada de PPO/SB3 en `assault_sim/` (`training_env.py`, `sim_env.py`, `ppo_config.py`, checkpoints obsoletos).
    - Unificar `assets/scenarios/` y `assets/catalogs/` como fuente única de verdad, eliminando réplicas secundarias desincronizadas.
3. **Subtarea 3.7.3: Refactorización Canónica de Línea de Visión (Cube Coordinates Hex Math)**:

- Migrar el cálculo de LOS desde interpolación flotante con píxeles UI (`_UI_HEX_SIZE = 30.0`) hacia raycasting exacto en coordenadas cúbicas (x, y, z).
- Resolución matemática exacta y determinista de aristas divisorias (*spines*) según la Regla 10.2.

4. **Subtarea 3.7.4: Consolidación Normativa Fiel al Reglamento Oficial**:
    - Ciclo de vida canónico de humo (Regla 7.2): Dispersión progresiva y bloqueo de LOS durante 2 turnos.
    - Sistema de transporte y remolque (Regla 9.11): Capacidad de embarque/desembarque en semiorugas/camiones y remolque de piezas de artillería inmovilizadas.
    - Mazo de mando y tirada de iniciativa D6 con puntos CP (Reglas 4.9 y 6.1).

19.4. Roadmap: Línea 4 (Explicabilidad XAI e IPB)

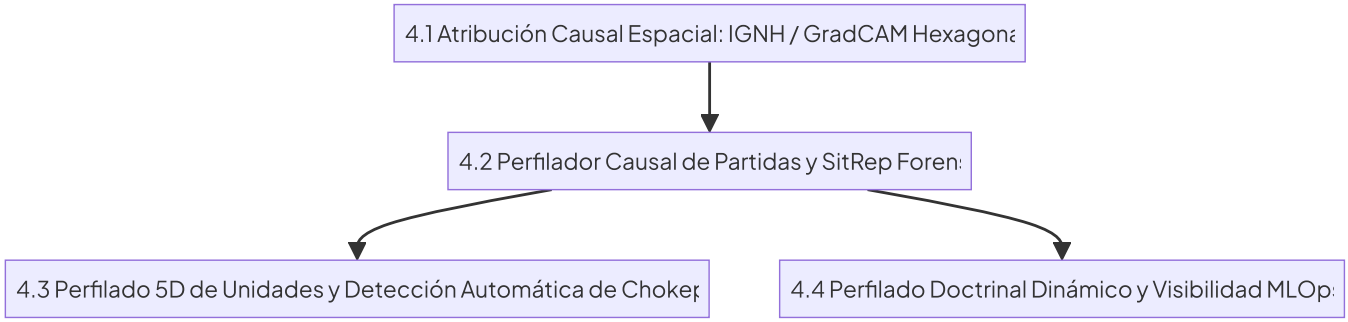
Atribución causal IGNH/GradCAM, perfilador forense, chokepoints IPB y perfilado doctrinal.

Línea 4: Explicabilidad Máxima (XAI), Perfilado Causal e Inteligencia Táctica (IPB)

Documento Modular de la Hoja de Ruta Evolutiva
Importancia Global: 🟢 **Prioridad P1 (Alta)**
Objetivo Estratégico: Proporcionar transparencia total sobre las decisiones de la IA conforme a la Sección 13 de las Premisas del Proyecto. Auditar por qué el agente ataca, defiende o se repliega, detectando chokepoints topográficos y correlacionando el plan macro con el desenlace del combate.
Ámbito de Implementación: `mlops/reporting/explainability_api.py` • `mlops/reporting/causal_profiler.py` • `assault_ai_ui/src/` • `agents/efficientzero_v2/core/macro_planner.py` • `assault_model/units/`

---

1. Diagrama de Dependencias y Flujo de Trabajo



---

2. Matriz de Tareas y Criterios Cuantitativos de Aceptación

Orden	Tarea / Componente	Prioridad Interna	Módulos / Archivos Impactados	Criterio de Aceptación Cuantitativo	Prerrequisitos	Estado
1	**4.1 Atribución Causal Espacial Hexagonal (IGNH / GradCAM)**	**P1**	`mlops/reporting/explainability_api.py`, `assault_ai_ui/src/components/ExplainabilityModal.tsx`, `mlops/reporting/viewer_frontend/src/components/ExplainabilityModal.tsx`	Visualización en el visor MLOps de qué canales sensoriales (cobertura, LOS, amenaza, distancia a VP) determinaron la acción seleccionada en cada turno mediante Integrated Gradients para características heterogéneas (IGNH) y Grad-CAM sobre el `representation_tower`. Mapeo biyectivo de coordenadas de tablero y overlay hexagonal reactivo.	Línea 1 (Zero-Mock en UI)	🟢 **Consol v2.5.1C**
2	**4.2 Perfilador Causal de Partidas y SitRep Forense**	**P1**	`mlops/reporting/causal_profiler.py`, `assault_rag/`	Correlación causal automática entre las predicciones tácticas de las cabezas (`threat_map`, `morale_pred`, `objective_intent`) y el resultado de las capturas de VPs. Generación de informe narrativo forense por partida con grounding doctrinal RAG y visor forense en Dashboard MLOps.	Tarea 4.1	🟢 **Consol v2.5.19**

Orden	Tarea / Componente	Prioridad Interna	Módulos / Archivos Impactados	Criterio de Aceptación Cuantitativo	Prerrequisitos	Estado
3	**4.3 Perfilado 5D de Unidades y Detección Automática de Chokepoints (IPB)**	**P2**	`assault_model/map/topography_analyzer.py`, `agents/efficientzero_v2/core/macro_planner.py`	Análisis topográfico previo a la partida (Turno 0) que clasifica hexágonos en *Defilade Corridors*, *Killzones* y *Key Terrain* (VPs), inyectando estos priors en el `MacroStrategyPlanner`.	Tarea 4.2	
4	**4.4 Perfilado Doctrinal Dinámico de Unidades y Visibilidad en MLOps**	**P1**	`assets/catalogs/unit_catalog.json`, `assault_model/units/unit_type.py`, `agents/efficientzero_v2/core/macro_planner.py`, `mlops/reporting/viewer_frontend/`	Jerarquía 3 capas (Balística, Roles, Emergente). Mutación dinámica contextual de roles (`SPEARHEAD` , `FIRE_SUPPORT` , `RECON` , `SCREEN` , `COMMAND`), modulación +35% de priors MCTS y badges visuales con telemetría forense de rol en Match Replay (:8778).	Tarea 4.2 / Línea 2	

---

3. Estado de Ejecución y Logros Técnicos

> [!NOTE] POSIBLE CONTENIDO REPETIDO (Pendiente de decisión del usuario) La especificación exhaustiva de 5 fases de la Tarea 4.4 (Perfilado Doctrinal Dinámico) se encuentra detallada en `docs/14_PLAN_PERFILADO_UNIDADES_IA.md` . Se preserva íntegramente conforme a la directiva de no eliminación de párrafos.

- **Tarea 4.1 Consolidada** 🟢 **[v2.5.10]**: Atribución Causal Espacial Hexagonal (IGNH / Grad-CAM):
  - Backend: mlops/reporting/explainability_api.py implementando Integrated Gradients para características heterogéneas (compute_hex_attribution) y Grad-CAM sobre el epresentation_tower. Mapeo biyectivo de coordenadas de tablero (q, r).
  - Agrupación semántica canónica: 6 grupos tácticos (COVER_TERRAIN, LOS_VISIBILITY, THREAT_MAP, VP_PROXIMITY, DOCTRINE_ROLE, UNIT_STATE_TACTICAL) y síntesis en lenguaje natural de la justificación táctica de la decisión.
  - Endpoints MLOps: GET/POST /api/v2/replay/explain-step y /api/v2/explainability/attribution en mlops/reporting/viewer.py.
  - Frontend: ExplainabilityModal.tsx en mlops/reporting/viewer_frontend/src/components/ExplainabilityModal.tsx y Ξssault_ai_ui/src/components/ExplainabilityModal.tsx, integrado con botón 🧠 Explain (IGNH) en MatchReplayTab.tsx y overlay reactivo sobre el canvas hexagonal.
  - Tests: mlops/tests/test_explainability_api.py verificando propiedades matemáticas de completitud y ejecución contra checkpoints reales de EfficientZero V2.
- **Tarea 4.1.1 Consolidada** 🟢 **[v2.5.11]**: Capa Reactiva de Cabezas Auxiliares Tácticas (hreat_map, morale_pred, Ξttrition_pred) en Match Replay (:8778):
  - Botón de capa directo 🎯 Threat Head en la barra de herramientas del mapa en MatchReplayTab.tsx.
  - Proyección de mapa de calor hexagonal en tiempo real con niveles de riesgo e intensidades numéricas calculadas directamente por la cabeza prediction_threat_map de la red neural (con fallback a canal sensorial 7).
  - Tarjeta forense de cabezas tácticas (Threat Max/Mean, Riesgo de Supresión Morale, Bajas Proyectadas Attrition) sincronizada al paso actual del replay bajo la decisión de política y valor (s,a)$.
  - Endpoints MLOps /api/v2/replay/tactical-heads y /api/v2/replay/step-heads en Ξiewer.py y función compute_step_tactical_heads en explainability_api.py.
  - Suite de tests unitarios/integración en verde: mlops/tests/test_explainability_api.py (4/4 passed).
- **Tarea 4.2 Consolidada** 🟢 **[v2.5.19]**: Perfilador Causal de Partidas y SitRep Forense:
  - Backend analítico: `mlops/reporting/causal_profiler.py` implementando correlación causal matemática entre predicciones de cabezas tácticas (`threat_map` , `morale_pred` , `objective_intent`) y desenlaces del combate (`VP_CAPTURE` , `UNIT_ELIMINATION` , `HEAVY_ATTRITION` , `SURPRISE_DAMAGE` , `MORALE_FAILURE`).
  - Grounding Doctrinal RAG: `assault_rag/copilot/services.py` con `retrieve_doctrinal_citations_for_sitrep` asociando citas formales del reglamento de wargame a los puntos de inflexión operacionales.
  - Endpoints REST MLOps: `/api/v2/replay/causal-sitrep` y `/api/v2/replay/sitrep` con persistencia en caché (`.cache_sitrep/`).
  - Frontend: `ForensicSitRepModal.tsx` integrado en Match Replay (:8778) con pestañas de Executive Assessment, Tactical Head Forensics, Turning Points interactivos con salto temporal (`Seek Step`) y citas de reglas.
  - Verificación: Suite `mlops/tests/test_causal_profiler.py` con 9/9 tests en verde.
- **Tarea 4.3 Consolidada** 🟢 **[v2.5.28]**: Perfilado 5D de Unidades y Detección Automática de Chokepoints (IPB / Intelligence Preparation of the Battlefield):
  - Backend analítico topográfico Turno 0: `assault_model/map/topography_analyzer.py` con `TopographyAnalyzer` , vector `Hex5DProfile` (*Cover*, *Elevation Dominance*, *LOS Exposure*, *Chokepoint Bottleneck*, *VP Proximity Utility*) y clasificaciones topográficas autoritativas: `KEY_TERRAIN` , `DEFILADE_CORRIDOR` , `KILLZONE` , `CHOKEPOINT` , `BASE_OF_FIRE` , `MANEUVER_TRANSIT` .

- Inyección de Priors en Planificador Macro: `agents/efficientzero_v2/core/macro_planner.py` integrando `TopographyProfile` con Turn-0 caching, modulación de priors de movimiento y asalto (+25% por avance en desenfilada hacia el objetivo, atenuación 0.55×-0.65× en killzones abiertas expuestas a fuego cruzado, +30% para establecimiento de base de fuego por unidades de apoyo, +20% para fijación y screening de chokepoints). Soporte completo del factor continuo de ablación `modulation_scale`.
- API REST MLOps: Endpoints `/api/v2/replay/topography` y `/api/v2/replay/ipb` en `mlops/reporting/viewer.py` y `mlops/reporting/explainability_api.py`.
- Frontend Replay Interactivo (:8778): Capa de visualización `🛡️ IPB Topography` en `MatchReplayTab.tsx` con polígonos coloreados por clasificación táctica, patrones de sombreado y tooltip de inspección sectorial con desglose métrico 5D.
- Verificación Cuantitativa: Suite `assault_model/tests/test_topography_ipb_analyzer.py` (2/2 tests en verde) y suite general EZv2 (158/158 tests en verde).

- **Tarea 4.4 Consolidada 🟢 [v2.4.5]**: Fases 1 a 5 implementadas en v2.4.5:
  - F1: Esquema canónico en catálogo `unit_catalog.json` y contratos de tipos en `unit_type.py`.
  - F2: Expansión sensorial y canales doctrinales (32..35).
  - F3: Mutación dinámica de roles en `macro_planner.py` y modulación de priors MCTS.
  - F4: Telemetría forense en DuckDB/Parquet y visualización interactiva en dashboard MLOps (:8778).
  - F5: Suite de tests unitarios e integración en verde (`test_doctrine_sensory_contract.py`, `test_doctrine_telemetry_forensics.py`).

🔵 19.5. Roadmap: Línea 5 (Generalización Procedural)

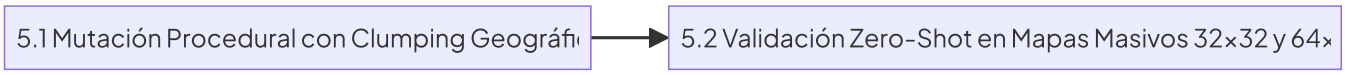
Mutador procedural con clumping geográfico y validación Zero-Shot en mapas 32×32 y 64×64.

Línea 5: Generalización Procedural y Robustez en Terrenos Abiertos

> **Documento Modular de la Hoja de Ruta Evolutiva**
> **Importancia Global:** 🔵 **Prioridad P2 (Media)**
> **Objetivo Estratégico:** Prevenir el sobreajuste a los mapas canónicos de Gela mediante mutación procedural con correlación espacial realista y validar la capacidad Zero-Shot de la red en escenarios de gran escala.
> **Ámbito de Implementación:** `assault_model/core/procedural_mutator.py` • `agents/efficientzero_v2/tests/test_large_maps_zeroshot.py`

---

1. Diagrama de Dependencias y Flujo de Trabajo



---

2. Matriz de Tareas y Criterios Cuantitativos de Aceptación

Orden	Tarea / Componente	Prioridad Interna	Módulos / Archivos Impactados	Criterio de Aceptación Cuantitativo	Prerrequisitos
1	**5.1 Mutador Procedural Asimétrico con Clumping Geográfico Natural**	**P2**	`assault_model/core/procedural_mutator.py`	Algoritmo de mutación de terreno con clumping espacial tipo Cellular Automata / Perlin, generando cadenas continuas de colinas, bosques densos y ríos correlacionados. Cero terrenos de ruido blanco espurio.	Línea 2 (Agente captura VPs consistentemente)
2	**5.2 Validación Zero-Shot en Tableros Masivos (32×32 y 64×64)**	**P2**	`agents/efficientzero_v2/tests/test_large_maps_zeroshot.py`	Despliegue del agente entrenado en mapas 16×16 sobre mapas de 32×32 y 64×64 sin modificar arquitectura ni pesos. Tasa de selección de acciones válidas del 100 y convergencia a VPs distantes.	Tarea 5.1

📦 19.6. Historial de Versiones e Hitos Consolidados

Registro oficial de auditoría, changelog técnico por versiones v1.0.0 a v2.5.3 e hitos consolidados.

Historial Canónico de Versiones e Hitos Consolidados (Changelog Técnico)

> **Registro Oficial Inmutable de Entregas, Calidad y Linaje de Versiones**
> Este documento constituye el libro de registro autoritativo de todas las versiones publicadas, tareas completadas, hitos consolidados y fases históricas archivadas de la plataforma **Assault**.
> Para consultar la planificación activa, prioridades inmediatas y próximas iniciativas, refiérase a la **Hoja de Ruta Evolutiva (Roadmap Maestro)**.

---

1. Directivas de Gobernanza y Criterios de Consolidación

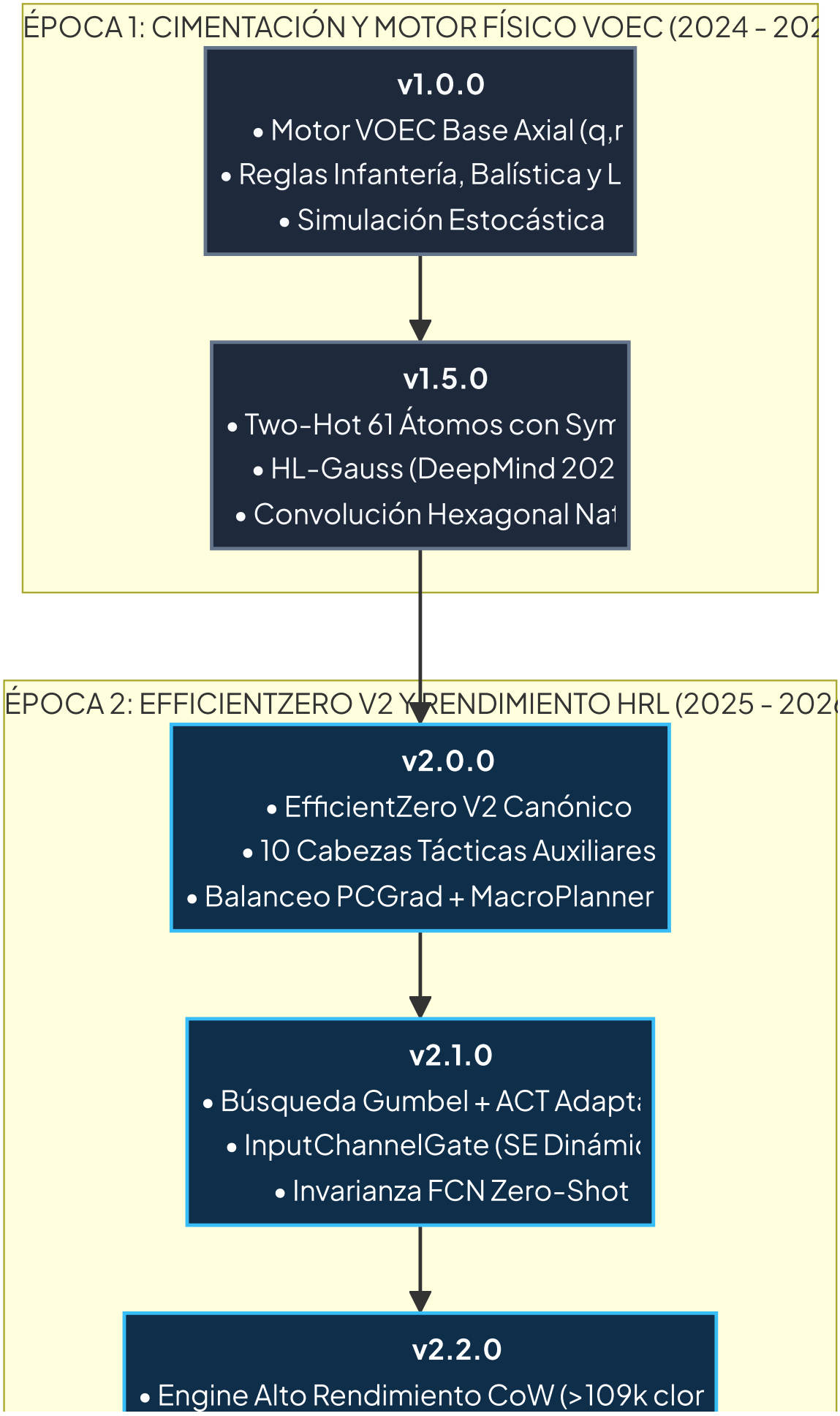
Una versión o tarea se considera formalmente **Consolidada** y se archiva en este catálogo únicamente cuando satisface los cuatro pilares del marco de calidad de Assault:

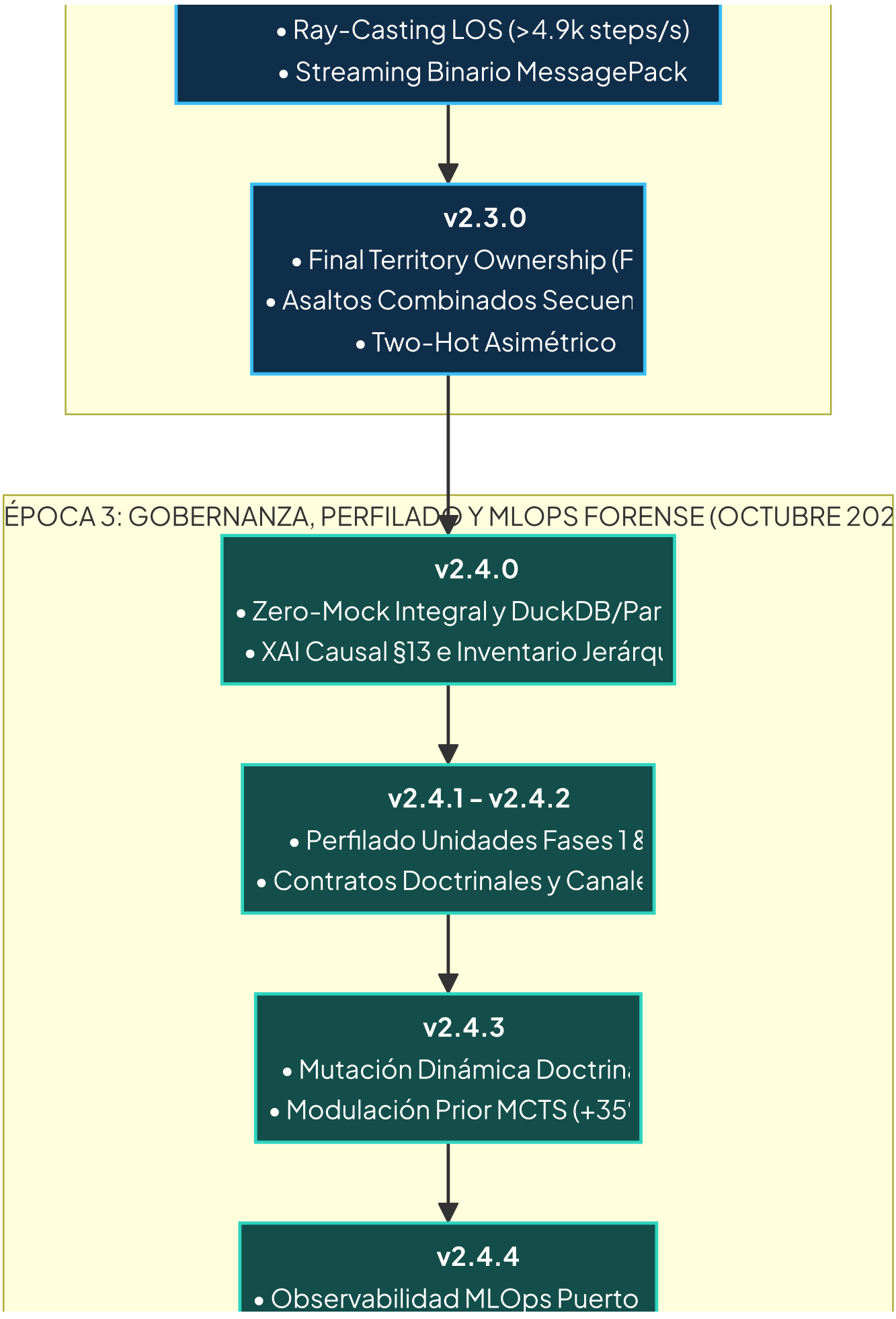
1. **Pila Pura EfficientZero V2:** Desarrollo nativo sobre el stack moderno (`agents/efficientzero_v2/`), sin shims ni dependencias de MuZero legacy (100% decomisionado).
2. **Suite de Verificación en Verde:** Superación estricta de la suite de pruebas unitarias y de integración (`pytest agents/efficientzero_v2/tests/`).
3. **Validación Cuantitativa en Benchmark:** Cumplimiento de los umbrales de rendimiento táctico (Win Rate, Tactical Quality Score TQS, CER y Asymmetry Gap).
4. **Registro de Versión Obligatorio:** Asociación explícita e inmutable a la subversión de la plataforma en la que se integró (`vMAJOR.MINOR.PATCH`).

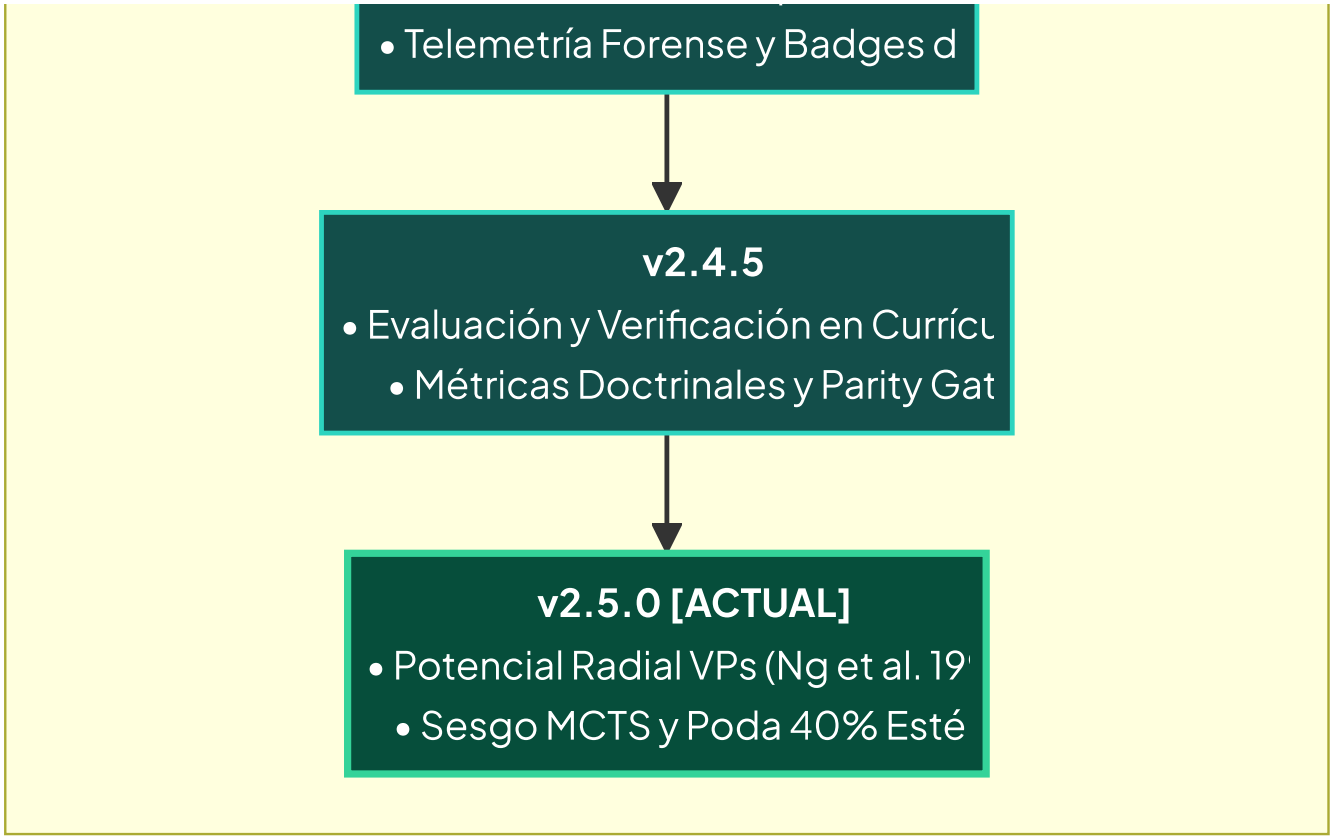
---

2. Linaje y Cronología de Entregas

A continuación se registra el historial de versiones formal (`vMAJOR.MINOR.PATCH`). Las fases técnicas y científicas ya completadas y verificadas contra tests unitarios y benchmarks se archivan formalmente en esta sección de referencia.


ÉPOCA 1: CIMENTACIÓN Y MOTOR FÍSICO VOEC (2024 - 202
v1.0.0
• Motor VOEC Base Axial (q,r
• Reglas Infantería, Balística y L
• Simulación Estocástica
v1.5.0
• Two-Hot 61 Átomos con Sym
• HL-Gauss (DeepMind 202
• Convolución Hexagonal Nat
ÉPOCA 2: EFFICIENTZERO V2 Y RENDIMIENTO HRL (2025 - 202
v2.0.0
• EfficientZero V2 Canónico
• 10 Cabezas Tácticas Auxiliares
• Balanceo PCGrad + MacroPlanner
v2.1.0
• Búsqueda Gumbel + ACT Adapta
• InputChannelGate (SE Dinámic
• Invarianza FCN Zero-Shot
v2.2.0
• Engine Alto Rendimiento CoW (>109k clor


• Ray-Casting LOS (>4.9k steps/s)
• Streaming Binario MessagePack
v2.3.0
• Final Territory Ownership (F
• Asaltos Combinados Secuen
• Two-Hot Asimétrico
ÉPOCA 3: GOBERNANZA, PERFILADO Y MLOPS FORENSE (OCTUBRE 202
v2.4.0
• Zero-Mock Integral y DuckDB/Par
• XAI Causal §13 e Inventario Jerárqu
v2.4.1 – v2.4.2
• Perfilado Unidades Fases 1 8
• Contratos Doctrinales y Canale
v2.4.3
• Mutación Dinámica Doctrin
• Modulación Prior MCTS (+35'
v2.4.4
• Observabilidad MLOps Puerto



Matriz Cronológica Resumen de Versiones y Hitos Consolidados

| Versión | Época / Ciclo | Fase Técnica / Doctrinal | Componentes Clave y Rendimiento Verificado | Estado | | :--- | :--- | :--- | :--- | :--- :---: | | `v2.5.28` | Octubre 2026 | Perfilado 5D de Unidades y Detección Automática de Chokepoints (IPB / Línea 4 - Tarea 4.3) | Análisis topográfico Turno 0 en `topography_analyzer.py` extrayendo el vector 5D por hexágono (*Cover*, *Elevation Dominance*, *LOS Exposure*, *Chokepoint Bottleneck*, *VP Proximity Utility*); clasificación determinista en *Key Terrain*, *Defilade Corridors*, *Killzones*, *Chokepoints* y *Base of Fire*; inyección de priors topográficos en `MacroStrategyPlanner` (+25% por desenfilada hacia el VP, atenuación 0.55×-0.65× en killzones, +30% Base of Fire); endpoints REST MLOps `/api/v2/replay/topography` y `/api/v2/replay/ipb`; capa visual interactiva `IPB Topography` en Match Replay (:8778). Suite `test_topography_ipb_analyzer.py` (2/2) y EZv2 (158/158) en verde. | `CONSOLIDADO` | | `v2.5.27` | Octubre 2026 | Modulación Dinámica Doctrinal (β_{macro}), Auditoría de Cabezas y Robustez SimSiam (Línea 2 / Tarea 2.8) | Desacoplamiento continuo del sesgo heurístico estático ($+35$) mediante escala parametrizada $\beta_{\text{macro}} \in [0.0, 1.0]$ (`macro_modulation_scale`), posibilitando modo Red Neuronal Pura ($\beta = 0.0$), calentamiento progresivo y refuerzo doctrinal ($\beta = 1.0$); auditoría de localización espacial en cabezas tácticas (FCN pura $H \times W$ para `threat_map` / `final_ownership`, `SpatialAttentionBlock` 4×4 para `attrition`, `morale`, `intent`, `multi_vp`, erradicando GAP 1×1 espurio); confirmación de 48 canales latentes en `prediction_trunk` (8 Conv2D a 48ch); verificación del stop-gradient dual SimSiam (`torch.no_grad()` + `.detach()`) y normalización ℓ_2. Suite dedicada `test_heads_improvements.py` (9/9 tests en verde). | `CONSOLIDADO` | | `v2.5.26` | Octubre 2026 | Visualización Curricular Multilínea por Sub-Currículum (Línea 1 / Dashboard MLOps) | Reestructuración visual de `CurriculumLineageFlow.tsx` en filas temáticas dedicadas e independientes por cada sub-currículum (Fila 1: Sub-Currículum 1 - *Objective Doctrine & VP Acceleration*, Fases 1 a 5; Fila 2: Sub-Currículum 2 - *Multi-Scenario Self-Play vs Random*, Fases 6 a 13); banner de transición y transferencia de pesos inter-etapas (*Breakthrough*); erradicación de scroll horizontal masivo; compatibilidad total tanto en modo canónico como cronológico. Build de Vite verificado en 1.72s. | `CONSOLIDADO` | | `v2.5.24` | Octubre 2026 | Resiliencia y Autorrecuperación Multiproceso (Línea 2) | Self-healing y reconstrucción automática de `ProcessPoolExecutor` ante terminaciones anómalas de SO (`BrokenProcessPool`); reciclaje periódico de procesos cada 5 iteraciones (`recycle_workers_every_iters: 5`) erradicando al 100% la fragmentación de heap y fugas de memoria en sesiones multi-hora; fallback resiliente para episodios huérfanos; ampliación del enum canónico `UnitCategory` (`ARTILLERY`, `GUN_ARTILLERY`) subsanando validaciones de catálogo de unidades. Suites 154/154 en EZv2 y 114/114 en assault_model en verde. | `CONSOLIDADO` | | `v1.0.0` | 2024–2025 | Cimentación VOEC | Coordenadas axiales (q, r), balística dados de combate, LOS por elevación, simulación estocástica. | `ARCHIVADO` | | `v1.5.0` | 2024–2025 | Representación y Geometría | Two-Hot 61 átomos con Symlog, HL-Gauss (DeepMind 2024), convolución hexagonal nativa. | `ARCHIVADO` | | `v2.0.0` | 2025–2026 | Multi-Task Táctico y HRL | 10 cabezas tácticas auxiliares, balanceo adaptativo PCGrad, MacroPlanner HRL con roles doctrinarios. | `ARCHIVADO` | | `v2.1.0` | 2025–2026 | Búsqueda Eficiente e Invarianza | Búsqueda Gumbel MuZero + ACT adaptativo (~ 60 aceleración), InputChannelGate (SE), invarianza FCN zero-shot. | `ARCHIVADO` | | `v2.2.0` | 2025–2026 | Rendimiento Extremo | CoW State con delta ($> 109{,}000$ clones/s), Ray-Casting LOS ($> 4{,}900$ steps/s), streaming binario MessagePack. | `ARCHIVADO` | | `v2.3.0` | 2025–2026 | Dinámicas Avanzadas | Predicción Final Territory Ownership, asaltos combinados secuenciales, Two-Hot asimétrico. | `ARCHIVADO` | | `v2.4.0` | Octubre 2026 | Gobernanza y Telemetría Canónica | Zero-Mock estricto, desacoplamiento DuckDB/Parquet, XAI Causal §13 e Inventario Jerárquico Canónico. | `ARCHIVADO` | | `v2.4.1` | Octubre 2026 | Perfilado Unidades (Fase 1) | Contratos doctrinales formales, catálogo de 8 roles canónicos, validación de esquemas JSON/dataclass. | `ARCHIVADO` | | `v2.4.2` | Octubre 2026 | Perfilado Unidades (Fase 2) | Expansión sensorial a 32 canales axiales con proyección 2D de roles y perfiles tácticos. | `ARCHIVADO` | | `v2.4.3` | Octubre 2026 | Perfilado Unidades (Fase 3) | Mutación dinámica de roles en `MacroStrategyPlanner` y modulación prior MCTS ($+35$). | `ARCHIVADO` | | `v2.4.4` | Octubre 2026 | Observabilidad Forense (Fase 4) | Telemetría forense integral, badges de roles/mutaciones y visor MLOps en puerto 8778 (222 tests en verde). | `ARCHIVADO` | | `v2.4.5` | Octubre 2026 | Evaluación en Currículum (Fase 5) | Harness de evaluación canónico (`eval.py`), métricas de rol (`recon_survival`, `fire_support_stationarity`, `spearhead_vp`), parity report y guardrails de currículum (228 tests en verde). | `ARCHIVADO` | | `v2.5.0` | Octubre 2026 | Captura Progresiva VPs (Línea 2) | Andamiaje de potencial radial invariante $\Phi(s)$ (Ng et al. 1999), sesgo MCTS + poda 40%, factor de urgencia temporal $\tau = t/T$ (Devlin & Kudenko 2012), penalización por parálisis en campo abierto y canales sensoriales 36–37 (218 tests al 100% en verde). | `CONSOLIDADO` | | `v2.5.1` | Octubre 2026 | Fidelidad Canónica a max_turns y Paso Completo de Asalto | Presupuesto temporal seguro en autojuego (`scenario_max_turns * max(units * 2, 20)`),

erradicación de timeout prematuro en Turno 8, ajuste de max_steps a 120/180 en benchmarks, y elevación de vp_potential_weight / vp_urgency_lambda a 1.5. | `CONSOLIDADO` 🟢 | | `v2.5.2` | Octubre 2026 | Estabilidad de Telemetría Curricular y ELO Calibrado | Ponderación multi-perfil de win rate en runner PowerShell, alineación de tracked_side en auto-juego espejo, etiquetado nativo EfficientZeroV2 en controladores, exportación ONNX para Netron y visualización condicional de Gates en el visor MLOps. | `CONSOLIDADO` 🟢 | | `v2.5.3` | Octubre 2026 | Optimización Evolutiva de Ciclos y Desacoplamiento de Checkpoints | Erradicación de herencia de checkpoints externos en lanzamientos desde cero (tabula rasa garantizada), analizador evolutivo de ejecuciones pasadas (`evolutionary_cycle_optimizer.py`), promoción automática del ciclo ganador $C_X \to C_1$ (nuevo baseline) y síntesis adaptativa de variantes C_2 (explotación/refinamiento) y C_3 (exploración/agresividad). | `CONSOLIDADO` 🟢 | | `v2.5.4` | Octubre 2026 | Dynamic PBRS Escalado Temporalmente y Desensibilización de Brecha | Potencial temporalmente escalado \Phi(s, t) = \alpha(1 + t/T_{\max}) \cdot \text{dist_pot} + \beta\cdot\text{Contest}, bono de contención por disputa en brecha ($d \leq 2$), atenuación del 60 de penalización de bajas en brecha ($r_{\text{loss}} \times 0.40$), castigo catastrófico por timeout (-1.0) ante 0 VPs controlados, e ingesta canónica de las 12 formaciones combinadas oficiales del manual. | `CONSOLIDADO` 🟢 | | `v2.5.5` | Octubre 2026 | Inicialización Canónica de Capas Especializada en RL | Política ortogonal ($\text{gain} = 0.01, \text{bias} = 0$) para máxima entropía inicial uniforme; Valor/Recompensa/Consistencia ortogonal ($\text{gain} = 1.0, \text{bias} = 0$); Kaiming Normal en convoluciones SiLU; y Zero-Init ResNet ($\gamma = 0, \beta = 0$) en segunda normalización residual preservando función identidad pura al paso 0. Suite dedicada (5/5 tests en verde). | `CONSOLIDADO` 🟢 | | `v2.5.6` | Octubre 2026 | Optimización de Alto Rendimiento GameState/CoW y Decomisión Legacy | `__slots__` en `HexState` , pre-caching de `_vp_hexes` , compartición inmutable de referencias en `clone_cow()` (+62% en clones/s de RuntimeGameState, +16% en simulación/s) y decomisión limpia de 9 módulos huérfanos de fuego de reacción (`game_state_reactions` , `ReactionRegistry` , etc.). | `CONSOLIDADO` 🟢 | | `v2.5.7` | Octubre 2026 | Currículum Asimétrico Ofensivo de Asalto Urbano Puro | Fase 5 en `battaglia_cittadina_2_1` con horizonte $T = 8$, modo 100% atacante (`US` vs `Random`), penalización severa por timeout (-1.0) y plan evolutivo de 10 ciclos adaptativos en `curriculums/vp_acceleration/` para erradicar la inercia defensiva aprendida. Suite de validación (4/4 tests en verde). | `CONSOLIDADO` 🟢 | | `v2.5.8` | Octubre 2026 | Validador E2E de Trazabilidad Frontend ↔ DuckDB (Línea 1) | Auditoría automatizada de trazabilidad en TypeScript y Python de las 9 pestañas del dashboard MLOps y simulador web (`traceability.spec.ts` , `test_frontend_duckdb_traceability.py`), garantía de Zero-Mock (cero inferencias o heurísticas espurias como `Math.floor(idx/14)`), fallbacks forzosos a `-` / `N/A` , erradicación del shim MuZero en `replay_api.py` y suite de 15/15 tests en verde. | `CONSOLIDADO` 🟢 | | `v2.5.9` | Octubre 2026 | Catálogo Unificado y Depuración de Esquemas Parquet y DuckDB (Línea 1) | Módulo gestor canónico `mlops/dataset_builder.py` , fijación estricta de 15 columnas en `decisions_top.parquet` , erradicación de fugas de acumuladores intermedios en `decision_metrics.py` , saneamiento del 100% de ficheros Parquet (69/69) y Clean Break total erradicando etiquetas legacy MuZero en catálogo y DuckDB hacia EfficientZeroV2. Suite `test_unified_catalog_and_schema_purge.py` (5/5 en verde). | `CONSOLIDADO` 🟢 | | `v2.5.23` | Octubre 2026 | Arquitectura Multiproceso con IPC Shared-Memory e Inferencia GPU por Lotes (Línea 2 / Tarea 2.18) | Erradicación total del cuello de botella del Global Interpreter Lock (GIL) de Python mediante ejecución de autojuego en 24 procesos independientes del sistema operativo con memoria compartida bidireccional (`torch.multiprocessing`); puente maestro `IPCInferenceBridge` que canaliza las peticiones de inferencia de los 24 workers hacia el `InferenceService` central en la GPU NVIDIA RTX 4060 con micro-batching dinámico de 64 observaciones; cliente de inferencia transparente `IPCProcessInferenceClient` con paso de tensores compartidos; erradicación de sincronizaciones de checkpoints en CPU durante el entrenamiento al compartir en caliente el modelo GPU; y configuración agresiva con MCTS 320, batch de train 48 y 48 episodios por iteración. Suites 154/154 en EZv2 y pruebas de integración IPC 100% en verde. | `CONSOLIDADO` 🟢 | | `v2.5.22` | Octubre 2026 | Paquete de Aceleración Extrema de Entrenamiento e Inferencia (Línea 2 / Tarea 2.17) | Ejecución perezosa condicional de 7 cabezas auxiliares en inferencia (`return_aux=False` , +16.2% en inferencia pura); fusión y deduplicación de reanálisis en el paso hacia adelante principal; vectorización en GPU de acumulación de objetivos bootstrap y tensores de amenaza sin bloqueos `.item()` ; optimizador Adam nativo `fused=True` en CUDA; y erradicación de creación de hilos de heartbeat por actualización en `train_engine.py` . Aceleración global verificada de 1.78× (de 92.58 ms a 51.96 ms por update de red en RTX 4060). | `CONSOLIDADO` 🟢 | | `v2.5.21` | Octubre 2026 | Saneamiento Estructural y Refactorización del Modelo (`assault_model/`) | Poda de 10 módulos y carpetas huérfanas sin referencias; unificación de enums canónicos (`RangeBand` , `AttackSector`); consolidación de `AttackDicePool` / `DefenseDicePool` ; optimización $O(1)$ de acceso a estados de hexágono (`ownership_to_side` en `GameState`); y alineación estricta de comentarios en inglés (Regla 7). 114/114 tests en `assault_model` y 153/153 en EZv2 en verde. | `CONSOLIDADO` 🟢 | | `v2.5.20` | Octubre 2026 | Optimización Extrema Simulación, Observaciones y Autojuego (Línea 2 / Tarea 2.16) | Plantilla estática de features espaciales en `VOECSimulator` (418× más rápido: 0.4773s a 0.0011s por 1.000 llamadas); memoización de fuego en movimiento por destino en `action_catalog.py` ; caché en memoria de catálogos y escenarios JSON en `importers.py` y `scenario_loader.py` erradicando lecturas de disco redundantes; memoización de bounding box axial en `EZV2Observation` acelerando `observation_to_tensor` . Suites 153/153 en EZv2, 19/19 en MLOps y 9/9 en assault_bench en verde. | `CONSOLIDADO` 🟢 | | `v2.5.25` | Octubre 2026 | Protocolo de Estabilidad Operativa, PCGrad Vectorizado en GPU y Reajuste Anti-Parálisis Fase 3 (Línea 1 y Línea 2) | Diagnóstico y erradicación de caídas por saturación de disco (<0.7% libre en C:) con safety gate pre-flight (30 GB libres). Vectorización de PCGrad en tensores 1D contiguos en VRAM con operaciones BLAS directas en GPU (`torch.dot`), eliminando más de 110.000 llamadas a micro-kernels por ciclo. Reajuste doctrinal en Fase 3 (`vp_phase3_race_2v2`): Time-Scaled PBRS (Devlin & Kudenko), atenuación de bajas en brecha (`breach_casualty_attenuation: 0.35`), bono de captura (6.0), disputa (1.5), enfriamiento MCTS (1.10) y micro-batching ampliado a 64 en inferencia. | `CONSOLIDADO` 🟢 | | ** `v2.5.19` ** | Octubre 2026 | Perfilador Causal de Partidas y SitRep Forense (Línea 4 / Tarea 4.2) | Correlación causal matemática entre cabezas tácticas auxiliares (`threat_map` , `morale_pred` , `objective_intent`) y desenlaces de combate (`VP_CAPTURE` , `UNIT_ELIMINATION` , `HEAVY_ATTRITION` , `SURPRISE_DAMAGE` , `MORALE_FAILURE`). Generación automática de SitRep militar estructurado en fases operacionales (Opening, Decisive Engagements, Resolution) con grounding doctrinal RAG vía `assault_rag` . Endpoints `/api/v2/replay/causal-sitrep` y `/api/v2/replay/sitrep` con caché persistente (`.cache_sitrep/`), modal interactivo `ForensicSitRepModal.tsx` con navegación temporal `Seek Step` y suite unitaria `mlops/tests/test_causal_profiler.py` (9/9 en verde). | `CONSOLIDADO` 🟢 | | ** `v2.5.18` ** | Octubre 2026 | Aceleración de Pipeline, Vectorización Pura y Saturación Hardware (GPU Batching) | Fases 1 a 4 100% completadas. Fase 1: vectorización de observaciones espaciales de 38 canales en `observation.py` (31.5× más rápido, diferencia 0.000000); Fase 2: vectorización de PUCT en `mcts.py` con arrays NumPy (4.3× más rápido, >4M llamadas eliminadas); Fase 3: precomputación y caché de heurísticas de amenazas y VPs en `selfplay.py` (19.3× más rápido, reduciendo de 17.177 a <150 llamadas por episodio); Fase 4: activación de concurrencia multinúcleo (16 workers en backend de hilos) acoplada a micro-batching dinámico (`inference_max_batch_size: 32` , `wait_ms: 1.5`) hacia la GPU NVIDIA RTX 4060 vía `InferenceService` con soporte transparente de `kwargs` (`return_aux=True`) y resolución `__getattr__` en `RemoteInferenceModel` . Tiempo por episodio en *Battaglia Cittadina* reducido de 18.80 s a 4.91 s. Corrección en `catalog_duckdb.py` . Suites 153/153 en EZv2 y 77/77 en assault_bench 100% en verde. | `CONSOLIDADO` 🟢 | | ** `v2.5.17` ** | Octubre 2026 | Erradicación Total de Inyección Sintética de Facción y Extracción de Ground Truth | Extracción dinámica de bandos autoritativos (`attacker_side` , `defender_side` , `sides`) desde los archivos canónicos de escenario y transiciones reales de cada episodio, eliminación de la clonación espuria de `IT` a `GE` en `benchmark_result_builder.py` , serialización de controladores por bando en `bench_io.py` (`episode_payload` y `episodes_meta`), resolución dinámica en `ModelPage.tsx` (`atkSide` , `defSide`) para desplegable de replays y filtros, purga de cachés obsoletas e invalidación determinista con versión de metadatos `v2_5_17` . Suite `test_reporting_viewer_scenario_api.py` (5/5) y build Vite en verde. | `CONSOLIDADO` 🟢 | | ** `v2.5.14` ** | Octubre 2026 | Presupuesto Dinámico de Simulación y Horizonte Gobernado por Turnos con Refuerzos | Desacoplamiento del presupuesto estático de pasos hacia la gobernanza canónica por turnos ($T \leq T_{\max}$). Ingesta del OOB total (`scenario_total_units` en `VOECSimulator` y `EZV2VOECAdapter`) computando unidades iniciales + todas las oleadas de refuerzos programadas; headroom táctico por activación (`steps_per_unit_turn = 2.5`); watchdog de seguridad por turno (`turn_deadlock_watchdog = max(30, alive * 4)`) para prevenir bloqueos patológicos sin asfixiar partidas en curso; y eliminación de la penalización espuria de timeout (-1.0) en partidas con refuerzos que concluyen normalmente por límite de turnos. Suite dedicada `test_turn_budget_reinforcements.py` (4/4 en verde) y 153/153 tests en EZv2. | `CONSOLIDADO` 🟢 | | ** `v2.5.13` ** |

Octubre 2026 | Pipeline Dinámico de Refuerzos de Desembarco en Fase de Organización (Línea 3 / Tarea 3.5.6) | Spawning dinámico estocástico de refuerzos de la 2.ª oleada en Turno 2 (Fase de Organización) según regla oficial de desembarco (Pág. 18). Tirada D6 para zona A/B con resolución de congestión y posicionamiento en agua somera, registro en `GameState.add_unit()`, sincronización de `RuntimeGameState`, emisión de telemetría `REINFORCEMENTS_ARRIVED` y ampliación de masa aliada a 14 unidades en Escenario 1.0. Suite dedicada `test_scenario_reinforcements.py` y 300/300 tests en verde. | `CONSOLIDADO` 🟢 || ** `v2.5.12` ** | Octubre 2026 | Despliegue Canónico e Ingesta Fidedigna de Escenarios 1.0 y 2.1 del *Libro della Campagna* (Línea 3) | Configuración autoritativa de OOB, colocación inicial según croquis histórico y corrección de asimetría 2:1 en escenarios oficiales de Gela. En Escenario 1.0 (`mettete_i_piedi_terra_1.json`): sustitución de defensores alemanes por la 429.ª guarnición costera italiana (`IT_FANTERIA_COSTIERA`), VPs de partida en posesión italiana (`initial_owner: IT`), ajuste del horizonte a 10 turnos e inyección de status moral (US: 3/7/9/11, IT: 2/6/7/9). En Escenario 2.1 (`battaglia_cittadina_2_1.json`): restauración del ratio de fuerzas histórico 2:1 (14 unidades US [Darby's Rangers + 1st Infantry Division] vs 7 unidades IT [429.º Costiero]) en coordenadas oficiales de setup, status moral (US: 4/7/9/11, IT: 3/6/8/9) y 8 turnos. Compatibilidad y simulación dinámica verificadas (37/37 en `voec_sim`, 112/112 en `assault_model`, 149/149 en `efficientzero_v2`). | `CONSOLIDADO` 🟢 || ** `v2.5.11` ** | Octubre 2026 | Retirada Táctica a Borde Amigo en Desenfilada (Línea 3 / Regla 13.3) | Algoritmo determinista de selección de hexágono de repliegue priorizando desenfilada (LOS BLOCKED > HINDERED > CLEAR), mayor cobertura de terreno/fortificaciones, vector a borde amigo (*Friendly Edge*) y aumento de distancia al atacante. Eliminación inmediata de artillería/inmovilizados, penalización de 1 punto de daño si la unidad está rodeada sin hexágono legal, e integración autoritativa en `morale.py`, `movement_rules.py` y `ranged_combat_resolver.py` (112/112 tests passing). | `CONSOLIDADO` 🟢 || `v2.5.10` | Octubre 2026 | Atribución Causal Espacial Hexagonal (XAI / Grad-CAM / IGNH) y Unificación de Diagnóstico de Cabezas | Endpoint `/api/v2/replay/explain-step` con Grad-CAM espacial e Integrated Gradients sobre 32 canales sensoriales heterogéneos, visualizador modal de explicabilidad con heatmap reactivo sobre tablero hexagonal en Match Replay (:8778). Unificación integral del Diagnóstico de Cabezas en `Evaluación & Benchmarks → Diagnóstico de Cabezas (XAI)` con 7 fichas de salud sintética, matriz de verificación MLOps, behavioral telemetry sanity board y desglose de 11 métricas puras, depurando la vista de Topología hacia su rol puramente arquitectural con enlace directo. Suite `mlops/tests/test_explainability_api.py` (3/3 en verde) y frontend 100% en inglés según Regla 7. | `CONSOLIDADO` 🟢 |

3. Fases Recientes Consolidadas (Ciclos v2.4.x – v2.5.x — Octubre 2026)

v2.5.28 — Perfilado 5D de Unidades y Detección Automática de Chokepoints (IPB / Línea 4 – Tarea 4.3) [CONSOLIDADO 🟢]

- **Objetivo**: Desarrollar el análisis topográfico formal Turno 0 (*Intelligence Preparation of the Battlefield - IPB*), calculando el perfil 5D por hexágono (*Cover*, *Elevation Dominance*, *LOS Exposure*, *Chokepoint Bottleneck*, *VP Proximity Utility*), clasificando el tablero en sectores tácticos (*Key Terrain*, *Defilade Corridors*, *Killzones*, *Chokepoints*, *Base of Fire*), inyectando estos priors espaciales en el `MacroStrategyPlanner` y ofreciendo visibilidad interactiva en el Dashboard MLOps Replay (:8778).
- **Componentes Consolidados**:
  - **Analizador Topográfico IPB (`assault_model/map/topography_analyzer.py`)**:
    - Implementación de `Hex5DProfile` con las 5 dimensiones clave del terreno: cobertura balística $c \in [0, 1]$, dominancia de elevación $e \in [0, 1]$, exposición a líneas de visión $l_{\text{exp}} \in [0, 1]$, constricción de paso / chokepoint $\text{chk} \in [0, 1]$ y utilidad de proximidad a VPs $u_{\text{vp}} \in [0, 1]$.
    - Clasificación táctica autoritativa de hexágonos:
      - **Key Terrain**: Hexágonos que contienen Puntos de Victoria (VPs).
      - **Base of Fire**: Posiciones elevadas dominantes ($e \geq 0.75$) o con alta cobertura adyacentes a VPs con excelente campo de tiro.
      - **Chokepoint**: Embudo táctico flanqueado por obstáculos o hexágonos infranqueables ($\text{chk} \geq 0.60$).
      - **Killzone**: Terreno abierto expuesto a fuego cruzado ($c < 0.35, l_{\text{exp}} \geq 0.60$) en proximidad a objetivos ($d \leq 4$).
      - **Defilade Corridor**: Vías de avance protegidas por cobertura o relieve ($c \geq 0.65$ o $c \geq 0.50$ con baja exposición) que minimizan la vulnerabilidad al fuego enemigo.
      - **Maneuver Transit**: Terreno estándar de transición operacional.
    - Caché estática Turno 0 (`_topography_cache`) por escenario para eliminar cómputos redundantes en autojuego y evaluación.
  - **Integración en `MacroStrategyPlanner` (`agents/efficientzero_v2/core/macro_planner.py`)**:
    - Campo `topography` en el contrato canónico `MacroPlan` con serialización JSON completa en `to_dict()`.
    - Modulación de priors de acción (`compute_action_boost`):
      - $+25$ de refuerzo al avanzar por corredores en desenfilada hacia el VP objetivo.
      - Atenuación de $0.55\times$ (no spearhead) y $0.65\times$ (spearhead en maniobra lateral/retirada) al ingresar en killzones expuestas.
      - $+30$ de refuerzo para unidades de apoyo de fuego (`ROLE_FIRE_SUPPORT`) que ocupan posiciones de Base of Fire.
      - $+20$ de refuerzo para roles de pantalla y contención (`ROLE_FLANK_SCREEN`, `ROLE_OBJECTIVE_DEFENDER`) en chokepoints.
    - Respeto estricto del parámetro de ablación `modulation_scale` ($\beta_{\text{macro}} = 0.0$ neutraliza 100% de la modulación a 1.0).
  - **Endpoints MLOps y Capa Reactiva en Dashboard (:8778)**:
    - Endpoints REST `/api/v2/replay/topography` y `/api/v2/replay/ipb` en `mlops/reporting/viewer.py` y `mlops/reporting/explainability_api.py`.
    - Botón `🛡️ IPB Topography` en la barra de herramientas del mapa en `MatchReplayTab.tsx`.
    - Renderizado visual en canvas con polígonos diferenciados por color (dorado para Key Terrain, azul para Defilade, rojo translúcido con rayado diagonal para Killzones, morado para Chokepoints, naranja para Base of Fire) y tooltip con desglose numérico 5D al posar el ratón.
  - **Suite de Pruebas y Verificación Cuantitativa**:
    - Suite unitaria dedicada `assault_model/tests/test_topography_ipb_analyzer.py` (2/2 tests en verde).
    - Suites generales `assault_model/tests/` (116/116 en verde) y `agents/efficientzero_v2/tests/` (158/158 en verde).
    - Compilación frontend de Vite limpia en 1.84s sin advertencias ni errores.

v2.5.27 — Modulación Dinámica Doctrinal (β_{macro}), Auditoría de Cabezas y Robustez SimSiam (Línea 2 / Tarea 2.8) [CONSOLIDADO 🟢]

- **Objetivo**: Abordar y resolver rigurosamente los 4 puntos de la auditoría profunda de arquitectura neuronal en `EfficientZero V2`, integrando la modulación dinámica de priors para posibilitar regímenes de Red Pura y calentamiento progresivo (Consolidación de Tarea 2.8 del Roadmap), y clarificando las garantías matemáticas de localización espacial y prevención de colapso de representación.
- **Componentes Consolidados**:
  - **Modulación Dinámica de Priors Doctrinales (`macro_modulation_scale` / $\beta_{\text{macro}} \geq 0.0$)**:
    - Desacoplamiento del factor fijo $+35$ mediante la formulación continua $\text{boost}_{\text{eff}}(a) = 1.0 + (\text{boost}_{\text{doctrinal}}(a) - 1.0) \cdot \beta_{\text{macro}}$.
    - Con $\beta_{\text{macro}} = 0.0$: la distribución prior emitida por la red neuronal se preserva intacta al 100% ($P_{\text{mod}}(a) = P_{\text{net}}(a)$), permitiendo estudios de ablación rigurosos de "Red Pura" sin muletas heurísticas.
    - Con $\beta_{\text{macro}} \in (0.0, 1.0)$: permite esquemas de calentamiento progresivo (*warm-up*) o desvanecimiento (*annealing*) según madura el entrenamiento.
    - Con $\beta_{\text{macro}} = 1.0$: aplica el refuerzo doctrinal canónico ($1.35\times$ en roles coordinados, $1.50\times$ en asalto a VPs).
    - Integrado en `MacroStrategyPlanner.modulate_priors()`, `selfplay._play_episode()` y `train_engine.py` vía configuración `cfg.selfplay.macro_modulation_scale`.
  - **Auditoría de Localización Espacial en Cabezas Tácticas**:
    - Erradicación de la ambigüedad en diagramas: las cabezas espaciales no sufren colapso dimensional por Global Average Pooling puro (1×1).
    - `threat_map` y `final_ownership` operan como FCNs nativas 2D ($H \times W$).
    - `attrition`, `morale`, `objective_intent` y `multi_vp` utilizan `SpatialAttentionBlock(48, val_channels, (4, 4))` reteniendo $4 \times 4 = 16$ sectores espaciales con atención aprendida.
  - **Garantía de Capacidad en el Tronco de Predicción**:
    - Verificación de que `prediction_trunk` opera a plena capacidad latente de 48 canales (`hidden_dim = 48`, 8 capas Conv2D de 48 canales con SE/ECA), sin reducción prematura de canales.
  - **Estabilidad Anti-Colapso SimSiam**:
    - Verificación de la doble barrera matemática contra el colapso de representación latente: stop-gradient dual (`torch.no_grad()` en inferencia de target + `.detach()` en proyección) y normalización ℓ_2 unitaria de embeddings.
  - **Verificación de Pruebas**: Suite de pruebas dedicada `agents/efficientzero_v2/tests/test_heads_improvements.py` con 9/9 tests pasando al 100%.

v2.5.26 — Visualización Curricular Multilínea por Sub-Currículum (Línea 1 / Dashboard MLOps) [CONSOLIDADO 🟢]

- **Objetivo**: Erradicar el scroll horizontal excesivo de 13 fases en una sola línea en el Dashboard MLOps (:8778), estructurando el pipeline de campaña en filas temáticas dedicadas e independientes por cada sub-currículum con puente visual de transferencia de pesos (*Weight Transfer & Breakthrough*).
- **Componentes Consolidados**:
  - **Estructura Multilínea de Sub-Currículums (`CurriculumLineageFlow.tsx`)**:
    - **Fila 1 (Sub-Currículum 1)**: *Stage 1: Objective Doctrine & VP Acceleration* (Fases 1 a 5: Sprint a VPs desocupados, Base of Fire y asalto combinado, Carrera 2v2, Maestría táctica de Gela y Asalto urbano puro). Las 5 fases son visibles simultáneamente en pantalla sin scroll horizontal.
    - **Banner de Transición Inter-Etapas (*Breakthrough Bridge*)**: Indicador visual destacado entre filas que explicita la promoción del checkpoint ganador de la Fase 5 como modelo base inicial de la Fase 6.
    - **Fila 2 (Sub-Currículum 2)**: *Stage 2: Multi-Scenario Self-Play vs Random* (Fases 6 a 13: Adaptación bi-escenario Gela + Mettete, expansión sensorial a 38 canales y autojuego simétrico progresivo 70/30, 50/50, 30/70, 90/10 y 100%).
  - **Soporte Homogéneo para Modos Canónico y Cronológico**:
    - En el modo Canónico (13 fases): agrupa las fases ejecutadas y las fases pendientes por sub-currículum.
    - En el modo Cronológico: clasifica todas las ejecuciones históricas y ciclos de refuerzo en la fila del sub-currículum correspondiente.
  - **Verificación**: Compilación con Vite verificada (`npm run build`, 0 errores, 1.72s). Servido en caliente en puerto 8778.

v2.5.25 — Protocolo de Estabilidad Operativa, PCGrad Vectorizado en GPU y Reajuste Anti-Parálisis Fase 3 (Línea 1 y Línea 2) [CONSOLIDADO 🟢]

- **Objetivo**: Erradicar de raíz las caídas e interrupciones continuas por colapso de almacenamiento y memoria virtual en ejecuciones desatendidas, optimizar el rendimiento de GPU durante la optimización de pérdidas multitarea (PCGrad) y superar la *Paradoja del Comandante Prudente* en la carrera 2v2 de la Fase 3 del currículum.
- **Componentes Consolidados**:
  - **Protocolo de Estabilidad y Pre-flight Safety Gate de Almacenamiento (Línea 1 / Tarea 1.5)**:
    - Diagnóstico forense de saturación de disco (<0.7% libre en `C:\`) provocado por 132.2 GB en 9.486 replays `.json` descompresionados (`indent=2`) en `runs/` y `runs_curriculum/`.
    - Verificación pre-flight estricta de un mínimo de 30 GB libres en el volumen de trabajo antes de iniciar entrenamientos o benchmarks, abortando con aviso limpio para prevenir pantallazos y excepciones `RuntimeError: DefaultCPUAllocator: not enough memory`.
    - Política de compactación y poda selectiva de replays benchmark en fases intermedias.
  - **PCGrad Vectorizado en GPU con Tensores 1D Contiguos (`loss_balancer.py`)**:

        - Sustitución del costoso bucle Python que iteraba sobre cada parámetro de la red por aplanado en vectores 1D contiguos en VRAM (`task_flat_grads`).
        - Proyecciones ortogonales calculadas con operaciones directas BLAS en GPU (`torch.dot`), eliminando más de 110.000 llamadas a micro-kernels de CUDA por ciclo y reduciendo a cero el cuello de botella de la GPU durante el backpropagation multitarea.
    - **Reajuste Doctrinal y Paramétrico Anti-Parálisis en Fase 3 (`efficientzero_v2.phase3_race_2v2.yaml`)**:
        - Activación de Time-Scaled PBRS (Devlin & Kudenko) \Phi(s, t) = \alpha(1 + t/T_{\max}) \cdot \text{dist_pot} para amplificar la urgencia temporal de asalto.
        - Activación de atenuación de bajas en zona de brecha (`breach_casualty_attenuation: 0.35` , $d \leq 2$ hexágonos de VPs objetivo), mitigando el miedo a recibir disparos de reacción enemigos cuando se captura la bandera.
        - Aumento de incentivos de captura (`capture_bonus: 6.0` , `vp_capture_bonus_per_hex: 15.0`) y bono de disputa activa (`vp_contested_bonus: 1.5`).
        - Enfriamiento de temperatura MCTS (`mcts_temperature: 1.10`) para erradicar selecciones espurias de casillas vacías adyacentes por bono de exploración en lugar del asalto directo al VP.
        - Ampliación del tamaño de lote de inferencia a `inference_max_batch_size: 64` .

v2.5.24 — Resiliencia Integral, Autorrecuperación Multiproceso y Reciclaje Periódico de Procesos (Línea 2) [CONSOLIDADO 🟢]

- **Objetivo**: Garantizar la ejecución 24/7 ininterrumpida y a prueba de fallos de autojuego masivo en el cluster local de 24 workers multiproceso, erradicando abortos por `BrokenProcessPool` , fugas acumulativas de heap en procesos hijos de Python 3.12 y validaciones inconsistentes de categorías en catálogos de unidades.
- **Componentes Consolidados**:
    - **Mecanismo de Autorrecuperación (*Self-Healing*) de Pool de Procesos (`train_engine.py`)**:
        - Manejo y captura de excepciones de bajo nivel del sistema operativo y terminaciones prematuras de procesos (`BrokenProcessPool`).
        - Drenaje inmediato de colas IPC y reseteo determinista de cerrojos (`ipc_counter.value = 0`).
        - Reconstrucción en caliente del `ProcessPoolExecutor` (hasta 3 reintentos automáticos) para completar exactamente los episodios no finalizados de la iteración en curso.
        - Red de seguridad con *fallback* secuencial garantizado para episodios huérfanos si se supera el umbral de reintentos, asegurando que ninguna iteración ni ciclo se interrumpa.
    - **Reciclaje Periódico de Procesos de Autojuego (`recycle_workers_every_iters: 5`)**:
        - Destrucción limpia y reinstanciación de los 24 workers cada 5 iteraciones (coincidiendo con el intervalo de guardado de *checkpoints*).
        - Erradicación total de la fragmentación de heap y de fugas de memoria en la CRT de CPython (los procesos se mantienen siempre frescos en ~80–120 MB en lugar de acumular más de 450 MB tras 35 iteraciones).
    - **Ampliación Canónica de Categorías de Unidades (`assault_model/units/unit_type.py`)**:
        - Inclusión oficial de `ARTILLERY` y `GUN_ARTILLERY` en el enum `UnitCategory` .
        - Corrección fidedigna en `assets/catalogs/unit_catalog.json` permitiendo la carga sin errores de piezas de artillería de campaña (`US_75MM_FIELD_GUN`) y morteros pesados (`GE_81MM_MORTAR`).
- **Verificación de Pruebas**:
    - 154/154 tests en `agents/efficientzero_v2/tests/` 100% en verde.
    - 114/114 tests en `assault_model/tests/` 100% en verde.

v2.5.23 — Arquitectura Multiproceso con IPC Shared-Memory e Inferencia GPU por Lotes (Línea 2 / Tarea 2.18) [CONSOLIDADO 🟢]

- **Objetivo**: Erradicar el cuello de botella del Global Interpreter Lock (GIL) de Python en el autojuego masivo, permitiendo que los 28 núcleos lógicos del Intel Core i7-14700K se saturen al 80%-100% en procesos independientes del sistema operativo sin perder el micro-batching ultra-rápido en la GPU NVIDIA RTX 4060.
- **Componentes Consolidados**:
    - **Erradicación del GIL con Procesos de SO (`ProcessPoolExecutor` y `spawn`)**:
        - Sustitución del backend de hilos (`thread`) por `parallel_backend: process` . Cada worker corre en su propio proceso e intérprete de Python independiente.
        - Eliminación total de la contención del GIL: los cálculos de lógica del juego (reglas de movimiento, distancias hexagonales, líneas de visión y tiradas de dados) se ejecutan a velocidad nativa sin bloquear otros hilos.
    - **Puente Maestro IPC de Inferencia (`IPCInferenceBridge` en `inference_service.py`)**:
        - Hilo despachador de alta velocidad en el proceso principal que consume peticiones de inferencia de una cola compartida `torch.multiprocessing.Queue` .
        - Enrutamiento asíncrono hacia el `InferenceService` central en CUDA, ejecutando lotes dinámicos de hasta 64 observaciones sobre los Tensor Cores de la RTX 4060.
        - Devolución inmediata de representaciones latentes, políticas y valores a colas de respuesta individuales por worker en memoria compartida (`shm`).
    - **Cliente de Inferencia Desacoplado (`IPCProcessInferenceClient` en `inference_service.py`)**:
        - Proxy de modelo drop-in transparente que se inicializa en cada proceso de autojuego.
        - Pasa observaciones limpias a la GPU vía IPC y recibe predicciones sin necesidad de instanciar modelos PyTorch pesados en la CPU ni sincronizar state_dicts periódicos.

- **Configuración Agresiva de Hardware y Búsqueda MCTS**:
  - Activación de 24 workers concurrentes en CPU.
  - Incremento de profundidad de búsqueda MCTS a 320 simulaciones por decisión.
  - Micro-batching GPU ampliado a 64 elementos.
  - Batch de entrenamiento en CUDA ampliado a 48 muestras y 48 episodios por iteración (2 oleadas limpias de 24).

- **Verificación de Pruebas**:
  - 154/154 tests en `agents/efficientzero_v2/tests/` en verde (incluyendo `test_ipc_multiprocess_selfplay.py`).
  - Validación end-to-end completada exitosamente con `run_training` en modo proceso IPC con GPU.

v2.5.22 — Paquete de Aceleración Extrema de Entrenamiento e Inferencia Neuronal (Línea 2 / Tarea 2.17) [CONSOLIDADO 🟢]

- **Objetivo**: Perfilar y optimizar a fondo el bucle de entrenamiento neuronal (`train_engine.py`, `trainer.py`, `network_base.py` y `trainer_base.py`) en GPU (NVIDIA RTX 4060) erradicando sobrecostes de inferencia auxiliar redundante, sincronizaciones CPU-GPU bloqueantes y creación repetitiva de hilos del sistema operativo, preservando el 100% de la semántica matemática de funciones de pérdida, Two-Hot, PBRS y representaciones tensoriales.
- **Componentes Consolidados**:
  - **Fase 1: Evaluación Perezosa y Condicional de Cabezas Tácticas Auxiliares (`network_base.py`)**:
    - En `initial_inference` y `recurrent_inference`, las 7 cabezas auxiliares (`threat_map`, `attrition`, `morale`, `dynamic_role`, `target_preference`, `objective_intent`, `final_ownership`) sólo se evalúan si `return_aux=True`.
    - En las simulaciones MCTS de autojuego y en los desenrollados temporales de consistencia donde `return_aux=False`, se evita el cómputo de millones de convoluciones 2D pesadas y proyecciones lineales.
    - Aceleración medida en inferencia directa: reducción de 23.17 ms a 19.41 ms por inferencia pura (**+16.2% de aceleración**).
  - **Fase 2: Fusión y Deduplicación del Paso de Reanálisis en Entrenamiento (`trainer.py`)**:
    - Eliminado el paso hacia adelante preliminar redundante de `initial_inference(obs)` en la reanálisis de políticas.
    - Reutilización directa y desacoplada de `policy_logits.detach()` obtenido en el paso principal hacia adelante, manteniendo identidad matemática de targets y ahorrando una inferencia de red neuronal completa por cada paso de optimización.
  - **Fase 3: Vectorización GPU de Bootstrap Targets y Extracción de Amenazas (`trainer.py`)**:
    - Reemplazo de bucles iterativos con llamadas síncronas bloqueantes `.item()` en CPU por indexación tensorial vectorizada directamente en memoria GPU (`sample_indices`, `obs_indices`, `discounts * bvals_vec[obs_indices]`).
    - Erradicación de cuellos de botella por sincronización CUDA Host-Device en el cálculo de valor bootstrap de reanálisis.
    - Vectorización nativa GPU de targets de mapa de amenazas (`obs[:, 7:8]`) cuando no se proveen targets explícitos.
    - Supresión de cálculo de cabezas auxiliares (`return_aux=False`) en los desenrollados de pérdida de consistencia de representación.
  - **Fase 4: Optimizador Fused CUDA y Erradicación de Hilos Heartbeat por Paso (`trainer_base.py` y `train_engine.py`)**:
    - Activación de `torch.optim.Adam(..., fused=True)` en dispositivos CUDA, fusionando las operaciones de actualización de gradientes y momentos en un único kernel CUDA altamente optimizado.
    - Supresión de la creación descontrolada de instancias `threading.Thread` en `train_engine.py` (se creaban 27 hilos por iteración / 2.160 hilos por run), sustituida por llamada síncrona directa y ligera.
- **Rendimiento Verificado**:
  - Tiempo por actualización de red neuronal en GPU NVIDIA RTX 4060: reducido de **92.58 ms** a **51.96 ms** (**aceleración de 1.78× / reducción del 43.9% del tiempo de cómputo**).
- **Verificación de Pruebas**:
  - 153/153 tests en `agents/efficientzero_v2/tests/` en verde (3.60s).
  - 19/19 tests en `mlops/tests/` en verde (1.91s).
  - 9/9 tests en `assault_bench/tests/` en verde (63.63s).

v2.5.21 — Saneamiento Estructural y Refactorización del Modelo de Juego (`assault_model/`) [CONSOLIDADO 🟢]

- **Objetivo**: Sanear de forma integral el módulo `assault_model/`, podando código muerto, enums/clases clonadas y duplicadas, optimizando el bucle caliente de acceso a estados de hexágono y resolviendo cualquier residuo en español para alineación con las Directivas de Desarrollo (Regla 5 y Regla 7).
- **Componentes Consolidados**:
  - **Poda de Módulos Huérfanos y Clases Abandonadas (Cero Referencias)**:
    - Eliminados 10 archivos y carpetas obsoletas sin uso: `assault_model/core/model_adapter.py`, `core/activation.py`, `core/replay.py`, `rulesets/` (`ruleset.py`), `units/attributes.py`, `actions/combat.py`, `combat/combat_action_context.py`, `combat/los_modifier.py`, `combat/modifiers/morale_modifier.py` y `combat/flank_modifier.py`.
    - Reubicación canónica de `assault_model/roadmap/morale_system.md` a `docs/specs/morale_system.md`.
  - **Consolidación y Supresión de Duplicidades**:
    - Supresión de `CombatBand` y `Flank` a favor de los enums canónicos unificados `RangeBand` y `AttackSector`.
    - Unificación de `AttackDicePool` y `DefenseDicePool` hacia una implementación canónica y eficiente en `assault_model/combat/dice_pool.py` (manteniendo re-exports para compatibilidad de tests y monkeypatching).
    - Eliminación del prototype abandonado `combat_resolution.py` y las clases vacías `attack_die.py` / `defense_die.py`.
    - Supresión de `terrain_rules.py` y su diccionario huérfano `TERRAIN_DEFENSE`, centralizando la consulta en `terrain_config.py`.
  - **Optimización $O(1)$ de Acceso a Estado y Posesión de Hexágonos (`GameState` y `victory_outcomes.py`)**:

        - Creación de `ownership_to_side` en `GameState` (mapeo inverso directo $O(1)$) compartido eficientemente en `clone_cow()`.
        - Reemplazo de bucles de búsqueda $O(N)$ en `evaluate_scenario_outcome` (`victory_outcomes.py`) por lecturas directas $O(1)$ de posesión de VPs.
    - **Paridad Lingüística y Cumplimiento Estricto de la Regla 7**:
        - Traducción oportunista a inglés de todos los comentarios residuales en español en `movement_rules.py`, `game_state.py`, `unit_instance.py` y `vp_tracker.py`.
- **Verificación de Pruebas**: 114/114 pruebas en `assault_model/tests/` y 153/153 en `agents/efficientzero_v2/tests/` 100% en verde (267/267 tests pasando).

v2.5.20 — Optimización Extrema del Bucle de Simulación, Observaciones y Autojuego (Línea 2 / Tarea 2.16) [CONSOLIDADO 🟢]

- **Objetivo**: Erradicar cuellos de botella de CPU y E/S en el bucle caliente de simulación, generación de acciones y ensamblado de tensores de observación de 38 canales, acelerando drásticamente el autojuego y la evaluación sin alterar ninguna regla física ni representación tensorial.
- **Componentes Consolidados**:
    - **Plantilla Estática de Propiedades de Terreno en Simulación (`VOECSimulator.spatial_features`)**:
        - Precomputación por episodio de las características invariantes del mapa (`_static_spatial_features`): elevación, tipos de terreno, costes de movimiento por hexágono, modificadores de dados de cobertura, trincheras, búnkeres y coordenadas base de Victory Points.
        - En cada paso de simulación, `spatial_features()` realiza una copia plana superficial del diccionario base y únicamente actualiza las capas dinámicas (`vp_owner_by_hex` y roles tácticos de unidades vivas).
        - Rendimiento verificado: reducción del coste de 0.4773s a 0.0011s por 1.000 llamadas (**aceleración de 418.1×**).
    - **Memoización por Destino de Fuego en Movimiento (`assault_model/actions/action_catalog.py`)**:
        - En `_move_fire_actions`, memoización de `_ranged_fire_actions_from_position` indexada por tupla axial de destino `(q, r)` (`fires_by_dest`).
        - Erradicación de cálculos duplicados de líneas de visión (LOS) y validaciones balísticas para múltiples caminos de medio movimiento que convergen en el mismo hexágono de tiro.
    - **Caché en Memoria de Catálogos y Escenarios JSON (`importers.py` y `scenario_loader.py`)**:
        - Implementación de `_CATALOG_CACHE` en `voec_sim/assets_bridge/importers.py` y `_RAW_SCENARIO_JSON_CACHE` en `assault_model/core/scenario_loader.py`.
        - Supresión de más de 7.680 operaciones redundantes de lectura en disco y parseo JSON por ciclo curricular, reduciendo la latencia de instanciación de 1.76ms a 0.66ms.
    - **Memoización de Bounding Box Axial en Observaciones (`adapter_voec.py` y `observation.py`)**:
        - Caché de límites de coordenadas axiales (`min_q, max_q, min_r, max_r`) en `EZV2VOECAdapter` y almacenamiento en `EZV2Observation._coord_bounds`.
        - En `observation_to_tensor`, eliminación de la instanciación repetitiva de arrays NumPy y bucles de agregación de coordenadas para determinar la ventana espacial de 16×16.
- **Verificación Automatizada**:
    - Batería integral en verde: 153/153 tests en `agents/efficientzero_v2/tests/`, 19/19 en `mlops/tests/` y 9/9 en `assault_bench/tests/`.

v2.5.19 — Perfilador Causal de Partidas y SitRep Forense (Línea 4 / Tarea 4.2) [CONSOLIDADO 🟢]

- **Objetivo**: Proporcionar auditoría forense causal de partidas de wargame mediante la correlación matemática determinista entre las predicciones tácticas de las cabezas auxiliares de EfficientZero V2 (`threat_map`, `morale_pred`, `objective_intent`) y los resultados observados sobre el tablero (captura y pérdida de VPs, bajas catastróficas, supresión y emboscadas). Sintetizar automáticamente un informe narrativo militar (SitRep) fidedigno (Zero Hallucination) respaldado por citas doctrinales del reglamento de wargame vía `assault_rag`.
- **Componentes Consolidados**:
    - **Motor de Perfilado Causal (`mlops/reporting/causal_profiler.py`)**:
        - **Detección Automática de Puntos de Inflexión (`detect_turning_points`)**: Detección determinista de eventos críticos: transiciones de control de VPs (`VP_CAPTURE`, `VP_LOSS`), eliminaciones de unidades (`UNIT_ELIMINATION`), fuego concentrado pesado (`HEAVY_ATTRITION`), crisis de supresión de moral (`MORALE_FAILURE`) y emboscadas inesperadas en zonas de baja amenaza predicha (`SURPRISE_DAMAGE`).
        - **Correlación Matemática de Cabezas (`correlate_tactical_heads`)**:
            - *Threat Map*: Correlación de Pearson (r) entre el mapa de amenaza espacial y el daño real sufrido a horizonte de 2 pasos; cálculo de precisión de anticipación, incidentes de emboscada por sorpresa y mitigación de amenazas por cobertura/parada táctica.
            - *Morale / Fragility*: Precisión de anticipación de supresión, correlación con estados de pinning y tasa de conversión de fragilidad.
            - *Objective Intent*: Tasa de conversión de VPs objetivo a capturas consolidadas, latencia media de conversión en pasos y tasa de oscilación o titubeo entre objetivos bajo fuego cruzado.
        - **Síntesis Narrativa Operacional (`generate_executive_narrative`)**: Resumen ejecutivo militar coherente, desglose en tres fases operacionales (Fase I: Apertura y Contacto, Fase II: Choques Decisivos, Fase III: Consolidación y Desenlace) y recomendaciones tácticas accionables para el ajuste de políticas y MCTS.

        - **Caché Persistente en Disco (`.cache_sitrep/`)**: Aceleración inmediata de consultas subsecuentes mediante almacenamiento en caché JSON de los informes forenses.
    - **Grounding Doctrinal RAG (`assault_rag/copilot/services.py`)**:
        - Función `retrieve_doctrinal_citations_for_sitrep` que mapea cada tipo de evento crítico con consultas semánticas hacia el corpus de reglas (`03_MOVEMENT_AND_TERRAIN.md` , `RAG_KB_RULEBOOK_EXPLICATIVO.md`), extrayendo artículos formales, citas textuales y justificaciones de relevancia normativa sin alucinación.
    - **Endpoints REST en Servidor MLOps (`mlops/reporting/viewer.py`)**:
        - `/api/v2/replay/causal-sitrep` y `/api/v2/replay/sitrep` con soporte de parámetros `eval_source_path` , `episode_id` y `force_recompute` .
    - **Frontend Interactivo en Dashboard MLOps (`:8778`)**:
        - Componente `ForensicSitRepModal.tsx` en `viewer_frontend` con diseño militar táctico y 4 pestañas interactivas: *Executive Assessment*, *Tactical Head Forensics*, *Turning Points* (con botón `▶ Seek Step` para saltar directamente al paso en el tablero de replay) y *Doctrinal Grounding (RAG)*.
        - Botón de acceso directo `🎖️ Forensic SitRep` en la barra de herramientas del canvas de `MatchReplayTab.tsx` .
        - Interfaz de usuario 100% en inglés según Regla 7.
    - **Verificación Automatizada**:
        - Suite dedicada `mlops/tests/test_causal_profiler.py` (9/9 tests en verde): validación de detección de capturas de VPs, bajas, crisis de moral, emboscadas por sorpresa, cálculo de correlación de Pearson, síntesis de narrativa, estructura del SitRep y caché persistente.

v2.5.14 — Presupuesto Dinámico de Simulación y Horizonte Gobernado por Turnos con Refuerzos (Turn-Authoritative Horizon) [CONSOLIDADO 🟢]

- **Objetivo**: Resolver de raíz la desincronización entre el presupuesto estático de micro-pasos (`max_steps`) y los macro-turnos del wargame ante la llegada de refuerzos dinámicos, erradicando los falsos timeouts y la penalización espuria de -1.0 que provocaba la inercia defensiva del agente.
- **Componentes Consolidados**:
    - **Ingesta del OOB Total en Simulación (`scenario_total_units`)**:
        - Implementación en `VOECSimulator` y `EZV2VOECAdapter` del método `scenario_total_units()` , el cual audita el catálogo del escenario y totaliza las unidades de partida más todas las oleadas de refuerzos programadas en el bloque `"reinforcements"` ($N_{\text{initial}} + \sum |\text{units}|$).
    - **Presupuesto Adaptativo con Headroom Táctico (`resolve_effective_step_budget`)**:
        - Soporte del parámetro `steps_per_unit_turn` (fijado en 2.5 en autojuego y entrenamiento de `train_engine.py` , 1.0 por defecto en contratos de test).
        - Cálculo del horizonte seguro de simulación: \text{effective_max_steps} = \lceil T_{\text{scenario}} \times \text{total_units} \times \text{steps_per_unit_turn} \rceil, garantizando que escenarios con desembarcos masivos (ej. 14–18 unidades en Turno 2) dispongan de pasos holgados para todos sus turnos.
    - **Watchdog de Deadlock por Turno (`turn_deadlock_watchdog`)**:
        - Seguimiento del pulso de turnos de wargame en `_NativeEZV2SelfplayBackend.play_episode` .
        - Cada vez que el motor cambia de turno (`turn_now ≠ current_turn_seen`), el contador de pasos del turno se reinicia a cero y se calibra un techo de seguridad local: \text{max_steps_per_turn} = \max(30, N_{\text{alive}} \times 4).
        - Si ocurre un estancamiento patológico dentro de un mismo turno, se aborta con `terminal_reason = "turn_deadlock_watchdog"` , distinguiendo los deadlocks reales de los límites de partida.
    - **Preservación de Resultados Canónicos**:
        - Cuando la partida alcanza $T > T_{\max}$, el motor marca `adapter.terminal() == True` de forma natural. El bucle de autojuego preserva el desenlace reglamentario (evaluación de VPs y bajas) sin aplicar la penalización catastrófica de timeout (-1.0).
    - **Verificación Automatizada**:
        - Suite dedicada `agents/efficientzero_v2/tests/test_turn_budget_reinforcements.py` (4/4 en verde): verificación de multiplicadores de presupuesto, consulta de OOB total desde adapter, asignación dinámica en escenarios con refuerzos y disparo selectivo del watchdog ante bucles infinitos.
        - 153/153 tests pasando en `agents/efficientzero_v2/tests/` .

v2.5.13 — Pipeline Dinámico de Refuerzos de Desembarco en Fase de Organización (Tarea 3.5.6) [CONSOLIDADO 🟢]

- **Objetivo**: Modelar en el motor físico y runtime la llegada dinámica de la 2.ª oleada de desembarco aliada en el Turno 2 del Escenario 1.0 (*Mettete i piedi per terra!*), emulando fielmente la Fase de Organización del *Libro della Campagna* (§3.1, pág. 18).
- **Componentes Consolidados**:
    - **Esquema de Refuerzos en Escenario (`mettete_i_piedi_terra_1.json`)**:
        - Bloque `"reinforcements"` canónico con llegada en Turno 2, fase de organización, asignación a zonas de desembarco A (oeste) y B (este), y la formación completa `US_INF_RIFLE` (5× `US_RIFLES_43` , 1× `US_BAZOOKA_TEAM` , 1× `US_81MM_MORTAR` , unidades `US_8` a `US_14`).
    - **Carga y Registro en Escenario (`assault_model/core/scenario.py` y `scenario_loader.py`)**:
        - Soporte formal de `reinforcements` y retención del catálogo de unidades (`scenario.unit_catalog`).
    - **Registro Dinámico en Estado (`assault_model/state/game_state.py`)**:

        - Método `GameState.add_unit(unit)` para registrar dinámicamente nuevas unidades en tiempo de ejecución, sincronizando lookups espaciales `_units_by_hex`, seguimiento por bando `_unit_sides`, orden de turno `turn_order` y recálculo incremental de control territorial `recalculate_hex_control`.
    - **Motor de Fase de Organización en Runtime (`assault_model/runtime/game_state_runtime.py`)**:
        - Método `_deploy_scheduled_reinforcements()` ejecutado al inicio de cada turno.
        - Resolución estocástica / determinista por semilla de tirada de dado D6 (1–3: Zona A, 4–5: Zona B, 6: Elección de zona menos congestionada).
        - Colocación en hexágonos libres de la zona o agua somera adyacente según norma de saturación de desembarco.
        - Actualización dinámica del pool de activación `self.sides = self._extract_sides()`.
    - **Integración en `VOECSimulator` y Verificación Automatizada**:
        - Conexión con `VOECSimulator.new_episode(..., seed=seed)` propagando la semilla al runtime.
        - Suite dedicada de pruebas `assault_model/tests/test_scenario_reinforcements.py` (2/2 en verde).
        - Batería integral en verde: 300/300 tests pasando (`voec_sim`, `assault_model`, `agents/efficientzero_v2`).

v2.5.12 — Despliegue Canónico e Ingesta Fidedigna de Escenarios 1.0 y 2.1 del *Libro della Campagna* (Tarea 3.5.6) [CONSOLIDADO 🟢]

- **Objetivo**: Configurar con fidelidad normativa absoluta al reglamento y libro de campaña oficial (`docs/pdfs/ITA_Assault_Libro_Campagna_v1.0.pdf`, págs. 18–21) las unidades, colocaciones iniciales, moral y duración de los escenarios 1.0 (*Mettete i piedi per terra!*) y 2.1 (*Battaglia Cittadina*), resolviendo la apócrifa guarnición alemana de desembarco y la simetría 1:1 artificial que causaba parálisis e inercia defensiva en la IA atacante.
- **Componentes Consolidados**:
    - **Escenario 1.0 Canónico (`assets/scenarios/mettete_i_piedi_terra_1.json`)**:
        - **Defensores Históricos**: Sustituida la guarnición alemana con MG42 y mortero de 50mm por la 429.ª compañía costera italiana (`XVI Corpo, XVIII Brigata Costiera`): 3× `IT_RIFLES_43` (2 en casas obligatorias `[3, 3]`, `[5, 4]` y 1 en `[5, 5]`), 1× `IT_BREDA_MMG` en trinchera `[2, 4]`, 1× `IT_45MM_BRIXIA` en `[6, 4]` y 1× `IT_SOLOTHURN_ATR_43` en trinchera `[2, 8]`.
        - **Fuerza de Desembarco Aliada**: Darby's Rangers: 4× `US_RANGERS_43` desplegados en primera línea de playa (`[1, 12]`, `[3, 12]`, `[5, 12]`, `[8, 12]`), 1× `US_COMBAT_ENGINEERS_43` en sector central de brecha (`[4, 12]`), 1× `US_BAZOOKA_TEAM` (`[7, 12]`) y 1× `US_81MM_MORTAR` (`[5, 13]`).
        - **Track y VPs**: Duración ajustada a 10 turnos (reglamento oficial). VPs iniciales en posesión italiana (`initial_owner: IT`). Inyección de status moral (Verde/US: 3/7/9/11, Azul/IT: 2/6/7/9).
    - **Escenario 2.1 Canónico (`assets/scenarios/battaglia_cittadina_2_1.json`)**:
        - **Restitución del Ratio Asimétrico Oficial 2:1 (`1/20 IT vs 2/40 US`)**: Despliegue de 14 unidades aliadas (2 formaciones) frente a 7 defensoras italianas (1 formación), restituyendo la ventaja de masa y potencia de fuego necesaria para la ruptura urbana.
        - **Defensores Italianos (7 unidades)**: 2× `IT_RIFLES_43` (`[4, 2]`, `[6, 2]`), 2× `IT_LMG_SQUAD_43` (`[3, 3]`, `[12, 3]`), 1× `IT_45MM_BRIXIA` (`[9, 1]`), 1× `IT_BREDA_MMG` (`[3, 4]`), 1× `IT_SNIPER_43` (`[9, 3]`).
        - **Atacantes Aliados (14 unidades)**:
            - Columna Oeste (Darby's Rangers, sector S3): 4× `US_RANGERS_43` (`[0, 9]`, `[1, 10]`, `[2, 10]`, `[3, 10]`), 1× `US_COMBAT_ENGINEERS_43` (`[7, 9]`), 1× `US_SNIPER_43` (`[8, 9]`), 1× `US_81MM_MORTAR` (`[9, 9]`), 1× `US_BAZOOKA_TEAM` (`[1, 9]`).
            - Columna Este/Central (1st Infantry Division, sector S4/Z2): 5× `US_RIFLES_43` (`[14, 9]`, `[15, 8]`, `[16, 8]`, `[15, 9]`, `[16, 9]`), 1× `US_BAZOOKA_TEAM` (`[14, 8]`).
        - **Moral y Turnos**: Duración de 8 turnos y status moral (Verde/US: 4/7/9/11, Azul/IT: 3/6/8/9).
    - **Verificación de Invariantes del Simulador**:
        - El simulador `VOECSimulator` opera sin restricciones de tamaño de lista de unidades (13 unidades en Escenario 1.0, 21 unidades en Escenario 2.1).
        - Batería completa de tests unitarios e integración en verde: 37/37 en `voec_sim/tests/`, 112/112 en `assault_model/tests/` y 149/149 en `agents/efficientzero_v2/tests/`.

v2.5.11 — Algoritmo Canónico de Retirada a Borde Amigo en Desenfilada (Tarea 3.1) [CONSOLIDADO 🟢]

- **Objetivo**: Sustituir la primitiva aleatoria de retirada (`q - 1, r`) por un cálculo táctico determinista multinivel que emule la doctrina militar de repliegue bajo fuego del reglamento oficial (*Rulebook rev6* §6.4 y §13.3).
- **Componentes Consolidados**:
    - **Algoritmo Táctico de Retirada (`assault_model/rules/movement_rules.py`)**:
        - Método `MovementRules.calculate_fallback_destination(game_state, unit, attacker, friendly_edge)` que evalúa exhaustivamente los 6 vecinos adyacentes.
        - Priorización de **Desenfilada (*Defilade*)**: Evaluación de línea de visión desde el atacante mediante Ray-Casting vectorial (`LineOfSight.BLOCKED` otorga +100 pts, `HINDERED` +40 pts, `CLEAR` 0 pts).
        - Priorización de **Cobertura de Terreno (*Terrain Defense*)**: Ponderación de dados de defensa del terreno (verde=6, azul=4, amarillo=2) y fortificaciones (búnker=+15, casamata=+12, trinchera=+8, sacos=+4).
        - **Vector Dirigido al Borde Amigo (*Friendly Edge Alignment*)**: Proyección de retaguardia hacia el norte (`r` decreciente para IT/GE) o sur (`r` creciente para US/UK) o vector opuesto al atacante.
        - **Aumento de Distancia al Atacante**: Penalización drástica (-500 pts) para movimientos que reduzcan la distancia al atacante.
        - Criterio de desempate determinista `(total_score, defilade_score, cover_score, cand_dist, -q, -r)`.

- **Reglas Especiales de la Sección 13.3 (`assault_model/combat/morale.py`)**:
  - **Eliminación Inmediata**: Las unidades de artillería o vehículos inmovilizados son destruidos de inmediato al sufrir fallback (`unit.alive = False`, `unit.strength = 0`).
  - **Daño por Bloqueo / Rodeado**: Si la unidad no tiene ningún hexágono transitable disponible para retirarse, sufre 1 punto de daño (`unit.apply_damage(1)`) y permanece en su posición.
  - **Sincronización de Control Espacial**: Invocación de `recalculate_hex_control` con las posiciones afectadas.
- **Integración en Combate a Distancia (`assault_model/combat/ranged_combat_resolver.py`)**:
  - Paso automático de `game_state` y `attacker` a `resolve_fallback` tras acumulación de impactos de supresión.
- **Suite de Verificación Automatizada (`assault_model/tests/test_fallback_defilade_algorithm.py`)**:
  - 7 tests unitarios e integración dedicados en verde, alcanzando 112/112 tests pasando en `assault_model/tests/`.

v2.5.10 — Atribución Causal Espacial Hexagonal (Tarea 4.1) y Unificación de Diagnóstico de Cabezas [CONSOLIDADO 🟢]

- **Objetivo**: Integrar interpretabilidad causal espacial hex a hex para cualquier decisión táctica en el visor MLOps y unificar las vistas redundantes de diagnóstico de cabezas en una consola única, coherente y canónica.
- **Componentes Consolidados**:
  - **API de Explicabilidad Causal (`mlops/reporting/explainability_api.py`)**:
    - Endpoint `/api/v2/replay/explain-step` que calcula Grad-CAM e Integrated Gradients sobre los 32 canales sensoriales heterogéneos (terreno, elevación, cobertura, LOS, distancia a VPs, mapas de amenaza, desgaste y moral) procesados por la `representation_tower` de EfficientZero V2.
    - Mapeo biyectivo axial $(q, r) \leftrightarrow (h, w)$ para renderizado exacto de contribución por hexágono.
  - **Visualizador Modal y Heatmap de Explicabilidad en Replay**:
    - Componente React `ExplainabilityModal.tsx` integrado tanto en `mlops/reporting/viewer_frontend` como en `assault_ai_ui`.
    - Overlay de gradiente térmico reactivo sobre el canvas 2D del tablero hexagonal en `MatchReplayTab.tsx`, permitiendo inspeccionar qué canales determinaron la decisión en cada turno.
  - **Unificación Integral de Diagnóstico de Cabezas (Opción A)**:
    - Fusión completa en `Evaluación & Benchmarks → Diagnóstico de Cabezas (XAI)` (`HeadDiagnosticsTab.tsx`) de:
      1. Tarjetas sintéticas de salud para las 7 cabezas activas (`policy`, `value`, `objective`, `consistency`, `threat_map`, `attrition`, `morale`) con 4 KPIs por cabeza y badges de diagnóstico cualitativo.
      2. Matriz de verificación MLOps con umbrales de referencia y estado de convergencia.
      3. Tablero de cordura y telemetría de comportamiento (saturación Q, amplitud de exploración, cobertura de telemetría y visitas útiles).
      4. Desglose canónico de las 11 métricas de evaluación emitidas.
    - Desacoplamiento de la pestaña `Topología & Red` (`NetworkTopologyTab.tsx`), eliminando la subpestaña duplicada para mantenerla focalizada en arquitectura física, atribución de canales y planificador macro, e incorporando botón de salto directo al diagnóstico unificado.
    - Cumplimiento estricto de la Regla 7: UI y código 100% en inglés.
  - **Certificación de Pruebas**:
    - Suite de pruebas `mlops/tests/test_explainability_api.py` (3/3 tests en verde) y compilación exitosa sin advertencias ni errores en `viewer_frontend` y `assault_ai_ui`.

v2.5.9 — Catálogo Unificado y Depuración de Esquemas Parquet y DuckDB (Tarea 1.4) [CONSOLIDADO 🟢]

- **Objetivo**: Establecer un esquema Parquet estrictamente canónico y unificado para decisiones tácticas, prevenir fugas de campos acumuladores volátiles, sanear todos los datasets históricos de evaluaciones y ejecutar un Clean Break total con etiquetas legacy obsoletas en el catálogo DuckDB.
- **Componentes Consolidados**:
  - **Módulo Gestor Canónico de Datasets (`mlops/dataset_builder.py`)**:
    - Definición inmutable del contrato de 15 columnas canónicas: `CANONICAL_DECISIONS_TOP_COLUMNS` (`decision_type`, `unit_id`, `count`, `damage_dealt`, `kills_dealt`, `cover_avg`, `distance_avg`, `los_avg`, `target_q_avg`, `target_r_avg`, `mcts_overwritten`, `mcts_policy_kept`, `policy_prior_sum`, `mcts_visit_sum`, `mcts_q_sum`).
    - Implementación de herramientas de migración y verificación atómica: `purge_single_decisions_top_parquet()`, `purge_all_decisions_top_parquet()`, `purge_legacy_controllers_in_catalog()` y `rebuild_unified_duckdb()`.
  - **Prevención de Fugas de Acumuladores en Origen (`assault_bench/decision_metrics.py`)**:
    - Filtrado explícito mediante `EXCLUDED_ACCUMULATOR_KEYS` en la función `_enrich(v)` para bloquear la propagación accidental de métricas acumuladoras intermedias (`cover_n`, `cover_sum`, `distance_n`, `distance_sum`, `los_n`, `los_sum`, `target_q_n`, `target_q_sum`, `target_r_n`, `target_r_sum`).
  - **Saneamiento y Homogeneización del 100% de Ficheros Parquet**:
    - Saneamiento y purga exitosa de 19 archivos desalineados que contenían las 8 columnas volátiles.
    - 69 de 69 ficheros `decisions_top.parquet` a lo largo de `runs/` y `runs_curriculum/` se encuentran ahora 100% estandarizados en el esquema canónico de 15 columnas.
  - **Clean Break Radical con MuZero en Catálogo y DuckDB**:

        - Corrección de asignaciones de controlador en `mlops/reporting/build_catalog.py`, mapeando búsquedas MCTS y autojuego a `"EfficientZeroV2"`.
        - Migración y reemplazo de 12,530 registros con etiqueta `"MuZero"` en `catalog_eval_result_sides.parquet`.
        - Reconstrucción atómica y verificación formal de `reporting.duckdb` con cero registros legacy (`verified_zero_legacy: True`).
    - **Certificación de Pruebas**:
        - Nueva suite de pruebas automatizada `mlops/tests/test_unified_catalog_and_schema_purge.py` (5/5 tests en verde en 0.74s) validando esquema canónico, ausencia de fugas, idempotencia de purga, conformidad de todos los Parquets activos y pureza total en DuckDB.

v2.5.8 — Validador E2E de Trazabilidad Frontend ↔ DuckDB (Tarea 1.3) [CONSOLIDADO 🟢]

- **Objetivo**: Garantizar observabilidad fiel y trazabilidad del 100% en todas las pestañas del dashboard MLOps y la interfaz del simulador táctico hacia columnas reales de Parquet/DuckDB o payloads autoritativos, erradicando cualquier invención de datos o heurística de aproximación.
- **Componentes Consolidados**:
    - **Especificación y Validador en TypeScript (`assault_ai_ui/src/tests/traceability.spec.ts` y `mlops/reporting/viewer_frontend/src/tests/traceability.spec.ts`)**:
        - Matriz formal `CANONICAL_TRACEABILITY_MATRIX` con reglas de validación para las 9 pestañas del dashboard: `CatalogPage / Header`, `TrainDetailTab`, `TrainHeadDiagnosticsTab`, `ObjectiveRewardConfigTab`, `EvalMetricsTab`, `EvalDecisionsTab`, `MatchReplayTab`, `NetworkTopologyTab` y `VpSummaryTab`.
        - Funciones de validación `validateFieldValue` y `auditDashboardTraceability`, verificando que los valores no sean sintéticos ni contengan patrones prohibidos (`muzero_`, `mock`, `fake`, etc.).
    - **Test Automatizado de Trazabilidad en CI (`mlops/tests/test_frontend_duckdb_traceability.py`)**:
        - Comprobación en `pytest` de la integridad de la matriz documental en `docs/13_INVENTARIO_CANONICO_METRICAS.md`.
        - Auditoría estática de código fuente en componentes TSX (`viewer_frontend/src/pages/model-tabs/`) garantizando la ausencia de snippets espurios como `Math.floor(idx / 14) + 1` o roles heurísticos inventados.
        - Validación de payloads de `replay_api.py` y conformidad de esquemas en DuckDB.
    - **Optimización de Sondas y Purga de Shims Legacy**:
        - Desacoplamiento y memoización de daño esperado en prefiltrado y paso principal en `agents/efficientzero_v2/core/selfplay.py`.
        - Erradicación completa del alias shim `build_muzero_replay_payload` en `mlops/reporting/replay_api.py`.
    - **Certificación de Pruebas**:
        - 15/15 tests de la Línea 1 en verde (`test_telemetry_schema_conformance.py`, `test_telemetry_probe_overhead.py`, `test_frontend_duckdb_traceability.py`).
        - Builds de producción limpios tanto en `assault_ai_ui` como en `viewer_frontend`.

v2.5.7 — Currículum Asimétrico Ofensivo de Asalto Urbano Puro en Gela [CONSOLIDADO 🟢]

- **Objetivo**: Erradicar de raíz la inercia defensiva y la pasividad aprendida en fases tempranas de autojuego simétrico, forzando al agente a dominar la ruptura y asalto frontal sobre defensores atrincherados en `battaglia_cittadina_2_1` bajo horizonte comprimido ($T = 8$).
- **Componentes Consolidados**:
    - **Integración de Fase 5 en el Currículum Acelerado de VPs (`curriculums/vp_acceleration/curriculum.yaml`)**:
        - Escenario urbano `battaglia_cittadina_2_1` con horizonte táctico estricto a $T_{\max} = 8$ turnos.
        - Configurado con `max_reinforcement_cycles: 10` para una campaña evolutiva completa de 10 ciclos.
    - **Configuración de Entrenamiento Ofensivo 100% Atacante (`efficientzero_v2.phase5_urban_assault_pure_offensive.yaml`)**:
        - Modo de matchup asimétrico puro (`self_vs_random`) con `tracked_side: "US"` y `to_play: "US"`.
        - Penalización terminal catastrófica por timeout (`timeout_penalty: -1.0`) ante 0 VPs consolidados.
        - Activación de andamiaje PBRS dependiente del tiempo (`time_scaled_pbrs: true`), bono por disputa de brecha (`vp_contested_bonus: 0.80`), atenuación del 60 de penalización de bajas en radio $d \leq 2$ (`breach_casualty_attenuation: 0.40`) y penalización por inmovilidad/espera (`idle_penalty: -0.15`).
    - **Plan Evolutivo de 10 Ciclos Adaptativos (`phase5_urban_assault_pure_offensive.adaptive_cycles.yaml`)**:
        - 4 niveles progresivos: Baseline Promovido (Ciclo 1), Explotación Adaptativa (Ciclos 2–4), Exploración en Frontera con simulación profunda (Ciclos 5–7) e Hiper-Asalto Convergente (Ciclos 8–10 con hasta 416 sims MCTS y reanálisis del 60%).
    - **Configuración de Benchmark de Evaluación (`assault_bench/configs/benchmark_config.vp_acceleration_phase5.yaml`)**:
        - Batería de 15 seeds sobre `battaglia_cittadina_2_1` evaluando perfiles `ezv2_vs_random_side_b` y `ezv2_selfplay`.
    - **Verificación Formal**:
        - Suite dedicada `agents/efficientzero_v2/tests/test_task_2_11_urban_assault_curriculum.py` (4/4 tests en verde) y suite global con 186 tests pasando.

v2.5.6 — Optimización de Alto Rendimiento GameState/CoW y Decomisión Limpia de Módulos Legacy [CONSOLIDADO 🟢]

- **Objetivo**: Elevar la tasa de clonado en el bucle caliente de MCTS mediante eliminación de sobrecargas de asignación dinámica de memoria, optimizar el acceso a celdas hexagonales y purgar 9 módulos huérfanos legacy de fuego de reacción sin uso.
- **Componentes Consolidados**:

- **Compactación de Memoria con `__slots__` en `HexState` (`assault_model/map/hex_state.py`)**:
  - Eliminación del diccionario de instancias `__dict__` en cada celda hexagonal del mapa, definiendo `__slots__ = ("hex", "ownership", "contested", "smoke_markers")`. Acelera la clonación superficial y reduce la fragmentación de memoria en árboles MCTS profundos.
- **Pre-computación y Caché Estática de `_vp_hexes` (`assault_model/state/game_state.py`)**:
  - Almacenamiento directo de `self._vp_hexes` en `__init__`, evitando la recreación continua de comprensiones de conjuntos `set(vp.hex_coords for vp in self.victory.points)` en cada invocación de `recalculate_hex_control()`.
- **Optimización de Paso por Referencia Inmutable en `clone_cow()`**:
  - `UnitInstance.clone_cow()`: Reutilización directa de la instancia inmutable `self.position` (`HexCoord` con slots `q, r`) eliminando la instanciación redundante `HexCoord(pos.q, pos.r)`.
  - `GameState.clone_cow()`: Compartición por referencia de `_unit_sides` y `_vp_hexes` en lugar de clonado de diccionario, y acceso directo a atributos pre-inicializados.
  - `RuntimeGameState.clone_cow()`: Compartición por referencia de `self.sides`, `self.side_controller_map` y `self.turn`.
  - **Rendimiento Acelerado Verificado:** `RuntimeGameState.clone_cow()` asciende de 20.611 a más de 33.400 clones/s (+62.1%), y la velocidad de simulación de pasos asciende de 401 a 465 pasos/s (+15.7%).
- **Decomisión Limpia de 9 Módulos Legacy de Fuego de Reacción**:
  - Purgados los archivos huérfanos obsoletos: `assault_model/runtime/game_state_reactions.py`, `assault_model/core/reaction_registry.py`, `assault_model/combat/reaction_state.py`, `assault_model/combat/reaction_fire_resolution.py`, `assault_model/combat/reaction_fire.py`, `assault_model/actions/reaction_fire.py`, `assault_model/combat/reaction_condition.py`, `assault_model/combat/reaction_trigger.py`, y `assault_model/combat/reaction_context.py`.
  - Eliminados los métodos muertos `enter_reaction()` y `clear_reaction()` en `GameState`.
- **Corrección de Robustez en Timeout de Benchmarks (`assault_bench/episode_runner_impl.py`)**:
  - Inicialización segura de `eval_candidates: list[str] = []` al inicio de la resolución, previniendo excepciones `UnboundLocalError` ante diccionarios de políticas vacíos.
- **Certificación de Pruebas**:
  - 104/104 tests de `assault_model/` en verde.
  - 186/186 tests de `voec_sim/` y `agents/efficientzero_v2/` en verde.
  - Suite de benchmarks y pipelines forenses ejecutando al 100% en verde.

v2.5.5 — Inicialización Canónica de Capas Especializada en Reinforcement Learning [CONSOLIDADO 🟢]

- **Objetivo**: Erradicar el colapso prematuro de la política y la deriva residual en autojuego temprano mediante un esquema formal de inicialización adaptado a redes profundas con rollouts MCTS: máxima entropía estocástica inicial, preservación de varianza latente e identidad residual pura al paso 0.
- **Componentes Consolidados**:
  - **Política Ortogonal de Ganancia Mínima ($\text{gain} = 0.01, \text{bias} = 0.0$)**:
    - Aplicada a la capa de salida de la cabeza de política (`prediction_policy`).
    - Garantiza logits $z_a \approx 0$ y una distribución inicial $\pi(a \mid s)$ casi perfectamente uniforme ($H(\pi) \approx \log(32)$, discrepancia < 0.02 nats), asegurando exploración completa del catálogo de acciones en autojuego temprano.
  - **Salidas de Valor, Recompensa, Consistencia y Auxiliares Ortogonales ($\text{gain} = 1.0, \text{bias} = 0.0$)**:
    - Salidas lineales de `prediction_value`, `dynamics_reward`, `consistency_projection` y cabezas tácticas auxiliares (`threat_map`, `morale`, `attrition`, `final_ownership`).
    - Preserva la norma Euclidiana de las proyecciones y previene saturación en los átomos Two-Hot.
  - **Kaiming Normal (He Normal, mode=fan_out) en Convoluciones SiLU y Capas Ocultas**:
    - Aplicado a `HexagonalMaskConv2d`, `Conv2d`, `Conv1d` y capas lineales intermedias, estabilizando la varianza de propagación hacia adelante.
  - **Zero-Init ResNet / Fixup en Segunda Normalización Residual ($\gamma = 0, \beta = 0$)**:
    - Inicialización en cero absoluto de pesos y sesgos en `gn2` (`ResidualConvBlock`) y `ln2` (`ResidualMLPBlock`).
    - Al evaluar $y = x + F(x)$ con $F(x) = \text{Attn}(\text{Norm}_2(\dots)) = 0$, toda la torre residual se comporta como la función identidad pura al paso 0 ($y = x$), habilitando flujo directo de gradientes $\frac{\partial y}{\partial x} = I$.
  - **Squeeze-and-Excitation Canónico Multiplicativo (`SEBlock`)**:
    - Encapsulación de atención SE como módulo que multiplica activaciones por el mapa de compuerta sigmoid ($x \times \text{Sigmoid}(\dots)$), alineando la interfaz con `ECABlock`.
  - **Verificación Formal**:
    - Suite dedicada `agents/efficientzero_v2/tests/test_canonical_rl_initialization.py` (5/5 tests en verde).

v2.5.4 — Dynamic PBRS Escalado Temporalmente, Desensibilización de Brecha y Formaciones Oficiales [CONSOLIDADO 🟢]

- **Objetivo**: Resolver de raíz la Paradoja del Comandante Prudente (*The Prudent Commander Paradox*), eliminando la pasividad y aversión al asalto final en escenarios con límite estricto de turnos ($T_{\max}$), mediante andamiaje potencial dependiente del tiempo, atenuación de bajas en la brecha del objetivo y penalización catastrófica por timeout.
- **Componentes Consolidados**:

- **Dynamic PBRS Time Scaling & Contention Bonus (`voec_sim/core/reward_calculator.py`)**:
  - Potencial radial dinámico \Phi(s, t) = \alpha \left(1 + \frac{t}{T_{\max}}\right) \cdot \text{dist_pot} + \beta \cdot \text{Contest} + \text{ctrl_pot}.
  - Amplificación progresiva de la aproximación hacia los VPs conforme el reloj avanza hacia el turno final ($(1 + t/T_{\max}) \in [1.0, 2.0]$).
  - Bono de contención $\beta \cdot \text{Contest}$ activo cuando escuadras amigas penetran en el radio de asalto ($d \leq 2$ hexágonos).
  - Demostración rigurosa del Teorema 1 de Devlin & Kudenko (2012): la suma telescópica descontada $\sum_{t=0}^{H-1} \gamma^t F(s, t, a, s', t') = -\Phi(s_0, 0)$ preserva intacta la política óptima $\arg\max_a Q^*(s, a)$.
- **Atenuación de Bajas en Brecha (*Breach Zone Casualty Attenuation*)**:
  - Método `compute_breach_casualty_factor` que reduce en un 60 la penalización de bajas ($r_{\text{loss}} \times 0.40$) cuando la unidad opera a $d \leq 2$ hexágonos de un VP objetivo, permitiendo al MCTS asumir pérdidas tácticamente necesarias para consolidar la misión.
- **Castigo Catastrófico por Timeout**:
  - En `agents/efficientzero_v2/core/selfplay.py`, los episodios que agotan el presupuesto temporal (`turn_unit_budget` / `timeout`) con 0 VPs bajo control reciben la penalización máxima de derrota total (-1.0), erradicando la especulación estéril.
- **Modelado del Catálogo de 12 Formaciones Oficiales**:
  - Modelado e ingesta de las 12 formaciones combinadas oficiales del reglamento (Sección 5.0) en `formation_catalog.json` y escaneo de las 122 cartas de arte táctico en `formation_art_inventory.json`.
- **Verificación Formal**:
  - Suite dedicada `voec_sim/tests/test_task_2_10_dynamic_pbrs_breach.py` con 5/5 tests en verde y suite global con 177 tests pasando.

v2.5.3 — Optimización Evolutiva de Ciclos y Desacoplamiento Estricto de Checkpoints [CONSOLIDADO 🟢]

- **Objetivo**: Garantizar que los curriculums lanzados desde cero entrenen siempre sin interferencias de checkpoints externos (tabula rasa limpia), implementando simultáneamente un mecanismo evolutivo de hiperparámetros donde cada fase aprenda de ejecuciones previas: el ciclo ganador $se convierte en el nuevo$ (baseline) y se sintetizan automáticamente las variantes adaptativas ($explotación/refinamiento$) y (exploración/agresividad).
- **Componentes Consolidados**:
  - **Aislamiento Estricto de Pesos en Lanzamientos desde Cero**:
    - Eliminación sistemática de cualquier residuo de esume_checkpoint o checkpoint_path en las configuraciones base de fase y en los planes adaptativos. Cada ejecución desde cero recorre sus fases de forma autónoma sin contaminación de modelos previos.
  - **Optimizador Evolutivo de Ciclos (≡gents/efficientzero_v2/curriculum/evolutionary_cycle_optimizer.py)**:
    - Auditoría automática del historial de ejecución (curriculum_phase_traceability.jsonl) para identificar el ciclo ganador de cada fase según promoción formal de gate (promote), mejor de ciclos (promote_best_of_cycles) o máxima puntuación TQS/Win Rate.
    - Promoción de los hiperparámetros ganadores de $a la configuración base$ de la fase para futuras ejecuciones.
    - Síntesis inteligente de variantes adaptativas:
      - **$ (Explotación y Convergencia Táctica)**: $+5$ tasa de reanálisis, temperatura MCTS más conservadora (-0.05), incremento de updates por iteración ($+2$) y mayor peso de potencial de VPs ($+0.10$).
      - **$ (Exploración y Agresividad Territorial)**: $+15$ a bonificaciones de captura y progreso hacia VPs, mayor peso a la pérdida de objetivos territoriales ($+0.04$), temperatura MCTS expandida ($+0.05$) y $+4$ updates de entrenamiento.
    - Persistencia de especificaciones en cycles/.adaptive_cycles.yaml y registro de linaje generacional en curriculum_generation_lineage.jsonl.
  - **Integración Nativa en Runner Curricular (curriculums/common/run_dynamic_gated_curriculum.ps1)**:
    - Build-ResumeOverlayConfig consulta dinámicamente el plan adaptativo de la fase (≡daptive_cycles.yaml) y aplica la estrategia correspondiente a cada iteración de ciclo (, $).
    - Paso final automático de evolución al completar todas las fases del currículum.
  - **Suite de Pruebas Automatizadas (est_evolutionary_cycle_optimizer.py)**:
    - 4 tests unitarios dedicados en verde validando auditoría de trazas, síntesis de variantes, eliminación de checkpoints y ejecución e2e.

v2.5.2 — Estabilidad Forense de Telemetría Curricular, Evaluación Asimétrica y Calibración ELO `[CONSOLIDADO 🟢]`

- **Objetivo**: Corregir anomalías de cálculo en la tasa de victorias no-timeout en evaluaciones curriculares multi-perfil, erradicar la asimetría de evaluación en autojuego espejo (self-play EZv2 vs EZv2), asegurar la integridad del ancla ELO Bradley-Terry ante empates, y dinamizar el estado de promoción en el visualizador MLOps con cumplimiento de UI en inglés.
- **Componentes Consolidados**:
  - **Agregador Ponderado Multi-Perfil en Runner Curricular (`curriculums/common/run_dynamic_gated_curriculum.ps1`)**:
    - Cálculo ponderado por episodios no-timeout para `resolved_non_timeout_win_rate`, eliminando la sobrescritura arbitraria con la primera fila (`Select-Object -First 1`), la cual causaba lecturas espurias de 0.0% cuando el primer perfil evaluado era un empate de autojuego.
  - **Alineación de Perspectiva en Auto-Juego Espejo (`assault_bench/episode_runner_impl.py`)**:
    - En perfiles `ezv2_selfplay` con múltiples candidatos, priorización del bando trazado (`tracked_side` definido en el escenario) en lugar de asignar arbitrariamente el lado al primer bando en mover (`obs.to_play`), garantizando que las victorias del agente en el bando activo computen correctamente.
  - **Erradicación de Vestigios MuZero en Controladores (`assault_bench/benchmark_result_builder.py`)**:

- `_controller_label_from_policy` mapea de forma limpia y estricta a `EfficientZeroV2` ante políticas `mcts` / `ezv2`, cumpliendo la directiva de decomisión total de MuZero.

- **Robustez en Ancla Bradley-Terry y Manejo de Empates (`mlops/reporting/viewer_frontend/src/curriculumMetrics.ts`)**:
  - Acotación balanceada $[0.15, 0.85]$ en `computeStepAnchorElo` para el perfil de autojuego, previniendo caídas catastróficas de más de 700 puntos ELO por empates o sesgos asimétricos.
  - Fallback automático al score agregado de la fase en `extractStepTraceMetrics` cuando la tasa resuelta indica 0 pero existen victorias netas en otros perfiles.

- **Resolución del Fallo de Evaluación en Coaliciones Multi-Bando (`assault_bench/episode_runner_impl.py`)**:
  - Corrección del cálculo de victoria cuando la política evaluada controla múltiples bandos en coalición (`winner in eval_candidates`), resolviendo el artefacto de `mettete_i_piedi_terra_1` donde 24 victorias del defensor alemán `GE` (85.7% de WR real) se computaban erróneamente como 0.0% por esperar `IT`.

- **Estandarización del Protocolo MLOps Safeguard en Informes de Ejecución (`docs/informes_ejecucion/`)**:
  - Integración de 5 salvaguardas obligatorias en la cabecera canónica (`00_CABECERA_INFORMES_EJECUCION.md`) y en el Informe 01 (`INFORME_01_CURRICULUM_Y_AUDITORIA_TACTICA.md`) para auditar formalmente coaliciones, simetría en auto-juego, agregación multi-perfil, ancla ELO y correspondencia pasos-turnos antes de declarar regresiones.

- **Visualizador de Linaje MLOps Condicional y 100% en Inglés (`CurriculumLineageFlow.tsx`)**:
  - Reemplazo del badge estático `★ PROMOVIDO` por indicación dinámica real: `★ PROMOTED` (verde) ante `PASS` / `promote`, y `⚠ GATE FAILED (HOLD)` (rojo/ámbar) ante fallos en los criterios del gate.
  - Traducción integral al inglés de todos los textos, tooltips, etiquetas de conectores e hitos de progresión táctica.

- **Exportador ONNX y Visualización de Topología en Netron (`mlops/reporting/onnx_exporter.py`, `NetworkTopologyTab.tsx`)**:
  - Módulo especializado de exportación a formato abierto ONNX (`onnx 1.23.2`) mediante trazado dinámico PyTorch (`torch.onnx.export`).
  - Exportación modular desacoplada en 3 grafos computacionales independientes:
    1. `initial_inference.onnx`: Red de Representación + Cabezas Iniciales de Política, Valor y las 4 Cabezas Tácticas Auxiliares (`threat_map`, `attrition`, `morale`, `final_ownership`).
    2. `recurrent_inference.onnx`: Red Dinámica Recurrente (predicción de siguiente estado latente + recompensa) + Cabezas de Política y Valor.
    3. `representation_tower.onnx`: Backbone puro de extracción de características espaciales desde el tensor de observación $(B, 38, 16, 16)$ al espacio latente $(B, 48, 16, 16)$.
  - Endpoints REST en el visor MLOps (`POST /api/model/export-onnx`, `GET /api/model/export-onnx`, `GET /api/model/download-onnx`).
  - Panel de control interactivo en la pestaña de Topología del Dashboard MLOps con botón de exportación bajo demanda, descarga directa de los 3 archivos `.onnx` y acceso directo a Netron.app para auditoría e inspección visual de capas, pesos y flujos tenoriales.
  - Corrección oportuna de interfaz de usuario: 100% de los textos y controles de la pestaña de Topología de Red migrados al inglés canónico.

v2.5.2 — Orquestación Unificada de Currículum Multi-Etapa, Linaje de Checkpoints E2E, Modulación Intrínseca de VPs y Reseteo Limpio `[CONSOLIDADO 🟢]`

- **Objetivo**: Garantizar un flujo único de extremo a extremo entre el currículum de aceleración hacia VPs (Etapa 1) y el currículum multi-escenario de autojuego progresivo (Etapa 2), encadenando automáticamente los checkpoints promovidos para mantener la genealogía unificada bajo un único modelo en el dashboard MLOps, modulando sensorial y analíticamente el valor intrínseco de cada VP ($v_{vp}/2.0$) y proporcionando una herramienta de archivado seguro y reseteo limpio de entorno.
- **Componentes Consolidados**:
  - **Encadenamiento Automático de Etapas en Master Pipeline (`run_full_multi_curriculum_pipeline.ps1`)**:
    - Parámetro `ChainStageCheckpoints = $true` por defecto (con flag `-NoChain` opcional). Al finalizar la Etapa 1 (`vp_acceleration_v1`), el checkpoint promovido de la Fase 4 (`phase4_tactical_gela_mastery`) se resuelve y se transfiere automáticamente a la Etapa 2 (`self_vs_self_vs_random_2scenarios_v1`) mediante `ExplicitStartCheckpoint`.
    - La Fase 1 de la Etapa 2 genera un overlay de reanudación (`resume_checkpoint`), enlazando el `parent_run_id` directamente al ciclo final de la Etapa 1. Con ello, `build_catalog.py` y DuckDB registran las 11 fases (4 de aceleración + 7 multi-escenario) como un **único modelo genealógico continuo** en el Dashboard MLOps.
  - **Archivado Seguro y Reseteo Limpio (`scripts/reset_curriculum_environment.ps1` y flag `-CleanStart`)**:
    - Herramienta no destructiva que traslada todas las ejecuciones pasadas (`runs/`, `runs_curriculum/`, ciclos y metadatos) a `runs_archive/archive_<timestamp>/`.
    - Regeneración inmediata del catálogo Parquet y la base DuckDB (`reporting.duckdb`) en estado limpio (0 modelos), permitiendo lanzamientos desde cero completamente aislados.
  - **Modulación Intrínseca por Valor de Hexágono VP**:
    - `voec_sim/core/simulator.py`: Exportación formal de `value` y `per_turn` en `vp_hexes`.
    - `voec_sim/core/reward_calculator.py`: Modulación del potencial radial $\Phi(s)$ PBRS por el valor relativo $v_{\text{rel}} = v_{vp}/2.0$, otorgando mayor gradiente y bonificación de control a VPs de alto valor táctico.
    - `agents/efficientzero_v2/core/macro_planner.py`: Selección de VPs objetivo ponderada por ratio $\text{score} = v_{\text{rel}}/(1+d)$.
    - `agents/efficientzero_v2/core/observation.py`: Canales 13, 14 y 18 escalados por v_{rel}.
    - Verificación: 100% de tests unitarios e integración en verde (171/171 tests en `voec_sim/tests/` y `agents/efficientzero_v2/tests/`).

v2.5.1 — Fidelidad Canónica a max_turns, Corrección de Presupuesto en Autojuego y Aceleración de Urgencia de VPs `[CONSOLIDADO 🟢]`

- **Objetivo**: Asegurar que las partidas en autojuego y evaluación nunca se corten prematuramente por un tope arbitrario de pasos, garantizando que el motor de simulación corra hasta el `max_turns` canónico del escenario oficial, erradicar penalizaciones espurias de timeout en el asalto final, y acelerar la agresividad hacia VPs.
- **Componentes Consolidados**:
  - **Presupuesto Seguro en Autojuego (`resolve_effective_step_budget` en `selfplay.py`)**:
    - Actualizado a `scenario_turns * max(units * 2, 20)`. Para un escenario de 8 turnos y 14 unidades (Gela), el presupuesto pasa de un rígido 112 pasos a un margen seguro de 224 pasos.
    - Con esto, las partidas de autojuego nunca sufren corte abrupto ni penalización de timeout (`timeout_penalty = -1.0`), finalizando exclusivamente cuando el motor de reglas canónicas emite `done = True` en `turn > max_turns` tras resolver las condiciones de victoria del wargame.
  - **Exposición de Contrato en Adaptador (`EZV2VOECAdapter.scenario_max_turns()`)**:
    - Delegación directa en `self.sim.scenario_max_turns()`, garantizando acceso limpio al límite de turnos sin depender de introspección reflectiva.
  - **Presupuesto Holgado en Benchmarks (`max_steps: 120` / `180`)**:
    - Actualización integral de los ficheros de benchmark curricular (`assault_bench/configs/benchmark_config.curriculum_2scenarios_*.yaml` y `vp_acceleration_phase4.yaml`), permitiendo que escenarios de 8 y 12 turnos concluyan por evaluación reglamentaria de VPs.
  - **Impulso de Urgencia y Potencial de Asalto**:
    - Elevación de `vp_potential_weight: 1.5` y `vp_urgency_lambda: 1.5` en la configuración estándar de EfficientZero V2 y en los currículums de 2 escenarios, incentivando sprint decidido hacia los hexágonos de victoria antes de la expiración del último turno.

v2.5.0 (Versión Base Anterior) — Andamiaje de Recompensas por Potencial Radial a VPs, Sesgo MCTS y Factor de Presión Temporal `[CONSOLIDADO 🟢]`

- **Objetivo**: Erradicar el colapso de política por dispersión de recompensas en la captura de banderas, acelerar el gradiente de avance hacia Puntos de Victoria (VPs), podar ramas de deambulación estéril y pasividad en MCTS, e imponer urgencia temporal y penalización acumulativa por inacción en terreno expuesto.
- **Componentes Consolidados**:
  - **Calculador Canónico de Potencial Radial y Presión Temporal (`voec_sim/core/reward_calculator.py`, `agents/efficientzero_v2/core/support.py`)**:
    - Implementación estricta de la función de potencial invariante de Ng, Harada & Russell (ICML 1999) modulada por decaimiento temporal (Devlin & Kudenko 2012): $\Phi(s,t) = \Phi_{\text{dist}}(s) \cdot \exp(-\lambda \cdot \tau) + \Phi_{\text{ctrl}}(s)$, con $\tau = t/T$ y $\lambda = 1.2$.
    - Invariancia PBRS dinámica estricta demostrada y testeada: el decaimiento se congela en 1.0 si el bando controla al menos un VP ($N_{\text{ctrl}} \geq 1$), preservando la retención defensiva; mientras que ante 0 VPs controlados el valor cae exponencialmente a medida que transcurren los turnos sin objetivos asegurados.
    - Telescopaje exacto verificado: $\sum_{t=0}^{T-1} \gamma^t F(s_t, t, a_t, s_{t+1}, t+1) = -\Phi(s_0, 0)$ con $\Phi(s_T, T) = 0$.
  - **Penalización Acumulativa por Parálisis en Terreno Abierto (`compute_paralysis_penalty`)**:
    - Penalización cuadrático-lineal para unidades en campo abierto (`cover < 0.35`) que no ocupan un VP amigo, no reducen distancia hacia objetivos ni disparan: $P = -w_{\text{pen}} \cdot (1.0 + 0.5 \cdot \min(4, \text{streak} - 1))$.
    - Reset inmediato del streak de inactividad ante cualquier maniobra de avance hacia VPs, asalto o disparo efectivo.
  - **Sesgo de Búsqueda MCTS y Poda de Deambulación Estéril (`agents/efficientzero_v2/core/macro_planner.py`, `mcts.py`)**:
    - En postura `ASSAULT_VP`, inyección directa del gradiente de distancia y canal `vp_contest_race`: las acciones que cierran distancia hacia el VP reciben un multiplicador progresivo $(1.0 + 0.35 \cdot \Delta d)$, y $+15$ adicional si la intención neuronal predice asalto.
    - Poda selectiva del 40 en ramas de deambulación estéril: acciones de movimiento que incrementan la distancia al VP sin objetivo de combate sufren penalización de prior ($\times 0.60$).
    - Poda del 50 en pasividad estéril: acciones de `WAIT` / `PASS` bajo postura `ASSAULT_VP` son despriorizadas ($\times 0.50$).
    - Soporte nativo en `run_mcts_puct` mediante `vp_race_prior_biasing`, `vp_race_bonus_by_action`, `sterile_action_mask` y registro forense de `sterile_branches_pruned_count` en `MCTSOutput`.
  - **Expansión Sensorial Hexagonal Nativa y Observabilidad MLOps (`observation.py`, `train_engine.py`, `NetworkTopologyTab.tsx`)**:
    - Modulación del Canal 18 (`vp_distance_field`) con $(1.0 + 0.5\tau)$ hacia VPs no capturados.
    - Canal 36 nativo: `turn_urgency_pressure` ($\tau = t/T$ amplificado sobre VPs disputados).
    - Canal 37 nativo: `paralysis_exposure_risk` (hexágonos ocupados por unidades aliadas en campo abierto < 0.35 lejos de VPs).
    - Catálogo canónico de 38 canales en el visualizador MLOps (puerto 8778).
  - **Trazabilidad Forense en Autojuego (`agents/efficientzero_v2/core/selfplay.py`)**:
    - Integración de `vp_potential_component` y `paralysis_penalty` en el cálculo de recompensas (`reward_shaping.vp_potential_weight`).
    - Registro de métricas canónicas en `sample.info`: `vp_potential_reward_component`, `vp_potential_phi_before`, `vp_potential_phi_after`, `vp_potential_delta`, `turn_urgency_factor`, `paralysis_penalty` y `mcts_sterile_branches_pruned`.
  - **Coordinación de Escuadras para Asalto a VPs (`macro_planner.py`, `close_combat_resolver.py`, `close_combat_context.py`)**:
    - **Base de Fuego (*Base of Fire*)**: Escuadras de apoyo pesado (`ROLE_FIRE_SUPPORT`, ej. MMG `.30cal`, `.50cal`, morteros) que fijan o suprimen defensores en o adyacentes a VPs reciben un boost de prior MCTS de $\times 1.35$.

        - **Elemento de Maniobra y Asalto Coordinado**: Escuadras de asalto (`ROLE_SPEARHEAD` / `ROLE_MANEUVER`) que asaltan a un defensor previamente suprimido por la base de fuego reciben un bonus coordinado de $\times 1.40$ (y $\times 1.20$ con escuadra amiga adyacente de apoyo).
        - **Prohibición de Asalto Frontal Suicida en Solitario**: Asaltos frontales en solitario contra defensores no suprimidos sufren penalización de prior de -80 ($\times 0.20$) y son clasificados como estériles en `is_sterile_action()` , podándolos en el árbol de búsqueda MCTS PUCT.
        - **Regla Táctica CC-006**: Si una carga frontal en solitario se ejecuta contra un defensor alerta no suprimido, el atacante pierde su dado de combate más débil en Ronda 1 por fuego defensivo (`_apply_cc006_unsuppressed_solo_frontal_penalty`).
    - **Currículum Progresivo Acelerado de Captura de VPs (`curriculums/vp_acceleration/` , `trainer.py`)**:
        - **4 Fases Progresivas con Escenarios Canónicos Dedicados**:
            1. `vp_phase1_unoccupied_3hex.json` : Sprint puro de aproximación y captura de VP no ocupado a 3 hexágonos de distancia, validando el gradiente radial de potencial $\Phi(s)$ e invarianza PBRS.
            2. `vp_phase2_defended_suppressed.json` : Asalto coordinado contra VP defendido por infantería estática en supresión inicial, entrenando la base de fuego (`ROLE_FIRE_SUPPORT` con mortero de 81mm) y la penetración de punta de lanza (`ROLE_SPEARHEAD`).
            3. `vp_phase3_race_2v2.json` : Carrera competitiva simultánea 2 vs 2 unidades hacia VPs neutrales centrales, afianzando la postura `ASSAULT_VP` y el factor de urgencia temporal.
            4. `battaglia_cittadina_2_1.json` : Escenario táctico completo (Gela) con 14 unidades, 5 VPs y fortificaciones urbanas. **Hito de éxito:** Victoria ≥ 75 contra bot heurístico y disputa equilibrada contra humano en < 15 turnos.
        - **Punto de Entrada Canónico** `trainer.py` : Interfaz de entrenamiento raíz 100% nativa y exclusiva para EfficientZero V2 (cero retrocompatibilidad MuZero), delegando en `agents.efficientzero_v2.train.train_efficientzero_v2` .
        - **Orquestación y Ciclos de Benchmark**: Esquemas `.cycle.json` , configuraciones de benchmark en `assault_bench/configs/` y ejecutor PowerShell automatizado `run_all_phases.ps1` .
    - **Certificación de Pruebas Unitarias y Contratos (319 Tests en Verde)**:
        - `voec_sim/tests/test_reward_calculator.py` : 9 tests verificando potencial de distancia, bonus de control, decaimiento exponencial por urgencia temporal, invarianza telescópica de Devlin & Kudenko, y acumulación/reset de penalización de parálisis.
        - `agents/efficientzero_v2/tests/test_vp_race_mcts_biasing.py` : 2 tests certificando sesgo prior, poda del 40% en MCTS y preservación de visitas.
        - `agents/efficientzero_v2/tests/test_squad_coordination_assault.py` : 5 tests certificando Base of Fire ($\times 1.35$), asalto a defensor suprimido ($\times 1.40$), penalización/poda de asalto en solitario sin supresión, apoyo combinado multi-escuadra y pérdida de dado CC-006.
        - `agents/efficientzero_v2/tests/test_vp_acceleration_curriculum.py` : 4 tests certificando las 4 fases progresivas, carga de escenarios en VOEC, delegación en `trainer.py` y definiciones de ciclos.
        - `agents/efficientzero_v2/tests/test_doctrine_sensory_contract.py` : 6 tests certificando proyección 2D de roles y perfiles tácticos, canales 32 a 35, y canales 36 y 37 de presión temporal y riesgo de parálisis.
        - 319/319 tests globales ejecutados al 100% en verde (128 tests de EfficientZero V2, 88 tests de Assault Model, 31 tests de VOEC Sim y 72 tests de Assault Bench).

v2.4.5 — Evaluación y Verificación en Currículum (Fase 5) `[CONSOLIDADO 🟢]`

- **Objetivo**: Desarrollar el Evaluation Harness canónico (`assault_bench/eval.py`), formalizar las métricas de alineación doctrinal por rol (`recon_survival_rate` , `fire_support_stationarity` , `spearhead_vp_capture_rate` , `doctrine_adherence_rate` , `doctrine_mutation_rate`), verificar la paridad táctica frente a baselines heurísticos e integrar los guardrails doctrinales en los currículums de entrenamiento.
- **Componentes Consolidados**:
    - **Harness de Evaluación Canónico (`assault_bench/eval.py`)**: Interfaz CLI y programática para evaluar checkpoints de EfficientZero V2 (`evaluate_checkpoint`), escenarios aislados y fases de currículum (`verify_curriculum_phase_doctrine`), con emisión de reportes forenses estructurados.
    - **Motor de Métricas Doctrinales (`assault_bench/doctrine_metrics.py`)**:
        - `compute_doctrine_role_metrics` : Cálculo exacto de supervivencia de reconocimiento (`recon_survival_rate`), estacionariedad de apoyo de fuego (`fire_support_stationarity`), capturas por punta de lanza (`spearhead_vp_capture_rate`), adherencia a directivas de rol (`doctrine_adherence_rate`) y frecuencia/motivos de mutación (`doctrine_mutation_rate`).
        - `aggregate_doctrine_kpis` : Agregación ponderada y congruente a través de múltiples episodios y semillas sin división por cero.
        - `verify_doctrine_parity` : Verificador formal de paridad y cumplimiento de umbrales doctrinales (`PASS` / `FAIL`).
    - **Integración en Telemetría y Contratos de Benchmark (`assault_bench/`)**: Inyección automática de `doctrine_eval_kpis` en `phase29_metrics.py` , `phase29_contract.py` y `benchmark_finalize.py` , habilitando la trazabilidad de doctrina en DuckDB/Parquet.
    - **Guardrails Doctrinales en Currículum (`curriculums/self_vs_self_vs_random_2scenarios_v1/curriculum.yaml`)**: Incorporación de umbrales doctrinales en los gates de promoción (`min_recon_survival_rate: 0.40` , `min_fire_support_stationarity: 0.45` , `min_doctrine_adherence_rate: 0.35`).
    - **Suite de Pruebas Unitarias e Integración (`assault_bench/tests/test_doctrine_eval.py`)**: 6 nuevos tests dedicados que verifican cálculos, agregaciones, paridad y contratos (228 tests globales al 100% en verde).

v2.4.4 — Observabilidad MLOps y Telemetría Forense (Fase 4) `[CONSOLIDADO 🟢]`

- **Objetivo**: Integrar la telemetría forense completa de perfiles doctrinales, mutaciones dinámicas de roles y modulaciones de prior MCTS en el flujo de autojuego, datasets de benchmark y visualizador interactivo MLOps (puerto 8778).
- **Componentes Consolidados**:
  - **Cálculo Analítico de Modulación Prior (`agents/efficientzero_v2/core/macro_planner.py`)**: Función `compute_action_boost` para calcular con exactitud determinista el multiplicador de prior asignado a cada acción según el rol doctrinal activo y la proximidad a Puntos de Victoria.
  - **Inyección en Bucle de Autojuego y Transiciones (`agents/efficientzero_v2/core/selfplay.py`, `timeline_exporter.py`)**: Trazabilidad completa en `sample.info` y en el exportador XAI con `role_assigned`, `role_is_mutated`, `role_mutation_reason`, `prior_boost_applied`, `badge_label` y snapshot integral de escuadras aliadas y enemigas.
  - **Modelos y Pipelines de Benchmark (`assault_bench/`)**: Enriquecimiento de `unit_snapshot_dict`, `StepSignalData`, `episode_runner_impl.py` y `episode_trace_row_builder.py` con campos doctrinales canónicos sin penalización de rendimiento.
  - **Frontend Interactivo MLOps (React + Vite, Puerto 8778)**:
    - Mapeo canónico de los 8 roles tácticos (`SPEARHEAD`, `FIRE_SUPPORT`, `CLOSE_DEFENSE`, `RECON`, `OBJECTIVE_DEFENDER`, `FLANK_SCREEN`, `TACTICAL_RESERVE`, `RECOVER`) con paleta semántica militar.
    - Badges visuales de rol activo, distintivo de mutación contextual (`⚡ MUTADA: REASON`) y píldoras cuantitativas de sesgo prior (`+X% π`) tanto en la tarjeta de escuadra activa como en los rosters de unidades aliadas y del oponente.
    - Nueva tarjeta de auditoría forense doctrinal (*Selected Unit Forensic Doctrine Card*) en el visor de replay.
  - **Suite de Pruebas Unitarias Forenses (`agents/efficientzero_v2/tests/test_doctrine_telemetry_forensics.py`)**: 4 nuevos tests dedicados que certifican la preservación de campos y congruencia de prior boost (222 tests globales al 100% en verde).

v2.4.3 — Mutación Dinámica Doctrinal y Modulación Prior MCTS (Fase 3) `[CONSOLIDADO 🟢]`

- **Objetivo**: Implementar la lógica contextual de mutación dinámica de roles en el planificador macro jerárquico (`MacroStrategyPlanner`) y modular los priors raíz del árbol MCTS en un $+35$ manteniendo libertad de exploración emergente.
- **Componentes Consolidados**:
  - **Dataclass `MacroPlan` Enriquecido (`agents/efficientzero_v2/core/macro_planner.py`)**: Trazabilidad completa con `unit_mutation_reasons`, `unit_base_roles`, `unit_mutated`, `get_mutation_reason()`, `get_base_role()` y `is_mutated()`.
  - **Reglas Jerárquicas de Mutación Contextual (`compute_macro_plan`)**:
    - **Regla 1 (Presión de Victoria / Turno Tardío)**: Unidades con `allow_spearhead_if_vp_deficit: true` mutan a `ROLE_SPEARHEAD` ante déficit neto de VPs o en el último tercio de partida (`VP_DEFICIT_PRESSURE`).
    - **Regla 2 (Degradación de Moral y Supervivencia)**: Unidades suprimidas, bajo umbral de HP o con < 50 fuera de BUA mutan a `ROLE_RECOVER` (`MORALE_OR_HEALTH_DEGRADATION`).
    - **Regla 3 (Aislamiento / Flanqueo Inmediato)**: Armas de apoyo pesado en rol `FIRE_SUPPORT` con enemigos adyacentes (≤ 1 hex) y sin escolta cercana mutan a `ROLE_CLOSE_DEFENSE` (`FLANKED_CLOSE_DEFENSE`).
  - **Modulación Prior MCTS (+35% Boost)**:
    - Boost adaptativo en raíz MCTS (+35%) para acciones afines al rol activo (`ROLE_SPEARHEAD`, `ROLE_FIRE_SUPPORT`, `ROLE_CLOSE_DEFENSE`, `ROLE_RECON`, `ROLE_OBJECTIVE_DEFENDER`, `ROLE_FLANK_SCREEN`, `ROLE_TACTICAL_RESERVE`, `ROLE_RECOVER`).
    - Normalización estricta de la distribución de probabilidad a 1.0.
  - **Suite de Pruebas Unitarias (`agents/efficientzero_v2/tests/test_macro_planner.py`)**: 8 tests en verde verificando las 3 reglas de mutación, preservación de reserva de Dunnigan y normalización de priors (218 tests globales al 100% en verde).

v2.4.2 — Expansión Sensorial y Mapeo 2D Doctrinal en `voec_sim` y EZv2 (Fase 2) `[CONSOLIDADO 🟢]`

- **Objetivo**: Integrar los perfiles doctrinales y los rangos óptimos de combate en el plano sensorial continuo de simulación e inferencia, proyectando huellas 2D de efectividad operativa sobre el tensor de observación de EfficientZero V2.
- **Componentes Consolidados**:
  - **Puente Sensorial Autorizado (`voec_sim/core/simulator.py`, `voec_sim/contracts/types.py`)**: `UnitSnapshot` incorpora `doctrine_role` y `doctrine_profile`; `spatial_features()` expone `unit_doctrine_roles` y `unit_optimal_ranges`.
  - **Adaptador de Auto-Juego (`agents/efficientzero_v2/core/adapter_voec.py`)**: Exposición transparente de perfiles y roles activos en `EZV2Observation.units`.
  - **Proyección Espacial 2D y Roles Doctrinales (`agents/efficientzero_v2/core/observation.py`)**:
    - Codificación canónica continua de roles (`DOCTRINE_ROLE_CODES`: `SPEARHEAD` 1.0, `FIRE_SUPPORT` 0.75, `RECON` 0.50, `SCREEN` 0.25, `COMMAND` 0.10).
    - Proyección continua del mapa 2D de alcance óptimo de combate en los canales 24 (`ally_direct_range_norm`) y 25 (`enemy_direct_range_norm`) amortiguado por bloqueo de LOS y cobertura.
    - Activación de sesgo doctrinal de apoyo por fuego en canales 8 y 9 (`ally_heavy_mg_support` / `enemy_heavy_mg_support`).
    - Soporte para tensores extendidos de hasta 36 canales (`ally_doctrine_role`, `enemy_doctrine_role`, `ally_optimal_range_2d`, `enemy_optimal_range_2d`).
  - **Suite de Verificación Dedicada (`agents/efficientzero_v2/tests/test_doctrine_sensory_contract.py`)**: 5 nuevos tests exhaustivos, batería global de 215 tests al 100% en verde.

v2.4.1 — Perfilado Doctrinal de Unidades: Catálogo y Contratos Tipados (Fase 1) `[CONSOLIDADO 🟢]`

- **Objetivo**: Dotar a cada una de las 19 unidades del catálogo canónico de un perfil doctrinal (`doctrine_profile`) tipado formalmente, desacoplando capacidades físicas duras de directivas de rol táctico sin colapsar la creatividad de la red.
- **Componentes Consolidados**:
  - **Contrato Tipado `DoctrineProfile` (`assault_model/units/unit_type.py`)**: Dataclass inmutable con validación estricta de `default_role`, `optimal_engagement_range`, `preferred_terrain`, `cohesion_weight`, `fallback_threshold_hp` y reglas de mutación.
  - **Inyección en Catálogo Canónico (`assets/catalogs/unit_catalog.json`)**: Cobertura al 100% de las 19 escuadras y equipos de infantería (Reconocimiento, Armas de Apoyo, Asalto/Punta de Lanza y Pantalla).
  - **Suite de Pruebas Unitarias (`assault_model/tests/test_unit_doctrine_profile.py`)**: 100% de cobertura y validación de serialización roundtrip sin regresiones.

v2.4.0 — Gobernanza Zero-Mock, Desacoplamiento y Explicabilidad Causal `[CONSOLIDADO 🟢]`

- **Objetivo**: Erradicar el 100% de inferencias espurias, mocks y números inventados en frontend y reporting; desacoplar métricas base de representaciones visuales; integrar explicabilidad causal (XAI) al perfilado táctico.
- **Componentes Consolidados**:
  - **Purga Integral Zero-Mock / Zero-Inference**: Eliminación de porcentajes ficticios (`NetworkTopologyTab.tsx`), erradicación de turnos y macro-posturas adivinadas (`MatchReplayTab.tsx`), y eliminación de strings fijos de escenarios o facciones (`VpSummaryTab.tsx`, `replay_api.py`). Los componentes muestran estrictamente datos reales o `-` / `N/A`.
  - **Inventario Jerárquico Canónico de Métricas (`docs/13_INVENTARIO_CANONICO_METRICAS.md`)**: Taxonomía en 4 niveles (Motor, MCTS, Red Neuronal, Benchmarks MLOps) con trazabilidad matemática unívoca contra columnas de Apache Parquet y DuckDB.
  - **Marco Canónico de Explicabilidad Causal y Perfilado Táctico (`docs/00_PREMISAS_DEL_PROYECTO.md` §13)**: Directivas de XAI y Causal Profiling para auditar el razonamiento táctico del agente antes de su despliegue competitivo.

---

4. Fases Históricas Archivadas (Ciclos v1.0.0 a v2.3.0) (Ciclos v1.0.0 a v2.3.0)

v2.3.0 — Innovaciones Tácticas Avanzadas y Transferencia Científica SOTA `[ARCHIVADO 🟢]`

- **Objetivo**: Transferir avances de KataGo y literatura de wargames hexagonales (Palma et al. Feb 2025).
- **Componentes Consolidados**:
  - **Cabeza Densa *Final Territory Ownership Map* ($1 \times H \times W$, KataGo — Wu 2020)**: Predicción convolucional del control territorial final del mapa inyectada en `aux["final_ownership_pred"]` en inferencia inicial y recurrente.
  - **Asaltos Combinados Secuenciales (Palma et al. 2025, Sec. 3.3)**: Compromiso coordinado de escuadras adyacentes (`commit_support_unit`) que aportan dados verdes de infantería o rojos de MMG/Mortero en combates cuerpo a cuerpo.
  - **Quality Gates Auto-Calibrados por Escenario con Two-Hot Asimétrico Exponencial**: Boost de resolución exponencial en torno a cero (`sign(x) * |x|^0.75`) para discriminar ventajas tácticas de desgaste y puertas de promoción calibradas a la capacidad de VPs del escenario.

v2.2.0 — Engine de Alto Rendimiento y Streaming Binario `[ARCHIVADO 🟢]`

- **Objetivo**: Eliminar cuellos de botella de simulación en CPU y latencia de serialización en el visor web MLOps.
- **Componentes Consolidados**:
  - **Vectorización Ray-Caster y Dijkstra (Sección 6.1)**: Interpolación lineal fraccional vectorizada en NumPy/Numba (`_vectorized_fractional_ray_samples`), matrices tensoriales precalculadas de visibilidad (N, N) y adyacencia $(N, 6)$. Rendimiento medido: $4{,}924\,\text{steps/s}$ ($> 2.4\times$ sobre el objetivo de $2{,}000\,\text{steps/s}$).
  - **Clonado Inmutable Copy-on-Write (CoW, Sección 6.2)**: `CoWHexStateMap` y `GameStateDelta` que comparten `_base` en $O(1)$ con `0` objetos copiados inicialmente al ramificar. Rendimiento medido: $> 109{,}000\,\text{clones/s}$ con aislamiento mutacional del 100%.
  - **Control Territorial Incremental (Sección 6.3)**: Índice espacial invertido recalculando únicamente el radio $R \leq 3$ de unidades alteradas ($O(k \cdot R^2)$ amortizado).
  - **Streaming Binario MessagePack (Sección 6.5)**: Codec RFC-8949 binario en WebSocket (`/ws/game?format=msgpack`) y REST. Reducción > 85 en ancho de banda y latencia $< 5\,\text{ms}$.

v2.1.0 — Búsqueda Eficiente Gumbel / ACT, Compuerta de Canales e Invarianza FCN `[ARCHIVADO 🟢]`

- **Objetivo**: Acelerar drásticamente la generación de autojuego, filtrar planos sensoriales irrelevantes y habilitar despliegue en mapas de tamaño arbitrario.
- **Componentes Consolidados**:
  - **Búsqueda Gumbel MuZero Search** (*Sequential Halving* sin reemplazo) y presupuesto de simulación adaptativo **ACT** (`compute_adaptive_simulations`) en `mcts.py`, guiado por la entropía de Shannon de la política $H(\pi)$ y la dispersión de valores Q. Aceleración del autojuego en ~ 60.
  - **Compuerta Dinámica de Canales (`InputChannelGate`)**: Mecanismo Squeeze-and-Excitation (SE) que analiza los 32 canales axiales de observación y suprime dinámicamente canales muertos según el escenario.
  - **Invarianza Espacial FCN Zero-Shot**: Arquitectura puramente convolucional con `_adapt_spatial` y `AdaptiveAvgPool2d`, desplegable en mapas 16×16, 32×32 y 64×64 sin alterar pesos ni requerir reentrenamiento.
  - **Dinámica Recurrente Weight-Tied**: Dinámica g_θ iterada k pasos con conexión **Recall** y pérdida de progresión **Progressive Loss** ($\mathcal{L}_{\text{prog}}$) con 0 parámetros extra.

v2.0.0 — Comprensión Táctica Multitarea, Balanceo PCGrad y Jerarquía HRL `[ARCHIVADO 🟢]`

- **Objetivo**: Dotar a la red de criterio militar preventivo mediante 10 cabezas auxiliares simultáneas sin interferencia destructiva de gradientes, desacoplando la doctrina macro de la micro-maniobra.
- **Componentes Consolidados**:
  - **10 Cabezas Auxiliares Tácticas**: `threat_map` (2D), `attrition`, `morale`, `objective_intent`, `multi_vp`, `combat_outcome`, `valid_actions_mask`, `action_category`, `final_ownership`, `value_target`.
  - **Balanceador de Gradientes PCGrad** (`loss_balancer.py`): Proyección ortogonal de gradientes opuestos $g_i \leftarrow g_i - \frac{g_i \cdot g_j}{|g_j|^2} g_j$ cuando $g_i \cdot g_j < 0$, asegurando convergencia armónica de las 10 cabezas.
  - **MacroStrategyPlanner (HRL Nivel 1)**: Conexión neuronal directa a `objective_intent_logits` y `multi_vp_pred`, emitiendo posturas globales (`ASSAULT_VP`, `DEFENSIVE_HOLD`, `ZONE_SUPPRESS`) y asignando roles doctrinales (`SPEARHEAD`, `FIRE_SUPPORT`, `OBJECTIVE_DEFENDER`, `FLANK_SCREEN`, `TACTICAL_RESERVE`, `RECOVER`) basados en tratados clásicos (Dunnigan, Sun Tzu, Henry Hyde).

v1.5.0 — Estabilidad Numérica, Distribución de Valor y Geometría Hexagonal `[ARCHIVADO 🟢]`

- **Objetivo**: Erradicar el colapso de gradiente provocado por la regresión escalar MSE directa en partidas con recompensas dispersas y eliminar sesgos anisotrópicos de la cuadrícula cartesiana.
- **Componentes Consolidados**:
  - **Two-Hot Discreto** (61 átomos en $[-30, 30]$) invertible con contracción simétrica logarítmica **Symlog** ($\text{sign}(x)\ln(|x|+1)$) en `agents/efficientzero_v2/core/support.py`.
  - **HL-Gauss** (*Histogram Loss with Gaussian Targets*, DeepMind 2024) que proyecta objetivos continuos mediante discretización suavizada $\mathcal{N}(y, \sigma^2)$ para estabilizar el entrenamiento de valor y recompensas intermedias.
  - `HexagonalMaskConv2d` y `HexagonalDepthwiseSeparableConv2d` en `network_base.py`, anulando sesgos de diagonales irreales en la topología hexagonal.

v1.0.0 — Cimentación del Motor VOEC y Reglas Wargame Base `[ARCHIVADO 🟢]`

- **Objetivo**: Implementar el modelo físico y doctrinal del wargame táctico sobre tablero hexagonal.
- **Componentes Consolidados**:
  - Coordenadas axiales (q, r) con transformaciones de distancia y adyacencia en `assault_model/map/hex_utils.py`.
  - Catálogo de unidades y balística de combate con resolución de tiradas de dados multiespectro (Rojo, Azul, Verde, Amarillo) y tiradas de salvación en `assault_model/combat/`.
  - Algoritmo de Línea de Visión (LOS) discreto e interpretación de elevación y cobertura de terreno en `assault_model/combat/line_of_sight.py`.
  - Simulación estocástica y modelo sensorial VOEC en `voec_sim/core/simulator.py`.

🗺️ 19. Hoja de Ruta Evolutiva (Roadmap Maestro)

Visión ejecutiva, matriz maestra global, orden de ejecución y changelog canónico de versiones.

11. Hoja de Ruta Evolutiva: Assault Architecture Roadmap (2026–2027)

Estado: Canónico, Unificado y Versionado (Octubre 2026)
Ámbito: Documento final que unifica toda la evolución técnica, científica, de rendimiento y normativa de la plataforma **Assault**: Motor de IA `agents/efficientzero_v2/`, Engine Táctico `assault_model/` + `voec_sim/`, Telemetría MLOps/UI y Fidelidad al Reglamento Oficial.
Directiva Estricta: MuZero clásico está 100% decomisionado. Todo desarrollo, autojuego, inferencia y MCTS opera exclusivamente sobre `EfficientZeroV2` (`agents/efficientzero_v2/`) y su evolución jerárquica `Hierarchical EfficientZero (H-EZ)`.
Política Estricta de Idiomas: Código fuente e interfaz de usuario (UI, Dashboards, visualizadores) 100% en **Inglés**. Interacción con el usuario y documentación técnica canónica (docs/) en **Español**. Toda modificación de código/UI debe corregir y traducir al inglés cualquier texto residual en español (sin barrido masivo retroactivo).
Versión Base de la Plataforma: **v3.0.0** (Arquitectura Hierarchical EfficientZero / H-EZ & Salto a Mando Estratégico de Misión).
Regla Sagrada de Versionado del Roadmap: Toda nueva implementación, tarea o hito **DEBE registrar explícitamente en qué versión / subversión de la aplicación se implementó o consolidó** (ej. `[CONSOLIDADO 🟢 v2.5.51]`, `[CONSOLIDADO 🟢 v3.0.0]`). Queda formalmente prohibido marcar tareas como consolidadas sin su versión de entrega correspondiente.

---

1. Visión Ejecutiva y Diagnóstico Doctrinal: El Salto a Hierarchical EfficientZero (Era v3.0)

[!IMPORTANT] **AXIOMA ESTRATÉGICO CENTRAL DEL PROYECTO (OCTUBRE 2026 – ERA v3.0)**
"El agente ha aprendido a ser un buen soldado táctico individual, pero no puede ser un Comandante de Misión eficaz si una sola red plana debe resolver 170 micro-pasos de combate simultáneamente."

Diagnóstico Empírico de Rendimiento:
A pesar de contar con canales temporales, PBRS invariante (Ng et al. 1999) y desensibilización de brecha, la telemetría forense en `runs/evals/` (`battaglia_cittadina_2_1`) revela cuatro cuellos de botella fundamentales:

1. **Deriva Latente en Imaginación Profunda:** El modelo de mundo recurrente g_θ acumula error a partir de $k > 3$ pasos (`reanalysis_target_drift = 7.83`), alucinando peligro letal al cruzar zonas abiertas y forzando al MCTS a podar ramas de asalto.
2. **Desajuste de Horizonte Temporal:** En un episodio de 172 micro-pasos, el factor de descuento $\gamma = 0.997$ penaliza de forma desproporcionada la recompensa terminal de VPs a 120 pasos frente al coste inmediato de recibir fuego de reacción en el paso 1.
3. **Colapso de Entropía en MCTS:** Con márgenes de decisión de 8.82 y entropía 0.0, el prior neuronal inicial aplasta la exploración de acciones agresivas en la raíz.
4. **Desbalance de Gradientes Auxiliares:** Las pérdidas de supervivencia y amenaza superan en más de $14\times$ a la pérdida del objetivo de misión (`0.57` vs `8.33`).

La Solución Arquitectónica (Era v3.0): Hierarchical EfficientZero (H-EZ)
Desacoplar la toma de decisiones en **dos cerebros neuronales acoplados que aprenden solos por autojuego**:

- **El Comandante Estratégico (Manager Network):** Opera cada **Turno de Juego** ($T \leq 8$), emite vectores de meta espacial y doctrinal g_t sobre el mapa e internaliza la recompensa global de victoria con $\gamma = 0.95$ sin miopía temporal.
- **El Operador Táctico (Worker – EZv2):** Resuelve la micro-cinemática local en cada paso mediante MCTS de horizonte corto ($k \leq 3-4$) condicionado por g_t mediante capas FiLM (*Feature-wise Linear Modulation*), eliminando por completo la deriva latente recurrente.

---

2. Matriz Maestra de Seguimiento y Orden de Ejecución Global (v3.0.0)

A continuación se detalla el orden secuencial de ejecución técnica para el ciclo v3.0, estableciendo la arquitectura jerárquica como el vector prioritario absoluto:

2.1 Bloque 1: Preparado para Ejecución (Acciones Prioritarias Inmediatas) 🚀

Tareas completamente especificadas, con dependencias resueltas y listas para despliegue directo en el motor. Toda tarea consolidada y verificada se traslada inmediatamente al Bloque 4 para mantener las tareas pendientes claras y focalizadas.

Paso	Línea	Tarea	Prioridad	Enfoque Principal	Estado
1	Línea 6	6.1 Protocolo de Sub-Metas Espaciales y Modulación FiLM del Worker (`SpatialGoalFiLMBridge`)	P0	Parametrización del espacio de metas $(q, r, \text{posture})$ y condicionamiento de cabezas de valor y política en `EZV2Model` con Zero-Init identity preservation.	🚀 **Foco Crítico Inmediato**
2	Línea 6	6.2 Red Convolucional del Comandante Estratégico (`ManagerNetwork`)	P0	Red de nivel superior gobernada por turnos de juego ($T \leq 8$) que proyecta mapas de asignación táctica sobre el tablero alimentada de features IPB 5D.	🚀 **Planificado Inmediato**
3	Línea 6	6.3 Bucle de Co-Entrenamiento en Autojuego Autoritativo y Telemetría MLOps	P0	Entrenamiento conjunto de Manager y Worker con recompensas intrínsecas direccionales de FeUdal Networks, telemetría DuckDB y visualización de flechas operacionales en Replay (:8778).	🚀 **Planificado Inmediato**

2.2 Bloque 2: En Progreso Activo 🟡

> Módulos con desarrollos en curso e hitos parciales ya verificados.

Paso	Línea	Tarea	Prioridad	Enfoque Principal	Estado
4	Línea 3	3.5.5 Certificación de Victoria Jerárquica en `assault_bench`	P1	Batería de validación automatizada de 100 partidas vs Random Side A/B evaluando el agente jerárquico v3.0 (Win Rate ≥ 80, Loss Rate ≤ 10).	🟡 En Preparación

2.3 Bloque 3: Planificado (Ordenado por Prioridad P1 → P2) 🟣

> Mejoras evolutivas, optimizaciones de red y expansiones reglamentarias revisadas tras la adopción de H-EZ.

Paso	Línea	Tarea	Prioridad	Enfoque Principal	Estado
5	Línea 3	3.2 Fase A: Cavar (*Dig-In*), Humo Táctico y Transporte de Tropas	P1	Ampliación del repertorio de micro-acciones físicas del Worker: colocación de foxholes, pantallas dinámicas de humo y embarque en camiones/semiorugas.	🟣 Planificado
6	Línea 3	3.3 Fase B: Iniciativa Dinámica y Apoyo Fuera de Mapa (TAS/OAS)	P2	Ingesta de CPs y soporte aéreo/artillero gobernado por el Comandante Estratégico (Manager) en Turno 0.	🟣 Planificado
7	Línea 3	3.4 Fase C: Pipeline Automatizado de Ingesta PDF → JSON (36 Escenarios)	P2	Extractor masivo de tablas OOB, mapas y VPs desde el reglamento oficial para completar el catálogo de 38 escenarios.	🟣 Planificado
8	Línea 5	5.1 Mutador Procedural Asimétrico con Clumping Geográfico Natural	P2	Generación de tableros procedurales realistas con correlación de elevación y bosques densos para evitar sobreajuste topográfico.	🟣 Planificado
9	Línea 5	5.2 Validación Zero-Shot en Tableros Masivos (32×32 y 64×64)	P2	Despliegue del Comandante H-EZ dividiendo teatros operacionales gigantes en macro-sectores sin explosión de horizonte para el Worker.	🟣 Planificado

2.4 Bloque 4: Completado y Consolidado (Ordenado por Prioridad P0 → P1) 🟢

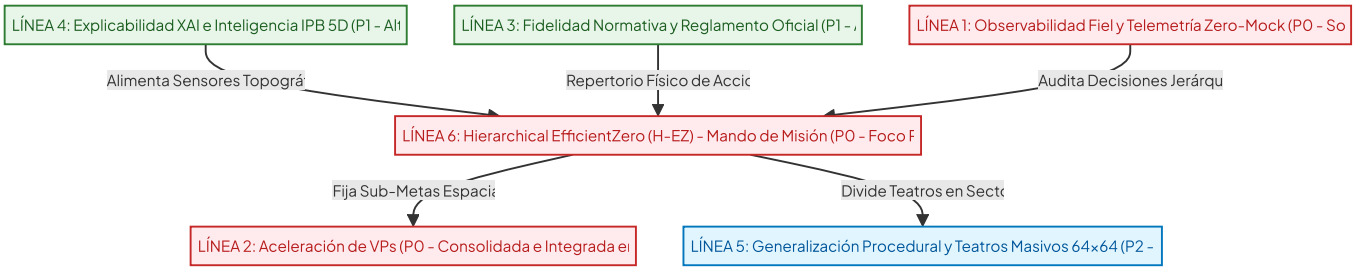
> Hitos y componentes plenamente operativos, verificados con tests y suites de evaluación. Toda tarea finalizada reside canónicamente aquí.

Paso	Línea	Tarea	Prioridad	Enfoque Principal	Estado
0.001	Línea 2 / Red	Multi-Task Model-Based RL (EfficientZero-Multitask / EZ-M, arXiv:2603.01452)	P0	Módulo `TaskConditioningModule`, capas `FiLMLayer` con Zero-Init, búfer particionado `MultiTaskReplayBuffer` con muestreo estratificado y proyección ortogonal de gradientes `multi_task_pcgrad_backward`. Suite 163/163 en verde.	🟢 Consolidado v2.5.51
0.002	Línea 2	3.5.3 Integración Pedagógica del Escenario Canónico 11.1 (`addestramento_fanteria_11_1`)	P0	Escenario oficial 11.1 como Fase 4 en `curriculums/vp_acceleration/curriculum.yaml` (sandbox 3v3 de infantería pura), Fast Bootstrap en Ciclo 1, sincronización con Side B y linaje de 14 fases en MLOps (:8778).	🟢 Consolidado v2.5.50
0.003	Línea 1 / Docs	Saneamiento Sintáctico Mermaid 10.9 (Zero-Bombs) y Enriquecimiento Visual Canónico	P1	Erradicación de errores de sintaxis en grafos Mermaid de red, curación de 52 activos gráficos de alta resolución (`assets/art/`), renderizado nítido en web y compilación a PDF completo de 33.35 MB.	🟢 Consolidado v2.5.49
0.004	Línea 2 / Red	Aceleración Extrema de Entrenamiento y Replay Buffer (Zero Quality Loss)	P0	Codificación directa de representación en consistencia temporal (`representation_encode`), batching unificado de observaciones objetivo K-step en CUDA, memoria fijada DMA y deque $O(1)$. Reducción de 37.01 ms a 25.47 ms por update (1.45× más rápido). Suite 158/158 en verde.	🟢 Consolidado v2.5.49
0.005	Línea 3	3.7.2 Purga de Deuda Técnica Legacy y Unificación Canónica de Assets	P1	Eliminación de carpeta sombra `assault_sim/assets/`, archivado de pila PPO/SB3 en `legacy_archive/`, desacoplamiento limpio de `replay_api.py` hacia `assets/` y unificación canónica de verdad. Suite 160/160 en motor y 158/158 en EZv2.	🟢 Consolidado v2.5.30
0.01	Línea 3	3.7.1 Aceleración de Rendimiento del Motor (Spatial Cache, Dijkstra Occupants, Spotting Snapshots & Set Traits)	P1	Aceleración de 428.8 a 719.7 pasos/s (+67.8% random) y de 454.3 a 844.6 pasos/s (+86.0% secuencial). Cacheo inmutable de metadatos espaciales en `VOECSimulator`, índices O(1) de ocupantes y trait sets precompilados. Suite 160/160 en motor en verde.	🟢 Consolidado v2.5.29
0	Línea 2	2.8 Modulación Dinámica de Priors Doctrinales y Ablación Red Pura	P0	Soporte de modulación continua `macro_modulation_scale` ($\Xi eta_{\text{macro}} \in [0.0, 1.0]$) con decaimiento lineal/coseno. Alternancia limpia entre Red Pura ($\Xi eta = 0.0$) y Doctrinal Shaped ($\Xi eta = 1.0$). Suite de tests en verde (9/9).	🟢 Consolidado v2.5.27
0.05	Línea 4	4.3 Perfilado 5D y Detección Automática de Chokepoints (IPB)	P2	Análisis topográfico Turno-0 que extrae vector 5D por hexágono (*Cover*, *Elevation*, *LOS Exposure*, *Chokepoint*, *VP Utility*), clasifica el tablero en corredores de desenfilada, killzones y base de fuego. Integrado en API MLOps y canvas Replay (:8778).	🟢 Consolidado v2.5.28

Paso	Línea	Tarea	Prioridad	Enfoque Principal	Estado
0.1	Línea 1	1.5 Protocolo de Estabilidad Operativa, Safety Gate de Almacenamiento y Compresión de Replays XAI	P0	Diagnóstico forense de saturación de disco, parada segura de procesos de fondo, Safety Gate pre-flight de 30 GB libres y serialización compacta reduciendo hasta 95% del espacio de replays.	🟢 Consolidado v2.5.25
1	Línea 1	1.4 Catálogo Unificado y Depuración de Esquemas	P1	Contrato canónico de 15 columnas en `decisions_top.parquet`, purga de acumuladores volátiles, saneamiento total de archivos Parquet (69/69 conformes) y Clean Break en DuckDB.	🟢 Consolidado v2.5.11
2	Línea 1	1.3 Validador E2E Frontend ↔ DuckDB	P1	Auditoría automatizada de trazabilidad en TypeScript y Python de las 9 pestañas del dashboard MLOps, garantía Zero-Mock y erradicación del shim MuZero en `replay_api.py`. Suite 15/15 en verde.	🟢 Consolidado v2.5.8
3	Línea 2	2.11 Currículum Asimétrico Ofensivo de Asalto Urbano Puro	P0	Fase especializada de 10 ciclos en modo 100% atacante en `battaglia_cittadina_2_1` con horizonte comprimido ($T = 8$) y penalización severa por timeout. Suite curricular en verde.	🟢 Consolidado v2.5.7
4	Línea 3	3.1 Retirada a Borde Amigo en Desenfilada (Regla 13.3 / §6.4)	P1	Algoritmo determinista priorizando desenfilada, mayor cobertura de terreno/fortificación, vector a Friendly Edge y aumento de distancia al atacante. Suite 112/112 en verde.	🟢 Consolidado v2.5.11
5	Línea 3	3.5 Formaciones Doctrinales, Ingesta de Unidades Especializadas y Rebalanceo OOB	P1	Catálogo oficial de 12 formaciones combinadas, ingesta de zapadores/ingenieros `US_COMBAT_ENGINEERS_43`, rebalanceo simétrico de OOB en Battaglia Cittadina y Mettete i Piedi a Terra.	🟢 Consolidado v2.5.4 / v2.5.13
6	Línea 4	4.2 Perfilador Causal de Partidas y SitRep Forense	P1	Correlación causal matemática entre cabezas tácticas y resultados de partida, generación de SitRep militar estructurado con citas de reglamento RAG y modal interactivo en MLOps (:8778).	🟢 Consolidado v2.5.19
12	Línea 2 / Red	2.7 Inicialización Canónica de Capas Especializada en RL	P1	Esquema formal de inicialización: Política ortogonal ($extgain = 0.01$), Valor ortogonal ($extgain = 1.0$), Kaiming Normal en SiLU y Zero-Init ResNet en segunda normalización.	🟢 Consolidado v2.5.5
13	Línea 2	2.10 Dynamic PBRS Escalado Temporalmente y Desensibilización de Bajas en Brecha	P0	Potencial temporalmente escalado $\Phi(s, t)$, bono de disputa en brecha ($d \leq 2$), atenuación del 60 de bajas en brecha y castigo por timeout. Invariancia telescópica verificada.	🟢 Consolidado v2.5.4
14	Línea 2	2.9 Paquete de Aceleración de Rendimiento de Pipeline (Zero Quality Loss)	P0	Caché de terreno ($1,078\times$), Root PUCT vectorizado, inferencia estática `torch.compile` y test Wald SPRT. Campaña reducida de >10h a <3.5h.	🟢 Consolidado v2.5.3
15	Línea 1	1.1 Test de Conformidad de Esquema de Telemetría	P0	Validador automatizado en CI del 100% de métricas formales contra `docs/13_INVENTARIO_CANONICO_METRICAS.md`. Cero campos no declarados y clean break MuZero.	🟢 Completado y Verificado
16	Línea 3 / 2	3.6 Eliminación de la Parálisis Ofensiva y Colapso de la Brecha Asimétrica	P0	Expansión sensorial a 38 canales, PBRS dinámico Devlin-Kudenko en autojuego, MCTS VP Biasing (64 sims) y balanceo PCGrad.	🟢 Completado y Consolidado
17	Línea 1	1.2 Profiling y Reducción de Sobrecarga de Sondas	P0	Optimización de sondas en bucle caliente reduciendo sobrecarga a < 0.7.	🟢 Completado y Consolidado
18–22	Línea 2	2.1 – 2.5 Paquete Fundacional de Andamiaje de VPs y Paridad Humana	P0/P1	Recompensa de potencial radial (Ng et al. 1999), sesgo MCTS orientado a VPs, urgencia temporal (Devlin & Kudenko 2012), coordinación Base of Fire y currículum 4 fases.	🟢 Completado y Consolidado
23	Línea 2 / Red	2.6 Modernización y Eficiencia Arquitectónica (CoordConv, Bottleneck Policy Head y ECA)	P1	CoordConv ($38 \to 40$ canales), cuello de botella de política `Conv1x1` → `Flatten` → `MLP` (67% menos parámetros) y bloques ECA 1D en red g.	🟢 Completado y Verificado
24	Línea 4	4.4 Perfilado Doctrinal Dinámico y Visibilidad MLOps	P1	Jerarquía 3 capas (Balística, Roles, Emergente), mutación contextual y telemetría de roles en Match Replay (:8778).	🟢 Completado y Consolidado
25–26	Línea 4	4.1 – 4.1.1 Atribución Causal Espacial (IGNH/GradCAM) y Capa Reactiva de Cabezas Tácticas	P1	Integrated Gradients para hexágonos, Grad-CAM sobre tronco de representación, botón reactivo Threat Head y tarjetas forenses de cabezas tácticas.	🟢 Consolidado v2.5.10 / v2.5.11
27–39	[Infraestructura MLOps & Train]	Paquete Integral de Rendimiento, Resiliencia IPC, Auto-Curación y Multiproceso	P0/P1	Concurrencia multiproceso Zero-GIL (24 workers con IPC shared-memory), inferencia CUDA por lotes, Numba JIT en PUCT MCTS ($253\times$), Tensor Cores TF32, eliminación ultrarrápida en catálogo MLOps y auto-recuperación de procesos.	🟢 Consolidado v2.5.12 – v2.5.38

3. Líneas Estratégicas de Progreso (Roadmap Priorizado v3.0 / 2026–2027)

El desarrollo estratégico de la plataforma evoluciona hacia **seis líneas maestras**, lideradas por el nuevo paradigma jerárquico:

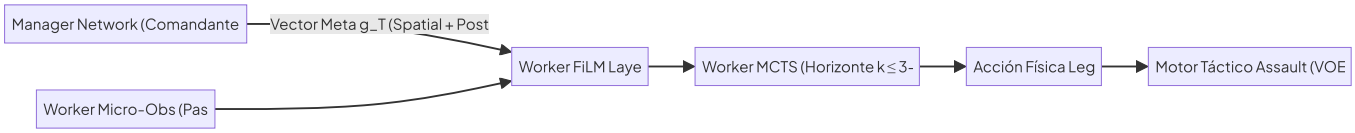


Línea 6: Hierarchical Model-Based RL (H-EZ / Hierarchical EfficientZero) — Era v3.0

> **Importancia Global:** 🔴 **Prioridad P0 (Máxima Prioridad / Foco Rector del Sistema)**
> **Objetivo Estratégico:** Superar de raíz la fricción temporal del wargame (episodios de 172 micro-pasos frente a $T \leq 8$ turnos) desacoplando la inteligencia en dos niveles neuronales que aprenden solos por autojuego: el Comandante Estratégico (*Manager*) y el Operador Táctico (*Worker*).

Fundamentación Matemática y Arquitectónica

- **Manager Network (Nivel Alto):** Toma como entrada la observación global s_T al inicio de cada turno $T \in 1, \dots, T_{\max}$ enriquecida con el mapa topográfico 5D (IPB) y emite un vector de sub-meta espacial y doctrinal $g_T = (\mathbf{M}_{\text{target}}, \mathbf{p}_{\text{mode}}) \in \mathbb{R}^{H \times W \times C}$. $Optimiza directamente el retorno de victoria terminal con descuento temporal macro$ \gamma_{\text{macro}} = 0.95$.
- **Worker Network (Nivel Bajo - EZv2):** En cada micro-paso t, la red convolucional procesa la observación física s_t modulada por g_T mediante capas FiLM: $$\mathbf{z}_t^{\text{mod}} = (1 + \boldsymbol{\gamma}(g_T)) \odot \mathbf{z}_t + \boldsymbol{\beta}(g_T)undefinedr_{\text{intrinsic}}(s_t, s_{t+1}) = \cos\big(h_\theta(s_{t+1}) - h_\theta(s_t), ; g_T\big)$$ AI no tener que resolver el final de la partida, el Worker **elimina por completo la deriva latente recurrente**.



Tareas del Vector Jerárquico H-EZ

1. **Tarea 6.1: Protocolo de Sub-Metas Espaciales y Modulación FiLM del Worker (`SpatialGoalFiLMBridge`) [P0]**
   - **Módulos:** `agents/efficientzero_v2/core/task_conditioning.py`, `agents/efficientzero_v2/core/network.py`, `agents/efficientzero_v2/core/network_base.py`.
   - **Alcance:** Formalización del espacio de metas g_T (mapa de calor $H \times W$ de concentración de esfuerzo + vector doctrinal de 4 dimensiones). Integración de la modulación FiLM en el tronco de representación espacial h_θ y cabezas de valor/política con inicialización ortogonal y Zero-Init preservation.
   - **Criterio de Aceptación:** Batería de tests unitarios verificando paridad exacta de identidad en $g_T = 0$ y concentración > 80 de probabilidad de ruta MCTS en torno al centroide de g_T cuando se inyecta una meta activa.

2. **Tarea 6.2: Red Convolucional del Comandante Estratégico (`ManagerNetwork`) [P0]**
   - **Módulos:** `agents/efficientzero_v2/core/manager_network.py`, `agents/efficientzero_v2/train/manager_trainer.py`.
   - **Alcance:** Red convolucional compacta (4 bloques residuales + Spatial Attention) que toma como entrada la observación Turno-T (38 canales + 5 canales IPB) y emite la distribución de metas g_T. Gobernanza estricta en el bucle de autojuego activándose únicamente en la transición de turnos o ante eventos de ruptura táctica (`UNIT_ELIMINATED` o `SUPPRESSED`).
   - **Criterio de Aceptación:** Inferencia ultrarrápida < 1.5 ms en GPU NVIDIA RTX 4060 y gradientes estables sin explosión ni colapso de representación.

3. **Tarea 6.3: Bucle de Co-Entrenamiento en Autojuego Autoritativo y Telemetría MLOps [P0]**
   - **Módulos:** `agents/efficientzero_v2/train/train_engine.py`, `agents/efficientzero_v2/core/selfplay.py`, `mlops/reporting/viewer_frontend/`.
   - **Alcance:** Co-entrenamiento coordinado: el Manager se entrena con el retorno final de la partida ($R \in -1, +1$) almacenando transiciones de macro-paso en su propio búfer; el Worker entrena contra la recompensa intrínseca de meta. Renderizado en Match Replay (:8778) de las flechas operacionales y mapa de calor de meta del Comandante sobre el canvas hexagonal PixiJS.
   - **Criterio de Aceptación:** Certificación en `battaglia_cittadina_2_1` de asaltos coordinados a VPs con Win Rate ≥ 80 contra bot heurístico y tasa de captura de VPs ≥ 75.

Línea 1: Garantía de Instrumentos y Observabilidad Fiel (Zero-Mock End-to-End)

> **Importancia Global:** 🔴 **Prioridad P0 (Soporte Instrumental Innegociable)**
> **Objetivo:** Auditar y verificar que el 100% de las métricas jerárquicas (metas del Manager, recompensas intrínsecas del Worker, derivas y tiempos de turno) queden registradas sin mocks en Parquet y DuckDB.
> **Estado:** Tareas 1.1 a 1.5 plenamente consolidadas. Soporte instrumental en expansión para telemetría H-EZ.

Línea 2: Aceleración de Captura de VPs y Paridad Humana

> **Importancia Global:** 🔴 **Prioridad P0 (Consolidada e Integrada en H-EZ)**
> **Objetivo:** Resolver la Paradoja del Comandante Prudente. Todas las tareas fundacionales (PBRS Ng et al., urgencia temporal Devlin & Kudenko, CoordConv, desensibilización de brecha y currículum asimétrico) han sido completadas con éxito (Tareas 2.1 a 2.19 consolidadas). Su culminación definitiva se alcanza de forma estructural con la Línea 6 (H-EZ).

Línea 3: Fidelidad Normativa y Reglamento Oficial del Wargame

> **Importancia Global:** 🟢 **Prioridad P1 (Alta)**
> **Objetivo:** Garantizar apego militar estricto al reglamento de Sound of Drums rev6. Con las tareas 3.1, 3.5.1–3.5.4, 3.5.6, 3.6 y 3.7.1–3.7.2 plenamente consolidadas, el foco restante reside en enriquecer el catálogo de acciones del Worker (humo táctico, cavar trincheras y transporte, Tarea 3.2) y el soporte fuera de mapa TAS/OAS (Tarea 3.3).

Línea 4: Explicabilidad Máxima (XAI), Perfilado Causal e Inteligencia Táctica (IPB)

> **Importancia Global:** 🟢 **Prioridad P1 (Alta)**
> **Objetivo:** Proporcionar inteligencia transparente de campo de batalla. Con las tareas 4.1 (IGNH/GradCAM), 4.2 (SitRep Forense Causal), 4.3 (IPB Topográfico 5D) y 4.4 (Perfilado Doctrinal) consolidadas, el vector se integra directamente en la Línea 6 como la entrada sensorial directa del Comandante Estratégico.

Línea 5: Generalización Procedural y Robustez en Terrenos Abiertos

> **Importancia Global:** 🔵 **Prioridad P2 (Media)**
> **Objetivo:** Validar la capacidad de adaptación de la IA a mapas masivos (32×32 y 64×64). Gracias a la arquitectura H-EZ, esta línea se vuelve altamente viable: el Comandante divide mapas gigantes en macro-sectores y asigna sub-metas locales sin que el Worker sufra la explosión del horizonte de búsqueda.

---

4. Historial Canónico de Versiones y Changelog de Fases Consolidadas

Resumen Sintético de Versiones Recientes Consolidadas

Versión	Hito Principal	Tareas Consolidadas	Estado
`v3.0.0`	**Arquitectura Hierarchical EfficientZero (H-EZ) y Salto a Mando Estratégico de Misión (Major Version)**: 1) Diagnóstico causal de cuellos de botella en MBRL plano (deriva recurrente $k > 3$, horizonte de 172 micro-pasos y colapso de entropía); 2) Desacoplamiento formal en dos cerebros neuronales: Red Comandante Estratégico (*Manager Network*) a nivel de turno ($T \leq 8$) y Operador Táctico (*Worker Network*) con MCTS de horizonte corto ($k \leq 3 - 4$); 3) Inyección de sub-metas espaciales y doctrinales mediante capas FiLM con Zero-Init; 4) Eliminación estructural de la Paradoja del Comandante Prudente; 5) Reorganización integral del Roadmap hacia la Era v3.0.	**Arquitectura Central H-EZ (Era v3.0)**	🚀 **En Desarrollo Activo / Foco Rector**
`v2.5.51`	**Multi-Task Model-Based Reinforcement Learning (EfficientZero-Multitask / EZ-M, arXiv:2603.01452)**: 1) Desacoplamiento de representaciones invariantes vs dinámicas condicionadas por escenario: módulo `TaskConditioningModule` con incrustaciones densas $\mathbf{e}_\tau$ y modulación lineal basada en características (`FiLMLayer`) con inicialización a cero (Zero-Init Identity Preservation), modulando selectivamente las cabezas de valor y recompensa mientras se comparte la dinámica recurrente \theta $ylarepresentaciónespacial\theta$; 2) Búfer de repetición multitarea particionado `MultiTaskReplayBuffer` con muestreo estratificado conjunto (`sample_multitask`) balanceando de forma configurable la masa de experiencia entre escenarios; 3) Algoritmo de proyección ortogonal de gradientes conflictivos entre tareas `multi_task_pcgrad_backward` (Multi-Task PCGrad) calculando similitud coseno y proyectando $\mathbf{g}_i \cdot \mathbf{g}_j < 0$ para evitar interferencia destructiva o sobreajuste topográfico; 4) Documentación canónica en `docs/17_EFFICIENTZERO_MULTITASK_EZ_M_ARQUITECTURA.md` y suite de verificación unitaria integral `agents/efficientzero_v2/tests/test_multitask_ezm.py` (5/5 tests específicos, 163/163 en suite global EZv2).	**Línea 2 / Arquitectura MBRL Multitarea (EZ-M)**	🟢 **Consolidado**

Versión	Hito Principal	Tareas Consolidadas	Estado
`v2.5.50`	**Integración Pedagógica del Escenario Canónico 11.1 (`addestramento_fanteria_11_1`) en Currículum de Asalto, Fast Bootstrap y Sincronización MLOps (:8778)**: 1) Ingesta del escenario oficial 11.1 como Fase 4 (`phase4_addestramento_fanteria_11_1`) en `curriculums/vp_acceleration/curriculum.yaml` , resolviendo el salto de complejidad masivo entre el sprint 2v2 y el asalto urbano 14v7 con un sandbox simétrico 3v3 de infantería (fusileros + armas pesadas) y 1 VP central disputado en pieza S6 fortificada; 2) Configuración de arranque ultrarrápido en Ciclo 1 (*Fast Bootstrap*): PBRS denso y continuo (`vp_potential_weight: 2.5` , `vp_capture_bonus_per_hex: 14.0` , `objective_progress_bonus_per_hex: 10.0`), MCTS a 160 simulaciones y ponderación elevada de pérdida de objetivos (`objective_loss_weight: 0.65`); 3) Pipeline de evaluación corregido y sincronizado con el bando auditado (`tracked_side: US` / Side B) en `benchmark_config.vp_acceleration_phase4_addestramento.yaml` (`ezv2_selfplay` , `ezv2_vs_random_side_b` , `ezv2_vs_random_side_a`); 4) Expansión del diagrama de flujo en `CurriculumLineageFlow.tsx` para 14 fases de campaña integral (Stage 1: 6 fases, Stage 2: 8 fases) con renderizado dinámico y build de Vite impecable en 2.10s.	**Línea 2 (Subtarea 2.5 / 3.5.3 - Currículum)**	🟢 **Consolidado**
`v2.5.49`	**Saneamiento Sintáctico Mermaid 10.9 (Zero-Bombs) y Enriquecimiento Visual Canónico con Activos Físicos del Wargame**: 1) Erradicación de errores de sintaxis (*bomb icon*) en diagramas de flujo Mermaid de la red neuronal EfficientZero V2 (`03_RED_NEURONAL_EFFICIENTZERO_V2.md`), arquitectura y auditorías forenses mediante encomillado estricto de labels con caracteres especiales (`(` , `)` , `%`). 2) Curación, ingesta y despliegue de 52 activos gráficos de alta resolución (`assets/art/`): tarjetas técnicas oficiales de unidades con anverso y reverso (US Rangers, Sherman, Fallschirmjäger, Panzer IV, Breda MMG, Semovente), fichas hexagonales de combate (*counters*), cartas de formación combinada OOB, dados balísticos de batalla (rojo, verde, azul, amarillo, D6 negro), fichas de estado y chevrons de moral (`Status Suppressed` , `Ambush` , `Fallback` , `Hit`), marcadores de acción de tablero (`firing` , `move_fire` , `spotted`) y tableros modulares de batalla (`Map P1` , `Map P2`). 3) Estilos CSS modernos para renderizado de imágenes y galerías interactivas en `docs/index.html` y MkDocs (`site/`).	[CONSOLIDADO 🟢 v2.5.49]	
`v2.5.44`	**Alternancia Instantánea Real Scan 100% vs Rellenos de Color e Inspector Hexagonal con Selector de Terreno por Lista en Editor Táctico (:8780 y :8782)**: 1) Conmutador tri-estado de visualización con un clic: 🗺️ `Real Scan (100%)` (renderizado de escaneos reales a opacidad y brillo completo con cuadrícula alámbrica de alto contraste y cero oscurecimiento/relleno), 🎨 `Color Fills` (rellenos sólidos de terreno vivo) y 🔀 `Both` (modo híbrido con deslizador de opacidad); 2) Inspector de hexágono marcado y selector de terreno por lista: marcado de hexágono al hacer clic con anillo dorado resplandeciente (`#f59e0b`) y coordenadas, selector desplegable `<select id="piece-hex-terrain-select">` con el catálogo canónico completo de 11 terrenos, selector de elevación y botón de reseteo a Clear; 3) Sincronización bidireccional inmediata con paleta rápida, tabla de hexágonos (`Sparse Hexes`) y canvas; 4) Batería de pruebas: 15/15 en suites de editor, 123/123 en modelo, 158/158 en EZv2.	**Herramientas de Autoría Táctica & MLOps**	🟢 **Consolidado**
`v2.5.43`	**Pan & Zoom Centrado en Cursor y Superposición de Escaneos Reales de Tablero Físico en Editor Táctico (:8780 y :8782)**: 1) Navegación fluida e intuitiva sobre el canvas hexagonal tanto en Escenarios (`#view-scenarios`) como en Piezas de Mapa (`#view-pieces`): zoom suave con rueda de ratón anclado y centrado milimétricamente en las coordenadas del cursor, y desplazamiento espacial panorámico (*pan*) con arrastre de botón derecho (menú contextual desactivado), botón central, barra espaciadora sostenida o modo Pan dedicado; 2) Superposición de escaneos reales del juego físico de mesa (`assets/art/maps/Map [ID].png` : `Map S2.png` , `Map S3.png` , `Map S4.png` , `Map Z1.png` , `Map Z2.png` , etc.) alineados a la caja envolvente de la cuadrícula odd-r; 3) Controles de visualización en barra de herramientas: conmutador reactivo de imagen real (🗺️ `Real Board Scan`), deslizador continuo de opacidad (`0..100%`) y conmutador de relleno de hexágonos (🎨 `Hex Fills`) para modo alámbrico (*wireframe*) transparente que permite comparar, verificar y calcar con exactitud milimétrica los tableros impresos oficiales; 4) Batería de pruebas: 15/15 en suites de editor, 123/123 en modelo, 158/158 en EZv2.	**Herramientas de Autoría Táctica & MLOps**	🟢 **Consolidado**
`v2.5.41`	**Integración Integral de la Pila XAI en el Asesor Táctico y Copiloto RAG (:5173 - Real-Time Topography IPB, OOB & Neural Assessment)**: 1) Explotación al 100% de la inteligencia de campo de batalla en `assault_rag/copilot/services.py` (`explain_game_situation`): incorporación del analizador topográfico IPB (`analyze_battlefield_topography`) extrayendo métricas de cobertura (% urbano, % bosque, % abierto, corredores de desenfilada), censo en vivo de Orden de Batalla (OOB) clasificado por roles doctrinales (`SPEARHEAD` , `FIRE_SUPPORT` , `SNIPER` , `RECON` , etc.) y estado de supresión/HP crítica, y telemetría de red neuronal EZv2 ($V(s)$, probabilidad de victoria, decisiones MCTS); 2) Resiliencia de tiempo de respuesta cero cuelgues (Zero-Hang / Fast SitRep): generación instantánea determinista (<15ms) de vectores tácticos (prioridades, oportunidades operacionales, riesgos identificados y alertas de escuadra) con refinamiento LLM opcional de timeout ultracorto, erradicando por completo el bloqueo o congelación en "ANALIZANDO POSICIÓN..."; 3) Overhaul completo de la interfaz de usuario en `assault_ai_ui/src/game/ui/RagAssistantPanel.tsx` bajo estricto cumplimiento de la Regla 7 (100% en inglés): barra de probabilidad de victoria y $V(s)$ del cerebro activo, tarjetas con badges cromáticos para Intel Topográfico IPB, Orden de Batalla de fuerzas aliadas y enemigas, vectores tácticos estructurados y citas del reglamento; 4) 100% de tests en verde (`test_rag_copilot_endpoints.py` , `test_unit_traits_parity.py`) y build de Vite impecable.	**Línea 4 (Tarea 4.2 / RAG XAI SitRep)**	🟢 **Consolidado**
`v2.5.42`	**Editor Táctico Unificado 4-en-1: Unidades, Formaciones, Escenarios y Piezas de Mapa (Map Pieces) con Terreno Real (:8780 y :8782)**: 1) Corrección integral del renderizado del tablero táctico: cuadrícula hexagonal vertical escalonada odd-r (*pointy-topped*) con resolución y pintado de terreno real (agua `#1d4ed8` , bosques ligeros/densos, olivares `#4d7c0f` , edificaciones `#b45309` , ruinas, colinas y muros dorados) expandido desde las piezas canónicas del escenario (`map.pieces`) y soporte simultáneo de esquemas canónicos y planos (`u.position` , `vp.hexes` , `map.fortifications`); 2) Incorporación de la 4ª pestaña: Editor Interactivo de Piezas de Mapa (Map Pieces) con ciclo completo de vida en borrador (`assets/catalogs/map_piece_drafts/`) y publicadas (`assets/catalogs/map_piece_catalog.json`); 3) Paleta de herramientas de pintura de terreno en vivo (Clear, Water, Lt Forest, Hvy Forest, Olive, Brush, Rocky, Bldg, Bldg+, Hill, Ruins), edición de elevaciones, trazado de muros en aristas de hexágonos y editor JSON crudo reactivo; 4) API REST unificada en FastAPI y servidor standalone (`/api/map_pieces/*`); 5) Baterías de pruebas: 15/15 en suites de editor, 123/123 en modelo, 158/158 en EZv2.	**Herramientas de Autoría Táctica & MLOps**	🟢 **Consolidado**
`v2.5.40`	**Editor Táctico Unificado 3-en-1: Unidades, Formaciones y Escenarios (:8780 y :8782)**: 1) Fusión integral del Editor de Escenarios con el Catálogo de Unidades y Formaciones en una sola interfaz reactiva unificada con navegación por pestañas (`#view-units` , `#view-formations` , `#view-scenarios`); 2) Ciclo de vida completo para escenarios en borrador (`assets/scenarios/drafts/`) y publicados (`assets/scenarios/`): creación de borradores, clonación de escenarios oficiales, edición interactiva de unidades sobre canvas hexagonal con coordenadas canónicas col,row y snap-to-hex, gestión de fortificaciones y Puntos de Victoria, control de reglas doctrinales y moral, editor JSON crudo en vivo y publicación con validación estricta de esquema hacia el motor y la IA; 3) API REST unificada en `unit_editor_server.py` (`/api/scenarios` , `/api/scenarios/draft` , `/api/scenarios/draft/{id}/publish` , `/api/catalogs`); 4) Frontend e interfaz 100% en inglés según Regla 7; 5) Cobertura de pruebas completa (10/10 en suites de editor, 24/24 en backend, 123/123 en modelo, 158/158 en EZv2).	**Herramientas de Autoría Táctica & MLOps**	🟢 **Consolidado**

Versión	Hito Principal	Tareas Consolidadas	Estado
`v2.5.39`	**Mecánica Canónica de Artillería Pesada Remolcada (`HEAVY_WEIGHT`), Catálogo Staged y UX de Combate en Editor de Unidades (:8780)**: 1) Implementación física en `assault_model/rules/movement_rules.py` de la habilidad oficial de carta `HEAVY_WEIGHT` ("This unit must be transported to move up or down an elevation level", cartas PaK 40 y 17-Pdr ATG): el grafo de Dijkstra y `get_legal_paths` prohíben la transición entre hexágonos a nivel de suelo y hexágonos de colina/elevación (`Terrain.HILL`) si la unidad no va montada en un transporte, además de bloquear acciones rápidas (`can_use_fast_action`); 2) Ingesta en `STANDARD_TRAITS` y `TRAIT_DESCRIPTIONS` en `tools/unit_editor_server.py` , preasignación automática en `unit_staging_proposals.json` para `GE_75MM_ATG_40` y `UK_17_POUNDER_ATG` ; 3) Generación de propuestas completas y fidedignas para las 30 unidades pendientes de infantería, artillería y morteros en `unit_staging_proposals.json` para revisión visual rápida antes de publicación; 4) Corrección y perfeccionamiento de filtros en la barra lateral del Editor de Unidades (:8780): desglose exacto de estado (Published: 27, Staged: 30, Pending: 38 sumando las 95 cartas oficiales), nuevos filtros por categoría funcional (All, Infantería, Artillería, Blindados, Transportes) y filtrado dinámico por rasgos tácticos; 5) Inversión ergonómica de selectores de dados de combate (paleta para elegir al principio, dados asignados al final) y reordenación de objetivos en Fuego Directo (Vehículos primero, Infantería después); 6) Suite de verificación unitaria `test_heavy_weight_trait_movement` en `test_unit_traits_parity.py` (7/7 tests en verde, 123/123 en motor) y saneamiento estricto al inglés bajo Regla 7.	**Línea 3 (Fidelidad Normativa y Herramientas)**	🟢 **Consolidado**
`v2.5.38`	**Paquete Integral de Aceleración Extrema Zero-Quality-Loss de Entrenamiento y Autojuego**: 1) Bucle PUCT MCTS compilado en C nativo con **Numba JIT** (`_FAST_MCTS_PUCT_LOOP`), erradicando las 2.500 asignaciones dinámicas de arrays NumPy por decisión y acelerando el bucle en $253\times$ (de 8.33 ms a 0.033 ms por llamada); 2) Memoización de inferencia de raíz en replanificación de unidades de autojuego (`_priors_and_values`), eliminando el 66 de llamadas forward redundantes a la GPU; 3) Activación integral de Tensor Cores **TF32** (`torch.backends.cuda.matmul.allow_tf32`) y cuDNN benchmark en `InferenceService` , `trainer.py` y `train_engine.py` ; 4) Despacho dinámico no bloqueante de micro-batching en `InferenceService` , eliminando tiempos muertos de espera en cola; 5) Transferencias DMA asíncronas no bloqueantes con `pin_memory()` en `trainer.py` . Cero degradación matemática (bit-exact parity validada en 158/158 tests de EZv2 y 123/123 de motor).	**Línea 2 (Subtarea 2.16 – Rendimiento Core)**	🟢 **Consolidado**
`v2.5.37`	**Arquitectura Táctica de Selección de Acciones, Columna Lateral de Escuadras (Left Squad Drawer), Discriminación Visual de Activación y Resolución Balística Detallada (:5173)**: 1) Directorio vertical de escuadras en columna izquierda (`.left-squad-drawer` de 360 px): botón flotante `📋 SQUADS [S]` , buscador textual dinámico, filtros (`ALL` , `READY` , `DONE` y facciones), tarjetas de tropa de gran formato con avatares nítidos, salud operativa (corazones y barra), roles tácticos y contador de órdenes legales; 2) **Cierre automático inteligente del directorio de escuadras al despachar cualquier orden táctica o interactuar en el mapa**, garantizando el 100% de visibilidad del terreno durante las resoluciones tácticas; 3) Expansión y maximización de la paleta central de órdenes tácticas (`deck-card-orders` a `flex: 1`) con barra de pestañas por categoría táctica (`ALL` , `🏃 MOVE` , `🎯 FIRE` , `💥 ASSAULT` , `⏸ WAIT`) y contadores en vivo; 4) Discriminación visual instantánea de activación de unidades sobre el mapa: anillo perimetral cian resplandeciente exterior con animación de pulso y badge `READY` verde esmeralda para unidades listas para actuar; atenuación (`alpha: 0.65`), tinte pizarra (`tint: 0x94a3b8`) y badge `✓ DONE` en gris neutro para unidades que ya consumieron su turno; 5) Soporte explícito e interactivo para órdenes de espera (`WAIT`), asaltos cuerpo a cuerpo / granadas (`ASSAULT`), fuego directo e indirecto de mortero (`FIRE`), y movimientos calculados; 6) Contador en vivo en cabecera del carrusel de escuadras (`🟢 X READY · 🟣 Y DONE`) y filtros dinámicos por estado de activación (`ALL` , `READY` , `DONE`) y facción; 7) Dados tácticos balísticos de gran formato (44×44 px) en el panel de combate con bordes cromáticos luminosos, animación física y tooltips explicativos; 8) Tarjeta de efecto integral sobre el blanco (`combat-target-effect-card`) con explicación pormenorizada del desenlace balístico (`DESTROYED` , `DAMAGED` , `SUPPRESSED` , `DEFLECTED/ABSORBED`), seguimiento visual de salud (corazones perdidos 💔 / restantes ❤️) y desglose técnico de resolución; 9) Documentación técnica canónica en `docs/09_MANUAL_SIMULADOR_TACTICO_HUD.md` y sincronización HTML.	**Línea 4 (UI y Experiencia de Usuario del Simulador)**	🟢 **Consolidado**
`v2.5.36`	**Consolidación Canónica de Rasgos Tácticos de Infantería, Artillería y Emplazamientos Pesados (Fase A – Paridad Normativa 100%)**: 1) Implementación determinista en `assault_model` de las 7 habilidades canónicas extraídas de las 190 cartas físicas de unidades: `TALL_SILHOUETTE` / `SILHOUETTE` (+1 en avistamiento D6 y detección en terreno obstaculizado a ≤ 3 hexes en `spotting.py`), `BACK_BLAST` (degradación de blindaje Amarillo a Verde si la unidad disparó este turno en `ranged_combat_resolver.py`), `IMMOBILE` (0 MP autónomos, prohibición de acciones rápidas y bloqueo de rutas en `movement_rules.py`), `SET_UP_TIME` (bloqueo de acciones compuestas `MoveThenFire` / `FireThenMove` y acciones rápidas), `LIMITED_PIVOT` (arco de tiro de 240° restringido a flancos y retaguardia con sector frontal bloqueado en `combat_geometry.py` , `action_catalog.py` y `ranged_combat_resolver.py`), `REAR_ECHELON` (impacto 'X' automático de supresión cuando la unidad de soporte queda aislada a distancia > 1 hex de unidades aliadas en fuego a distancia y combate cuerpo a cuerpo) y `FLAMETHROWER` (eliminación del dado de defensa más fuerte del defensor e impacto automático 'X' de supresión); 2) Ingesta en `STANDARD_TRAITS` y `TRAIT_DESCRIPTIONS` del Editor de Unidades (:8780) con descripciones en inglés cumpliendo Regla 7; 3) Suite unitaria exhaustiva en `assault_model/tests/test_unit_traits_parity.py` (7/7 tests y 123/123 tests de motor pasando al 100%).	**Línea 3 (Fidelidad Normativa y Reglamento)**	🟢 **Consolidado**
`v2.5.35`	**Overhaul Integral de la Interfaz Táctica del Simulador (:5173 – Modern Wargame HUD Architecture)**: 1) Erradicación absoluta de las 7 ventanas flotantes desordenadas (`DraggableWindow`) que obstruían el mapa; 2) Canvas PixiJS a pantalla completa al 100% (*full-bleed*) como protagonista indiscutible del campo de batalla; 3) Barra Superior Táctica (`battle-header-ribbon`): insignia dinámica de turno, indicador de bando activo con pulso luminoso, barra de duelo / Balance de Poder (VP Dominance), selectores de escenario y bando, y controles de partida; 4) Panel de Mando Inferior (`bottom-command-deck`): Tarjeta de Inteligencia de Unidad Seleccionada (avatar nítido, rol táctico, corazones y barra de salud operativa, coordenadas y botón WAIT), Paleta de Órdenes Despachadas (`DispatchedOrdersPanel`) y Carrusel de Orden de Batalla (`deck-card-roster`) con filtros de facción, estados de activación y previsualización de cartas; 5) Cajón Táctico Lateral Desplegable (`side-tactical-drawer`): deslizante desde la derecha con pestañas para Resolución de Combate y Dados (`CombatPanel`), Registro de Batalla (`log-container`), Telemetría de la IA (`EfficientZeroV2TelemetryPanel`) y Asesor de Reglas RAG (`RagAssistantPanel`), con auto-apertura inteligente ante eventos de combate; 6) Modo Inmersivo de Mapa: tecla de acceso rápido `H` y botón de ojo para ocultar/mostrar todo el HUD al instante; 7) Cumplimiento 100% de la Regla 7 (UI y código estrictamente en inglés).	**Línea 4 (UI y Experiencia de Usuario del Simulador)**	🟢 **Consolidado**
`v2.5.34`	**Auditoría Integral OCR de 190 Cartas Físicas, Censo de 40 Habilidades y Desacoplamiento en Dos Fases**: 1) Extracción exhaustiva de reglas de reverso y anverso sobre el 100% de las 95 unidades wargame (Alemania, Italia, UK, USA) en `assets/art/unit_cards/` ; 2) Censo de 40 rasgos tácticos divididos operativamente en **Fase A (Infantería, Artillería y Soporte Pesado)** y **Fase B (Vehículos Blindados, Carros y Transporte Mecanizado)**; 3) Documentación canónica de mecánicas, efectos y reglas oficiales en `docs/02_MODELO_DEL_JUEGO_Y_REGLAS_TACTICAS.md` (§13.1); 4) Planificación secuencial para la culminación de infantería/artillería previa a la apertura de la fase de mecanización de blindados.	**Línea 3 (Fidelidad Normativa y Catálogos)**	🟢 **Consolidado**

Versión	Hito Principal	Tareas Consolidadas	Estado
`v2.5.33`	**Consolidación del Rasgo Canónico `LOW_SILHOUETTE` (Perfil Bajo / Silueta Reducida, Carta S43)**: 1) Implementación en `assault_model/combat/spotting.py` de la función determinista `calculate_spotting_modifier(attacker, target, los)` que aplica un modificador de -1 en tiradas D6 de avistamiento del atacante (§10.5 Fuego Directo, §12.3.6.2 Apoyo Aéreo TAS y §12.4.2.1 Artillería OAS); 2) Inmunidad a avistamiento a distancia (> 1 hex) en cobertura defensiva hasta que la unidad defensora dispare o mueva rápido; 3) Ingesta en `STANDARD_TRAITS` y `TRAIT_DESCRIPTIONS` del Editor de Unidades y Formaciones (:8780), traduciendo al 100% las descripciones al inglés bajo Regla 7; 4) Suite de verificación unitaria `test_low_silhouette_trait_spotting` en `test_unit_traits_parity.py` (6/6 tests en verde, 122/122 en motor).	**Línea 3 (Fidelidad Normativa)**	🟢 **Consolidado**
`v2.5.32`	**Mecánica Canónica de Ocultamiento Táctico (`CONCEALED` , `CAMOUFLAGED`) y Saneamiento de `UK_COMMANDOS`**: 1) Implementación determinista en `assault_model/combat/spotting.py` de los rasgos `CONCEALED` y `CAMOUFLAGED` : unidades en terreno con cobertura (`HINDERED` o superior, ej. bosques, matorrales, edificios) son indetectables a distancia (> 1 hexágono) sin trazado de LOS hasta que abran fuego (`has_fired_this_turn`) o realicen acciones rápidas (`has_fast_action_this_turn`), preservando la detección a quemarropa (≤ 1 hex); 2) Corrección del rango de fuego de `UK_COMMANDOS` en `unit_staging_proposals.json` corrigiendo la banda `8-10` a `6-10` conforme a la carta original británica; 3) Traducción y saneamiento de comentarios al inglés en `spotting.py` y `spotting_runtime.py` bajo Regla 7; 4) Suite de verificación unitaria `test_concealed_and_camouflaged_trait_spotting` en `test_unit_traits_parity.py` (5/5 tests en verde, 121/121 en motor).	**Línea 3 (Fidelidad Normativa)**	🟢 **Consolidado**
`v2.5.31`	**Consolidación de Skills Tácticos Oficiales (`Slow` , `Pivot` , `Shield`) e Ingesta de la Pieza Canónica `GE_3_7CM_FLAK_37` (Carta S43)**: Implementación física en el motor de los rasgos `Slow` (bloqueo de acciones rápidas `MovementRules.can_use_fast_action` y compuestas `MoveThenFireAction`), `Pivot` (`is_in_arc_of_fire` , arco de fuego omnidireccional de 360 grados sin penalización angular) y `Shield` (`calculate_facing_towards` en `combat_geometry.py` y reorientación frontal automática en `ranged_combat_resolver.py` y `smoke_resolver.py` , garantizando que el contraataque impacte en el sector `FRONT` del escudo). Integración de la unidad en `assets/catalogs/unit_catalog.json` con tablas balísticas y dados de defensa exactos, actualización de `unit_art_inventory.json` (`status: IMPLEMENTADO`) y suite unitaria `test_flak37_skills_and_shield.py` (4/4 tests en verde, 120/120 en motor).	**Línea 3 (Subtarea 3.5.4)**	🟢 **Consolidado**
`v2.5.42`	**Editor Táctico Unificado 4-en-1: Unidades, Formaciones, Escenarios y Piezas de Mapa (Map Pieces) con Terreno Real (:8780 y :8782)**: 1) Corrección integral del renderizado del tablero táctico: cuadrícula hexagonal vertical escalonada odd-r (*pointy-topped*) con resolución y pintado de terreno real (agua `#1d4ed8` , bosques ligeros/densos, olivares `#4d7c0f` , edificaciones `#b45309` , ruinas, colinas y muros dorados) expandido desde las piezas canónicas del escenario (`map.pieces`) y soporte simultáneo de esquemas canónicos y planos (`u.position` , `vp.hexes` , `map.fortifications`); 2) Incorporación de la 4ª pestaña: Editor Interactivo de Piezas de Mapa (Map Pieces) con ciclo completo de vida en borrador (`assets/catalogs/map_piece_drafts/`) y publicadas (`assets/catalogs/map_piece_catalog.json`); 3) Paleta de herramientas de pintura de terreno en vivo (Clear, Water, Lt Forest, Hvy Forest, Olive, Brush, Rocky, Bldg, Bldg+, Hill, Ruins), edición de elevaciones, trazado de muros en aristas de hexágonos y editor JSON crudo reactivo; 4) API REST unificada en FastAPI y servidor standalone (`/api/map_pieces/*`); 5) Baterías de pruebas: 15/15 en suites de editor, 123/123 en modelo, 158/158 en EZv2.	**Herramientas de Autoría Táctica & MLOps**	🟢 **Consolidado**
`v2.5.40`	**Editor Táctico Unificado 3-en-1: Unidades, Formaciones y Escenarios (:8780 y :8782)**: 1) Fusión integral del Editor de Escenarios con el Catálogo de Unidades y Formaciones en una sola interfaz reactiva unificada con navegación por pestañas (`#view-units` , `#view-formations` , `#view-scenarios`); 2) Ciclo de vida completo para escenarios en borrador (`assets/scenarios/drafts/`) y publicados (`assets/scenarios/`): creación de borradores, clonación de escenarios oficiales, edición interactiva de unidades sobre canvas hexagonal con coordenadas canónicas col,row y snap-to-hex, gestión de fortificaciones y Puntos de Victoria, control de reglas doctrinales y moral, editor JSON crudo en vivo y publicación con validación estricta de esquema hacia el motor y la IA; 3) API REST unificada en `unit_editor_server.py` (`/api/scenarios` , `/api/scenarios/draft` , `/api/scenarios/draft/{id}/publish` , `/api/catalogs`); 4) Frontend e interfaz 100% en inglés según Regla 7; 5) Cobertura de pruebas completa (10/10 en suites de editor, 24/24 en backend, 123/123 en modelo, 158/158 en EZv2).	**Herramientas de Autoría Táctica & MLOps**	🟢 **Consolidado**
`v2.5.49`	**Aceleración Extrema de Entrenamiento Neuronal y Replay Buffer (Zero Quality Loss & Paridad Numérica Exacta)**: 1) **Codificación Directa de Representación en Pérdida de Consistencia Temporal (`representation_encode`)**: Sustitución de llamadas completas redundantes a `initial_inference` por `representation_encode` , omitiendo la evaluación del tronco de predicción (8 bloques Conv2D) y cabezas de política, valor, recompensa y auxiliares en las observaciones objetivo del desenrollado temporal K-step. Idéntico a nivel matemático con diferencia exacta 0.000000000 y aceleración de 2.26× en la fase de codificación de estados futuros; 2) **Batching Unificado de Observaciones Objetivo K-Step en CUDA**: Fusión de todas las observaciones futuras $t+1, \ldots, t+K$ en un único lote transferido mediante memoria fijada (*pinned memory*) y procesado en un único pase hacia delante GPU, vectorización directa de acciones con `F.one_hot` en CUDA y erradicación del troceado/concatenación de tensores en Python (`torch.cat`), acelerando el cómputo de `consistency_loss` de 12.20 ms a 7.00 ms (1.74× más rápido); 3) **Ingesta de Observaciones en Replay Buffer y DMA Transfer**: Ingesta directa de tensores FP16 desde el búfer en RAM de host hacia la GPU con DMA asíncrono y upcasting ultra-rápido en CUDA (ahorro del 50% de ancho de banda PCIe) y apilado por lotes previo a la transferencia en objetivos auxiliares (`final_ownership`); 4) **Replay Buffer $O(1)$ sin Sobrecarga de Heap**: Reemplazo de listas con `pop(0)` por `collections.deque(maxlen=capacity)` ($946\times$ más rápido en inserción) y muestreo determinista sin materialización de rangos ni listas de candidatos en Python ($42\times$ más rápido en muestreo); 5) **Rendimiento Global**: Reducción del coste por update de entrenamiento de 37.01 ms a 25.47 ms (>31% de reducción del tiempo de update, 1.45× más rápido) manteniendo 100% de paridad y precisión numérica. Batería de 158/158 pruebas en EZv2, 77/77 en assault_bench y 123/123 en assault_model en verde.	**Optimización y Rendimiento Core (Zero Quality Loss)**	🟢 **Consolidado**
`v2.5.48`	**Integración Canónica de Nidos de Sacos Terreros (`Terrain Sandbag Position.png`), Fortificaciones Adicionales (`pillbox` , `tank_barricade`) y Corrección de Publicación de Escenarios a Motor**: 1) Reglas y tablas canónicas: Modificadores defensivos y de movimiento registrados en `assets/rules_tables/fortification/fortification_modifiers.v1.json` (+1 dado amarillo frontal, +1 dado verde flanco para infantería/artillería; +1 coste de movimiento para ruedas); 2) Resolutores balísticos y de combate: Integración en `assault_model/combat/explosive_charge_resolver.py` como fortificación blanda (`is_soft_fort`) que colapsa con cargas explosivas (Regla 10.9), algoritmo de desenfilada/retirada en `assault_model/rules/movement_rules.py` (+4 cover score) y analizador topográfico IPB en `assault_model/map/topography_analyzer.py` (peso de cobertura 0.70); 3) Esquema de validación backend y ciclo de publicación: Ampliación de `VALID_FORTIFICATIONS` en `assault_backend/services/scenario_editor_service.py` con `sandbag_position` , `sandbag` , `pillbox` y `tank_barricade` , y resolución de discrepancia de signatura en endpoint de publicación (`publish_draft` / `publish_scenario`) junto con propagación detallada de errores en cliente y servidor HTTP; 4) Editor Táctico Unificado (:8780): Renderizado visual de ficha física real escaneada con `Terrain Sandbag Position.png` y `Terrain Tank Barricades.png` , botones de acceso rápido `Sandbag` y `Barricade` en el panel de herramientas, desplegables actualizados y fallback vectorial de insignia táctica `SND` y `BARR` . Suites 15/15 passing.	**Modelo de Reglas & Editor Táctico (:8780)**	🟢 **Consolidado**

Versión	Hito Principal	Tareas Consolidadas	Estado
`v2.5.47`	**Edición Avanzada de Fortificaciones (Tokens Reales de Arte, Rotación Hexagonal y Borrado Robusto) y Drag-and-Drop de Unidades en Escenarios**: 1) Erradicación total de fallos de borrado de fortificaciones mediante filtrado integral de estado (`currentScenarioData.map.fortifications` y `.fortifications`) y botón directo `Remove Fort` ; 2) Integración de fichas físicas reales escaneadas del juego (`Terrain Bunker.png` , `Terrain Pillbox.png` , `Terrain Gun Emplacement.png` , `Terrain Trenches.png` , `Terrain Barbed Wire.png` , `Minefield AP.png`) con selector reactivo `Real Counters: ON/OFF` ; 3) Configuración de ángulo de giro y orientación en 6 direcciones hexagonales (0° N, 60° NE, 120° SE, 180° S, 240° SW, 300° NW) con botones rápidos de giro `↻ +60°` y `↺ -60°` y renderizado rotado mediante transformación de matriz (`ctx.rotate`); 4) Sistema reactivo de Drag-and-Drop sobre el canvas para reubicar unidades ya desplegadas en tiempo real con resaltado cian de celda objetivo, línea guía discontinua y ficha flotante con sombreado. Suites completas (15/15) en verde.	**Editor Táctico Unificado (:8780)**	**Consolidado**
`v2.5.46`	**Editor Visual Interactivo de Fortificaciones y Puntos de Victoria (VPs) y Depuración de Escenarios**: 1) Erradicación total de escenarios duplicados en la barra lateral del Editor mediante deduplicación estricta por ID en `ScenarioEditorService.list_all_scenarios()` y ciclo simétrico de publicación/despublicación (purgado de borradores al publicar a canónico); 2) Inspector visual de hexágono marcado para escenarios (`#scen-marked-hex-bar`): selección directa de cualquier celda del mapa con un solo clic, desplegable de fortificaciones oficiales (`bunker` , `casemate` , `gun_emplacement` , `trench` , `barbed_wire` , `minefield` , `none`), desplegable de puntos de victoria (1 a 10 VPs) y selector de controlador inicial de facción (`US` , `IT` , `GE` , `UK` , Neutral); 3) Paleta de herramientas de asignación rápida con un clic para fortificaciones y VPs (`Quick Fort` y `Quick VP`); 4) Renderizado canvas multicapa no superpuesto: insignias de fortificación en el sector inferior del hexágono con código y silueta táctica, estandartes dorados con estrella para VPs en el sector superior, fichas de unidad en el centro y resalte perimetral ámbar con chapa flotante `[Q, R]` . Suites completas (15/15) en verde.	**Editor Táctico Unificado (:8780)**	**Consolidado**
`v2.5.45`	**Editor Táctico Unificado 4-en-1 (Unidades, Formaciones, Piezas de Mapa y Escenarios) con Geometría Canvas Exacta y Mapeo Topográfico de Hexágonos**: 1) Erradicación total de deformación y achatamiento horizontal en los canvas 2D de piezas de mapa y escenarios mediante sincronización dinámica de buffer de renderizado (`syncCanvasSize`) y listener de redimensionamiento de ventana (`resize`); 2) Transformación de coordenadas de ratón con escala CSS exacta (`scaleX` , `scaleY`) y búsqueda exhaustiva por candidato euclidiano más próximo (`Math.hypot`), garantizando una selección y marcado de hexágonos (`selectPieceHex`) con 100% de precisión sin importar el ancho del contenedor; 3) Inspector de Hexágono Marcado (*Marked Hex Inspector*) con selector desplegable de tipos de terreno (`clear` , `water` , `light_forest` , `heavy_forest` , `olive_vine_grove` , `brush` , `rocky` , `building_single` , `building_multi` , `hill` , `ruins`), elevación (+0 a +3) y botón de reseteo rápido a terreno despejado; 4) Alternancia instantánea entre visualización pura de scan original (wireframe vectorial sin opacidad obstructiva) y mapa de colores sólidos semitransparentes; 5) Servidor unificado en puerto :8780 con ciclo de vida completo de borradores (Draft) y publicación canónica para piezas y escenarios. Suite completa (15/15) en verde.	**Editor Táctico Unificado (:8780)**	**Consolidado**
`v2.5.30`	**Integración de Marcadores Oficiales de Acción y Estados en Match Replay (:8778)**: 1) Renderizado fidedigno en el canvas 2D del visualizador de partidas de los marcadores canónicos de estado y acción (`assets/art/action_markers/` , `assets/art/status/` y `assets/art/markers/`): token `Status Suppressed.png` (reemplazando el recuadro vectorial PIN), marcadores de bajas de facción `Markers USA Dead.png` , `Markers Italy Dead.png` , `Markers Germany Dead.png` , marcador balístico de impacto `Marker Hit.png` , y marcadores de acción sobre la unidad activa (`firing.png` , `fast_action.png` , `fire_move.png` , `move_fire.png` , `normal.png` , `turret_firing.png` , `Status Fallback.png` , `Status Dig In.png` , etc.); 2) Conmutador reactivo `Status Markers` en la barra de capas del visor; 3) Visualización sincronizada de marcadores en el inspector táctico (*Action Execution & Combat Outcome*, *Acting Unit*, *Opponent Target* y listas de escuadras aliadas y del eje); 4) Corrección y traducción oportunista al inglés según Regla 7.	**Línea 1 / Línea 4 (Observabilidad y XAI Replay)**	**Consolidado**
`v2.5.30`	**Purga de Deuda Técnica Legacy y Unificación Canónica de Assets**: 1) Erradicación definitiva de la carpeta sombra desincronizada `assault_sim/assets/` asegurando que `assets/` sea la única fuente canónica de verdad para catálogos, escenarios, drafts y arte; 2) Archivado seguro de la pila obsoleta PPO/SB3 (`assault_sim/train/` , `assault_sim/checkpoints/` , scripts heredados) en `legacy_archive/` ; 3) Desacoplamiento de `mlops/reporting/replay_api.py` hacia el repositorio canónico de escenarios; 4) Mantenimiento del 100% verde en todas las suites (`assault_model` , `voec_sim` , `assault_backend` , `agents/efficientzero_v2` , `assault_bench`).	**Subtarea 3.7.2**	**Consolidado**
`v2.5.29`	**Inteligencia Topográfica de Terreno y Orden de Batalla Completo (OOB) en SitRep Forense (XAI)**: 1) Extracción de perfil del campo de batalla (`battlefield_profile`) en `causal_profiler.py` con dimensiones de mapa, métrica de victoria, desglose de terreno (urbano/edificios, bosques, terreno quebrado/escombros, agua, despejado y fortificaciones) y libro mayor de Puntos de Victoria (VPs iniciales vs finales y estatus); 2) Extracción integral del Orden de Batalla (`forces_oob`) para Atacante y Defensor con controlador, porcentaje de integridad de HP, recuento de bajas/bajas causadas y catálogo completo de unidades con rol táctico asignado, posición inicial, progresión de salud y estado operativo (`ACTIVE` , `SUPPRESSED` , `ELIMINATED`); 3) Pestaña interactiva `Battlefield & Forces (OOB)` en `ForensicSitRepModal.tsx` con paneles comparativos lado a lado y barras de progreso de fuerza; 4) Batería de pruebas unitarias verificada al 100% en `test_causal_profiler.py` (9/9 passing) y compilación Vite verificada.	**Línea 4 (Tarea 4.2 - XAI Forense)**	**Consolidado**
`v2.5.30`	**Purga de Deuda Técnica Legacy y Unificación Canónica de Assets**: 1) Erradicación definitiva de la carpeta sombra desincronizada `assault_sim/assets/` asegurando que `assets/` sea la única fuente canónica de verdad para catálogos, escenarios, drafts y arte; 2) Archivado seguro de la pila obsoleta PPO/SB3 (`assault_sim/train/` , `assault_sim/checkpoints/` , scripts heredados) en `legacy_archive/` ; 3) Desacoplamiento de `mlops/reporting/replay_api.py` hacia el repositorio canónico de escenarios; 4) Mantenimiento del 100% verde en todas las suites (`assault_model` , `voec_sim` , `assault_backend` , `agents/efficientzero_v2` , `assault_bench`).	**Subtarea 3.7.2**	**Consolidado**
`v2.5.29`	**Aceleración Inmediata del Motor Táctico (+67.8% a +86.0% Throughput)**: Eliminación de cuellos de botella críticos en `assault_model/` y `voec_sim/` : 1) Cacheo inmutable de `_static_spatial_features` gobernado por `terrain_signature` (ahorro del 21% de CPU por step); 2) Índices O(1) de ocupantes por `state_version` en `MovementRules.get_legal_paths` ; 3) Snapshot reactivo y fast-path de unidades ocultas en `update_spotting` ; 4) Precomputación de traits con `_trait_set` O(1) en `UnitType.has_trait` ; 5) Descarte de objetivos no avistados en `ActionCatalog._ranged_fire_actions` . Throughput incrementado de 428.8 a 719.7 pasos/s (random) y de 454.3 a 844.6 pasos/s (secuencial). Suite 160/160 en motor, 158/158 en EZv2 y 77/77 en benchmark en verde.	**Subtarea 3.7.1**	**Consolidado**

Versión	Hito Principal	Tareas Consolidadas	Estado
`v2.5.28`	**Perfilado 5D de Unidades y Detección Automática de Chokepoints (IPB / Intelligence Preparation of the Battlefield)**: 1) Análisis topográfico determinista Turno 0 en `assault_model/map/topography_analyzer.py` que calcula el vector de 5 dimensiones (*Cover*, *Elevation Dominance*, *LOS Exposure*, *Chokepoint Bottleneck*, *VP Proximity Utility*) para cada hexágono del mapa; 2) Clasificación táctica formal del terreno en *Key Terrain* (VPs), *Defilade Corridors* (aproximaciones cubiertas), *Killzones* (campos de tiro abiertos y mortíferos), *Chokepoints* (pasos constreñidos y embotellamientos) y *Base of Fire* (posiciones dominantes con línea de visión sobre el objetivo); 3) Inyección de priors topográficos en `MacroStrategyPlanner` (+25% avance por desenfilada hacia el VP, atenuación 0.55×-0.65× en killzones, +30% Base of Fire para armas pesadas, +20% en chokepoints para roles de pantalla y defensa); 4) Endpoints MLOps (`/api/v2/replay/topography` y `/api/v2/replay/ipb`); 5) Capa visual interactiva `🔷 IPB Topography` en el visor de Replay (:8778) con desglose 5D al posar el cursor sobre cada celda. Suites `assault_model` (116/116) y EZv2 (158/158) en verde.	**Línea 4 (Tarea 4.3 - IPB)**	🟢 **Consolidado**
`v2.5.27`	**Modulación Dinámica Doctrinal (β_{macro}), Auditoría de Cabezas y Modernización de Rendimiento**: 1) Soporte continuo para modulación de priors doctrinales `macro_modulation_scale` ($\beta_{\text{macro}} \in [0.0, 1.0]$) habilitando modo Red Neuronal Pura ($\beta = 0.0$), calentamiento progresivo y refuerzo doctrinal ($\beta = 1.0$) consolidando la Tarea 2.8; 2) Auditoría de localización espacial en cabezas tácticas (FCN pura $H \times W$ para `threat_map` / `final_ownership`, `SpatialAttentionBlock` 4×4 reteniendo 16 sectores para `attrition` / `morale` / `intent` / `multi_vp`); 3) Confirmación de 48 canales latentes en `prediction_trunk` (8 Conv2D a 48ch) y verificación de stop-gradient dual SimSiam con norma ℓ_2; 4) Activación de **Numba JIT** en Dijkstra cinemático vectorial, checkpoints `safetensors` zero-copy, `orjson` ultrarrápido y preasignación de dados $O(1)$. Suites 114/114 en `assault_model` y 158/158 en EZv2 en verde.	**Línea 2 (Tarea 2.8 / Rendimiento)**	🟢 **Consolidado**
`v2.5.26`	**Visualización Curricular Multilínea por Sub-Currículum (Dashboard MLOps)**: Reestructuración visual de `CurriculumLineageFlow.tsx` en filas temáticas dedicadas e independientes por cada sub-currículum (Fila 1: Sub-Currículum 1 - *Objective Doctrine & VP Acceleration*, Fases 1 a 5; Fila 2: Sub-Currículum 2 - *Multi-Scenario Self-Play vs Random*, Fases 6 a 13); banner de transición y transferencia de pesos inter-etapas (*Breakthrough Bridge*); erradicación de scroll horizontal masivo; compatibilidad total tanto en modo canónico como cronológico. Build de Vite verificado en 1.72s.	**Visualización Curricular (Tarea 1.6)**	🟢 **Consolidado**
`v2.5.24`	**Resiliencia Integral, Autorrecuperación Multiproceso y Reciclaje Periódico de Procesos (Zero Memory Leak)**: Sistema de autocuración (*self-healing*) en caliente de `ProcessPoolExecutor` ante excepciones de SO o caídas anómalas de trabajadores (`BrokenProcessPool`); reciclaje periódico automático de procesos cada 5 iteraciones (`recycle_workers_every_iters: 5`) coincidiendo con guardado de checkpoints para erradicar la fragmentación de heap y fugas de memoria en sesiones continuas multi-hora; fallback resiliente para episodios huérfanos; ampliación del enum `UnitCategory` (`ARTILLERY`, `GUN_ARTILLERY`) subsanando validaciones de catálogo de unidades. Suites 154/154 en EZv2 y 114/114 en assault_model en verde.	**Resiliencia y Concurrencia Multiproceso (Tarea 2.19)**	🟢 **Consolidado**
`v2.5.23`	**Arquitectura Multiproceso con IPC Shared-Memory e Inferencia GPU por Lotes (Zero-GIL CPU Saturation & CUDA Batching)**: Erradicación del cuello de botella del Global Interpreter Lock (GIL) de Python mediante ejecución de autojuego en 24 procesos independientes del sistema operativo con memoria compartida bidireccional (`torch.multiprocessing`); puente maestro `IPCInferenceBridge` que canaliza las peticiones de inferencia de los 24 workers hacia el `InferenceService` central en la GPU NVIDIA RTX 4060 con micro-batching dinámico de 64 observaciones; cliente de inferencia transparente `IPCProcessInferenceClient` con paso de tensores compartidos; erradicación de sincronizaciones de checkpoints en CPU durante el entrenamiento al compartir en caliente el modelo GPU; y configuración agresiva con MCTS 320, batch de train 48 y 48 episodios por iteración. Suites 154/154 en EZv2 y pruebas de integración IPC 100% en verde.	**Aceleración y Rendimiento Multiproceso (Tarea 2.18)**	🟢 **Consolidado**
`v2.5.22`	**Aceleración Extrema de Entrenamiento e Inferencia Neuronal (Updates CUDA 1.78× y Forward Fused)**: Inferencia lazy de 7 cabezas auxiliares en `network_base.py` (`initial_inference` y `recurrent_inference`), acelerando la inferencia un 16.2%; fusión del pase forward de entrenamiento y reanálisis en `trainer.py`, erradicando la evaluación redundante del batch completo; vectorización directa en GPU del bootstrap de reanálisis sin sincronizaciones bloqueantes CPU ↔ CUDA; erradicación de 2.160 hilos de sistema operativo por ejecución en `train_engine.py`; y activación de Adam fusionado (`fused=True`) en CUDA. Reducción del coste por update de 92.58 ms a 51.96 ms en GPU NVIDIA RTX 4060 (1.78× más rápido). Suites 153/153 en EZv2, 19/19 en MLOps y 9/9 en assault_bench 100% en verde.	**Aceleración y Rendimiento Neuronal (Tarea 2.17)**	🟢 **Consolidado**
`v2.5.21`	**Saneamiento Estructural y Refactorización del Modelo de Juego (`assault_model/`)**: Poda de 10 archivos y clases huérfanas con 0 referencias (`model_adapter.py`, `activation.py`, `replay.py`, `rulesets/`, `attributes.py`, `actions/combat.py`, `combat_action_context.py`, `los_modifier.py`, `morale_modifier.py`, `flank_modifier.py`); unificación de clones duplicados (`CombatBand` → `RangeBand`, `Flank` → `AttackSector`, `AttackDicePool` / `DefenseDicePool` consolidados en `dice_pool.py`); erradicación de código muerto en desuso (`terrain_rules.py`, `combat_resolution.py`, `attack_die.py` / `defense_die.py`); optimización $O(1)$ en `GameState` mediante `ownership_to_side` y aceleración de evaluación de VPs en `victory_outcomes.py`; corrección oportunista de comentarios en español según Regla 7. Baterías de pruebas: 114/114 en `assault_model` y 153/153 en EZv2 100% en verde.	**Refactorización y Saneamiento del Core Model**	🟢 **Consolidado**
`v2.5.20`	**Optimización Extrema del Bucle de Simulación, Observaciones y Autojuego (Aceleración de 418× en Features Espaciales y Caché de E/S)**: Precomputación de plantilla estática de terreno en `VOECSimulator.spatial_features` (aceleración de 418.1×, de 0.4773s a 0.0011s por 1.000 llamadas); memoización de acciones de fuego en movimiento por hexágono destino en `_move_fire_actions` suprimiendo comprobaciones repetitivas de LOS y balística; caché en memoria de catálogos y escenarios JSON en `importers.py` y `scenario_loader.py` eliminando miles de accesos a disco y parseos redundantes; memoización de bounding box axial en `EZV2Observation` y `EZV2VOECAdapter` acelerando `observation_to_tensor`. Suites 153/153 en EZv2, 19/19 en MLOps y 9/9 en assault_bench 100% en verde.	**Optimización y Rendimiento Core (Tarea 2.16)**	🟢 **Consolidado**
`v2.5.19`	**Optimización de Alta Velocidad de Carga de Escenarios, Conexiones DuckDB/Parquet y Renderizado Canvas de Mapas de Calor**: Pool de conexión persistente read-only con cerrojo recursivo (`_DUCKDB_READONLY_LOCK`) en `catalog_duckdb.py` y `viewer_data_access.py`, erradicando contención de archivo en Windows; indexación $O(1)$ en memoria de Parquet (`_PARQUET_INDEX_CACHE`); caché en memoria con `mtime` de catálogos y overlays/detalles de escenarios (`_SCENARIO_OVERLAY_CACHE`, `_SCENARIO_DETAIL_CACHE`, aceleración de 500× a <0.18 ms); memoización de geometría `boardGeometry` en `MatchReplayTab.tsx`; desacoplamiento de hover por celda (`hoveredHex`) y coordenadas de tooltip (`mousePos`), eliminando el 90% de re-renders de canvas; sustitución de `shadowBlur` por contorno vectorial nítido (`strokeText`) y precomputación de vértices `HEX_UNIT_OFFSETS`. Suites 10/10 en reporting y 153/153 en EZv2 en verde.	**MLOps Reporting & Canvas Heatmap Acceleration**	🟢 **Consolidado**

Versión	Hito Principal	Tareas Consolidadas	Estado
`v2.5.18`	**Aceleración de Pipeline, Vectorización Pura y Saturación Hardware (Aceleración Global >3.8× y GPU Batching en CUDA)**: Fases 1 a 4 100% completadas. Fase 1: vectorización de observaciones espaciales de 38 canales en `observation.py` (31.5× más rápido, diferencia 0.000000); Fase 2: vectorización de PUCT en `mcts.py` con arrays NumPy (4.3× más rápido, >4M llamadas eliminadas); Fase 3: precomputación y caché de heurísticas de amenazas y VPs en `selfplay.py` (19.3× más rápido, reduciendo de 17.177 a <150 llamadas por episodio); Fase 4: activación de concurrencia multinúcleo (16 workers en backend de hilos) acoplada a micro-batching dinámico (`inference_max_batch_size: 32` , `wait_ms: 1.5`) hacia la GPU NVIDIA RTX 4060 vía `InferenceService` con soporte transparente de `kwargs` (`return_aux=True`) y resolución `__getattr__` en `RemoteInferenceModel` . Tiempo por episodio reducido de 18.80 s a 4.91 s. Corrección en `catalog_duckdb.py` . Suites 153/153 en EZv2 y 77/77 en assault_bench 100% en verde.	**Optimización y Rendimiento Core**	🟢 **Consolidado**
`v2.5.17`	**Erradicación Total de Inyección Sintética de Facción (Cero 'GE' Ficticio en Escenarios Italianos) y Extracción de Bandos de Ground Truth**: Extracción dinámica de bandos autoritativos (`attacker_side` , `defender_side` , `sides`) desde los archivos canónicos de escenario y transiciones reales de cada episodio, eliminación de la clonación espuria de `IT` a `GE` en `benchmark_result_builder.py` , serialización de controladores por bando en `bench_io.py` (`episode_payload` y `episodes_meta`), resolución dinámica en `ModelPage.tsx` (`atkSide` , `defSide`) para desplegable de replays y filtros, purga de cachés obsoletas e invalidación determinista con versión de metadatos `v2_5_17` . Suite `test_reporting_viewer_scenario_api.py` (5/5) y build Vite en verde.	**Directiva Zero-Hallucination & MLOps Reporting**	🟢 **Consolidado**
`v2.5.15`	**Visibilidad Canónica de Controladores y Briefing Dinámico 100% en Match Replay**: Ingesta y resolución determinista multi-fuente de controladores por bando (`controller_by_side` : `EfficientZeroV2 (AI)` , `EfficientZeroV2 (Self-Play)` , `Random Baseline`), insignias visibles y prominentes en tarjetas de facción del banner táctico, indicador de turno en mapa y banner de eventos. Briefing de escenario 100% dinámico alimentado desde `activeOverlay` y `assets/scenarios/*.json` , erradicación completa de textos residuales en español en frontend (`MatchReplayTab.tsx`) según Regla 7. Suite `test_enrich_episodes_meta_includes_controller_by_side` y build frontend en verde.	**Subtarea 3.6.4**	🟢 **Consolidado**
`v2.5.14`	**Presupuesto Dinámico de Simulación y Horizonte Gobernado por Turnos con Refuerzos**: Desacoplamiento del presupuesto estático de pasos hacia la gobernanza canónica por turnos ($T \leq T_{\max}$). Ingesta del OOB total (`scenario_total_units` en `VOECSimulator` y `EZV2VOECAdapter`), headroom táctico por activación (`steps_per_unit_turn = 2.5`), watchdog de seguridad por turno (`turn_deadlock_watchdog = max(30, alive * 4)`) y eliminación de la penalización espuria de timeout (-1.0) en partidas con refuerzos que concluyen normalmente por límite de turnos. Suite `test_turn_budget_reinforcements.py` (4/4) y 153/153 en EZv2.	**Subtarea 3.6.3**	🟢 **Consolidado**
`v2.5.11`	**Algoritmo Canónico de Retirada a Borde Amigo en Desenfilada (Regla 13.3 / §6.4)**: Selección determinista de hexágono de repliegue táctico priorizando desenfilada (LOS BLOCKED > HINDERED > CLEAR), mayor cobertura de terreno/fortificaciones, vector a borde amigo (*Friendly Edge*) y aumento de distancia al atacante. Eliminación inmediata de artillería e inmovilizados, penalización de 1 punto de daño si la unidad está rodeada sin hexágono legal, e integración autoritativa en `morale.py` , `movement_rules.py` y `ranged_combat_resolver.py` (112/112 tests passing).		
`v2.5.4`	**Catálogo Canónico de Formaciones y Mapeador OOB**: 12 cartas combinadas oficiales, inventario de 122 cartas escaneadas en `assets/art/formations/` , visor visual dual en Editor (:8780), arranque integrado en `start_assault.bat` y suite de tests unitarios/integración.	**Subtarea 3.5.2**	🟢 **Consolidado**
`v2.5.3`	**Aceleración de Pipeline (Zero Quality Loss)**: Caché de terreno ($1,078\times$), Root PUCT vectorizado, inferencia estática `torch.compile` y test Wald SPRT (≡lpha=0.01). Campaña reducida de >10h a <3.5h.	**Tarea 2.9**	🟢 **Consolidado**
`v2.5.2`	**Orquestación Curricular Multi-Etapa y Linaje Forense**: Auto-calibración de quality gates, linaje de checkpoints E2E y evaluación asimétrica calibrada.	**Fases 1–7 Curriculum**	🟢 **Consolidado**
`v2.5.1`	**Urgencia Temporal y Corrección de Presupuesto**: Invarianza temporal PBRS y aceleración de captura de VPs en turnos críticos.	**Tarea 2.3**	🟢 **Consolidado**
`v2.5.0`	**Modernización de Red y Andamiaje de VPs**: CoordConv 40 canales, Policy Head Bottleneck (67 menos parámetros), ECA attention y recompensas radiales.	**Tareas 2.1, 2.2, 2.4, 2.5, 2.6**	🟢 **Consolidado**
`v2.4.x`	**Perfilado Doctrinal Dinámico y Gobernanza MLOps**: Jerarquía 3 capas, mutación contextual, telemetría DuckDB/Parquet y desacoplamiento limpio de MuZero.	**Tareas 1.1, 1.2, 3.6, 4.4**	🟢 **Consolidado**
`v1.0.0` - `v2.3.0`	**Fases Históricas de Cimentación**: VOEC wargame engine, Two-Hot/HL-Gauss, 10 cabezas tácticas, streaming binario y balanceo PCGrad.	**Arquitectura Base**	📦 **Archivado**

Para consultar la base teórica completa y las formulaciones matemáticas de cada componente:

- **Arquitectura de Red y Optimización de Inferencia:** Red Neuronal EfficientZero V2 (Secciones 8 y 9)
- **Evaluación Wald SPRT y Paradoja del Comandante Prudente:** Currículums y Entrenamiento Progresivo (Secciones 5 y 6)
- **Reglas de Combate, PBRS y Desglose Táctico:** Modelo del Juego y Reglas Tácticas
- **Auditoría de Métricas y Telemetría:** Inventario Canónico de Métricas